

AI ENGINEERING FROM SCRATCH.

Deep Learning: Networks, Vision, and Speech

VOLUME II / VI · 58 CHAPTERS · PHASES 03 · 04 · 06

EDITION 2026.09 · A SNAPSHOT OF A LIVING COURSE · NEWEST BUILD ALWAYS AT THE LINK BELOW

aiengineeringfromscratch.com

github.com/rohitg00/ai-engineering-from-scratch

Contents

About This Volume	1
How to Use This Book	2
Part I — Deep Learning Core	3
The Perceptron	4
Multi-Layer Networks and Forward Pass	12
Backpropagation from Scratch	20
Activation Functions	30
Loss Functions	41
Optimizers	51
Regularization	61
Weight Initialization and Training Stability	73
Learning Rate Schedules and Warmup	81
Build Your Own Mini Framework	90
Introduction to PyTorch	103
Introduction to JAX	115
Debugging Neural Networks	126
Part II — Computer Vision	141
Image Fundamentals — Pixels, Channels, Color Spaces	142
Convolutions from Scratch	152
CNNs — LeNet to ResNet	160
Image Classification	169
Transfer Learning & Fine-Tuning	179
Object Detection — YOLO from Scratch	187
Semantic Segmentation — U-Net	196
Instance Segmentation — Mask R-CNN	205

Image Generation — GANs	212
Image Generation — Diffusion Models	219
Stable Diffusion — Architecture & Fine-Tuning	226
Video Understanding — Temporal Modeling	233
3D Vision — Point Clouds & NeRFs	240
Vision Transformers (ViT)	247
Real-Time Vision — Edge Deployment	254
Build a Complete Vision Pipeline — Capstone	261
Self-Supervised Vision — SimCLR, DINO, MAE	269
Open-Vocabulary Vision — CLIP	275
OCR & Document Understanding	280
Image Retrieval & Metric Learning	286
Keypoint Detection & Pose Estimation	292
3D Gaussian Splatting from Scratch	298
Diffusion Transformers & Rectified Flow	307
SAM 3 & Open-Vocabulary Segmentation	315
Vision-Language Models — The ViT-MLP-LLM Pattern	322
Monocular Depth & Geometry Estimation	329
Multi-Object Tracking & Video Memory	335
World Models & Video Diffusion	342
Part III — Speech & Audio	350
Audio Fundamentals — Waveforms, Sampling, Fourier Transform	351
Spectrograms, Mel Scale & Audio Features	356
Audio Classification — From k-NN on MFCCs to AST and BEATs	361
Speech Recognition (ASR) — CTC, RNN-T, Attention	366
Whisper — Architecture & Fine-Tuning	372
Speaker Recognition & Verification	378
Text-to-Speech (TTS) — From Tacotron to F5 and Kokoro	383
Voice Cloning & Voice Conversion	388
Music Generation — MusicGen, Stable Audio, Suno, and the Licensing Earthquake	393

Audio-Language Models — Qwen2.5-Omni, Audio Flamingo, GPT-4o Audio	398
Real-Time Audio Processing	404
Build a Voice Assistant Pipeline — The Phase 6 Capstone	409
Neural Audio Codecs — EnCodec, SNAC, Mimi, DAC and the Semantic-Acoustic Split	414
Voice Activity Detection & Turn-Taking — Silero, Cobra, and the Flush Trick	420
Streaming Speech-to-Speech — Moshi, Hibiki, and Full-Duplex Dialogue	425
Voice Anti-Spoofing & Audio Watermarking — ASVspoof 5, AudioSeal, WaveVerify	431
Audio Evaluation — WER, MOS, UTMOS, MMAU, FAD, and the Open Leaderboard	437

About This Volume

This is Volume 2 of *AI Engineering from Scratch*, a six-volume compilation of the open course of the same name. Each volume stands alone; cross-references cite course phase numbers, which map to volumes like this:

Vol	Title	Course phases
1	Foundations — Math, Tooling, and Classical Machine Learning	00, 01, 02
2	Deep Learning — Networks, Vision, and Speech	03, 04, 06
3	Language — NLP Foundations and the Transformer	05, 07
4	Large Language Models — Generation, Reinforcement, Pretraining, and Engineering	08, 09, 10, 11
5	Agents — Multimodality, Protocols, Autonomy, and Swarms	12, 13, 14, 15, 16
6	Production — Infrastructure, Safety, and Capstones	17, 18, 19

The chapters in this volume come from course phases 03, 04, 06. Chapter prerequisites name phases, not volumes; use the table above to translate.

How to Use This Book

This volume is one loop of a larger machine, and it works best when you run the whole loop:

1. **Read the chapter here.** The prose, the derivations, and the code walkthroughs are complete on the page.
2. **Run the code from the repository.** Every chapter has a code/ directory with a working implementation you can run and break: <https://github.com/rohitg00/ai-engineering-from-scratch>
3. **Open the web edition for what paper cannot do.** Animated figures you can watch and drag, and a quiz per chapter that grades itself: <https://aiengineeringfromscratch.com>

The repository is the living edition. Lessons are updated as the field moves; the book is a snapshot with a version number. When they disagree, the repo is right.

Learning with an AI

This course is built to be read by agents as well as people. The machine-readable index of every lesson lives at <https://aiengineeringfromscratch.com/llms.txt>. If you learn with an AI assistant, paste this and go:

I am working through *AI Engineering from Scratch, Volume 2: Deep Learning*. Fetch <https://aiengineeringfromscratch.com/llms.txt>, find the lesson I name, and act as my tutor: quiz me on its Key Terms, review my solutions to its Exercises, and walk me through its code from the repository.

Part I — Deep Learning Core

Course phase 03. Live edition with animated figures and quizzes: <https://aiengineeringfromscratch.com/catalog.html>

The Perceptron

The perceptron is the atom of neural networks. Split it open and you find weights, a bias, and a decision.

Type: Build **Languages:** Python **Prerequisites:** Phase 1 (Linear Algebra Intuition) **Time:** ~60 minutes

Learning Objectives

- Implement a perceptron from scratch in Python, including the weight update rule and step activation function
- Explain why a single perceptron can only solve linearly separable problems and demonstrate the XOR failure case
- Construct a multi-layer perceptron by composing OR, NAND, and AND gates to solve XOR
- Train a two-layer network with sigmoid activation and backpropagation to learn XOR automatically

The Problem

You know vectors and dot products. You know that a matrix transforms inputs into outputs. But how does a machine *learn* which transformation to use?

The perceptron answers this. It's the simplest possible learning machine: take some inputs, multiply by weights, add a bias, and make a binary decision. Then adjust. That's it. Every neural network ever built is layers of this idea stacked together.

Understanding the perceptron means understanding what “learning” actually means in code: adjusting numbers until the output matches reality.

The Concept

One Neuron, One Decision

A perceptron takes n inputs, multiplies each by a weight, sums them up, adds a bias, and passes the result through an activation function.

Diagram. Rendered live in the web edition: <https://aiengineeringfromscratch.com/lesson?path=phases/03-deep-learning-core/01-the-perceptron>

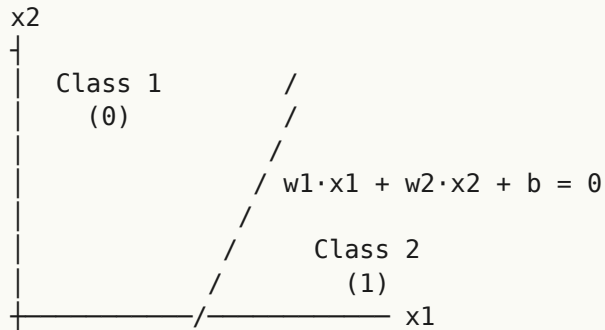
The step function is brutal: if the weighted sum plus bias is ≥ 0 , output 1. Otherwise, output 0.

$$\text{step}(z) = \begin{cases} 1 & \text{if } z \geq 0 \\ 0 & \text{if } z < 0 \end{cases}$$

This is a linear classifier. The weights and bias define a line (or hyperplane in higher dimensions) that splits the input space into two regions.

The Decision Boundary

For two inputs, the perceptron draws a line through 2D space:



Everything on one side of the line outputs 0. Everything on the other side outputs 1. Training moves this line until it correctly separates the classes.

The Learning Rule

The perceptron learning rule is simple:

For each training example (x, y_{true}) :

$y_{\text{pred}} = \text{predict}(x)$

$\text{error} = y_{\text{true}} - y_{\text{pred}}$

For each weight:

$w_i = w_i + \text{learning_rate} * \text{error} * x_i$

$\text{bias} = \text{bias} + \text{learning_rate} * \text{error}$

If the prediction is correct, $\text{error} = 0$, nothing changes. If it predicts 0 but should be 1, weights increase. If it predicts 1 but should be 0, weights decrease. The learning rate controls how big each adjustment is.

The XOR Problem

Here's where it breaks. Look at these logic gates:

AND gate:

x1	x2	out
0	0	0
0	1	0
1	0	0
1	1	1

OR gate:

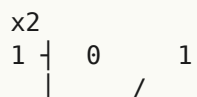
x1	x2	out
0	0	0
0	1	1
1	0	1
1	1	1

XOR gate:

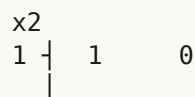
x1	x2	out
0	0	0
0	1	1
1	0	1
1	1	0

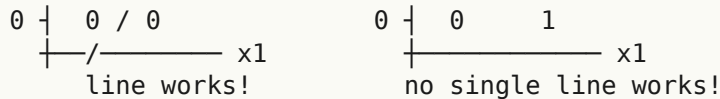
AND and OR are linearly separable: you can draw a single line to separate the 0s from the 1s. XOR is not. No single line can separate $[0,1]$ and $[1,0]$ from $[0,0]$ and $[1,1]$.

AND (separable):



XOR (not separable):





This is a fundamental limit. A single perceptron can only solve linearly separable problems. Minsky and Papert proved this in 1969 and it nearly killed neural network research for a decade.

The fix: stack perceptrons into layers. A multi-layer perceptron can solve XOR by combining two linear decisions into a nonlinear one.

Interactive figure: perceptron-boundary. This one moves. Watch it animate and drag its controls in the web edition: <https://aiengineeringfromscratch.com/lesson?path=phases/03-deep-learning-core/01-the-perceptron>

Build It

Step 1: The Perceptron class

```
class Perceptron:
    def __init__(self, n_inputs, learning_rate=0.1):
        self.weights = [0.0] * n_inputs
        self.bias = 0.0
        self.lr = learning_rate

    def predict(self, inputs):
        total = sum(w * x for w, x in zip(self.weights, inputs))
        total += self.bias
        return 1 if total >= 0 else 0

    def train(self, training_data, epochs=100):
        for epoch in range(epochs):
            errors = 0
            for inputs, target in training_data:
                prediction = self.predict(inputs)
                error = target - prediction
                if error != 0:
                    errors += 1
                    for i in range(len(self.weights)):
                        self.weights[i] += self.lr * error * inputs[i]
                        self.bias += self.lr * error
            if errors == 0:
                print(f"Converged at epoch {epoch + 1}")
                return
        print(f"Did not converge after {epochs} epochs")
```

Step 2: Train on logic gates

```
and_data = [
    ([0, 0], 0),
    ([0, 1], 0),
    ([1, 0], 0),
    ([1, 1], 1),
]
```

```

or_data = [
    ([0, 0], 0),
    ([0, 1], 1),
    ([1, 0], 1),
    ([1, 1], 1),
]

not_data = [
    ([0], 1),
    ([1], 0),
]

print("=== AND Gate ===")
p_and = Perceptron(2)
p_and.train(and_data)
for inputs, _ in and_data:
    print(f" {inputs} -> {p_and.predict(inputs)}")

print("\n=== OR Gate ===")
p_or = Perceptron(2)
p_or.train(or_data)
for inputs, _ in or_data:
    print(f" {inputs} -> {p_or.predict(inputs)}")

print("\n=== NOT Gate ===")
p_not = Perceptron(1)
p_not.train(not_data)
for inputs, _ in not_data:
    print(f" {inputs} -> {p_not.predict(inputs)}")

```

Step 3: Watch XOR fail

```

xor_data = [
    ([0, 0], 0),
    ([0, 1], 1),
    ([1, 0], 1),
    ([1, 1], 0),
]

print("\n=== XOR Gate (single perceptron) ===")
p_xor = Perceptron(2)
p_xor.train(xor_data, epochs=1000)
for inputs, expected in xor_data:
    result = p_xor.predict(inputs)
    status = "OK" if result == expected else "WRONG"
    print(f" {inputs} -> {result} (expected {expected}) {status}")

```

It will never converge. This is the hard proof that a single perceptron cannot learn XOR.

Step 4: Solve XOR with two layers

The trick: $XOR = (x1 \text{ OR } x2) \text{ AND NOT } (x1 \text{ AND } x2)$. Combine three perceptrons:

Diagram. Rendered live in the web edition: <https://aiengineeringfromscratch.com/lesson?path=phases/03-deep-learning-core/01-the-perceptron>

```
def xor_network(x1, x2):
    or_neuron = Perceptron(2)
    or_neuron.weights = [1.0, 1.0]
    or_neuron.bias = -0.5

    nand_neuron = Perceptron(2)
    nand_neuron.weights = [-1.0, -1.0]
    nand_neuron.bias = 1.5

    and_neuron = Perceptron(2)
    and_neuron.weights = [1.0, 1.0]
    and_neuron.bias = -1.5

    hidden1 = or_neuron.predict([x1, x2])
    hidden2 = nand_neuron.predict([x1, x2])
    output = and_neuron.predict([hidden1, hidden2])
    return output

print("\n=== XOR Gate (multi-layer network) ===")
for inputs, expected in xor_data:
    result = xor_network(inputs[0], inputs[1])
    print(f" {inputs} -> {result} (expected {expected})")
```

All four cases correct. Stacking perceptrons into layers creates decision boundaries that no single perceptron can produce.

Step 5: Train a Two-Layer Network

Step 4 hand-wired the weights. That works for XOR, but not for real problems where you don't know the right weights in advance. The fix: replace the step function with sigmoid and learn the weights automatically through backpropagation.

```
class TwoLayerNetwork:
    def __init__(self, learning_rate=0.5):
        import random
        random.seed(0)
        self.w_hidden = [[random.uniform(-1, 1), random.uniform(-1, 1)] for _ in
            ↪ range(2)]
        self.b_hidden = [random.uniform(-1, 1), random.uniform(-1, 1)]
        self.w_output = [random.uniform(-1, 1), random.uniform(-1, 1)]
        self.b_output = random.uniform(-1, 1)
        self.lr = learning_rate

    def sigmoid(self, x):
        import math
        x = max(-500, min(500, x))
        return 1.0 / (1.0 + math.exp(-x))

    def forward(self, inputs):
        self.inputs = inputs
        self.hidden_outputs = []
```

```

        for i in range(2):
            z = sum(w * x for w, x in zip(self.w_hidden[i], inputs)) +
↪ self.b_hidden[i]
            self.hidden_outputs.append(self.sigmoid(z))
            z_out = sum(w * h for w, h in zip(self.w_output, self.hidden_outputs)) +
↪ self.b_output
            self.output = self.sigmoid(z_out)
            return self.output

def train(self, training_data, epochs=10000):
    for epoch in range(epochs):
        total_error = 0
        for inputs, target in training_data:
            output = self.forward(inputs)
            error = target - output
            total_error += error ** 2

        d_output = error * output * (1 - output)

        saved_w_output = self.w_output[:]
        hidden_deltas = []
        for i in range(2):
            h = self.hidden_outputs[i]
            hd = d_output * saved_w_output[i] * h * (1 - h)
            hidden_deltas.append(hd)

        for i in range(2):
            self.w_output[i] += self.lr * d_output * self.hidden_outputs[i]
            self.b_output += self.lr * d_output

        for i in range(2):
            for j in range(len(inputs)):
                self.w_hidden[i][j] += self.lr * hidden_deltas[i] * inputs[j]
                self.b_hidden[i] += self.lr * hidden_deltas[i]

net = TwoLayerNetwork(learning_rate=2.0)
net.train(xor_data, epochs=10000)
for inputs, expected in xor_data:
    result = net.forward(inputs)
    predicted = 1 if result >= 0.5 else 0
    print(f" {inputs} -> {result:.4f} (rounded: {predicted}, expected
↪ {expected})")

```

Two key differences from Step 4. First, sigmoid replaces the step function - it's smooth, so gradients exist. Second, the train method propagates error backward from output to hidden layer, adjusting every weight proportionally to its contribution to the error. That's backpropagation in 20 lines.

This is the bridge to Lesson 03. The math behind `d_output` and `hidden_deltas` is the chain rule applied to the network graph. We'll derive it properly there.

Use It

Everything you just built from scratch exists in one import:

```
from sklearn.linear_model import Perceptron as SkPerceptron
import numpy as np
```

```
X = np.array([[0,0],[0,1],[1,0],[1,1]])
y = np.array([0, 0, 0, 1])
```

```
clf = SkPerceptron(max_iter=100, tol=1e-3)
clf.fit(X, y)
print([clf.predict([x])[0] for x in X])
```

Five lines. Your 30-line Perceptron class does the same thing. The sklearn version adds convergence checks, multiple loss functions, and sparse input support - but the core loop is identical: weighted sum, step function, weight update on error.

The real gap shows up at scale. What changes in production networks:

- The step function becomes sigmoid, ReLU, or other smooth activations
- Weights are learned automatically via backpropagation (Lesson 03)
- Layers get deeper: 3, 10, 100+ layers
- The same principle holds: each layer creates new features from the previous layer's outputs

A single perceptron can only draw straight lines. Stack them, and you can draw any shape.

This chapter ships an artifact. The course version of this lesson produces a reusable prompt or agent skill. It lives in the repository, ready to install: <https://github.com/rohitg00/ai-engineering-from-scratch/tree/main/phases/03-deep-learning-core/01-the-perceptron>

Exercises

Starter code and the lesson's working implementation: <https://github.com/rohitg00/ai-engineering-from-scratch/tree/main/phases/03-deep-learning-core/01-the-perceptron/code>

1. Train a perceptron on a NAND gate (the universal gate - any logic circuit can be built from NAND). Verify its weights and bias form a valid decision boundary.
2. Modify the Perceptron class to track the decision boundary ($w_1x_1 + w_2x_2 + b = 0$) at each epoch. Print how the line shifts during training on the AND gate.
3. Build a 3-input perceptron that outputs 1 only when at least 2 of the 3 inputs are 1 (a majority vote function). Is this linearly separable? Why?

Key Terms

Term	What people say	What it actually means
Perceptron	"A fake neuron"	A linear classifier: dot product of inputs and weights, plus bias, through a step function
Weight	"How important an input is"	A multiplier that scales each input's contribution to the decision
Bias	"The threshold"	A constant that shifts the decision boundary, letting the perceptron fire even with zero inputs
Activation function	"The thing that squishes values"	A function applied after the weighted sum - step function for perceptrons, sigmoid/ReLU for modern networks

Term	What people say	What it actually means
Linearly separable	"You can draw a line between them"	A dataset where a single hyperplane can perfectly separate the classes
XOR problem	"The thing perceptrons can't do"	Proof that single-layer networks cannot learn non-linearly-separable functions
Decision boundary	"Where the classifier switches"	The hyperplane $w \cdot x + b = 0$ that divides input space into two classes
Multi-layer perceptron	"A real neural network"	Perceptrons stacked in layers, where each layer's output feeds the next layer's input

Further Reading

- Frank Rosenblatt, "The Perceptron: A Probabilistic Model for Information Storage and Organization in the Brain" (1958) - the original paper that started it all
- Minsky & Papert, "Perceptrons" (1969) - the book that proved XOR was unsolvable by single-layer networks and killed perceptron research for a decade
- Michael Nielsen, "Neural Networks and Deep Learning", Chapter 1 (<http://neuralnetworksanddeeplearning.com/>) - free online, best visual explanation of how perceptrons compose into networks

Continue online. The living edition of this chapter has more than the page can hold:

- Animated, interactive figures and the web text: <https://aiengineeringfromscratch.com/lesson?path=phases/03-deep-learning-core/01-the-perceptron>
- Runnable code for every step: <https://github.com/rohitg00/ai-engineering-from-scratch/tree/main/phases/03-deep-learning-core/01-the-perceptron/code>
- The chapter quiz, graded in the browser: <https://aiengineeringfromscratch.com/lesson?path=phases/03-deep-learning-core/01-the-perceptron>

The repository moves faster than any printing. When the book and the repo disagree, trust the repo.

Multi-Layer Networks and Forward Pass

One neuron draws a line. Stack them, and you can draw anything.

Type: Build **Languages:** Python **Prerequisites:** Phase 01 (Math Foundations), Lesson 03.01 (The Perceptron) **Time:** ~90 minutes

Learning Objectives

- Build a multi-layer network from scratch with Layer and Network classes that perform a complete forward pass
- Trace matrix dimensions through each layer of a network and identify shape mismatches
- Explain how stacking nonlinear activations enables a network to learn curved decision boundaries
- Solve the XOR problem using a 2-2-1 architecture with hand-tuned sigmoid weights

The Problem

A single neuron is a line drawer. That's it. One straight line through your data. Every real problem in AI - image recognition, language understanding, playing Go - requires curves. Stacking neurons into layers is how you get curves.

In 1969, Minsky and Papert proved this limitation was fatal: a single-layer network cannot learn XOR. Not "struggles to learn" - mathematically cannot. The XOR truth table places [0,1] and [1,0] on one side, [0,0] and [1,1] on the other. No single line separates them.

This killed neural network funding for over a decade. The fix was obvious in hindsight: stop using one layer. Stack neurons into layers. Let the first layer carve the input space into new features, and let the second layer combine those features into decisions no single line could make.

That stack is the multi-layer network. It is the foundation of every deep learning model in production today. The forward pass - data flowing from input through hidden layers to output - is the first thing you need to build before anything else works.

The Concept

Layers: Input, Hidden, Output

A multi-layer network has three types of layers:

Input layer - not really a layer. It holds your raw data. Two features means two input nodes. No computation happens here.

Hidden layers - where the work happens. Each neuron takes every output from the previous layer, applies weights and a bias, then passes the result through an activation function. "Hidden" because you never see these values directly in the training data.

Output layer - the final answer. For binary classification, one neuron with sigmoid. For multi-class, one neuron per class.

Diagram. Rendered live in the web edition: <https://aiengineeringfromscratch.com/lesson?path=phases/03-deep-learning-core/02-multi-layer-networks>

This is a 2-3-1 network. Two inputs, three hidden neurons, one output. Every connection carries a weight. Every neuron (except input) carries a bias.

Each layer produces a vector of numbers called a hidden state. For text, hidden states increase dimensionality - encoding a word as 768 numbers to capture semantic meaning. For images, they reduce dimensionality - compressing millions of pixels into a manageable representation. The hidden state is where the learning lives.

Neurons and Activations

Each neuron does three things:

1. Multiply every input by its corresponding weight
2. Sum all the products and add a bias
3. Pass the sum through an activation function

For now, the activation is sigmoid:

$$\text{sigmoid}(z) = 1 / (1 + e^{(-z)})$$

Sigmoid squashes any number into the range (0, 1). Large positive inputs push toward 1. Large negative inputs push toward 0. Zero maps to 0.5. This smooth curve is what makes learning possible - unlike the perceptron's hard step, sigmoid has a gradient everywhere.

Forward Pass: How Data Flows

The forward pass pushes input data through the network, layer by layer, until it reaches the output. No learning happens during the forward pass. It is pure computation: multiply, add, activate, repeat.

Diagram. Rendered live in the web edition: <https://aiengineeringfromscratch.com/lesson?path=phases/03-deep-learning-core/02-multi-layer-networks>

At each layer, three operations happen in sequence:

$$\begin{aligned} z &= W * \text{input} + b && \text{(linear transformation)} \\ a &= \text{sigmoid}(z) && \text{(activation)} \end{aligned}$$

The output of one layer becomes the input to the next. That is the entire forward pass.

Matrix Dimensions

Tracking dimensions is the single most important debugging skill in deep learning. Here is the 2-3-1 network:

Step	Operation	Dimensions	Result Shape
Input	x	-	(2,)
Hidden linear	W1 * x + b1	W1: (3, 2), b1: (3,)	(3,)
Hidden activation	sigmoid(z1)	-	(3,)

Step	Operation	Dimensions	Result Shape
Output linear	$W2 * h + b2$	$W2: (1, 3), b2: (1,)$	$(1,)$
Output activation	$\text{sigmoid}(z2)$	-	$(1,)$

The rule: weight matrix W at layer k has shape $(\text{neurons_in_layer_k}, \text{neurons_in_layer_k_minus_1})$. Rows match the current layer. Columns match the previous layer. If the shapes do not line up, you have a bug.

Universal Approximation Theorem

In 1989, George Cybenko proved something remarkable: a neural network with a single hidden layer and enough neurons can approximate any continuous function to any desired accuracy.

This does not mean one hidden layer is always best. It means the architecture is theoretically capable. In practice, deeper networks (more layers, fewer neurons per layer) learn the same functions with far fewer total parameters than shallow-wide networks. That is why deep learning works.

The intuition: each neuron in the hidden layer learns one “bump” or feature. Enough bumps placed in the right locations can approximate any smooth curve. More neurons, more bumps, better approximation.

Diagram. Rendered live in the web edition: <https://aiengineeringfromscratch.com/lesson?path=phases/03-deep-learning-core/02-multi-layer-networks>

Composability

Neural networks are composable. You can stack them, chain them, run them in parallel. A Whisper model uses an encoder network to process audio and a separate decoder network to generate text. Modern LLMs are decoder-only. BERT is encoder-only. T5 is encoder-decoder. The architecture choice defines what the model can do.

Interactive figure: mlp-forward. This one moves. Watch it animate and drag its controls in the web edition: <https://aiengineeringfromscratch.com/lesson?path=phases/03-deep-learning-core/02-multi-layer-networks>

Build It

Pure Python. No numpy. Every matrix operation written from scratch.

Step 1: Sigmoid Activation

```
import math

def sigmoid(x):
    x = max(-500.0, min(500.0, x))
    return 1.0 / (1.0 + math.exp(-x))
```

The clamp to $[-500, 500]$ prevents overflow. $\text{math.exp}(500)$ is large but finite. $\text{math.exp}(1000)$ is infinity.

Step 2: Layer Class

The most important operation in all of deep learning is matrix multiplication. Every layer, every attention head, every forward pass – it's matmuls all the way down. A linear layer takes an input vector, multiplies it by a weight matrix, and adds a bias vector: $y = Wx + b$. That single equation is 90% of the compute in a neural network.

A layer holds a weight matrix and a bias vector. Its forward method takes an input vector and returns the activated output.

```
class Layer:
    def __init__(self, n_inputs, n_neurons, weights=None, biases=None):
        if weights is not None:
            self.weights = weights
        else:
            import random
            self.weights = [
                [random.uniform(-1, 1) for _ in range(n_inputs)]
                for _ in range(n_neurons)
            ]
        if biases is not None:
            self.biases = biases
        else:
            self.biases = [0.0] * n_neurons

    def forward(self, inputs):
        self.last_input = inputs
        self.last_output = []
        for neuron_idx in range(len(self.weights)):
            z = sum(
                w * x for w, x in zip(self.weights[neuron_idx], inputs)
            )
            z += self.biases[neuron_idx]
            self.last_output.append(sigmoid(z))
        return self.last_output
```

The weight matrix has shape (n_neurons, n_inputs). Each row is one neuron's weights across all inputs. The forward method loops through neurons, computes the weighted sum plus bias, applies sigmoid, and collects the results.

Step 3: Network Class

A network is a list of layers. The forward pass chains them: output of layer k feeds into layer k+1.

```
class Network:
    def __init__(self, layers):
        self.layers = layers

    def forward(self, inputs):
        current = inputs
        for layer in self.layers:
            current = layer.forward(current)
        return current
```

That is the entire forward pass. Four lines of logic. Data goes in, flows through every layer, comes out the other side.

Step 4: XOR with Hand-Tuned Weights

In Lesson 01, we solved XOR by combining OR, NAND, and AND perceptrons. Now do the same thing with our Layer and Network classes. The 2-2-1 architecture: two inputs, two hidden neurons, one output.

```
hidden = Layer(
    n_inputs=2,
    n_neurons=2,
    weights=[[20.0, 20.0], [-20.0, -20.0]],
    biases=[-10.0, 30.0],
)

output = Layer(
    n_inputs=2,
    n_neurons=1,
    weights=[[20.0, 20.0]],
    biases=[-30.0],
)

xor_net = Network([hidden, output])

xor_data = [
    ([0, 0], 0),
    ([0, 1], 1),
    ([1, 0], 1),
    ([1, 1], 0),
]

for inputs, expected in xor_data:
    result = xor_net.forward(inputs)
    predicted = 1 if result[0] >= 0.5 else 0
    print(f" {inputs} -> {result[0]:.6f} (rounded: {predicted}, expected:
    ↪ {expected})")
```

The large weights (20, -20) make sigmoid act like a step function. The first hidden neuron approximates OR. The second approximates NAND. The output neuron combines them into AND, which is XOR.

Step 5: Circle Classification

A harder problem: classify 2D points as inside or outside a circle of radius 0.5 centered at the origin. This requires a curved decision boundary – impossible for a single perceptron.

```
import random
import math

random.seed(42)

data = []
for _ in range(200):
    x = random.uniform(-1, 1)
    y = random.uniform(-1, 1)
    label = 1 if (x * x + y * y) < 0.25 else 0
    data.append([x, y], label))
```

```
circle_net = Network([
    Layer(n_inputs=2, n_neurons=8),
    Layer(n_inputs=8, n_neurons=1),
])
```

With random weights, the network will not classify well. But the forward pass still runs. This is the point – the forward pass is just computation. Learning the right weights is backpropagation, coming in Lesson 03.

```
correct = 0
for inputs, expected in data:
    result = circle_net.forward(inputs)
    predicted = 1 if result[0] >= 0.5 else 0
    if predicted == expected:
        correct += 1

print(f"Accuracy with random weights: {correct}/{len(data)}")
↪ ({100*correct/len(data):.1f}%)")
```

Random weights give poor accuracy – often worse than guessing the majority class. After training (Lesson 03), this same architecture with 8 hidden neurons will draw a curved boundary that separates inside from outside.

Use It

PyTorch does everything above in four lines:

```
import torch
import torch.nn as nn

model = nn.Sequential(
    nn.Linear(2, 8),
    nn.Sigmoid(),
    nn.Linear(8, 1),
    nn.Sigmoid(),
)

x = torch.tensor([[0.0, 0.0], [0.0, 1.0], [1.0, 0.0], [1.0, 1.0]])
output = model(x)
print(output)
```

`nn.Linear(2, 8)` is your Layer class: weight matrix of shape (8, 2), bias vector of shape (8,). `nn.Sigmoid()` is your sigmoid function applied element-wise. `nn.Sequential` is your Network class: chain layers in order.

The difference is speed and scale. PyTorch runs on GPUs, handles batches of millions of samples, and automatically computes gradients for backpropagation. But the forward pass logic is identical to what you just built from scratch.

This chapter ships an artifact. The course version of this lesson produces a reusable prompt or agent skill. It lives in the repository, ready to install: <https://github.com/rohitg00/ai-engineering-from-scratch/tree/main/phases/03-deep-learning-core/02-multi-layer-networks>

Exercises

Starter code and the lesson's working implementation: <https://github.com/rohitg00/ai-engineering-from-scratch/tree/main/phases/03-deep-learning-core/02-multi-layer-networks/code>

1. Build a 2-4-2-1 network (two hidden layers) and run the forward pass on XOR data with random weights. Print the intermediate hidden layer outputs to see how the representation transforms at each layer.
2. Change the hidden layer size in the circle classifier from 8 to 2, then to 32. Run the forward pass with random weights each time. Does the number of hidden neurons change the output range or distribution? Why?
3. Implement a `count_parameters` method on the `Network` class that returns the total number of trainable weights and biases. Test it on a 784-256-128-10 network (the classic MNIST architecture). How many parameters does it have?
4. Build a forward pass for a 3-4-4-2 network. Feed it RGB color values (normalized to 0-1) and observe the two outputs. This is the architecture for a simple color classifier with two classes.
5. Replace sigmoid with a "leaky step" function: return $0.01 * z$ if $z < 0$, else 1.0. Run the forward pass on XOR with the same hand-tuned weights from Step 4. Does it still work? Why is the smooth sigmoid preferred over hard cutoffs?

Key Terms

Term	What people say	What it actually means
Forward pass	"Running the model"	Pushing input through every layer - multiply by weights, add bias, activate - to produce an output
Hidden layer	"The middle part"	Any layer between input and output whose values are not directly observed in the data
Multi-layer network	"A deep neural network"	Layers of neurons stacked sequentially, where each layer's output feeds the next layer's input
Activation function	"The nonlinearity"	A function applied after the linear transformation that introduces curves into the decision boundary
Sigmoid	"The S-curve"	$\text{sigma}(z) = 1/(1+e^{(-z)})$, squashes any real number to (0,1), smooth and differentiable everywhere
Weight matrix	"The parameters"	A matrix W of shape (current_layer_neurons, previous_layer_neurons) containing learnable connection strengths
Bias vector	"The offset"	A vector added after the matrix multiply that lets neurons activate even when all inputs are zero
Universal approximation	"Neural nets can learn anything"	A single hidden layer with enough neurons can approximate any continuous function - but "enough" can mean billions
Linear transformation	"The matrix multiply step"	$z = W * x + b$, the computation before activation, which maps inputs to a new space

Term	What people say	What it actually means
Decision boundary	“Where the classifier switches”	The surface in input space where the network output crosses the classification threshold

Further Reading

- Michael Nielsen, “Neural Networks and Deep Learning”, Chapter 1-2 (<http://neuralnetworksanddeeplearning.com/>) - the clearest free explanation of forward passes and network structure, with interactive visualizations
- Cybenko, “Approximation by Superpositions of a Sigmoidal Function” (1989) - the original universal approximation theorem paper, surprisingly readable
- 3Blue1Brown, “But what is a neural network?” (<https://www.youtube.com/watch?v=airAruvnKk>) - 20-minute visual walkthrough of layers, weights, and forward passes that builds the right mental model
- Goodfellow, Bengio, Courville, “Deep Learning”, Chapter 6 (<https://www.deeplearningbook.org/>) - the standard reference for multi-layer networks, free online

Continue online. The living edition of this chapter has more than the page can hold:

- Animated, interactive figures and the web text: <https://aiengineeringfromscratch.com/lesson?path=phases/03-deep-learning-core/02-multi-layer-networks>
- Runnable code for every step: <https://github.com/rohitg00/ai-engineering-from-scratch/tree/main/phases/03-deep-learning-core/02-multi-layer-networks/code>
- The chapter quiz, graded in the browser: <https://aiengineeringfromscratch.com/lesson?path=phases/03-deep-learning-core/02-multi-layer-networks>

The repository moves faster than any printing. When the book and the repo disagree, trust the repo.

Backpropagation from Scratch

Backpropagation is the algorithm that makes learning possible. Without it, neural networks are just expensive random number generators.

Type: Build **Languages:** Python **Prerequisites:** Lesson 03.02 (Multi-Layer Networks)
Time: ~120 minutes

Learning Objectives

- Implement a Value-based autograd engine that builds a computational graph and computes gradients via topological sort
- Derive the backward pass for addition, multiplication, and sigmoid using the chain rule
- Train a multi-layer network on XOR and circle classification using only your from-scratch backpropagation engine
- Identify the vanishing gradient problem in deep sigmoid networks and explain why gradients shrink exponentially

The Problem

Your network has a single hidden layer with 768 inputs and 3072 outputs. That's 2,359,296 weights. It made a wrong prediction. Which weights caused the error? Testing each weight individually means 2.3 million forward passes. Backpropagation computes all 2.3 million gradients in a single backward pass. That's not an optimization. That's the difference between trainable and impossible.

The naive approach: take one weight, nudge it by a tiny amount, run the forward pass again, measure whether the loss went up or down. That gives you the gradient for that weight. Now do it for every weight in the network. Multiply by thousands of training steps and millions of data points. You'd need geological time to train anything useful.

Backpropagation solves this. One forward pass, one backward pass, all gradients computed. The trick is the chain rule from calculus, applied systematically to a computational graph. This is the algorithm that made deep learning practical. Without it, we'd still be stuck on toy problems.

The Concept

The Chain Rule, Applied to Networks

You saw the chain rule in Phase 01, Lesson 05. Quick recap: if $y = f(g(x))$, then $dy/dx = f'(g(x)) * g'(x)$. You multiply derivatives along the chain.

In a neural network, the "chain" is the sequence of operations from input to loss. Each layer applies weights, adds biases, passes through an activation. The loss function compares the final output to the target. Backpropagation traces this chain backward, computing how each operation contributed to the error.

Computational Graphs

Every forward pass builds a graph. Each node is an operation (multiply, add, sigmoid). Each edge carries a value forward and a gradient backward.

Diagram. Rendered live in the web edition: <https://aiengineeringfromscratch.com/lesson?path=phases/03-deep-learning-core/03-backpropagation>

Forward pass: values flow left to right. x and w produce $z_1 = w \cdot x$. Add b to get z_2 . Sigmoid gives activation a . Compare a to target y using the loss function.

Backward pass: gradients flow right to left. Start with dL/da (how loss changes with the activation). Multiply by da/dz_2 (sigmoid derivative). That gives dL/dz_2 . Split into dL/db (which equals dL/dz_2 , since $z_2 = z_1 + b$) and dL/dz_1 . Then $dL/dw = dL/dz_1 \cdot x$ and $dL/dx = dL/dz_1 \cdot w$.

Every node in the graph has one job during the backward pass: take the gradient coming from above, multiply by its local derivative, and pass it down.

Forward vs Backward

Diagram. Rendered live in the web edition: <https://aiengineeringfromscratch.com/lesson?path=phases/03-deep-learning-core/03-backpropagation>

The forward pass stores every intermediate value: z , a , the inputs to each layer. The backward pass needs these stored values to compute gradients. This is the memory-computation tradeoff at the heart of backprop. You trade memory (storing activations) for speed (one pass instead of millions).

Gradient Flow Through a Network

For a 3-layer network, gradients chain through every layer:

Diagram. Rendered live in the web edition: <https://aiengineeringfromscratch.com/lesson?path=phases/03-deep-learning-core/03-backpropagation>

At each layer, the gradient gets multiplied by the sigmoid derivative. The sigmoid derivative is $a \cdot (1 - a)$, which maxes out at 0.25 (when $a = 0.5$). Three layers deep, the gradient has been multiplied by at most $0.25^3 = 0.0156$. Ten layers deep: $0.25^{10} = 0.000001$.

Vanishing Gradients

This is the vanishing gradient problem. Sigmoid squashes its output between 0 and 1. Its derivative is always less than 0.25. Stack enough sigmoid layers and gradients shrink to nothing. Early layers barely learn because they receive near-zero gradients.

sigmoid(z): Output range $[0, 1]$
sigmoid'(z): Max value 0.25 (at $z = 0$)

After 5 layers: gradient $\cdot 0.25^5 = 0.001 \times$ original
After 10 layers: gradient $\cdot 0.25^{10} = 0.000001 \times$ original

This is why deep sigmoid networks are nearly impossible to train. The fix - ReLU and its variants - is the subject of Lesson 04. For now, understand that backprop works perfectly. The problem is what it's working through.

Deriving Gradients for a 2-Layer Network

Concrete math for a network with input x , hidden layer with sigmoid, output layer with sigmoid, and MSE loss.

Forward pass:

```
z1 = W1 * x + b1
a1 = sigmoid(z1)
z2 = W2 * a1 + b2
a2 = sigmoid(z2)
L = (a2 - y)^2
```

Backward pass (applying chain rule step by step):

```
dL/da2 = 2(a2 - y)
da2/dz2 = a2 * (1 - a2)
dL/dz2 = dL/da2 * da2/dz2 = 2(a2 - y) * a2 * (1 - a2)

dL/dW2 = dL/dz2 * a1
dL/db2 = dL/dz2

dL/da1 = dL/dz2 * W2
da1/dz1 = a1 * (1 - a1)
dL/dz1 = dL/da1 * da1/dz1

dL/dW1 = dL/dz1 * x
dL/db1 = dL/dz1
```

Every gradient is a product of local derivatives traced back from the loss. That's all backpropagation is.

Interactive figure: backprop-vanishing. This one moves. Watch it animate and drag its controls in the web edition: <https://aiengineeringfromscratch.com/lesson?path=phases/03-deep-learning-core/03-backpropagation>

Build It

Step 1: The Value Node

Every number in our computation becomes a Value. It stores its data, its gradient, and how it was created (so it knows how to compute gradients backward).

```
class Value:
    def __init__(self, data, children=(), op=''):
        self.data = data
        self.grad = 0.0
        self._backward = lambda: None
        self._children = set(children)
        self._op = op

    def __repr__(self):
        return f"Value(data={self.data:.4f}, grad={self.grad:.4f})"
```

No gradient yet (0.0). No backward function yet (no-op). The `_children` track which Values produced this one, so we can topologically sort the graph later.

Step 2: Operations with Backward Functions

Each operation creates a new Value and defines how gradients flow backward through it.

```
def __add__(self, other):
    other = other if isinstance(other, Value) else Value(other)
    out = Value(self.data + other.data, (self, other), '+')

    def _backward():
        self.grad += out.grad
        other.grad += out.grad

    out._backward = _backward
    return out

def __mul__(self, other):
    other = other if isinstance(other, Value) else Value(other)
    out = Value(self.data * other.data, (self, other), '*')

    def _backward():
        self.grad += other.data * out.grad
        other.grad += self.data * out.grad

    out._backward = _backward
    return out
```

For addition: $d(a+b)/da = 1$, $d(a+b)/db = 1$. So both inputs get the output's gradient directly.

For multiplication: $d(ab)/da = b$, $d(ab)/db = a$. Each input gets the other's value times the output gradient.

The += is critical. A Value might be used in multiple operations. Its gradient is the sum of gradients from all paths.

Step 3: Sigmoid and Loss

```
import math

def sigmoid(self):
    x = self.data
    x = max(-500, min(500, x))
    s = 1.0 / (1.0 + math.exp(-x))
    out = Value(s, (self,), 'sigmoid')

    def _backward():
        self.grad += (s * (1 - s)) * out.grad

    out._backward = _backward
    return out
```

Sigmoid derivative: $\text{sigmoid}(x) * (1 - \text{sigmoid}(x))$. We computed $\text{sigmoid}(x) = s$ during the forward pass. Reuse it. No extra work.

```
def mse_loss(predicted, target):
    diff = predicted - target
    return diff * diff
```

MSE for a single output: $(\text{predicted} - \text{target})^2$. We express subtraction as addition with a negated Value.

Step 4: Backward Pass

Topological sort ensures we process nodes in the right order – a node's gradient is fully accumulated before we propagate through it.

```
def backward(self):
    topo = []
    visited = set()

    def build_topo(v):
        if v not in visited:
            visited.add(v)
            for child in v._children:
                build_topo(child)
            topo.append(v)

    build_topo(self)
    self.grad = 1.0
    for v in reversed(topo):
        v._backward()
```

Start at the loss (gradient = 1.0, since $dL/dL = 1$). Walk backward through the sorted graph. Each node's `_backward` pushes gradients to its children.

Step 5: Layer and Network

```
import random

class Neuron:
    def __init__(self, n_inputs):
        scale = (2.0 / n_inputs) ** 0.5
        self.weights = [Value(random.uniform(-scale, scale)) for _ in
            range(n_inputs)]
        self.bias = Value(0.0)

    def __call__(self, x):
        act = sum((wi * xi for wi, xi in zip(self.weights, x)), self.bias)
        return act.sigmoid()

    def parameters(self):
        return self.weights + [self.bias]

class Layer:
    def __init__(self, n_inputs, n_outputs):
        self.neurons = [Neuron(n_inputs) for _ in range(n_outputs)]

    def __call__(self, x):
        out = [n(x) for n in self.neurons]
        return out[0] if len(out) == 1 else out
```

```

def parameters(self):
    params = []
    for n in self.neurons:
        params.extend(n.parameters())
    return params

class Network:
    def __init__(self, sizes):
        self.layers = []
        for i in range(len(sizes) - 1):
            self.layers.append(Layer(sizes[i], sizes[i + 1]))

    def __call__(self, x):
        for layer in self.layers:
            x = layer(x)
            if not isinstance(x, list):
                x = [x]
        return x[0] if len(x) == 1 else x

    def parameters(self):
        params = []
        for layer in self.layers:
            params.extend(layer.parameters())
        return params

    def zero_grad(self):
        for p in self.parameters():
            p.grad = 0.0

```

A Neuron takes inputs, computes weighted sum + bias, and applies sigmoid. Weight initialization scales by $\sqrt{2/n_inputs}$ to prevent sigmoid saturation in deeper networks. A Layer is a list of Neurons. A Network is a list of Layers. The `parameters()` method collects all learnable Values so we can update them.

Step 6: Train on XOR

```

random.seed(42)
net = Network([2, 4, 1])

xor_data = [
    ([0.0, 0.0], 0.0),
    ([0.0, 1.0], 1.0),
    ([1.0, 0.0], 1.0),
    ([1.0, 1.0], 0.0),
]

learning_rate = 1.0

for epoch in range(1000):
    total_loss = Value(0.0)
    for inputs, target in xor_data:
        x = [Value(i) for i in inputs]
        pred = net(x)

```

```

        loss = mse_loss(pred, target)
        total_loss = total_loss + loss

    net.zero_grad()
    total_loss.backward()

    for p in net.parameters():
        p.data -= learning_rate * p.grad

    if epoch % 100 == 0:
        print(f"Epoch {epoch:4d} | Loss: {total_loss.data:.6f}")

print("\nXOR Results:")
for inputs, target in xor_data:
    x = [Value(i) for i in inputs]
    pred = net(x)
    print(f"    {inputs} -> {pred.data:.4f} (expected {target})")

```

Watch the loss decrease. From random predictions to correct XOR outputs, driven entirely by backpropagation computing gradients and nudging weights in the right direction.

Step 7: Circle Classification

In Lesson 02, you hand-tuned weights for circle classification. Now let the network learn them.

```

random.seed(7)

def generate_circle_data(n=100):
    data = []
    for _ in range(n):
        x1 = random.uniform(-1.5, 1.5)
        x2 = random.uniform(-1.5, 1.5)
        label = 1.0 if x1 * x1 + x2 * x2 < 1.0 else 0.0
        data.append([x1, x2], label)
    return data

circle_data = generate_circle_data(80)

circle_net = Network([2, 8, 1])
learning_rate = 0.5

for epoch in range(2000):
    random.shuffle(circle_data)
    total_loss_val = 0.0
    for inputs, target in circle_data:
        x = [Value(i) for i in inputs]
        pred = circle_net(x)
        loss = mse_loss(pred, target)
        circle_net.zero_grad()
        loss.backward()
        for p in circle_net.parameters():
            p.data -= learning_rate * p.grad
    total_loss_val += loss.data

```

```

if epoch % 200 == 0:
    correct = 0
    for inputs, target in circle_data:
        x = [Value(i) for i in inputs]
        pred = circle_net(x)
        predicted_class = 1.0 if pred.data > 0.5 else 0.0
        if predicted_class == target:
            correct += 1
    accuracy = correct / len(circle_data) * 100
    print(f"Epoch {epoch:4d} | Loss: {total_loss_val:.4f} | Accuracy:
    ↪ {accuracy:.1f}%")

```

We use online SGD here - update weights after each sample instead of accumulating the full batch. This breaks symmetry faster and avoids sigmoid saturation on the full loss landscape. Shuffling the data each epoch prevents the network from memorizing the order.

No hand-tuning. The network discovers the circular decision boundary on its own. That's the power of backpropagation: you define the architecture, the loss function, and the data. The algorithm figures out the weights.

Use It

PyTorch does everything above in a few lines. The core idea is identical - autograd builds a computational graph during the forward pass and traces it backward to compute gradients.

```

import torch
import torch.nn as nn

```

```

model = nn.Sequential(
    nn.Linear(2, 4),
    nn.Sigmoid(),
    nn.Linear(4, 1),
    nn.Sigmoid(),
)
optimizer = torch.optim.SGD(model.parameters(), lr=1.0)
criterion = nn.MSELoss()

```

```

X = torch.tensor([[0,0],[0,1],[1,0],[1,1]], dtype=torch.float32)
y = torch.tensor([[0],[1],[1],[0]], dtype=torch.float32)

```

```

for epoch in range(1000):
    pred = model(X)
    loss = criterion(pred, y)
    optimizer.zero_grad()
    loss.backward()
    optimizer.step()

```

```

print("PyTorch XOR Results:")
with torch.no_grad():
    for i in range(4):
        pred = model(X[i])
        print(f" {X[i].tolist()} -> {pred.item():.4f} (expected {y[i].item()})")

```

loss.backward() is your total_loss.backward(). optimizer.step() is your manual p.data -= lr * p.grad. optimizer.zero_grad() is your net.zero_grad(). Same algorithm,

industrial-strength implementation. PyTorch handles GPU acceleration, mixed precision, gradient checkpointing, and hundreds of layer types. But the backward pass is the same chain rule applied to the same computational graph.

Training runs the forward pass, then the backward pass, then updates weights. Inference runs only the forward pass. No gradients, no updates. This distinction matters because inference is what happens in production. When you call an API like Claude or GPT, you're running inference - your prompt flows forward through the network, and tokens come out the other end. No weights change. Understanding backprop matters because it shaped every weight in that network.

This chapter ships an artifact. The course version of this lesson produces a reusable prompt or agent skill. It lives in the repository, ready to install: <https://github.com/rohitg00/ai-engineering-from-scratch/tree/main/phases/03-deep-learning-core/03-backpropagation>

Exercises

Starter code and the lesson's working implementation: <https://github.com/rohitg00/ai-engineering-from-scratch/tree/main/phases/03-deep-learning-core/03-backpropagation/code>

1. Add a `__sub__` method to the `Value` class ($a - b = a + (-1 * b)$). Then implement a `__neg__` method. Verify that the gradients are correct by comparing with manual calculation for a simple expression like $(a - b)^2$.
2. Add a `relu` method to `Value` (output $\max(0, x)$, derivative is 1 if $x > 0$, else 0). Replace sigmoid with `relu` in the hidden layers and train on XOR again. Compare convergence speed. You should see faster training - this previews Lesson 04.
3. Implement a `__pow__` method on `Value` for integer powers. Use it to replace `mse_loss` with a proper $(\text{predicted} - \text{target}) ** 2$ expression. Verify gradients match the original implementation.
4. Add gradient clipping to the training loop: after calling `backward()`, clip all gradients to $[-1, 1]$. Train a deeper network (4+ layers with sigmoid) and compare loss curves with and without clipping. This is your first defense against exploding gradients.
5. Build a visualization: after training on XOR, print the gradient of every parameter in the network. Identify which layer has the smallest gradients. This demonstrates the vanishing gradient problem you read about in the Concept section.

Key Terms

Term	What people say	What it actually means
Backpropagation	"The network learns"	An algorithm that computes dL/dw for every weight by applying the chain rule backward through the computational graph
Computational graph	"The network structure"	A directed acyclic graph where nodes are operations and edges carry values (forward) and gradients (backward)
Chain rule	"Multiply the derivatives"	If $y = f(g(x))$, then $dy/dx = f'(g(x)) * g'(x)$ - the mathematical foundation of backpropagation

Term	What people say	What it actually means
Gradient	“The direction of steepest ascent”	The partial derivative of the loss with respect to a parameter – tells you how to change that parameter to reduce the loss
Vanishing gradient	“Deep networks don’t learn”	Gradients shrink exponentially as they propagate through layers with saturating activations like sigmoid
Forward pass	“Running the network”	Computing the output from inputs by sequentially applying each layer’s operations and storing intermediate values
Backward pass	“Computing gradients”	Traversing the computational graph in reverse, accumulating gradients at each node using the chain rule
Learning rate	“How fast it learns”	A scalar that controls the step size when updating weights: $w_{\text{new}} = w_{\text{old}} - \text{lr} * \text{gradient}$
Topological sort	“The right order”	An ordering of graph nodes where each node appears after all nodes it depends on – ensures gradients are fully accumulated before propagation
Autograd	“Automatic differentiation”	A system that builds computational graphs during forward computation and automatically computes gradients – what PyTorch’s engine does

Further Reading

- Rumelhart, Hinton & Williams, “Learning representations by back-propagating errors” (1986) – the paper that made backpropagation mainstream and unlocked multi-layer network training
- 3Blue1Brown, “Neural Networks” series (https://www.youtube.com/playlist?list=PLZHQObOWTQDNU6R1_67000Dx_ZCJB-3pi) – the best visual explanation of backpropagation and gradient flow through networks

Continue online. The living edition of this chapter has more than the page can hold:

- Animated, interactive figures and the web text: <https://aiengineeringfromscratch.com/lesson?path=phases/03-deep-learning-core/03-backpropagation>
- Runnable code for every step: <https://github.com/rohitg00/ai-engineering-from-scratch/tree/main/phases/03-deep-learning-core/03-backpropagation/code>
- The chapter quiz, graded in the browser: <https://aiengineeringfromscratch.com/lesson?path=phases/03-deep-learning-core/03-backpropagation>

The repository moves faster than any printing. When the book and the repo disagree, trust the repo.

Activation Functions

Without nonlinearity, your 100-layer network is a fancy matrix multiply. Activations are the gates that let neural networks think in curves.

Type: Build **Languages:** Python **Prerequisites:** Lesson 03.03 (Backpropagation) **Time:** ~75 minutes

Learning Objectives

- Implement sigmoid, tanh, ReLU, Leaky ReLU, GELU, Swish, and softmax with their derivatives from scratch
- Diagnose the vanishing gradient problem by measuring activation magnitudes through 10+ layers with different activations
- Detect dead neurons in a ReLU network and explain why GELU avoids this failure mode
- Select the correct activation function for a given architecture (transformer, CNN, RNN, output layer)

The Problem

Stack two linear transformations: $y = W_2(W_1x + b_1) + b_2$. Expand it: $y = W_2W_1x + W_2b_1 + b_2$. That's just $y = Ax + c$ - a single linear transformation. No matter how many linear layers you stack, the result collapses to one matrix multiply. Your 100-layer network has the same representational power as a single layer.

This is not a theoretical curiosity. It means a deep linear network literally cannot learn XOR, cannot classify a spiral dataset, cannot recognize a face. Without activation functions, depth is an illusion.

Activation functions break the linearity. They warp the output of each layer through a non-linear function, giving the network the ability to bend decision boundaries, approximate arbitrary functions, and actually learn. But pick the wrong activation and your gradients vanish to zero (sigmoid in deep networks), explode to infinity (unbounded activations without careful initialization), or your neurons die permanently (ReLU with large negative biases). The choice of activation function directly determines whether your network learns at all.

The Concept

Why Nonlinearity Is Necessary

Matrix multiplication is composable. Multiplying a vector by matrix A then matrix B is identical to multiplying by AB. This means stacking ten linear layers is mathematically equivalent to one linear layer with one big matrix. All those parameters, all that depth - wasted. You need something to break the chain. That's what activation functions do.

Here is the proof. A linear layer computes $f(x) = Wx + b$. Stack two:

Layer 1: $h = W1 * x + b1$

Layer 2: $y = W2 * h + b2$

Substitute:

$$y = W2 * (W1 * x + b1) + b2$$

$$y = (W2 * W1) * x + (W2 * b1 + b2)$$

$$y = A * x + c$$

One layer. Insert a nonlinear activation $g()$ between layers:

$$h = g(W1 * x + b1)$$

$$y = W2 * h + b2$$

Now the substitution breaks. $W2 * g(W1 * x + b1) + b2$ cannot be reduced to a single linear transformation. The network can represent nonlinear functions. Each additional layer with an activation adds representational capacity.

Sigmoid

The original activation function for neural networks.

$$\text{sigmoid}(x) = 1 / (1 + e^{(-x)})$$

Output range: (0, 1). Smooth, differentiable, maps any real number to a probability-like value.

The derivative:

$$\text{sigmoid}'(x) = \text{sigmoid}(x) * (1 - \text{sigmoid}(x))$$

The maximum value of this derivative is 0.25, occurring at $x = 0$. In backpropagation, gradients multiply through layers. Ten layers of sigmoid means the gradient gets multiplied by at most 0.25 ten times:

$$0.25^{10} = 0.000000953674$$

Less than one millionth of the original signal. This is the vanishing gradient problem. Gradients in early layers become so small that weights barely update. The network appears to learn – loss decreases in later layers – but the first layers are frozen. Deep sigmoid networks simply do not train.

Additional problem: sigmoid outputs are always positive (0 to 1), which means gradients on weights are always the same sign. This causes zig-zagging during gradient descent.

Tanh

The centered version of sigmoid.

$$\tanh(x) = (e^x - e^{(-x)}) / (e^x + e^{(-x)})$$

Output range: (-1, 1). Zero-centered, which eliminates the zig-zag problem.

The derivative:

$$\tanh'(x) = 1 - \tanh(x)^2$$

Maximum derivative is 1.0 at $x = 0$ – four times better than sigmoid. But the vanishing gradient problem still exists. For large positive or negative inputs, the derivative approaches zero. Ten layers still crush the gradient, just less aggressively.

ReLU: The Breakthrough

Rectified Linear Unit. Popularized for deep learning by Nair and Hinton in 2010 (the function itself dates to Fukushima's 1969 work), it changed everything.

$$\text{relu}(x) = \max(0, x)$$

Output range: $[0, \infty)$. The derivative is trivially simple:

$$\text{relu}'(x) = \begin{cases} 1 & \text{if } x > 0 \\ 0 & \text{if } x \leq 0 \end{cases}$$

No vanishing gradient for positive inputs. The gradient is exactly 1, passed straight through. This is why deep networks became trainable - ReLU preserves gradient magnitude across layers.

But there is a failure mode: the dead neuron problem. If a neuron's weighted input is always negative (due to a large negative bias or unfortunate weight initialization), its output is always zero, its gradient is always zero, and it never updates. It is permanently dead. In practice, 10-40% of neurons in a ReLU network can die during training.

Leaky ReLU

The simplest fix for dead neurons.

$$\text{leaky_relu}(x) = \begin{cases} x & \text{if } x > 0 \\ \alpha * x & \text{if } x \leq 0 \end{cases}$$

Where α is a small constant, typically 0.01. The negative side has a small slope instead of zero, so dead neurons still get a gradient signal and can recover.

GELU: The Modern Default

Gaussian Error Linear Unit. Introduced by Hendrycks and Gimpel in 2016. Default activation in BERT, GPT, and most modern transformers.

$$\text{gelu}(x) = x * \Phi(x)$$

Where $\Phi(x)$ is the cumulative distribution function of the standard normal distribution. The approximation used in practice:

$$\text{gelu}(x) \approx 0.5 * x * (1 + \tanh(\sqrt{2/\pi}) * (x + 0.044715 * x^3))$$

GELU is smooth everywhere, allows small negative values (unlike ReLU which hard-clips to zero), and has a probabilistic interpretation: it weights each input by how likely it is to be positive under a Gaussian distribution. This smooth gating outperforms ReLU in transformer architectures because it provides better gradient flow and avoids the dead neuron problem entirely.

Swish / SiLU

Self-gated activation discovered by Ramachandran et al. in 2017 through automated search.

$$\text{swish}(x) = x * \text{sigmoid}(x)$$

Swish is formally $x * \text{sigmoid}(x)$. Google discovered it through automated search over activation function space - a neural network designing parts of neural networks.

Like GELU, it is smooth, non-monotonic, and allows small negative values. The difference is subtle: Swish uses sigmoid for gating while GELU uses the Gaussian CDF. In practice,

performance is nearly identical. Swish is used in EfficientNet and some vision models. GELU dominates in language models.

Softmax: The Output Activation

Not used in hidden layers. Softmax converts a vector of raw scores (logits) into a probability distribution.

$$\text{softmax}(x_i) = e^{(x_i)} / \sum(e^{(x_j)} \text{ for all } j)$$

Every output is between 0 and 1. All outputs sum to 1. This makes it the standard final activation for multi-class classification. The largest logit gets the highest probability, but unlike argmax, softmax is differentiable and preserves information about relative confidence.

Comparison of Shapes

Diagram. Rendered live in the web edition: <https://aiengineeringfromscratch.com/lesson?path=phases/03-deep-learning-core/04-activation-functions>

Gradient Flow Comparison

Diagram. Rendered live in the web edition: <https://aiengineeringfromscratch.com/lesson?path=phases/03-deep-learning-core/04-activation-functions>

Which Activation When

Diagram. Rendered live in the web edition: <https://aiengineeringfromscratch.com/lesson?path=phases/03-deep-learning-core/04-activation-functions>

Interactive figure: softmax-temperature. This one moves. Watch it animate and drag its controls in the web edition: <https://aiengineeringfromscratch.com/lesson?path=phases/03-deep-learning-core/04-activation-functions>

Build It

Step 1: Implement All Activation Functions with Derivatives

Each function takes a single float and returns a float. Each derivative function takes the same input and returns the gradient.

```
import math

def sigmoid(x):
    x = max(-500, min(500, x))
    return 1.0 / (1.0 + math.exp(-x))

def sigmoid_derivative(x):
    s = sigmoid(x)
    return s * (1 - s)

def tanh_act(x):
    return math.tanh(x)

def tanh_derivative(x):
```

```

    t = math.tanh(x)
    return 1 - t * t

def relu(x):
    return max(0.0, x)

def relu_derivative(x):
    return 1.0 if x > 0 else 0.0

def leaky_relu(x, alpha=0.01):
    return x if x > 0 else alpha * x

def leaky_relu_derivative(x, alpha=0.01):
    return 1.0 if x > 0 else alpha

def gelu(x):
    return 0.5 * x * (1 + math.tanh(math.sqrt(2 / math.pi) * (x + 0.044715 * x **
↪ 3)))

def gelu_derivative(x):
    phi = 0.5 * (1 + math.erf(x / math.sqrt(2)))
    pdf = math.exp(-0.5 * x * x) / math.sqrt(2 * math.pi)
    return phi + x * pdf

def swish(x):
    return x * sigmoid(x)

def swish_derivative(x):
    s = sigmoid(x)
    return s + x * s * (1 - s)

def softmax(xs):
    max_x = max(xs)
    exps = [math.exp(x - max_x) for x in xs]
    total = sum(exps)
    return [e / total for e in exps]

```

Step 2: Visualize Where Gradients Die

Compute the gradient at 100 evenly-spaced points from -5 to 5. Print a text histogram showing where each activation's gradient is near-zero.

```

def gradient_scan(name, derivative_fn, start=-5, end=5, n=100):
    step = (end - start) / n
    near_zero = 0
    healthy = 0
    for i in range(n):
        x = start + i * step
        g = derivative_fn(x)
        if abs(g) < 0.01:
            near_zero += 1
        else:
            healthy += 1
    pct_dead = near_zero / n * 100

```

```

print(f"{name:15s}: {healthy:3d} healthy, {near_zero:3d} near-zero
    ↪ ({pct_dead:.0f}% dead zone)")

gradient_scan("Sigmoid", sigmoid_derivative)
gradient_scan("Tanh", tanh_derivative)
gradient_scan("ReLU", relu_derivative)
gradient_scan("Leaky ReLU", leaky_relu_derivative)
gradient_scan("GELU", gelu_derivative)
gradient_scan("Swish", swish_derivative)

```

Step 3: Vanishing Gradient Experiment

Forward-pass a signal through N layers using sigmoid vs ReLU. Measure how the activation magnitude changes.

```

import random

def vanishing_gradient_experiment(activation_fn, name, n_layers=10, n_inputs=5):
    random.seed(42)
    values = [random.gauss(0, 1) for _ in range(n_inputs)]

    print(f"\n{name} through {n_layers} layers:")
    for layer in range(n_layers):
        weights = [random.gauss(0, 1) for _ in range(n_inputs)]
        z = sum(w * v for w, v in zip(weights, values))
        activated = activation_fn(z)
        magnitude = abs(activated)
        bar = "#" * int(magnitude * 20)
        print(f"  Layer {layer+1:2d}: magnitude = {magnitude:.6f} {bar}")
        values = [activated] * n_inputs

vanishing_gradient_experiment(sigmoid, "Sigmoid")
vanishing_gradient_experiment(relu, "ReLU")
vanishing_gradient_experiment(gelu, "GELU")

```

Step 4: Dead Neuron Detector

Create a ReLU network, pass random inputs through it, count how many neurons never fire.

```

def dead_neuron_detector(n_inputs=5, hidden_size=20, n_samples=1000):
    random.seed(0)
    weights = [[random.gauss(0, 1) for _ in range(n_inputs)] for _ in
    ↪ range(hidden_size)]
    biases = [random.gauss(0, 1) for _ in range(hidden_size)]

    fire_counts = [0] * hidden_size

    for _ in range(n_samples):
        inputs = [random.gauss(0, 1) for _ in range(n_inputs)]
        for neuron_idx in range(hidden_size):
            z = sum(w * x for w, x in zip(weights[neuron_idx], inputs)) +
            ↪ biases[neuron_idx]
            if relu(z) > 0:
                fire_counts[neuron_idx] += 1

```

```

dead = sum(1 for c in fire_counts if c == 0)
rarely_fire = sum(1 for c in fire_counts if 0 < c < n_samples * 0.05)
healthy = hidden_size - dead - rarely_fire

print(f"\nDead Neuron Report ({hidden_size} neurons, {n_samples} samples):")
print(f"  Dead (never fired):      {dead}")
print(f"  Barely alive (<5%):        {rarely_fire}")
print(f"  Healthy:                    {healthy}")
print(f"  Dead neuron rate:          {dead/hidden_size*100:.1f}%")

for i, c in enumerate(fire_counts):
    status = "DEAD" if c == 0 else "WEAK" if c < n_samples * 0.05 else "OK"
    bar = "#" * (c * 40 // n_samples)
    print(f"  Neuron {i:2d}: {c:4d}/{n_samples} fires [{status:4s}] {bar}")

dead_neuron_detector()

```

Step 5: Training Comparison - Sigmoid vs ReLU vs GELU

Train the same two-layer network on the circle dataset (points inside a circle = class 1, outside = class 0) with three different activations. Compare convergence speed.

```

def make_circle_data(n=200, seed=42):
    random.seed(seed)
    data = []
    for _ in range(n):
        x = random.uniform(-2, 2)
        y = random.uniform(-2, 2)
        label = 1.0 if x * x + y * y < 1.5 else 0.0
        data.append([x, y], label))
    return data

class ActivationNetwork:
    def __init__(self, activation_fn, activation_deriv, hidden_size=8, lr=0.1):
        random.seed(0)
        self.act = activation_fn
        self.act_d = activation_deriv
        self.lr = lr
        self.hidden_size = hidden_size

        self.w1 = [[random.gauss(0, 0.5) for _ in range(2)] for _ in
                    range(hidden_size)]
        self.b1 = [0.0] * hidden_size
        self.w2 = [random.gauss(0, 0.5) for _ in range(hidden_size)]
        self.b2 = 0.0

    def forward(self, x):
        self.x = x
        self.z1 = []
        self.h = []
        for i in range(self.hidden_size):
            z = self.w1[i][0] * x[0] + self.w1[i][1] * x[1] + self.b1[i]
            self.z1.append(z)

```



```

        self.h.append(self.act(z))

    self.z2 = sum(self.w2[i] * self.h[i] for i in range(self.hidden_size)) +
        ↪ self.b2
    self.out = sigmoid(self.z2)
    return self.out

def backward(self, target):
    error = self.out - target
    d_out = error * self.out * (1 - self.out)

    for i in range(self.hidden_size):
        d_h = d_out * self.w2[i] * self.act_d(self.z1[i])
        self.w2[i] -= self.lr * d_out * self.h[i]
        for j in range(2):
            self.w1[i][j] -= self.lr * d_h * self.x[j]
        self.b1[i] -= self.lr * d_h
    self.b2 -= self.lr * d_out

def train(self, data, epochs=200):
    losses = []
    for epoch in range(epochs):
        total_loss = 0
        correct = 0
        for x, y in data:
            pred = self.forward(x)
            self.backward(y)
            total_loss += (pred - y) ** 2
            if (pred >= 0.5) == (y >= 0.5):
                correct += 1
        avg_loss = total_loss / len(data)
        accuracy = correct / len(data) * 100
        losses.append(avg_loss)
        if epoch % 50 == 0 or epoch == epochs - 1:
            print(f"    Epoch {epoch:3d}: loss={avg_loss:.4f},
                ↪ accuracy={accuracy:.1f}%")
    return losses

data = make_circle_data()

configs = [
    ("Sigmoid", sigmoid, sigmoid_derivative),
    ("ReLU", relu, relu_derivative),
    ("GELU", gelu, gelu_derivative),
]

results = {}
for name, act_fn, act_d_fn in configs:
    print(f"\n=== Training with {name} ===")
    net = ActivationNetwork(act_fn, act_d_fn, hidden_size=8, lr=0.1)
    losses = net.train(data, epochs=200)
    results[name] = losses

```

```
print("\n=== Final Loss Comparison ===")
for name, losses in results.items():
    print(f" {name:10s}: start={losses[0]:.4f} -> end={losses[-1]:.4f}
    ↪ (improvement: {(1 - losses[-1]/losses[0])*100:.1f}%)" )
```

Use It

PyTorch provides all of these as both functional and module forms:

```
import torch
import torch.nn as nn
import torch.nn.functional as F

x = torch.randn(4, 10)

relu_out = F.relu(x)
gelu_out = F.gelu(x)
sigmoid_out = torch.sigmoid(x)
swish_out = F.silu(x)

logits = torch.randn(4, 5)
probs = F.softmax(logits, dim=1)

model = nn.Sequential(
    nn.Linear(10, 64),
    nn.GELU(),
    nn.Linear(64, 32),
    nn.GELU(),
    nn.Linear(32, 5),
)
```

Hidden layers in a transformer: GELU. Hidden layers in a CNN: ReLU. Output layer for classification: softmax. Output layer for regression: none (linear). Output layer for probabilities: sigmoid. That's it. Start with these defaults. Change them only when you have evidence.

RNNs and LSTMs use tanh for hidden state and sigmoid for gates, but if you're building from scratch today, you're probably not using RNNs. If neurons are dying in your ReLU network, switch to GELU. Don't reach for Leaky ReLU unless you have a specific reason – GELU solves the dead neuron problem and gives better gradient flow.

This chapter ships an artifact. The course version of this lesson produces a reusable prompt or agent skill. It lives in the repository, ready to install: <https://github.com/rohitg00/ai-engineering-from-scratch/tree/main/phases/03-deep-learning-core/04-activation-functions>

Exercises

Starter code and the lesson's working implementation: <https://github.com/rohitg00/ai-engineering-from-scratch/tree/main/phases/03-deep-learning-core/04-activation-functions/code>

1. Implement Parametric ReLU (PReLU) where the negative slope alpha is a learnable parameter. Train it on the circle dataset and compare to fixed Leaky ReLU.
2. Run the vanishing gradient experiment with 50 layers instead of 10. Plot the magnitude at each layer for sigmoid, tanh, ReLU, and GELU. At which layer does each activation's signal effectively reach zero?

3. Implement the ELU (Exponential Linear Unit): $\text{elu}(x) = x$ if $x > 0$, $\alpha * (e^{-x} - 1)$ if $x \leq 0$. Compare its dead neuron rate to ReLU on the same network.
4. Build a “gradient health monitor” that runs during training: at each epoch, compute the average gradient magnitude at each layer. Print a warning when any layer’s gradient drops below 0.001 or exceeds 100.
5. Modify the training comparison to use the XOR dataset from Lesson 01 instead of circles. Which activation converges fastest on XOR? Why does this differ from the circle results?

Key Terms

Term	What people say	What it actually means
Activation function	“The nonlinear part”	A function applied to each neuron’s output that breaks linearity, enabling the network to learn nonlinear mappings
Vanishing gradient	“Gradients disappear in deep networks”	Gradients shrink exponentially through layers when the activation’s derivative is less than 1, making early layers untrainable
Exploding gradient	“Gradients blow up”	Gradients grow exponentially through layers when the effective multiplier exceeds 1, causing unstable training
Dead neuron	“A neuron that stopped learning”	A ReLU neuron whose input is permanently negative, producing zero output and zero gradient
Sigmoid	“Squishes values to 0-1”	The logistic function $1/(1+e^{-x})$, historically important but causes vanishing gradients in deep networks
ReLU	“Clips negatives to zero”	$\max(0, x)$ – the activation that made deep learning practical by preserving gradient magnitude
GELU	“The transformer activation”	Gaussian Error Linear Unit, a smooth activation that weights inputs by their probability of being positive
Swish/SiLU	“Self-gated ReLU”	$x * \text{sigmoid}(x)$, discovered through automated search, used in EfficientNet
Softmax	“Turns scores into probabilities”	Normalizes a vector of logits into a probability distribution where all values are in (0,1) and sum to 1
Leaky ReLU	“ReLU that doesn’t die”	$\max(\alpha * x, x)$ where α is small (0.01), preventing dead neurons by allowing small negative gradients
Saturation	“The flat part of sigmoid”	Regions where an activation’s derivative approaches zero, blocking gradient flow
Logit	“The raw score before softmax”	The unnormalized output of the final layer before applying softmax or sigmoid

Further Reading

- Nair & Hinton, “Rectified Linear Units Improve Restricted Boltzmann Machines” (2010) – the paper that introduced ReLU and enabled training of deep networks

- Hendrycks & Gimpel, “Gaussian Error Linear Units (GELUs)” (2016) - introduced the activation function that became the default for transformers
- Ramachandran et al., “Searching for Activation Functions” (2017) - used automated search to discover Swish, showing that activation design can be automated
- Glorot & Bengio, “Understanding the difficulty of training deep feedforward neural networks” (2010) - the paper that diagnosed vanishing/exploding gradients and proposed Xavier initialization
- Goodfellow, Bengio, Courville, “Deep Learning” Chapter 6.3 (<https://www.deeplearningbook.org/>) - rigorous treatment of hidden units and activation functions

Continue online. The living edition of this chapter has more than the page can hold:

- Animated, interactive figures and the web text: <https://aiengineeringfromscratch.com/lesson?path=phases/03-deep-learning-core/04-activation-functions>
- Runnable code for every step: <https://github.com/rohitg00/ai-engineering-from-scratch/tree/main/phases/03-deep-learning-core/04-activation-functions/code>
- The chapter quiz, graded in the browser: <https://aiengineeringfromscratch.com/lesson?path=phases/03-deep-learning-core/04-activation-functions>

The repository moves faster than any printing. When the book and the repo disagree, trust the repo.

Loss Functions

Your network makes a prediction. The ground truth says otherwise. How wrong is it? That number is the loss. Pick the wrong loss function and your model optimizes for the wrong thing entirely.

Type: Build **Languages:** Python **Prerequisites:** Lesson 03.04 (Activation Functions) **Time:** ~75 minutes

Learning Objectives

- Implement MSE, binary cross-entropy, categorical cross-entropy, and contrastive loss (InfoNCE) from scratch with their gradients
- Explain why MSE fails for classification by demonstrating the “predict 0.5 for everything” failure mode
- Apply label smoothing to cross-entropy and describe how it prevents overconfident predictions
- Choose the correct loss function for regression, binary classification, multi-class classification, and embedding learning tasks

The Problem

A model minimizing MSE on a classification problem will confidently predict 0.5 for everything. It’s minimizing loss. It’s also useless.

The loss function is the only thing your model actually optimizes. Not accuracy. Not F1 score. Not whatever metric you report to your manager. The optimizer takes the gradient of the loss function and adjusts weights to make that number smaller. If the loss function doesn’t capture what you care about, the model will find the mathematically cheapest way to satisfy it, and that way is almost never what you wanted.

Here is a concrete example. You have a binary classification task. Two classes, 50/50 split. You use MSE as your loss. The model predicts 0.5 for every single input. The average MSE is 0.25, which is the minimum possible without actually learning anything. The model has zero discriminative ability but it has technically minimized your loss function. Switch to cross-entropy and the same model is forced to push predictions toward 0 or 1, because $-\log(0.5) = 0.693$ is a terrible loss, while $-\log(0.99) = 0.01$ rewards confident correct predictions. The choice of loss function is the difference between a model that learns and a model that games the metric.

It gets worse. In self-supervised learning, you don’t even have labels. Contrastive loss defines the learning signal entirely: what counts as similar, what counts as different, and how hard the model should push them apart. Get contrastive loss wrong and your embeddings collapse to a single point – every input maps to the same vector. Technically zero loss. Completely worthless.

The Concept

Mean Squared Error (MSE)

The default for regression. Compute the squared difference between prediction and target, average over all samples.

$$\text{MSE} = (1/n) * \text{sum}((y_{\text{pred}} - y_{\text{true}})^2)$$

Why squaring matters: it penalizes large errors quadratically. An error of 2 costs 4x as much as an error of 1. An error of 10 costs 100x. This makes MSE sensitive to outliers – a single wildly wrong prediction dominates the loss.

Real numbers: if your model predicts housing prices and is off by \$10,000 on most houses but off by \$200,000 on one mansion, MSE will aggressively try to fix that one mansion, potentially hurting performance on the other 99 houses.

The gradient of MSE with respect to a prediction is:

$$d\text{MSE}/dy_{\text{pred}} = (2/n) * (y_{\text{pred}} - y_{\text{true}})$$

Linear in the error. Bigger errors get bigger gradients. This is a feature for regression (large errors need large corrections) and a bug for classification (you want to penalize confident wrong answers exponentially, not linearly).

Cross-Entropy Loss

The loss function for classification. Rooted in information theory – it measures the divergence between the predicted probability distribution and the true distribution.

Binary Cross-Entropy (BCE):

$$\text{BCE} = -(y * \log(p) + (1 - y) * \log(1 - p))$$

Where y is the true label (0 or 1) and p is the predicted probability.

Why $-\log(p)$ works: when the true label is 1 and you predict $p = 0.99$, the loss is $-\log(0.99) = 0.01$. When you predict $p = 0.01$, the loss is $-\log(0.01) = 4.6$. That 460x difference is why cross-entropy works. It brutally punishes confident wrong predictions while barely penalizing confident correct ones.

The gradient tells the same story:

$$dBCE/dp = -(y/p) + (1-y)/(1-p)$$

When $y = 1$ and p is near zero, the gradient is $-1/p$ which approaches negative infinity. The model gets an enormous signal to fix its mistake. When p is near 1, the gradient is tiny. Already correct, nothing to fix.

Categorical Cross-Entropy:

For multi-class classification with one-hot encoded targets.

$$\text{CCE} = -\text{sum}(y_i * \log(p_i))$$

Only the true class contributes to the loss (because all other y_i are zero). If there are 10 classes and the correct class gets probability 0.1 (random guessing), the loss is $-\log(0.1) = 2.3$. If the correct class gets probability 0.9, the loss is $-\log(0.9) = 0.105$. The model learns to concentrate probability mass on the right answer.

Why MSE Fails for Classification

Diagram. Rendered live in the web edition: <https://aiengineeringfromscratch.com/lesson?path=phases/03-deep-learning-core/05-loss-functions>

MSE gradients flatten when predictions are near 0 or 1 (due to sigmoid saturation). Cross-entropy gradients compensate for this – the $-\log$ cancels the sigmoid’s flat regions, giving strong gradients exactly where they are needed most.

Label Smoothing

Standard one-hot labels say “this is 100% class 3 and 0% everything else.” That’s a strong claim. Label smoothing softens it:

$$\text{smooth_label} = (1 - \alpha) * \text{one_hot} + \alpha / \text{num_classes}$$

With $\alpha = 0.1$ and 10 classes: instead of $[0, 0, 1, 0, \dots]$, the target becomes $[0.01, 0.01, 0.91, 0.01, \dots]$. The model targets 0.91 instead of 1.0.

Why this works: a model trying to output exactly 1.0 through a softmax needs to push logits to infinity. This causes overconfidence, hurts generalization, and makes the model brittle to distribution shift. Label smoothing caps the target at 0.9 (with $\alpha=0.1$), keeping logits in a reasonable range. GPT and most modern models use label smoothing or its equivalent.

Contrastive Loss

No labels. No classes. Just pairs of inputs and the question: are these similar or different?

SimCLR-style contrastive loss (NT-Xent / InfoNCE):

Take one image. Create two augmented views of it (crop, rotate, color jitter). These are the “positive pair” – they should have similar embeddings. Every other image in the batch forms a “negative pair” – they should have different embeddings.

$$L = -\log(\exp(\text{sim}(z_i, z_j) / \tau) / \sum(\exp(\text{sim}(z_i, z_k) / \tau)))$$

Where $\text{sim}()$ is cosine similarity, z_i and z_j are the positive pair, the sum is over all negatives, and τ (temperature) controls how sharp the distribution is. Lower temperature = harder negatives = more aggressive separation.

Real numbers: batch size 256 means 255 negatives per positive pair. Temperature $\tau = 0.07$ (SimCLR default). The loss looks like a softmax over similarities – it wants the positive pair’s similarity to be highest among all 256 options.

Triplet Loss:

Takes three inputs: anchor, positive (same class), negative (different class).

$$L = \max(0, d(\text{anchor}, \text{positive}) - d(\text{anchor}, \text{negative}) + \text{margin})$$

The margin (typically 0.2-1.0) enforces a minimum gap between positive and negative distances. If the negative is already far enough away, the loss is zero – no gradient, no update. This makes training efficient but requires careful triplet mining (choosing hard negatives that are close to the anchor).

Focal Loss

For imbalanced datasets. Standard cross-entropy treats all correctly classified examples equally. Focal loss down-weights easy examples:

$$FL = -\alpha * (1 - p_t)^\gamma * \log(p_t)$$

Where p_t is the predicted probability of the true class and gamma controls the focusing. With $\gamma = 0$, this is standard cross-entropy. With $\gamma = 2$ (the default):

- Easy example ($p_t = 0.9$): $\text{weight} = (0.1)^2 = 0.01$. Effectively ignored.
- Hard example ($p_t = 0.1$): $\text{weight} = (0.9)^2 = 0.81$. Full gradient signal.

Focal loss was introduced by Lin et al. for object detection, where 99% of candidate regions are background (easy negatives). Without focal loss, the model drowns in easy background examples and never learns to detect objects. With it, the model focuses its capacity on the hard, ambiguous cases that matter.

Loss Function Decision Tree

Diagram. Rendered live in the web edition: <https://aiengineeringfromscratch.com/lesson?path=phases/03-deep-learning-core/05-loss-functions>

Loss Landscape

Diagram. Rendered live in the web edition: <https://aiengineeringfromscratch.com/lesson?path=phases/03-deep-learning-core/05-loss-functions>

Interactive figure: cross-entropy-loss. This one moves. Watch it animate and drag its controls in the web edition: <https://aiengineeringfromscratch.com/lesson?path=phases/03-deep-learning-core/05-loss-functions>

Build It

Step 1: MSE and Its Gradient

```
def mse(predictions, targets):
    n = len(predictions)
    total = 0.0
    for p, t in zip(predictions, targets):
        total += (p - t) ** 2
    return total / n

def mse_gradient(predictions, targets):
    n = len(predictions)
    grads = []
    for p, t in zip(predictions, targets):
        grads.append(2.0 * (p - t) / n)
    return grads
```

Step 2: Binary Cross-Entropy

The $\log(0)$ problem is real. If the model predicts exactly 0 for a positive example, $\log(0) =$ negative infinity. Clipping prevents this.

```
import math

def binary_cross_entropy(predictions, targets, eps=1e-15):
    n = len(predictions)
    total = 0.0
    for p, t in zip(predictions, targets):
```



```

        p_clipped = max(eps, min(1 - eps, p))
        total += -(t * math.log(p_clipped) + (1 - t) * math.log(1 - p_clipped))
    return total / n

def bce_gradient(predictions, targets, eps=1e-15):
    grads = []
    for p, t in zip(predictions, targets):
        p_clipped = max(eps, min(1 - eps, p))
        grads.append(-(t / p_clipped) + (1 - t) / (1 - p_clipped))
    return grads

```

Step 3: Categorical Cross-Entropy with Softmax

Softmax converts raw logits to probabilities. Then we compute the cross-entropy against one-hot targets.

```

def softmax(logits):
    max_val = max(logits)
    exps = [math.exp(x - max_val) for x in logits]
    total = sum(exps)
    return [e / total for e in exps]

def categorical_cross_entropy(logits, target_index, eps=1e-15):
    probs = softmax(logits)
    p = max(eps, probs[target_index])
    return -math.log(p)

def cce_gradient(logits, target_index):
    probs = softmax(logits)
    grads = list(probs)
    grads[target_index] -= 1.0
    return grads

```

The gradient of softmax + cross-entropy simplifies beautifully: it's just (predicted probability - 1) for the true class, and (predicted probability) for all other classes. This elegant simplification is not a coincidence - it's why softmax and cross-entropy are paired.

Step 4: Label Smoothing

```

def label_smoothed_cce(logits, target_index, num_classes, alpha=0.1, eps=1e-15):
    probs = softmax(logits)
    loss = 0.0
    for i in range(num_classes):
        if i == target_index:
            smooth_target = 1.0 - alpha + alpha / num_classes
        else:
            smooth_target = alpha / num_classes
        p = max(eps, probs[i])
        loss += -smooth_target * math.log(p)
    return loss

```

Step 5: Contrastive Loss (Simplified InfoNCE)

```
def cosine_similarity(a, b):
    dot = sum(x * y for x, y in zip(a, b))
    norm_a = math.sqrt(sum(x * x for x in a))
    norm_b = math.sqrt(sum(x * x for x in b))
    if norm_a < 1e-10 or norm_b < 1e-10:
        return 0.0
    return dot / (norm_a * norm_b)

def contrastive_loss(anchor, positive, negatives, temperature=0.07):
    sim_pos = cosine_similarity(anchor, positive) / temperature
    sim_negs = [cosine_similarity(anchor, neg) / temperature for neg in negatives]

    max_sim = max(sim_pos, max(sim_negs)) if sim_negs else sim_pos
    exp_pos = math.exp(sim_pos - max_sim)
    exp_negs = [math.exp(s - max_sim) for s in sim_negs]
    total_exp = exp_pos + sum(exp_negs)

    return -math.log(max(1e-15, exp_pos / total_exp))
```

Step 6: MSE vs Cross-Entropy on Classification

Train the same network from lesson 04 (circle dataset) with both loss functions. Watch cross-entropy converge faster.

```
import random

def sigmoid(x):
    x = max(-500, min(500, x))
    return 1.0 / (1.0 + math.exp(-x))

def make_circle_data(n=200, seed=42):
    random.seed(seed)
    data = []
    for _ in range(n):
        x = random.uniform(-2, 2)
        y = random.uniform(-2, 2)
        label = 1.0 if x * x + y * y < 1.5 else 0.0
        data.append([x, y], label)
    return data

class LossComparisonNetwork:
    def __init__(self, loss_type="bce", hidden_size=8, lr=0.1):
        random.seed(0)
        self.loss_type = loss_type
        self.lr = lr
        self.hidden_size = hidden_size

        self.w1 = [[random.gauss(0, 0.5) for _ in range(2)] for _ in
                    range(hidden_size)]
        self.b1 = [0.0] * hidden_size
        self.w2 = [random.gauss(0, 0.5) for _ in range(hidden_size)]
```

```

self.b2 = 0.0

def forward(self, x):
    self.x = x
    self.z1 = []
    self.h = []
    for i in range(self.hidden_size):
        z = self.w1[i][0] * x[0] + self.w1[i][1] * x[1] + self.b1[i]
        self.z1.append(z)
        self.h.append(max(0.0, z))

    self.z2 = sum(self.w2[i] * self.h[i] for i in range(self.hidden_size)) +
        ↪ self.b2
    self.out = sigmoid(self.z2)
    return self.out

def backward(self, target):
    if self.loss_type == "mse":
        d_loss = 2.0 * (self.out - target)
    else:
        eps = 1e-15
        p = max(eps, min(1 - eps, self.out))
        d_loss = -(target / p) + (1 - target) / (1 - p)

    d_sigmoid = self.out * (1 - self.out)
    d_out = d_loss * d_sigmoid

    for i in range(self.hidden_size):
        d_relu = 1.0 if self.z1[i] > 0 else 0.0
        d_h = d_out * self.w2[i] * d_relu
        self.w2[i] -= self.lr * d_out * self.h[i]
        for j in range(2):
            self.w1[i][j] -= self.lr * d_h * self.x[j]
        self.b1[i] -= self.lr * d_h
    self.b2 -= self.lr * d_out

def compute_loss(self, pred, target):
    if self.loss_type == "mse":
        return (pred - target) ** 2
    else:
        eps = 1e-15
        p = max(eps, min(1 - eps, pred))
        return -(target * math.log(p) + (1 - target) * math.log(1 - p))

def train(self, data, epochs=200):
    losses = []
    for epoch in range(epochs):
        total_loss = 0.0
        correct = 0
        for x, y in data:
            pred = self.forward(x)
            self.backward(y)
            total_loss += self.compute_loss(pred, y)
            if (pred >= 0.5) == (y >= 0.5):

```

```

        correct += 1
    avg_loss = total_loss / len(data)
    accuracy = correct / len(data) * 100
    losses.append((avg_loss, accuracy))
    if epoch % 50 == 0 or epoch == epochs - 1:
        print(f"    Epoch {epoch:3d}: loss={avg_loss:.4f},
              ↪ accuracy={accuracy:.1f}%")
    return losses

```

Use It

PyTorch provides all standard loss functions with numerical stability built in:

```

import torch
import torch.nn as nn
import torch.nn.functional as F

predictions = torch.tensor([0.9, 0.1, 0.7], requires_grad=True)
targets = torch.tensor([1.0, 0.0, 1.0])

mse_loss = F.mse_loss(predictions, targets)
bce_loss = F.binary_cross_entropy(predictions, targets)

logits = torch.randn(4, 10)
labels = torch.tensor([3, 7, 1, 9])
ce_loss = F.cross_entropy(logits, labels)
ce_smooth = F.cross_entropy(logits, labels, label_smoothing=0.1)

```

Use `F.cross_entropy` (not `F.nll_loss` plus manual softmax). It combines log-softmax and negative log-likelihood in one numerically stable operation. Applying softmax separately then taking the log is less stable – you lose precision in the subtraction of large exponentials.

For contrastive learning, most teams use custom implementations or libraries like `lightly` or `pytorch-metric-learning`. The core loop is always the same: compute pairwise similarities, create the softmax over positives and negatives, backpropagate.

This chapter ships an artifact. The course version of this lesson produces a reusable prompt or agent skill. It lives in the repository, ready to install: <https://github.com/rohitg00/ai-engineering-from-scratch/tree/main/phases/03-deep-learning-core/05-loss-functions>

Exercises

Starter code and the lesson's working implementation: <https://github.com/rohitg00/ai-engineering-from-scratch/tree/main/phases/03-deep-learning-core/05-loss-functions/code>

1. Implement Huber loss (smooth L1 loss), which is MSE for small errors and MAE for large errors. Train a regression network predicting $y = \sin(x)$ with MSE vs Huber when 5% of training targets have random noise added (outliers). Compare final test error.
2. Add focal loss to the binary classification training loop. Create an imbalanced dataset (90% class 0, 10% class 1). Compare standard BCE vs focal loss ($\gamma=2$) on the minority class recall after 200 epochs.
3. Implement triplet loss with semi-hard negative mining. Generate 2D embedding data for 5 classes. For each anchor, find the hardest negative that is still farther than the positive

(semi-hard). Compare convergence to random triplet selection.

4. Run the MSE vs cross-entropy comparison but track gradient magnitudes at each layer during training. Plot the average gradient norm per epoch. Verify that cross-entropy produces larger gradients in early epochs when the model is most uncertain.
5. Implement KL divergence loss and verify that minimizing $KL(\text{true} || \text{predicted})$ gives the same gradients as cross-entropy when the true distribution is one-hot. Then try soft targets (like knowledge distillation) where the “true” distribution comes from a teacher model’s softmax output.

Key Terms

Term	What people say	What it actually means
Loss function	“How wrong the model is”	A differentiable function mapping predictions and targets to a scalar that the optimizer minimizes
MSE	“Average squared error”	Mean of squared differences between predictions and targets; penalizes large errors quadratically
Cross-entropy	“The classification loss”	Measures divergence between predicted probability distribution and true distribution using $-\log(p)$
Binary cross-entropy	“BCE”	Cross-entropy for two classes: $-(y \log(p) + (1-y) \log(1-p))$
Label smoothing	“Softening the targets”	Replacing hard 0/1 targets with soft values (e.g., 0.1/0.9) to prevent overconfidence and improve generalization
Contrastive loss	“Pull together, push apart”	A loss that learns representations by making similar pairs close and dissimilar pairs far in embedding space
InfoNCE	“The CLIP/SimCLR loss”	Normalized temperature-scaled cross-entropy over similarity scores; treats contrastive learning as classification
Focal loss	“The imbalanced data fix”	Cross-entropy weighted by $(1-p_t)^\gamma$ to down-weight easy examples and focus on hard ones
Triplet loss	“Anchor-positive-negative”	Pushes anchor closer to positive than negative by at least a margin in embedding space
Temperature	“Sharpness knob”	A scalar divisor on logits/similarities that controls how peaked the resulting distribution is; lower = sharper

Further Reading

- Lin et al., “Focal Loss for Dense Object Detection” (2017) – introduced focal loss for handling extreme class imbalance in object detection (RetinaNet)
- Chen et al., “A Simple Framework for Contrastive Learning of Visual Representations” (SimCLR, 2020) – defined the modern contrastive learning pipeline with NT-Xent loss

- Szegedy et al., “Rethinking the Inception Architecture” (2016) - introduced label smoothing as a regularization technique, now standard in most large models
- Hinton et al., “Distilling the Knowledge in a Neural Network” (2015) - knowledge distillation using soft targets and KL divergence, foundational for model compression

Continue online. The living edition of this chapter has more than the page can hold:

- Animated, interactive figures and the web text: <https://aiengineeringfromscratch.com/lesson?path=phases/03-deep-learning-core/05-loss-functions>
- Runnable code for every step: <https://github.com/rohitg00/ai-engineering-from-scratch/tree/main/phases/03-deep-learning-core/05-loss-functions/code>
- The chapter quiz, graded in the browser: <https://aiengineeringfromscratch.com/lesson?path=phases/03-deep-learning-core/05-loss-functions>

The repository moves faster than any printing. When the book and the repo disagree, trust the repo.

Optimizers

Gradient descent tells you which direction to move. It says nothing about how far or how fast. SGD is a compass. Adam is GPS with traffic data.

Type: Build **Languages:** Python **Prerequisites:** Lesson 03.05 (Loss Functions) **Time:** ~75 minutes

Learning Objectives

- Implement SGD, SGD with momentum, Adam, and AdamW optimizers from scratch in Python
- Explain how Adam's bias correction compensates for zero-initialized moment estimates in early training steps
- Demonstrate why AdamW produces better generalization than Adam with L2 regularization on the same task
- Select the appropriate optimizer and default hyperparameters for transformers, CNNs, GANs, and fine-tuning

The Problem

You computed the gradients. You know that weight #4,721 should decrease by 0.003 to reduce the loss. But 0.003 in what units? Scaled by what? And should you move the same amount on step 1 as on step 1,000?

Vanilla gradient descent applies the same learning rate to every parameter on every step: $w = w - lr * \text{gradient}$. This creates three problems that make training neural networks painful in practice.

First, oscillation. The loss landscape is rarely shaped like a smooth bowl. It's more like a long, narrow valley. The gradient points across the valley (steep direction), not along it (shallow direction). Gradient descent bounces back and forth across the narrow dimension while making tiny progress along the useful one. You've seen this: loss drops fast then plateaus, not because the model converged but because it's oscillating.

Second, one learning rate for all parameters is wrong. Some weights need large updates (they're in the early, underfitting stage). Others need tiny updates (they're near their optimal value). A learning rate that works for the former destroys the latter, and vice versa.

Third, saddle points. In high dimensions, the loss landscape has vast flat regions where the gradient is near zero. Vanilla SGD crawls through these at the speed of the gradient, which is effectively zero. The model looks stuck. It isn't stuck – it's in a flat region with useful descent on the other side. But SGD has no mechanism to push through.

Adam solves all three. It maintains two running averages per parameter – the mean gradient (momentum, handles oscillation) and the mean squared gradient (adaptive rate, handles different scales). Combined with bias correction for the first few steps, it gives you a single optimizer that works on 80% of problems with default hyperparameters. This lesson builds it from scratch so you understand exactly when and why it fails on the other 20%.

The Concept

Stochastic Gradient Descent (SGD)

The simplest optimizer. Compute the gradient on a mini-batch and step in the opposite direction.

$$w = w - lr * \text{gradient}$$

The “stochastic” means you use a random subset (mini-batch) of data to estimate the gradient, rather than the full dataset. This noise is actually useful – it helps escape sharp local minima. But the noise also causes oscillation.

Learning rate is the only knob. Too high: the loss diverges. Too low: training takes forever. The optimal value depends on the architecture, the data, the batch size, and the current stage of training. For vanilla SGD on modern networks, typical values range from 0.01 to 0.1. But even within a single training run, the ideal learning rate changes.

Momentum

The ball-rolling-downhill analogy is overused but accurate. Instead of stepping by the gradient alone, you maintain a velocity that accumulates past gradients.

$$m_t = \text{beta} * m_{t-1} + \text{gradient}$$

$$w = w - lr * m_t$$

Beta (typically 0.9) controls how much history to keep. With beta = 0.9, the momentum is roughly the average of the last 10 gradients ($1 / (1 - 0.9) = 10$).

Why this fixes oscillation: gradients that point in the same direction accumulate. Gradients that flip direction cancel out. In that narrow valley, the “across” component flips sign each step and gets dampened. The “along” component stays consistent and gets amplified. The result is smooth acceleration in the useful direction.

Real numbers: SGD alone on a badly conditioned loss landscape might take 10,000 steps. SGD with momentum (beta=0.9) typically takes 3,000-5,000 steps on the same problem. The speedup is not marginal.

RMSProp

The first per-parameter adaptive learning rate method that actually worked. Proposed by Hinton in a Coursera lecture (never formally published).

$$s_t = \text{beta} * s_{t-1} + (1 - \text{beta}) * \text{gradient}^2$$

$$w = w - lr * \text{gradient} / (\text{sqrt}(s_t) + \text{epsilon})$$

s_t tracks the running average of squared gradients. Parameters with consistently large gradients get divided by a large number (smaller effective learning rate). Parameters with small gradients get divided by a small number (larger effective learning rate).

This solves the “one learning rate for all parameters” problem. A weight that’s already been getting large updates is probably near its target – slow it down. A weight that’s been getting tiny updates might be undertrained – speed it up.

Epsilon (typically $1e-8$) prevents division by zero when a parameter hasn’t been updated.

Adam: Momentum + RMSProp

Adam combines both ideas. It maintains two exponential moving averages per parameter:


```

m_t = beta1 * m_{t-1} + (1 - beta1) * gradient      (first moment: mean)
v_t = beta2 * v_{t-1} + (1 - beta2) * gradient^2    (second moment: variance)

```

Bias correction is the key detail most explanations skip. At step 1, $m_1 = (1 - \beta_1) * \text{gradient}$. With $\beta_1 = 0.9$, that's $0.1 * \text{gradient}$ – ten times too small. The moving average hasn't warmed up yet. Bias correction compensates:

```

m_hat = m_t / (1 - beta1^t)
v_hat = v_t / (1 - beta2^t)

```

At step 1 with $\beta_1 = 0.9$: $m_hat = m_1 / (1 - 0.9) = m_1 / 0.1 =$ the actual gradient. At step 100: $(1 - 0.9^{100})$ is approximately 1.0, so the correction vanishes. Bias correction matters for the first ~10 steps and is irrelevant after ~50.

The update:

```

w = w - lr * m_hat / (sqrt(v_hat) + epsilon)

```

Adam defaults: $lr = 0.001$, $\beta_1 = 0.9$, $\beta_2 = 0.999$, $\epsilon = 1e-8$. These defaults work for 80% of problems. When they don't, change lr first. Then β_2 . Almost never change β_1 or ϵ .

AdamW: Weight Decay Done Right

L2 regularization adds $\lambda * w^2$ to the loss. In vanilla SGD, this is equivalent to weight decay (subtracting $\lambda * w$ from the weight at each step). In Adam, this equivalence breaks.

The Loshchilov & Hutter insight: when you add L2 to the loss and then Adam processes the gradient, the adaptive learning rate scales the regularization term too. Parameters with large gradient variance get less regularization. Parameters with small variance get more. This is not what you want – you want uniform regularization regardless of the gradient statistics.

AdamW fixes this by applying weight decay directly to the weights, after the Adam update:

```

w = w - lr * m_hat / (sqrt(v_hat) + epsilon) - lr * lambda * w

```

The weight decay term ($lr * \lambda * w$) is not scaled by Adam's adaptive factor. Every parameter gets the same proportional shrinkage.

This seems like a minor detail. It's not. AdamW converges to better solutions than Adam + L2 regularization on virtually every task. It's the default optimizer in PyTorch for training transformers, diffusion models, and most modern architectures. BERT, GPT, LLaMA, Stable Diffusion – all trained with AdamW.

Learning Rate: The Most Important Hyperparameter

Diagram. Rendered live in the web edition: <https://aiengineeringfromscratch.com/lesson?path=phases/03-deep-learning-core/06-optimizers>

If you tune one hyperparameter, tune the learning rate. A 10x change in learning rate matters more than any architectural decision you'll make. Common defaults:

- SGD: $lr = 0.01$ to 0.1
- Adam/AdamW: $lr = 1e-4$ to $3e-4$
- Fine-tuning pretrained models: $lr = 1e-5$ to $5e-5$
- Learning rate warmup: linear ramp over first 1-10% of steps

Optimizer Comparison

Diagram. Rendered live in the web edition: <https://aiengineeringfromscratch.com/lesson?path=phases/03-deep-learning-core/06-optimizers>

When Each Optimizer Wins

Diagram. Rendered live in the web edition: <https://aiengineeringfromscratch.com/lesson?path=phases/03-deep-learning-core/06-optimizers>

Interactive figure: optimizer-trajectory. This one moves. Watch it animate and drag its controls in the web edition: <https://aiengineeringfromscratch.com/lesson?path=phases/03-deep-learning-core/06-optimizers>

Build It

Step 1: Vanilla SGD

```
class SGD:
    def __init__(self, lr=0.01):
        self.lr = lr

    def step(self, params, grads):
        for i in range(len(params)):
            params[i] -= self.lr * grads[i]
```

Step 2: SGD with Momentum

```
class SGDMomentum:
    def __init__(self, lr=0.01, beta=0.9):
        self.lr = lr
        self.beta = beta
        self.velocities = None

    def step(self, params, grads):
        if self.velocities is None:
            self.velocities = [0.0] * len(params)
        for i in range(len(params)):
            self.velocities[i] = self.beta * self.velocities[i] + grads[i]
            params[i] -= self.lr * self.velocities[i]
```

Step 3: Adam

```
import math

class Adam:
    def __init__(self, lr=0.001, beta1=0.9, beta2=0.999, epsilon=1e-8):
        self.lr = lr
        self.beta1 = beta1
        self.beta2 = beta2
        self.epsilon = epsilon
        self.m = None
        self.v = None
```

```

self.t = 0

def step(self, params, grads):
    if self.m is None:
        self.m = [0.0] * len(params)
        self.v = [0.0] * len(params)

    self.t += 1

    for i in range(len(params)):
        self.m[i] = self.beta1 * self.m[i] + (1 - self.beta1) * grads[i]
        self.v[i] = self.beta2 * self.v[i] + (1 - self.beta2) * grads[i] ** 2

        m_hat = self.m[i] / (1 - self.beta1 ** self.t)
        v_hat = self.v[i] / (1 - self.beta2 ** self.t)

        params[i] -= self.lr * m_hat / (math.sqrt(v_hat) + self.epsilon)

```

Step 4: AdamW

```

class AdamW:
    def __init__(self, lr=0.001, beta1=0.9, beta2=0.999, epsilon=1e-8,
        ↪ weight_decay=0.01):
        self.lr = lr
        self.beta1 = beta1
        self.beta2 = beta2
        self.epsilon = epsilon
        self.weight_decay = weight_decay
        self.m = None
        self.v = None
        self.t = 0

    def step(self, params, grads):
        if self.m is None:
            self.m = [0.0] * len(params)
            self.v = [0.0] * len(params)

        self.t += 1

        for i in range(len(params)):
            self.m[i] = self.beta1 * self.m[i] + (1 - self.beta1) * grads[i]
            self.v[i] = self.beta2 * self.v[i] + (1 - self.beta2) * grads[i] ** 2

            m_hat = self.m[i] / (1 - self.beta1 ** self.t)
            v_hat = self.v[i] / (1 - self.beta2 ** self.t)

            params[i] -= self.lr * m_hat / (math.sqrt(v_hat) + self.epsilon)
            params[i] -= self.lr * self.weight_decay * params[i]

```

Step 5: Training Comparison

Train the same two-layer network on the circle dataset from lesson 05 with all four optimizers. Compare convergence.

```

import random

def sigmoid(x):
    x = max(-500, min(500, x))
    return 1.0 / (1.0 + math.exp(-x))

def make_circle_data(n=200, seed=42):
    random.seed(seed)
    data = []
    for _ in range(n):
        x = random.uniform(-2, 2)
        y = random.uniform(-2, 2)
        label = 1.0 if x * x + y * y < 1.5 else 0.0
        data.append([x, y], label)
    return data

class OptimizerTestNetwork:
    def __init__(self, optimizer, hidden_size=8):
        random.seed(0)
        self.hidden_size = hidden_size
        self.optimizer = optimizer

        self.w1 = [[random.gauss(0, 0.5) for _ in range(2)] for _ in
                    range(hidden_size)]
        self.b1 = [0.0] * hidden_size
        self.w2 = [random.gauss(0, 0.5) for _ in range(hidden_size)]
        self.b2 = 0.0

    def get_params(self):
        params = []
        for row in self.w1:
            params.extend(row)
        params.extend(self.b1)
        params.extend(self.w2)
        params.append(self.b2)
        return params

    def set_params(self, params):
        idx = 0
        for i in range(self.hidden_size):
            for j in range(2):
                self.w1[i][j] = params[idx]
                idx += 1
            self.b1[i] = params[idx]
            idx += 1
        for i in range(self.hidden_size):
            self.w2[i] = params[idx]
            idx += 1
        self.b2 = params[idx]

    def forward(self, x):
        self.x = x

```

```

self.z1 = []
self.h = []
for i in range(self.hidden_size):
    z = self.w1[i][0] * x[0] + self.w1[i][1] * x[1] + self.b1[i]
    self.z1.append(z)
    self.h.append(max(0.0, z))

self.z2 = sum(self.w2[i] * self.h[i] for i in range(self.hidden_size)) +
    ↪ self.b2
self.out = sigmoid(self.z2)
return self.out

def compute_grads(self, target):
    eps = 1e-15
    p = max(eps, min(1 - eps, self.out))
    d_loss = -(target / p) + (1 - target) / (1 - p)
    d_sigmoid = self.out * (1 - self.out)
    d_out = d_loss * d_sigmoid

    grads = [0.0] * (self.hidden_size * 2 + self.hidden_size + self.hidden_size
    ↪ + 1)
    idx = 0
    for i in range(self.hidden_size):
        d_relu = 1.0 if self.z1[i] > 0 else 0.0
        d_h = d_out * self.w2[i] * d_relu
        grads[idx] = d_h * self.x[0]
        grads[idx + 1] = d_h * self.x[1]
        idx += 2

    for i in range(self.hidden_size):
        d_relu = 1.0 if self.z1[i] > 0 else 0.0
        grads[idx] = d_out * self.w2[i] * d_relu
        idx += 1

    for i in range(self.hidden_size):
        grads[idx] = d_out * self.h[i]
        idx += 1

    grads[idx] = d_out
    return grads

def train(self, data, epochs=300):
    losses = []
    for epoch in range(epochs):
        total_loss = 0.0
        correct = 0
        for x, y in data:
            pred = self.forward(x)
            grads = self.compute_grads(y)
            params = self.get_params()
            self.optimizer.step(params, grads)
            self.set_params(params)

        eps = 1e-15

```

```

        p = max(eps, min(1 - eps, pred))
        total_loss += -(y * math.log(p) + (1 - y) * math.log(1 - p))
        if (pred >= 0.5) == (y >= 0.5):
            correct += 1
    avg_loss = total_loss / len(data)
    accuracy = correct / len(data) * 100
    losses.append((avg_loss, accuracy))
    if epoch % 75 == 0 or epoch == epochs - 1:
        print(f"    Epoch {epoch:3d}: loss={avg_loss:.4f},
              ↪ accuracy={accuracy:.1f}%")
    return losses

```

Use It

PyTorch optimizers handle parameter groups, gradient clipping, and learning rate scheduling:

```

import torch
import torch.optim as optim

model = torch.nn.Sequential(
    torch.nn.Linear(784, 256),
    torch.nn.ReLU(),
    torch.nn.Linear(256, 10),
)

optimizer = optim.AdamW(model.parameters(), lr=3e-4, weight_decay=0.01)

scheduler = optim.lr_scheduler.CosineAnnealingLR(optimizer, T_max=100)

for epoch in range(100):
    optimizer.zero_grad()
    output = model(torch.randn(32, 784))
    loss = torch.nn.functional.cross_entropy(output, torch.randint(0, 10, (32,)))
    loss.backward()
    torch.nn.utils.clip_grad_norm_(model.parameters(), max_norm=1.0)
    optimizer.step()
    scheduler.step()

```

The pattern is always: zero_grad, forward, loss, backward, (clip), step, (schedule). Memorize this order. Getting it wrong (e.g., calling scheduler.step() before optimizer.step()) is a common source of subtle bugs.

For CNNs, many practitioners still prefer SGD + momentum (lr=0.1, momentum=0.9, weight decay=1e-4) with a step or cosine schedule. SGD finds flatter minima, which often generalize better. For transformers and LLMs, AdamW with warmup + cosine decay is the universal default. Don't fight the consensus without a measured reason.

This chapter ships an artifact. The course version of this lesson produces a reusable prompt or agent skill. It lives in the repository, ready to install: <https://github.com/rohitg00/ai-engineering-from-scratch/tree/main/phases/03-deep-learning-core/06-optimizers>

Exercises

Starter code and the lesson's working implementation: <https://github.com/rohitg00/ai-engineering-from-scratch/tree/main/phases/03-deep-learning-core/06-optimizers/code>

1. Implement Nesterov momentum, where you compute the gradient at the “lookahead” position ($w - lr * \beta * v$) instead of the current position. Compare convergence to standard momentum on the circle dataset.
2. Implement a learning rate warmup schedule: linear ramp from 0 to `max_lr` over the first 10% of training steps, then cosine decay to 0. Train with Adam + warmup vs Adam without warmup. Measure how many epochs it takes to reach 90% accuracy on the circle dataset.
3. Track the effective learning rate for each parameter during Adam training. The effective rate is $lr * m_hat / (\sqrt{v_hat} + \epsilon)$. Plot the distribution of effective rates after 10, 50, and 200 steps. Are all parameters being updated at the same speed?
4. Implement gradient clipping (clip by global norm). Set the max gradient norm to 1.0. Train with and without clipping using a high learning rate (`lr=0.01` for Adam). Count how many runs diverge (loss goes to NaN) with and without clipping over 10 random seeds.
5. Compare Adam vs AdamW on a network with large weights. Initialize all weights to random values in $[-5, 5]$ (much larger than normal). Train for 200 epochs with `weight_decay=0.1`. Plot the L2 norm of weights over training for both optimizers. AdamW should show faster weight shrinkage.

Key Terms

Term	What people say	What it actually means
Learning rate	“Step size”	The scalar multiplier on the gradient update; the single most impactful hyperparameter in training
SGD	“Basic gradient descent”	Stochastic gradient descent: update weights by subtracting $lr * \text{gradient}$, computed on a mini-batch
Momentum	“Rolling ball analogy”	Exponential moving average of past gradients; dampens oscillation and accelerates consistent directions
RMSProp	“Adaptive learning rate”	Divides each parameter's gradient by the running RMS of its recent gradients; equalizes learning rates
Adam	“The default optimizer”	Combines momentum (first moment) and RMSProp (second moment) with bias correction for the initial steps
AdamW	“Adam done right”	Adam with decoupled weight decay; applies regularization directly to weights rather than through the gradient
Bias correction	“Warmup for running averages”	Dividing by $(1 - \beta^t)$ to compensate for the zero-initialization of Adam's moment estimates

Term	What people say	What it actually means
Weight decay	“Shrink the weights”	Subtracting a fraction of the weight value at each step; a regularizer that penalizes large weights
Learning rate schedule	“Changing lr over time”	A function that adjusts the learning rate during training; warmup + cosine decay is the modern default
Gradient clipping	“Capping the gradient norm”	Scaling down the gradient vector when its norm exceeds a threshold; prevents exploding gradient updates

Further Reading

- Kingma & Ba, “Adam: A Method for Stochastic Optimization” (2014) - the original Adam paper with convergence analysis and the bias correction derivation
- Loshchilov & Hutter, “Decoupled Weight Decay Regularization” (2017) - proved that L2 regularization and weight decay are not equivalent in Adam, and proposed AdamW
- Smith, “Cyclical Learning Rates for Training Neural Networks” (2017) - introduced the LR range test and cyclical schedules that remove the need to tune a fixed learning rate
- Ruder, “An Overview of Gradient Descent Optimization Algorithms” (2016) - the best single survey of all optimizer variants, with clear comparisons and intuitions

Continue online. The living edition of this chapter has more than the page can hold:

- Animated, interactive figures and the web text: <https://aiengineeringfromscratch.com/lesson?path=phases/03-deep-learning-core/06-optimizers>
- Runnable code for every step: <https://github.com/rohitg00/ai-engineering-from-scratch/tree/main/phases/03-deep-learning-core/06-optimizers/code>
- The chapter quiz, graded in the browser: <https://aiengineeringfromscratch.com/lesson?path=phases/03-deep-learning-core/06-optimizers>

The repository moves faster than any printing. When the book and the repo disagree, trust the repo.

Regularization

Your model gets 99% on training data and 60% on test data. It memorized instead of learning. Regularization is the tax you impose on complexity to force generalization.

Type: Build **Languages:** Python **Prerequisites:** Lesson 03.06 (Optimizers) **Time:** ~75 minutes

Learning Objectives

- Implement dropout with inverted scaling, L2 weight decay, batch normalization, layer normalization, and RMSNorm from scratch
- Measure the train-test accuracy gap and diagnose overfitting using regularization experiments
- Explain why transformers use LayerNorm instead of BatchNorm and why modern LLMs prefer RMSNorm
- Apply the correct combination of regularization techniques based on the severity of overfitting

The Problem

A neural network with enough parameters can memorize any dataset. This is not a hypothetical – Zhang et al. (2017) proved it by training standard networks on ImageNet with random labels. The networks reached near-zero training loss on completely random label assignments. They memorized a million random input-output pairs with no pattern to learn. Training loss was perfect. Test accuracy was zero.

This is the overfitting problem, and it gets worse as models get larger. GPT-3 has 175 billion parameters. The training set has about 500 billion tokens. With that many parameters, the model has enough capacity to memorize significant chunks of the training data verbatim. Without regularization, it would just regurgitate training examples instead of learning generalizable patterns.

The gap between training performance and test performance is the overfitting gap. Every technique in this lesson attacks that gap from a different angle. Dropout forces the network to not rely on any single neuron. Weight decay prevents any single weight from growing too large. Batch normalization smooths the loss landscape so the optimizer finds flatter, more generalizable minima. Layer normalization does the same thing but works where batch normalization fails (small batches, variable-length sequences). RMSNorm does it 10% faster by dropping the mean calculation. Each technique is simple. Together, they're the difference between a model that memorizes and one that generalizes.

The Concept

The Overfitting Spectrum

Every model sits somewhere on a spectrum from underfitting (too simple to capture the pattern) to overfitting (so complex it captures noise). The sweet spot is in between, and regularization pushes models toward it from the overfit side.

Diagram. Rendered live in the web edition: <https://aiengineeringfromscratch.com/lesson?path=phases/03-deep-learning-core/07-regularization>

Dropout

The simplest regularization technique with the most elegant interpretation. During training, randomly set each neuron's output to zero with probability p .

$\text{output} = \text{activation}(z) * \text{mask}$ where $\text{mask}[i] \sim \text{Bernoulli}(1 - p)$

With $p = 0.5$, half the neurons are zeroed on every forward pass. The network must learn redundant representations because it can't predict which neurons will be available. This prevents co-adaptation - neurons learning to rely on specific other neurons being present.

The ensemble interpretation: a network with N neurons and dropout creates 2^N possible subnetworks (every combination of which neurons are on or off). Training with dropout approximately trains all 2^N subnetworks simultaneously, each on different mini-batches. At test time, you use all neurons (no dropout) and scale outputs by $(1 - p)$ to match the expected value during training. This is equivalent to averaging the predictions of 2^N subnetworks - a massive ensemble from a single model.

In practice, the scaling is applied during training instead of testing (inverted dropout):

During training: $\text{output} = \text{activation}(z) * \text{mask} / (1 - p)$

During testing: $\text{output} = \text{activation}(z)$ (no change needed)

This is cleaner because test code doesn't need to know about dropout at all.

Default rates: $p = 0.1$ for transformers, $p = 0.5$ for MLPs, $p = 0.2-0.3$ for CNNs. Higher dropout = stronger regularization = more underfitting risk.

Weight Decay (L2 Regularization)

Add the squared magnitude of all weights to the loss:

$\text{total_loss} = \text{task_loss} + (\lambda / 2) * \sum(w_i^2)$

The gradient of the regularization term is $\lambda * w$. This means at every step, each weight is shrunk toward zero by a fraction proportional to its magnitude. Large weights get penalized more. The model is pushed toward solutions where no single weight dominates.

Why this helps generalization: overfit models tend to have large weights that amplify noise in the training data. Weight decay keeps weights small, which limits the model's effective capacity and forces it to rely on robust, generalizable features rather than memorized quirks.

The λ hyperparameter controls the strength. Typical values:

- 0.01 for AdamW on transformers
- $1e-4$ for SGD on CNNs
- 0.1 for heavily overfit models

As discussed in lesson 06: weight decay and L2 regularization are equivalent in SGD but not in Adam. Always use AdamW (decoupled weight decay) when training with Adam.

Batch Normalization

Normalize the output of each layer across the mini-batch before passing it to the next layer.

For a mini-batch of activations at some layer:

```
mu = (1/B) * sum(x_i)           (batch mean)
sigma^2 = (1/B) * sum((x_i - mu)^2) (batch variance)
x_hat = (x_i - mu) / sqrt(sigma^2 + eps) (normalize)
y = gamma * x_hat + beta       (scale and shift)
```

Gamma and beta are learnable parameters that let the network undo the normalization if that's optimal. Without them, you'd be forcing every layer's output to be zero-mean unit-variance, which might not be what the network wants.

Training vs inference split: During training, mu and sigma come from the current mini-batch. During inference, you use running averages accumulated during training (exponential moving average with momentum = 0.1, meaning 90% old + 10% new).

Why BatchNorm works is still debated. The original paper claimed it reduces “internal co-variate shift” (the distribution of layer inputs changing as earlier layers update). Santurkar et al. (2018) showed this explanation is wrong. The actual reason: BatchNorm makes the loss landscape smoother. The gradients are more predictive, the Lipschitz constants are smaller, and the optimizer can take larger steps safely. This is why BatchNorm lets you use higher learning rates and converge faster.

BatchNorm has a fundamental limitation: it depends on batch statistics. With batch size 1, the mean and variance are meaningless. With small batches (< 32), the statistics are noisy and hurt performance. This matters for tasks like object detection (where memory limits batch size) and language modeling (where sequence lengths vary).

Layer Normalization

Normalize across features instead of across the batch. For a single sample:

```
mu = (1/D) * sum(x_j)           (feature mean)
sigma^2 = (1/D) * sum((x_j - mu)^2) (feature variance)
x_hat = (x_j - mu) / sqrt(sigma^2 + eps)
y = gamma * x_hat + beta
```

D is the feature dimension. Each sample is normalized independently - no dependence on batch size. This is why transformers use LayerNorm instead of BatchNorm. Sequences have variable lengths, batch sizes are often small (or 1 during generation), and the computation is identical between training and inference.

LayerNorm in transformers is applied after each self-attention block and each feed-forward block (Post-LN), or before them (Pre-LN, which is more stable for training).

RMSNorm

LayerNorm without the mean subtraction. Proposed by Zhang & Sennrich (2019).

```
rms = sqrt((1/D) * sum(x_j^2))
y = gamma * x / rms
```

That's it. No mean computation, no beta parameter. The observation: the re-centering (mean subtraction) in LayerNorm contributes very little to the model's performance, but costs computation. Removing it gives the same accuracy with about 10% less overhead.

LLaMA, LLaMA 2, LLaMA 3, Mistral, and most modern LLMs use RMSNorm instead of LayerNorm. At the scale of billions of parameters and trillions of tokens, that 10% savings is significant.

Normalization Comparison

Diagram. Rendered live in the web edition: <https://aiengineeringfromscratch.com/lesson?path=phases/03-deep-learning-core/07-regularization>

Data Augmentation as Regularization

Not a model modification but a data modification. Transform training inputs while preserving labels:

- Images: random crop, flip, rotation, color jitter, cutout
- Text: synonym replacement, back-translation, random deletion
- Audio: time stretch, pitch shift, noise addition

The effect is identical to regularization: it increases the effective size of the training set, making it harder for the model to memorize specific examples. A model that only sees each image once in its original form can memorize it. A model that sees 50 augmented versions of each image is forced to learn the invariant structure.

Early Stopping

The simplest regularizer: stop training when validation loss starts increasing. The model hasn't overfit yet at that point. In practice, you track validation loss every epoch, save the best model, and continue training for a "patience" window (typically 5-20 epochs). If validation loss doesn't improve within the patience window, you stop and load the best saved model.

When to Apply What

Diagram. Rendered live in the web edition: <https://aiengineeringfromscratch.com/lesson?path=phases/03-deep-learning-core/07-regularization>

Interactive figure: l2-regularization. This one moves. Watch it animate and drag its controls in the web edition: <https://aiengineeringfromscratch.com/lesson?path=phases/03-deep-learning-core/07-regularization>

Build It

Step 1: Dropout (Train and Eval Mode)

```
import random
import math

class Dropout:
    def __init__(self, p=0.5):
        self.p = p
```

```

        self.training = True
        self.mask = None

    def forward(self, x):
        if not self.training:
            return list(x)
        self.mask = []
        output = []
        for val in x:
            if random.random() < self.p:
                self.mask.append(0)
                output.append(0.0)
            else:
                self.mask.append(1)
                output.append(val / (1 - self.p))
        return output

    def backward(self, grad_output):
        grads = []
        for g, m in zip(grad_output, self.mask):
            if m == 0:
                grads.append(0.0)
            else:
                grads.append(g / (1 - self.p))
        return grads

```

Step 2: L2 Weight Decay

```

def l2_regularization(weights, lambda_reg):
    penalty = 0.0
    for w in weights:
        penalty += w * w
    return lambda_reg * 0.5 * penalty

def l2_gradient(weights, lambda_reg):
    return [lambda_reg * w for w in weights]

```

Step 3: Batch Normalization

```

class BatchNorm:
    def __init__(self, num_features, momentum=0.1, eps=1e-5):
        self.gamma = [1.0] * num_features
        self.beta = [0.0] * num_features
        self.eps = eps
        self.momentum = momentum
        self.running_mean = [0.0] * num_features
        self.running_var = [1.0] * num_features
        self.training = True
        self.num_features = num_features

    def forward(self, batch):
        batch_size = len(batch)
        if self.training:

```

```

mean = [0.0] * self.num_features
for sample in batch:
    for j in range(self.num_features):
        mean[j] += sample[j]
mean = [m / batch_size for m in mean]

var = [0.0] * self.num_features
for sample in batch:
    for j in range(self.num_features):
        var[j] += (sample[j] - mean[j]) ** 2
var = [v / batch_size for v in var]

for j in range(self.num_features):
    self.running_mean[j] = (1 - self.momentum) * self.running_mean[j] +
        ↪ self.momentum * mean[j]
    self.running_var[j] = (1 - self.momentum) * self.running_var[j] +
        ↪ self.momentum * var[j]
else:
    mean = list(self.running_mean)
    var = list(self.running_var)

self.x_hat = []
output = []
for sample in batch:
    normalized = []
    out_sample = []
    for j in range(self.num_features):
        x_h = (sample[j] - mean[j]) / math.sqrt(var[j] + self.eps)
        normalized.append(x_h)
        out_sample.append(self.gamma[j] * x_h + self.beta[j])
    self.x_hat.append(normalized)
    output.append(out_sample)
return output

```

Step 4: Layer Normalization

```

class LayerNorm:
    def __init__(self, num_features, eps=1e-5):
        self.gamma = [1.0] * num_features
        self.beta = [0.0] * num_features
        self.eps = eps
        self.num_features = num_features

    def forward(self, x):
        mean = sum(x) / len(x)
        var = sum((xi - mean) ** 2 for xi in x) / len(x)

        self.x_hat = []
        output = []
        for j in range(self.num_features):
            x_h = (x[j] - mean) / math.sqrt(var + self.eps)
            self.x_hat.append(x_h)
            output.append(self.gamma[j] * x_h + self.beta[j])
        return output

```

Step 5: RMSNorm

```
class RMSNorm:
    def __init__(self, num_features, eps=1e-6):
        self.gamma = [1.0] * num_features
        self.eps = eps
        self.num_features = num_features

    def forward(self, x):
        rms = math.sqrt(sum(xi * xi for xi in x) / len(x) + self.eps)
        output = []
        for j in range(self.num_features):
            output.append(self.gamma[j] * x[j] / rms)
        return output
```

Step 6: Training With and Without Regularization

```
def sigmoid(x):
    x = max(-500, min(500, x))
    return 1.0 / (1.0 + math.exp(-x))

def make_circle_data(n=200, seed=42):
    random.seed(seed)
    data = []
    for _ in range(n):
        x = random.uniform(-2, 2)
        y = random.uniform(-2, 2)
        label = 1.0 if x * x + y * y < 1.5 else 0.0
        data.append([x, y], label)
    return data

class RegularizedNetwork:
    def __init__(self, hidden_size=16, lr=0.05, dropout_p=0.0, weight_decay=0.0):
        random.seed(0)
        self.hidden_size = hidden_size
        self.lr = lr
        self.dropout_p = dropout_p
        self.weight_decay = weight_decay
        self.dropout = Dropout(p=dropout_p) if dropout_p > 0 else None

        self.w1 = [[random.gauss(0, 0.5) for _ in range(2)] for _ in
                    range(hidden_size)]
        self.b1 = [0.0] * hidden_size
        self.w2 = [random.gauss(0, 0.5) for _ in range(hidden_size)]
        self.b2 = 0.0

    def forward(self, x, training=True):
        self.x = x
        self.z1 = []
        self.h = []
        for i in range(self.hidden_size):
            z = self.w1[i][0] * x[0] + self.w1[i][1] * x[1] + self.b1[i]
```

```

        self.z1.append(z)
        self.h.append(max(0.0, z))

    if self.dropout and training:
        self.dropout.training = True
        self.h = self.dropout.forward(self.h)
    elif self.dropout:
        self.dropout.training = False
        self.h = self.dropout.forward(self.h)

    self.z2 = sum(self.w2[i] * self.h[i] for i in range(self.hidden_size)) +
        ↪ self.b2
    self.out = sigmoid(self.z2)
    return self.out

def backward(self, target):
    eps = 1e-15
    p = max(eps, min(1 - eps, self.out))
    d_loss = -(target / p) + (1 - target) / (1 - p)
    d_sigmoid = self.out * (1 - self.out)
    d_out = d_loss * d_sigmoid

    for i in range(self.hidden_size):
        d_relu = 1.0 if self.z1[i] > 0 else 0.0
        d_h = d_out * self.w2[i] * d_relu
        self.w2[i] -= self.lr * (d_out * self.h[i] + self.weight_decay *
            ↪ self.w2[i])
        for j in range(2):
            self.w1[i][j] -= self.lr * (d_h * self.x[j] + self.weight_decay *
                ↪ self.w1[i][j])
        self.b1[i] -= self.lr * d_h
    self.b2 -= self.lr * d_out

def evaluate(self, data):
    correct = 0
    total_loss = 0.0
    for x, y in data:
        pred = self.forward(x, training=False)
        eps = 1e-15
        p = max(eps, min(1 - eps, pred))
        total_loss += -(y * math.log(p) + (1 - y) * math.log(1 - p))
        if (pred >= 0.5) == (y >= 0.5):
            correct += 1
    return total_loss / len(data), correct / len(data) * 100

def train_model(self, train_data, test_data, epochs=300):
    history = []
    for epoch in range(epochs):
        total_loss = 0.0
        correct = 0
        for x, y in train_data:
            pred = self.forward(x, training=True)
            self.backward(y)
            eps = 1e-15

```



```

        p = max(eps, min(1 - eps, pred))
        total_loss += -(y * math.log(p) + (1 - y) * math.log(1 - p))
        if (pred >= 0.5) == (y >= 0.5):
            correct += 1
    train_loss = total_loss / len(train_data)
    train_acc = correct / len(train_data) * 100
    test_loss, test_acc = self.evaluate(test_data)
    history.append((train_loss, train_acc, test_loss, test_acc))
    if epoch % 75 == 0 or epoch == epochs - 1:
        gap = train_acc - test_acc
        print(f"    Epoch {epoch:3d}: train_acc={train_acc:.1f}%,
              ↪ test_acc={test_acc:.1f}%, gap={gap:.1f}%")
    return history

```

Use It

PyTorch provides all normalization and regularization as modules:

```

import torch
import torch.nn as nn

```

```

model = nn.Sequential(
    nn.Linear(784, 256),
    nn.BatchNorm1d(256),
    nn.ReLU(),
    nn.Dropout(0.3),
    nn.Linear(256, 128),
    nn.BatchNorm1d(128),
    nn.ReLU(),
    nn.Dropout(0.3),
    nn.Linear(128, 10),
)

```

```

model.train()
out_train = model(torch.randn(32, 784))

```

```

model.eval()
out_test = model(torch.randn(1, 784))

```

The `model.train()` / `model.eval()` toggle is critical. It switches dropout on/off and tells BatchNorm to use batch statistics vs running statistics. Forgetting `model.eval()` before inference is one of the most common bugs in deep learning. Your test accuracy will fluctuate randomly because dropout is still active and BatchNorm is using mini-batch statistics.

For transformers, the pattern is different:

```

class TransformerBlock(nn.Module):
    def __init__(self, d_model=512, nhead=8, dropout=0.1):
        super().__init__()
        self.attention = nn.MultiheadAttention(d_model, nhead, dropout=dropout)
        self.norm1 = nn.LayerNorm(d_model)
        self.ff = nn.Sequential(
            nn.Linear(d_model, d_model * 4),
            nn.GELU(),
            nn.Linear(d_model * 4, d_model),

```

```

        nn.Dropout(dropout),
    )
    self.norm2 = nn.LayerNorm(d_model)
    self.dropout = nn.Dropout(dropout)

    def forward(self, x):
        attended, _ = self.attention(x, x, x)
        x = self.norm1(x + self.dropout(attended))
        x = self.norm2(x + self.ff(x))
        return x

```

LayerNorm, not BatchNorm. Dropout $p=0.1$, not $p=0.5$. These are the transformer defaults.

This chapter ships an artifact. The course version of this lesson produces a reusable prompt or agent skill. It lives in the repository, ready to install: <https://github.com/rohitg00/ai-engineering-from-scratch/tree/main/phases/03-deep-learning-core/07-regularization>

Exercises

Starter code and the lesson's working implementation: <https://github.com/rohitg00/ai-engineering-from-scratch/tree/main/phases/03-deep-learning-core/07-regularization/code>

1. Implement spatial dropout for 2D data: instead of dropping individual neurons, drop entire feature channels. Simulate this by treating groups of consecutive features as channels and dropping whole groups. Compare the train-test gap to standard dropout on the circle dataset with `hidden_size=32`.
2. Implement label smoothing from lesson 05 combined with dropout from this lesson. Train with four configurations: neither, dropout only, label smoothing only, both. Measure the final train-test accuracy gap for each. Which combination gives the smallest gap?
3. Add a BatchNorm layer between the hidden layer and the activation in your circle-dataset network. Train with and without BatchNorm at learning rates 0.01, 0.05, and 0.1. BatchNorm should allow stable training at higher learning rates where the vanilla network diverges.
4. Implement early stopping: track test loss each epoch, save the best weights, and stop if test loss hasn't improved for 20 epochs. Run the regularized network for 1000 epochs. Report which epoch had the best test accuracy and how many epochs of computation you saved.
5. Compare LayerNorm vs RMSNorm on a 4-layer network (not just 2). Initialize both with the same weights. Train for 200 epochs and compare final accuracy, training speed (time per epoch), and gradient magnitudes at the first layer. Verify that RMSNorm is faster with the same accuracy.

Key Terms

Term	What people say	What it actually means
Overfitting	“Model memorized the data”	When a model’s training performance significantly exceeds its test performance, indicating it learned noise rather than signal
Regularization	“Preventing overfitting”	Any technique that constrains model complexity to improve generalization: dropout, weight decay, normalization, augmentation
Dropout	“Random neuron deletion”	Zeroing random neurons during training with probability p , forcing redundant representations; equivalent to training an ensemble
Weight decay	“L2 penalty”	Shrinking all weights toward zero by subtracting $\lambda * w$ at each step; penalizes complexity through weight magnitude
Batch normalization	“Normalize per batch”	Normalizing layer outputs across the batch dimension using batch statistics during training and running averages during inference
Layer normalization	“Normalize per sample”	Normalizing across features within each sample; batch-independent, used in transformers where batch size varies
RMSNorm	“LayerNorm without the mean”	Root mean square normalization; drops the mean subtraction from LayerNorm for 10% speedup with equal accuracy
Early stopping	“Stop before overfit”	Halting training when validation loss stops improving; the simplest regularizer, often used alongside others
Data augmentation	“More data from less”	Transforming training inputs (flip, crop, noise) to increase effective dataset size and force invariance learning
Generalization gap	“Train-test split”	The difference between training and test performance; regularization aims to minimize this gap

Further Reading

- Srivastava et al., “Dropout: A Simple Way to Prevent Neural Networks from Overfitting” (2014) – the original dropout paper with the ensemble interpretation and extensive experiments
- Ioffe & Szegedy, “Batch Normalization: Accelerating Deep Network Training by Reducing Internal Covariate Shift” (2015) – introduced BatchNorm and its training procedure, one of the most cited deep learning papers
- Zhang & Sennrich, “Root Mean Square Layer Normalization” (2019) – showed RMSNorm matches LayerNorm accuracy with reduced computation; adopted by LLaMA and Mistral
- Zhang et al., “Understanding Deep Learning Requires Rethinking Generalization” (2017) – the landmark paper showing neural networks can memorize random labels, challenging traditional views of generalization

Continue online. The living edition of this chapter has more than the page can hold:

- Animated, interactive figures and the web text: <https://aiengineeringfromscratch.com/lesson?path=phases/03-deep-learning-core/07-regularization>
- Runnable code for every step: <https://github.com/rohitg00/ai-engineering-from-scratch/tree/main/phases/03-deep-learning-core/07-regularization/code>
- The chapter quiz, graded in the browser: <https://aiengineeringfromscratch.com/lesson?path=phases/03-deep-learning-core/07-regularization>

The repository moves faster than any printing. When the book and the repo disagree, trust the repo.

Weight Initialization and Training Stability

Initialize wrong and training never starts. Initialize right and 50 layers train as smoothly as 3.

Type: Build **Languages:** Python **Prerequisites:** Lesson 03.04 (Activation Functions), Lesson 03.07 (Regularization) **Time:** ~90 minutes

Learning Objectives

- Implement zero, random, Xavier/Glorot, and Kaiming/He initialization strategies and measure their effect on activation magnitudes through 50 layers
- Derive why Xavier init uses $\text{Var}(w) = 2/(\text{fan_in} + \text{fan_out})$ and Kaiming uses $\text{Var}(w) = 2/\text{fan_in}$
- Demonstrate the symmetry problem with zero initialization and explain why random scale alone is insufficient
- Match the correct initialization strategy to the activation function: Xavier for sigmoid/tanh, Kaiming for ReLU/GELU

The Problem

Initialize all weights to zero. Nothing learns. Every neuron computes the same function, receives the same gradient, and updates identically. After 10,000 epochs, your 512-neuron hidden layer is still 512 copies of the same neuron. You paid for 512 parameters and got 1.

Initialize them too large. Activations explode through the network. By layer 10, values hit $1e15$. By layer 20, they overflow to infinity. Gradients follow the same trajectory in reverse.

Initialize them randomly from a standard normal distribution. Works for 3 layers. At 50 layers, the signal collapses to zero or detonates to infinity depending on whether the random scale was slightly too small or slightly too large. The boundary between “works” and “broken” is razor-thin.

Weight initialization is the most underrated decision in deep learning. Architecture gets papers. Optimizers get blog posts. Initialization gets a footnote. But get it wrong and nothing else matters – your network is dead before training begins.

The Concept

The Symmetry Problem

Every neuron in a layer has the same structure: multiply inputs by weights, add bias, apply activation. If all weights start at the same value (zero is the extreme case), every neuron computes the same output. During backpropagation, every neuron receives the same gradient. During the update step, every neuron changes by the same amount.

You're stuck. The network has hundreds of parameters, but they all move in lockstep. This is called symmetry, and random initialization is the brute-force way to break it. Each neuron starts at a different point in weight space, so each learns a different feature.

But "random" is not enough. The *scale* of the randomness determines whether the network trains.

Variance Propagation Through Layers

Consider a single layer with fan_in inputs:

$$z = w_1 \cdot x_1 + w_2 \cdot x_2 + \dots + w_n \cdot x_n$$

If each weight w_i is drawn from a distribution with variance $\text{Var}(w)$ and each input x_i has variance $\text{Var}(x)$, the output variance is:

$$\text{Var}(z) = \text{fan_in} * \text{Var}(w) * \text{Var}(x)$$

If $\text{Var}(w) = 1$ and $\text{fan_in} = 512$, the output variance is 512x the input variance. After 10 layers: $512^{10} = 1.2e27$. Your signal has exploded.

If $\text{Var}(w) = 0.001$, the output variance shrinks by $0.001 * 512 = 0.512$ per layer. After 10 layers: $0.512^{10} = 0.00013$. Your signal has vanished.

The goal: choose $\text{Var}(w)$ so that $\text{Var}(z) = \text{Var}(x)$. Signal magnitude stays constant across layers.

Xavier/Glorot Initialization

Glorot and Bengio (2010) derived the solution for sigmoid and tanh activations. To keep variance constant in both the forward and backward pass:

$$\text{Var}(w) = 2 / (\text{fan_in} + \text{fan_out})$$

In practice, weights are drawn from:

$$w \sim \text{Uniform}(-\text{limit}, \text{limit}) \quad \text{where } \text{limit} = \sqrt{6 / (\text{fan_in} + \text{fan_out})}$$

or:

$$w \sim \text{Normal}(0, \sqrt{2 / (\text{fan_in} + \text{fan_out})})$$

This works because sigmoid and tanh are roughly linear near zero, where properly initialized activations live. The variance stays stable through dozens of layers.

Kaiming/He Initialization

ReLU kills half the outputs (everything negative becomes zero). The effective fan_in is halved because on average half the inputs are zeroed. Xavier init doesn't account for this - it underestimates the variance needed.

He et al. (2015) adjusted the formula:

$$\text{Var}(w) = 2 / \text{fan_in}$$

Weights are drawn from:

$$w \sim \text{Normal}(0, \sqrt{2 / \text{fan_in}})$$

The factor of 2 compensates for ReLU zeroing half the activations. Without it, the signal shrinks by $\sim 0.5x$ per layer. With 50 layers: $0.5^{50} = 8.8e-16$. Kaiming init prevents this.

Transformer Initialization

GPT-2 introduced a different pattern. Residual connections add the output of each sub-layer to its input:

$$x = x + \text{sublayer}(x)$$

Each addition increases variance. With N residual layers, variance grows proportionally to N . GPT-2 scales the weights of residual layers by $1/\sqrt{2N}$, where N is the number of layers. This keeps the accumulated signal magnitude stable.

Llama 3 (405B parameters, 126 layers) uses a similar scheme. Without this scaling, the residual stream would grow unbounded through 126 layers of attention and feedforward blocks.

Diagram. Rendered live in the web edition: <https://aiengineeringfromscratch.com/lesson?path=phases/03-deep-learning-core/08-weight-initialization>

Activation Magnitude Through 50 Layers

Diagram. Rendered live in the web edition: <https://aiengineeringfromscratch.com/lesson?path=phases/03-deep-learning-core/08-weight-initialization>

Choosing the Right Init

Diagram. Rendered live in the web edition: <https://aiengineeringfromscratch.com/lesson?path=phases/03-deep-learning-core/08-weight-initialization>

Interactive figure: weight-init-variance. This one moves. Watch it animate and drag its controls in the web edition: <https://aiengineeringfromscratch.com/lesson?path=phases/03-deep-learning-core/08-weight-initialization>

Build It

Step 1: Initialization Strategies

Four ways to initialize a weight matrix. Each returns a list of lists (a 2D matrix) with `fan_in` columns and `fan_out` rows.

```
import math
import random

def zero_init(fan_in, fan_out):
    return [[0.0 for _ in range(fan_in)] for _ in range(fan_out)]

def random_init(fan_in, fan_out, scale=1.0):
    return [[random.gauss(0, scale) for _ in range(fan_in)] for _ in
            range(fan_out)]

def xavier_init(fan_in, fan_out):
    std = math.sqrt(2.0 / (fan_in + fan_out))
    return [[random.gauss(0, std) for _ in range(fan_in)] for _ in range(fan_out)]
```

```
def kaiming_init(fan_in, fan_out):
    std = math.sqrt(2.0 / fan_in)
    return [[random.gauss(0, std) for _ in range(fan_in)] for _ in range(fan_out)]
```

Step 2: Activation Functions

We need sigmoid, tanh, and ReLU to test each init strategy with its intended activation.

```
def sigmoid(x):
    x = max(-500, min(500, x))
    return 1.0 / (1.0 + math.exp(-x))
```

```
def tanh_act(x):
    return math.tanh(x)
```

```
def relu(x):
    return max(0.0, x)
```

Step 3: Forward Pass Through 50 Layers

Pass random data through a deep network and measure mean activation magnitude at each layer.

```
def forward_deep(init_fn, activation_fn, n_layers=50, width=64, n_samples=100):
    random.seed(42)
    layer_magnitudes = []

    inputs = [[random.gauss(0, 1) for _ in range(width)] for _ in range(n_samples)]

    for layer_idx in range(n_layers):
        weights = init_fn(width, width)
        biases = [0.0] * width

        new_inputs = []
        for sample in inputs:
            output = []
            for neuron_idx in range(width):
                z = sum(weights[neuron_idx][j] * sample[j] for j in range(width)) +
                ↪ biases[neuron_idx]
                output.append(activation_fn(z))
            new_inputs.append(output)
        inputs = new_inputs

        magnitudes = []
        for sample in inputs:
            magnitudes.append(sum(abs(v) for v in sample) / width)
        mean_mag = sum(magnitudes) / len(magnitudes)
        layer_magnitudes.append(mean_mag)

    return layer_magnitudes
```


Step 4: The Experiment

Run all combinations: zero init, random $N(0,1)$, random $N(0,0.01)$, Xavier with sigmoid, Xavier with tanh, Kaiming with ReLU. Print the magnitude at key layers.

```
def run_experiment():
    configs = [
        ("Zero init + Sigmoid", lambda fi, fo: zero_init(fi, fo), sigmoid),
        ("Random N(0,1) + ReLU", lambda fi, fo: random_init(fi, fo, 1.0), relu),
        ("Random N(0,0.01) + ReLU", lambda fi, fo: random_init(fi, fo, 0.01), relu),
        ("Xavier + Sigmoid", xavier_init, sigmoid),
        ("Xavier + Tanh", xavier_init, tanh_act),
        ("Kaiming + ReLU", kaiming_init, relu),
    ]

    print(f"{'Strategy':<30} {'L1':>10} {'L5':>10} {'L10':>10} {'L25':>10}"
          ↪ {'L50':>10}")
    print("-" * 80)

    for name, init_fn, act_fn in configs:
        mags = forward_deep(init_fn, act_fn)
        row = f"{'name':<30}"
        for idx in [0, 4, 9, 24, 49]:
            val = mags[idx]
            if val > 1e6:
                row += f" {'EXPLODED':>10}"
            elif val < 1e-6:
                row += f" {'VANISHED':>10}"
            else:
                row += f" {val:>10.4f}"
        print(row)
```

Step 5: Symmetry Demonstration

Show that zero init produces identical neurons.

```
def symmetry_demo():
    random.seed(42)
    weights = zero_init(2, 4)
    biases = [0.0] * 4

    inputs = [0.5, -0.3]
    outputs = []
    for neuron_idx in range(4):
        z = sum(weights[neuron_idx][j] * inputs[j] for j in range(2)) +
        ↪ biases[neuron_idx]
        outputs.append(sigmoid(z))

    print("\nSymmetry Demo (4 neurons, zero init):")
    for i, out in enumerate(outputs):
        print(f" Neuron {i}: output = {out:.6f}")
    all_same = all(abs(outputs[i] - outputs[0]) < 1e-10 for i in
    ↪ range(len(outputs)))
    print(f" All identical: {all_same}")
    print(f" Effective parameters: 1 (not {len(weights) * len(weights[0])})")
```

Step 6: Layer-by-Layer Magnitude Report

Print a visual bar chart of activation magnitudes through 50 layers.

```
def magnitude_report(name, magnitudes):
    print(f"\n{name}:")
    for i, mag in enumerate(magnitudes):
        if i % 5 == 0 or i == len(magnitudes) - 1:
            if mag > 1e6:
                bar = "X" * 50 + " EXPLODED"
            elif mag < 1e-6:
                bar = "." + " VANISHED"
            else:
                bar_len = min(50, max(1, int(mag * 10)))
                bar = "#" * bar_len
            print(f" Layer {i+1:3d}: {bar} ({mag:.6f})")
```

Use It

PyTorch provides these as built-in functions:

```
import torch
import torch.nn as nn
```

```
layer = nn.Linear(512, 256)
```

```
nn.init.xavier_uniform_(layer.weight)
nn.init.xavier_normal_(layer.weight)
```

```
nn.init.kaiming_uniform_(layer.weight, nonlinearity='relu')
nn.init.kaiming_normal_(layer.weight, nonlinearity='relu')
```

```
nn.init.zeros_(layer.bias)
```

When you call `nn.Linear(512, 256)`, PyTorch defaults to Kaiming uniform initialization. That's why most simple networks "just work" – PyTorch already made the right choice. But when you build custom architectures or go deeper than 20 layers, you need to understand what's happening and potentially override the default.

For transformers, HuggingFace models typically handle initialization in their `_init_weights` method. GPT-2's implementation scales residual projections by $1/\sqrt{N}$. If you're building a transformer from scratch, you need to add this yourself.

This chapter ships an artifact. The course version of this lesson produces a reusable prompt or agent skill. It lives in the repository, ready to install: <https://github.com/rohitg00/ai-engineering-from-scratch/tree/main/phases/03-deep-learning-core/08-weight-initialization>

Exercises

Starter code and the lesson's working implementation: <https://github.com/rohitg00/ai-engineering-from-scratch/tree/main/phases/03-deep-learning-core/08-weight-initialization/code>

1. Add LeCun initialization ($\text{Var} = 1/\text{fan_in}$, designed for SELU activation). Run the 50-layer experiment with LeCun init + tanh and compare to Xavier + tanh.

2. Implement the GPT-2 residual scaling: multiply the output of each layer by $1/\sqrt{2N}$ before adding to the residual stream. Run 50 layers with and without scaling, measure how fast the residual magnitude grows.
3. Create an “init health check” function that takes a network’s layer dimensions and activation type, then recommends the correct initialization and warns if the current init will cause problems.
4. Run the experiment with `fan_in = 16` vs `fan_in = 1024`. Xavier and Kaiming adapt to `fan_in`, but random init doesn’t. Show how the gap between “works” and “breaks” widens with larger layers.
5. Implement orthogonal initialization (generate a random matrix, compute its SVD, use the orthogonal matrix U). Compare to Kaiming for ReLU networks at 50 layers.

Key Terms

Term	What people say	What it actually means
Weight initialization	“Set starting weights randomly”	The strategy for choosing initial weight values that determines whether a network can train at all
Symmetry breaking	“Make neurons different”	Using random initialization to ensure neurons learn distinct features instead of computing identical functions
Fan-in	“Number of inputs to a neuron”	The number of incoming connections, which determines how input variance accumulates in the weighted sum
Fan-out	“Number of outputs from a neuron”	The number of outgoing connections, relevant for maintaining gradient variance during backpropagation
Xavier/Glorot init	“The sigmoid initialization”	$\text{Var}(w) = 2/(\text{fan_in} + \text{fan_out})$, designed to preserve variance through sigmoid and tanh activations
Kaiming/He init	“The ReLU initialization”	$\text{Var}(w) = 2/\text{fan_in}$, accounts for ReLU zeroing half the activations
Variance propagation	“How signals grow or shrink through layers”	The mathematical analysis of how activation variance changes layer by layer based on weight scale
Residual scaling	“GPT-2’s init trick”	Scaling residual connection weights by $1/\sqrt{2N}$ to prevent variance growth through N transformer layers
Dead network	“Nothing trains”	A network where poor initialization causes all gradients to be zero or all activations to saturate
Exploding activations	“Values go to infinity”	When weight variance is too high, causing activation magnitudes to grow exponentially through layers

Further Reading

- Glorot & Bengio, “Understanding the difficulty of training deep feedforward neural networks” (2010) – the original Xavier initialization paper with variance analysis

- He et al., “Delving Deep into Rectifiers” (2015) – introduced Kaiming initialization for ReLU networks
- Radford et al., “Language Models are Unsupervised Multitask Learners” (2019) – GPT-2 paper with residual scaling initialization
- Mishkin & Matas, “All You Need is a Good Init” (2016) – layer-sequential unit-variance initialization, an empirical alternative to analytical formulas

Continue online. The living edition of this chapter has more than the page can hold:

- Animated, interactive figures and the web text: <https://aiengineeringfromscratch.com/lesson?path=phases/03-deep-learning-core/08-weight-initialization>
- Runnable code for every step: <https://github.com/rohitg00/ai-engineering-from-scratch/tree/main/phases/03-deep-learning-core/08-weight-initialization/code>
- The chapter quiz, graded in the browser: <https://aiengineeringfromscratch.com/lesson?path=phases/03-deep-learning-core/08-weight-initialization>

The repository moves faster than any printing. When the book and the repo disagree, trust the repo.

Learning Rate Schedules and Warmup

The learning rate is the single most important hyperparameter. Not the architecture. Not the dataset size. Not the activation function. The learning rate. If you tune nothing else, tune this.

Type: Build **Languages:** Python **Prerequisites:** Lesson 03.06 (Optimizers), Lesson 03.08 (Weight Initialization) **Time:** ~90 minutes

Learning Objectives

- Implement constant, step decay, cosine annealing, warmup + cosine, and 1 cycle learning rate schedules from scratch
- Demonstrate the three failure modes of learning rate selection: divergence (too high), stalling (too low), and oscillation (no decay)
- Explain why warmup is necessary for Adam-based optimizers and how it stabilizes early training
- Compare convergence speed across all five schedules on the same task and select the appropriate one for a given training budget

The Problem

Set the learning rate to 0.1. Training diverges – loss jumps to infinity in 3 steps. Set it to 0.0001. Training crawls – after 100 epochs, the model has barely moved from random. Set it to 0.01. Training works for 50 epochs, then the loss oscillates around a minimum it can never reach because the steps are too large.

The optimal learning rate is not a constant. It changes during training. Early on, you want large steps to cover ground quickly. Late in training, you want tiny steps to settle into a sharp minimum. The difference between a 90% accurate model and a 95% accurate model is often just the schedule.

Every major model published in the last three years uses a learning rate schedule. Llama 3 used peak $lr=3e-4$ with 2000 warmup steps and cosine decay to $3e-5$. GPT-3 used $lr=6e-4$ with warmup over 375 million tokens. These are not arbitrary choices. They are the result of extensive hyperparameter sweeps that cost millions of dollars.

You need to understand schedules because the defaults will not work for your problem. When you fine-tune a pretrained model, the right schedule is different than training from scratch. When you increase batch size, the warmup period needs to change. When training breaks at step 10,000, you need to know whether it's a schedule problem or something else.

The Concept

Constant Learning Rate

The simplest approach. Pick a number, use it for every step.

$$lr(t) = lr_0$$

Rarely optimal. It's either too high for the end of training (oscillation around the minimum) or too low for the beginning (wasted compute on tiny steps). Works fine for small models and debugging. A terrible choice for anything that trains for more than an hour.

Step Decay

The old-school approach from the ResNet era. Cut the learning rate by a factor (usually 10x) at fixed epochs.

$$lr(t) = lr_0 * \gamma^{\lfloor \text{epoch} / \text{step_size} \rfloor}$$

Where $\gamma = 0.1$ and $\text{step_size} = 30$ means: lr drops by 10x every 30 epochs. ResNet-50 used this - $lr=0.1$, drop by 10x at epochs 30, 60, and 90.

The problem: the optimal decay points depend on the dataset and architecture. Move to a different problem and you need to re-tune when to drop. The transitions are abrupt - loss can spike when the rate suddenly changes.

Cosine Annealing

Smooth decay from the maximum learning rate to a minimum, following a cosine curve:

$$lr(t) = lr_{\min} + 0.5 * (lr_{\max} - lr_{\min}) * (1 + \cos(\pi * t / T))$$

Where t is the current step and T is the total number of steps.

At $t=0$, the cosine term is 1, so $lr = lr_{\max}$. At $t=T$, the cosine term is -1, so $lr = lr_{\min}$. The decay is gentle at first, accelerates in the middle, and becomes gentle again near the end.

This is the default for most modern training runs. No hyperparameters to tune beyond $lr_{\max}$ and $lr_{\min}$. The cosine shape matches the empirical observation that most learning happens in the middle of training - you want reasonable step sizes during that critical period.

Warmup: Why You Start Small

Adam and other adaptive optimizers maintain running estimates of gradient mean and variance. At step 0, these estimates are initialized to zero. The first few gradient updates are based on garbage statistics. If your learning rate is large during this period, the model takes huge, poorly-directed steps.

Warmup fixes this. Start with a tiny learning rate (often $lr_{\max} / \text{warmup_steps}$ or even zero) and linearly ramp up to $lr_{\max}$ over the first N steps. By the time you reach the full learning rate, Adam's statistics have stabilized.

$$lr(t) = lr_{\max} * (t / \text{warmup_steps}) \quad \text{for } t < \text{warmup_steps}$$

Typical warmup: 1-5% of total training steps. Llama 3 trained for ~1.8 trillion tokens and warmed up for 2000 steps. GPT-3 warmed up over 375 million tokens.

Linear Warmup + Cosine Decay

The modern default. Ramp up linearly, then decay with cosine:

```
if t < warmup_steps:
    lr(t) = lr_max * (t / warmup_steps)
else:
```

```
progress = (t - warmup_steps) / (total_steps - warmup_steps)
lr(t) = lr_min + 0.5 * (lr_max - lr_min) * (1 + cos(pi * progress))
```

This is what Llama, GPT, PaLM, and most modern transformers use. The warmup prevents early instability. The cosine decay settles the model into a good minimum.

1cycle Policy

Leslie Smith's discovery (2018): ramp the learning rate up from a low value to a high value in the first half of training, then ramp it back down in the second half. Counterintuitive – why would you *increase* the learning rate midway through?

The theory: a high learning rate acts as regularization by adding noise to the optimization trajectory. The model explores more of the loss landscape during the ramp-up phase, finding better basins. The ramp-down phase then refines within the best basin found.

```
Phase 1 (0 to T/2):    lr ramps from lr_max/25 to lr_max
Phase 2 (T/2 to T):    lr ramps from lr_max to lr_max/10000
```

1cycle often trains faster than cosine annealing for a fixed compute budget. The tradeoff: you must know the total number of steps in advance.

Schedule Shapes

Diagram. Rendered live in the web edition: <https://aiengineeringfromscratch.com/lesson?path=phases/03-deep-learning-core/09-learning-rate-schedules>

Decision Flowchart

Diagram. Rendered live in the web edition: <https://aiengineeringfromscratch.com/lesson?path=phases/03-deep-learning-core/09-learning-rate-schedules>

Real Numbers from Published Models

Diagram. Rendered live in the web edition: <https://aiengineeringfromscratch.com/lesson?path=phases/03-deep-learning-core/09-learning-rate-schedules>

Interactive figure: lr-schedule. This one moves. Watch it animate and drag its controls in the web edition: <https://aiengineeringfromscratch.com/lesson?path=phases/03-deep-learning-core/09-learning-rate-schedules>

Build It

Step 1: Schedule Functions

Each function takes the current step and returns the learning rate at that step.

```
import math
```

```
def constant_schedule(step, lr=0.01, **kwargs):
    return lr
```

```
def step_decay_schedule(step, lr=0.1, step_size=100, gamma=0.1, **kwargs):
```

```

    return lr * (gamma ** (step // step_size))

def cosine_schedule(step, lr=0.01, total_steps=1000, lr_min=1e-5, **kwargs):
    if step >= total_steps:
        return lr_min
    return lr_min + 0.5 * (lr - lr_min) * (1 + math.cos(math.pi * step /
↪ total_steps))

def warmup_cosine_schedule(step, lr=0.01, total_steps=1000, warmup_steps=100,
↪ lr_min=1e-5, **kwargs):
    if total_steps <= warmup_steps:
        return lr * (step / max(warmup_steps, 1))
    if step < warmup_steps:
        return lr * step / warmup_steps
    progress = (step - warmup_steps) / (total_steps - warmup_steps)
    return lr_min + 0.5 * (lr - lr_min) * (1 + math.cos(math.pi * progress))

def one_cycle_schedule(step, lr=0.01, total_steps=1000, **kwargs):
    mid = max(total_steps // 2, 1)
    if step < mid:
        return (lr / 25) + (lr - lr / 25) * step / mid
    else:
        progress = (step - mid) / max(total_steps - mid, 1)
        return lr * (1 - progress) + (lr / 10000) * progress

```

Step 2: Visualize All Schedules

Print a text-based plot showing how each schedule evolves over training.

```

def visualize_schedule(name, schedule_fn, total_steps=500, **kwargs):
    steps = list(range(0, total_steps, total_steps // 20))
    if total_steps - 1 not in steps:
        steps.append(total_steps - 1)

    lrs = [schedule_fn(s, total_steps=total_steps, **kwargs) for s in steps]
    max_lr = max(lrs) if max(lrs) > 0 else 1.0

    print(f"\n{name}:")
    for s, lr_val in zip(steps, lrs):
        bar_len = int(lr_val / max_lr * 40)
        bar = "#" * bar_len
        print(f" Step {s:4d}: lr={lr_val:.6f} {bar}")

```

Step 3: Training Network

A simple two-layer network on the circle dataset, same as previous lessons, but now we vary the schedule.

```
import random
```

```
def sigmoid(x):
```



```

x = max(-500, min(500, x))
return 1.0 / (1.0 + math.exp(-x))

def relu(x):
    return max(0.0, x)

def relu_deriv(x):
    return 1.0 if x > 0 else 0.0

def make_circle_data(n=200, seed=42):
    random.seed(seed)
    data = []
    for _ in range(n):
        x = random.uniform(-2, 2)
        y = random.uniform(-2, 2)
        label = 1.0 if x * x + y * y < 1.5 else 0.0
        data.append([x, y], label))
    return data

def train_with_schedule(schedule_fn, schedule_name, data, epochs=300,
    ↪ base_lr=0.05, **kwargs):
    random.seed(0)
    hidden_size = 8
    total_steps = epochs * len(data)

    std = math.sqrt(2.0 / 2)
    w1 = [[random.gauss(0, std) for _ in range(2)] for _ in range(hidden_size)]
    b1 = [0.0] * hidden_size
    w2 = [random.gauss(0, std) for _ in range(hidden_size)]
    b2 = 0.0

    step = 0
    epoch_losses = []

    for epoch in range(epochs):
        total_loss = 0
        correct = 0

        for x, target in data:
            lr = schedule_fn(step, lr=base_lr, total_steps=total_steps, **kwargs)

            z1 = []
            h = []
            for i in range(hidden_size):
                z = w1[i][0] * x[0] + w1[i][1] * x[1] + b1[i]
                z1.append(z)
                h.append(relu(z))

            z2 = sum(w2[i] * h[i] for i in range(hidden_size)) + b2
            out = sigmoid(z2)

```

```

error = out - target
d_out = error * out * (1 - out)

for i in range(hidden_size):
    d_h = d_out * w2[i] * relu_deriv(z1[i])
    w2[i] -= lr * d_out * h[i]
    for j in range(2):
        w1[i][j] -= lr * d_h * x[j]
    b1[i] -= lr * d_h
b2 -= lr * d_out

total_loss += (out - target) ** 2
if (out >= 0.5) == (target >= 0.5):
    correct += 1
step += 1

avg_loss = total_loss / len(data)
accuracy = correct / len(data) * 100
epoch_losses.append(avg_loss)

return epoch_losses

```

Step 4: Compare All Schedules

Train the same network with each schedule and compare final loss and convergence behavior.

```

def compare_schedules(data):
    configs = [
        ("Constant", constant_schedule, {}),
        ("Step Decay", step_decay_schedule, {"step_size": 15000, "gamma": 0.1}),
        ("Cosine", cosine_schedule, {"lr_min": 1e-5}),
        ("Warmup+Cosine", warmup_cosine_schedule, {"warmup_steps": 3000, "lr_min":
↪ 1e-5}),
        ("1cycle", one_cycle_schedule, {}),
    ]

    print(f"\n{'Schedule':<20} {'Start Loss':>12} {'Mid Loss':>12} {'End
↪ Loss':>12} {'Best Loss':>12}")
    print("-" * 70)

    for name, schedule_fn, extra_kwargs in configs:
        losses = train_with_schedule(schedule_fn, name, data, epochs=300,
↪ base_lr=0.05, **extra_kwargs)
        mid_idx = len(losses) // 2
        best = min(losses)
        print(f"{name:<20} {losses[0]:>12.6f} {losses[mid_idx]:>12.6f}
↪ {losses[-1]:>12.6f} {best:>12.6f}")

```

Step 5: LR Too High vs Too Low

Demonstrate the three failure modes: too high (divergence), too low (crawling), and just right.

```

def lr_sensitivity(data):
    learning_rates = [1.0, 0.1, 0.01, 0.001, 0.0001]

```

```

print("\nLR Sensitivity (constant schedule, 100 epochs):")
print(f"  {'LR':>10} {'Start Loss':>12} {'End Loss':>12} {'Status':>15}")
print("  " + "-" * 52)

for lr in learning_rates:
    losses = train_with_schedule(constant_schedule, f"lr={lr}", data,
    epochs=100, base_lr=lr)
    start = losses[0]
    end = losses[-1]

    if end > start or math.isnan(end) or end > 1.0:
        status = "DIVERGED"
    elif end > start * 0.9:
        status = "BARELY MOVED"
    elif end < 0.15:
        status = "CONVERGED"
    else:
        status = "LEARNING"

    end_str = f"{end:.6f}" if not math.isnan(end) else "NaN"
    print(f"  {lr:>10.4f} {start:>12.6f} {end_str:>12} {status:>15}")

```

Use It

PyTorch provides schedulers in `torch.optim.lr_scheduler`:

```

import torch
import torch.optim as optim
from torch.optim.lr_scheduler import CosineAnnealingLR, OneCycleLR, StepLR

model = nn.Sequential(nn.Linear(10, 64), nn.ReLU(), nn.Linear(64, 1))
optimizer = optim.Adam(model.parameters(), lr=3e-4)

scheduler = CosineAnnealingLR(optimizer, T_max=1000, eta_min=1e-5)

for step in range(1000):
    loss = train_step(model, optimizer)
    scheduler.step()

```

For warmup + cosine, use a lambda scheduler or the `get_cosine_schedule_with_warmup` from HuggingFace:

```

from transformers import get_cosine_schedule_with_warmup

scheduler = get_cosine_schedule_with_warmup(
    optimizer,
    num_warmup_steps=2000,
    num_training_steps=100000,
)

```

The HuggingFace function is what most Llama and GPT fine-tuning scripts use. When in doubt, use warmup + cosine with warmup = 3-5% of total steps. It works for almost everything.

This chapter ships an artifact. The course version of this lesson produces a reusable

prompt or agent skill. It lives in the repository, ready to install: <https://github.com/rohitg00/ai-engineering-from-scratch/tree/main/phases/03-deep-learning-core/09-learning-rate-schedules>

Exercises

Starter code and the lesson's working implementation: <https://github.com/rohitg00/ai-engineering-from-scratch/tree/main/phases/03-deep-learning-core/09-learning-rate-schedules/code>

1. Implement exponential decay: $lr(t) = lr_0 * \gamma^t$ where $\gamma = 0.999$. Compare to cosine annealing on the circle dataset.
2. Implement the learning rate range test (Leslie Smith): train for a few hundred steps while exponentially increasing the LR from $1e-7$ to 1. Plot loss vs LR. The optimal max LR is just before the loss starts increasing.
3. Train with warmup + cosine but vary the warmup length: 0%, 1%, 5%, 10%, 20% of total steps. Find the sweet spot where training is most stable.
4. Implement cosine annealing with warm restarts (SGDR): reset the learning rate to lr_max every T steps and decay again. Compare to standard cosine on a longer training run.
5. Build a “schedule surgeon” that monitors training loss and automatically switches from warmup to cosine when the loss stabilizes, and reduces lr if the loss plateaus for too long.

Key Terms

Term	What people say	What it actually means
Learning rate	“How fast the model learns”	The scalar that multiplies the gradient to determine the parameter update size
Schedule	“Change the LR over time”	A function that maps training step to learning rate, designed to optimize convergence
Warmup	“Start with a small LR”	Linearly ramping the LR from near-zero to the target value over the first N steps to stabilize optimizer statistics
Cosine annealing	“Smooth LR decay”	Decreasing the LR following a cosine curve from lr_max to lr_min over training
Step decay	“Drop LR at milestones”	Multiplying the LR by a factor (usually 0.1) at fixed epoch intervals
1 cycle policy	“Up then down”	Leslie Smith’s method of ramping LR up then down in a single cycle for faster convergence
LR range test	“Find the best learning rate”	Training briefly while increasing LR to find the value where loss starts diverging
Cosine with warm restarts	“Reset and repeat”	Periodically resetting the LR to lr_max and decaying again (SGDR)
Eta min	“The floor for the LR”	The minimum learning rate that the schedule decays to

Term	What people say	What it actually means
Peak learning rate	“The maximum LR”	The highest LR reached during training, typically after warmup

Further Reading

- Loshchilov & Hutter, “SGDR: Stochastic Gradient Descent with Warm Restarts” (2017) – introduced cosine annealing and warm restarts
- Smith, “Super-Convergence: Very Fast Training of Neural Networks Using Large Learning Rates” (2018) – the 1cycle policy paper
- Touvron et al., “Llama 2: Open Foundation and Fine-Tuned Chat Models” (2023) – documents the warmup + cosine schedule used at scale
- Goyal et al., “Accurate, Large Minibatch SGD: Training ImageNet in 1 Hour” (2017) – linear scaling rule and warmup for large batch training

Continue online. The living edition of this chapter has more than the page can hold:

- Animated, interactive figures and the web text: <https://aiengineeringfromscratch.com/lesson?path=phases/03-deep-learning-core/09-learning-rate-schedules>
- Runnable code for every step: <https://github.com/rohitg00/ai-engineering-from-scratch/tree/main/phases/03-deep-learning-core/09-learning-rate-schedules/code>
- The chapter quiz, graded in the browser: <https://aiengineeringfromscratch.com/lesson?path=phases/03-deep-learning-core/09-learning-rate-schedules>

The repository moves faster than any printing. When the book and the repo disagree, trust the repo.

Build Your Own Mini Framework

You have built neurons, layers, networks, backprop, activations, loss functions, optimizers, regularization, initialization, and LR schedules. All as separate pieces. Now wire them together into a framework. Not PyTorch. Not TensorFlow. Yours.

Type: Build **Languages:** Python **Prerequisites:** All of Phase 03 (Lessons 01-09) **Time:** ~120 minutes

Learning Objectives

- Build a complete deep learning framework (~500 lines) with Module, Linear, ReLU, Sigmoid, Dropout, BatchNorm, Sequential, loss functions, optimizers, and DataLoader
- Explain the Module abstraction (forward, backward, parameters) and why train/eval mode toggling is necessary
- Wire all components into a working training loop that trains a 4-layer network on circle classification
- Map each component of your framework to its PyTorch equivalent (nn.Module, nn.Sequential, optim.Adam, DataLoader)

The Problem

You have ten lessons of building blocks scattered across separate files. A Value class here, a training loop there, weight initialization in another file, learning rate schedules in yet another. To train a network, you copy-paste from five different lessons and wire them together by hand.

That is what frameworks solve. PyTorch gives you nn.Module, nn.Sequential, optim.Adam, DataLoader, and a training loop pattern that ties them together. TensorFlow gives you keras.Layer, keras.Sequential, keras.optimizers.Adam. These are not magic. They are organizational patterns that make it possible to define, train, and evaluate networks without reinventing the plumbing every time.

You are going to build the same thing in ~500 lines of Python. No numpy. No external dependencies. A framework that can define any feedforward network, train it with SGD or Adam, batch the data, apply dropout and batch normalization, use any activation, and schedule the learning rate.

When you finish, you will understand exactly what happens when you write `model = nn.Sequential(...)` in PyTorch. You will understand why `model.train()` and `model.eval()` exist. You will understand why `optimizer.zero_grad()` is a separate call. You will understand all of it, because you built all of it.

The Concept

The Module Abstraction

Every layer in PyTorch inherits from nn.Module. A Module has three responsibilities:

1. **forward()** - compute the output given inputs
2. **parameters()** - return all trainable weights
3. **backward()** - compute gradients (handled by autograd in PyTorch, explicit in ours)

A Linear layer is a Module. A ReLU activation is a Module. A dropout layer is a Module. A batch normalization layer is a Module. They all have the same interface.

Sequential Container

`nn.Sequential` chains Modules. Forward pass: feed data through Module 1, then Module 2, then Module 3. Backward pass: reverse the chain. The container itself is a Module - it has `forward()`, `parameters()`, and `backward()`. This is the composite pattern: a sequence of Modules is itself a Module.

Training vs Evaluation Mode

Dropout randomly zeroes neurons during training but passes everything through during evaluation. Batch normalization uses batch statistics during training but running averages during evaluation. The `train()` and `eval()` methods toggle this behavior. Every Module has a `training` flag.

Optimizer

The optimizer updates parameters using their gradients. SGD: `param -= lr * grad`. Adam: maintains momentum and variance estimates, then updates. The optimizer does not know about the network architecture - it only sees a flat list of parameters and their gradients.

DataLoader

Batching matters for two reasons. First, you cannot fit the entire dataset in memory for large problems. Second, mini-batch gradient descent provides noise that helps escape local minima. The `DataLoader` splits data into batches and optionally shuffles between epochs.

Framework Architecture

Diagram. Rendered live in the web edition: <https://aiengineeringfromscratch.com/lesson?path=phases/03-deep-learning-core/10-mini-framework>

Training Loop

Diagram. Rendered live in the web edition: <https://aiengineeringfromscratch.com/lesson?path=phases/03-deep-learning-core/10-mini-framework>

Module Hierarchy

Diagram. Rendered live in the web edition: <https://aiengineeringfromscratch.com/lesson?path=phases/03-deep-learning-core/10-mini-framework>

Interactive figure: gradient-clipping. This one moves. Watch it animate and drag its controls in the web edition: <https://aiengineeringfromscratch.com/lesson?path=phases/03-deep-learning-core/10-mini-framework>

Build It

Step 1: Module Base Class

The abstract interface that every layer implements.

```
class Module:
    def __init__(self):
        self.training = True

    def forward(self, x):
        raise NotImplementedError

    def backward(self, grad):
        raise NotImplementedError

    def parameters(self):
        return []

    def train(self):
        self.training = True

    def eval(self):
        self.training = False
```

Step 2: Linear Layer

The fundamental building block. Stores weights and biases, computes $Wx + b$ forward, and weight/input gradients backward.

```
import math
import random

class Linear(Module):
    def __init__(self, fan_in, fan_out):
        super().__init__()
        std = math.sqrt(2.0 / fan_in)
        self.weights = [[random.gauss(0, std) for _ in range(fan_in)] for _ in
            ↪ range(fan_out)]
        self.biases = [0.0] * fan_out
        self.weight_grads = [[0.0] * fan_in for _ in range(fan_out)]
        self.bias_grads = [0.0] * fan_out
        self.fan_in = fan_in
        self.fan_out = fan_out
        self.input = None

    def forward(self, x):
        self.input = x
        output = []
        for i in range(self.fan_out):
            val = self.biases[i]
            for j in range(self.fan_in):
                val += self.weights[i][j] * x[j]
            output.append(val)
```



```

    return output

def backward(self, grad):
    input_grad = [0.0] * self.fan_in
    for i in range(self.fan_out):
        self.bias_grads[i] += grad[i]
        for j in range(self.fan_in):
            self.weight_grads[i][j] += grad[i] * self.input[j]
            input_grad[j] += grad[i] * self.weights[i][j]
    return input_grad

def parameters(self):
    params = []
    for i in range(self.fan_out):
        for j in range(self.fan_in):
            params.append((self.weights, i, j, self.weight_grads))
        params.append((self.biases, i, None, self.bias_grads))
    return params

```

Step 3: Activation Modules

ReLU, Sigmoid, and Tanh as Modules. Each caches what it needs for the backward pass.

```

class ReLU(Module):
    def __init__(self):
        super().__init__()
        self.mask = None

    def forward(self, x):
        self.mask = [1.0 if v > 0 else 0.0 for v in x]
        return [max(0.0, v) for v in x]

    def backward(self, grad):
        return [g * m for g, m in zip(grad, self.mask)]

class Sigmoid(Module):
    def __init__(self):
        super().__init__()
        self.output = None

    def forward(self, x):
        self.output = []
        for v in x:
            v = max(-500, min(500, v))
            self.output.append(1.0 / (1.0 + math.exp(-v)))
        return self.output

    def backward(self, grad):
        return [g * o * (1 - o) for g, o in zip(grad, self.output)]

class Tanh(Module):
    def __init__(self):

```

```

    super().__init__()
    self.output = None

    def forward(self, x):
        self.output = [math.tanh(v) for v in x]
        return self.output

    def backward(self, grad):
        return [g * (1 - o * o) for g, o in zip(grad, self.output)]

```

Step 4: Dropout Module

Randomly zeroes elements during training. Scales remaining elements by $1/(1-p)$ so expected values stay the same. Does nothing during eval.

```

class Dropout(Module):
    def __init__(self, p=0.5):
        super().__init__()
        self.p = p
        self.mask = None

    def forward(self, x):
        if not self.training:
            return x
        self.mask = [0.0 if random.random() < self.p else 1.0 / (1 - self.p) for _
            ↪ in x]
        return [v * m for v, m in zip(x, self.mask)]

    def backward(self, grad):
        if self.mask is None:
            return grad
        return [g * m for g, m in zip(grad, self.mask)]

```

Step 5: BatchNorm Module

Normalizes activations to zero mean and unit variance per feature across the batch. Maintains running statistics for eval mode.

```

class BatchNorm(Module):
    def __init__(self, size, momentum=0.1, eps=1e-5):
        super().__init__()
        self.size = size
        self.gamma = [1.0] * size
        self.beta = [0.0] * size
        self.gamma_grads = [0.0] * size
        self.beta_grads = [0.0] * size
        self.running_mean = [0.0] * size
        self.running_var = [1.0] * size
        self.momentum = momentum
        self.eps = eps
        self.x_norm = None
        self.std_inv = None
        self.batch_input = None

    def forward_batch(self, batch):

```

```

batch_size = len(batch)
output_batch = []

if self.training:
    mean = [0.0] * self.size
    for sample in batch:
        for j in range(self.size):
            mean[j] += sample[j]
    mean = [m / batch_size for m in mean]

    var = [0.0] * self.size
    for sample in batch:
        for j in range(self.size):
            var[j] += (sample[j] - mean[j]) ** 2
    var = [v / batch_size for v in var]

    self.std_inv = [1.0 / math.sqrt(v + self.eps) for v in var]

    self.x_norm = []
    self.batch_input = batch
    for sample in batch:
        normed = [(sample[j] - mean[j]) * self.std_inv[j] for j in
↪ range(self.size)]
        self.x_norm.append(normed)
        output = [self.gamma[j] * normed[j] + self.beta[j] for j in
↪ range(self.size)]
        output_batch.append(output)

        for j in range(self.size):
            self.running_mean[j] = (1 - self.momentum) * self.running_mean[j] +
↪ self.momentum * mean[j]
            self.running_var[j] = (1 - self.momentum) * self.running_var[j] +
↪ self.momentum * var[j]
    else:
        std_inv = [1.0 / math.sqrt(v + self.eps) for v in self.running_var]
        for sample in batch:
            normed = [(sample[j] - self.running_mean[j]) * std_inv[j] for j in
↪ range(self.size)]
            output = [self.gamma[j] * normed[j] + self.beta[j] for j in
↪ range(self.size)]
            output_batch.append(output)

    return output_batch

def forward(self, x):
    result = self.forward_batch([x])
    return result[0]

def backward(self, grad):
    if self.x_norm is None:
        return grad
    for j in range(self.size):
        self.gamma_grads[j] += self.x_norm[0][j] * grad[j]
        self.beta_grads[j] += grad[j]

```

```

    return [grad[j] * self.gamma[j] * self.std_inv[j] for j in range(self.size)]

def parameters(self):
    params = []
    for j in range(self.size):
        params.append((self.gamma, j, None, self.gamma_grads))
        params.append((self.beta, j, None, self.beta_grads))
    return params

```

Step 6: Sequential Container

Chains modules. Forward goes left-to-right, backward goes right-to-left.

```

class Sequential(Module):
    def __init__(self, *modules):
        super().__init__()
        self.modules = list(modules)

    def forward(self, x):
        for module in self.modules:
            x = module.forward(x)
        return x

    def backward(self, grad):
        for module in reversed(self.modules):
            grad = module.backward(grad)
        return grad

    def parameters(self):
        params = []
        for module in self.modules:
            params.extend(module.parameters())
        return params

    def train(self):
        self.training = True
        for module in self.modules:
            module.train()

    def eval(self):
        self.training = False
        for module in self.modules:
            module.eval()

```

Step 7: Loss Functions

MSE and Binary Cross-Entropy. Each returns the loss value and provides a backward() that returns the gradient.

```

class MSELoss:
    def __call__(self, predicted, target):
        self.predicted = predicted
        self.target = target
        n = len(predicted)
        self.loss = sum((p - t) ** 2 for p, t in zip(predicted, target)) / n

```

```

        return self.loss

    def backward(self):
        n = len(self.predicted)
        return [2 * (p - t) / n for p, t in zip(self.predicted, self.target)]

class BCELoss:
    def __call__(self, predicted, target):
        self.predicted = predicted
        self.target = target
        eps = 1e-7
        n = len(predicted)
        self.loss = 0
        for p, t in zip(predicted, target):
            p = max(eps, min(1 - eps, p))
            self.loss += -(t * math.log(p) + (1 - t) * math.log(1 - p))
        self.loss /= n
        return self.loss

    def backward(self):
        eps = 1e-7
        n = len(self.predicted)
        grads = []
        for p, t in zip(self.predicted, self.target):
            p = max(eps, min(1 - eps, p))
            grads.append((-t / p + (1 - t) / (1 - p)) / n)
        return grads

```

Step 8: SGD and Adam Optimizers

Both take a parameter list and update weights using gradients.

```

class SGD:
    def __init__(self, parameters, lr=0.01):
        self.params = parameters
        self.lr = lr

    def step(self):
        for container, i, j, grad_container in self.params:
            if j is not None:
                container[i][j] -= self.lr * grad_container[i][j]
            else:
                container[i] -= self.lr * grad_container[i]

    def zero_grad(self):
        for container, i, j, grad_container in self.params:
            if j is not None:
                grad_container[i][j] = 0.0
            else:
                grad_container[i] = 0.0

class Adam:

```

```

def __init__(self, parameters, lr=0.001, beta1=0.9, beta2=0.999, eps=1e-8):
    self.params = parameters
    self.lr = lr
    self.beta1 = beta1
    self.beta2 = beta2
    self.eps = eps
    self.t = 0
    self.m = [0.0] * len(parameters)
    self.v = [0.0] * len(parameters)

def step(self):
    self.t += 1
    for idx, (container, i, j, grad_container) in enumerate(self.params):
        if j is not None:
            g = grad_container[i][j]
        else:
            g = grad_container[i]

        self.m[idx] = self.beta1 * self.m[idx] + (1 - self.beta1) * g
        self.v[idx] = self.beta2 * self.v[idx] + (1 - self.beta2) * g * g

        m_hat = self.m[idx] / (1 - self.beta1 ** self.t)
        v_hat = self.v[idx] / (1 - self.beta2 ** self.t)

        update = self.lr * m_hat / (math.sqrt(v_hat) + self.eps)

        if j is not None:
            container[i][j] -= update
        else:
            container[i] -= update

def zero_grad(self):
    for container, i, j, grad_container in self.params:
        if j is not None:
            grad_container[i][j] = 0.0
        else:
            grad_container[i] = 0.0

```

Step 9: DataLoader

Splits data into batches, optionally shuffles each epoch.

```

class DataLoader:
    def __init__(self, data, batch_size=32, shuffle=True):
        self.data = data
        self.batch_size = batch_size
        self.shuffle = shuffle

    def __iter__(self):
        indices = list(range(len(self.data)))
        if self.shuffle:
            random.shuffle(indices)
        for start in range(0, len(indices), self.batch_size):
            batch_indices = indices[start:start + self.batch_size]

```

```

        batch = [self.data[i] for i in batch_indices]
        inputs = [item[0] for item in batch]
        targets = [item[1] for item in batch]
        yield inputs, targets

    def __len__(self):
        return (len(self.data) + self.batch_size - 1) // self.batch_size

```

Step 10: Train a 4-Layer Network on Circle Classification

Wire everything together. Define a model, pick a loss, pick an optimizer, run the training loop.

```

def make_circle_data(n=500, seed=42):
    random.seed(seed)
    data = []
    for _ in range(n):
        x = random.uniform(-2, 2)
        y = random.uniform(-2, 2)
        label = 1.0 if x * x + y * y < 1.5 else 0.0
        data.append([x, y], [label])
    return data

def train():
    random.seed(42)

    model = Sequential(
        Linear(2, 16),
        ReLU(),
        Linear(16, 16),
        ReLU(),
        Linear(16, 8),
        ReLU(),
        Linear(8, 1),
        Sigmoid(),
    )

    criterion = BCELoss()
    optimizer = Adam(model.parameters(), lr=0.01)

    data = make_circle_data(500)
    split = int(len(data) * 0.8)
    train_data = data[:split]
    test_data = data[split:]

    loader = DataLoader(train_data, batch_size=16, shuffle=True)

    model.train()

    for epoch in range(100):
        total_loss = 0
        total_correct = 0
        total_samples = 0

```

```

for batch_inputs, batch_targets in loader:
    batch_loss = 0
    for x, t in zip(batch_inputs, batch_targets):
        pred = model.forward(x)
        loss = criterion(pred, t)
        batch_loss += loss

    optimizer.zero_grad()
    grad = criterion.backward()
    model.backward(grad)
    optimizer.step()

    predicted_class = 1.0 if pred[0] >= 0.5 else 0.0
    if predicted_class == t[0]:
        total_correct += 1
    total_samples += 1

    total_loss += batch_loss

avg_loss = total_loss / total_samples
accuracy = total_correct / total_samples * 100

if epoch % 10 == 0 or epoch == 99:
    print(f"Epoch {epoch:3d} | Loss: {avg_loss:.6f} | Train Accuracy:
    ↪ {accuracy:.1f}%")

model.eval()
correct = 0
for x, t in test_data:
    pred = model.forward(x)
    predicted_class = 1.0 if pred[0] >= 0.5 else 0.0
    if predicted_class == t[0]:
        correct += 1
test_accuracy = correct / len(test_data) * 100
print(f"\nTest Accuracy: {test_accuracy:.1f}% ({correct}/{len(test_data)})")

return model, test_accuracy

```

Use It

Here is the PyTorch equivalent of what you just built:

```

import torch
import torch.nn as nn
from torch.utils.data import DataLoader, TensorDataset

```

```

model = nn.Sequential(
    nn.Linear(2, 16),
    nn.ReLU(),
    nn.Linear(16, 16),
    nn.ReLU(),
    nn.Linear(16, 8),
    nn.ReLU(),

```



```

        nn.Linear(8, 1),
        nn.Sigmoid(),
    )

criterion = nn.BCELoss()
optimizer = torch.optim.Adam(model.parameters(), lr=0.01)

for epoch in range(100):
    model.train()
    for inputs, targets in dataloader:
        optimizer.zero_grad()
        predictions = model(inputs)
        loss = criterion(predictions, targets)
        loss.backward()
        optimizer.step()

    model.eval()
    with torch.no_grad():
        test_predictions = model(test_inputs)

```

The structure is identical. Sequential, Linear, ReLU, Sigmoid, BCELoss, Adam, zero_grad, backward, step, train, eval. Every concept maps one-to-one. The difference is that PyTorch handles autograd automatically (no need to implement backward() in each module), runs on GPU, and has been optimized for years. But the bones are the same.

Now when you see PyTorch code, you know exactly what is happening at every line. That understanding is the whole point.

This chapter ships an artifact. The course version of this lesson produces a reusable prompt or agent skill. It lives in the repository, ready to install: <https://github.com/rohitg00/ai-engineering-from-scratch/tree/main/phases/03-deep-learning-core/10-mini-framework>

Exercises

Starter code and the lesson's working implementation: <https://github.com/rohitg00/ai-engineering-from-scratch/tree/main/phases/03-deep-learning-core/10-mini-framework/code>

1. Add a SoftmaxCrossEntropyLoss class for multi-class classification. Softmax the predictions, compute cross-entropy loss, and handle the combined backward pass. Test it on a 3-class spiral dataset.
2. Implement learning rate scheduling in the optimizer: add a set_lr() method and wire in the cosine schedule from Lesson 09. Train the circle classifier with warmup + cosine and compare to constant LR.
3. Add a save() and load() method to Sequential that serializes all weights to a JSON file and loads them back. Verify that a loaded model produces the same predictions as the original.
4. Implement weight decay (L2 regularization) in the Adam optimizer. Add a weight_decay parameter that shrinks weights toward zero each step. Compare training with decay=0 vs decay=0.01.
5. Replace the per-sample training loop with proper mini-batch gradient accumulation: accumulate gradients across all samples in a batch, then divide by batch size and take one optimizer step. Measure whether this changes convergence speed.

Key Terms

Term	What people say	What it actually means
Module	"A layer"	The base abstraction in a framework – anything with forward(), backward(), and parameters()
Sequential	"Stack layers in order"	A container that chains modules, applying them in sequence for forward and reverse for backward
Forward pass	"Run the network"	Computing the output by passing input through each module in order
Backward pass	"Compute gradients"	Propagating the loss gradient through each module in reverse to compute parameter gradients
Parameters	"The trainable weights"	All values in the network that the optimizer can update – weights and biases
Optimizer	"The thing that updates weights"	An algorithm that uses gradients to update parameters, implementing SGD, Adam, or other rules
DataLoader	"The thing that feeds data"	An iterator that splits a dataset into batches, optionally shuffling between epochs
Training mode	"model.train()"	A flag that enables stochastic behavior like dropout and batch normalization with batch stats
Evaluation mode	"model.eval()"	A flag that disables dropout and uses running statistics for batch normalization
Zero grad	"Clear the gradients"	Resetting all parameter gradients to zero before computing the next batch's gradients

Further Reading

- Paszke et al., "PyTorch: An Imperative Style, High-Performance Deep Learning Library" (2019) – the paper describing PyTorch's design decisions
- Chollet, "Deep Learning with Python, Second Edition" (2021) – Chapter 3 covers Keras internals with the same module/layer abstraction
- Johnson, "Tiny-DNN" (<https://github.com/tiny-dnn/tiny-dnn>) – a header-only C++ deep learning framework for understanding framework internals

Continue online. The living edition of this chapter has more than the page can hold:

- Animated, interactive figures and the web text: <https://aiengineeringfromscratch.com/lesson?path=phases/03-deep-learning-core/10-mini-framework>
- Runnable code for every step: <https://github.com/rohitg00/ai-engineering-from-scratch/tree/main/phases/03-deep-learning-core/10-mini-framework/code>
- The chapter quiz, graded in the browser: <https://aiengineeringfromscratch.com/lesson?path=phases/03-deep-learning-core/10-mini-framework>

The repository moves faster than any printing. When the book and the repo disagree, trust the repo.

Introduction to PyTorch

You built the engine from pistons and crankshafts. Now learn the one everyone actually drives.

Type: Build **Languages:** Python **Prerequisites:** Lesson 03.10 (Build Your Own Mini Framework) **Time:** ~75 minutes

Learning Objectives

- Build and train neural networks using PyTorch's `nn.Module`, `nn.Sequential`, and `autograd`
- Use PyTorch tensors, GPU acceleration, and the standard training loop (`zero_grad`, `forward`, `loss`, `backward`, `step`)
- Convert your from-scratch mini framework components to their PyTorch equivalents
- Profile and compare training speed between your pure-Python framework and PyTorch on the same task

The Problem

You have a working mini framework. Linear layers, ReLU, dropout, batch norm, Adam, a `DataLoader`, a training loop. It trains a 4-layer network on a circle classification problem in pure Python.

It is also 500x slower than PyTorch on the same problem.

Your mini framework processes one sample at a time with nested Python loops. PyTorch dispatches the same operations to optimized C++/CUDA kernels that run on GPU. On a single NVIDIA A100, PyTorch trains a ResNet-50 (25.6M parameters) on ImageNet (1.28M images) in about 6 hours. Your framework would take roughly 3,000 hours on the same task - if it didn't run out of memory first.

Speed is not the only gap. Your framework has no GPU support. No automatic differentiation - you hand-wrote `backward()` for every module. No serialization. No distributed training. No mixed precision. No way to debug gradient flow without print statements.

PyTorch fills every one of these gaps. And it does so while keeping the exact same mental model you already built: `Module`, `forward()`, `parameters()`, `backward()`, `optimizer.step()`. The concepts transfer one-to-one. The syntax is nearly identical. The difference is that PyTorch wraps a decade of systems engineering behind the same interface you designed from scratch.

The Concept

Why PyTorch Won

In 2015, TensorFlow required you to define a static computation graph before running anything. You built the graph, compiled it, then fed data through it. Debugging meant staring at graph visualizations. Changing the architecture meant rebuilding the graph from scratch.

PyTorch launched in 2017 with a different philosophy: eager execution. You write Python. It runs immediately. `y = model(x)` actually computes `y` right now, not “add a node to a graph that will compute `y` later.” This meant standard Python debugging tools worked. `print()` worked. `pdb` worked. `if/else` in your forward pass worked.

By 2020, the market had spoken. PyTorch’s share in ML research papers went from 7% (2017) to over 75% (2022). Meta, Google DeepMind, OpenAI, Anthropic, and Hugging Face all use PyTorch as their primary framework. TensorFlow 2.x adopted eager execution in response – tacit admission that PyTorch’s design was correct.

The lesson: developer experience compounds. A framework that is 10% slower but 50% faster to debug wins every time.

Tensors

A tensor is a multi-dimensional array with three critical properties: shape, dtype, and device.

```
import torch
```

```
x = torch.zeros(3, 4)           # shape: (3, 4), dtype: float32, device: cpu
x = torch.randn(2, 3, 224, 224) # batch of 2 RGB images, 224x224
x = torch.tensor([1, 2, 3])     # from a Python list
```

Shape is the dimensionality. A scalar is shape `()`, a vector is `(n,)`, a matrix is `(m, n)`, a batch of images is `(batch, channels, height, width)`.

Dtype controls precision and memory.

dtype	Bits	Range	Use case
float32	32	~7 decimal digits	Default training
float16	16	~3.3 decimal digits	Mixed precision
bfloat16	16	Same range as float32, less precision	LLM training
int8	8	-128 to 127	Quantized inference

Device determines where computation happens.

```
device = torch.device("cuda" if torch.cuda.is_available() else "cpu")
x = torch.randn(3, 4, device=device)
x = x.to("cuda")
x = x.cpu()
```

Every operation requires all tensors on the same device. This is the #1 PyTorch error beginners hit: `RuntimeError: Expected all tensors to be on the same device. Fix it by moving everything to the same device before computation.`

Reshaping is constant-time – it changes the metadata, not the data.

```
x = torch.randn(2, 3, 4)
x.view(2, 12)           # reshape to (2, 12) -- must be contiguous
x.reshape(6, 4)         # reshape to (6, 4) -- works always
x.permute(2, 0, 1)      # reorder dimensions
x.unsqueeze(0)          # add dimension: (1, 2, 3, 4)
x.squeeze()             # remove size-1 dimensions
```

Autograd

Your mini framework required you to implement `backward()` for every module. PyTorch does not. It records every operation on tensors into a directed acyclic graph (the computational graph) and then traverses that graph in reverse to compute gradients automatically.

Diagram. Rendered live in the web edition: <https://aiengineeringfromscratch.com/lesson?path=phases/03-deep-learning-core/11-intro-to-pytorch>

The key difference from your framework: PyTorch uses tape-based autodiff. Every operation appends to a “tape” during the forward pass. Calling `.backward()` replays the tape in reverse.

```
x = torch.randn(3, requires_grad=True)
y = x ** 2 + 3 * x
z = y.sum()
z.backward()
print(x.grad)  # dz/dx = 2x + 3
```

Three rules of autograd:

1. Only leaf tensors with `requires_grad=True` accumulate gradients
2. Gradients accumulate by default – call `optimizer.zero_grad()` before each backward pass
3. `torch.no_grad()` disables gradient tracking (use during evaluation)

nn.Module

`nn.Module` is the base class for every neural network component in PyTorch. You already built this abstraction in Lesson 10. PyTorch’s version adds automatic parameter registration, recursive module discovery, device management, and state dict serialization.

```
import torch.nn as nn

class MLP(nn.Module):
    def __init__(self, input_dim, hidden_dim, output_dim):
        super().__init__()
        self.layer1 = nn.Linear(input_dim, hidden_dim)
        self.relu = nn.ReLU()
        self.layer2 = nn.Linear(hidden_dim, output_dim)

    def forward(self, x):
        x = self.layer1(x)
        x = self.relu(x)
        x = self.layer2(x)
        return x
```

When you assign an `nn.Module` or `nn.Parameter` as an attribute in `__init__`, PyTorch automatically registers it. `model.parameters()` recursively collects every registered parameter. This is why you never have to manually gather weights like you did in the mini framework.

Key building blocks:

Module	What it does	Parameters
<code>nn.Linear(in, out)</code>	$Wx + b$	<code>in*out + out</code>
<code>nn.Conv2d(in_ch, out_ch, k)</code>	2D convolution	<code>in_ch*out_ch*k*k + out_ch</code>

Module	What it does	Parameters
<code>nn.BatchNorm1d(features)</code>	Normalize activations	2 * features
<code>nn.Dropout(p)</code>	Random zeroing	0
<code>nn.ReLU()</code>	$\max(0, x)$	0
<code>nn.GELU()</code>	Gaussian error linear	0
<code>nn.Embedding(vocab, dim)</code>	Lookup table	vocab * dim
<code>nn.LayerNorm(dim)</code>	Per-sample normalization	2 * dim

Loss Functions and Optimizers

PyTorch ships production-ready versions of everything you built.

Loss functions (from `torch.nn`):

Loss	Task	Input
<code>nn.MSELoss()</code>	Regression	Any shape
<code>nn.CrossEntropyLoss()</code>	Multi-class classification	Logits (not softmax)
<code>nn.BCEWithLogitsLoss()</code>	Binary classification	Logits (not sigmoid)
<code>nn.L1Loss()</code>	Regression (robust)	Any shape
<code>nn.CTCLoss()</code>	Sequence alignment	Log probabilities

Note: `CrossEntropyLoss` combines `LogSoftmax` + `NLLLoss` internally. Pass raw logits, not softmax outputs. This is a common mistake that produces wrong gradients silently.

Optimizers (from `torch.optim`):

Optimizer	When to use	Typical LR
<code>SGD(params, lr, momentum)</code>	CNNs, well-tuned pipelines	0.01-0.1
<code>Adam(params, lr)</code>	Default starting point	1e-3
<code>AdamW(params, lr, weight_decay)</code>	Transformers, fine-tuning	1e-4-1e-3
<code>LBFGS(params)</code>	Small-scale, second-order	1.0

The Training Loop

Every PyTorch training loop follows the same 5-step pattern. You already know this from Lesson 10.

Diagram. Rendered live in the web edition: <https://aiengineeringfromscratch.com/lesson?path=phases/03-deep-learning-core/11-intro-to-pytorch>

The canonical pattern:

```
for epoch in range(num_epochs):
    model.train()
    for inputs, targets in train_loader:
        inputs, targets = inputs.to(device), targets.to(device)
        optimizer.zero_grad()
        outputs = model(inputs)
```

```

    loss = criterion(outputs, targets)
    loss.backward()
    optimizer.step()

```

Five lines inside the batch loop. Five lines that trained GPT-4, Stable Diffusion, and LLaMA. The architecture changes. The data changes. These five lines do not.

Dataset and DataLoader

PyTorch's Dataset is an abstract class with two methods: `__len__` and `__getitem__`. DataLoader wraps it with batching, shuffling, and multi-process data loading.

```

from torch.utils.data import Dataset, DataLoader

```

```

class MNISTDataset(Dataset):
    def __init__(self, images, labels):
        self.images = images
        self.labels = labels

    def __len__(self):
        return len(self.labels)

    def __getitem__(self, idx):
        return self.images[idx], self.labels[idx]

```

```

loader = DataLoader(dataset, batch_size=64, shuffle=True, num_workers=4)

```

`num_workers=4` spawns 4 processes to load data in parallel while the GPU trains on the current batch. On disk-bound workloads (large images, audio), this alone can double training speed.

GPU Training

Moving a model to GPU:

```

device = torch.device("cuda" if torch.cuda.is_available() else "cpu")
model = model.to(device)

```

This recursively moves every parameter and buffer to the GPU. Then move each batch during training:

```

inputs, targets = inputs.to(device), targets.to(device)

```

Mixed precision halves memory usage and doubles throughput on modern GPUs (A100, H100, RTX 4090) by running forward/backward in float16 while keeping the master weights in float32:

```

from torch.amp import autocast, GradScaler

```

```

scaler = GradScaler()
for inputs, targets in loader:
    with autocast(device_type="cuda"):
        outputs = model(inputs)
        loss = criterion(outputs, targets)
    scaler.scale(loss).backward()
    scaler.step(optimizer)
    scaler.update()
    optimizer.zero_grad()

```

Comparison: Mini Framework vs PyTorch vs JAX

Feature	Mini Framework (L10)	PyTorch	JAX
Autodiff	Manual backward()	Tape-based autograd	Functional transforms
Execution	Eager (Python loops)	Eager (C++ kernels)	Traced + JIT compiled
GPU support	No	Yes (CUDA, ROCm, MPS)	Yes (CUDA, TPU)
Speed (MNIST MLP)	~300s/epoch	~0.5s/epoch	~0.3s/epoch
Module system	Custom Module class	nn.Module	Stateless functions (Flax/Equinox)
Debugging	print()	print(), pdb, breakpoint()	Harder (JIT tracing breaks print)
Ecosystem	None	Hugging Face, Lightning, timm	Flax, Optax, Orbax
Learning curve	You built it	Moderate	Steep (functional paradigm)
Production use	Toy problems	Meta, OpenAI, Anthropic, HF	Google DeepMind, Midjourney

Interactive figure: dropout-mask. This one moves. Watch it animate and drag its controls in the web edition: <https://aiengineeringfromscratch.com/lesson?path=phases/03-deep-learning-core/11-intro-to-pytorch>

Build It

A 3-layer MLP trained on MNIST using only PyTorch primitives. No high-level wrappers. No torchvision.datasets. We download and parse the raw data ourselves.

Step 1: Load MNIST From Raw Files

MNIST ships as 4 gzipped files: training images (60,000 x 28 x 28), training labels, test images (10,000 x 28 x 28), test labels. We download them and parse the binary format.

```
import torch
import torch.nn as nn
```



```

import struct
import gzip
import urllib.request
import os

def download_mnist(path="./mnist_data"):
    base_url = "https://storage.googleapis.com/cvdf-datasets/mnist/"
    files = [
        "train-images-idx3-ubyte.gz",
        "train-labels-idx1-ubyte.gz",
        "t10k-images-idx3-ubyte.gz",
        "t10k-labels-idx1-ubyte.gz",
    ]
    os.makedirs(path, exist_ok=True)
    for f in files:
        filepath = os.path.join(path, f)
        if not os.path.exists(filepath):
            urllib.request.urlretrieve(base_url + f, filepath)

def load_images(filepath):
    with gzip.open(filepath, "rb") as f:
        magic, num, rows, cols = struct.unpack(">IIII", f.read(16))
        data = f.read()
        images = torch.frombuffer(bytearray(data), dtype=torch.uint8)
        images = images.reshape(num, rows * cols).float() / 255.0
    return images

def load_labels(filepath):
    with gzip.open(filepath, "rb") as f:
        magic, num = struct.unpack(">II", f.read(8))
        data = f.read()
        labels = torch.frombuffer(bytearray(data), dtype=torch.uint8).long()
    return labels

```

Step 2: Define the Model

A 3-layer MLP: 784 -> 256 -> 128 -> 10. ReLU activations. Dropout for regularization. No batch norm to keep it simple.

```

class MNISTModel(nn.Module):
    def __init__(self):
        super().__init__()
        self.net = nn.Sequential(
            nn.Linear(784, 256),
            nn.ReLU(),
            nn.Dropout(0.2),
            nn.Linear(256, 128),
            nn.ReLU(),
            nn.Dropout(0.2),
            nn.Linear(128, 10),
        )

    def forward(self, x):
        return self.net(x)

```

The output layer produces 10 raw logits (one per digit). No softmax – CrossEntropyLoss handles that internally.

Parameter count: $784256 + 256 + 256128 + 128 + 128 \cdot 10 + 10 = 235,146$. Tiny by modern standards. GPT-2 small has 124M. This trains in seconds.

Step 3: Training Loop

The canonical forward-loss-backward-step pattern.

```
def train_one_epoch(model, loader, criterion, optimizer, device):
    model.train()
    total_loss = 0
    correct = 0
    total = 0
    for images, labels in loader:
        images, labels = images.to(device), labels.to(device)
        optimizer.zero_grad()
        outputs = model(images)
        loss = criterion(outputs, labels)
        loss.backward()
        optimizer.step()
        total_loss += loss.item() * images.size(0)
        _, predicted = outputs.max(1)
        correct += predicted.eq(labels).sum().item()
        total += labels.size(0)
    return total_loss / total, correct / total
```

```
def evaluate(model, loader, criterion, device):
    model.eval()
    total_loss = 0
    correct = 0
    total = 0
    with torch.no_grad():
        for images, labels in loader:
            images, labels = images.to(device), labels.to(device)
            outputs = model(images)
            loss = criterion(outputs, labels)
            total_loss += loss.item() * images.size(0)
            _, predicted = outputs.max(1)
            correct += predicted.eq(labels).sum().item()
            total += labels.size(0)
    return total_loss / total, correct / total
```

Note `torch.no_grad()` during evaluation. This disables autograd, reducing memory usage and speeding up inference. Without it, PyTorch builds a computational graph you never use.

Step 4: Wire Everything Together

```
def main():
    device = torch.device("cuda" if torch.cuda.is_available() else "cpu")

    download_mnist()
    train_images = load_images("./mnist_data/train-images-idx3-ubyte.gz")
```

```

train_labels = load_labels("./mnist_data/train-labels-idx1-ubyte.gz")
test_images = load_images("./mnist_data/t10k-images-idx3-ubyte.gz")
test_labels = load_labels("./mnist_data/t10k-labels-idx1-ubyte.gz")

train_dataset = torch.utils.data.TensorDataset(train_images, train_labels)
test_dataset = torch.utils.data.TensorDataset(test_images, test_labels)
train_loader = torch.utils.data.DataLoader(
    train_dataset, batch_size=64, shuffle=True
)
test_loader = torch.utils.data.DataLoader(
    test_dataset, batch_size=256, shuffle=False
)

model = MNISTModel().to(device)
criterion = nn.CrossEntropyLoss()
optimizer = torch.optim.Adam(model.parameters(), lr=1e-3)

num_params = sum(p.numel() for p in model.parameters())
print(f"Device: {device}")
print(f"Parameters: {num_params:,}")
print(f"Train samples: {len(train_dataset):,}")
print(f"Test samples: {len(test_dataset):,}")
print()

for epoch in range(10):
    train_loss, train_acc = train_one_epoch(
        model, train_loader, criterion, optimizer, device
    )
    test_loss, test_acc = evaluate(
        model, test_loader, criterion, device
    )
    print(
        f"Epoch {epoch+1:2d} | "
        f"Train Loss: {train_loss:.4f} | Train Acc: {train_acc:.4f} | "
        f"Test Loss: {test_loss:.4f} | Test Acc: {test_acc:.4f}"
    )

torch.save(model.state_dict(), "mnist_mlp.pt")
print(f"\nModel saved to mnist_mlp.pt")
print(f"Final test accuracy: {test_acc:.4f}")

```

Expected output after 10 epochs: ~97.8% test accuracy. Training time on CPU: ~30 seconds. On GPU: ~5 seconds. On your mini framework with the same architecture: ~45 minutes.

Use It

Quick Comparison: Mini Framework vs PyTorch

Mini Framework (Lesson 10)	PyTorch
<code>model = Sequential(Linear(784, 256), ReLU(), ...)</code>	<code>model = nn.Sequential(nn.Linear(784, 256), nn.ReLU(), ...)</code>
<code>pred = model.forward(x)</code> <code>optimizer.zero_grad()</code>	<code>pred = model(x)</code> <code>optimizer.zero_grad()</code>
<code>grad = criterion.backward()</code> then <code>model.backward(grad)</code> <code>optimizer.step()</code>	<code>loss.backward()</code> <code>optimizer.step()</code>
No GPU Manual backward for every module	<code>model.to("cuda")</code> Autograd handles everything

The interface is nearly identical. The difference is everything under the hood.

Saving and Loading Models

```
torch.save(model.state_dict(), "model.pt")

model = MNISTModel()
model.load_state_dict(torch.load("model.pt", weights_only=True))
model.eval()
```

Always save `state_dict()` (the parameter dictionary), not the model object. Saving the model object uses pickle, which breaks when you refactor code. State dicts are portable.

Learning Rate Scheduling

```
scheduler = torch.optim.lr_scheduler.CosineAnnealingLR(
    optimizer, T_max=10
)
for epoch in range(10):
    train_one_epoch(model, train_loader, criterion, optimizer, device)
    scheduler.step()
```

PyTorch ships 15+ schedulers: StepLR, ExponentialLR, CosineAnnealingLR, OneCycleLR, ReduceLROnPlateau. All plug into the same optimizer interface.

This chapter ships an artifact. The course version of this lesson produces a reusable prompt or agent skill. It lives in the repository, ready to install: <https://github.com/rohitg00/ai-engineering-from-scratch/tree/main/phases/03-deep-learning-core/11-intro-to-pytorch>

Exercises

Starter code and the lesson's working implementation: <https://github.com/rohitg00/ai-engineering-from-scratch/tree/main/phases/03-deep-learning-core/11-intro-to-pytorch/code>

1. **Add batch normalization.** Insert `nn.BatchNorm1d` after each linear layer (before the activation). Compare test accuracy and training speed vs the dropout-only version. Batch norm should reach 98%+ in fewer epochs.

2. **Implement a learning rate finder.** Train for one epoch with exponentially increasing learning rate (from $1e-7$ to 1.0). Plot loss vs LR. The optimal LR is just before the loss starts climbing. Use this to pick a better LR for the MNIST model.
3. **Port to GPU with mixed precision.** Add `torch.amp.autocast` and `GradScaler` to the training loop. Measure throughput (samples/second) with and without mixed precision on GPU. On an A100, expect $\sim 2x$ speedup.
4. **Build a custom Dataset.** Download Fashion-MNIST (same format as MNIST but with clothing items). Implement a `FashionMNISTDataset(Dataset)` class with `__getitem__` and `__len__`. Train the same MLP and compare accuracy. Fashion-MNIST is harder - expect $\sim 88\%$ vs $\sim 98\%$.
5. **Replace Adam with SGD + momentum.** Train with `SGD(params, lr=0.01, momentum=0.9)`. Compare convergence curves. Then add a `CosineAnnealingLR` scheduler and see if SGD catches up to Adam by epoch 10.

Key Terms

Term	What people say	What it actually means
Tensor	"A multi-dimensional array"	A typed, device-aware array with automatic differentiation support baked into every operation
Autograd	"Automatic backprop"	A tape-based system that records operations during forward pass, then replays them in reverse to compute exact gradients
nn.Module	"A layer"	The base class for any differentiable computation block - registers parameters, supports nesting, handles train/eval modes
state_dict	"The model weights"	An <code>OrderedDict</code> mapping parameter names to tensors - the portable, serializable representation of a trained model
.backward()	"Compute gradients"	Traverse the computational graph in reverse, computing and accumulating gradients for every leaf tensor with <code>requires_grad=True</code>
.to(device)	"Move to GPU"	Recursively transfer all parameters and buffers to the specified device (CPU, CUDA, MPS)
DataLoader	"The data pipeline"	An iterator that batches, shuffles, and optionally parallelizes data loading from a Dataset
Mixed precision	"Use float16"	Train with float16 forward/backward for speed while keeping float32 master weights for numerical stability
Eager execution	"Run it now"	Operations execute immediately when called, not deferred to a later compilation step - the core design choice that differentiates PyTorch from TF 1.x
zero_grad	"Reset gradients"	Set all parameter gradients to zero before the next backward pass, since PyTorch accumulates gradients by default

Further Reading

- Paszke et al., “PyTorch: An Imperative Style, High-Performance Deep Learning Library” (2019) – the original paper explaining PyTorch’s design tradeoffs
- PyTorch Tutorials: “Learning PyTorch with Examples” (https://pytorch.org/tutorials/beginner/pytorch_with_examples.html) – the official path from tensors to nn.Module
- PyTorch Performance Tuning Guide (https://pytorch.org/tutorials/recipes/recipes/tuning_guide.html) – mixed precision, DataLoader workers, pinned memory, and other production optimizations
- Horace He, “Making Deep Learning Go Brrrr” (https://horace.io/brrr_intro.html) – why GPU training is fast, with PyTorch-specific optimization strategies

Continue online. The living edition of this chapter has more than the page can hold:

- Animated, interactive figures and the web text: <https://aiengineeringfromscratch.com/lesson?path=phases/03-deep-learning-core/11-intro-to-pytorch>
- Runnable code for every step: <https://github.com/rohitg00/ai-engineering-from-scratch/tree/main/phases/03-deep-learning-core/11-intro-to-pytorch/code>
- The chapter quiz, graded in the browser: <https://aiengineeringfromscratch.com/lesson?path=phases/03-deep-learning-core/11-intro-to-pytorch>

The repository moves faster than any printing. When the book and the repo disagree, trust the repo.

Introduction to JAX

PyTorch mutates tensors. TensorFlow builds graphs. JAX compiles pure functions. That last one changes how you think about deep learning.

Type: Build **Languages:** Python **Prerequisites:** Phase 03 Lessons 01-10, basic NumPy
Time: ~90 minutes

Learning Objectives

- Write pure-function neural network code using JAX's functional API (`jax.numpy`, `jax.grad`, `jax.jit`, `jax.vmap`)
- Explain the key design difference between PyTorch's eager mutation and JAX's functional compilation model
- Apply `jit` compilation and `vmap` vectorization to accelerate training loops compared to naive Python
- Train a simple network in JAX and contrast the explicit state management with PyTorch's object-oriented approach

The Problem

You know how to build neural networks in PyTorch. You define an `nn.Module`, call `.backward()`, step the optimizer. It works. Millions of people use it.

But PyTorch has a constraint baked into its DNA: it traces operations eagerly, one at a time, in Python. Every tensor + tensor is a separate kernel launch. Every training step re-interprets the same Python code. This works fine until you need to train a 540-billion-parameter model across 2,048 TPUs. Then the overhead kills you.

Google DeepMind trains Gemini on JAX. Anthropic trained Claude on JAX. These are not small operations - they are the largest neural network training runs on Earth. They chose JAX because it treats your training loop as a compilable program, not a sequence of Python calls.

JAX is NumPy with three superpowers: automatic differentiation, JIT compilation to XLA, and automatic vectorization. You write a function that processes one example. JAX gives you a function that processes a batch, computes gradients, compiles to machine code, and runs across multiple devices. All without changing the original function.

The Concept

The JAX Philosophy

JAX is a functional framework. No classes, no mutable state, no `.backward()` method. Instead:

PyTorch	JAX
<code>nn.Module</code> class with state	Pure function: <code>f(params, x) -> y</code>
<code>loss.backward()</code>	<code>jax.grad(loss_fn)(params, x, y)</code>
Eager execution	JIT compilation via XLA
<code>for x in batch:</code> manual loop	<code>jax.vmap(f)</code> auto-vectorization
<code>DataParallel</code> / FSDP	<code>jax.pmap(f)</code> auto-parallelism
<code>Mutable model.parameters()</code>	Immutable pytree of arrays

This is not a style preference. It is a compiler constraint. JIT compilation requires pure functions - same inputs always produce same outputs, no side effects. That restriction is what makes 100x speedups possible.

jax.numpy: The Familiar Surface

JAX reimplements the NumPy API on accelerators:

```
import jax.numpy as jnp

a = jnp.array([1.0, 2.0, 3.0])
b = jnp.array([4.0, 5.0, 6.0])
c = jnp.dot(a, b)
```

Same function names. Same broadcasting rules. Same slicing semantics. But the arrays live on GPU/TPU, and every operation is traceable by the compiler.

One critical difference: JAX arrays are immutable. No `a[0] = 5`. Instead: `a = a.at[0].set(5)`. This feels awkward for a week, then it clicks - immutability is what makes transformations like `grad`, `jit`, and `vmap` composable.

jax.grad: Functional Autodiff

PyTorch attaches gradients to tensors (`.grad`). JAX attaches gradients to functions.

```
import jax

def f(x):
    return x ** 2

df = jax.grad(f)
df(3.0)
```

`jax.grad` takes a function and returns a new function that computes the gradient. No `.backward()` call. No computation graph stored on tensors. The gradient is just another function you can call, compose, or JIT-compile.

This composes arbitrarily:

```
d2f = jax.grad(jax.grad(f))
d2f(3.0)
```

Second derivatives. Third derivatives. Jacobians. Hessians. All by composing `grad`. PyTorch can do this too (`torch.autograd.functional.hessian`), but it is bolted on. In JAX, it is the foundation.

The constraint: grad only works on pure functions. No print statements inside (they run during tracing, not execution). No mutation of external state. No random number generation without explicit key management.

jit: Compile to XLA

```
@jax.jit
def train_step(params, x, y):
    loss = loss_fn(params, x, y)
    return loss
```

```
fast_step = jax.jit(train_step)
```

On the first call, JAX traces the function – it records which operations happen, without executing them. Then it hands that trace to XLA (Accelerated Linear Algebra), Google’s compiler for TPUs and GPUs. XLA fuses operations, eliminates redundant memory copies, and generates optimized machine code.

Subsequent calls skip Python entirely. The compiled code runs on the accelerator at C++ speed.

When JIT helps: - Training steps (same computation repeated thousands of times) - Inference (same model, different inputs) - Any function called more than once with similar-shaped inputs

When JIT hurts: - Functions with Python control flow that depends on values (if $x > 0$ where x is a traced array) - One-shot computations (compilation overhead exceeds runtime) - Debugging (tracing hides the actual execution)

The control flow restriction is real. `jax.lax.cond` replaces `if/else`. `jax.lax.scan` replaces for loops. These are not optional – they are the price of compilation.

vmap: Automatic Vectorization

You write a function that processes one example:

```
def predict(params, x):
    return jnp.dot(params['w'], x) + params['b']
```

`vmap` lifts it to process a batch:

```
batch_predict = jax.vmap(predict, in_axes=(None, 0))
```

`in_axes=(None, 0)` means: do not batch over `params` (shared), batch over axis 0 of `x`. No manual for loop. No reshaping. No batch dimension threading. JAX figures out the batch dimension and vectorizes the entire computation.

This is not syntactic sugar. `vmap` generates fused vectorized code that runs 10-100x faster than a Python loop. And it composes with `jit` and `grad`:

```
per_example_grads = jax.vmap(jax.grad(loss_fn), in_axes=(None, 0, 0))
```

Per-example gradients. One line. This is nearly impossible in PyTorch without hacks.

pmap: Data Parallelism Across Devices

```
parallel_step = jax.pmap(train_step, axis_name='devices')
```

`pmap` replicates the function across all available devices (GPUs/TPUs) and splits the batch. Inside the function, `jax.lax.pmean` and `jax.lax.psum` synchronize gradients across devices.

Google trains Gemini across thousands of TPU v5e chips using `pmap` (and its successor `shard_map`). The programming model: write the single-device version, wrap with `pmap`, done.

Pytrees: The Universal Data Structure

JAX operates on “pytrees” – nested combinations of lists, tuples, dicts, and arrays. Your model parameters are a pytree:

```
params = {
    'layer1': {'w': jnp.zeros((784, 256)), 'b': jnp.zeros(256)},
    'layer2': {'w': jnp.zeros((256, 128)), 'b': jnp.zeros(128)},
    'layer3': {'w': jnp.zeros((128, 10)), 'b': jnp.zeros(10)},
}
```

Every JAX transformation – `grad`, `jit`, `vmap` – knows how to traverse pytrees. `jax.tree.map(f, tree)` applies `f` to every leaf. This is how optimizers update all parameters at once:

```
params = jax.tree.map(lambda p, g: p - lr * g, params, grads)
```

No `.parameters()` method. No parameter registration. The tree structure is the model.

Functional vs Object-Oriented

PyTorch stores state inside objects:

```
class Model(nn.Module):
    def __init__(self):
        self.linear = nn.Linear(784, 10)

    def forward(self, x):
        return self.linear(x)
```

JAX uses pure functions with explicit state:

```
def predict(params, x):
    return jnp.dot(x, params['w']) + params['b']
```

The params are passed in. Nothing is stored. Nothing is mutated. This makes every function testable, composable, and compilable. It also means you manage the params yourself – or use a library like Flax or Equinox.

The JAX Ecosystem

JAX gives you primitives. Libraries give you ergonomics:

Library	Role	Style
Flax (Google)	Neural network layers	<code>nn.Module</code> with explicit state
Equinox (Patrick Kidger)	Neural network layers	Pytree-based, Pythonic
Optax (DeepMind)	Optimizers + LR schedules	Composable gradient transforms
Orbax (Google)	Checkpointing	Save/restore pytrees
CLU (Google)	Metrics + logging	Training loop utilities

Optax is the standard optimizer library. It separates the gradient transformation (Adam, SGD, clipping) from the parameter update, making it trivial to compose:

```
optimizer = optax.chain(
    optax.clip_by_global_norm(1.0),
    optax.adam(learning_rate=1e-3),
)
```

When to Use JAX vs PyTorch

Factor	JAX	PyTorch
TPU support	First-class (Google built both)	Community-maintained (torch_xla)
GPU support	Good (CUDA via XLA)	Best-in-class (native CUDA)
Debugging	Hard (tracing + compilation)	Easy (eager, line-by-line)
Ecosystem	Research-focused (Flax, Equinox)	Massive (HuggingFace, torchvision, etc.)
Hiring	Niche (Google/DeepMind/Anthropic)	Mainstream (everywhere)
Large-scale training	Superior (XLA, pmap, mesh)	Good (FSDP, DeepSpeed)
Prototyping speed	Slower (functional overhead)	Faster (mutate and go)
Production inference	TensorFlow Serving, Vertex AI	TorchServe, Triton, ONNX
Who uses it	DeepMind (Gemini), Anthropic (Claude)	Meta (Llama), OpenAI (GPT), Stability AI

The honest answer: use PyTorch unless you have a specific reason to use JAX. Those reasons are – TPU access, need for per-example gradients, multi-device training at massive scale, or working at Google/DeepMind/Anthropic.

Random Numbers in JAX

JAX does not have a global random state. Every random operation requires an explicit PRNG key:

```
key = jax.random.PRNGKey(42)
key1, key2 = jax.random.split(key)
w = jax.random.normal(key1, shape=(784, 256))
```

This is annoying at first. But it guarantees reproducibility across devices and compilations – a property that PyTorch’s `torch.manual_seed` cannot guarantee in multi-GPU settings.

Interactive figure: batchnorm-effect. This one moves. Watch it animate and drag its controls in the web edition: <https://aiengineeringfromscratch.com/lesson?path=phases/03-deep-learning-core/12-intro-to-jax>

Build It

Step 1: Setup and Data

We will train a 3-layer MLP on MNIST using JAX and Optax. 784 inputs, two hidden layers of 256 and 128 neurons, 10 output classes.

```
import jax
import jax.numpy as jnp
from jax import random
import optax

def get_mnist_data():
    from sklearn.datasets import fetch_openml
    mnist = fetch_openml('mnist_784', version=1, as_frame=False, parser='auto')
    X = mnist.data.astype('float32') / 255.0
    y = mnist.target.astype('int')
    X_train, X_test = X[:60000], X[60000:]
    y_train, y_test = y[:60000], y[60000:]
    return X_train, y_train, X_test, y_test
```

Step 2: Initialize Parameters

No class. Just a function that returns a pytree:

```
def init_params(key):
    k1, k2, k3 = random.split(key, 3)
    scale1 = jnp.sqrt(2.0 / 784)
    scale2 = jnp.sqrt(2.0 / 256)
    scale3 = jnp.sqrt(2.0 / 128)
    params = {
        'layer1': {
            'w': scale1 * random.normal(k1, (784, 256)),
            'b': jnp.zeros(256),
        },
        'layer2': {
            'w': scale2 * random.normal(k2, (256, 128)),
            'b': jnp.zeros(128),
        },
        'layer3': {
            'w': scale3 * random.normal(k3, (128, 10)),
            'b': jnp.zeros(10),
        },
    }
    return params
```

He-initialization, done manually. Three PRNG keys split from one seed. Every weight is an immutable array in a nested dict.

Step 3: Forward Pass

```
def forward(params, x):
    x = jnp.dot(x, params['layer1']['w']) + params['layer1']['b']
    x = jax.nn.relu(x)
    x = jnp.dot(x, params['layer2']['w']) + params['layer2']['b']
```

```

x = jax.nn.relu(x)
x = jnp.dot(x, params['layer3']['w']) + params['layer3']['b']
return x

def loss_fn(params, x, y):
    logits = forward(params, x)
    one_hot = jax.nn.one_hot(y, 10)
    return -jnp.mean(jnp.sum(jax.nn.log_softmax(logits) * one_hot, axis=-1))

```

Pure functions. Params in, prediction out. No self, no stored state. `loss_fn` computes cross-entropy from scratch - softmax, log, negative mean.

Step 4: JIT-Compiled Training Step

```

@jax.jit
def train_step(params, opt_state, x, y):
    loss, grads = jax.value_and_grad(loss_fn)(params, x, y)
    updates, opt_state = optimizer.update(grads, opt_state, params)
    params = optax.apply_updates(params, updates)
    return params, opt_state, loss

@jax.jit
def accuracy(params, x, y):
    logits = forward(params, x)
    preds = jnp.argmax(logits, axis=-1)
    return jnp.mean(preds == y)

```

`jax.value_and_grad` returns both the loss value and the gradients in one pass. The `@jax.jit` decorator compiles both functions to XLA. After the first call, each training step runs without touching Python.

Step 5: Training Loop

```

optimizer = optax.adam(learning_rate=1e-3)

X_train, y_train, X_test, y_test = get_mnist_data()
X_train, X_test = jnp.array(X_train), jnp.array(X_test)
y_train, y_test = jnp.array(y_train), jnp.array(y_test)

key = random.PRNGKey(0)
params = init_params(key)
opt_state = optimizer.init(params)

batch_size = 128
n_epochs = 10

for epoch in range(n_epochs):
    key, subkey = random.split(key)
    perm = random.permutation(subkey, len(X_train))
    X_shuffled = X_train[perm]
    y_shuffled = y_train[perm]

    epoch_loss = 0.0
    n_batches = len(X_train) // batch_size

```

```

for i in range(n_batches):
    start = i * batch_size
    xb = X_shuffled[start:start + batch_size]
    yb = y_shuffled[start:start + batch_size]
    params, opt_state, loss = train_step(params, opt_state, xb, yb)
    epoch_loss += loss

train_acc = accuracy(params, X_train[:5000], y_train[:5000])
test_acc = accuracy(params, X_test, y_test)
print(f"Epoch {epoch + 1:2d} | Loss: {epoch_loss / n_batches:.4f} | "
      f"Train Acc: {train_acc:.4f} | Test Acc: {test_acc:.4f}")

```

10 epochs. ~97% test accuracy. The first epoch is slow (JIT compilation). Epochs 2-10 are fast.

Notice what is missing: no `.zero_grad()`, no `.backward()`, no `.step()`. The entire update is one composed function call. Gradients are computed, transformed by Adam, and applied to parameters - all inside `train_step`.

Use It

Flax: The Google Standard

Flax is the most common JAX neural network library. It adds `nn.Module` back, but with explicit state management:

```

import flax.linen as nn

class MLP(nn.Module):
    @nn.compact
    def __call__(self, x):
        x = nn.Dense(256)(x)
        x = nn.relu(x)
        x = nn.Dense(128)(x)
        x = nn.relu(x)
        x = nn.Dense(10)(x)
        return x

```

```

model = MLP()
params = model.init(jax.random.PRNGKey(0), jnp.ones((1, 784)))
logits = model.apply(params, x_batch)

```

Same structure as PyTorch, but `params` is separate from the model. `model.init()` creates `params`. `model.apply(params, x)` runs the forward pass. The model object has no state.

Equinox: The Pythonic Alternative

Equinox (by Patrick Kidger) represents models as pytrees:

```

import equinox as eqx

model = eqx.nn.MLP(
    in_size=784, out_size=10, width_size=256, depth=2,
    activation=jax.nn.relu, key=jax.random.PRNGKey(0))

```

```
)
logits = model(x)
```

The model itself is a pytree. No `.apply()` needed. Parameters are just the model's leaves. This is closer to how JAX thinks.

Optax: Composable Optimizers

Optax decouples the gradient transformation from the update:

```
schedule = optax.warmup_cosine_decay_schedule(
    init_value=0.0, peak_value=1e-3,
    warmup_steps=1000, decay_steps=50000
)

optimizer = optax.chain(
    optax.clip_by_global_norm(1.0),
    optax.adamw(learning_rate=schedule, weight_decay=0.01),
)
```

Gradient clipping, learning rate warmup, weight decay – all composed as a chain of transforms. Each transform sees the gradients, modifies them, and passes them to the next. No monolithic optimizer class.

This chapter ships an artifact. The course version of this lesson produces a reusable prompt or agent skill. It lives in the repository, ready to install: <https://github.com/rohitg00/ai-engineering-from-scratch/tree/main/phases/03-deep-learning-core/12-intro-to-jax>

Exercises

Starter code and the lesson's working implementation: <https://github.com/rohitg00/ai-engineering-from-scratch/tree/main/phases/03-deep-learning-core/12-intro-to-jax/code>

1. Add dropout to the MLP. In JAX, dropout requires a PRNG key – thread a key through the forward pass and split it for each dropout layer. Compare test accuracy with and without.
2. Use `jax.vmap` to compute per-example gradients for a batch of 32 MNIST images. Compute the gradient norm for each example. Which examples have the largest gradients, and why?
3. Replace the manual forward function with a generic `mlp_forward(params, x)` that works for any number of layers. Use `jax.tree.leaves` to determine the depth automatically.
4. Benchmark the training step with and without `@jax.jit`. Time 100 steps of each. How large is the speedup on your hardware? What is the compilation overhead on the first call?
5. Implement gradient clipping by composing `optax.chain(optax.clip_by_global_norm(1.0), optax.adam(1e-3))`. Train with and without clipping. Plot the gradient norm over training to see the effect.

Key Terms

Term	What people say	What it actually means
XLA	“The thing that makes JAX fast”	Accelerated Linear Algebra – a compiler that fuses operations and generates optimized GPU/TPU kernels from a computation graph
JIT	“Just-in-time compilation”	JAX traces the function on first call, compiles to XLA, then runs the compiled version on subsequent calls
Pure function	“No side effects”	A function where the output depends only on inputs – no global state, no mutation, no randomness without explicit keys
vmap	“Auto-batching”	Transforms a function that processes one example into one that processes a batch, without rewriting
pmap	“Auto-parallelism”	Replicates a function across multiple devices and splits the input batch
Pytree	“Nested dict of arrays”	Any nested structure of lists, tuples, dicts, and arrays that JAX can traverse and transform
Tracing	“Recording the computation”	JAX executes the function with abstract values to build a computation graph, without computing real results
Functional autodiff	“grad of a function”	Computing derivatives by transforming functions, not by attaching gradient storage to tensors
Optax	“JAX’s optimizer library”	A composable library of gradient transformations – Adam, SGD, clipping, scheduling – that chain together
Flax	“JAX’s nn.Module”	Google’s neural network library for JAX, adding layer abstractions while keeping state explicit

Further Reading

- JAX documentation: <https://jax.readthedocs.io/> – the official docs, with excellent tutorials on grad, jit, and vmap
- “JAX: composable transformations of Python+NumPy programs” (Bradbury et al., 2018) – the original paper explaining the design philosophy
- Flax documentation: <https://flax.readthedocs.io/> – Google’s neural network library for JAX
- Patrick Kidger, “Equinox: neural networks in JAX via callable PyTrees and filtered transformations” (2021) – the Pythonic alternative to Flax
- DeepMind, “Optax: composable gradient transformation and optimisation” – the standard optimizer library
- “You Don’t Know JAX” (Colin Raffel, 2020) – a practical guide to JAX gotchas and patterns, from one of the T5 authors

Continue online. The living edition of this chapter has more than the page can hold:

- Animated, interactive figures and the web text: <https://aiengineeringfromscratch.com/lesson?path=phases/03-deep-learning-core/12-intro-to-jax>
- Runnable code for every step: <https://github.com/rohitg00/ai-engineering-from-scratch/tree/main/phases/03-deep-learning-core/12-intro-to-jax/code>

- The chapter quiz, graded in the browser: <https://aiengineeringfromscratch.com/lesson?path=phases/03-deep-learning-core/12-intro-to-jax>

The repository moves faster than any printing. When the book and the repo disagree, trust the repo.

Debugging Neural Networks

Your network compiled. It ran. It produced a number. The number is wrong and nothing crashed. Welcome to the hardest kind of debugging – the kind where there is no error message.

Type: Build **Languages:** Python, PyTorch **Prerequisites:** Phase 03 Lessons 01-10 (especially backpropagation, loss functions, optimizers) **Time:** ~90 minutes

Learning Objectives

- Diagnose common neural network failures (NaN loss, flat loss curve, overfitting, oscillation) using systematic debugging strategies
- Apply the “overfit one batch” technique to verify that your model architecture and training loop are correct
- Inspect gradient magnitudes, activation distributions, and weight norms to identify vanishing/exploding gradient problems
- Build a debugging checklist that covers data pipeline, model architecture, loss function, optimizer, and learning rate issues

The Problem

Traditional software crashes when it is broken. A null pointer throws an exception. A type mismatch fails at compile time. An off-by-one error produces a clearly wrong output.

Neural networks do not give you that luxury.

A broken neural network runs to completion, prints a loss value, and outputs predictions. The loss might decrease. The predictions might look plausible. But the model is silently wrong – learning shortcuts, memorizing noise, or converging to a useless local minimum. Google researchers estimated that 60-70% of ML debugging time is spent on “silent” bugs that produce no errors but degrade model quality.

The difference between a working model and a broken one is often a single misplaced line: a missing `zero_grad()`, a transposed dimension, a learning rate off by 10x. the canonical “Recipe for Training Neural Networks” (2019) opens with this: “The most common neural net mistakes are bugs that don’t crash.”

This lesson teaches you to find those bugs.

The Concept

The Debugging Mindset

Forget print-and-pray debugging. Neural network debugging requires a systematic approach because the feedback loop is slow (minutes to hours per training run) and the symptoms are ambiguous (bad loss could mean 20 different things).

The golden rule: **start simple, add complexity one piece at a time, and verify each piece independently.**

Diagram. Rendered live in the web edition: <https://aiengineeringfromscratch.com/lesson?path=phases/03-deep-learning-core/13-debugging-neural-networks>

Symptom 1: Loss Not Decreasing

This is the most common complaint. The training loop runs, epochs tick by, and the loss stays flat or oscillates wildly.

Wrong learning rate. Too high: loss oscillates or jumps to NaN. Too low: loss decreases so slowly it looks flat. For Adam, start at 1e-3. For SGD, start at 1e-1 or 1e-2. Always try 3 learning rates spanning 10x each (e.g., 1e-2, 1e-3, 1e-4) before concluding something else is wrong.

Dead ReLUs. If a ReLU neuron receives a large negative input, it outputs 0 and its gradient is 0. It never activates again. If enough neurons die, the network cannot learn. Check: print the fraction of activations that are exactly 0 after each ReLU layer. If >50% are dead, switch to LeakyReLU or reduce the learning rate.

Vanishing gradients. In deep networks with sigmoid or tanh activations, gradients shrink exponentially as they propagate backward. By the time they reach the first layer, they are ~ 0 . The first layers stop learning. Fix: use ReLU/GELU, add residual connections, or use batch normalization.

Exploding gradients. The opposite problem – gradients grow exponentially. Common in RNNs and very deep networks. Loss jumps to NaN. Fix: gradient clipping (`torch.nn.utils.clip_grad_norm_`), lower learning rate, or add normalization.

Symptom 2: Loss Decreasing But Model is Bad

The loss goes down. Training accuracy hits 99%. But test accuracy is 55%. Or the model produces nonsensical outputs on real data.

Overfitting. The model memorizes training data instead of learning patterns. Gap between training and validation loss grows over time. Fix: more data, dropout, weight decay, early stopping, data augmentation.

Data leakage. Test data leaked into training. Accuracy is suspiciously high. Common causes: shuffling before splitting, preprocessing with statistics from the full dataset, duplicate samples across splits. Fix: split first, preprocess second, check for duplicates.

Label errors. 5-10% of labels in most real datasets are wrong (Northcutt et al., 2021 – “Pervasive Label Errors in Test Sets”). The model learns the noise. Fix: use confident learning to find and fix mislabeled examples, or use loss truncation to ignore high-loss samples.

Symptom 3: NaN or Inf in Loss

The loss value becomes nan or inf. Training is dead.

Learning rate too high. Gradient updates overshoot so far that weights explode. Fix: reduce by 10x.

log(0) or log(negative). Cross-entropy loss computes $\log(p)$. If your model outputs exactly 0 or a negative probability, the log explodes. Fix: clamp predictions to $[\text{eps}, 1-\text{eps}]$ where $\text{eps}=1e-7$.

Division by zero. Batch normalization divides by standard deviation. A batch with constant values has $\text{std}=0$. Fix: add epsilon to the denominator (PyTorch does this by default, but custom implementations might not).

Numerical overflow. Large activations fed into $\exp()$ produce Inf. Softmax is especially prone. Fix: subtract the max before exponentiating (the log-sum-exp trick).

Technique 1: Gradient Checking

Compare your analytical gradients (from backprop) to numerical gradients (from finite differences). If they disagree, your backward pass has a bug.

Numerical gradient for parameter w :

```
grad_numerical = (loss(w + eps) - loss(w - eps)) / (2 * eps)
```

Agreement metric (relative difference):

```
rel_diff = |grad_analytical - grad_numerical| / max(|grad_analytical|,
↪ |grad_numerical|, 1e-8)
```

If $\text{rel_diff} < 1e-5$: correct. If $\text{rel_diff} > 1e-3$: almost certainly a bug.

Diagram. Rendered live in the web edition: <https://aiengineeringfromscratch.com/lesson?path=phases/03-deep-learning-core/13-debugging-neural-networks>

Technique 2: Activation Statistics

Monitor the mean and standard deviation of activations after each layer during training. Healthy networks maintain activations with mean near 0 and std near 1 (after normalization) or at least bounded.

Health indicator	Mean	Std	Diagnosis
Healthy	~ 0	~ 1	Network is learning normally
Saturated	$\gg 0$ or $\ll 0$	~ 0	Activations stuck at extreme values
Dead	0	0	Neurons are dead (all zeros)
Exploding	$\gg 10$	$\gg 10$	Activations growing without bound

Technique 3: Gradient Flow Visualization

Plot the average gradient magnitude for each layer. In a healthy network, gradient magnitudes should be roughly similar across layers. If early layers have gradients 1000x smaller than later layers, you have vanishing gradients.

Diagram. Rendered live in the web edition: <https://aiengineeringfromscratch.com/lesson?path=phases/03-deep-learning-core/13-debugging-neural-networks>

Diagram. Rendered live in the web edition: <https://aiengineeringfromscratch.com/lesson?path=phases/03-deep-learning-core/13-debugging-neural-networks>

Technique 4: The Overfit-One-Batch Test

The single most important debugging technique in deep learning.

Take one small batch (8-32 samples). Train on it for 100+ iterations. The loss should go to nearly zero and training accuracy should hit 100%. If it does not, your model or training loop has a fundamental bug – do not proceed to full training.

This test catches: - Broken loss functions - Broken backward passes - Architecture too small to represent the data - Optimizer not connected to model parameters - Data and labels mis-aligned

This takes 30 seconds to run and saves hours of debugging full training runs.

Technique 5: Learning Rate Finder

Leslie Smith (2017) proposed sweeping the learning rate from very small ($1e-7$) to very large (10) over one epoch while recording the loss. Plot loss vs learning rate. The optimal learning rate is roughly 10x smaller than the rate where loss starts decreasing fastest.

Diagram. Rendered live in the web edition: <https://aiengineeringfromscratch.com/lesson?path=phases/03-deep-learning-core/13-debugging-neural-networks>

Best LR in this example: $\sim 1e-3$ (one order of magnitude before the steepest point).

Common PyTorch Bugs

These are the bugs that waste the most collective hours in the PyTorch community:

Bug	Symptom	Fix
Forgetting optimizer. <code>zero_grad()</code>	Gradients accumulate across batches, loss oscillates	Add <code>optimizer.zero_grad()</code> before <code>loss.backward()</code>
Forgetting <code>model.eval()</code> at test time	Dropout and batch norm behave differently, test accuracy varies between runs	Add <code>model.eval()</code> and <code>torch.no_grad()</code>
Wrong tensor shapes	Silent broadcasting produces wrong results, no error	Print shapes after every operation during debugging
CPU/GPU mismatch	<code>RuntimeError: expected CUDA tensor</code>	Use <code>.to(device)</code> on model AND data
Not detaching tensors	Computation graph grows forever, OOM	Use <code>.detach()</code> or with <code>torch.no_grad()</code>
In-place operations breaking autograd	<code>RuntimeError: modified by in-place operation</code>	Replace <code>x += 1</code> with <code>x = x + 1</code>
Data not normalized	Loss stuck at random-chance level	Normalize inputs to <code>mean=0, std=1</code>
Labels as wrong dtype	Cross-entropy expects Long, got Float	Cast labels: <code>labels.long()</code>

The Master Debugging Table

Symptom	Likely cause	First thing to try
Loss stuck at $-\log(1/\text{num_classes})$	Model predicting uniform distribution	Check data pipeline, verify labels match inputs
Loss NaN after a few steps	Learning rate too high	Reduce LR by 10x
Loss NaN immediately	$\log(0)$ or division by zero	Add epsilon to log/division operations
Loss oscillating wildly	LR too high or batch size too small	Reduce LR, increase batch size
Loss decreasing then plateaus	LR too high for fine-tuning phase	Add LR schedule (cosine or step decay)
Training acc high, test acc low	Overfitting	Add dropout, weight decay, more data
Training acc = test acc = chance	Model not learning anything	Run overfit-one-batch test
Training acc = test acc but both low	Underfitting	Bigger model, more layers, more features
Gradients all zero	Dead ReLUs or detached computation graph	Switch to LeakyReLU, check <code>.requires_grad</code>
Out of memory during training	Batch too large or graph not freed	Reduce batch size, use <code>torch.no_grad()</code> for eval

Interactive figure: learning-curves. This one moves. Watch it animate and drag its controls in the web edition: <https://aiengineeringfromscratch.com/lesson?path=phases/03-deep-learning-core/13-debugging-neural-networks>

Build It

A diagnostic toolkit that monitors activations, gradients, and loss curves. You will deliberately break a network and use the toolkit to diagnose each problem.

Step 1: The NetworkDebugger Class

Hooks into a PyTorch model to record activation and gradient statistics per layer.

```
import torch
import torch.nn as nn
import math

class NetworkDebugger:
    def __init__(self, model):
        self.model = model
        self.activation_stats = {}
        self.gradient_stats = {}
        self.loss_history = []
        self.lr_losses = []
        self.hooks = []
        self._register_hooks()

    def _register_hooks(self):
```

```

        for name, module in self.model.named_modules():
            if isinstance(module, (nn.Linear, nn.Conv2d, nn.ReLU, nn.LeakyReLU)):
                hook =
    ↪ module.register_forward_hook(self._make_activation_hook(name))
                self.hooks.append(hook)
                hook =
    ↪ module.register_full_backward_hook(self._make_gradient_hook(name))
                self.hooks.append(hook)

    def _make_activation_hook(self, name):
        def hook(module, input, output):
            with torch.no_grad():
                out = output.detach().float()
                self.activation_stats[name] = {
                    "mean": out.mean().item(),
                    "std": out.std().item(),
                    "fraction_zero": (out == 0).float().mean().item(),
                    "min": out.min().item(),
                    "max": out.max().item(),
                }
        return hook

    def _make_gradient_hook(self, name):
        def hook(module, grad_input, grad_output):
            if grad_output[0] is not None:
                with torch.no_grad():
                    grad = grad_output[0].detach().float()
                    self.gradient_stats[name] = {
                        "mean": grad.mean().item(),
                        "std": grad.std().item(),
                        "abs_mean": grad.abs().mean().item(),
                        "max": grad.abs().max().item(),
                    }
        return hook

    def record_loss(self, loss_value):
        self.loss_history.append(loss_value)

    def check_loss_health(self):
        if len(self.loss_history) < 2:
            return "NOT_ENOUGH_DATA"
        recent = self.loss_history[-10:]
        if any(math.isnan(v) or math.isinf(v) for v in recent):
            return "NAN_OR_INF"
        if len(self.loss_history) >= 20:
            first_half = sum(self.loss_history[:10]) / 10
            second_half = sum(self.loss_history[-10:]) / 10
            if second_half >= first_half * 0.99:
                return "NOT_DECREASING"
        if len(recent) >= 5:
            diffs = [recent[i+1] - recent[i] for i in range(len(recent)-1)]
            if max(diffs) - min(diffs) > 2 * abs(sum(diffs) / len(diffs)):
                return "OSCILLATING"
        return "HEALTHY"

```

```

def check_activations(self):
    issues = []
    for name, stats in self.activation_stats.items():
        if stats["fraction_zero"] > 0.5:
            issues.append(f"DEAD_NEURONS: {name} has
↪ {stats['fraction_zero']:.0%} zero activations")
        if abs(stats["mean"]) > 10:
            issues.append(f"EXPLODING_ACTIVATIONS: {name}
↪ mean={stats['mean']:.2f}")
        if stats["std"] < 1e-6:
            issues.append(f"COLLAPSED_ACTIVATIONS: {name}
↪ std={stats['std']:.2e}")
    return issues if issues else ["HEALTHY"]

def check_gradients(self):
    issues = []
    grad_magnitudes = []
    for name, stats in self.gradient_stats.items():
        grad_magnitudes.append((name, stats["abs_mean"]))
        if stats["abs_mean"] < 1e-7:
            issues.append(f"VANISHING_GRADIENT: {name}
↪ abs_mean={stats['abs_mean']:.2e}")
        if stats["abs_mean"] > 100:
            issues.append(f"EXPLODING_GRADIENT: {name}
↪ abs_mean={stats['abs_mean']:.2e}")
        if len(grad_magnitudes) >= 2:
            first_mag = grad_magnitudes[0][1]
            last_mag = grad_magnitudes[-1][1]
            if last_mag > 0 and first_mag / last_mag > 100:
                issues.append(f"GRADIENT_RATIO: first/last =
↪ {first_mag/last_mag:.0f}x (vanishing)")
    return issues if issues else ["HEALTHY"]

def print_report(self):
    print("\n=== NETWORK DEBUGGER REPORT ===")
    print(f"\nLoss health: {self.check_loss_health()}")
    if self.loss_history:
        print(f"  Last 5 losses: {[f'{v:.4f}' for v in
↪ self.loss_history[-5:]]}")
    print("\nActivation diagnostics:")
    for item in self.check_activations():
        print(f"  {item}")
    print("\nGradient diagnostics:")
    for item in self.check_gradients():
        print(f"  {item}")
    print("\nPer-layer activation stats:")
    for name, stats in self.activation_stats.items():
        print(f"  {name}: mean={stats['mean']:.4f} std={stats['std']:.4f}
↪ zero={stats['fraction_zero']:.1%}")
    print("\nPer-layer gradient stats:")
    for name, stats in self.gradient_stats.items():
        print(f"  {name}: abs_mean={stats['abs_mean']:.2e}
↪ max={stats['max']:.2e}")

```



```
def remove_hooks(self):
    for hook in self.hooks:
        hook.remove()
    self.hooks.clear()
```

Step 2: The Overfit-One-Batch Test

```
def overfit_one_batch(model, x_batch, y_batch, criterion, lr=0.01, steps=200):
    optimizer = torch.optim.Adam(model.parameters(), lr=lr)
    model.train()
    print("\n=== OVERFIT ONE BATCH TEST ===")
    print(f"Batch size: {x_batch.shape[0]}, Steps: {steps}")

    for step in range(steps):
        optimizer.zero_grad()
        output = model(x_batch)
        loss = criterion(output, y_batch)
        loss.backward()
        optimizer.step()

        if step % 50 == 0 or step == steps - 1:
            with torch.no_grad():
                preds = (output > 0).float() if output.shape[-1] == 1 else
                ↪ output.argmax(dim=1)
                targets = y_batch if y_batch.dim() == 1 else y_batch.squeeze()
                acc = (preds.squeeze() == targets).float().mean().item()
                print(f" Step {step:3d} | Loss: {loss.item():.6f} | Accuracy:
                ↪ {acc:.1%}")

    final_loss = loss.item()
    if final_loss > 0.1:
        print(f"\n FAIL: Loss did not converge ({final_loss:.4f}). Model or
        ↪ training loop is broken.")
        return False
    print(f"\n PASS: Loss converged to {final_loss:.6f}")
    return True
```

Step 3: Learning Rate Finder

```
def find_learning_rate(model, x_data, y_data, criterion, start_lr=1e-7, end_lr=10,
    ↪ steps=100):
    import copy
    original_state = copy.deepcopy(model.state_dict())
    optimizer = torch.optim.SGD(model.parameters(), lr=start_lr)
    lr_mult = (end_lr / start_lr) ** (1 / steps)

    model.train()
    results = []
    best_loss = float("inf")
    current_lr = start_lr

    print("\n=== LEARNING RATE FINDER ===")
```

```

for step in range(steps):
    optimizer.zero_grad()
    output = model(x_data)
    loss = criterion(output, y_data)

    if math.isnan(loss.item()) or loss.item() > best_loss * 10:
        break

    best_loss = min(best_loss, loss.item())
    results.append((current_lr, loss.item()))

    loss.backward()
    optimizer.step()

    current_lr *= lr_mult
    for param_group in optimizer.param_groups:
        param_group["lr"] = current_lr

model.load_state_dict(original_state)

if len(results) < 10:
    print(" Could not complete LR sweep -- loss diverged too quickly")
    return results

min_loss_idx = min(range(len(results)), key=lambda i: results[i][1])
suggested_lr = results[max(0, min_loss_idx - 10)][0]

print(f" Swept {len(results)} steps from {start_lr:.0e} to
    ↪ {results[-1][0]:.0e}")
print(f" Minimum loss {results[min_loss_idx][1]:.4f} at
    ↪ lr={results[min_loss_idx][0]:.2e}")
print(f" Suggested learning rate: {suggested_lr:.2e}")

return results

```

Step 4: Gradient Checker

```

def _flat_to_multi_index(flat_idx, shape):
    multi_idx = []
    remaining = flat_idx
    for dim in reversed(shape):
        multi_idx.insert(0, remaining % dim)
        remaining //= dim
    return tuple(multi_idx)

def gradient_check(model, x, y, criterion, eps=1e-4):
    model.train()
    x_double = x.double()
    y_double = y.double()
    model_double = model.double()

    print("\n=== GRADIENT CHECK ===")

```

```

overall_max_diff = 0
checked = 0

for name, param in model_double.named_parameters():
    if not param.requires_grad:
        continue

    layer_max_diff = 0

    model_double.zero_grad()
    output = model_double(x_double)
    loss = criterion(output, y_double)
    loss.backward()
    analytical_grad = param.grad.clone()

    num_checks = min(5, param.numel())
    for i in range(num_checks):
        idx = _flat_to_multi_index(i, param.shape)
        original = param.data[idx].item()

        param.data[idx] = original + eps
        with torch.no_grad():
            loss_plus = criterion(model_double(x_double), y_double).item()

        param.data[idx] = original - eps
        with torch.no_grad():
            loss_minus = criterion(model_double(x_double), y_double).item()

        param.data[idx] = original

        numerical = (loss_plus - loss_minus) / (2 * eps)
        analytical = analytical_grad[idx].item()

        denom = max(abs(numerical), abs(analytical), 1e-8)
        rel_diff = abs(numerical - analytical) / denom

        layer_max_diff = max(layer_max_diff, rel_diff)
        checked += 1

    overall_max_diff = max(overall_max_diff, layer_max_diff)
    status = "OK" if layer_max_diff < 1e-5 else "MISMATCH"
    print(f" {name}: max_rel_diff={layer_max_diff:.2e} [{status}]")

model.float()

print(f"\n Checked {checked} parameters")
if overall_max_diff < 1e-5:
    print(" PASS: Gradients match (rel_diff < 1e-5)")
elif overall_max_diff < 1e-3:
    print(" WARN: Small differences (1e-5 < rel_diff < 1e-3)")
else:
    print(" FAIL: Gradient mismatch detected (rel_diff > 1e-3)")
return overall_max_diff

```

Step 5: Deliberately Broken Networks

Now apply the toolkit to broken networks and diagnose each one.

```
def demo_broken_networks():
    torch.manual_seed(42)
    x = torch.randn(64, 10)
    y = (x[:, 0] > 0).long()

    print("\n" + "=" * 60)
    print("BUG 1: Learning rate too high (lr=10)")
    print("=" * 60)
    model1 = nn.Sequential(nn.Linear(10, 32), nn.ReLU(), nn.Linear(32, 2))
    debugger1 = NetworkDebugger(model1)
    optimizer1 = torch.optim.SGD(model1.parameters(), lr=10.0)
    criterion = nn.CrossEntropyLoss()
    for step in range(20):
        optimizer1.zero_grad()
        out = model1(x)
        loss = criterion(out, y)
        debugger1.record_loss(loss.item())
        loss.backward()
        optimizer1.step()
    debugger1.print_report()
    debugger1.remove_hooks()

    print("\n" + "=" * 60)
    print("BUG 2: Dead ReLUs from bad initialization")
    print("=" * 60)
    model2 = nn.Sequential(nn.Linear(10, 32), nn.ReLU(), nn.Linear(32, 32),
    ↪ nn.ReLU(), nn.Linear(32, 2))
    with torch.no_grad():
        for m in model2.modules():
            if isinstance(m, nn.Linear):
                m.weight.fill_(-1.0)
                m.bias.fill_(-5.0)
    debugger2 = NetworkDebugger(model2)
    optimizer2 = torch.optim.Adam(model2.parameters(), lr=1e-3)
    for step in range(50):
        optimizer2.zero_grad()
        out = model2(x)
        loss = criterion(out, y)
        debugger2.record_loss(loss.item())
        loss.backward()
        optimizer2.step()
    debugger2.print_report()
    debugger2.remove_hooks()

    print("\n" + "=" * 60)
    print("BUG 3: Missing zero_grad (gradients accumulate)")
    print("=" * 60)
    model3 = nn.Sequential(nn.Linear(10, 32), nn.ReLU(), nn.Linear(32, 2))
    debugger3 = NetworkDebugger(model3)
    optimizer3 = torch.optim.SGD(model3.parameters(), lr=0.01)
    for step in range(50):
```

```

        out = model3(x)
        loss = criterion(out, y)
        debugger3.record_loss(loss.item())
        loss.backward()
        optimizer3.step()
    debugger3.print_report()
    debugger3.remove_hooks()

    print("\n" + "=" * 60)
    print("HEALTHY NETWORK: Correct setup for comparison")
    print("=" * 60)
    model_good = nn.Sequential(nn.Linear(10, 32), nn.ReLU(), nn.Linear(32, 2))
    debugger_good = NetworkDebugger(model_good)
    optimizer_good = torch.optim.Adam(model_good.parameters(), lr=1e-3)
    for step in range(50):
        optimizer_good.zero_grad()
        out = model_good(x)
        loss = criterion(out, y)
        debugger_good.record_loss(loss.item())
        loss.backward()
        optimizer_good.step()
    debugger_good.print_report()
    debugger_good.remove_hooks()

    print("\n" + "=" * 60)
    print("OVERFIT-ONE-BATCH TEST (healthy model)")
    print("=" * 60)
    model_test = nn.Sequential(nn.Linear(10, 32), nn.ReLU(), nn.Linear(32, 2))
    overfit_one_batch(model_test, x[:8], y[:8], criterion)

    print("\n" + "=" * 60)
    print("LEARNING RATE FINDER")
    print("=" * 60)
    model_lr = nn.Sequential(nn.Linear(10, 32), nn.ReLU(), nn.Linear(32, 2))
    find_learning_rate(model_lr, x, y, criterion)

    print("\n" + "=" * 60)
    print("GRADIENT CHECK")
    print("=" * 60)
    model_grad = nn.Sequential(nn.Linear(10, 8), nn.ReLU(), nn.Linear(8, 2))
    gradient_check(model_grad, x[:4], y[:4], criterion)

```

Use It

PyTorch Built-in Tools

```

import torch
import torch.nn as nn

model = nn.Sequential(
    nn.Linear(768, 256),
    nn.ReLU(),
    nn.Linear(256, 10),

```

```

)

with torch.autograd.detect_anomaly():
    output = model(input_tensor)
    loss = criterion(output, target)
    loss.backward()

for name, param in model.named_parameters():
    if param.grad is not None:
        print(f"{name}: grad_mean={param.grad.abs().mean():.2e}")

```

Weights & Biases Integration

```

import wandb

wandb.init(project="debug-training")

for epoch in range(100):
    loss = train_one_epoch()
    wandb.log({
        "loss": loss,
        "lr": optimizer.param_groups[0]["lr"],
        "grad_norm": torch.nn.utils.clip_grad_norm_(model.parameters(),
            ↪ float("inf")),
    })

    for name, param in model.named_parameters():
        if param.grad is not None:
            wandb.log({f"grad/{name}": wandb.Histogram(param.grad.cpu().numpy())})

```

TensorBoard

```

from torch.utils.tensorboard import SummaryWriter

writer = SummaryWriter("runs/debug_experiment")

for epoch in range(100):
    loss = train_one_epoch()
    writer.add_scalar("Loss/train", loss, epoch)

    for name, param in model.named_parameters():
        writer.add_histogram(f"weights/{name}", param, epoch)
        if param.grad is not None:
            writer.add_histogram(f"gradients/{name}", param.grad, epoch)

```

The Debug Checklist (Before Full Training)

1. Run overfit-one-batch test. If it fails, stop.
2. Print model summary – verify parameter count is reasonable.
3. Run a single forward pass with random data – check output shape.
4. Train for 5 epochs – verify loss decreases.
5. Check activation statistics – no dead layers, no explosions.
6. Check gradient flow – no vanishing, no exploding.

7. Verify data pipeline - print 5 random samples with labels.

This chapter ships an artifact. The course version of this lesson produces a reusable prompt or agent skill. It lives in the repository, ready to install: <https://github.com/rohitg00/ai-engineering-from-scratch/tree/main/phases/03-deep-learning-core/13-debugging-neural-networks>

Exercises

Starter code and the lesson's working implementation: <https://github.com/rohitg00/ai-engineering-from-scratch/tree/main/phases/03-deep-learning-core/13-debugging-neural-networks/code>

1. **Add an exploding gradient detector.** Modify the NetworkDebugger to detect when gradients exceed a threshold and automatically suggest a gradient clipping value. Test it on a 20-layer network with no normalization.
2. **Build a dead neuron resurrector.** Write a function that identifies dead ReLU neurons (always outputting 0) and reinitializes their incoming weights with Kaiming initialization. Show that this recovers a network where >70% of neurons are dead.
3. **Implement the learning rate finder with plotting.** Extend `find_learning_rate` to save results as a CSV and write a separate script that reads the CSV and displays the LR vs loss curve using matplotlib. Identify the optimal LR for ResNet-18 on CIFAR-10.
4. **Create a data pipeline validator.** Write a function that checks for: duplicate samples across train/test splits, label distribution imbalance (>10:1 ratio), input normalization (mean near 0, std near 1), and NaN/Inf values in the data. Run it on a deliberately corrupted dataset.
5. **Debug a real failure.** Take the mini-framework from Lesson 10, introduce a subtle bug (e.g., transpose the weight matrix in backward), and use gradient checking to locate exactly which parameter has incorrect gradients. Document the debugging process.

Key Terms

Term	What people say	What it actually means
Silent bug	"It runs but gives bad results"	A bug that produces no error but degrades model quality - the dominant failure mode in ML
Dead ReLU	"The neurons died"	A ReLU neuron whose input is always negative, so it outputs 0 and receives 0 gradient permanently
Vanishing gradients	"Early layers stop learning"	Gradients shrink exponentially through layers, making weights in early layers effectively frozen
Exploding gradients	"Loss went to NaN"	Gradients grow exponentially through layers, causing weight updates so large they overflow
Gradient checking	"Verify backprop is correct"	Comparing analytical gradients from backprop to numerical gradients from finite differences

Term	What people say	What it actually means
Overfit-one-batch	“The most important debug test”	Training on a single small batch to verify the model CAN learn – if it cannot, something is fundamentally broken
LR finder	“Sweep to find the right learning rate”	Exponentially increasing the learning rate over one epoch and picking the rate just before loss diverges
Data leakage	“Test data leaked into training”	When information from the test set contaminates training, producing artificially high accuracy
Activation statistics	“Monitor layer health”	Tracking mean, std, and zero-fraction of each layer’s output to detect dead, saturated, or exploding neurons
Gradient clipping	“Cap the gradient magnitude”	Scaling gradients down when their norm exceeds a threshold, preventing exploding gradient updates

Further Reading

- Smith, “Cyclical Learning Rates for Training Neural Networks” (2017) – the paper introducing the learning rate range test (LR finder)
- Northcutt et al., “Pervasive Label Errors in Test Sets Destabilize Machine Learning Benchmarks” (2021) – demonstrates that 3-6% of labels in ImageNet, CIFAR-10, and other major benchmarks are wrong
- Zhang et al., “Understanding Deep Learning Requires Rethinking Generalization” (2017) – the paper showing neural networks can memorize random labels, which is why the overfit-one-batch test works
- PyTorch documentation on `torch.autograd.detect_anomaly` and `torch.autograd.set_detect_anomaly` for built-in NaN/Inf detection

Continue online. The living edition of this chapter has more than the page can hold:

- Animated, interactive figures and the web text: <https://aiengineeringfromscratch.com/lesson?path=phases/03-deep-learning-core/13-debugging-neural-networks>
- Runnable code for every step: <https://github.com/rohitg00/ai-engineering-from-scratch/tree/main/phases/03-deep-learning-core/13-debugging-neural-networks/code>
- The chapter quiz, graded in the browser: <https://aiengineeringfromscratch.com/lesson?path=phases/03-deep-learning-core/13-debugging-neural-networks>

The repository moves faster than any printing. When the book and the repo disagree, trust the repo.

Part II — Computer Vision

Course phase 04. Live edition with animated figures and quizzes: <https://aiengineeringfromscratch.com/catalog.html>

Image Fundamentals — Pixels, Channels, Color Spaces

An image is a tensor of light samples. Every vision model you will ever use starts from this one fact.

Type: Build **Languages:** Python **Prerequisites:** Phase 1 Lesson 12 (Tensor Operations), Phase 3 Lesson 11 (Intro to PyTorch) **Time:** ~45 minutes

Learning Objectives

- Explain how a continuous scene gets discretized into pixels and why sampling/quantization decisions set the ceiling on every downstream model
- Read, slice, and inspect images as NumPy arrays and switch fluently between HWC and CHW layouts
- Convert between RGB, grayscale, HSV, and YCbCr and justify why each color space exists
- Apply pixel-level preprocessing (normalize, standardize, resize, channel-first) exactly as pretrained PyTorch vision models expect it

The Problem

Every paper you will read, every pretrained weight you will download, every vision API you will call assumes a specific encoding of the input. Pass a uint8 image where the model wants float32 and it will still run — and silently produce garbage. Feed BGR to a network trained on RGB and accuracy collapses by ten points. Hand a model channels-last input when it expects channels-first and the first conv layer treats height as a feature channel. None of this throws an error. It just ruins your metrics and you spend a week hunting for a bug that lives in how you loaded the file.

A convolution is not complicated once you know what it is sliding over. The hard part is that “an image” means different things to a camera, a JPEG decoder, PIL, OpenCV, torchvision, and a CUDA kernel. Each stack has its own axis order, byte range, and channel convention. A vision engineer who cannot keep these straight ships broken pipelines.

This lesson fixes the foundation so the rest of the phase can build on it. By the end you will know what a pixel is, why there are three numbers per pixel instead of one, what “normalize with ImageNet stats” actually does, and how to move between the two or three layouts that every other lesson in this phase will assume.

The Concept

The full preprocessing pipeline at a glance

Every production vision system is the same sequence of reversible transforms. Get one step wrong and the model sees a different input than it was trained on.

Diagram. Rendered live in the web edition: <https://aiengineeringfromscratch.com/lesson?path=phases/04-computer-vision/01-image-fundamentals>

The two red and blue boxes are where 80% of silent failures live: missing standardization and wrong layout.

A pixel is a sample, not a square

A camera sensor counts photons that land on a grid of tiny detectors. Each detector integrates light for a fraction of a second and emits a voltage proportional to how many photons hit it. The sensor then discretizes that voltage into an integer. One detector becomes one pixel.

Continuous scene (infinite detail)	Sensor grid (H x W detectors)	Digital image (H x W integers)
~~~~~ ~ ~ ~ ~ light ~ ~~~~~	<pre> +---+---+---+---+                   +---+---+---+---+                   +---+---+---+---+ </pre>	<pre> 210 198 180 155 120 205 195 178 152 118 200 190 175 150 115 195 185 170 148 112 188 180 165 145 108 </pre>

Two choices happen at this step and they fix the ceiling on everything downstream:

- **Spatial sampling** decides how many detectors per degree of the scene. Too few, and edges become jagged (aliasing). Too many, and storage and compute explode.
- **Intensity quantization** decides how finely the voltage is bucketed. 8 bits gives 256 levels and is standard for display. 10, 12, 16 bits give smoother gradients and matter for medical imaging, HDR, and raw sensor pipelines.

A pixel is not a coloured square with area. It is a single measurement. When you resize or rotate, you are resampling that measurement grid.

## Why three channels

One detector counts photons across the whole visible spectrum — that is grayscale. To get colour, the sensor covers the grid with a mosaic of red, green, and blue filters. After demosaicing, every spatial location has three integers: the response of the red-filtered detector, green-filtered, and blue-filtered nearby. Those three integers are a pixel's RGB triplet.

One pixel in memory:

```
(R, G, B) = (210, 140, 30)  <- reddish-orange
```

An H x W RGB image:

```
shape (H, W, 3)    stored as    H rows of W pixels of 3 values
                        each in [0, 255] for uint8
```

Three is not magic. Depth cameras add a Z channel. Satellites add infrared and ultraviolet bands. Medical scans often have one channel (X-ray, CT) or many (hyperspectral). The number of channels is the last axis; conv layers learn to mix across it.

## Two layout conventions: HWC and CHW

Same tensor, two orderings. Every library picks one.

HWC (height, width, channels)

```

W ->
+-----+-----+-----+
H |R G B|R G B|R G B|
| +-----+-----+-----+
v |R G B|R G B|R G B|
+-----+-----+-----+

```

CHW (channels, height, width)

```

H ->
+-----+-----+
C |R R R R R R|
| +-----+-----+
v |G G G G G G|
+-----+-----+
  |B B B B B B|
+-----+-----+

```

PIL, OpenCV, matplotlib,  
almost every image file on disk

PyTorch, most deep learning  
frameworks, cuDNN kernels

CHW exists because convolution kernels slide across H and W. Keeping the channel axis first means each kernel sees a contiguous 2D plane per channel, which vectorizes cleanly. Disk formats keep HWC because that matches how scanlines come out of a sensor.

The one-line conversion you will type a thousand times:

```

img_chw = img_hwc.transpose(2, 0, 1)    # NumPy
img_chw = img_hwc.permute(2, 0, 1)      # PyTorch tensor

```

Memory layout, visualised:

**Diagram.** Rendered live in the web edition: <https://aiengineeringfromscratch.com/lesson?path=phases/04-computer-vision/01-image-fundamentals>

## Byte ranges and dtype

Three conventions dominate:

Convention	dtype	Range	Where you see it
Raw	uint8	[0, 255]	Files on disk, PIL, OpenCV output
Normalized	float32	[0.0, 1.0]	After <code>img.astype('float32') / 255</code>
Standardized	float32	roughly [-2, +2]	After subtracting mean and dividing by std

Convolutional networks were trained on standardized inputs. ImageNet stats mean=[0.485, 0.456, 0.406], std=[0.229, 0.224, 0.225] are the arithmetic mean and standard deviation of the three channels over the full ImageNet training set, computed on [0, 1] normalized pixels. Feeding raw uint8 into a model that expects standardized float is the single most common silent failure in applied vision.

## Color spaces and why they exist

RGB is the capture format but it is not always the most useful representation for a model.

RGB	HSV	YCbCr / YUV
R red	H hue (angle 0-360)	Y luminance (brightness)
G green	S saturation (0-1)	Cb chroma blue-yellow
B blue	V value/brightness (0-1)	Cr chroma red-green

Linear to sensor output	Separates color from brightness. Useful for color thresholding, UI sliders, simple filters	Separates brightness from color. JPEG and most video codecs compress the chroma channels harder because the human eye is less sensitive to chroma detail than to Y.
-------------------------	--------------------------------------------------------------------------------------------	---------------------------------------------------------------------------------------------------------------------------------------------------------------------

For most modern CNNs you feed RGB. You meet other spaces when:

- **HSV** — classical CV code, color-based segmentation, white-balancing.
- **YCbCr** — reading JPEG internals, video pipelines, super-resolution models that operate on Y only.
- **Grayscale** — OCR, document models, any case where color is nuisance variable rather than signal.

Grayscale from RGB is a weighted sum, not an average, because the human eye is more sensitive to green than to red or blue:

$$Y = 0.299 R + 0.587 G + 0.114 B \quad (\text{ITU-R BT.601, the classic weights})$$

## Aspect ratio, resizing, and interpolation

Every model has a fixed input size (224x224 for most ImageNet classifiers, 384x384 or 512x512 for modern detectors). Your images rarely match. The three resize choices that matter:

- **Resize shorter side, then center crop** — the standard ImageNet recipe. Preserves aspect ratio, throws away a strip of edge pixels.
- **Resize and pad** — preserves aspect ratio and every pixel, adds black bars. Standard for detection and OCR.
- **Resize directly to target** — stretches the image. Cheap, distorts geometry, fine for many classification tasks.

The interpolation method decides how intermediate pixels are computed when the new grid does not align with the old one:

Nearest neighbour	fastest, blocky, only choice for masks/labels
Bilinear	fast, smooth, default for most image resizing
Bicubic	slower, sharper on upscaling
Lanczos	slowest, best quality, used for final display

Rule of thumb: bilinear for training, bicubic or lanczos for assets you will look at, nearest for anything containing integer class IDs.

**Interactive figure: conv-output-size.** This one moves. Watch it animate and drag its controls in the web edition: <https://aiengineeringfromscratch.com/lesson?path=phases/04-computer-vision/01-image-fundamentals>

## Build It

### Step 1: Build an image tensor and inspect its shape

Start with a deterministic synthetic image so the first lab runs offline with only NumPy. File decoding is a separate boundary: once a JPEG or PNG decoder returns RGB bytes, every tensor operation below is the same.

```
import numpy as np
```

```
def synthetic_rgb(h=128, w=192, seed=0):
    rng = np.random.default_rng(seed)
    yy, xx = np.meshgrid(np.linspace(0, 1, h), np.linspace(0, 1, w), indexing="ij")
    r = (np.sin(xx * 6) * 0.5 + 0.5) * 255
    g = yy * 255
    b = (1 - yy) * xx * 255
    rgb = np.stack([r, g, b], axis=-1) + rng.normal(0, 6, (h, w, 3))
    return np.clip(rgb, 0, 255).astype(np.uint8)
```

```
arr = synthetic_rgb()
```

```
print(f"type: {type(arr).__name__}")
print(f"dtype: {arr.dtype}")
print(f"shape: {arr.shape}      # (H, W, C)")
print(f"min: {arr.min()}")
print(f"max: {arr.max()}")
print(f"pixel at (0, 0): {arr[0, 0]}")
```

Expected output: shape: (H, W, 3), dtype: uint8, range [0, 255]. That is the canonical decoded representation whether the bytes came from a camera, an image decoder, or this synthetic generator.

## Step 2: Split channels and re-order layout

Pull out R, G, B separately, then convert from HWC to CHW for PyTorch.

```
R = arr[:, :, 0]
G = arr[:, :, 1]
B = arr[:, :, 2]
print(f"R shape: {R.shape}, mean: {R.mean():.1f}")
print(f"G shape: {G.shape}, mean: {G.mean():.1f}")
print(f"B shape: {B.shape}, mean: {B.mean():.1f}")
```

```
arr_chw = arr.transpose(2, 0, 1)
print(f"\nHWC shape: {arr.shape}")
print(f"CHW shape: {arr_chw.shape}")
```

Three grayscale planes, one per channel. CHW just reorders the axes; no data copy is strictly required when the memory layout allows it.

## Step 3: Grayscale and HSV conversions

Weighted-sum grayscale, then a manual RGB-to-HSV.

```
def rgb_to_grayscale(rgb):
    weights = np.array([0.299, 0.587, 0.114], dtype=np.float32)
    return (rgb.astype(np.float32) @ weights).astype(np.uint8)
```

```
def rgb_to_hsv(rgb):
    rgb_f = rgb.astype(np.float32) / 255.0
    r, g, b = rgb_f[..., 0], rgb_f[..., 1], rgb_f[..., 2]
    cmax = np.max(rgb_f, axis=-1)
    cmin = np.min(rgb_f, axis=-1)
    delta = cmax - cmin

    h = np.zeros_like(cmax)
```

```

mask = delta > 0
argmax = np.argmax(rgb_f, axis=-1)
rmax = mask & (argmax == 0)
gmax = mask & (argmax == 1)
bmax = mask & (argmax == 2)
h[rmax] = ((g[rmax] - b[rmax]) / delta[rmax]) % 6
h[gmax] = ((b[gmax] - r[gmax]) / delta[gmax]) + 2
h[bmax] = ((r[bmax] - g[bmax]) / delta[bmax]) + 4
h = h * 60.0

s = np.divide(delta, cmax, out=np.zeros_like(delta), where=cmax > 0)
v = cmax
return np.stack([h, s, v], axis=-1)

gray = rgb_to_grayscale(arr)
hsv = rgb_to_hsv(arr)
print(f"gray shape: {gray.shape}, range: [{gray.min()}, {gray.max()}]")
print(f"hsv shape: {hsv.shape}")
print(f"hue range: [{hsv[..., 0].min():.1f}, {hsv[..., 0].max():.1f}] degrees")
print(f"sat range: [{hsv[..., 1].min():.2f}, {hsv[..., 1].max():.2f}]")
print(f"val range: [{hsv[..., 2].min():.2f}, {hsv[..., 2].max():.2f}]")

```

Hue comes out in degrees, saturation and value in [0, 1]. That matches the OpenCV `hsv_full` convention.

#### Step 4: Normalize, standardize, and reverse it

Go from raw bytes to the exact tensor a pretrained ImageNet model expects, then back.

```

mean = np.array([0.485, 0.456, 0.406], dtype=np.float32)
std = np.array([0.229, 0.224, 0.225], dtype=np.float32)

def preprocess_imagenet(rgb_uint8):
    x = rgb_uint8.astype(np.float32) / 255.0
    x = (x - mean) / std
    x = x.transpose(2, 0, 1)
    return x

def deprocess_imagenet(chw_float32):
    x = chw_float32.transpose(1, 2, 0)
    x = x * std + mean
    x = np.clip(x * 255.0, 0, 255).astype(np.uint8)
    return x

x = preprocess_imagenet(arr)
print(f"preprocessed shape: {x.shape}      # (C, H, W)")
print(f"preprocessed dtype: {x.dtype}")
print(f"preprocessed mean per channel: {x.mean(axis=(1, 2)).round(3)}")
print(f"preprocessed std per channel: {x.std(axis=(1, 2)).round(3)}")

roundtrip = deprocess_imagenet(x)
max_diff = np.abs(roundtrip.astype(int) - arr.astype(int)).max()
print(f"roundtrip max pixel diff: {max_diff}      # should be 0 or 1")

```

Per-channel mean should be close to zero, std close to one. The preprocess/deprocess pair is

exactly what every torchvision.transforms.Normalize call is doing under the hood.

## Step 5: Resize from scratch

Nearest neighbor rounds each output coordinate to one source pixel. Bilinear interpolation finds the four surrounding pixels and blends them by distance. Both implementations below use endpoint-aligned coordinates so the first and last source pixels stay fixed.

```
def resize_coordinates(source_length, target_length):
    if target_length == 1:
        return np.zeros(1, dtype=np.float32)
    return np.linspace(0, source_length - 1, target_length, dtype=np.float32)

def nearest_resize(image, target_height, target_width):
    y = np rint(resize_coordinates(image.shape[0], target_height)).astype(int)
    x = np rint(resize_coordinates(image.shape[1], target_width)).astype(int)
    return image[y[:, None], x[None, :]]

def bilinear_resize(image, target_height, target_width):
    y = resize_coordinates(image.shape[0], target_height)
    x = resize_coordinates(image.shape[1], target_width)
    y0 = np.floor(y).astype(int)
    x0 = np.floor(x).astype(int)
    y1 = np.minimum(y0 + 1, image.shape[0] - 1)
    x1 = np.minimum(x0 + 1, image.shape[1] - 1)
    wy = (y - y0)[:, None, None]
    wx = (x - x0)[None, :, None]

    source = image.astype(np.float32)
    top = source[y0[:, None], x0[None, :]] * (1 - wx)
    top += source[y0[:, None], x1[None, :]] * wx
    bottom = source[y1[:, None], x0[None, :]] * (1 - wx)
    bottom += source[y1[:, None], x1[None, :]] * wx
    result = top * (1 - wy) + bottom * wy
    return np.clip(np rint(result), 0, 255).astype(image.dtype)

target_height = arr.shape[0] * 3
target_width = arr.shape[1] * 3
nearest = nearest_resize(arr, target_height, target_width)
bilinear = bilinear_resize(arr, target_height, target_width)

def local_roughness(x):
    gy = np.diff(x.astype(float), axis=0)
    gx = np.diff(x.astype(float), axis=1)
    return float(np.abs(gy).mean() + np.abs(gx).mean())

for name, out in [("nearest", nearest), ("bilinear", bilinear)]:
    print(f"{name:>8}  shape={out.shape}  roughness={local_roughness(out):6.2f}")
```

Nearest scores highest on roughness because it keeps hard edges. Bilinear is smoother because every new pixel blends two positions on each axis. The runnable companion extends the same separable idea to four neighbors per axis with a Catmull-Rom cubic kernel, then prints all three results without an image library.



## Use It

PyTorch performs the same operations on batched, device-aware tensors. The code below resizes the shorter side, takes a center crop, standardizes each channel, and produces the NCHW tensor a pretrained model expects.

```
import torch
import torch.nn.functional as F

image_hwc = torch.from_numpy(synthetic_rgb(256, 320))
batch = image_hwc.permute(2, 0, 1).unsqueeze(0).float() / 255.0

height, width = batch.shape[-2:]
scale = 256 / min(height, width)
resized_height = round(height * scale)
resized_width = round(width * scale)
batch = F.interpolate(
    batch,
    size=(resized_height, resized_width),
    mode="bilinear",
    align_corners=False,
    antialias=True,
)

top = (resized_height - 224) // 2
left = (resized_width - 224) // 2
batch = batch[:, :, top:top + 224, left:left + 224]

mean = torch.tensor([0.485, 0.456, 0.406]).view(1, 3, 1, 1)
std = torch.tensor([0.229, 0.224, 0.225]).view(1, 3, 1, 1)
batch = (batch - mean) / std

print(f"tensor dtype: {batch.dtype}")
print(f"batched shape: {tuple(batch.shape)}")
print(f"per-channel mean: {batch.mean(dim=(0, 2, 3)).tolist()}")
print(f"per-channel std: {batch.std(dim=(0, 2, 3)).tolist()}")
```

Four steps, in this exact order: convert bytes to float and swap HWC to NCHW, resize the shorter side to 256, take a 224x224 center crop, then subtract the ImageNet mean and divide by its standard deviation. Reversing that order silently changes what reaches the model.

**This chapter ships an artifact.** The course version of this lesson produces a reusable prompt or agent skill. It lives in the repository, ready to install: <https://github.com/rohitg00/ai-engineering-from-scratch/tree/main/phases/04-computer-vision/01-image-fundamentals>

## Exercises

Starter code and the lesson's working implementation: <https://github.com/rohitg00/ai-engineering-from-scratch/tree/main/phases/04-computer-vision/01-image-fundamentals/code>

1. **(Easy)** Create a 2x2 RGB uint8 array with four distinct colors. Convert HWC to CHW and back, print both shapes, and prove the round trip preserves every value.
2. **(Medium)** Write `standardize(img, mean, std)` and its inverse that together pass a `roundtrip_max_diff <= 1` test on any uint8 image. Your functions must work on a single

image in HWC and on a batch in NCHW with the same call.

3. **(Hard)** Take a 3-channel ImageNet-standardized tensor and run it through a 1x1 conv that learns a weighted mixture of RGB into a single grayscale channel. Initialize the weights to  $[0.299, 0.587, 0.114]$ , freeze them, and verify the output matches your manual `rgb_to_grayscale` to within floating-point error. What other classical color-space transforms can be written as 1x1 convolutions?

## Key Terms

Term	What people say	What it actually means
Pixel	“A coloured square”	One sample of light intensity at one grid location — three numbers for colour, one for grayscale
Channel	“The colour”	One of the parallel spatial grids stacked into an image tensor; last axis in HWC, first in CHW
HWC / CHW	“The shape”	Axis orderings for an image tensor; disk and PIL use HWC, PyTorch and cuDNN use CHW
Normalize	“Scale the image”	Divide by 255 so pixels live in $[0, 1]$ — necessary but not sufficient
Standardize	“Zero-center”	Subtract mean and divide by std per channel so the input distribution matches what the model was trained on
Grayscale conversion	“Average the channels”	A weighted sum with coefficients $0.299/0.587/0.114$ that matches human luminance perception
Interpolation	“How resize picks pixels”	The rule that decides output values when the new grid does not align with the old one — nearest for labels, bilinear for training, bicubic for display
Aspect ratio	“Width over height”	The ratio that distinguishes “resize and pad” from “resize and stretch”

## Further Reading

- Charles Poynton — A Guided Tour of Color Space — the clearest technical treatment of why there are so many color spaces and when each one matters
- PyTorch Vision Transforms Docs — the full pipeline of transforms you will actually compose in production
- How JPEG Works (Colt McAnlis) — a sharp visual tour of chroma subsampling, DCT, and why JPEG encodes YCbCr rather than RGB
- ImageNet Preprocessing Conventions (torchvision models) — the source of truth for `mean=[0.485, 0.456, 0.406]` and why every model in the zoo expects it

**Continue online.** The living edition of this chapter has more than the page can hold:

- Animated, interactive figures and the web text: <https://aiengineeringfromscratch.com/lesson?path=phases/04-computer-vision/01-image-fundamentals>
- Runnable code for every step: <https://github.com/rohitg00/ai-engineering-from-scratch/tree/main/phases/04-computer-vision/01-image-fundamentals/code>

- The chapter quiz, graded in the browser: <https://aiengineeringfromscratch.com/lesson?path=phases/04-computer-vision/01-image-fundamentals>

The repository moves faster than any printing. When the book and the repo disagree, trust the repo.

# Convolutions from Scratch

---

A convolution is a tiny dense layer you slide across an image, sharing the same weights at every location.

**Type:** Build **Languages:** Python **Prerequisites:** Phase 3 (Deep Learning Core), Phase 4 Lesson 01 (Image Fundamentals) **Time:** ~75 minutes

## Learning Objectives

- Implement 2D convolution from scratch using only NumPy, including the nested-loop version and a vectorised `im2col` version
- Compute output spatial size for any combination of input size, kernel size, padding, and stride, and justify the  $(H - K + 2P) / S + 1$  formula
- Hand-design kernels (edge, blur, sharpen, Sobel) and explain why each one produces the pattern of activations it does
- Stack convolutions into a feature extractor and connect the depth-of-the-stack to the size of the receptive field

## The Problem

A fully connected layer on a 224x224 RGB image would need  $224 * 224 * 3 = 150,528$  input weights per neuron. A single hidden layer with 1,000 units is already 150 million parameters — before you have learnt anything useful. Worse, that layer has no notion that a dog in the top-left and a dog in the bottom-right are the same pattern. It treats every pixel position as independent, which is exactly wrong for images: translating a cat by three pixels should not force the network to relearn the concept.

The two properties an image model needs are **translation equivariance** (the output shifts when the input shifts) and **parameter sharing** (the same feature detector runs everywhere). Dense layers give you neither. Convolution gives you both for free.

Convolution was not invented for deep learning. It is the same operation that powers JPEG compression, Gaussian blur in Photoshop, edge detection in industrial vision, and every audio filter ever shipped. The reason CNNs dominated ImageNet from 2012 to 2020 is that convolution is the correct prior for data where nearby values are related and the same pattern can appear anywhere.

## The Concept

### One kernel, sliding

A 2D convolution takes a small weight matrix called the kernel (or filter), slides it across the input, and at each location computes the sum of element-wise products. That sum becomes one output pixel.

**Diagram.** Rendered live in the web edition: <https://aiengineeringfromscratch.com/lesson?path=phases/04-computer-vision/02-convolutions-from-scratch>

A concrete 3x3 example on a 5x5 input (no padding, stride 1):

Input X (5 x 5):

```
1 2 0 1 2
0 1 3 1 0
2 1 0 2 1
1 0 2 1 3
2 1 1 0 1
```

Kernel W (3 x 3):

```
1 0 -1
2 0 -2
1 0 -1
```

The kernel slides across every valid 3 x 3 window. Output Y is 3 x 3:

```
Y[0,0] = sum( W * X[0:3, 0:3] )
Y[0,1] = sum( W * X[0:3, 1:4] )
Y[0,2] = sum( W * X[0:3, 2:5] )
Y[1,0] = sum( W * X[1:4, 0:3] )
... and so on
```

That one formula — **shared weights, locality, sliding window** — is the entire idea. Everything else is bookkeeping.

## Output size formula

Given input spatial size H, kernel size K, padding P, stride S:

$H_{out} = \text{floor}( (H - K + 2P) / S ) + 1$

Memorise this. You will compute it dozens of times per architecture.

Scenario	H	K	P	S	H _{out}
Valid conv, no padding	32	3	0	1	30
Same conv (preserves size)	32	3	1	1	32
Downsample by 2	32	3	1	2	16
Pool 2x2	32	2	0	2	16
Large receptive field	32	7	3	2	16

“Same padding” means pick P so that  $H_{out} == H$  when  $S == 1$ . For odd K, that is  $P = (K - 1) / 2$ . That is why 3x3 kernels dominate — they are the smallest odd kernel that still has a centre.

## Padding

Without padding, every convolution shrinks the feature map. Stack 20 of them and your 224x224 image becomes 184x184, which wastes compute on the border and complicates residual connections that need matching shapes.

Zero padding (P = 1) on a 5 x 5 input:

```
0 0 0 0 0 0 0
0 1 2 0 1 2 0
0 0 1 3 1 0 0
0 2 1 0 2 1 0
```

Now the kernel can centre on pixel

0	1	0	2	1	3	0	(0, 0) and still have three rows and
0	2	1	1	0	1	0	three columns of values to multiply.
0	0	0	0	0	0	0	

Modes you meet in practice: zero (most common), reflect (mirror the edge, avoids hard borders in generative models), replicate (copy the edge), circular (wrap around, used in toroidal problems).

## Stride

Stride is the step size of the slide. `stride=1` is the default. `stride=2` halves the spatial dimensions and is the classic way to downsample inside a CNN without a separate pooling layer — every modern architecture (ResNet, ConvNeXt, MobileNet) uses strided convs in place of max-pool somewhere.

Stride 1 on a 5 x 5 input, 3 x 3 kernel:

starts: (0,0) (0,1) (0,2)	-> output row 0
(1,0) (1,1) (1,2)	-> output row 1
(2,0) (2,1) (2,2)	-> output row 2

Output: 3 x 3

Stride 2 on the same input:

starts: (0,0) (0,2)	-> output row 0
(2,0) (2,2)	-> output row 1

Output: 2 x 2

## Multiple input channels

Real images have three channels. A 3x3 convolution on an RGB input is actually a 3x3x3 volume: one 3x3 slice per input channel. At each spatial position, you multiply and sum across all three slices and add a bias.

Input:	(C _{in} , H, W)	3 x 5 x 5
Kernel:	(C _{in} , K, K)	3 x 3 x 3 (one kernel)
Output:	(1, H', W')	2D map

For a layer that produces C_{out} output channels, you stack C_{out} kernels:

Weight:	(C _{out} , C _{in} , K, K)	e.g. 64 x 3 x 3 x 3
Output:	(C _{out} , H', W')	64 x 3 x 3

Parameter count: C_{out} * C_{in} * K * K + C_{out} (the + C_{out} is biases)

That last line is the one you will calculate when planning a model. A 64-channel 3x3 conv on a 3-channel input has  $64 * 3 * 3 * 3 + 64 = 1,792$  parameters. Cheap.

## The im2col trick

Nested loops are easy to read but slow. GPUs want big matrix multiplies. The trick: flatten every receptive-field window of the input into one column of a big matrix, flatten the kernel into a row, and the whole convolution becomes a single matmul.

**Diagram.** Rendered live in the web edition: <https://aiengineeringfromscratch.com/lesson?path=phases/04-computer-vision/02-convolutions-from-scratch>

Every production conv implementation is some variant of this plus cache-tiling tricks (direct conv, Winograd, FFT conv for large kernels). Understand im2col and you understand the core.

## Receptive field

A single 3x3 conv looks at 9 input pixels. Stack two 3x3 convs and a neuron in the second layer looks at 5x5 input pixels. Three 3x3 convs give 7x7. In general:

RF after L stacked K x K convs (stride 1) =  $1 + L * (K - 1)$

With strides: RF grows multiplicatively with stride along each layer.

The entire reason “3x3 all the way down” works (VGG, ResNet, ConvNeXt) is that two 3x3 convs see the same input area as one 5x5 conv but with fewer parameters and an extra non-linearity in between.

**Interactive figure: convolution-kernel.** This one moves. Watch it animate and drag its controls in the web edition: <https://aiengineeringfromscratch.com/lesson?path=phases/04-computer-vision/02-convolutions-from-scratch>

## Build It

### Step 1: Pad an array

Start with the smallest primitive: a function that pads with zeros around an H x W array.

```
import numpy as np

def pad2d(x, p):
    if p == 0:
        return x
    h, w = x.shape[-2:]
    out = np.zeros(x.shape[:-2] + (h + 2 * p, w + 2 * p), dtype=x.dtype)
    out[..., p:p + h, p:p + w] = x
    return out

x = np.arange(9).reshape(3, 3)
print(x)
print()
print(pad2d(x, 1))
```

The trailing-axes trick `x.shape[-2]` means the same function works on (H, W), (C, H, W), or (N, C, H, W) without modification.

### Step 2: 2D convolution with nested loops

The reference implementation — slow, but unambiguous. This is what `torch.nn.functional.conv2d` does in principle.

```
def conv2d_naive(x, w, b=None, stride=1, padding=0):
    c_in, h, w_in = x.shape
    c_out, c_in_w, kh, kw = w.shape
```

```

assert c_in == c_in_w

x_pad = pad2d(x, padding)
h_out = (h + 2 * padding - kh) // stride + 1
w_out = (w_in + 2 * padding - kw) // stride + 1

out = np.zeros((c_out, h_out, w_out), dtype=np.float32)
for oc in range(c_out):
    for i in range(h_out):
        for j in range(w_out):
            hs = i * stride
            ws = j * stride
            patch = x_pad[:, hs:hs + kh, ws:ws + kw]
            out[oc, i, j] = np.sum(patch * w[oc])
    if b is not None:
        out[oc] += b[oc]
return out

```

Four nested loops (output channel, row, column, plus the implicit sum over  $C_{in}$ ,  $kh$ ,  $kw$ ). This is the ground truth you will check every faster implementation against.

### Step 3: Verify with a hand-designed kernel

Build a vertical Sobel kernel, apply it to a synthetic step image, and watch the vertical edge light up.

```

def synthetic_step_image():
    img = np.zeros((1, 16, 16), dtype=np.float32)
    img[:, :, 8:] = 1.0
    return img

sobel_x = np.array([
    [-1, 0, 1],
    [-2, 0, 2],
    [-1, 0, 1]
], dtype=np.float32)[None]

x = synthetic_step_image()
y = conv2d_naive(x, sobel_x, padding=1)
print(y[0].round(1))

```

Expect large positive values on column 7 (left-to-right brightness increase) and zeros everywhere else. That single print is your sanity check that the math is right.

### Step 4: im2col

Convert every kernel-sized window in the input into a column of a matrix. For  $C_{in}=3$ ,  $K=3$ , each column is 27 numbers.

```

def im2col(x, kh, kw, stride=1, padding=0):
    c_in, h, w = x.shape
    x_pad = pad2d(x, padding)
    h_out = (h + 2 * padding - kh) // stride + 1
    w_out = (w + 2 * padding - kw) // stride + 1

    cols = np.zeros((c_in * kh * kw, h_out * w_out), dtype=x.dtype)

```



```

col = 0
for i in range(h_out):
    for j in range(w_out):
        hs = i * stride
        ws = j * stride
        patch = x_pad[:, hs:hs + kh, ws:ws + kw]
        cols[:, col] = patch.reshape(-1)
        col += 1
return cols, h_out, w_out

```

It is still a Python loop, but now the heavy lifting will be a single vectorised matmul.

## Step 5: Fast conv via im2col + matmul

Replace the quadruple loop with one matrix multiplication.

```

def conv2d_im2col(x, w, b=None, stride=1, padding=0):
    c_out, c_in, kh, kw = w.shape
    cols, h_out, w_out = im2col(x, kh, kw, stride, padding)
    w_flat = w.reshape(c_out, -1)
    out = w_flat @ cols
    if b is not None:
        out += b[:, None]
    return out.reshape(c_out, h_out, w_out)

```

Correctness check: run both implementations and compare.

```

rng = np.random.default_rng(0)
x = rng.normal(0, 1, (3, 16, 16)).astype(np.float32)
w = rng.normal(0, 1, (8, 3, 3, 3)).astype(np.float32)
b = rng.normal(0, 1, (8,)).astype(np.float32)

y_naive = conv2d_naive(x, w, b, padding=1)
y_im2col = conv2d_im2col(x, w, b, padding=1)

print(f"max abs diff: {np.max(np.abs(y_naive - y_im2col)):.2e}")

```

max abs diff should be around 1e-5 — the difference is floating-point accumulation order, not a bug.

## Step 6: A bank of hand-designed kernels

Five filters that show what a single conv layer can express before any training.

```

KERNELS = {
    "identity": np.array([[0, 0, 0], [0, 1, 0], [0, 0, 0]], dtype=np.float32),
    "blur_3x3": np.ones((3, 3), dtype=np.float32) / 9.0,
    "sharpen": np.array([[0, -1, 0], [-1, 5, -1], [0, -1, 0]], dtype=np.float32),
    "sobel_x": np.array([[ -1, 0, 1], [-2, 0, 2], [-1, 0, 1]], dtype=np.float32),
    "sobel_y": np.array([[ -1, -2, -1], [0, 0, 0], [1, 2, 1]], dtype=np.float32),
}

def apply_kernel(img2d, kernel):
    x = img2d[None].astype(np.float32)
    w = kernel[None, None]
    return conv2d_im2col(x, w, padding=1)[0]

```

Applied to any grayscale image, blur softens, sharpen crisps up edges, Sobel-x lights up vertical edges, Sobel-y lights up horizontal edges. These are exactly the patterns that the *first* trained conv layer in AlexNet and VGG ended up learning — because a good image model needs edge and blob detectors no matter what task comes later.

## Use It

PyTorch's `nn.Conv2d` wraps the same operation with autograd, CUDA kernels, and cuDNN optimisation. Shape semantics are identical.

```
import torch
import torch.nn as nn

conv = nn.Conv2d(in_channels=3, out_channels=64, kernel_size=3, stride=1,
    ↪ padding=1)
print(conv)
print(f"weight shape: {tuple(conv.weight.shape)} # (C_out, C_in, K, K)")
print(f"bias shape: {tuple(conv.bias.shape)}")
print(f"param count: {sum(p.numel() for p in conv.parameters())}")

x = torch.randn(8, 3, 224, 224)
y = conv(x)
print(f"\ninput shape: {tuple(x.shape)}")
print(f"output shape: {tuple(y.shape)}")
```

Swap `padding=1` for `padding=0` and the output drops to  $222 \times 222$ . Swap `stride=1` for `stride=2` and it drops to  $112 \times 112$ . Same formula you memorised above.

**This chapter ships an artifact.** The course version of this lesson produces a reusable prompt or agent skill. It lives in the repository, ready to install: <https://github.com/rohitg00/ai-engineering-from-scratch/tree/main/phases/04-computer-vision/02-convolutions-from-scratch>

## Exercises

Starter code and the lesson's working implementation: <https://github.com/rohitg00/ai-engineering-from-scratch/tree/main/phases/04-computer-vision/02-convolutions-from-scratch/code>

1. **(Easy)** Given a  $128 \times 128$  grayscale input and a stack of `[Conv3x3(s=1,p=1), Conv3x3(s=2,p=1), Conv3x3(s=1,p=1), Conv3x3(s=2,p=1)]`, compute the output spatial size and the receptive field at each layer by hand. Verify with a PyTorch `nn.Sequential` of dummy convs.
2. **(Medium)** Extend `conv2d_naive` and `conv2d_im2col` to accept a `groups` argument. Show that `groups=C_in=C_out` reproduces a depthwise convolution and that its parameter count is  $C * K * K$  instead of  $C * C * K * K$ .
3. **(Hard)** Implement the backward pass of `conv2d_im2col` by hand: given the gradient of the output, compute the gradient of `x` and `w`. Verify against `torch.autograd.grad` on the same inputs and weights. The trick: the gradient of `im2col` is `col2im`, and it has to accumulate overlapping windows.

## Key Terms

Term	What people say	What it actually means
Convolution	“Sliding a filter”	A learnable dot product applied at every spatial location with shared weights; mathematically a cross-correlation, but everyone calls it convolution
Kernel / filter	“The feature detector”	A small weight tensor of shape (C_in, K, K) whose dot product with a window of input produces one output pixel
Stride	“How far you jump”	The step size between consecutive kernel placements; stride 2 halves each spatial dimension
Padding	“Zeros on the edges”	Extra values added around the input so the kernel can centre on border pixels; same padding keeps output size equal to input size
Receptive field	“How much the neuron sees”	The patch of original input that a given output activation depends on, growing with depth and stride
im2col	“The GEMM trick”	Rearranging every receptive window into columns so convolution becomes one big matrix multiply — the core of every fast conv kernel
Depthwise conv	“One kernel per channel”	A conv with groups == C_in, computing each output channel from only its matching input channel; the backbone of MobileNet and ConvNeXt
Translation equivariance	“Shift in, shift out”	Property that shifting the input by k pixels shifts the output by k pixels; comes for free with shared weights

## Further Reading

- A guide to convolution arithmetic for deep learning (Dumoulin & Visin, 2016) — the definitive diagrams of padding/stride/dilation that every course quietly copies
- CS231n: Convolutional Neural Networks for Visual Recognition — the canonical lecture notes, including the original im2col explanation
- The Annotated ConvNet (fast.ai) — a notebook that walks from manual convolution to a trained digit classifier
- Receptive Field Arithmetic for CNNs (Dang Ha The Hien) — the paper-quality interactive explainer of receptive field calculations

**Continue online.** The living edition of this chapter has more than the page can hold:

- Animated, interactive figures and the web text: <https://aiengineeringfromscratch.com/lesson?path=phases/04-computer-vision/02-convolutions-from-scratch>
- Runnable code for every step: <https://github.com/rohitg00/ai-engineering-from-scratch/tree/main/phases/04-computer-vision/02-convolutions-from-scratch/code>
- The chapter quiz, graded in the browser: <https://aiengineeringfromscratch.com/lesson?path=phases/04-computer-vision/02-convolutions-from-scratch>

The repository moves faster than any printing. When the book and the repo disagree, trust the repo.

# CNNs — LeNet to ResNet

---

Every major CNN of the last thirty years is the same conv-nonlinearity-downsample recipe with one new idea bolted on. Learn the ideas in order.

**Type:** Learn + Build **Languages:** Python **Prerequisites:** Phase 3 Lesson 11 (PyTorch), Phase 4 Lesson 01 (Image Fundamentals), Phase 4 Lesson 02 (Convolutions from Scratch) **Time:** ~75 minutes

## Learning Objectives

- Trace the architectural lineage LeNet-5 -> AlexNet -> VGG -> Inception -> ResNet and state the single new idea each family contributed
- Implement LeNet-5, a VGG-style block, and a ResNet BasicBlock in PyTorch, each under 40 lines
- Explain why residual connections turn a 1,000-layer network from untrainable into state-of-the-art
- Read a modern backbone (ResNet-18, ResNet-50) and predict its output shape, receptive field, and parameter count before looking at the source

## The Problem

In 2011, the best ImageNet classifier scored around 74% top-5 accuracy. In 2012 AlexNet scored 85%. In 2015 ResNet scored 96%. No new data. No new GPU generation. The gains came from architecture ideas. A working vision engineer has to know which idea came from which paper because every production backbone you ship in 2026 is a recombination of those same pieces — and because the ideas keep transferring: grouped convs went from CNNs to transformers, residual connections went from ResNet to every LLM in existence, batch normalisation lives in diffusion models.

Studying these networks in order also immunises you against a common mistake: reaching for the biggest available model when a LeNet-sized network would solve the problem. MNIST does not need a ResNet. Knowing the scaling curve of each family tells you where to sit on it.

## The Concept

### The four ideas that changed vision

**Diagram.** Rendered live in the web edition: <https://aiengineeringfromscratch.com/lesson?path=phases/04-computer-vision/03-cnns-lenet-to-resnet>

Nothing else in classical vision mattered as much as these four jumps.

## LeNet-5 (1998)

Yann LeCun’s digit recogniser. 60,000 parameters. Two conv-pool blocks, two fully connected layers, tanh activations. It defined the template every CNN inherits:

```
input (1, 32, 32)
  conv 5x5 -> (6, 28, 28)
  avg pool 2x2 -> (6, 14, 14)
  conv 5x5 -> (16, 10, 10)
  avg pool 2x2 -> (16, 5, 5)
  flatten -> 400
  dense -> 120
  dense -> 84
  dense -> 10
```

Everything the modern world calls a CNN — alternating convolutions and downsampling feeding a small classifier head — is LeNet with more layers, bigger channels, and better activations.

## AlexNet (2012)

Three changes that together broke ImageNet:

1. **ReLU** instead of tanh. Gradients stop vanishing. Training speeds up by a factor of six.
2. **Dropout** in the fully connected head. Regularisation becomes a layer, not a trick.
3. **Depth and width**. Five conv layers, three dense layers, 60M parameters, trained on two GPUs with the model split across them.

The paper’s Figure 2 still shows the GPU split as two parallel streams. That parallelism was a hardware workaround, not an architectural insight — but the three ideas above are still in every model you use.

## VGG (2014)

VGG asked: what happens if you only use 3x3 convolutions and you go deep?

```
stack:  conv 3x3 -> conv 3x3 -> pool 2x2
repeat: 16 or 19 conv layers
```

Two 3x3 convs see the same 5x5 input area as one 5x5 conv but with fewer parameters ( $29C^2 = 18C^2$  vs  $25C^2$ ) and an extra ReLU in between. VGG turned this observation into an entire architecture. The simplicity — one block type, repeated — made it the reference point for everything that came after.

Cost: 138M parameters, slow to train, expensive at inference.

## Inception (2014, same year)

Google’s answer to “what kernel size should I use?” was: all of them, in parallel.

**Diagram.** Rendered live in the web edition: <https://aiengineeringfromscratch.com/lesson?path=phases/04-computer-vision/03-cnns-lenet-to-resnet>

Each branch specialises — 1x1 for channel mixing, 3x3 for local texture, 5x5 for larger patterns, pooling for shift-invariant features — and the concat lets the next layer pick whichever branch is useful. Inception v1 used 1x1 convolutions inside each branch as a bottleneck to keep parameter counts sane.

## The degradation problem

By 2015, VGG-19 worked and VGG-32 did not. Depth was supposed to help, but past ~20 layers both training and test loss got worse. That is not overfitting. That is the optimiser failing to find useful weights because gradients shrink multiplicatively through every layer.

Plain deep network:

$$y = f_L( f_{L-1}( \dots f_1(x) \dots ) )$$

Gradient wrt early layer:

$$dL/dW_1 = dL/dy * df_L/df_{L-1} * \dots * df_2/df_1 * df_1/dW_1$$

Each multiplicative term has magnitude roughly (weight magnitude) * (activation gain).  
 $\hookrightarrow$  gain).

Stack 100 of them with gains  $< 1$  and the gradient is effectively zero.

VGG worked at 19 layers because batch norm (published simultaneously) kept activations well-scaled. But even batch norm could not rescue depth beyond 30-ish layers.

## ResNet (2015)

He, Zhang, Ren, Sun proposed one change that fixed everything:

standard block:  $y = F(x)$

residual block:  $y = F(x) + x$

The  $+ x$  means the layer can always choose to do nothing by driving  $F(x)$  to zero. A 1,000-layer ResNet is now at most as bad as a 1-layer network, because every extra block has a trivial escape hatch. With that guarantee, the optimiser is willing to make every block *slightly* useful — and slightly useful, stacked 100 times, is state-of-the-art.

**Diagram.** Rendered live in the web edition: <https://aiengineeringfromscratch.com/lesson?path=phases/04-computer-vision/03-cnns-lenet-to-resnet>

Two variants of the block show up everywhere:

- **BasicBlock** (ResNet-18, ResNet-34): two 3x3 convs, skip around both.
- **Bottleneck** (ResNet-50, -101, -152): 1x1 down, 3x3 middle, 1x1 up, skip around the trio. Cheaper when channel counts are high.

When the skip has to cross a downsample (stride=2), the identity path is replaced with a 1x1 stride=2 conv to match shapes.

## Why residuals matter beyond vision

The idea was not really about image classification. It was about turning deep networks from “cross-your-fingers and hope gradients survive” into a reliable, scalable engineering tool. Every transformer you will read about next phase has the exact same skip connection in every block. Without ResNet, there is no GPT.

**Interactive figure: pooling.** This one moves. Watch it animate and drag its controls in the web edition: <https://aiengineeringfromscratch.com/lesson?path=phases/04-computer-vision/03-cnns-lenet-to-resnet>

## Build It

### Step 1: LeNet-5

A minimal, faithful LeNet. Tanh activations, average pooling. The only concession to modernity is that we use `nn.CrossEntropyLoss` downstream instead of the original Gaussian connections.

```
import torch
import torch.nn as nn
import torch.nn.functional as F

class LeNet5(nn.Module):
    def __init__(self, num_classes=10):
        super().__init__()
        self.conv1 = nn.Conv2d(1, 6, kernel_size=5)
        self.conv2 = nn.Conv2d(6, 16, kernel_size=5)
        self.pool = nn.AvgPool2d(2)
        self.fc1 = nn.Linear(16 * 5 * 5, 120)
        self.fc2 = nn.Linear(120, 84)
        self.fc3 = nn.Linear(84, num_classes)

    def forward(self, x):
        x = self.pool(torch.tanh(self.conv1(x)))
        x = self.pool(torch.tanh(self.conv2(x)))
        x = torch.flatten(x, 1)
        x = torch.tanh(self.fc1(x))
        x = torch.tanh(self.fc2(x))
        return self.fc3(x)

net = LeNet5()
x = torch.randn(1, 1, 32, 32)
print(f"output: {net(x).shape}")
print(f"params: {sum(p.numel() for p in net.parameters()):,}")
```

Expected output: output: `torch.Size([1, 10])`, params: 61,706. That is the entire digit classifier that started modern vision.

### Step 2: A VGG block

One reusable block: two 3x3 convs, ReLU, batch norm, max pool.

```
class VGGBlock(nn.Module):
    def __init__(self, in_c, out_c):
        super().__init__()
        self.conv1 = nn.Conv2d(in_c, out_c, kernel_size=3, padding=1)
        self.bn1 = nn.BatchNorm2d(out_c)
        self.conv2 = nn.Conv2d(out_c, out_c, kernel_size=3, padding=1)
        self.bn2 = nn.BatchNorm2d(out_c)
        self.pool = nn.MaxPool2d(2)

    def forward(self, x):
        x = F.relu(self.bn1(self.conv1(x)))
        x = F.relu(self.bn2(self.conv2(x)))
        return self.pool(x)
```

```

class MiniVGG(nn.Module):
    def __init__(self, num_classes=10):
        super().__init__()
        self.stack = nn.Sequential(
            VGGBlock(3, 32),
            VGGBlock(32, 64),
            VGGBlock(64, 128),
        )
        self.head = nn.Sequential(
            nn.AdaptiveAvgPool2d(1),
            nn.Flatten(),
            nn.Linear(128, num_classes),
        )

    def forward(self, x):
        return self.head(self.stack(x))

```

```

net = MiniVGG()
x = torch.randn(1, 3, 32, 32)
print(f"output: {net(x).shape}")
print(f"params: {sum(p.numel() for p in net.parameters()):,}")

```

Three VGG blocks on CIFAR-sized input, an adaptive pool, one linear layer. ~290k parameters. Plenty for CIFAR-10.

### Step 3: A ResNet BasicBlock

The core building block of ResNet-18 and ResNet-34.

```

class BasicBlock(nn.Module):
    def __init__(self, in_c, out_c, stride=1):
        super().__init__()
        self.conv1 = nn.Conv2d(in_c, out_c, kernel_size=3, stride=stride,
            ↪ padding=1, bias=False)
        self.bn1 = nn.BatchNorm2d(out_c)
        self.conv2 = nn.Conv2d(out_c, out_c, kernel_size=3, stride=1, padding=1,
            ↪ bias=False)
        self.bn2 = nn.BatchNorm2d(out_c)
        if stride != 1 or in_c != out_c:
            self.shortcut = nn.Sequential(
                nn.Conv2d(in_c, out_c, kernel_size=1, stride=stride, bias=False),
                nn.BatchNorm2d(out_c),
            )
        else:
            self.shortcut = nn.Identity()

    def forward(self, x):
        out = F.relu(self.bn1(self.conv1(x)))
        out = self.bn2(self.conv2(out))
        out = out + self.shortcut(x)
        return F.relu(out)

```

bias=False on conv layers is a batch-norm convention — BN's beta parameter already handles the bias, so carrying conv bias as well is a waste. The shortcut only needs a real conv when



stride or channel count changes; otherwise it is a no-op identity.

## Step 4: A tiny ResNet

Stack four groups of BasicBlocks to get a working ResNet for CIFAR-sized inputs.

```
class TinyResNet(nn.Module):
    def __init__(self, num_classes=10):
        super().__init__()
        self.stem = nn.Sequential(
            nn.Conv2d(3, 32, kernel_size=3, stride=1, padding=1, bias=False),
            nn.BatchNorm2d(32),
            nn.ReLU(inplace=True),
        )
        self.layer1 = self._make_group(32, 32, num_blocks=2, stride=1)
        self.layer2 = self._make_group(32, 64, num_blocks=2, stride=2)
        self.layer3 = self._make_group(64, 128, num_blocks=2, stride=2)
        self.layer4 = self._make_group(128, 256, num_blocks=2, stride=2)
        self.head = nn.Sequential(
            nn.AdaptiveAvgPool2d(1),
            nn.Flatten(),
            nn.Linear(256, num_classes),
        )

    def _make_group(self, in_c, out_c, num_blocks, stride):
        blocks = [BasicBlock(in_c, out_c, stride=stride)]
        for _ in range(num_blocks - 1):
            blocks.append(BasicBlock(out_c, out_c, stride=1))
        return nn.Sequential(*blocks)

    def forward(self, x):
        x = self.stem(x)
        x = self.layer1(x)
        x = self.layer2(x)
        x = self.layer3(x)
        x = self.layer4(x)
        return self.head(x)
```

```
net = TinyResNet()
x = torch.randn(1, 3, 32, 32)
print(f"output: {net(x).shape}")
print(f"params: {sum(p.numel() for p in net.parameters()):,}")
```

Four groups of two blocks each. Stride 2 at the start of groups 2, 3, 4. Channel count doubles at every downsample. Roughly 2.8M parameters. That is the standard recipe that scales cleanly up to ResNet-152.

## Step 5: Compare parameter-to-feature efficiency

Run the same input through all three networks and compare parameter counts.

```
def summary(name, net, x):
    y = net(x)
    params = sum(p.numel() for p in net.parameters())
```

```

print(f"{name:12s} input {tuple(x.shape)} -> output {tuple(y.shape)} params
    ↪ {params:>10,}")

x = torch.randn(1, 3, 32, 32)
summary("LeNet5", LeNet5(), torch.randn(1, 1, 32, 32))
summary("MiniVGG", MiniVGG(), x)
summary("TinyResNet", TinyResNet(), x)

```

Three models, three eras, three orders of magnitude in parameter count. For CIFAR-10 accuracy, you need roughly: LeNet 60%, MiniVGG 89%, TinyResNet 93% after a few epochs of training.

## Use It

`torchvision.models` gives you pretrained versions of all of the above. The call signature is identical across families, which is exactly the point of the backbone abstraction.

```
from torchvision.models import resnet18, ResNet18_Weights, vgg16, VGG16_Weights
```

```

r18 = resnet18(weights=ResNet18_Weights.IMAGENET1K_V1)
r18.eval()

```

```

print(f"ResNet-18 params: {sum(p.numel() for p in r18.parameters()):,}")
print(r18.layer1[0])
print()

```

```

v16 = vgg16(weights=VGG16_Weights.IMAGENET1K_V1)
v16.eval()
print(f"VGG-16 params: {sum(p.numel() for p in v16.parameters()):,}")

```

ResNet-18 has 11.7M parameters. VGG-16 has 138M. Similar ImageNet top-1 accuracy (69.8% vs 71.6%). Residual connections buy you a 12x parameter efficiency win. That is why ResNet variants dominated from 2016 until ViT arrived in 2021 — and still dominate real-world deployments where compute is the constraint.

For transfer learning, the recipe is always the same: load pretrained, freeze the backbone, replace the classifier head.

```

for p in r18.parameters():
    p.requires_grad = False
r18.fc = nn.Linear(r18.fc.in_features, 10)

```

Three lines. You now have a 10-class CIFAR classifier that inherits the representations ImageNet paid for.

**This chapter ships an artifact.** The course version of this lesson produces a reusable prompt or agent skill. It lives in the repository, ready to install: <https://github.com/rohitg00/ai-engineering-from-scratch/tree/main/phases/04-computer-vision/03-cnns-lenet-to-resnet>

## Exercises

Starter code and the lesson's working implementation: <https://github.com/rohitg00/ai-engineering-from-scratch/tree/main/phases/04-computer-vision/03-cnns-lenet-to-resnet/code>

1. **(Easy)** Count parameters by hand for TinyResNet layer by layer. Compare against `sum(p.numel() for p in net.parameters())`. Where does the majority of the parameter budget go — convs, BN, or the classifier head?
2. **(Medium)** Implement the Bottleneck block (1x1 -> 3x3 -> 1x1 with skip) and use it to build a ResNet-50-style network for CIFAR. Compare params against TinyResNet.
3. **(Hard)** Remove the skip connection from BasicBlock, train a 34-block “plain” network and a 34-block ResNet on CIFAR-10 for 10 epochs each. Plot training loss vs epoch for both. Reproduce the He et al. Figure 1 result where the plain deep network converges to higher loss than its shallower twin.

## Key Terms

Term	What people say	What it actually means
Backbone	“The model”	The stack of convolutional blocks that produces the feature map fed to the task head
Residual connection	“Skip connection”	$y = F(x) + x$ ; lets the optimiser learn identity by setting $F$ to zero, which makes arbitrary depth trainable
BasicBlock	“Two 3x3 convs with a skip”	The ResNet-18/34 building block: conv-BN-ReLU-conv-BN-add-ReLU
Bottleneck	“1x1 down, 3x3, 1x1 up”	The ResNet-50/101/152 block; cheap at high channel counts because the 3x3 runs on a reduced width
Degradation problem	“Deeper is worse”	Past ~20 plain conv layers, both training and test error increase; solved by residual connections, not by more data
Stem	“The first layer”	The initial conv that converts 3-channel input into the base feature width; usually 7x7 stride 2 for ImageNet, 3x3 stride 1 for CIFAR
Head	“The classifier”	The layers after the final backbone block: adaptive pool, flatten, linear(s)
Transfer learning	“Pretrained weights”	Loading a backbone trained on ImageNet and fine-tuning only the head on your task

## Further Reading

- Deep Residual Learning for Image Recognition (He et al., 2015) — the ResNet paper; every figure is worth studying
- Very Deep Convolutional Networks (Simonyan & Zisserman, 2014) — the VGG paper; still the best reference for “why 3x3”
- ImageNet Classification with Deep CNNs (Krizhevsky et al., 2012) — AlexNet; the paper that ended the hand-crafted-feature era
- Going Deeper with Convolutions (Szegedy et al., 2014) — Inception v1; the parallel-filter idea that still shows up in vision transformers

**Continue online.** The living edition of this chapter has more than the page can hold:

- Animated, interactive figures and the web text: <https://aiengineeringfromscratch.com/lesson?path=phases/04-computer-vision/03-cnns-lenet-to-resnet>

- Runnable code for every step: <https://github.com/rohitg00/ai-engineering-from-scratch/tree/main/phases/04-computer-vision/03-cnns-lenet-to-resnet/code>
- The chapter quiz, graded in the browser: <https://aiengineeringfromscratch.com/lesson?path=phases/04-computer-vision/03-cnns-lenet-to-resnet>

The repository moves faster than any printing. When the book and the repo disagree, trust the repo.

# Image Classification

---

A classifier is a function from pixels to a probability distribution over classes. Everything else is plumbing.

**Type:** Build **Languages:** Python **Prerequisites:** Phase 2 Lesson 09 (Model Evaluation), Phase 3 Lesson 10 (Mini Framework), Phase 4 Lesson 03 (CNNs) **Time:** ~75 minutes

## Learning Objectives

- Build an end-to-end image classification pipeline on CIFAR-10: dataset, augmentation, model, training loop, evaluation
- Explain the role of each component (dataloader, loss, optimizer, scheduler, augmentation) and predict how breaking any one of them manifests in the loss curve
- Implement mixup, cutout, and label smoothing from scratch and justify when each is worth adding
- Read a confusion matrix and a per-class precision/recall table to diagnose dataset and model failures beyond aggregate accuracy

## The Problem

Every vision task that ships reduces to image classification at some level. Detection classifies regions. Segmentation classifies pixels. Retrieval ranks by similarity to class centroids. Getting classification right — the dataset loop, the augmentation policy, the loss, the evaluation — is the skill that transfers to every other task in the phase.

Most classification bugs are not in the model. They live in the pipeline: a broken normalisation, an unshuffled training set, augmentation that distorts labels, a validation split contaminated by training data, a learning rate that silently diverges after epoch 30. A CNN that would hit 93% on CIFAR-10 with a correct setup commonly scores 70-75% with a broken one, and the loss curve looks plausible the whole time.

This lesson wires the entire pipeline by hand so every part is inspectable. You will not use anything from `torchvision.datasets` that could hide a bug.

## The Concept

### The classification pipeline

**Diagram.** Rendered live in the web edition: <https://aiengineeringfromscratch.com/lesson?path=phases/04-computer-vision/04-image-classification>

Every line in this loop is where a bug can live. Cross-entropy takes raw logits, not softmax outputs, so any `model(x).softmax()` before the loss quietly computes the wrong gradient. Augmentations apply to inputs only, not labels — except for mixup, which mixes both. `optimizer.zero_grad()` must happen once per step; skipping it accumulates gradients and looks

like a wildly unstable learning rate. Each of those bugs flattens the learning curve without throwing an error.

## Cross-entropy, logits, and softmax

A classifier produces  $C$  numbers per image called logits. Applying softmax converts them into a probability distribution:

$$\text{softmax}(z)_i = \exp(z_i) / \sum_j \exp(z_j)$$

Cross-entropy measures the negative log probability of the correct class:

$$\begin{aligned} \text{CE}(z, y) &= -\log(\text{softmax}(z)_y) \\ &= -z_y + \log(\sum_j \exp(z_j)) \end{aligned}$$

The right-hand form is the numerically stable one (log-sum-exp). PyTorch's `nn.CrossEntropy Loss` fuses softmax + NLL in one op and takes raw logits directly. Applying softmax yourself first is almost always a bug — you compute  $\log(\text{softmax}(\text{softmax}(z)))$ , a meaningless quantity.

## Why augmentation works

A CNN has inductive bias for translation (from weight sharing) but no built-in invariance to crops, flips, colour jitter, or occlusion. The only way to teach it those invariances is to show it pixels that exercise them. Every random transform during training is a way of saying: “these two images have the same label; learn the features that ignore the difference.”

```
Original crop: "dog facing left"
Flip:         "dog facing right"      <- same label, different pixels
Rotate(+15):  "dog, slight tilt"
Colour jitter: "dog in warmer light"
RandomErasing: "dog with patch missing"
```

The rule: augmentation must preserve the label. Cutout and rotation on a digit can flip “6” into “9”; for that dataset you use smaller rotation ranges and pick augmentations that respect digit-specific invariances.

## Mixup and cutmix

Ordinary augmentation transforms pixels but keeps labels one-hot. **Mixup** and **cutmix** break that by interpolating both.

```
Mixup:
  lambda ~ Beta(a, a)
  x = lambda * x_i + (1 - lambda) * x_j
  y = lambda * y_i + (1 - lambda) * y_j
```

```
Cutmix:
  paste a random rectangle of x_j into x_i
  y = area-weighted mix of y_i and y_j
```

Why it helps: the model stops memorising spiky one-hot targets and learns to interpolate between classes. Training loss goes up, test accuracy goes up. It is the single cheapest robustness upgrade for any classifier.

## Label smoothing

A cousin of mixup. Instead of training against  $[0, 0, 1, 0, 0]$ , train against  $[\text{eps}/C, \text{eps}/C, 1-\text{eps}, \text{eps}/C, \text{eps}/C]$  for a small  $\text{eps}$  like 0.1. Stops the model from producing arbitrarily sharp logits and improves calibration at almost no cost. Built into `nn.CrossEntropyLoss(label_smoothing=0.1)` since PyTorch 1.10.

## Evaluation beyond accuracy

Aggregate accuracy hides imbalance. A 90-10 binary classifier that always predicts the majority class scores 90%. The tools that actually tell you what is happening:

- **Per-class accuracy** — one number per class; immediately surfaces underperforming categories.
- **Confusion matrix** —  $C \times C$  grid with row  $i$  col  $j$  = count of true class  $i$  predicted as class  $j$ ; the diagonal is correct, the off-diagonals are where your model lives.
- **Top-1 / Top-5** — whether the correct class is in the top 1 or top 5 predictions; Top-5 matters for ImageNet because classes like “Norwich terrier” vs “Norfolk terrier” are genuinely ambiguous.
- **Calibration (ECE)** — does a 0.8 confidence prediction get it right 80% of the time? Modern networks are systematically over-confident; fix with temperature scaling or label smoothing.

**Interactive figure: receptive-field.** This one moves. Watch it animate and drag its controls in the web edition: <https://aiengineeringfromscratch.com/lesson?path=phases/04-computer-vision/04-image-classification>

## Build It

### Step 1: A deterministic synthetic dataset

CIFAR-10 lives on disk. To make this lesson reproducible and fast we build a synthetic dataset that looks like CIFAR — 32x32 RGB images with class-specific structure the model must learn. The exact same pipeline works unchanged on real CIFAR-10.

```
import numpy as np
import torch
from torch.utils.data import Dataset

def synthetic_cifar(num_per_class=1000, num_classes=10, seed=0):
    rng = np.random.default_rng(seed)
    X = []
    Y = []
    for c in range(num_classes):
        centre = rng.uniform(0, 1, (3,))
        freq = 2 + c
        for _ in range(num_per_class):
            yy, xx = np.meshgrid(np.linspace(0, 1, 32), np.linspace(0, 1, 32),
            indexing="ij")
            r = np.sin(xx * freq) * 0.5 + centre[0]
            g = np.cos(yy * freq) * 0.5 + centre[1]
            b = (xx + yy) * 0.5 * centre[2]
            img = np.stack([r, g, b], axis=-1)
            img += rng.normal(0, 0.08, img.shape)
```

```

        img = np.clip(img, 0, 1)
        X.append(img.astype(np.float32))
        Y.append(c)
X = np.stack(X)
Y = np.array(Y)
idx = rng.permutation(len(X))
return X[idx], Y[idx]

```

```

class ArrayDataset(Dataset):
    def __init__(self, X, Y, transform=None):
        self.X = X
        self.Y = Y
        self.transform = transform

    def __len__(self):
        return len(self.X)

    def __getitem__(self, i):
        img = self.X[i]
        if self.transform is not None:
            img = self.transform(img)
        img = torch.from_numpy(img).permute(2, 0, 1)
        return img, int(self.Y[i])

```

Each class gets its own colour palette and frequency pattern, plus Gaussian noise to force the model to learn the signal rather than memorise pixels. Ten classes, one thousand images each, permuted.

## Step 2: Normalisation and augmentation

The two transforms that every vision pipeline has.

```

def standardize(mean, std):
    mean = np.array(mean, dtype=np.float32)
    std = np.array(std, dtype=np.float32)
    def _fn(img):
        return (img - mean) / std
    return _fn

def random_hflip(p=0.5):
    def _fn(img):
        if np.random.random() < p:
            return img[:, ::-1, :].copy()
        return img
    return _fn

def random_crop(pad=4):
    def _fn(img):
        h, w = img.shape[:2]
        padded = np.pad(img, ((pad, pad), (pad, pad)), (0, 0), mode="reflect")
        y = np.random.randint(0, 2 * pad)
        x = np.random.randint(0, 2 * pad)

```



```

        return padded[y:y + h, x:x + w, :]
    return _fn

def compose(*fns):
    def _fn(img):
        for fn in fns:
            img = fn(img)
        return img
    return _fn

```

Reflect-pad before crop, not zero-pad, because black borders are a signal the model would learn to ignore in a non-useful way.

### Step 3: Mixup

Mixes two images and two labels inside the training step. Implemented as a batch transform so it lives next to the forward pass rather than inside the dataset.

```

def mixup_batch(x, y, num_classes, alpha=0.2):
    if alpha <= 0:
        return x, torch.nn.functional.one_hot(y, num_classes).float()
    lam = float(np.random.beta(alpha, alpha))
    idx = torch.randperm(x.size(0), device=x.device)
    x_mixed = lam * x + (1 - lam) * x[idx]
    y_onehot = torch.nn.functional.one_hot(y, num_classes).float()
    y_mixed = lam * y_onehot + (1 - lam) * y_onehot[idx]
    return x_mixed, y_mixed

def soft_cross_entropy(logits, soft_targets):
    log_probs = torch.log_softmax(logits, dim=-1)
    return -(soft_targets * log_probs).sum(dim=-1).mean()

```

`soft_cross_entropy` is cross-entropy against a soft-label distribution. It reduces to the usual one-hot case when the target is exactly one-hot.

### Step 4: The training loop

The complete recipe: one pass over the data, gradients once per batch, scheduler stepped once per epoch.

```

import torch
import torch.nn as nn
from torch.utils.data import DataLoader
from torch.optim import SGD
from torch.optim.lr_scheduler import CosineAnnealingLR

def train_one_epoch(model, loader, optimizer, device, num_classes,
    ↪ use_mixup=True):
    model.train()
    total, correct, loss_sum = 0, 0, 0.0
    for x, y in loader:
        x, y = x.to(device), y.to(device)
        if use_mixup:
            x_m, y_soft = mixup_batch(x, y, num_classes)

```

```

        logits = model(x_m)
        loss = soft_cross_entropy(logits, y_soft)
    else:
        logits = model(x)
        loss = nn.functional.cross_entropy(logits, y, label_smoothing=0.1)
    optimizer.zero_grad()
    loss.backward()
    optimizer.step()
    loss_sum += loss.item() * x.size(0)
    total += x.size(0)
    # Training accuracy vs the un-mixed labels `y` is only an approximation
    # when mixup is on (the model saw soft targets, not y). Treat it as a
    # rough progress signal; rely on val accuracy for real performance.
    with torch.no_grad():
        pred = logits.argmax(dim=-1)
        correct += (pred == y).sum().item()
    return loss_sum / total, correct / total

```

```

@torch.no_grad()
def evaluate(model, loader, device, num_classes):
    model.eval()
    total, correct = 0, 0
    loss_sum = 0.0
    cm = torch.zeros(num_classes, num_classes, dtype=torch.long)
    for x, y in loader:
        x, y = x.to(device), y.to(device)
        logits = model(x)
        loss = nn.functional.cross_entropy(logits, y)
        pred = logits.argmax(dim=-1)
        for t, p in zip(y.cpu(), pred.cpu()):
            cm[t, p] += 1
        loss_sum += loss.item() * x.size(0)
        total += x.size(0)
        correct += (pred == y).sum().item()
    return loss_sum / total, correct / total, cm

```

Five invariants you check every time you write a training loop:

1. `model.train()` before training, `model.eval()` before evaluation — flips dropout and batchnorm behaviour.
2. `.zero_grad()` before `.backward()`.
3. `.item()` when accumulating metrics so nothing keeps the computation graph alive.
4. `@torch.no_grad()` during evaluation — saves memory and time, prevents subtle accidents.
5. Argmax against raw logits, not softmax — same result, one fewer op.

## Step 5: Put it together

Use the TinyResNet from the previous lesson, train for a few epochs, evaluate.

```

from main import synthetic_cifar, ArrayDataset
from main import standardize, random_hflip, random_crop, compose
from main import mixup_batch, soft_cross_entropy
from main import train_one_epoch, evaluate

```

```

# TinyResNet comes from the previous lesson (03-cnns-lenet-to-resnet).
# Adjust the import path to wherever you stored the previous lesson's code.
from cnns_lenet_to_resnet import TinyResNet # example placeholder

X, Y = synthetic_cifar(num_per_class=500)
split = int(0.9 * len(X))
X_train, Y_train = X[:split], Y[:split]
X_val, Y_val = X[split:], Y[split:]

mean = [0.5, 0.5, 0.5]
std = [0.25, 0.25, 0.25]
train_tf = compose(random_hflip(), random_crop(pad=4), standardize(mean, std))
eval_tf = standardize(mean, std)

train_ds = ArrayDataset(X_train, Y_train, transform=train_tf)
val_ds = ArrayDataset(X_val, Y_val, transform=eval_tf)

train_loader = DataLoader(train_ds, batch_size=128, shuffle=True, num_workers=0)
val_loader = DataLoader(val_ds, batch_size=256, shuffle=False, num_workers=0)

device = "cuda" if torch.cuda.is_available() else "cpu"
model = TinyResNet(num_classes=10).to(device)
optimizer = SGD(model.parameters(), lr=0.1, momentum=0.9, weight_decay=5e-4,
    ↪ nesterov=True)
scheduler = CosineAnnealingLR(optimizer, T_max=10)

for epoch in range(10):
    tr_loss, tr_acc = train_one_epoch(model, train_loader, optimizer, device, 10,
    ↪ use_mixup=True)
    va_loss, va_acc, _ = evaluate(model, val_loader, device, 10)
    scheduler.step()
    print(f"epoch {epoch:2d}  lr {scheduler.get_last_lr()[0]:.4f}  "
          f"train {tr_loss:.3f}/{tr_acc:.3f}  val {va_loss:.3f}/{va_acc:.3f}")

```

On the synthetic dataset, this gets to near-perfect validation accuracy within five epochs, which is the point: the pipeline is correct, the model can learn what is learnable. Swap the dataset for real CIFAR-10 and the same loop trains to ~90% without changes.

## Step 6: Read the confusion matrix

Accuracy alone never tells you where the model is failing. The confusion matrix does.

```

def print_confusion(cm, labels=None):
    c = cm.shape[0]
    labels = labels or [str(i) for i in range(c)]
    print(f"{'':>6}" + "".join(f"{l:>5}" for l in labels))
    for i in range(c):
        row = cm[i].tolist()
        print(f"{labels[i]:>6}" + "".join(f"{v:>5}" for v in row))
    print()
    tp = cm.diag().float()
    fp = cm.sum(dim=0).float() - tp
    fn = cm.sum(dim=1).float() - tp
    prec = tp / (tp + fp).clamp_min(1)
    rec = tp / (tp + fn).clamp_min(1)

```

```

f1 = 2 * prec * rec / (prec + rec).clamp_min(1e-9)
for i in range(c):
    print(f"{labels[i]:>6}  prec {prec[i]:.3f}  rec {rec[i]:.3f}  f1
        ↪ {f1[i]:.3f}")

_, _, cm = evaluate(model, val_loader, device, 10)
print_confusion(cm)

```

Rows are true classes, columns are predictions. A cluster of off-diagonal counts between classes 3 and 5 means the model confuses those two and gives you a starting point for targeted data collection or a class-specific augmentation.

## Use It

torchvision wraps everything above into idiomatic components. For real CIFAR-10 the full pipeline is four lines plus a training loop.

```

from torchvision.datasets import CIFAR10
from torchvision.transforms import Compose, RandomCrop, RandomHorizontalFlip,
    ↪ ToTensor, Normalize

mean = (0.4914, 0.4822, 0.4465)
std = (0.2470, 0.2435, 0.2616)
train_tf = Compose([
    RandomCrop(32, padding=4, padding_mode="reflect"),
    RandomHorizontalFlip(),
    ToTensor(),
    Normalize(mean, std),
])
eval_tf = Compose([ToTensor(), Normalize(mean, std)])

train_ds = CIFAR10(root="./data", train=True, download=True, transform=train_tf)
val_ds   = CIFAR10(root="./data", train=False, download=True, transform=eval_tf)

```

Two things to notice: the mean/std are **dataset-specific** — computed on the CIFAR-10 training set, not ImageNet — and the reflect pad is the community-default crop policy. Copy-pasting ImageNet stats here is a ~1% accuracy leak that nobody catches until someone profiles the model.

**This chapter ships an artifact.** The course version of this lesson produces a reusable prompt or agent skill. It lives in the repository, ready to install: <https://github.com/rohitg00/ai-engineering-from-scratch/tree/main/phases/04-computer-vision/04-image-classification>

## Exercises

Starter code and the lesson's working implementation: <https://github.com/rohitg00/ai-engineering-from-scratch/tree/main/phases/04-computer-vision/04-image-classification/code>

1. **(Easy)** Train the same model with and without mixup for five epochs on the synthetic dataset. Plot train and val loss for both. Explain why train loss with mixup is higher yet val accuracy is similar or better.
2. **(Medium)** Implement Cutout — zero out a random 8x8 square in each training image — and run an ablation vs no augmentation, hflip+crop, hflip+crop+cutout, hflip+crop+mixup. Report val accuracy for each.

3. **(Hard)** Build a CIFAR-100 pipeline (100 classes, same input size) and reproduce a ResNet-34 training run to within 1% of published accuracy. Extras: sweep three learning rates and two weight decays, log to a local CSV, produce the final confusion-matrix-top-confusions table.

## Key Terms

Term	What people say	What it actually means
Logits	“Raw outputs”	The pre-softmax vector of C numbers per image; cross-entropy expects these, not softmaxed values
Cross-entropy	“The loss”	Negative log-probability of the correct class; combines log-softmax and NLL in one stable op
DataLoader	“The batcher”	Wraps a dataset with shuffling, batching, and (optional) multi-worker loading; gets blamed for half of training bugs
Augmentation	“Random transforms”	Any pixel-level transform at training time that preserves the label; teaches invariances the CNN does not have natively
Mixup / Cutmix	“Mix two images”	Blend both inputs and labels so the classifier learns smooth interpolations instead of hard boundaries
Label smoothing	“Softer targets”	Replace one-hot with $(1-\text{eps}, \text{eps}/(C-1), \dots)$ ; improves calibration and slightly boosts accuracy
Top-k accuracy	“Top-5”	The correct class is in the k highest-probability predictions; used on datasets with genuinely ambiguous classes
Confusion matrix	“Where errors live”	C x C table where entry (i, j) counts images of true class i predicted as j; diagonal is right, off-diagonal tells you what to fix

## Further Reading

- CS231n: Training Neural Networks — still the clearest tour of the training pipeline at a single page
- Bag of Tricks for Image Classification (He et al., 2019) — every small trick that together adds 3-4% to ResNet accuracy on ImageNet
- mixup: Beyond Empirical Risk Minimization (Zhang et al., 2017) — the original mixup paper; three pages of theory plus convincing experiments
- Why temperature scaling matters (Guo et al., 2017) — the paper that proved modern networks are miscalibrated and fixed it with one scalar parameter

**Continue online.** The living edition of this chapter has more than the page can hold:

- Animated, interactive figures and the web text: <https://aiengineeringfromscratch.com/lesson?path=phases/04-computer-vision/04-image-classification>
- Runnable code for every step: <https://github.com/rohitg00/ai-engineering-from-scratch/tree/main/phases/04-computer-vision/04-image-classification/code>

- The chapter quiz, graded in the browser: <https://aiengineeringfromscratch.com/lesson?path=phases/04-computer-vision/04-image-classification>

The repository moves faster than any printing. When the book and the repo disagree, trust the repo.

# Transfer Learning & Fine-Tuning

---

Somebody else spent a million GPU hours teaching a network what edges, textures, and object parts look like. You should borrow those features before training your own.

**Type:** Build **Languages:** Python **Prerequisites:** Phase 4 Lesson 03 (CNNs), Phase 4 Lesson 04 (Image Classification) **Time:** ~75 minutes

## Learning Objectives

- Distinguish feature extraction from fine-tuning and pick the right one based on dataset size, domain distance, and compute budget
- Load a pretrained backbone, replace its classifier head, and train only the head to a working baseline in under 20 lines
- Progressively unfreeze layers with discriminative learning rates so early generic features get smaller updates than late task-specific ones
- Diagnose the three common failures: feature drift from too-high LR on unfrozen blocks, BN statistics collapse on tiny datasets, and catastrophic forgetting

## The Problem

Training a ResNet-50 on ImageNet costs around 2,000 GPU-hours. Very few teams have that budget for every task they ship. What almost every team actually ships is a pretrained backbone with a new head trained on a few hundred or few thousand task-specific images.

This is not a shortcut. The first conv block of any ImageNet-trained CNN learns edges and Gabor-like filters. The next few blocks learn textures and simple motifs. The middle blocks learn object parts. The final blocks learn combinations that start to look like the 1,000 ImageNet categories. The first 90% of that hierarchy transfers almost unchanged to medical imaging, industrial inspection, satellite data, and every other vision task — because nature has a limited vocabulary of edges and textures. The last 10% is what you actually train.

Getting transfer right has three bugs waiting for you: destroying pretrained features with a too-high learning rate, starving the model of information by freezing too much, and letting BatchNorm's running statistics drift toward a tiny dataset that the rest of the network never learnt from. This lesson walks each of them on purpose.

## The Concept

### Feature extraction vs fine-tuning

Two regimes, picked by how much you trust the pretrained features and how much data you have.

**Diagram.** Rendered live in the web edition: <https://aiengineeringfromscratch.com/lesson?path=phases/04-computer-vision/05-transfer-learning>

Rules of thumb:

Dataset size	Domain distance	Recipe
< 1k images	close to ImageNet	Freeze backbone, train head only
1k-10k	close	Freeze first 2-3 stages, fine-tune the rest
10k-100k	any	Fine-tune end-to-end with discriminative LR
100k+	far	Fine-tune everything; consider training from scratch if domain is far enough

“Close to ImageNet” roughly means natural RGB photos with object-like content. Medical CT scans, overhead satellite imagery, and microscopy are far domains — the features still help, but you will need to let more layers adapt.

## Why freezing works at all

The ImageNet features a CNN learns are not specialised to the 1,000 categories. They are specialised to the statistics of natural images: edges at specific orientations, textures, contrast patterns, shape primitives. Those statistics are stable across almost every visual domain a human can name. That is why a model trained on ImageNet and evaluated zero-shot on CIFAR-10 with just a new linear head (no fine-tuning of the backbone) reaches 80%+ accuracy. The head is learning which of the already-learned features to weight for this task.

## Discriminative learning rates

When you do unfreeze, early layers should train slower than late layers. Early layers encode generic features that you want to preserve; late layers encode task-specific structure that you need to move a lot.

Typical recipe:

```

stage 0 (stem + first group): lr = base_lr / 100    (mostly fixed)
stage 1:                      lr = base_lr / 10
stage 2:                      lr = base_lr / 3
stage 3 (last backbone group): lr = base_lr
head:                         lr = base_lr (or slightly higher)

```

In PyTorch this is just a list of parameter groups passed to the optimizer. One model, five learning rates, zero extra code.

## The BatchNorm problem

BN layers hold `running_mean` and `running_var` buffers that were computed on ImageNet. If your task has a different pixel distribution — different lighting, different sensor, different colour space — those buffers are wrong. Three options in order of preference:



1. **Fine-tune with BN in train mode.** Let BN update its running statistics along with everything else. Default choice when the task dataset is medium-sized ( $\geq 5k$  examples).
2. **Freeze BN in eval mode.** Keep the ImageNet statistics and train only the weights. Correct when your dataset is small enough that BN's moving average would be noisy.
3. **Replace BN with GroupNorm.** Removes the moving-average problem entirely. Used in detection and segmentation backbones where batch size per GPU is tiny.

Getting this wrong silently tanks accuracy by 5-15%.

## Head design

The classifier head is 1-3 linear layers plus an optional dropout. Every torchvision backbone ships a default head that you replace:

```
backbone.fc = nn.Linear(backbone.fc.in_features, num_classes)      # ResNet
backbone.classifier[1] = nn.Linear(..., num_classes)              #
    ↪ EfficientNet, MobileNet
backbone.heads.head = nn.Linear(..., num_classes)                  #
    ↪ torchvision ViT
```

For small datasets, a single linear layer is usually enough. Adding a hidden layer (Linear  $\rightarrow$  ReLU  $\rightarrow$  Dropout  $\rightarrow$  Linear) helps when the task distribution is farther from the backbone's training distribution.

## Layer-wise LR decay

A smoother version of discriminative LR used in modern fine-tuning (BEiT, DINOv2, ViT-B fine-tunes). Instead of grouping layers into stages, give every layer a slightly smaller LR than the one above it:

$$\text{lr_layer_k} = \text{base_lr} * \text{decay}^{(L - k)}$$

With decay = 0.75 and L = 12 transformer blocks, the first block trains at  $0.75^{11} \approx 0.04x$  the head's LR. Matters more for transformer fine-tunes than for CNNs, where stage-grouped LR are usually enough.

## What to evaluate

Transfer-learning runs need two numbers you would not track on a scratch run:

- **Pretrained-only accuracy** — the head's accuracy with the backbone frozen. This is your floor.
- **Fine-tuned accuracy** — the same model after end-to-end training. This is your ceiling.

If fine-tuned is less than pretrained-only, you have a learning-rate or BN bug. Always print both.

**Interactive figure: transfer-learning.** This one moves. Watch it animate and drag its controls in the web edition: <https://aiengineeringfromscratch.com/lesson?path=phases/04-computer-vision/05-transfer-learning>

## Build It

### Step 1: Load a pretrained backbone and inspect it

```
import torch
import torch.nn as nn
from torchvision.models import resnet18, ResNet18_Weights

backbone = resnet18(weights=ResNet18_Weights.IMAGENET1K_V1)
print(backbone)
print()
print("classifier head:", backbone.fc)
print("feature dim:", backbone.fc.in_features)
```

ResNet18 has four stages (layer1..layer4) plus a stem and a fc head. Every torchvision classification backbone has an analogous structure.

### Step 2: Feature extraction — freeze everything, replace the head

```
def make_feature_extractor(num_classes=10):
    model = resnet18(weights=ResNet18_Weights.IMAGENET1K_V1)
    for p in model.parameters():
        p.requires_grad = False
    model.fc = nn.Linear(model.fc.in_features, num_classes)
    return model

model = make_feature_extractor(num_classes=10)
trainable = sum(p.numel() for p in model.parameters() if p.requires_grad)
frozen = sum(p.numel() for p in model.parameters() if not p.requires_grad)
print(f"trainable: {trainable:>10,}")
print(f"frozen: {frozen:>10,}")
```

Only model.fc is trainable. The backbone is a frozen feature extractor.

### Step 3: Discriminative fine-tuning

A utility that builds parameter groups with stage-specific learning rates.

```
def discriminative_param_groups(model, base_lr=1e-3, decay=0.3):
    stages = [
        ["conv1", "bn1"],
        ["layer1"],
        ["layer2"],
        ["layer3"],
        ["layer4"],
        ["fc"],
    ]
    groups = []
    for i, names in enumerate(stages):
        lr = base_lr * (decay ** (len(stages) - 1 - i))
        params = [p for n, p in model.named_parameters()
                   if any(n.startswith(k) for k in names)]
        if params:
            groups.append({"params": params, "lr": lr, "name": "_".join(names)})
    return groups
```

```

model = resnet18(weights=ResNet18_Weights.IMAGENET1K_V1)
model.fc = nn.Linear(model.fc.in_features, 10)
for p in model.parameters():
    p.requires_grad = True

groups = discriminative_param_groups(model)
for g in groups:
    print(f"{g['name']:>10s}  lr={g['lr']:.2e}  params={sum(p.numel() for p in
    ↪ g['params']):>8,}")

```

decay=0.3 means each stage trains at 30% of the rate of the next one. fc gets base_lr, layer4 gets  $0.3 * \text{base_lr}$ , conv1 gets  $0.3^5 * \text{base_lr} \approx 0.00243 * \text{base_lr}$ . Extreme sounding; empirically it works.

## Step 4: BatchNorm handling

Helper to freeze BN running statistics without freezing its weights.

```

def freeze_bn_stats(model):
    for m in model.modules():
        if isinstance(m, (nn.BatchNorm1d, nn.BatchNorm2d, nn.BatchNorm3d)):
            m.eval()
            for p in m.parameters():
                p.requires_grad = False
    return model

```

Call it after you set `model.train()` at the start of every epoch. `model.train()` flips everything to training mode; this reverses it only for BN layers.

## Step 5: A minimal end-to-end fine-tuning loop

```

from torch.optim import SGD
from torch.utils.data import DataLoader
from torch.optim.lr_scheduler import CosineAnnealingLR
import torch.nn.functional as F

def fine_tune(model, train_loader, val_loader, device, epochs=5, base_lr=1e-3,
    ↪ freeze_bn=False):
    model = model.to(device)
    groups = discriminative_param_groups(model, base_lr=base_lr)
    optimizer = SGD(groups, momentum=0.9, weight_decay=1e-4, nesterov=True)
    scheduler = CosineAnnealingLR(optimizer, T_max=epochs)

    for epoch in range(epochs):
        model.train()
        if freeze_bn:
            freeze_bn_stats(model)
        tr_loss, tr_correct, tr_total = 0.0, 0, 0
        for x, y in train_loader:
            x, y = x.to(device), y.to(device)
            logits = model(x)
            loss = F.cross_entropy(logits, y, label_smoothing=0.1)
            optimizer.zero_grad()
            loss.backward()

```

```

optimizer.step()
tr_loss += loss.item() * x.size(0)
tr_total += x.size(0)
tr_correct += (logits.argmax(-1) == y).sum().item()
scheduler.step()

model.eval()
va_total, va_correct = 0, 0
with torch.no_grad():
    for x, y in val_loader:
        x, y = x.to(device), y.to(device)
        pred = model(x).argmax(-1)
        va_total += x.size(0)
        va_correct += (pred == y).sum().item()
print(f"epoch {epoch}  train
↪ {tr_loss/tr_total:.3f}/{tr_correct/tr_total:.3f}  "
      f"val {va_correct/va_total:.3f}")
return model

```

Five epochs with the above recipe on CIFAR-10 takes ResNet18-IMAGENET1K_V1 from ~70% zero-shot linear-probe accuracy to ~93% fine-tuned accuracy. The head alone would plateau around 86% without ever touching the backbone.

## Step 6: Progressive unfreezing

A schedule that unfreezes one stage per epoch from the end toward the beginning. Mitigates feature drift at the cost of some extra epochs.

```

def progressive_unfreeze_schedule(model):
    stages = ["layer4", "layer3", "layer2", "layer1"]
    yielded = set()

    def start():
        for p in model.parameters():
            p.requires_grad = False
        for p in model.fc.parameters():
            p.requires_grad = True

    def unfreeze(epoch):
        if epoch < len(stages):
            name = stages[epoch]
            yielded.add(name)
            for n, p in model.named_parameters():
                if n.startswith(name):
                    p.requires_grad = True
            return name
        return None

    return start, unfreeze

```

Call `start()` once before the first epoch. Call `unfreeze(epoch)` at the start of each epoch. Rebuild the optimizer whenever the set of trainable parameters changes, otherwise the frozen params still hold cached moments that confuse it.

## Use It

For most real tasks, `torchvision.models` + three lines is enough. The heavier machinery above matters when you run into the problems that library defaults cannot fix.

```
from torchvision.models import resnet50, ResNet50_Weights
```

```
model = resnet50(weights=ResNet50_Weights.IMAGENET1K_V2)
model.fc = nn.Linear(model.fc.in_features, num_classes)
optimizer = torch.optim.AdamW(model.parameters(), lr=1e-4, weight_decay=1e-4)
```

Two other production-grade defaults:

- `timm` ships ~800 pretrained vision backbones with a consistent API (`timm.create_model("resnet50", pretrained=True, num_classes=10)`). For any fine-tune beyond the `torchvision` zoo, it is the standard.
- For transformers, `transformers.AutoModelForImageClassification.from_pretrained(name, num_labels=N)` gives you ViT / BEiT / DeiT with the same loading semantics as text models.

**This chapter ships an artifact.** The course version of this lesson produces a reusable prompt or agent skill. It lives in the repository, ready to install: <https://github.com/rohitg00/ai-engineering-from-scratch/tree/main/phases/04-computer-vision/05-transfer-learning>

## Exercises

Starter code and the lesson's working implementation: <https://github.com/rohitg00/ai-engineering-from-scratch/tree/main/phases/04-computer-vision/05-transfer-learning/code>

1. **(Easy)** Train a ResNet18 as a linear probe (backbone frozen) and as a full fine-tune on the same synthetic-CIFAR dataset. Report both accuracies side by side. Explain which gap tells you the features transfer well and which tells you they do not.
2. **(Medium)** Introduce a bug on purpose: set `base_lr = 1e-1` on the backbone stage instead of the head. Show the training loss explode, then recover by applying the `discriminative_param_groups` helper. Record the LR at which each stage starts diverging.
3. **(Hard)** Take a medical imaging dataset (e.g. CheXpert-small, PatchCamelyon, or HAM10000) and compare three regimes: (a) ImageNet-pretrained frozen backbone + linear head; (b) ImageNet-pretrained fine-tune end-to-end; (c) scratch training. Report accuracy and compute cost for each. At what dataset size does scratch training become competitive?

## Key Terms

Term	What people say	What it actually means
Feature extraction	"Freeze and train head"	Backbone parameters frozen, only the new classifier head receives gradient
Fine-tuning	"Retrain end-to-end"	All parameters trainable, usually with much smaller LR than scratch training
Discriminative LR	"Smaller LR for early layers"	Optimizer parameter groups where early-stage LR is a fraction of late-stage LR
Layer-wise LR decay	"Smooth LR gradient"	Per-layer LR multiplied by $\text{decay}^{(L - k)}$ ; common in transformer fine-tunes

Term	What people say	What it actually means
Catastrophic forgetting	“The model lost ImageNet”	A too-high LR overwrites pretrained features before the new task signal is learnt
BN statistics drift	“Running mean is wrong”	BatchNorm running_mean/var computed on a different distribution than the current task, silently hurting accuracy
Linear probe	“Frozen backbone + linear head”	Evaluation of pretrained features — accuracy of the best linear classifier on top of the frozen representation
Catastrophic collapse	“Everything predicts one class”	Happens when fine-tuning with an LR high enough to destroy features before gradients from the head can stabilise

## Further Reading

- How transferable are features in deep neural networks? (Yosinski et al., 2014) — the paper that quantified feature transferability across layers
- Universal Language Model Fine-tuning (ULMFiT, Howard & Ruder, 2018) — the original discriminative LR / progressive unfreezing recipe; the ideas transfer directly to vision
- timm documentation — the reference for modern vision backbones and the exact fine-tune defaults they were trained with
- A Simple Framework for Linear-Probe Evaluation (Kornblith et al., 2019) — why linear-probe accuracy matters and how to report it correctly

**Continue online.** The living edition of this chapter has more than the page can hold:

- Animated, interactive figures and the web text: <https://aiengineeringfromscratch.com/lesson?path=phases/04-computer-vision/05-transfer-learning>
- Runnable code for every step: <https://github.com/rohitg00/ai-engineering-from-scratch/tree/main/phases/04-computer-vision/05-transfer-learning/code>
- The chapter quiz, graded in the browser: <https://aiengineeringfromscratch.com/lesson?path=phases/04-computer-vision/05-transfer-learning>

The repository moves faster than any printing. When the book and the repo disagree, trust the repo.

# Object Detection — YOLO from Scratch

---

Detection is classification plus regression, run at every position in a feature map, then cleaned up with non-maximum suppression.

**Type:** Build **Languages:** Python **Prerequisites:** Phase 4 Lesson 03 (CNNs), Phase 4 Lesson 04 (Image Classification), Phase 4 Lesson 05 (Transfer Learning) **Time:** ~75 minutes

## Learning Objectives

- Explain the grid-and-anchor design that turns detection into a dense prediction problem and state what every number in the output tensor means
- Compute Intersection-over-Union between boxes and implement non-maximum suppression from scratch
- Build a minimal YOLO-style head on top of a pretrained backbone, including the classification, objectness, and box-regression losses
- Read a detection metric row (precision@0.5, recall, mAP@0.5, mAP@0.5:0.95) and pick which knob to turn next

## The Problem

Classification says “this image is a dog.” Detection says “there is a dog at pixels (112, 40, 280, 210), there is a cat at (400, 180, 560, 310), and nothing else in the frame.” That one structural change — predicting a variable number of labelled boxes instead of one label per image — is what every autonomous system, every surveillance product, every document layout parser, and every factory vision line depends on.

Detection is also where every engineering trade-off in vision shows up at once. You want boxes that are accurate (regression head), you want the right class for each box (classification head), you want the model to know when there is nothing to detect (objectness score), and you want exactly one prediction per real object (non-maximum suppression). Miss any of these and the pipeline either misses objects, reports hallucinated boxes, or predicts the same object fifteen times in slightly different positions.

YOLO (You Only Look Once, Redmon et al. 2016) was the design that made all of this run in real time by doing it with a single forward pass of a conv net, and the same structural decisions are still the backbone of modern detectors (YOLOv8, YOLOv9, YOLO-NAS, RT-DETR). Learn the core and every variant becomes a rearrangement of the same parts.

## The Concept

### Detection as dense prediction

A classifier outputs  $C$  numbers per image. A YOLO-style detector outputs  $(S \times S \times (5 + C))$  numbers per image, where  $S$  is the spatial grid size.

**Diagram.** Rendered live in the web edition: <https://aiengineeringfromscratch.com/lesson?path=phases/04-computer-vision/06-object-detection-yolo>

Each of the  $S * S$  grid cells predicts  $B$  boxes. For each box:

- 4 numbers describe geometry:  $tx, ty, tw, th$ .
- 1 number is the objectness score: “is there an object centred in this cell?”
- $C$  numbers are class probabilities.

Total per cell:  $B * (5 + C)$ . For VOC with  $S=13, B=2, C=20$ , that is 50 numbers per cell.

## Why grids and anchors

Plain regression would predict  $(x, y, w, h)$  for every object as an absolute coordinate. That is hard for a conv network because translating the image should not translate all predictions by the same amount — each object is spatially anchored. The grid answers this by assigning each ground-truth box to the grid cell its centre falls in; only that cell is responsible for that object.

Anchors address a second problem. A  $3 \times 3$  conv cannot easily regress a 500-pixel-wide box out of a 16-pixel receptive field feature cell. Instead, we pre-define  $B$  prior box shapes (anchors) per cell and predict small deltas from each anchor. The model learns to pick the right anchor and nudge it rather than regress from nothing.

Anchor box priors (example for  $416 \times 416$  input):

```
small:   (30, 60)
medium:  (75, 170)
large:   (200, 380)
```

At each grid cell, every anchor emits  $(tx, ty, tw, th, obj, c_1, \dots, c_C)$ .

Modern detectors often use FPN with different anchor sets per resolution — small anchors on shallow high-resolution maps, large anchors on deep low-resolution maps. Same idea, more scales.

## Decoding predictions

The raw  $tx, ty, tw, th$  are not box coordinates; they are regression targets to be transformed before plotting:

```
centre x = (sigmoid(tx) + cell_x) * stride
centre y = (sigmoid(ty) + cell_y) * stride
width    = anchor_w * exp(tw)
height   = anchor_h * exp(th)
```

`sigmoid` keeps centre offsets inside the cell. `exp` lets the width scale freely from the anchor without a sign flip. `stride` scales the grid coordinates back to pixels. This decode step is the same in every YOLO version since v2.

## IoU

Detection’s universal similarity metric between two boxes:

$\text{IoU}(A, B) = \text{area}(A \text{ intersect } B) / \text{area}(A \text{ union } B)$



IoU = 1 means identical; IoU = 0 means no overlap. IoU between the prediction and the ground-truth box is what decides whether a prediction counts as a true positive (typically  $\text{IoU} \geq 0.5$ ). IoU between two predictions is what NMS uses to deduplicate.

## Non-maximum suppression

A conv network trained on adjacent anchors will often predict overlapping boxes for the same object. NMS keeps the highest-confidence prediction and deletes any other prediction with IoU above a threshold.

```
NMS(boxes, scores, iou_threshold):
    sort boxes by score descending
    keep = []
    while boxes not empty:
        pick the top-scoring box, add to keep
        remove every box with IoU > iou_threshold to the picked box
    return keep
```

Typical threshold: 0.45 for object detection. Recent detectors replace standard NMS with soft-NMS, DIOU-NMS, or learn the suppression directly (RT-DETR) but the structural purpose is the same.

## The loss

YOLO loss is three losses added with weights:

```
L = lambda_coord * L_box(pred, target, where obj=1)
  + lambda_obj    * L_obj(pred, 1,      where obj=1)
  + lambda_noobj  * L_obj(pred, 0,      where obj=0)
  + lambda_cls    * L_cls(pred, target, where obj=1)
```

Only cells that contain an object contribute to the box-regression and classification losses. Cells without objects contribute only to the objectness loss (teaching the model to stay silent).  $\lambda_{\text{noobj}}$  is usually small ( $\sim 0.5$ ) because the vast majority of cells are empty and would otherwise dominate the total loss.

Modern variants swap MSE box loss for CIOU / DIOU (which optimise IoU directly), use focal loss for class imbalance, and balance objectness with quality focal loss. The three-component structure is unchanged.

## Detection metrics

Accuracy does not transfer to detection. Four numbers that do:

- **Precision@IoU=0.5** — of the predictions counted as positives, how many are actually correct.
- **Recall@IoU=0.5** — of the real objects, how many did we find.
- **AP@0.5** — precision-recall curve area at IoU threshold 0.5; one number per class.
- **mAP@0.5:0.95** — average of AP over IoU thresholds 0.5, 0.55, ..., 0.95. The COCO metric; strictest and most informative.

Report all four. A detector that is strong on mAP@0.5 but weak on mAP@0.5:0.95 is localising roughly but not tightly; fix with better box-regression loss. A detector with high precision and low recall is too conservative; lower the confidence threshold or increase the objectness weight.

**Interactive figure: object-detection-nms.** This one moves. Watch it animate and drag its controls in the web edition: <https://aiengineeringfromscratch.com/lesson?path=phases/04-computer-vision/06-object-detection-yolo>

## Build It

### Step 1: IoU

The workhorse of the whole lesson. Works on two arrays of boxes in (x1, y1, x2, y2) format.

```
import numpy as np

def box_iou(boxes_a, boxes_b):
    ax1, ay1, ax2, ay2 = boxes_a[:, 0], boxes_a[:, 1], boxes_a[:, 2], boxes_a[:, 3]
    bx1, by1, bx2, by2 = boxes_b[:, 0], boxes_b[:, 1], boxes_b[:, 2], boxes_b[:, 3]

    inter_x1 = np.maximum(ax1[:, None], bx1[None, :])
    inter_y1 = np.maximum(ay1[:, None], by1[None, :])
    inter_x2 = np.minimum(ax2[:, None], bx2[None, :])
    inter_y2 = np.minimum(ay2[:, None], by2[None, :])

    inter_w = np.clip(inter_x2 - inter_x1, 0, None)
    inter_h = np.clip(inter_y2 - inter_y1, 0, None)
    inter = inter_w * inter_h

    area_a = (ax2 - ax1) * (ay2 - ay1)
    area_b = (bx2 - bx1) * (by2 - by1)
    union = area_a[:, None] + area_b[None, :] - inter
    return inter / np.clip(union, 1e-8, None)
```

Returns an (N_a, N_b) matrix of pairwise IoUs. Use it against a single ground-truth box by making one of the arrays shape (1, 4).

### Step 2: Non-max suppression

```
def nms(boxes, scores, iou_threshold=0.45):
    order = np.argsort(-scores)
    keep = []
    while len(order) > 0:
        i = order[0]
        keep.append(i)
        if len(order) == 1:
            break
        rest = order[1:]
        ious = box_iou(boxes[[i]], boxes[rest])[0]
        order = rest[ious <= iou_threshold]
    return np.array(keep, dtype=np.int64)
```

Deterministic,  $O(N \log N)$  from the sort, and matches the behaviour of `torchvision.ops.nms` on identical inputs.

### Step 3: Box encoding and decoding

Convert between pixel coordinates and the (tx, ty, tw, th) targets that the network actually regresses.

```
def encode(box_xyxy, cell_x, cell_y, stride, anchor_wh):
    x1, y1, x2, y2 = box_xyxy
    cx = 0.5 * (x1 + x2)
    cy = 0.5 * (y1 + y2)
    w = x2 - x1
    h = y2 - y1
    tx = cx / stride - cell_x
    ty = cy / stride - cell_y
    tw = np.log(w / anchor_wh[0] + 1e-8)
    th = np.log(h / anchor_wh[1] + 1e-8)
    return np.array([tx, ty, tw, th])

def decode(tx_ty_tw_th, cell_x, cell_y, stride, anchor_wh):
    tx, ty, tw, th = tx_ty_tw_th
    cx = (sigmoid(tx) + cell_x) * stride
    cy = (sigmoid(ty) + cell_y) * stride
    w = anchor_wh[0] * np.exp(tw)
    h = anchor_wh[1] * np.exp(th)
    return np.array([cx - w / 2, cy - h / 2, cx + w / 2, cy + h / 2])

def sigmoid(x):
    return 1.0 / (1.0 + np.exp(-x))
```

Test: encode a box then decode — you should get back something very close to the original (up to the sigmoid inverse not being perfectly invertible when tx is not in the post-sigmoid range).

### Step 4: A minimal YOLO head

One 1x1 conv on a feature map, reshaping to (B, S, S, num_anchors, 5 + C).

```
import torch
import torch.nn as nn

class YOLOHead(nn.Module):
    def __init__(self, in_c, num_anchors, num_classes):
        super().__init__()
        self.num_anchors = num_anchors
        self.num_classes = num_classes
        self.conv = nn.Conv2d(in_c, num_anchors * (5 + num_classes), kernel_size=1)

    def forward(self, x):
        n, _, h, w = x.shape
        y = self.conv(x)
        y = y.view(n, self.num_anchors, 5 + self.num_classes, h, w)
        y = y.permute(0, 3, 4, 1, 2).contiguous()
        return y
```

Output shape: (N, H, W, num_anchors, 5 + C). The last dimension holds [tx, ty, tw, th,

obj, cls_0, ..., cls_{C-1}].

## Step 5: Ground-truth assignment

For every ground-truth box, decide which (cell, anchor) is responsible.

```
def assign_targets(boxes_xyxy, classes, anchors, stride, grid_size, num_classes):
    num_anchors = len(anchors)
    target = np.zeros((grid_size, grid_size, num_anchors, 5 + num_classes),
        dtype=np.float32)
    has_obj = np.zeros((grid_size, grid_size, num_anchors), dtype=bool)

    for box, cls in zip(boxes_xyxy, classes):
        x1, y1, x2, y2 = box
        cx, cy = 0.5 * (x1 + x2), 0.5 * (y1 + y2)
        gx, gy = int(cx / stride), int(cy / stride)
        bw, bh = x2 - x1, y2 - y1

        ious = np.array([
            (min(bw, aw) * min(bh, ah)) / (bw * bh + aw * ah - min(bw, aw) *
        ↪ min(bh, ah))
            for aw, ah in anchors
        ])
        best = int(np.argmax(ious))
        aw, ah = anchors[best]

        target[gy, gx, best, 0] = cx / stride - gx
        target[gy, gx, best, 1] = cy / stride - gy
        target[gy, gx, best, 2] = np.log(bw / aw + 1e-8)
        target[gy, gx, best, 3] = np.log(bh / ah + 1e-8)
        target[gy, gx, best, 4] = 1.0
        target[gy, gx, best, 5 + cls] = 1.0
        has_obj[gy, gx, best] = True
    return target, has_obj
```

Anchor selection is “best shape IoU with the ground truth” — a cheap proxy that matches the YOLOv2/v3 assignment. v5 and later use more sophisticated strategies (task-aligned matching, dynamic k) that refine the same idea.

## Step 6: The three losses

```
def yolo_loss(pred, target, has_obj, lambda_coord=5.0, lambda_obj=1.0,
    ↪ lambda_noobj=0.5, lambda_cls=1.0):
    has_obj_t = torch.from_numpy(has_obj).bool()
    target_t = torch.from_numpy(target).float()

    # box-regression loss: only on cells with objects
    box_pred = pred[..., :4][has_obj_t]
    box_true = target_t[..., :4][has_obj_t]
    loss_box = torch.nn.functional.mse_loss(box_pred, box_true, reduction="sum")

    # objectness loss
    obj_pred = pred[..., 4]
    obj_true = target_t[..., 4]
```

```

loss_obj_pos = torch.nn.functional.binary_cross_entropy_with_logits(
    obj_pred[has_obj_t], obj_true[has_obj_t], reduction="sum")
loss_obj_neg = torch.nn.functional.binary_cross_entropy_with_logits(
    obj_pred[~has_obj_t], obj_true[~has_obj_t], reduction="sum")

# classification loss on cells with objects
cls_pred = pred[..., 5:][has_obj_t]
cls_true = target_t[..., 5:][has_obj_t]
loss_cls = torch.nn.functional.binary_cross_entropy_with_logits(
    cls_pred, cls_true, reduction="sum")

total = (lambda_coord * loss_box
        + lambda_obj * loss_obj_pos
        + lambda_noobj * loss_obj_neg
        + lambda_cls * loss_cls)
return total, {"box": loss_box.item(), "obj_pos": loss_obj_pos.item(),
              "obj_neg": loss_obj_neg.item(), "cls": loss_cls.item()}

```

Five hyper-parameters that every YOLO tutorial either hardcodes or sweeps. The ratios matter: `lambda_coord=5`, `lambda_noobj=0.5` mirrors the original YOLOv1 paper and still works as a reasonable default.

## Step 7: Inference pipeline

Decode the raw head output, apply sigmoid/exp, threshold on objectness, and NMS.

```

def postprocess(pred_tensor, anchors, stride, img_size, conf_threshold=0.25,
    ↪ iou_threshold=0.45):
    pred = pred_tensor.detach().cpu().numpy()
    grid_h, grid_w = pred.shape[1], pred.shape[2]
    num_anchors = len(anchors)

    boxes, scores, classes = [], [], []
    for gy in range(grid_h):
        for gx in range(grid_w):
            for a in range(num_anchors):
                tx, ty, tw, th, obj, *cls = pred[0, gy, gx, a]
                score = sigmoid(obj) * sigmoid(np.array(cls)).max()
                if score < conf_threshold:
                    continue
                cls_idx = int(np.argmax(cls))
                cx = (sigmoid(tx) + gx) * stride
                cy = (sigmoid(ty) + gy) * stride
                w = anchors[a][0] * np.exp(tw)
                h = anchors[a][1] * np.exp(th)
                boxes.append([cx - w / 2, cy - h / 2, cx + w / 2, cy + h / 2])
                scores.append(float(score))
                classes.append(cls_idx)

    if not boxes:
        return np.zeros((0, 4)), np.zeros((0,)), np.zeros((0,)), dtype=int)
    boxes = np.array(boxes)
    scores = np.array(scores)
    classes = np.array(classes)
    keep = nms(boxes, scores, iou_threshold)

```

```
return boxes[keep], scores[keep], classes[keep]
```

That is the complete eval path: head -> decode -> threshold -> NMS.

## Use It

`torchvision.models.detection` ships production detectors with the same conceptual structure. Loading a pretrained model takes three lines.

```
import torch
from torchvision.models.detection import fasterrcnn_resnet50_fpn_v2

model = fasterrcnn_resnet50_fpn_v2(weights="DEFAULT")
model.eval()
with torch.no_grad():
    predictions = model([torch.randn(3, 400, 600)])
print(predictions[0].keys())
print(f"boxes: {predictions[0]['boxes'].shape}")
print(f"scores: {predictions[0]['scores'].shape}")
print(f"labels: {predictions[0]['labels'].shape}")
```

For real-time inference pipelines, ultralytics (YOLOv8/v9) is the standard: `from ultralytics import YOLO; model = YOLO('yolov8n.pt'); model(img)`. The model handles decoding and NMS internally and returns the same boxes / scores / labels triple you built above.

**This chapter ships an artifact.** The course version of this lesson produces a reusable prompt or agent skill. It lives in the repository, ready to install: <https://github.com/rohitg00/ai-engineering-from-scratch/tree/main/phases/04-computer-vision/06-object-detection-yolo>

## Exercises

Starter code and the lesson’s working implementation: <https://github.com/rohitg00/ai-engineering-from-scratch/tree/main/phases/04-computer-vision/06-object-detection-yolo/code>

1. **(Easy)** Implement `box_iou` and run it against `torchvision.ops.box_iou` on 1,000 random box pairs. Verify max absolute difference is below  $1e-6$ .
2. **(Medium)** Port `yolo_loss` to a version that uses CIoU box loss instead of MSE. Show on a 100-image synthetic dataset that CIoU converges to a better final mAP@0.5:0.95 than MSE in the same number of epochs.
3. **(Hard)** Implement multi-scale inference: feed the same image at three resolutions through the model, union the box predictions, and run a single NMS at the end. Measure the mAP lift vs single-scale inference on a held-out set.

## Key Terms

Term	What people say	What it actually means
Anchor	“Box prior”	A pre-defined box shape at each grid cell from which the network predicts deltas instead of absolute coordinates
IoU	“Overlap”	Intersection-over-union of two boxes; the universal similarity measure in detection

Term	What people say	What it actually means
NMS	“Deduplicate”	Greedy algorithm that keeps highest-score predictions and removes overlapping ones above a threshold
Objectness	“Is there something here”	Per-anchor, per-cell scalar predicting whether an object is centred in that cell
Grid stride	“Downsample factor”	Pixels per grid cell; a 416-px input with a 13-grid head has stride 32
mAP	“Mean average precision”	Average of the area under the precision-recall curve, averaged over classes and (for COCO) IoU thresholds
AP@0.5	“PASCAL VOC AP”	Average precision with IoU threshold 0.5; the lenient version of the metric
mAP@0.5:0.95	“COCO AP”	Average over IoU thresholds 0.5..0.95 step 0.05; the strict version and current community standard

## Further Reading

- YOLOv1: You Only Look Once (Redmon et al., 2016) — the founding paper; every YOLO since is a refinement of this structure
- YOLOv3 (Redmon & Farhadi, 2018) — the paper that introduced multi-scale FPN-style heads; still the clearest diagram
- Ultralytics YOLOv8 docs — the current production reference; covers dataset formats, augmentations, training recipes
- The Illustrated Guide to Object Detection (Jonathan Hui) — best plain-English tour of the full detector zoo; priceless for understanding how DETR, RetinaNet, FCOS, and YOLO relate

**Continue online.** The living edition of this chapter has more than the page can hold:

- Animated, interactive figures and the web text: <https://aiengineeringfromscratch.com/lesson?path=phases/04-computer-vision/06-object-detection-yolo>
- Runnable code for every step: <https://github.com/rohitg00/ai-engineering-from-scratch/tree/main/phases/04-computer-vision/06-object-detection-yolo/code>
- The chapter quiz, graded in the browser: <https://aiengineeringfromscratch.com/lesson?path=phases/04-computer-vision/06-object-detection-yolo>

The repository moves faster than any printing. When the book and the repo disagree, trust the repo.

# Semantic Segmentation — U-Net

---

Segmentation is classification at every pixel. U-Net makes it work by pairing a downsampling encoder with an upsampling decoder and wiring skip connections between them.

**Type:** Build **Languages:** Python **Prerequisites:** Phase 4 Lesson 03 (CNNs), Phase 4 Lesson 04 (Image Classification) **Time:** ~75 minutes

## Learning Objectives

- Distinguish semantic, instance, and panoptic segmentation and pick the right task for a given problem
- Build a U-Net from scratch in PyTorch with encoder blocks, a bottleneck, a decoder with transposed convolutions, and skip connections
- Implement pixel-wise cross-entropy, Dice loss, and the combined loss that is the current default for medical and industrial segmentation
- Read IoU and Dice metrics per class and diagnose whether a bad score comes from small-object recall, boundary accuracy, or class imbalance

## The Problem

Classification outputs one label per image. Detection outputs a handful of boxes per image. Segmentation outputs one label per pixel. For an input of size  $H \times W$ , the output is a tensor of shape  $H \times W$  (semantic) or  $H \times W \times N_{\text{instances}}$  (instance). That is millions of predictions per image, not one.

The structure of segmentation is why it powers almost every dense-prediction vision product: medical imaging (tumour masks), autonomous driving (road, lane, obstacle), satellite (building footprints, crop boundaries), document parsing (layout zones), robotics (graspable regions). None of those tasks can be solved by putting a box around the object; they need the exact silhouette.

The architectural problem is simple to state and not simple to solve: you need the network to see the global context of an image (what kind of scene is this) and the local pixel detail (exactly which pixel is road vs pavement) simultaneously. A standard CNN compresses spatially to gain context, which throws away the detail. U-Net was the design that got both.

## The Concept

### Semantic vs instance vs panoptic

**Diagram.** Rendered live in the web edition: <https://aiengineeringfromscratch.com/lesson?path=phases/04-computer-vision/07-semantic-segmentation-unet>

- **Semantic** says “this pixel is road, that pixel is car.” Two cars next to each other collapse into a single blob.



- **Instance** says “this pixel is car #3, that pixel is car #5.” Ignores background stuff (“stuff” = sky, road, grass).
- **Panoptic** unifies both: every pixel gets a class label, every instance gets a unique id, stuff and things both segmented.

This lesson covers semantic. The next lesson (Mask R-CNN) covers instance.

## The U-Net shape

**Diagram.** Rendered live in the web edition: <https://aiengineeringfromscratch.com/lesson?path=phases/04-computer-vision/07-semantic-segmentation-unet>

The encoder halves spatial resolution four times and doubles channels. The decoder reverses: doubles spatial resolution four times and halves channels. The skip connections concatenate matching encoder features with decoder features at every resolution. The final 1x1 conv maps 64 -> num_classes at full resolution.

Why skip connections are necessary: the decoder has seen only small feature maps by the time it tries to output pixel-level predictions. Without the skips it cannot localise edges accurately because that information was compressed away in the encoder. Skip connections hand it the high-resolution feature maps the encoder computed on the way down.

## Transposed vs bilinear upsample

The decoder has to expand spatial dimensions. Two options:

- **Transposed convolution** (`nn.ConvTranspose2d`) — learnable upsample. Historical U-Net default. Can produce checkerboard artifacts if stride and kernel size do not divide evenly.
- **Bilinear upsample + 3x3 conv** — smooth upsample followed by a conv. Fewer artifacts, fewer parameters, now the modern default.

Both appear in the wild. For a first U-Net, bilinear is safer.

## Cross-entropy on a pixel grid

For semantic segmentation with C classes, the model output is (N, C, H, W). The target is (N, H, W) with integer class IDs. Cross-entropy is identical to the classification case, just applied at every spatial position:

Loss = mean over (n, h, w) of  $-\log(\text{softmax}(\text{logits}[n, :, h, w])[\text{target}[n, h, w]])$

`F.cross_entropy` in PyTorch handles this shape natively. No reshape needed.

## Dice loss and why you need it

Cross-entropy treats every pixel equally. That is wrong when one class dominates the frame (medical imaging: 99% background, 1% tumour). The network can score 99% accuracy by predicting background everywhere and still be useless.

Dice loss solves this by directly optimising the overlap between predicted and true mask:

$\text{Dice}(p, y) = 2 * \text{sum}(p * y) / (\text{sum}(p) + \text{sum}(y) + \text{epsilon})$

$\text{Dice_loss} = 1 - \text{Dice}$

where p is the sigmoid/softmax probability map for a class and y is the binary ground-truth mask. The loss is zero only when the overlap is perfect. Because it is ratio-based, class imbalance is irrelevant.

In practice, use the **combined loss**:

$$L = L_{\text{cross_entropy}} + \lambda * L_{\text{dice}} \quad (\lambda \sim 1)$$

Cross-entropy gives stable gradients early in training; Dice focuses the tail of training on actually matching the mask shape. This combination is the medical-imaging default and hard to beat on any class-imbalanced dataset.

## Evaluation metrics

- **Pixel accuracy** — percent of pixels predicted correctly. Cheap. Broken on imbalanced data for the same reason as accuracy in classification.
- **IoU per class** — intersection over union for each class's mask; average across classes = mIoU.
- **Dice (F1 on pixels)** — similar to IoU;  $\text{Dice} = 2 * \text{IoU} / (1 + \text{IoU})$ . Medical imaging prefers Dice, driving community prefers IoU; they are monotonically related.
- **Boundary F1** — measures how close predicted boundaries are to ground-truth boundaries, penalising even small shifts. Important for high-precision tasks like semiconductor inspection.

Report IoU per class, not just mIoU. Mean IoU hides a class at 15% when nine others are at 85%.

## Input resolution trade-off

U-Net's encoder halves resolution four times, so the input must be divisible by 16. Medical images are often 512x512 or 1024x1024. Autonomous-driving crops are 2048x1024. The memory cost of U-Net scales with  $H * W * C_{\text{max}}$ , and at 1024x1024 with 1024 bottleneck channels the forward pass already uses gigabytes of VRAM.

Two standard workarounds: 1. Tile the input — process 256x256 tiles with overlap and stitch. 2. Replace the bottleneck with dilated convolutions that keep spatial resolution higher but widen receptive field (the DeepLab family).

For a first model, a 256x256 input with a 64-channel-base U-Net trains comfortably on 8 GB VRAM.

**Interactive figure: segmentation-flood.** This one moves. Watch it animate and drag its controls in the web edition: <https://aiengineeringfromscratch.com/lesson?path=phases/04-computer-vision/07-semantic-segmentation-unet>

## Build It

### Step 1: Encoder block

Two 3x3 convs with batch norm and ReLU. The first conv changes channel count; the second keeps it.

```
import torch
import torch.nn as nn
import torch.nn.functional as F

class DoubleConv(nn.Module):
    def __init__(self, in_c, out_c):
        super().__init__()
        self.net = nn.Sequential(
```

```

        nn.Conv2d(in_c, out_c, kernel_size=3, padding=1, bias=False),
        nn.BatchNorm2d(out_c),
        nn.ReLU(inplace=True),
        nn.Conv2d(out_c, out_c, kernel_size=3, padding=1, bias=False),
        nn.BatchNorm2d(out_c),
        nn.ReLU(inplace=True),
    )

    def forward(self, x):
        return self.net(x)

```

This block is reused throughout. `bias=False` because BN's beta handles the bias.

## Step 2: Down and up blocks

```

class Down(nn.Module):
    def __init__(self, in_c, out_c):
        super().__init__()
        self.net = nn.Sequential(
            nn.MaxPool2d(2),
            DoubleConv(in_c, out_c),
        )

    def forward(self, x):
        return self.net(x)

class Up(nn.Module):
    def __init__(self, in_c, out_c):
        super().__init__()
        self.up = nn.Upsample(scale_factor=2, mode="bilinear", align_corners=False)
        self.conv = DoubleConv(in_c, out_c)

    def forward(self, x, skip):
        x = self.up(x)
        if x.shape[-2:] != skip.shape[-2:]:
            x = F.interpolate(x, size=skip.shape[-2:], mode="bilinear",
↪ align_corners=False)
        x = torch.cat([skip, x], dim=1)
        return self.conv(x)

```

The spatial-only shape check (`shape[-2:]`) handles inputs whose dimensions are not divisible by 16; a safe `F.interpolate` aligns the tensor before the concat. Comparing the full shape would also trigger on channel-count differences, which should be a loud error, not a silent interpolate.

## Step 3: The U-Net

```

class UNet(nn.Module):
    def __init__(self, in_channels=3, num_classes=2, base=64):
        super().__init__()
        self.inc = DoubleConv(in_channels, base)
        self.d1 = Down(base, base * 2)
        self.d2 = Down(base * 2, base * 4)

```

```

self.d3 = Down(base * 4, base * 8)
self.d4 = Down(base * 8, base * 16)
self.u1 = Up(base * 16 + base * 8, base * 8)
self.u2 = Up(base * 8 + base * 4, base * 4)
self.u3 = Up(base * 4 + base * 2, base * 2)
self.u4 = Up(base * 2 + base, base)
self.outc = nn.Conv2d(base, num_classes, kernel_size=1)

def forward(self, x):
    x1 = self.inc(x)
    x2 = self.d1(x1)
    x3 = self.d2(x2)
    x4 = self.d3(x3)
    x5 = self.d4(x4)
    x = self.u1(x5, x4)
    x = self.u2(x, x3)
    x = self.u3(x, x2)
    x = self.u4(x, x1)
    return self.outc(x)

net = UNet(in_channels=3, num_classes=2, base=32)
x = torch.randn(1, 3, 256, 256)
print(f"output: {net(x).shape}")
print(f"params: {sum(p.numel() for p in net.parameters()):,}")

Output shape (1, 2, 256, 256) — same spatial size as the input, num_classes channels. About
7.7M parameters at base=32.

```

## Step 4: Losses

```

def dice_loss(logits, targets, num_classes, eps=1e-6):
    probs = F.softmax(logits, dim=1)
    targets_one_hot = F.one_hot(targets, num_classes).permute(0, 3, 1, 2).float()
    dims = (0, 2, 3)
    intersection = (probs * targets_one_hot).sum(dim=dims)
    denom = probs.sum(dim=dims) + targets_one_hot.sum(dim=dims)
    dice = (2 * intersection + eps) / (denom + eps)
    return 1 - dice.mean()

def combined_loss(logits, targets, num_classes, lam=1.0):
    ce = F.cross_entropy(logits, targets)
    dc = dice_loss(logits, targets, num_classes)
    return ce + lam * dc, {"ce": ce.item(), "dice": dc.item()}

```

Dice is computed per class then averaged (macro Dice). The eps prevents division by zero on classes absent from the batch.

## Step 5: IoU metric

```

@torch.no_grad()
def iou_per_class(logits, targets, num_classes):
    preds = logits.argmax(dim=1)
    ious = torch.zeros(num_classes)

```

```

for c in range(num_classes):
    pred_c = (preds == c)
    true_c = (targets == c)
    inter = (pred_c & true_c).sum().float()
    union = (pred_c | true_c).sum().float()
    ious[c] = (inter / union) if union > 0 else torch.tensor(float("nan"))
return ious

```

Returns a vector of length C. nan marks classes absent from the batch — do not average over those when computing mIoU.

## Step 6: Synthetic dataset for end-to-end verification

Generate shapes on coloured backgrounds so the network has to learn shape, not pixel colour.

```

import numpy as np
from torch.utils.data import Dataset, DataLoader

def synthetic_segmentation(num_samples=200, size=64, seed=0):
    rng = np.random.default_rng(seed)
    images = np.zeros((num_samples, size, size, 3), dtype=np.float32)
    masks = np.zeros((num_samples, size, size), dtype=np.int64)
    for i in range(num_samples):
        bg = rng.uniform(0, 1, (3,))
        images[i] = bg
        masks[i] = 0
        num_shapes = rng.integers(1, 4)
        for _ in range(num_shapes):
            cls = int(rng.integers(1, 3))
            color = rng.uniform(0, 1, (3,))
            cx, cy = rng.integers(10, size - 10, size=2)
            r = int(rng.integers(4, 12))
            yy, xx = np.meshgrid(np.arange(size), np.arange(size), indexing="ij")
            if cls == 1:
                mask = (xx - cx) ** 2 + (yy - cy) ** 2 < r ** 2
            else:
                mask = (np.abs(xx - cx) < r) & (np.abs(yy - cy) < r)
            images[i][mask] = color
            masks[i][mask] = cls
        images[i] += rng.normal(0, 0.02, images[i].shape)
        images[i] = np.clip(images[i], 0, 1)
    return images, masks

class SegDataset(Dataset):
    def __init__(self, images, masks):
        self.images = images
        self.masks = masks

    def __len__(self):
        return len(self.images)

    def __getitem__(self, i):
        img = torch.from_numpy(self.images[i]).permute(2, 0, 1).float()
        mask = torch.from_numpy(self.masks[i]).long()

```

```
return img, mask
```

Three classes: background (0), circles (1), squares (2). The network must learn to distinguish shape.

## Step 7: Training loop

```
def train_one_epoch(model, loader, optimizer, device, num_classes):
    model.train()
    loss_sum, total = 0.0, 0
    iou_sum = torch.zeros(num_classes)
    for x, y in loader:
        x, y = x.to(device), y.to(device)
        logits = model(x)
        loss, _ = combined_loss(logits, y, num_classes)
        optimizer.zero_grad()
        loss.backward()
        optimizer.step()
        loss_sum += loss.item() * x.size(0)
        total += x.size(0)
        iou_sum += iou_per_class(logits, y, num_classes).nan_to_num(0)
    return loss_sum / total, iou_sum / len(loader)
```

Run this for 10-30 epochs on the synthetic dataset and watch mIoU climb past 0.9 for the shape classes. Note the `nan_to_num(0)` treats classes absent from a batch as zero; for accurate per-class IoU, mask by presence and use `torch.nanmean` across batches at evaluation time rather than averaging here.

## Use It

For production, `segmentation_models_pytorch` (“smp”) wraps every standard segmentation architecture with any torchvision or timm backbone. Three lines:

```
import segmentation_models_pytorch as smp

model = smp.Unet(
    encoder_name="resnet34",
    encoder_weights="imagenet",
    in_channels=3,
    classes=3,
)
```

Also worth knowing for real work: - **DeepLabV3+** replaces max-pool-based downsampling with dilated convs so the bottleneck keeps resolution; faster boundaries on satellite and driving data. - **SegFormer** swaps the conv encoder for a hierarchical transformer; current SOTA on many benchmarks. - **Mask2Former** / **OneFormer** unify semantic, instance, and panoptic segmentation in a single architecture.

All three are drop-in replacements in smp or transformers with the same data loader.

**This chapter ships an artifact.** The course version of this lesson produces a reusable prompt or agent skill. It lives in the repository, ready to install: <https://github.com/rohitg00/ai-engineering-from-scratch/tree/main/phases/04-computer-vision/07-semantic-segmentation-unet>

## Exercises

Starter code and the lesson’s working implementation: <https://github.com/rohitg00/ai-engineering-from-scratch/tree/main/phases/04-computer-vision/07-semantic-segmentation-unet/code>

1. **(Easy)** Implement `bce_dice_loss` for a binary segmentation task (foreground vs background). Verify on a synthetic two-class dataset that the combined loss converges faster than BCE alone when the foreground is 5% of pixels.
2. **(Medium)** Replace the `nn.Upsample + conv` up-block with a `nn.ConvTranspose2d` up-block. Train both on the synthetic dataset and compare mIoU. Observe where checkerboard artifacts appear in the transposed-conv version.
3. **(Hard)** Take a real segmentation dataset (Oxford-IIIT Pets, Cityscapes mini split, or a medical subset) and train the U-Net to within 2 IoU points of the `smp.Unet` reference. Report per-class IoU and identify which classes benefit most from adding Dice to the loss.

## Key Terms

Term	What people say	What it actually means
Semantic segmentation	“Label every pixel”	Per-pixel classification into C classes; instances of the same class merge
Instance segmentation	“Label every object”	Separates distinct instances of the same class; foreground-only
Panoptic segmentation	“Semantic + instance”	Every pixel gets a class; every thing instance also gets a unique id
Skip connection	“U-Net bridge”	Concatenation of encoder features into matching-resolution decoder features; preserves high-frequency detail
Transposed conv	“Deconvolution”	Learnable upsampling; can produce checkerboard artifacts
Dice loss	“Overlap loss”	1 - 2
mIoU	“Mean intersection over union”	Average IoU across classes; the community-standard metric for segmentation
Boundary F1	“Boundary accuracy”	F1 score computed on boundary pixels only; matters for precision-critical tasks

## Further Reading

- U-Net: Convolutional Networks for Biomedical Image Segmentation (Ronneberger et al., 2015) — the original paper; the figure everyone copies is on page 2
- Fully Convolutional Networks (Long et al., 2015) — the paper that first made segmentation an end-to-end conv problem
- `segmentation_models_pytorch` — the reference for production segmentation; every standard architecture plus every standard loss

- Lessons learned from training SOTA segmentation (kaggle.com competitions) — a walk-through of why TTA, pseudo-labeling, and class weights matter on real data

**Continue online.** The living edition of this chapter has more than the page can hold:

- Animated, interactive figures and the web text: <https://aiengineeringfromscratch.com/lesson?path=phases/04-computer-vision/07-semantic-segmentation-unet>
- Runnable code for every step: <https://github.com/rohitg00/ai-engineering-from-scratch/tree/main/phases/04-computer-vision/07-semantic-segmentation-unet/code>
- The chapter quiz, graded in the browser: <https://aiengineeringfromscratch.com/lesson?path=phases/04-computer-vision/07-semantic-segmentation-unet>

The repository moves faster than any printing. When the book and the repo disagree, trust the repo.



# Instance Segmentation — Mask R-CNN

---

Add a tiny mask branch to a Faster R-CNN detector and you have instance segmentation. The hard part is RoIAlign, and it is harder than it looks.

**Type:** Build + Learn **Languages:** Python **Prerequisites:** Phase 4 Lesson 06 (YOLO), Phase 4 Lesson 07 (U-Net) **Time:** ~75 minutes

## Learning Objectives

- Trace the Mask R-CNN architecture end-to-end: backbone, FPN, RPN, RoIAlign, box head, mask head
- Implement RoIAlign from scratch and explain why RoIPool is no longer used
- Use the torchvision maskrcnn_resnet50_fpn_v2 pretrained model for production-quality instance masks and read its output format correctly
- Fine-tune Mask R-CNN on a small custom dataset by replacing the box and mask heads and keeping the backbone frozen

## The Problem

Semantic segmentation gives you one mask per class. Instance segmentation gives you one mask per object, even when two objects share a class. Counting individuals, tracking across frames, and measuring things (the bounding box of each brick in a wall, each cell in a microscope image) all demand instance segmentation.

Mask R-CNN (He et al., 2017) solved this by reframing instance segmentation as detection-plus-a-mask. The design was so clean that for the next five years almost every instance segmentation paper was a Mask R-CNN variant, and the torchvision implementation is still the production default for small to medium datasets.

The hard engineering problem is sampling: how do you crop a fixed-size feature region out of a proposal box whose corners do not align with pixel boundaries? Getting that wrong costs tenths of a mAP point everywhere. RoIAlign is the answer.

## The Concept

### The architecture

**Diagram.** Rendered live in the web edition: <https://aiengineeringfromscratch.com/lesson?path=phases/04-computer-vision/08-instance-segmentation-mask-rcnn>

Five pieces to understand:

1. **Backbone** — ResNet-50 or ResNet-101 trained on ImageNet. Produces a hierarchy of feature maps at strides 4, 8, 16, 32.

2. **FPN (Feature Pyramid Network)** — top-down + lateral connections that give every level C channels of semantic-rich features. Detection queries the FPN level matching the object size.
3. **RPN (Region Proposal Network)** — a small conv head that, at every anchor position, predicts “is there an object here?” and “how do I refine the box?”. Produces ~1000 proposals per image.
4. **RoIAlign** — samples a fixed-size (e.g. 7x7) feature patch from any box on any FPN level. Bilinear sampling, no quantisation.
5. **Heads** — two-layer box head that refines the box and picks a class, plus a small conv head that outputs a 28x28 binary mask for each proposal.

## Why RoIAlign, not RoIPool

The original Fast R-CNN used RoIPool, which splits a proposal box into a grid, takes the maximum feature in each cell, and rounds all coordinates to integers. That rounding misaligns the feature map from the input pixel coordinates by up to a full feature-map pixel — small on a 224x224 image, catastrophic when the feature map is stride 32.

RoIPool:

```
box (34.7, 51.3, 98.2, 142.9)
round -> (34, 51, 98, 142)
split grid -> round each cell boundary
misalignment accumulates at every step
```

RoIAlign:

```
box (34.7, 51.3, 98.2, 142.9)
sample at exact float coordinates using bilinear interpolation
no rounding anywhere
```

RoIAlign lifts mask AP by 3-4 points on COCO for free. Every detector that cares about localisation now uses it — YOLOv7 seg, RT-DETR, Mask2Former alike.

## The RPN in one paragraph

At every position of a feature map, place K anchor boxes of different sizes and shapes. Predict an objectness score for each anchor and a regression offset to turn the anchor into a better-fitting box. Keep the top ~1,000 boxes by score, apply NMS at IoU 0.7, and hand the survivors to the heads. The RPN is trained with its own mini-loss — the same structure as the YOLO loss from Lesson 6, just with two classes (object / no object).

## The mask head

For each proposal (after RoIAlign) the mask head is a tiny FCN: four 3x3 convs, a 2x deconv, a final 1x1 conv that produces `num_classes` output channels at 28x28 resolution. Only the channel corresponding to the predicted class is kept; the others are ignored. This decouples mask prediction from classification.

Upsample the 28x28 mask to the proposal’s original pixel size to produce the final binary mask.

## Losses

Mask R-CNN has four losses added together:

$$L = L_{\text{rpn_cls}} + L_{\text{rpn_box}} + L_{\text{box_cls}} + L_{\text{box_reg}} + L_{\text{mask}}$$

- `L_rpn_cls`, `L_rpn_box` — objectness + box regression for the RPN proposals.
- `L_box_cls` — cross-entropy over (C+1) classes (including background) on the head's classifier.
- `L_box_reg` — smooth L1 on the head's box refinement.
- `L_mask` — per-pixel binary cross-entropy on the 28x28 mask output.

Each loss has its own default weight; the torchvision implementation exposes them as constructor arguments.

## Output format

`torchvision.models.detection.maskrcnn_resnet50_fpn_v2` returns a list of dicts, one per image:

```
{
    "boxes": (N, 4) in (x1, y1, x2, y2) pixel coordinates,
    "labels": (N,) class IDs, 0 = background so indices are 1-based,
    "scores": (N,) confidence scores,
    "masks": (N, 1, H, W) float masks in [0, 1] – threshold at 0.5 for binary,
}
```

The mask is full image resolution already. The 28x28 head output has been upsampled internally.

**Interactive figure: cv3-roialign-sampling.** This one moves. Watch it animate and drag its controls in the web edition: <https://aiengineeringfromscratch.com/lesson?path=phases/04-computer-vision/08-instance-segmentation-mask-rcnn>

## Build It

### Step 1: RoIAlign from scratch

This is the one component of Mask R-CNN that is simpler to understand as code than as prose.

```
import torch
import torch.nn.functional as F

def roi_align_single(feature, box, output_size=7, spatial_scale=1 / 16.0):
    """
    feature: (C, H, W) single-image feature map
    box: (x1, y1, x2, y2) in original image pixel coordinates
    output_size: side of the output grid (7 for box head, 14 for mask head)
    spatial_scale: reciprocal of the feature map stride
    """
    C, H, W = feature.shape
    x1, y1, x2, y2 = [c * spatial_scale - 0.5 for c in box]
    bin_w = (x2 - x1) / output_size
    bin_h = (y2 - y1) / output_size

    grid_y = torch.linspace(y1 + bin_h / 2, y2 - bin_h / 2, output_size)
    grid_x = torch.linspace(x1 + bin_w / 2, x2 - bin_w / 2, output_size)
    yy, xx = torch.meshgrid(grid_y, grid_x, indexing="ij")

    gx = 2 * (xx + 0.5) / W - 1
    gy = 2 * (yy + 0.5) / H - 1
```

```

grid = torch.stack([gx, gy], dim=-1).unsqueeze(0)
sampled = F.grid_sample(feature.unsqueeze(0), grid, mode="bilinear",
                        align_corners=False)
return sampled.squeeze(0)

```

Every number is at a bilinearly-sampled position. No rounding, no quantisation, no dropped gradients.

## Step 2: Compare to torchvision's RoIAlign

```

from torchvision.ops import roi_align

feature = torch.randn(1, 16, 50, 50)
boxes = torch.tensor([[0, 10, 20, 100, 90]], dtype=torch.float32) # (batch_idx,
    ↪ x1, y1, x2, y2)

ours = roi_align_single(feature[0], boxes[0, 1:].tolist(), output_size=7,
    ↪ spatial_scale=1/4)
theirs = roi_align(feature, boxes, output_size=(7, 7), spatial_scale=1/4,
    ↪ sampling_ratio=1, aligned=True)[0]

print(f"shape ours: {tuple(ours.shape)}")
print(f"shape theirs: {tuple(theirs.shape)}")
print(f"max|diff|: {(ours - theirs).abs().max().item():.3e}")

```

With `sampling_ratio=1` and `aligned=True`, the two match to within  $1e-5$ .

## Step 3: Load a pretrained Mask R-CNN

```

import torch
from torchvision.models.detection import maskrcnn_resnet50_fpn_v2,
    ↪ MaskRCNN_ResNet50_FPN_V2_Weights

model = maskrcnn_resnet50_fpn_v2(weights=MaskRCNN_ResNet50_FPN_V2_Weights.DEFAULT)
model.eval()
print(f"params: {sum(p.numel() for p in model.parameters()):,}")
print(f"classes (including background):
    ↪ {len(model.roi_heads.box_predictor.cls_score.out_features * [0])}")

```

46M parameters, 91 classes (COCO). The first class (id 0) is background; everything the model actually detects starts at id 1.

## Step 4: Run inference

```

with torch.no_grad():
    x = torch.randn(3, 400, 600)
    predictions = model([x])
p = predictions[0]
print(f"boxes: {tuple(p['boxes'].shape)}")
print(f"labels: {tuple(p['labels'].shape)}")
print(f"scores: {tuple(p['scores'].shape)}")
print(f"masks: {tuple(p['masks'].shape)}")

```

The mask tensor is shape (N, 1, H, W). Threshold at 0.5 to get a binary mask per object:

```
binary_masks = (p['masks'] > 0.5).squeeze(1)  # (N, H, W) boolean
```

## Step 5: Swap the heads for a custom class count

The common fine-tuning recipe: reuse the backbone, FPN, and RPN; replace the two classifier heads.

```
from torchvision.models.detection.faster_rcnn import FastRCNNPredictor
from torchvision.models.detection.mask_rcnn import MaskRCNNPredictor

def build_custom_maskrcnn(num_classes):
    model =
    ↪ maskrcnn_resnet50_fpn_v2(weights=MaskRCNN_ResNet50_FPN_V2_Weights.DEFAULT)
    in_features = model.roi_heads.box_predictor.cls_score.in_features
    model.roi_heads.box_predictor = FastRCNNPredictor(in_features, num_classes)
    in_features_mask = model.roi_heads.mask_predictor.conv5_mask.in_channels
    hidden_layer = 256
    model.roi_heads.mask_predictor = MaskRCNNPredictor(in_features_mask,
    ↪ hidden_layer, num_classes)
    return model
```

```
custom = build_custom_maskrcnn(num_classes=5)
print(f"custom cls_score.out_features:
    ↪ {custom.roi_heads.box_predictor.cls_score.out_features}")
```

num_classes must include the background class, so a dataset with 4 object classes uses num_classes=5.

## Step 6: Freeze what does not need training

On small datasets, freeze the backbone and the FPN. Only the RPN objectness + regression and the two heads learn.

```
def freeze_backbone_and_fpn(model):
    # torchvision Mask R-CNN packs the FPN inside `model.backbone` (as
    # `model.backbone.fpn`), so iterating `model.backbone.parameters()` covers
    # both the ResNet feature layers and the FPN lateral/output convs.
    for p in model.backbone.parameters():
        p.requires_grad = False
    return model
```

```
custom = freeze_backbone_and_fpn(custom)
trainable = sum(p.numel() for p in custom.parameters() if p.requires_grad)
print(f"trainable after freeze: {trainable:,}")
```

On 500-image datasets this is the difference between convergence and overfitting.

## Use It

The full training loop for Mask R-CNN in torchvision is 40 lines and does not change meaningfully between tasks — swap datasets and go.

```
def train_step(model, images, targets, optimizer):
    model.train()
    loss_dict = model(images, targets)
```

```

losses = sum(loss for loss in loss_dict.values())
optimizer.zero_grad()
losses.backward()
optimizer.step()
return {k: v.item() for k, v in loss_dict.items()}

```

The targets list must have per-image dicts with boxes, labels, and masks (as (num_instances, H, W) binary tensors). The model returns a dict of four losses during training and a list of predictions during eval, keyed on `model.training`.

The pycocotools evaluator produces mAP@IoU=0.5:0.95 both for boxes and for masks; you need both numbers to know if the box head or the mask head is the bottleneck.

**This chapter ships an artifact.** The course version of this lesson produces a reusable prompt or agent skill. It lives in the repository, ready to install: <https://github.com/rohitg00/ai-engineering-from-scratch/tree/main/phases/04-computer-vision/08-instance-segmentation-mask-rcnn>

## Exercises

Starter code and the lesson’s working implementation: <https://github.com/rohitg00/ai-engineering-from-scratch/tree/main/phases/04-computer-vision/08-instance-segmentation-mask-rcnn/code>

1. **(Easy)** Verify your RoIAlign against `torchvision.ops.roi_align` on 100 random boxes. Report the max absolute difference. Also run RoIPool (pre-2017 behaviour) and show it diverges by ~1-2 feature-map pixels on boxes near the border.
2. **(Medium)** Fine-tune `maskrcnn_resnet50_fpn_v2` on a 50-image custom dataset (any two classes: balloons, fish, pothole, logos). Freeze the backbone, train for 20 epochs, report mask AP@0.5.
3. **(Hard)** Replace Mask R-CNN’s mask head with one that predicts at 56x56 instead of 28x28. Measure mAP@IoU=0.75 before and after. Explain why the gain (or lack of one) matches the expected boundary-precision / memory trade-off.

## Key Terms

Term	What people say	What it actually means
Mask R-CNN	“Detection plus masks”	Faster R-CNN + a small FCN head that predicts a 28x28 mask per proposal per class
FPN	“Feature pyramid”	Top-down + lateral connections that give every stride level C channels of semantic-rich features
RPN	“Region proposer”	A small conv head that produces ~1000 object/no-object proposals per image
RoIAlign	“No-rounding crop”	Bilinearly samples a fixed-size feature grid from any float-coordinate box
RoIPool	“Pre-2017 crop”	Same purpose as RoIAlign but rounds box coordinates; obsolete
Mask AP	“Instance mAP”	Average precision computed with mask IoU instead of box IoU; the COCO instance segmentation metric

Term	What people say	What it actually means
Binary mask head	“Per-class mask”	Predicts one binary mask per class for each proposal; only the predicted class’s channel is kept
Background class	“Class 0”	The catch-all “no object” class; indices for real classes start at 1

## Further Reading

- Mask R-CNN (He et al., 2017) — the paper; section 3 on RoIAlign is the critical read
- FPN: Feature Pyramid Networks (Lin et al., 2017) — the FPN paper; every modern detector uses it
- torchvision Mask R-CNN tutorial — the reference for the fine-tuning loop
- Detectron2 model zoo — production implementations with trained weights for nearly every detection and segmentation variant

**Continue online.** The living edition of this chapter has more than the page can hold:

- Animated, interactive figures and the web text: <https://aiengineeringfromscratch.com/lesson?path=phases/04-computer-vision/08-instance-segmentation-mask-rcnn>
- Runnable code for every step: <https://github.com/rohitg00/ai-engineering-from-scratch/tree/main/phases/04-computer-vision/08-instance-segmentation-mask-rcnn/code>
- The chapter quiz, graded in the browser: <https://aiengineeringfromscratch.com/lesson?path=phases/04-computer-vision/08-instance-segmentation-mask-rcnn>

The repository moves faster than any printing. When the book and the repo disagree, trust the repo.

# Image Generation — GANs

---

A GAN is two neural networks in a fixed game. One draws, one critiques. They get better together until the drawings fool the critic.

**Type:** Build **Languages:** Python **Prerequisites:** Phase 4 Lesson 03 (CNNs), Phase 3 Lesson 06 (Optimizers), Phase 3 Lesson 07 (Regularization) **Time:** ~75 minutes

## Learning Objectives

- Explain the minimax game between generator and discriminator and why the equilibrium corresponds to  $p_{\text{model}} = p_{\text{data}}$
- Implement a DCGAN in PyTorch and get it to generate coherent 32x32 synthetic images in under 60 lines
- Stabilise GAN training with the three standard tricks: non-saturating loss, spectral norm, TTUR (two-timescale update rule)
- Read training curves that distinguish healthy convergence from mode collapse, oscillation, and discriminator-wins-completely

## The Problem

Classification teaches a network to map images to labels. Generation inverts the problem: sample new images that look like they came from the same distribution. There is no “correct” output you can diff against; there is only a distribution you want to mimic.

The standard loss functions (MSE, cross-entropy) cannot measure “did this sample come from the real distribution.” Minimising per-pixel error produces blurry averages, not realistic samples. The breakthrough was to learn the loss: train a second network whose job is to tell real from fake, and use its judgement to push the generator.

GANs (Goodfellow et al., 2014) defined that framework. By 2018 StyleGAN was producing 1024x1024 faces indistinguishable from photographs. Diffusion models have since taken the throne on quality and controllability, but every trick that makes diffusion practical — normalisation choices, latent spaces, feature losses — was first understood on GANs.

## The Concept

### The two networks

**Diagram.** Rendered live in the web edition: <https://aiengineeringfromscratch.com/lesson?path=phases/04-computer-vision/09-image-generation-gans>

The **generator**  $G$  takes a vector of noise  $z$  and outputs an image. The **discriminator**  $D$  takes an image and outputs a single scalar: the probability that the image is real.



## The game

G wants D to be wrong. D wants to be right. Formally:

$$\min_G \max_D \mathbb{E}_x[\log D(x)] + \mathbb{E}_z[\log(1 - D(G(z)))]$$

Read right to left: D is maximising accuracy on real ( $\log D(\text{real})$ ) and fake ( $\log(1 - D(\text{fake}))$ ) images. G is minimising D's accuracy on fakes — it wants  $D(G(z))$  to be high.

Goodfellow proved that this minimax has a global equilibrium where  $p_G = p_{\text{data}}$ , D outputs 0.5 everywhere, and the Jensen-Shannon divergence between generated and real distributions is zero. The hard part is getting there.

## Non-saturating loss

The form above is numerically unstable. Early in training,  $D(G(z))$  is near zero for every fake, so  $\log(1 - D(G(z)))$  has vanishing gradients with respect to G. The fix: flip G's loss.

$$\begin{aligned} L_D &= -\mathbb{E}_x[\log D(x)] - \mathbb{E}_z[\log(1 - D(G(z)))] \\ L_G &= -\mathbb{E}_z[\log D(G(z))] \quad \# \text{ non-saturating} \end{aligned}$$

Now when  $D(G(z))$  is near zero, G's loss is large and its gradient is informative. Every modern GAN trains with this variant.

## DCGAN architecture rules

Radford, Metz, Chintala (2015) distilled years of failed experiments into five rules that make GAN training stable:

1. Replace pooling with strided convs (both nets).
2. Use batch norm in both generator and discriminator, except output of G and input of D.
3. Remove fully connected layers on deeper architectures.
4. G uses ReLU on all layers except output (tanh for output in  $[-1, 1]$ ).
5. D uses LeakyReLU (negative_slope=0.2) on all layers.

Every modern conv-based GAN (StyleGAN, BigGAN, GigaGAN) still starts from these rules and replaces pieces one at a time.

## Failure modes and their signatures

**Diagram.** Rendered live in the web edition: <https://aiengineeringfromscratch.com/lesson?path=phases/04-computer-vision/09-image-generation-gans>

- **Mode collapse:** G finds one image that fools D and produces only that. Fix: add mini-batch discrimination, spectral norm, or label-conditioning.
- **Discriminator wins:** D becomes too strong too fast, G's gradients vanish. Fix: smaller D, lower D learning rate, or apply label smoothing on the real labels.
- **Oscillation:** the two nets trade wins without ever approaching equilibrium. Fix: TTUR (D learns faster than G by a factor of 2-4), or switch to Wasserstein loss.

## Evaluation

GANs have no ground truth, so how do you know they are working?

- **Sample inspection** — just look at 64 samples at the end of every epoch. Non-negotiable.
- **FID (Fréchet Inception Distance)** — distance between Inception-v3 feature distributions of real and generated sets. Lower is better. Community standard.
- **Inception Score** — older, more brittle; prefer FID.

- **Precision/Recall for generative models** — measures quality (precision) and coverage (recall) separately. More informative than FID alone.

For a small synthetic-data run, sample inspection is enough.

**Interactive figure: cv-gan-image.** This one moves. Watch it animate and drag its controls in the web edition: <https://aiengineeringfromscratch.com/lesson?path=phases/04-computer-vision/09-image-generation-gans>

## Build It

### Step 1: Generator

A small DCGAN generator that takes 64-dim noise and produces a 32x32 image.

```
import torch
import torch.nn as nn

class Generator(nn.Module):
    def __init__(self, z_dim=64, img_channels=3, feat=64):
        super().__init__()
        self.net = nn.Sequential(
            nn.ConvTranspose2d(z_dim, feat * 4, kernel_size=4, stride=1, padding=0,
↪ bias=False),
            nn.BatchNorm2d(feat * 4),
            nn.ReLU(inplace=True),
            nn.ConvTranspose2d(feat * 4, feat * 2, kernel_size=4, stride=2,
↪ padding=1, bias=False),
            nn.BatchNorm2d(feat * 2),
            nn.ReLU(inplace=True),
            nn.ConvTranspose2d(feat * 2, feat, kernel_size=4, stride=2, padding=1,
↪ bias=False),
            nn.BatchNorm2d(feat),
            nn.ReLU(inplace=True),
            nn.ConvTranspose2d(feat, img_channels, kernel_size=4, stride=2,
↪ padding=1, bias=False),
            nn.Tanh(),
        )

    def forward(self, z):
        return self.net(z.view(z.size(0), -1, 1, 1))
```

Four transposed convs, each with `kernel_size=4`, `stride=2`, `padding=1` so they cleanly double spatial size. Output activations in `[-1, 1]` via `tanh`.

### Step 2: Discriminator

Mirror of the generator. LeakyReLU, strided convs, ends with a scalar logit.

```
class Discriminator(nn.Module):
    def __init__(self, img_channels=3, feat=64):
        super().__init__()
        self.net = nn.Sequential(
            nn.Conv2d(img_channels, feat, kernel_size=4, stride=2, padding=1),
            nn.LeakyReLU(0.2, inplace=True),
```

```

        nn.Conv2d(feats, feat * 2, kernel_size=4, stride=2, padding=1,
        ↪ bias=False),
        nn.BatchNorm2d(feats * 2),
        nn.LeakyReLU(0.2, inplace=True),
        nn.Conv2d(feats * 2, feat * 4, kernel_size=4, stride=2, padding=1,
        ↪ bias=False),
        nn.BatchNorm2d(feats * 4),
        nn.LeakyReLU(0.2, inplace=True),
        nn.Conv2d(feats * 4, 1, kernel_size=4, stride=1, padding=0),
    )

    def forward(self, x):
        return self.net(x).view(-1)

```

The last conv reduces a 4x4 feature map to 1x1. Output is a single scalar per image; apply sigmoid only during loss computation.

### Step 3: Training step

Alternate: update D once, then G once, every batch.

```

import torch.nn.functional as F

def train_step(G, D, real, z, opt_g, opt_d, device):
    real = real.to(device)
    bs = real.size(0)

    # D step
    opt_d.zero_grad()
    d_real = D(real)
    d_fake = D(G(z).detach())
    loss_d = (F.binary_cross_entropy_with_logits(d_real, torch.ones_like(d_real))
              + F.binary_cross_entropy_with_logits(d_fake,
              ↪ torch.zeros_like(d_fake)))
    loss_d.backward()
    opt_d.step()

    # G step
    opt_g.zero_grad()
    d_fake = D(G(z))
    loss_g = F.binary_cross_entropy_with_logits(d_fake, torch.ones_like(d_fake))
    loss_g.backward()
    opt_g.step()

    return loss_d.item(), loss_g.item()

```

`G(z).detach()` in the D step is critical: we do not want gradients flowing into G during its update. Forgetting that is the classic beginner bug.

### Step 4: Full training loop on synthetic shapes

```

from torch.utils.data import DataLoader, TensorDataset
import numpy as np

def synthetic_images(num=2000, size=32, seed=0):

```

```

rng = np.random.default_rng(seed)
imgs = np.zeros((num, 3, size, size), dtype=np.float32) - 1.0
for i in range(num):
    r = rng.uniform(6, 12)
    cx, cy = rng.uniform(r, size - r, size=2)
    yy, xx = np.meshgrid(np.arange(size), np.arange(size), indexing="ij")
    mask = (xx - cx) ** 2 + (yy - cy) ** 2 < r ** 2
    color = rng.uniform(-0.5, 1.0, size=3)
    for c in range(3):
        imgs[i, c][mask] = color[c]
return torch.from_numpy(imgs)

device = "cuda" if torch.cuda.is_available() else "cpu"
data = synthetic_images()
loader = DataLoader(TensorDataset(data), batch_size=64, shuffle=True)

G = Generator(z_dim=64, img_channels=3, feat=32).to(device)
D = Discriminator(img_channels=3, feat=32).to(device)
opt_g = torch.optim.Adam(G.parameters(), lr=2e-4, betas=(0.5, 0.999))
opt_d = torch.optim.Adam(D.parameters(), lr=2e-4, betas=(0.5, 0.999))

for epoch in range(10):
    for (batch,) in loader:
        z = torch.randn(batch.size(0), 64, device=device)
        ld, lg = train_step(G, D, batch, z, opt_g, opt_d, device)
    print(f"epoch {epoch} D {ld:.3f} G {lg:.3f}")

```

Adam(lr=2e-4, betas=(0.5, 0.999)) is the DCGAN default — the low beta1 keeps the momentum term from stabilising the adversarial game too much.

## Step 5: Sampling

```

@torch.no_grad()
def sample(G, n=16, z_dim=64, device="cpu"):
    G.eval()
    z = torch.randn(n, z_dim, device=device)
    imgs = G(z)
    imgs = (imgs + 1) / 2
    return imgs.clamp(0, 1)

```

Always switch to eval mode before sampling. For DCGAN this matters because batch norm running stats are used instead of the batch's stats.

## Step 6: Spectral normalisation

A drop-in replacement for BN in the discriminator that guarantees the network is 1-Lipschitz. Fixes most “D wins too hard” failures.

```

from torch.nn.utils import spectral_norm

def build_sn_discriminator(img_channels=3, feat=64):
    return nn.Sequential(
        spectral_norm(nn.Conv2d(img_channels, feat, 4, 2, 1)),
        nn.LeakyReLU(0.2, inplace=True),
        spectral_norm(nn.Conv2d(feat, feat * 2, 4, 2, 1)),

```

```

nn.LeakyReLU(0.2, inplace=True),
spectral_norm(nn.Conv2d(feats * 2, feat * 4, 4, 2, 1)),
nn.LeakyReLU(0.2, inplace=True),
spectral_norm(nn.Conv2d(feats * 4, 1, 4, 1, 0)),
)

```

Swap `Discriminator` for `build_sn_discriminator()` and you often do not need the TTUR trick. Spectral norm is the easiest single robustness upgrade you can apply.

## Use It

For serious generation, use pretrained weights or switch to diffusion. Two standard libraries:

- `torch_fidelity` computes FID / IS on your generator without writing custom eval code.
- `pytorch-gan-zoo` (legacy) and `StudioGAN` ship tested implementations of DCGAN, WGAN-GP, SN-GAN, StyleGAN, and BigGAN.

In 2026, GANs are still the best choice for: real-time image generation (latency <10 ms), style transfer, image-to-image translation with precise control (Pix2Pix, CycleGAN). Diffusion wins on photorealism and text conditioning.

**This chapter ships an artifact.** The course version of this lesson produces a reusable prompt or agent skill. It lives in the repository, ready to install: <https://github.com/rohitg00/ai-engineering-from-scratch/tree/main/phases/04-computer-vision/09-image-generation-gans>

## Exercises

Starter code and the lesson's working implementation: <https://github.com/rohitg00/ai-engineering-from-scratch/tree/main/phases/04-computer-vision/09-image-generation-gans/code>

1. **(Easy)** Train the DCGAN above on the synthetic circle dataset and save a grid of 16 samples at the end of each epoch. By which epoch do the generated circles become clearly circular?
2. **(Medium)** Replace the discriminator's batch norm with spectral norm. Train both versions side by side. Which one converges faster? Which one has lower variance across three seeds?
3. **(Hard)** Implement a conditional DCGAN: feed the class label into both G and D (concat one-hot to the noise in G, concat a class embedding channel in D). Train on the synthetic "circles vs squares" dataset from lesson 7 and show that class conditioning works by sampling with specific labels.

## Key Terms

Term	What people say	What it actually means
Generator (G)	"The draws-stuff net"	Maps noise to images; trained to fool the discriminator
Discriminator (D)	"The critic"	Binary classifier; trained to distinguish real from generated images
Minimax	"The game"	min over G, max over D of an adversarial loss; equilibrium is $p_G = p_{\text{data}}$

Term	What people say	What it actually means
Non-saturating loss	“The numerically sane version”	G’s loss is $-\log(D(G(z)))$ instead of $\log(1 - D(G(z)))$ to avoid vanishing gradients early in training
Mode collapse	“Generator makes one thing”	G produces only a small subset of the data distribution; fix with SN, minibatch discrimination, or larger batch
TTUR	“Two learning rates”	D learns faster than G, typically by a factor of 2-4; stabilises training
Spectral norm	“1-Lipschitz layer”	A weight-normalisation that bounds each layer’s Lipschitz constant; stops D from becoming arbitrarily steep
FID	“Fréchet Inception Distance”	Distance between Inception-v3 feature distributions of real and generated sets; the standard evaluation metric

## Further Reading

- Generative Adversarial Networks (Goodfellow et al., 2014) — the paper that started it all
- DCGAN (Radford, Metz, Chintala, 2015) — the architecture rules that made GANs trainable
- Spectral Normalization for GANs (Miyato et al., 2018) — the single most useful stabilisation trick
- StyleGAN3 (Karras et al., 2021) — the SOTA GAN; reads like a greatest-hits album of every trick from the last decade

**Continue online.** The living edition of this chapter has more than the page can hold:

- Animated, interactive figures and the web text: <https://aiengineeringfromscratch.com/lesson?path=phases/04-computer-vision/09-image-generation-gans>
- Runnable code for every step: <https://github.com/rohitg00/ai-engineering-from-scratch/tree/main/phases/04-computer-vision/09-image-generation-gans/code>
- The chapter quiz, graded in the browser: <https://aiengineeringfromscratch.com/lesson?path=phases/04-computer-vision/09-image-generation-gans>

The repository moves faster than any printing. When the book and the repo disagree, trust the repo.

# Image Generation — Diffusion Models

---

A diffusion model learns to denoise. Train it to remove a tiny bit of noise from a noisy image, repeat that backwards a thousand times, and you have an image generator.

**Type:** Build **Languages:** Python **Prerequisites:** Phase 4 Lesson 07 (U-Net), Phase 1 Lesson 06 (Probability), Phase 3 Lesson 06 (Optimizers) **Time:** ~75 minutes

## Learning Objectives

- Derive the forward noising process  $x_0 \rightarrow x_1 \rightarrow \dots \rightarrow x_T$  and explain why the closed-form  $q(x_t | x_0)$  holds for any  $t$
- Implement a DDPM-style training objective that regresses the noise added at each step, and a sampler that walks back from pure noise to an image
- Build a time-conditioned U-Net (small enough to train on CPU) that predicts the noise for any timestep
- Explain the difference between DDPM and DDIM sampling, and when each is appropriate (Lesson 23 covers flow matching and rectified flow in depth)

## The Problem

GANs generate one-shot: noise in, image out, one forward pass. They are fast and hard to train. Diffusion models generate iteratively: start from pure noise, denoise in small steps, image emerges. They are slow and easy to train. For the last five years the latter property has dominated: any small team can train a diffusion model and get reasonable samples; GAN training is a craft you learn over years of failed runs.

Beyond training stability, diffusion's iterative structure is what unlocks everything modern image generation does: text conditioning, inpainting, image editing, super-resolution, controllable style. Each step of the sampling loop is a place to inject a new constraint. That hook is why Stable Diffusion, Imagen, DALL-E 3, Midjourney, and every controllable image model you will use are all diffusion-based.

This lesson builds the minimal DDPM: forward noising, backward denoising, training loop. The next lesson (Stable Diffusion) wires it into a production system with a VAE, a text encoder, and classifier-free guidance.

## The Concept

### The forward process

Take an image  $x_0$ . Add a tiny amount of Gaussian noise to get  $x_1$ . Add a tiny amount more to get  $x_2$ . Keep going for  $T$  steps until  $x_T$  is nearly indistinguishable from pure Gaussian noise.

$$q(x_t | x_{\{t-1\}}) = N(x_t; \sqrt{1 - \beta_t} * x_{\{t-1\}}, \beta_t * I)$$

$\beta_t$  is a small variance schedule, typically linear from 0.0001 to 0.02 over  $T=1000$  steps. Each step slightly shrinks the signal and injects fresh noise.

## The closed-form jump

Adding noise one step at a time is a Markov chain, but the math folds: you can sample  $x_t$  directly from  $x_0$  in one step.

```
Define alpha_t = 1 - beta_t
Define alpha_bar_t = prod_{s=1..t} alpha_s
```

Then:

$$q(x_t | x_0) = N(x_t; \sqrt{\alpha_{\bar{t}}} * x_0, (1 - \alpha_{\bar{t}}) * I)$$

Equivalently:

```
x_t = sqrt(alpha_bar_t) * x_0 + sqrt(1 - alpha_bar_t) * epsilon
where epsilon ~ N(0, I)
```

This single equation is the whole reason diffusion is practical. During training you pick a random  $t$ , sample  $x_t$  directly from  $x_0$ , and train in one step — no simulation of the full Markov chain needed.

## The reverse process

The forward process is fixed. The reverse process  $p(x_{t-1} | x_t)$  is what the neural network learns. Diffusion models do not predict  $x_{t-1}$  directly; they predict the noise  $\epsilon$  added at step  $t$ , and the math derives  $x_{t-1}$  from it.

**Diagram.** Rendered live in the web edition: <https://aiengineeringfromscratch.com/lesson?path=phases/04-computer-vision/10-image-generation-diffusion>

## The training loss

For every training step:

1. Sample a real image  $x_0$ .
2. Sample a timestep  $t$  uniformly from  $[1, T]$ .
3. Sample noise  $\epsilon \sim N(0, I)$ .
4. Compute  $x_t = \sqrt{\alpha_{\bar{t}}} * x_0 + \sqrt{1 - \alpha_{\bar{t}}} * \epsilon$ .
5. Predict  $\epsilon_{\theta}(x_t, t)$  with the network.
6. Minimise  $\| \epsilon - \epsilon_{\theta}(x_t, t) \|^2$ .

That is it. The neural network learns to predict the noise at any timestep. The loss is MSE. There is no adversarial game, no collapse, no oscillation.

## The sampler (DDPM)

To generate: start from  $x_T \sim N(0, I)$  and walk backwards one step at a time.

```
for t = T, T-1, ..., 1:
    eps = model(x_t, t)
    x_{t-1} = (1 / sqrt(alpha_t)) * (x_t - (beta_t / sqrt(1 - alpha_bar_t)) * eps)
    ↪ + sqrt(beta_t) * z
    where z ~ N(0, I) if t > 1, else 0
return x_0
```



The key is that even though the reverse conditional is not known in closed form in general, for this specific Gaussian forward process it is. The ugly-looking coefficients are what Bayes' rule gives you.

## Why 1000 steps

The forward noise schedule is chosen so each step adds just enough noise that the reverse step is nearly Gaussian. Too few steps and the reverse step is far from Gaussian, the network cannot model it well. Too many steps and sampling becomes expensive with diminishing gain.  $T=1000$  with a linear schedule is the DDPM default.

## DDIM: 20x faster sampling

Training is the same. Sampling changes. DDIM (Song et al., 2020) defines a deterministic reverse process that skips timesteps without retraining. Sampling in 50 steps with DDIM gives near-1000-step DDPM quality. Every production system uses DDIM or an even faster variant (DPM-Solver, Euler ancestral).

## Time conditioning

The network  $\epsilon_{\theta}(x_t, t)$  needs to know which timestep it is denoising. Modern diffusion models inject  $t$  via sinusoidal time embeddings (same idea as positional encoding in transformers) that get added to feature maps at every U-Net level.

```
t_embedding = sinusoidal(t)
feature_map += MLP(t_embedding)
```

Without time conditioning the network has to guess the noise level from the image itself, which works but is much less sample-efficient.

**Interactive figure: cv-diffusion-image.** This one moves. Watch it animate and drag its controls in the web edition: <https://aiengineeringfromscratch.com/lesson?path=phases/04-computer-vision/10-image-generation-diffusion>

# Build It

## Step 1: Noise schedule

```
import torch

def linear_beta_schedule(T=1000, beta_start=1e-4, beta_end=2e-2):
    return torch.linspace(beta_start, beta_end, T)

def precompute_schedule(betas):
    alphas = 1.0 - betas
    alphas_cumprod = torch.cumprod(alphas, dim=0)
    return {
        "betas": betas,
        "alphas": alphas,
        "alphas_cumprod": alphas_cumprod,
        "sqrt_alphas_cumprod": torch.sqrt(alphas_cumprod),
        "sqrt_one_minus_alphas_cumprod": torch.sqrt(1.0 - alphas_cumprod),
        "sqrt_recip_alphas": torch.sqrt(1.0 / alphas),
```

```
}

```

```
schedule = precompute_schedule(linear_beta_schedule(T=1000))
```

Precompute once, gather by index during training and sampling.

## Step 2: Forward diffusion (q_sample)

```
def q_sample(x0, t, noise, schedule):
    sqrt_a = schedule["sqrt_alphas_cumprod"][t].view(-1, 1, 1, 1)
    sqrt_one_minus_a = schedule["sqrt_one_minus_alphas_cumprod"][t].view(-1, 1, 1,
    ↪ 1)
    return sqrt_a * x0 + sqrt_one_minus_a * noise
```

One-line closed form. `t` is a batch of timesteps, one per image in the batch.

## Step 3: A tiny time-conditioned U-Net

```
import torch.nn as nn
import torch.nn.functional as F
import math

def timestep_embedding(t, dim=64):
    half = dim // 2
    freqs = torch.exp(-math.log(10000) * torch.arange(half, device=t.device) /
    ↪ half)
    args = t[:, None].float() * freqs[None]
    emb = torch.cat([args.sin(), args.cos()], dim=-1)
    return emb

class TinyUNet(nn.Module):
    def __init__(self, img_channels=3, base=32, t_dim=64):
        super().__init__()
        self.t_mlp = nn.Sequential(
            nn.Linear(t_dim, base * 4),
            nn.SiLU(),
            nn.Linear(base * 4, base * 4),
        )
        self.t_dim = t_dim
        self.enc1 = nn.Conv2d(img_channels, base, 3, padding=1)
        self.enc2 = nn.Conv2d(base, base * 2, 4, stride=2, padding=1)
        self.mid = nn.Conv2d(base * 2, base * 2, 3, padding=1)
        self.dec1 = nn.ConvTranspose2d(base * 2, base, 4, stride=2, padding=1)
        self.dec2 = nn.Conv2d(base * 2, img_channels, 3, padding=1)
        self.time_proj = nn.Linear(base * 4, base * 2)

    def forward(self, x, t):
        t_emb = timestep_embedding(t, self.t_dim)
        t_emb = self.t_mlp(t_emb)
        t_proj = self.time_proj(t_emb)[:, :, None, None]

        h1 = F.silu(self.enc1(x))
        h2 = F.silu(self.enc2(h1)) + t_proj
```

```

h3 = F.silu(self.mid(h2))
d1 = F.silu(self.dec1(h3))
d2 = torch.cat([d1, h1], dim=1)
return self.dec2(d2)

```

Two-level U-Net with time conditioning injected at the bottleneck. Scale up the depth and width for real images.

## Step 4: Training loop

```

def train_step(model, x0, schedule, optimizer, device, T=1000):
    model.train()
    x0 = x0.to(device)
    bs = x0.size(0)
    t = torch.randint(0, T, (bs,), device=device)
    noise = torch.randn_like(x0)
    x_t = q_sample(x0, t, noise, schedule)
    pred = model(x_t, t)
    loss = F.mse_loss(pred, noise)
    optimizer.zero_grad()
    loss.backward()
    optimizer.step()
    return loss.item()

```

That is the entire training loop. No GAN game, no specialised loss, one MSE call.

## Step 5: Sampler (DDPM)

```

@torch.no_grad()
def sample(model, schedule, shape, T=1000, device="cpu"):
    model.eval()
    x = torch.randn(shape, device=device)
    betas = schedule["betas"].to(device)
    sqrt_one_minus_a = schedule["sqrt_one_minus_alphas_cumprod"].to(device)
    sqrt_recip_alphas = schedule["sqrt_recip_alphas"].to(device)

    for t in reversed(range(T)):
        t_batch = torch.full((shape[0],), t, dtype=torch.long, device=device)
        eps = model(x, t_batch)
        coef = betas[t] / sqrt_one_minus_a[t]
        mean = sqrt_recip_alphas[t] * (x - coef * eps)
        if t > 0:
            x = mean + torch.sqrt(betas[t]) * torch.randn_like(x)
        else:
            x = mean
    return x

```

1000 forward passes to produce one batch of samples. In real code you would swap this for a DDIM 50-step sampler.

## Step 6: DDIM sampler (deterministic, ~20x faster)

```

@torch.no_grad()
def sample_ddim(model, schedule, shape, steps=50, T=1000, device="cpu", eta=0.0):

```

```

model.eval()
x = torch.randn(shape, device=device)
alphas_cumprod = schedule["alphas_cumprod"].to(device)

ts = torch.linspace(T - 1, 0, steps + 1).long()
for i in range(steps):
    t = ts[i]
    t_prev = ts[i + 1]
    t_batch = torch.full((shape[0],), t, dtype=torch.long, device=device)
    eps = model(x, t_batch)
    a_t = alphas_cumprod[t]
    a_prev = alphas_cumprod[t_prev] if t_prev >= 0 else torch.tensor(1.0,
↪ device=device)
    x0_pred = (x - torch.sqrt(1 - a_t) * eps) / torch.sqrt(a_t)
    sigma = eta * torch.sqrt((1 - a_prev) / (1 - a_t) * (1 - a_t / a_prev))
    dir_xt = torch.sqrt(1 - a_prev - sigma ** 2) * eps
    noise = sigma * torch.randn_like(x) if eta > 0 else 0
    x = torch.sqrt(a_prev) * x0_pred + dir_xt + noise
return x

```

$\eta=0$  is fully deterministic (same noise input always produces the same output).  $\eta=1$  recovers DDPM.

## Use It

For production work, use diffusers:

```

from diffusers import DDPM Scheduler, UNet2DModel

UNET = UNet2DModel(sample_size=32, in_channels=3, out_channels=3,
↪ layers_per_block=2)
scheduler = DDPM Scheduler(num_train_timesteps=1000)

```

The library ships ready-made schedulers (DDPM, DDIM, DPM-Solver, Euler, Heun), configurable U-Nets, pipelines for text-to-image and image-to-image, and LoRA fine-tuning helpers.

For research, k-diffusion (Katherine Crowson) has the most faithful reference implementations and the best sampling variants.

**This chapter ships an artifact.** The course version of this lesson produces a reusable prompt or agent skill. It lives in the repository, ready to install: <https://github.com/rohitg00/ai-engineering-from-scratch/tree/main/phases/04-computer-vision/10-image-generation-diffusion>

## Exercises

Starter code and the lesson's working implementation: <https://github.com/rohitg00/ai-engineering-from-scratch/tree/main/phases/04-computer-vision/10-image-generation-diffusion/code>

1. **(Easy)** Visualise the forward process: take one image and plot  $x_t$  at  $t$  in  $[0, 100, 250, 500, 750, 1000]$ . Verify that  $x_{1000}$  looks like pure Gaussian noise.
2. **(Medium)** Train the TinyUNet on the synthetic-circles dataset for 20 epochs and sample 16 circles. Compare DDPM (1000 steps) and DDIM (50 steps) sampling — do they produce similar images from the same noise seed?

3. **(Hard)** Implement a cosine noise schedule (Nichol & Dhariwal, 2021):  $\alpha_{\bar{t}} = \cos^2((t/T + s) / (1 + s) * \pi / 2)$ . Train the same model with linear and cosine schedules and show that cosine gives better samples at low step counts.

## Key Terms

Term	What people say	What it actually means
Forward process	“Add noise over time”	Fixed Markov chain that corrupts an image into Gaussian noise over T steps
Reverse process	“Denoise step by step”	Learned distribution that walks back from noise to image
Epsilon prediction	“Predict the noise”	The training target: $\epsilon_{\theta}(x_t, t)$ predicts the noise added at step t
Beta schedule	“Noise amounts”	Sequence of T small variances that define how much noise enters per step
$\alpha_{\bar{t}}$	“Cumulative retain factor”	Product of $(1 - \beta_s)$ up to time t; bigger t means less signal left
DDPM sampler	“Ancestral, stochastic”	Samples each $x_{t-1}$ from its conditional Gaussian; 1000 steps
DDIM sampler	“Deterministic, fast”	Rewrites sampling as a deterministic ODE; 20-100 steps with similar quality
Time conditioning	“Tell the model which t”	Sinusoidal embedding of t injected into the U-Net so it knows the noise level

## Further Reading

- Denoising Diffusion Probabilistic Models (Ho et al., 2020) — the paper that made diffusion practical and beat GANs on FID
- Improved DDPM (Nichol & Dhariwal, 2021) — cosine schedule and v-parameterisation
- DDIM (Song, Meng, Ermon, 2020) — the deterministic sampler that made real-time inference possible
- Elucidating the Design Space of Diffusion (Karras et al., 2022) — a unified view of every diffusion design choice; current best reference

**Continue online.** The living edition of this chapter has more than the page can hold:

- Animated, interactive figures and the web text: <https://aiengineeringfromscratch.com/lesson?path=phases/04-computer-vision/10-image-generation-diffusion>
- Runnable code for every step: <https://github.com/rohitg00/ai-engineering-from-scratch/tree/main/phases/04-computer-vision/10-image-generation-diffusion/code>
- The chapter quiz, graded in the browser: <https://aiengineeringfromscratch.com/lesson?path=phases/04-computer-vision/10-image-generation-diffusion>

The repository moves faster than any printing. When the book and the repo disagree, trust the repo.

# Stable Diffusion — Architecture & Fine-Tuning

---

Stable Diffusion is a DDPM that runs in the latent space of a pretrained VAE, conditioned on text via cross-attention, sampled with a fast deterministic ODE solver, and steered by classifier-free guidance.

**Type:** Learn + Use **Languages:** Python **Prerequisites:** Phase 4 Lesson 10 (Diffusion), Phase 7 Lesson 02 (Self-Attention) **Time:** ~75 minutes

## Learning Objectives

- Trace the five pieces of a Stable Diffusion pipeline: VAE, text encoder, U-Net, scheduler, safety checker — and what each of them actually does
- Explain latent diffusion and why training in a  $4 \times 64 \times 64$  latent space (instead of a  $3 \times 512 \times 512$  image) reduces compute by 48x without quality loss
- Use diffusers to generate images, run image-to-image, inpainting, and ControlNet-guided generation
- Fine-tune Stable Diffusion with LoRA on a small custom dataset and load the LoRA adapter at inference

## The Problem

Training a DDPM directly on  $512 \times 512$  RGB images is expensive. Every training step backprops through a U-Net that sees  $3 \times 512 \times 512 = 786,432$  input values, and sampling takes 50+ forward passes through that same U-Net. At the quality level of Stable Diffusion 1.5 (released 2022), pixel-space diffusion would need roughly 256 GPU-months of training and 10-30 seconds per image on a consumer GPU.

The trick that made open-weight text-to-image practical was **latent diffusion** (Rombach et al., CVPR 2022). Train a VAE that maps a  $3 \times 512 \times 512$  image to a  $4 \times 64 \times 64$  latent tensor and back, then do the diffusion in that latent space. Compute drops by  $(3 \times 512 \times 512) / (4 \times 64 \times 64) = 48x$ . Sampling drops from tens of seconds to under two seconds on the same GPU.

Almost every modern image-generation model — SDXL, SD3, FLUX, HunyuanDiT, Wan-Video — is a latent diffusion model with variations on the autoencoder, the denoiser (U-Net or DiT), and the text conditioning. Learn Stable Diffusion and you have learnt the template.

## The Concept

### The pipeline

**Diagram.** Rendered live in the web edition: <https://aiengineeringfromscratch.com/lesson?path=phases/04-computer-vision/11-stable-diffusion>

- **VAE** — frozen autoencoder. Encoder turns image into latents (used for img2img and training). Decoder turns latents back into an image.
- **Text encoder** — CLIP text encoder (SD 1.x/2.x), CLIP-L + CLIP-G (SDXL), or T5-XXL (SD3/FLUX). Produces a sequence of token embeddings.
- **U-Net** — the denoiser. Has cross-attention layers that attend from latents to the text embedding at every resolution level.
- **Scheduler** — the sampling algorithm (DDIM, Euler, DPM-Solver++). Picks sigmas, blends predicted noise back into the latent.
- **Safety checker** — optional NSFW / illegal-content filter on the output image.

## Classifier-free guidance (CFG)

Plain text conditioning learns  $\epsilon_{\theta}(x_t, t, c)$  for every prompt  $c$ . CFG trains the same network with  $c$  dropped 10% of the time (replaced by an empty embedding), giving a single model that predicts both the conditional and the unconditional noise. At inference:

$$\epsilon_{\text{ps}} = \epsilon_{\text{ps_uncond}} + w * (\epsilon_{\text{ps_cond}} - \epsilon_{\text{ps_uncond}})$$

$w$  is the guidance scale.  $w=0$  is unconditional,  $w=1$  is plain conditional,  $w>1$  pushes the output toward being “more conditioned on the prompt” at the cost of diversity. SD default is  $w=7.5$ .

CFG is the reason text-to-image works at production quality. Without it, prompts bias the output weakly; with it, prompts dominate.

## Latent space geometry

The VAE’s 4-channel latent is not just a compressed image. It is a manifold where arithmetic roughly corresponds to semantic edits (prompt engineering + interpolation both live here), and where the diffusion U-Net has been trained to spend its entire modelling budget. Decoding a random  $4 \times 64 \times 64$  latent does not produce a random-looking image — it produces garbage, because only a specific submanifold of latents decodes to valid images.

Two consequences:

1. **Img2img** = encode image to latent, add partial noise, run the denoiser, decode. Image structure survives because encoding is near-invertible; content changes based on the prompt.
2. **Inpainting** = same as img2img but the denoiser only updates masked regions; unmasked regions are kept at the encoded latent.

## The U-Net architecture

The SD U-Net is a big version of the TinyUNet from Lesson 10 with three additions:

- **Transformer blocks** at every spatial resolution, containing self-attention + cross-attention to the text embedding.
- **Time embedding** via MLP on sinusoidal encoding.
- **Skip connections** between encoder and decoder at matching resolutions.

Total parameters in SD 1.5: ~860M. SDXL: ~2.6B. FLUX: ~12B. The jump in params is mostly in attention layers.

## LoRA fine-tuning

Full fine-tuning of Stable Diffusion needs 20+ GB of VRAM and updates 860M parameters. LoRA (Low-Rank Adaptation) keeps the base model frozen and injects small rank-decomposition matrices into the attention layers. A LoRA adapter for SD is typically 10-50

MB, trains in 10-60 minutes on a single consumer GPU, and loads at inference time as a drop-in modification.

Original:  $W_q : (d_{in}, d_{out})$  frozen  
 LoRA:  $W_q + \alpha * (A @ B)$  where  $A : (d_{in}, r), B : (r, d_{out})$

$r$  is typically 4-32.

LoRA is how almost every community fine-tune is distributed. CivitAI and Hugging Face host millions of them.

## Schedulers you will see

- **DDIM** — deterministic, ~50 steps, simple.
- **Euler ancestral** — stochastic, 30-50 steps, slightly more creative samples.
- **DPM-Solver++ 2M Karras** — deterministic, 20-30 steps, production default.
- **LCM / TCD / Turbo** — consistency models and distilled variants; 1-4 steps at the cost of some quality.

Swapping schedulers is a one-line change in diffusers and sometimes fixes sample issues without any retraining.

**Interactive figure: cv3-latent-compression.** This one moves. Watch it animate and drag its controls in the web edition: <https://aiengineeringfromscratch.com/lesson?path=phases/04-computer-vision/11-stable-diffusion>

## Build It

This lesson uses diffusers end-to-end rather than rebuilding Stable Diffusion from scratch. The pieces you would need to rebuild (VAE, text encoder, U-Net, scheduler) are topics of their own lessons; here the goal is fluency with the production API.

### Step 1: Text-to-image

```
import torch
from diffusers import StableDiffusionPipeline

pipe = StableDiffusionPipeline.from_pretrained(
    "runwayml/stable-diffusion-v1-5",
    torch_dtype=torch.float16,
).to("cuda")

image = pipe(
    prompt="a dog riding a skateboard in tokyo, studio ghibli style",
    guidance_scale=7.5,
    num_inference_steps=25,
    generator=torch.Generator("cuda").manual_seed(42),
).images[0]
image.save("dog.png")
```

float16 halves VRAM with no visible quality loss. `num_inference_steps=25` with the default DPM-Solver++ matches `num_inference_steps=50` with DDIM.



## Step 2: Swap the scheduler

```
from diffusers import DPMSolverMultistepScheduler, EulerAncestralDiscreteScheduler

pipe.scheduler = DPMSolverMultistepScheduler.from_config(pipe.scheduler.config)
pipe.scheduler =
    ↪ EulerAncestralDiscreteScheduler.from_config(pipe.scheduler.config)
```

Scheduler state is decoupled from U-Net weights. You can train on DDPM and sample with any scheduler.

## Step 3: Image-to-image

```
from diffusers import StableDiffusionImg2ImgPipeline
from PIL import Image

img2img = StableDiffusionImg2ImgPipeline.from_pretrained(
    "runwayml/stable-diffusion-v1-5",
    torch_dtype=torch.float16,
).to("cuda")

init_image = Image.open("dog.png").convert("RGB").resize((512, 512))
out = img2img(
    prompt="a dog riding a skateboard, oil painting",
    image=init_image,
    strength=0.6,
    guidance_scale=7.5,
).images[0]
```

strength is how much noise to add before denoising (0.0 = unchanged, 1.0 = full regeneration). 0.5-0.7 is the standard range for style transfer.

## Step 4: Inpainting

```
from diffusers import StableDiffusionInpaintPipeline

inpaint = StableDiffusionInpaintPipeline.from_pretrained(
    "runwayml/stable-diffusion-inpainting",
    torch_dtype=torch.float16,
).to("cuda")

image = Image.open("dog.png").convert("RGB").resize((512, 512))
mask = Image.open("dog_mask.png").convert("L").resize((512, 512))

out = inpaint(
    prompt="a cat",
    image=image,
    mask_image=mask,
    guidance_scale=7.5,
).images[0]
```

White pixels in the mask are the area to regenerate. Black pixels are preserved.

## Step 5: LoRA loading

```
pipe.load_lora_weights("sayakpaul/sd-lora-ghibli")
pipe.fuse_lora(lora_scale=0.8)
```

```
image = pipe(prompt="a village square in ghibli style").images[0]
```

`lora_scale` controls strength; 0.0 = no effect, 1.0 = full effect. `fuse_lora` bakes the adapter into the weights in place for speed, but prevents swapping. Call `pipe.unfuse_lora()` before loading a different adapter.

## Step 6: LoRA training (sketch)

Real LoRA training lives in `peft` or `diffusers.training`. The outline:

*# Pseudocode*

```
for step, batch in enumerate(dataloader):
    images, prompts = batch
    latents = vae.encode(images).latent_dist.sample() * 0.18215

    t = torch.randint(0, num_train_timesteps, (batch_size,))
    noise = torch.randn_like(latents)
    noisy_latents = scheduler.add_noise(latents, noise, t)

    text_emb = text_encoder(tokenizer(prompts))

    pred_noise = unet(noisy_latents, t, text_emb) # LoRA weights injected here

    loss = F.mse_loss(pred_noise, noise)
    loss.backward()
    optimizer.step()
```

Only the LoRA matrices receive gradient; the base U-Net, VAE, and text encoder are frozen. With a batch size of 1 and gradient checkpointing this fits in 8 GB of VRAM.

## Use It

In production, the decisions you actually make:

- **Model family:** SD 1.5 for open-source community fine-tunes, SDXL for higher fidelity, SD3 / FLUX for state of the art and strict licensing requirements.
- **Scheduler:** DPM-Solver++ 2M Karras for 20-30 steps, LCM-LoRA when latency is under 1s.
- **Precision:** float16 on 4080/4090, bfloat16 on A100 and newer, int8 (via bitsandbytes or compel) when VRAM is tight.
- **Conditioning:** plain text works; for stronger control, add ControlNet (canny, depth, pose) on top of the base pipeline.

For batch generation, AUT01111 / ComfyUI are the community tools; for production APIs, diff users + accelerate or optimum-nvidia with TensorRT compilation.

**This chapter ships an artifact.** The course version of this lesson produces a reusable prompt or agent skill. It lives in the repository, ready to install: <https://github.com/rohitg00/ai-engineering-from-scratch/tree/main/phases/04-computer-vision/11-stable-diffusion>

## Exercises

Starter code and the lesson’s working implementation: <https://github.com/rohitg00/ai-engineering-from-scratch/tree/main/phases/04-computer-vision/11-stable-diffusion/code>

1. **(Easy)** Generate the same prompt with `guidance_scale` in [1, 3, 5, 7.5, 10, 15]. Describe how the image changes. At what guidance value do artefacts appear?
2. **(Medium)** Take any real photograph, run it through `StableDiffusionImg2ImgPipeline` at strength in [0.2, 0.4, 0.6, 0.8, 1.0]. Which strength preserves composition while changing style? Why does 1.0 ignore the input entirely?
3. **(Hard)** Train a LoRA on 10-20 images of a single subject (a pet, a logo, a character) and generate novel scenes with that subject in them. Report the LoRA rank and training steps that produced the best identity preservation without overfitting to the input images.

## Key Terms

Term	What people say	What it actually means
Latent diffusion	“Diffuse in latents”	Run the entire DDPM in the VAE latent space (4x64x64) instead of pixel space (3x512x512); 48x compute saving
VAE scale factor	“0.18215”	Constant that rescales the VAE’s raw latent to roughly unit variance; hardcoded in every SD pipeline
Classifier-free guidance	“CFG”	Mix conditional and unconditional noise predictions; the single most impactful inference knob
Scheduler	“Sampler”	The algorithm that turns noise + model predictions into a denoised latent trajectory
LoRA	“Low-rank adapter”	Small rank-decomposition matrices that fine-tune attention layers without touching base weights
Cross-attention	“Text-image attention”	Attention from latent tokens to text tokens; injects prompt information at every U-Net level
ControlNet	“Structure conditioning”	A separately-trained adapter that steers SD with an extra input (canny, depth, pose, segmentation)
DPM-Solver++	“The default scheduler”	Second-order deterministic ODE solver; best quality at low step counts (20-30) in 2026

## Further Reading

- High-Resolution Image Synthesis with Latent Diffusion (Rombach et al., 2022) — the Stable Diffusion paper; includes every ablation that justifies the design
- Classifier-Free Diffusion Guidance (Ho & Salimans, 2022) — the CFG paper
- LoRA: Low-Rank Adaptation of Large Language Models (Hu et al., 2021) — LoRA was NLP-first; it transferred to SD with almost no changes
- diffusers documentation — the reference for every SD / SDXL / SD3 / FLUX pipeline

**Continue online.** The living edition of this chapter has more than the page can hold:

- Animated, interactive figures and the web text: <https://aiengineeringfromscratch.com/lesson?path=phases/04-computer-vision/11-stable-diffusion>
- Runnable code for every step: <https://github.com/rohitg00/ai-engineering-from-scratch/tree/main/phases/04-computer-vision/11-stable-diffusion/code>
- The chapter quiz, graded in the browser: <https://aiengineeringfromscratch.com/lesson?path=phases/04-computer-vision/11-stable-diffusion>

The repository moves faster than any printing. When the book and the repo disagree, trust the repo.

# Video Understanding — Temporal Modelling

---

A video is a sequence of images plus the physics that connects them. Every video model either treats time as an extra axis (3D conv), a sequence to attend over (transformer), or a feature to extract once and pool (2D+pool).

**Type:** Learn + Build **Languages:** Python **Prerequisites:** Phase 4 Lesson 03 (CNNs), Phase 4 Lesson 04 (Image Classification) **Time:** ~45 minutes

## Learning Objectives

- Distinguish the three main video-modelling approaches (2D+pool, 3D conv, spatio-temporal transformer) and predict their cost and accuracy trade-offs
- Implement frame sampling, temporal pooling, and a 2D+pool baseline classifier in PyTorch
- Explain why I3D's "inflated" 3D kernels transfer well from ImageNet weights and what a factorised (2+1)D conv does differently
- Read the standard action-recognition datasets and metrics: Kinetics-400/600, UCF101, Something-Something V2; top-1 accuracy at the clip and video level

## The Problem

A 30-second video at 30 fps is 900 images. Naively, video classification is image classification run 900 times followed by some kind of aggregation. That works when the action is visible in almost every frame (sports, cooking, exercise videos) and fails badly when the action is defined by motion itself: "pushing something from left to right" looks like two still objects in every single frame.

The core question for every video architecture is: when does temporal structure get modelled, and how? The answer drives everything else — compute cost, pretraining strategy, whether you can reuse ImageNet weights, what datasets the model trains on.

This lesson is deliberately shorter than the static-image lessons. The core image machinery is already in place, and video understanding is mostly about the temporal story: sampling, modelling, and aggregating.

## The Concept

### The three architectural families

**Diagram.** Rendered live in the web edition: <https://aiengineeringfromscratch.com/lesson?path=phases/04-computer-vision/12-video-understanding>

## 2D + pool

Take a 2D CNN (ResNet, EfficientNet, ViT). Run it independently on every sampled frame. Average (or max-pool, or attention-pool) the per-frame embeddings. Feed the pooled vector to a classifier.

Pros: - ImageNet pretraining transfers directly. - Simplest to implement. - Cheap:  $T$  frames * single-image inference cost.

Cons: - Cannot model motion. Action = aggregate of appearances. - Temporal pooling is order-invariant; “open door” and “close door” look the same.

When to use: appearance-heavy tasks, transfer learning on small video datasets, initial baselines.

## 3D convolutions

Replace 2D (H, W) kernels with 3D (T, H, W) kernels. The network convolves over both space and time. Early family: C3D, I3D, SlowFast.

I3D trick: take a pretrained 2D ImageNet model, “inflate” each 2D kernel by copying it along a new time axis. A 3x3 2D conv becomes a 3x3x3 3D conv. This gives the 3D model strong pretrained weights instead of training from scratch.

Pros: - Directly models motion. - I3D inflation gives free transfer learning.

Cons: -  $T/8$  more FLOPs than the 2D counterpart (for temporal kernel of 3 stacked 3 times). - Temporal kernels are small; long-range motion needs a pyramid or dual-stream approach.

When to use: action recognition where motion is the signal (Something-Something V2, Kinetics with motion-heavy classes).

## Spatio-temporal transformers

Tokenise the video into a grid of space-time patches and attend across all of them. TimeS-former, ViViT, Video Swin, VideoMAE.

Attention patterns that matter: - **Joint** — one big attention over (t, h, w). Quadratic in  $T*H*W$ ; expensive. - **Divided** — two attentions per block: one over time, one over space. Linear-ish scaling. - **Factorised** — time attention alternates with space attention across blocks.

Pros: - SOTA accuracy on every major benchmark. - Transfers from image transformers (ViT) via patch inflation. - Supports long-context video via sparse attention.

Cons: - Compute-hungry. - Requires careful attention pattern choice or runtime balloons.

When to use: large datasets, high-fidelity video understanding, multi-modal video+text tasks.

## Frame sampling

A 10-second clip at 30 fps is 300 frames; feeding all 300 to any model is wasteful. Standard strategies:

- **Uniform sampling** — pick  $T$  frames evenly across the clip. Default for 2D+pool.
- **Dense sampling** — random contiguous  $T$ -frame window. Common for 3D convs because motion requires neighbouring frames.
- **Multi-clip** — sample multiple  $T$ -frame windows from the same video, classify each, average predictions at test time.

$T$  is usually 8, 16, 32, or 64. Higher  $T$  = more temporal signal at more compute.

## Evaluation

Two levels: - **Clip-level accuracy** — model sees one T-frame clip, reports top-k. - **Video-level accuracy** — average clip-level predictions across multiple clips per video; higher and more stable.

Always report both. A model that scores 78% clip / 82% video is relying heavily on test-time averaging; one that scores 80% / 81% is more robust per-clip.

## Datasets you will meet

- **Kinetics-400 / 600 / 700** — the general-purpose action dataset. 400k clips; YouTube URLs (many now dead).
- **Something-Something V2** — motion-defined actions (“moving X from left to right”). Cannot be solved by 2D+pool.
- **UCF-101, HMDB-51** — older, smaller, still reported.
- **AVA** — action *localisation* in space and time; harder than classification.

**Interactive figure: v4-video-temporal.** This one moves. Watch it animate and drag its controls in the web edition: <https://aiengineeringfromscratch.com/lesson?path=phases/04-computer-vision/12-video-understanding>

## Build It

### Step 1: Frame sampler

Uniform and dense samplers that work on a list of frames (or a video tensor).

```
import numpy as np

def sample_uniform(num_frames_total, T):
    if num_frames_total <= T:
        return list(range(num_frames_total)) + [num_frames_total - 1] * (T -
            ↪ num_frames_total)
    step = num_frames_total / T
    return [int(i * step) for i in range(T)]

def sample_dense(num_frames_total, T, rng=None):
    rng = rng or np.random.default_rng()
    if num_frames_total <= T:
        return list(range(num_frames_total)) + [num_frames_total - 1] * (T -
            ↪ num_frames_total)
    start = int(rng.integers(0, num_frames_total - T + 1))
    return list(range(start, start + T))
```

Both return T indices that you use to slice the video tensor.

### Step 2: A 2D+pool baseline

Run a 2D ResNet-18 over every frame, average-pool features, classify.

```
import torch
import torch.nn as nn
from torchvision.models import resnet18, ResNet18_Weights
```

```

class FramePool(nn.Module):
    def __init__(self, num_classes=400, pretrained=True):
        super().__init__()
        weights = ResNet18_Weights.IMAGENET1K_V1 if pretrained else None
        backbone = resnet18(weights=weights)
        self.features = nn.Sequential(*(list(backbone.children())[:-1])) # global
        ↪ avg pool kept
        self.head = nn.Linear(512, num_classes)

    def forward(self, x):
        # x: (N, T, 3, H, W)
        N, T = x.shape[:2]
        x = x.view(N * T, *x.shape[2:])
        feats = self.features(x).view(N, T, -1)
        pooled = feats.mean(dim=1)
        return self.head(pooled)

model = FramePool(num_classes=10)
x = torch.randn(2, 8, 3, 224, 224)
print(f"output: {model(x).shape}")
print(f"params: {sum(p.numel() for p in model.parameters()):,}")

```

Eleven million parameters, ImageNet pretrained, runs per-frame, averages, classifies. This baseline is often within 5-10 points of proper 3D models on appearance-heavy tasks — sometimes better, because it reuses a stronger ImageNet backbone.

### Step 3: An I3D-style inflated 3D conv

Turn a single 2D conv into a 3D conv by repeating weights along a new time axis.

```

def inflate_2d_to_3d(conv2d, time_kernel=3):
    out_c, in_c, kh, kw = conv2d.weight.shape
    weight_3d = conv2d.weight.data.unsqueeze(2) # (out, in, 1, kh, kw)
    weight_3d = weight_3d.repeat(1, 1, time_kernel, 1, 1) / time_kernel
    conv3d = nn.Conv3d(in_c, out_c, kernel_size=(time_kernel, kh, kw),
        padding=(time_kernel // 2, conv2d.padding[0],
    ↪ conv2d.padding[1]),
        stride=(1, conv2d.stride[0], conv2d.stride[1]),
        bias=False)
    conv3d.weight.data = weight_3d
    return conv3d

conv2d = nn.Conv2d(3, 64, kernel_size=3, padding=1, bias=False)
conv3d = inflate_2d_to_3d(conv2d, time_kernel=3)
print(f"2D weight shape: {tuple(conv2d.weight.shape)}")
print(f"3D weight shape: {tuple(conv3d.weight.shape)}")
x = torch.randn(1, 3, 8, 56, 56)
print(f"3D output shape: {tuple(conv3d(x).shape)}")

```

The division by `time_kernel` keeps the activation magnitudes roughly constant — important for not breaking batch-norm statistics on the first pass.



## Step 4: Factorised (2+1)D conv

Split a 3D conv into a 2D (spatial) and a 1D (temporal) conv. Same receptive field, fewer parameters, better accuracy on some benchmarks.

```
class Conv2Plus1D(nn.Module):
    def __init__(self, in_c, out_c, kernel_size=3):
        super().__init__()
        mid_c = (in_c * out_c * kernel_size * kernel_size * kernel_size) \
            // (in_c * kernel_size * kernel_size + out_c * kernel_size)
        self.spatial = nn.Conv3d(in_c, mid_c, kernel_size=(1, kernel_size,
            ↪ kernel_size),
                                padding=(0, kernel_size // 2, kernel_size // 2),
                                bias=False)
        self.bn = nn.BatchNorm3d(mid_c)
        self.act = nn.ReLU(inplace=True)
        self.temporal = nn.Conv3d(mid_c, out_c, kernel_size=(kernel_size, 1, 1),
                                padding=(kernel_size // 2, 0, 0), bias=False)

    def forward(self, x):
        return self.temporal(self.act(self.bn(self.spatial(x))))

c = Conv2Plus1D(3, 64)
x = torch.randn(1, 3, 8, 56, 56)
print(f"(2+1)D output: {tuple(c(x).shape)}")
```

A full R(2+1)D network is the same as a ResNet-18 with every 3x3 conv replaced by Conv2Plus1D.

## Use It

Two libraries cover production video work:

- torchvision.models.video — R(2+1)D, MViT, Swin3D with pretrained Kinetics weights. Same API as image models.
- pytorchvideo (Meta) — model zoo, data loaders for Kinetics / SSv2 / AVA, standard transforms.

For Vision-Language video models (video captioning, video QA), use transformers (VideoMAE, VideoLLaMA, InternVideo).

**This chapter ships an artifact.** The course version of this lesson produces a reusable prompt or agent skill. It lives in the repository, ready to install: <https://github.com/rohitg00/ai-engineering-from-scratch/tree/main/phases/04-computer-vision/12-video-understanding>

## Exercises

Starter code and the lesson’s working implementation: <https://github.com/rohitg00/ai-engineering-from-scratch/tree/main/phases/04-computer-vision/12-video-understanding/code>

1. **(Easy)** Compute FLOPs (approximate) for FramePool with T=8 vs an I3D-style 3D ResNet with T=8. Justify why 2D+pool is 3-5x cheaper.
2. **(Medium)** Generate a synthetic video dataset: random balls moving in random directions, labelled by direction of motion (“left-to-right”, “right-to-left”, “diagonal-up”). Train

FramePool on it. Show that it achieves near-chance accuracy, proving appearance alone is insufficient for motion tasks.

3. **(Hard)** Build an R(2+1)D-18 by replacing every Conv2d in a ResNet-18 with Conv2Plus1D. Inflate the first conv’s weights from an ImageNet-pretrained ResNet-18. Train on the motion dataset from exercise 2 and beat FramePool.

## Key Terms

Term	What people say	What it actually means
2D + pool	“Per-frame classifier”	Run a 2D CNN on every sampled frame, average-pool features across time, classify
3D convolution	“Spatio-temporal kernel”	Kernel that convolves over (T, H, W); can model motion natively
Inflation	“Lift 2D weights to 3D”	Initialise 3D conv weights by repeating a 2D conv’s weights along the new time axis, then divide by kernel_T to preserve activation scale
(2+1)D	“Factorised conv”	Split 3D into 2D spatial + 1D temporal; fewer parameters, extra non-linearity between
Divided attention	“Time then space”	Transformer block with two attentions per layer: one over tokens at the same frame, one over tokens at the same position
Clip	“T-frame window”	A sampled subsequence of T frames; the unit a video model consumes
Clip vs video accuracy	“Two eval settings”	Clip = one sample per video, video = average across multiple sampled clips
Kinetics	“The ImageNet of video”	400-700 action classes, 300k+ YouTube clips, the standard video pretraining corpus

## Further Reading

- I3D: Quo Vadis, Action Recognition (Carreira & Zisserman, 2017) — introduces inflation and the Kinetics dataset
- R(2+1)D: A Closer Look at Spatiotemporal Convolutions (Tran et al., 2018) — factorised conv, still a strong baseline
- TimeSformer: Is Space-Time Attention All You Need? (Bertasius et al., 2021) — the first strong video transformer
- VideoMAE (Tong et al., 2022) — masked autoencoder pretraining for video; current dominant pretraining recipe

**Continue online.** The living edition of this chapter has more than the page can hold:

- Animated, interactive figures and the web text: <https://aiengineeringfromscratch.com/lesson?path=phases/04-computer-vision/12-video-understanding>
- Runnable code for every step: <https://github.com/rohitg00/ai-engineering-from-scratch/tree/main/phases/04-computer-vision/12-video-understanding/code>
- The chapter quiz, graded in the browser: <https://aiengineeringfromscratch.com/lesson?path=phases/04-computer-vision/12-video-understanding>

The repository moves faster than any printing. When the book and the repo disagree, trust the repo.

# 3D Vision — Point Clouds & NeRFs

---

3D vision comes in two flavours. Point clouds are the sensor's raw output. NeRFs are the learned volumetric field. Both answer "what is where in space."

**Type:** Learn + Build **Languages:** Python **Prerequisites:** Phase 4 Lesson 03 (CNNs), Phase 1 Lesson 12 (Tensor Operations) **Time:** ~45 minutes

## Learning Objectives

- Distinguish explicit (point cloud, mesh, voxel) and implicit (signed distance field, NeRF) 3D representations and when each is used
- Understand PointNet's symmetric-function trick that makes a neural network permutation-invariant over an unordered set of points
- Trace a NeRF forward pass: ray casting, volumetric rendering, positional encoding, MLP density+colour head
- Use nerfstudio or instant-ngp for pretrained 3D reconstruction from a small set of posed images

## The Problem

A camera produces a 2D image. A LIDAR produces a set of 3D points with no ordering. A structure-from-motion pipeline produces a sparse cloud of 3D keypoints. A NeRF reconstructs an entire 3D scene from a handful of posed images. All of these are "vision" but none of them look like the dense tensor a CNN wants.

3D vision matters because almost every high-value robot task runs in 3D: grasping, obstacle avoidance, navigation, AR occlusion, 3D content capture. A vision engineer who only understands 2D images is locked out of the fastest-growing slice of the field (AR/VR content, robotics, autonomous driving stacks, NeRF-based 3D reconstruction for real-estate or construction).

The two representations dominate for different reasons. Point clouds are what sensors give you for free. NeRFs and their successors (3D Gaussian splatting, neural SDFs) are what you get when you ask a neural network to learn a scene.

## The Concept

### Point clouds

A point cloud is an unordered set of  $N$  points in  $\mathbb{R}^3$ , optionally each with features (colour, intensity, normal).

```
cloud = [  
    (x1, y1, z1, r1, g1, b1),  
    (x2, y2, z2, r2, g2, b2),  
    ...
```

```
(xN, yN, zN, rN, gN, bN),
]
```

No grid, no connectivity. Two properties make this hard for neural networks:

- **Permutation invariance** — the output must not depend on point order.
- **Variable N** — a single model must handle clouds of different sizes.

PointNet (Qi et al., 2017) solved both with one idea: apply a shared MLP to every point, then aggregate with a symmetric function (max pool). The result is a fixed-size vector that does not depend on order.

$$f(P) = \max_{\{p \in P\}} \text{MLP}(p)$$

This is the entire core of PointNet. Deeper variants (PointNet++, Point Transformer) add hierarchical sampling and local aggregation but the symmetric-function trick is unchanged.

## The PointNet architecture

**Diagram.** Rendered live in the web edition: <https://aiengineeringfromscratch.com/lesson?path=phases/04-computer-vision/13-3d-vision-nerf>

“Shared MLP” means the same MLP runs on every point independently. Implemented as a 1x1 conv over the point dimension for efficiency.

## Neural Radiance Fields (NeRFs)

NeRFs (Mildenhall et al., 2020) took the question “can we reconstruct a 3D scene from N photos?” and answered with a neural network that is the scene. The network maps (x, y, z, viewing_direction) to (density, colour). Rendering a new view is a ray-casting loop over this network.

NeRF MLP: (x, y, z, theta, phi) -> (sigma, r, g, b)

To render a pixel (u, v) of a new view:

1. Cast a ray from the camera through pixel (u, v)
2. Sample points along the ray at distances t₁, t₂, ..., t_N
3. Query the MLP at each point
4. Composite the colours weighted by (1 - exp(-sigma * dt))
5. The sum is the rendered pixel colour

A loss compares the rendered pixel to the ground-truth pixel in the training photos. Backprop through the rendering step updates the MLP. No 3D ground truth, no explicit geometry — the scene is stored in the MLP weights.

## Positional encoding in NeRF

A vanilla MLP on (x, y, z) cannot represent high-frequency details because MLPs are spectrally biased toward low frequencies. NeRF fixes this by encoding each coordinate into a Fourier feature vector before the MLP:

$$\text{gamma}(p) = (\sin(2^0 \pi p), \cos(2^0 \pi p), \sin(2^1 \pi p), \cos(2^1 \pi p), \dots)$$

Up to L=10 frequency levels. This is the same trick transformers use for positions, and it appears again in diffusion time conditioning (Lesson 10). Without it, NeRFs look blurry.

## Volumetric rendering

$$C(r) = \sum_i T_i * (1 - \exp(-\sigma_i * \delta_i)) * c_i$$

$$T_i = \exp(-\sum_{j<i} \sigma_j * \delta_j)$$

$$\delta_i = t_{i+1} - t_i$$

$T_i$  is transmittance — how much light survives to point  $i$ .  $(1 - \exp(-\sigma_i * \delta_i))$  is the opacity at point  $i$ .  $c_i$  is the colour. The final pixel is a weighted sum along the ray.

## What replaced NeRFs

Pure NeRFs are slow to train (hours) and slow to render (seconds per image). The lineage since:

- **Instant-NGP** (2022) — hash-grid encoding replaces the MLP’s position input; trains in seconds.
- **Mip-NeRF 360** — handles unbounded scenes and anti-aliasing.
- **3D Gaussian Splatting** (2023) — replaces the volumetric field with millions of 3D Gaussians; trains in minutes, renders in real time. The current production default.

Almost every real NeRF product in 2026 is actually 3D Gaussian splatting. The mental model is still NeRF.

## Datasets and benchmarks

- **ShapeNet** — classification and segmentation of 3D CAD models as point clouds.
- **ScanNet** — real indoor scans for segmentation.
- **KITTI** — outdoor LIDAR point clouds for autonomous driving.
- **NeRF Synthetic / Blended MVS** — posed-image datasets for view synthesis.
- **Mip-NeRF 360** dataset — unbounded real scenes.

**Interactive figure: nerf-rays.** This one moves. Watch it animate and drag its controls in the web edition: <https://aiengineeringfromscratch.com/lesson?path=phases/04-computer-vision/13-3d-vision-nerf>

## Build It

### Step 1: PointNet classifier

```
import torch
import torch.nn as nn

class PointNet(nn.Module):
    def __init__(self, num_classes=10):
        super().__init__()
        self.mlp1 = nn.Sequential(
            nn.Conv1d(3, 64, 1), nn.BatchNorm1d(64), nn.ReLU(inplace=True),
            nn.Conv1d(64, 64, 1), nn.BatchNorm1d(64), nn.ReLU(inplace=True),
        )
        self.mlp2 = nn.Sequential(
            nn.Conv1d(64, 128, 1), nn.BatchNorm1d(128), nn.ReLU(inplace=True),
            nn.Conv1d(128, 1024, 1), nn.BatchNorm1d(1024), nn.ReLU(inplace=True),
        )
        self.head = nn.Sequential(
```

```

        nn.Linear(1024, 512),    nn.BatchNorm1d(512),    nn.ReLU(inplace=True),
        nn.Dropout(0.3),
        nn.Linear(512, 256),    nn.BatchNorm1d(256),    nn.ReLU(inplace=True),
        nn.Dropout(0.3),
        nn.Linear(256, num_classes),
    )

    def forward(self, x):
        # x: (N, 3, num_points) – transposed for Conv1d
        x = self.mlp1(x)
        x = self.mlp2(x)
        x = torch.max(x, dim=-1)[0]          # (N, 1024)
        return self.head(x)

pts = torch.randn(4, 3, 1024)
net = PointNet(num_classes=10)
print(f"output: {net(pts).shape}")
print(f"params: {sum(p.numel() for p in net.parameters()):,}")

```

About 1.6M parameters. Runs on 1,024 points per cloud.

## Step 2: Positional encoding

```

def positional_encoding(x, L=10):
    """
    x: (... , D) -> (... , D * 2 * L)
    """
    freqs = 2.0 ** torch.arange(L, dtype=x.dtype, device=x.device)
    args = x.unsqueeze(-1) * freqs * 3.141592653589793
    sinc = torch.cat([args.sin(), args.cos()], dim=-1)
    return sinc.reshape(*x.shape[:-1], -1)

x = torch.randn(5, 3)
y = positional_encoding(x, L=10)
print(f"input: {x.shape}")
print(f"encoded: {y.shape}      # (5, 60)")

```

Multiplying by  $2^l \cdot \pi$  gives progressively higher frequencies.

## Step 3: Tiny NeRF MLP

```

class TinyNeRF(nn.Module):
    def __init__(self, L_pos=10, L_dir=4, hidden=128):
        super().__init__()
        self.L_pos = L_pos
        self.L_dir = L_dir
        pos_dim = 3 * 2 * L_pos
        dir_dim = 3 * 2 * L_dir
        self.trunk = nn.Sequential(
            nn.Linear(pos_dim, hidden), nn.ReLU(inplace=True),
            nn.Linear(hidden, hidden), nn.ReLU(inplace=True),
            nn.Linear(hidden, hidden), nn.ReLU(inplace=True),
            nn.Linear(hidden, hidden), nn.ReLU(inplace=True),
        )

```

```

self.sigma = nn.Linear(hidden, 1)
self.color = nn.Sequential(
    nn.Linear(hidden + dir_dim, hidden // 2), nn.ReLU(inplace=True),
    nn.Linear(hidden // 2, 3), nn.Sigmoid(),
)

def forward(self, x, d):
    x_enc = positional_encoding(x, self.L_pos)
    d_enc = positional_encoding(d, self.L_dir)
    h = self.trunk(x_enc)
    sigma = torch.relu(self.sigma(h)).squeeze(-1)
    rgb = self.color(torch.cat([h, d_enc], dim=-1))
    return sigma, rgb

nerf = TinyNeRF()
x = torch.randn(128, 3)
d = torch.randn(128, 3)
s, c = nerf(x, d)
print(f"sigma: {s.shape}    rgb: {c.shape}")

```

Tiny compared to the original NeRF (which has 2 MLP trunks of depth 8). Enough to demonstrate the architecture.

#### Step 4: Volumetric rendering along a ray

```

def volumetric_render(sigma, rgb, t_vals):
    """
    sigma: (... , N_samples)
    rgb:    (... , N_samples, 3)
    t_vals: (N_samples,) distances along the ray
    """
    delta = torch.cat([t_vals[1:] - t_vals[:-1], torch.full_like(t_vals[:1],
        ↪ 1e10)])
    alpha = 1.0 - torch.exp(-sigma * delta)
    trans = torch.cumprod(torch.cat([torch.ones_like(alpha[..., :1]), 1.0 - alpha
        ↪ + 1e-10], dim=-1), dim=-1)[..., :-1]
    weights = alpha * trans
    rendered = (weights.unsqueeze(-1) * rgb).sum(dim=-2)
    depth = (weights * t_vals).sum(dim=-1)
    return rendered, depth, weights

N = 64
t_vals = torch.linspace(2.0, 6.0, N)
sigma = torch.rand(N) * 0.5
rgb = torch.rand(N, 3)
rendered, depth, weights = volumetric_render(sigma, rgb, t_vals)
print(f"rendered colour: {rendered.tolist()}")
print(f"depth:           {depth.item():.2f}")

```

One ray, 64 samples, composite to a single RGB pixel and a depth.



## Use It

For real work:

- nerfstudio (Tancik et al.) — the current reference library for NeRF / Instant-NGP / Gaussian Splatting. Command-line plus a web viewer.
- pytorch3d (Meta) — differentiable rendering, point-cloud utilities, mesh ops.
- open3d — point cloud processing, registration, visualisation.

For deployment, 3D Gaussian splatting has largely replaced pure NeRFs because it renders 100x faster. The reconstruction quality is comparable.

**This chapter ships an artifact.** The course version of this lesson produces a reusable prompt or agent skill. It lives in the repository, ready to install: <https://github.com/rohitg00/ai-engineering-from-scratch/tree/main/phases/04-computer-vision/13-3d-vision-nerf>

## Exercises

Starter code and the lesson’s working implementation: <https://github.com/rohitg00/ai-engineering-from-scratch/tree/main/phases/04-computer-vision/13-3d-vision-nerf/code>

1. **(Easy)** Show that PointNet is permutation-invariant: run the same cloud through twice, once with points shuffled. Verify outputs are identical up to floating-point noise.
2. **(Medium)** Implement a minimal ray-generation function that, given camera intrinsics and pose, produces ray origins and directions for every pixel of an H x W image.
3. **(Hard)** Train a TinyNeRF on a synthetic dataset of rendered views of a coloured cube (generated via differentiable rendering or a simple ray tracer). Report rendering loss at epoch 1, 10, and 100. At what epoch does the model produce recognisable views?

## Key Terms

Term	What people say	What it actually means
Point cloud	“3D points from LIDAR”	Unordered set of (x, y, z) + optional features per point
PointNet	“First neural net on point clouds”	Shared MLP per point + symmetric (max) pool; permutation-invariant by construction
NeRF	“MLP that is the scene”	Network mapping (x, y, z, dir) to (density, colour); rendered by ray casting
Positional encoding	“Fourier features”	Encode each coordinate into sin/cos at multiple frequencies to overcome MLP low-frequency bias
Volumetric rendering	“Ray integration”	Composite samples along a ray into a single pixel using transmittance and alpha
Instant-NGP	“Hash-grid NeRF”	Replaces NeRF’s coordinate MLP with a multi-resolution hash grid; 100-1000x faster
3D Gaussian splatting	“Millions of Gaussians”	Scene = collection of 3D Gaussians; renders in real time, trains in minutes
SDF	“Signed distance field”	Function returning signed distance to the nearest surface; another implicit representation

## Further Reading

- PointNet (Qi et al., 2017) — the permutation-invariant classifier
- NeRF (Mildenhall et al., 2020) — the paper that made 3D reconstruction from photos a neural-net problem
- Instant-NGP (Müller et al., 2022) — hash grids, 1000x speedup
- 3D Gaussian Splatting (Kerbl et al., 2023) — the architecture that replaced NeRFs in production

**Continue online.** The living edition of this chapter has more than the page can hold:

- Animated, interactive figures and the web text: <https://aiengineeringfromscratch.com/lesson?path=phases/04-computer-vision/13-3d-vision-nerf>
- Runnable code for every step: <https://github.com/rohitg00/ai-engineering-from-scratch/tree/main/phases/04-computer-vision/13-3d-vision-nerf/code>
- The chapter quiz, graded in the browser: <https://aiengineeringfromscratch.com/lesson?path=phases/04-computer-vision/13-3d-vision-nerf>

The repository moves faster than any printing. When the book and the repo disagree, trust the repo.

# Vision Transformers (ViT)

---

Cut the image into patches, treat each patch as a word, run a standard transformer.  
Don't look back.

**Type:** Build **Languages:** Python **Prerequisites:** Phase 7 Lesson 02 (Self-Attention), Phase 4 Lesson 04 (Image Classification) **Time:** ~45 minutes

## Learning Objectives

- Implement patch embedding, learned positional embedding, class token, and transformer encoder blocks from scratch to build a minimal ViT
- Explain why ViT was thought to need massive pretraining data until DeiT and MAE proved otherwise
- Compare ViT, Swin, and ConvNeXt on their architectural priors (none, local window attention, conv backbone)
- Fine-tune a pretrained ViT on a small dataset using timm and the standard linear-probe / fine-tune recipe

## The Problem

For a decade, convolution was synonymous with computer vision. CNNs had strong inductive biases — locality, translation equivariance — that nobody thought you could replace. Then Dosovitskiy et al. (2020) showed that a plain transformer applied to flattened image patches, with no convolutional machinery at all, could match or beat the best CNNs at scale.

The catch was “at scale.” ViT on ImageNet-1k lost to ResNet. ViT pretrained on ImageNet-21k or JFT-300M then fine-tuned on ImageNet-1k beat it. The conclusion was that transformers lacked useful priors but could learn them from enough data. Subsequent work (DeiT, MAE, DINO) showed that with the right training recipes — strong augmentation, self-supervised pretraining, distillation — ViTs train fine on small data too.

By 2026, pure CNNs are still competitive on edge devices (ConvNeXt is the strongest), but transformers dominate everything else: segmentation (Mask2Former, SegFormer), detection (DETR, RT-DETR), multimodal (CLIP, SigLIP), video (VideoMAE, VJEPa). The ViT block structure is the one to know.

## The Concept

### The pipeline

**Diagram.** Rendered live in the web edition: <https://aiengineeringfromscratch.com/lesson?path=phases/04-computer-vision/14-vision-transformers>

Seven steps. Patches -> tokens -> attention -> classifier. Every variant (DeiT, Swin, ConvNeXt, MAE pretraining) changes one or two of the seven and leaves the rest alone.

## Patch embedding

The first conv is the secret. Kernel size 16, stride 16, so a 224x224 image becomes a 14x14 grid of 16x16 patches, each projected to a 768-dim embedding. That single conv both patchifies and linearly projects.

Input: (3, 224, 224)

Conv (3 -> 768, k=16, s=16, no padding):

Output: (768, 14, 14)

Flatten spatial: (196, 768)

196 patches = 196 tokens. Each token's feature dimension is 768 (ViT-B), 1024 (ViT-L), or 1280 (ViT-H).

## Class token

A single learned vector prepended to the sequence:

tokens = [CLS; patch_1; patch_2; ...; patch_196]    shape (197, 768)

After N transformer blocks, the [CLS] output is the global image representation. Classification head reads only this one vector.

## Positional embedding

Transformers have no built-in notion of spatial position. Add a learned vector to every token:

tokens = tokens + learned_pos_embedding    (also shape (197, 768))

The embedding is a parameter of the model; gradient-based training adapts it to 2D image structure. Sinusoidal 2D alternatives exist but are rarely used in practice.

## Transformer encoder block

Standard. Multi-head self-attention, MLP, residual connections, pre-LayerNorm.

$x = x + \text{MSA}(\text{LN}(x))$

$x = x + \text{MLP}(\text{LN}(x))$

MLP is two-layer with GELU: Linear(d -> 4d) -> GELU -> Linear(4d -> d)

ViT-B/16 stacks 12 of these blocks, each with 12 attention heads, totalling 86M parameters.

## Why pre-LN

Early transformers used post-LN ( $x = \text{LN}(x + \text{sublayer}(x))$ ) and struggled to train past 6-8 layers without warmup. Pre-LN ( $x = x + \text{sublayer}(\text{LN}(x))$ ) trains deeper networks stably without warmup. Every ViT and every modern LLM uses pre-LN.

## Patch size trade-off

- 16x16 patches -> 196 tokens, standard.
- 32x32 patches -> 49 tokens, faster but lower resolution.
- 8x8 patches -> 784 tokens, finer but  $O(n^2)$  attention cost scales badly.

Bigger patches = fewer tokens = faster but less spatial detail. SwinV2 uses 4x4 patches in hierarchical windows.

## DeiT's recipe for training ViT on ImageNet-1k

The original ViT needed JFT-300M to beat CNNs. DeiT (Touvron et al., 2020) trained ViT-B to 81.8% top-1 on ImageNet-1k alone with four changes:

1. Heavy augmentation: RandAugment, Mixup, CutMix, Random Erasing.
2. Stochastic depth (drop entire blocks at random during training).
3. Repeated augmentation (same image sampled 3 times per batch).
4. Distillation from a CNN teacher (optional, lifts accuracy further).

Every modern ViT training recipe descends from DeiT.

## Swin vs ConvNeXt

- **Swin** (Liu et al., 2021) — window-based attention. Each block attends within a local window; alternating blocks shift the window to mix information across windows. Brings back a CNN-like locality prior while keeping the attention operator.
- **ConvNeXt** (Liu et al., 2022) — redesigned CNN that matches Swin's architecture choices (depthwise convs, LayerNorm, GELU, inverted bottleneck). Showed that the gap is not "attention vs convolution" but "modern training recipe + architecture."

In 2026, ConvNeXt-V2 and Swin-V2 are both production-grade; the right choice depends on your inference stack (ConvNeXt compiles better for edge) and pretraining corpus.

## MAE pretraining

Masked Autoencoder (He et al., 2022): mask 75% of patches at random, train the encoder to process only the visible 25%, train a small decoder to reconstruct the masked patches from the encoder's output. After pretraining, discard the decoder and fine-tune the encoder.

MAE makes ViT trainable on ImageNet-1k alone, hits SOTA, and is the current default self-supervised recipe.

**Interactive figure: batchnorm-inference.** This one moves. Watch it animate and drag its controls in the web edition: <https://aiengineeringfromscratch.com/lesson?path=phases/04-computer-vision/14-vision-transformers>

## Build It

### Step 1: Patch embedding

```
import torch
import torch.nn as nn

class PatchEmbedding(nn.Module):
    def __init__(self, in_channels=3, patch_size=16, dim=192, image_size=64):
        super().__init__()
        assert image_size % patch_size == 0
        self.proj = nn.Conv2d(in_channels, dim, kernel_size=patch_size,
                               ↪ stride=patch_size)
        num_patches = (image_size // patch_size) ** 2
        self.num_patches = num_patches

    def forward(self, x):
```

```
x = self.proj(x)
return x.flatten(2).transpose(1, 2)
```

One conv, one flatten, one transpose. That is the entire image-to-tokens step.

## Step 2: Transformer block

Pre-LN, multi-head self-attention, MLP with GELU, residual connections.

```
class Block(nn.Module):
    def __init__(self, dim, num_heads, mlp_ratio=4, dropout=0.0):
        super().__init__()
        self.ln1 = nn.LayerNorm(dim)
        self.attn = nn.MultiheadAttention(dim, num_heads, dropout=dropout,
            ↪ batch_first=True)
        self.ln2 = nn.LayerNorm(dim)
        self.mlp = nn.Sequential(
            nn.Linear(dim, dim * mlp_ratio),
            nn.GELU(),
            nn.Dropout(dropout),
            nn.Linear(dim * mlp_ratio, dim),
            nn.Dropout(dropout),
        )

    def forward(self, x):
        a, _ = self.attn(self.ln1(x), self.ln1(x), self.ln1(x), need_weights=False)
        x = x + a
        x = x + self.mlp(self.ln2(x))
        return x
```

`nn.MultiheadAttention` handles the splitting into heads, the scaled dot-product, and the output projection. `batch_first=True` so shapes are (N, seq, dim).

## Step 3: The ViT

```
class ViT(nn.Module):
    def __init__(self, image_size=64, patch_size=16, in_channels=3,
        num_classes=10, dim=192, depth=6, num_heads=3, mlp_ratio=4):
        super().__init__()
        self.patch = PatchEmbedding(in_channels, patch_size, dim, image_size)
        num_patches = self.patch.num_patches
        self.cls_token = nn.Parameter(torch.zeros(1, 1, dim))
        self.pos_embed = nn.Parameter(torch.zeros(1, num_patches + 1, dim))
        self.blocks = nn.ModuleList([
            Block(dim, num_heads, mlp_ratio) for _ in range(depth)
        ])
        self.ln = nn.LayerNorm(dim)
        self.head = nn.Linear(dim, num_classes)
        nn.init.trunc_normal_(self.pos_embed, std=0.02)
        nn.init.trunc_normal_(self.cls_token, std=0.02)

    def forward(self, x):
        x = self.patch(x)
        cls = self.cls_token.expand(x.size(0), -1, -1)
        x = torch.cat([cls, x], dim=1)
```

```

x = x + self.pos_embed
for blk in self.blocks:
    x = blk(x)
x = self.ln(x[:, 0])
return self.head(x)

```

```

vit = ViT(image_size=64, patch_size=16, num_classes=10, dim=192, depth=6,
    num_heads=3)
x = torch.randn(2, 3, 64, 64)
print(f"output: {vit(x).shape}")
print(f"params: {sum(p.numel() for p in vit.parameters()),}")

```

About 2.8M parameters — a tiny ViT tractable on CPU. Real ViT-B is 86M; same class definition with dim=768, depth=12, num_heads=12.

## Step 4: Sanity check — single image inference

```

logits = vit(torch.randn(1, 3, 64, 64))
print(f"logits: {logits}")
print(f"probs: {logits.softmax(-1)}")

```

Should run without error. Probabilities sum to 1.

## Use It

timm ships every ViT variant with ImageNet pretrained weights. One line:

```
import timm
```

```
model = timm.create_model("vit_base_patch16_224", pretrained=True, num_classes=10)
```

timm is the production default for vision transformers in 2026. Supports ViT, DeiT, Swin, Swin-V2, ConvNeXt, ConvNeXt-V2, MaxViT, MViT, EfficientFormer, and dozens of others under the same API.

For multi-modal work (image + text), transformers ships CLIP, SigLIP, BLIP-2, LLaVA. The image encoder in all of those is a ViT variant.

**This chapter ships an artifact.** The course version of this lesson produces a reusable prompt or agent skill. It lives in the repository, ready to install: <https://github.com/rohitg00/ai-engineering-from-scratch/tree/main/phases/04-computer-vision/14-vision-transformers>

## Exercises

Starter code and the lesson's working implementation: <https://github.com/rohitg00/ai-engineering-from-scratch/tree/main/phases/04-computer-vision/14-vision-transformers/code>

1. **(Easy)** Print the shapes of every intermediate tensor for a forward pass through the tiny ViT above. Confirm: input (N, 3, 64, 64) -> patches (N, 16, 192) -> with CLS (N, 17, 192) -> classifier input (N, 192) -> output (N, num_classes).
2. **(Medium)** Fine-tune a pretrained timm ViT-S/16 on the synthetic-CIFAR dataset from Lesson 4. Compare against ResNet-18 fine-tuning on the same data. Report training time and final accuracy.

3. **(Hard)** Implement MAE pretraining for the tiny ViT: mask 75% of patches, train the encoder + a small decoder to reconstruct the masked patches. Evaluate linear-probe accuracy on the synthetic data before and after pretraining.

## Key Terms

Term	What people say	What it actually means
Patch embedding	“The first conv”	A conv with kernel size = stride = patch size; turns the image into a grid of token embeddings
Class token	“[CLS]”	A learned vector prepended to the token sequence; its final output is the global image representation
Positional embedding	“Learned pos”	A learned vector added to every token so the transformer knows where each patch came from
Pre-LN	“LayerNorm before sublayer”	The stable transformer variant: $x + \text{sublayer}(\text{LN}(x))$ instead of $\text{LN}(x + \text{sublayer}(x))$
Multi-head attention	“Parallel attention”	Standard transformer attention split into <code>num_heads</code> independent subspaces, concatenated afterwards
ViT-B/16	“Base, patch 16”	The canonical size: <code>dim=768</code> , <code>depth=12</code> , <code>heads=12</code> , <code>patch_size=16</code> , <code>image=224</code> ; ~86M params
DeiT	“Data-efficient ViT”	ViT trained on ImageNet-1k alone with strong augmentation; proved large pretraining datasets are not strictly required
MAE	“Masked autoencoder”	Self-supervised pretraining: mask 75% of patches, reconstruct; the dominant ViT pretraining recipe

## Further Reading

- An Image is Worth 16x16 Words (Dosovitskiy et al., 2020) — the ViT paper
- DeiT: Data-efficient Image Transformers (Touvron et al., 2020) — how to train ViT on ImageNet-1k alone
- Masked Autoencoders are Scalable Vision Learners (He et al., 2022) — MAE pretraining
- timm documentation — the reference for every vision transformer you will use in production

**Continue online.** The living edition of this chapter has more than the page can hold:

- Animated, interactive figures and the web text: <https://aiengineeringfromscratch.com/lesson?path=phases/04-computer-vision/14-vision-transformers>
- Runnable code for every step: <https://github.com/rohitg00/ai-engineering-from-scratch/tree/main/phases/04-computer-vision/14-vision-transformers/code>
- The chapter quiz, graded in the browser: <https://aiengineeringfromscratch.com/lesson?path=phases/04-computer-vision/14-vision-transformers>



The repository moves faster than any printing. When the book and the repo disagree, trust the repo.

# Real-Time Vision — Edge Deployment

---

Edge inference is the discipline of getting a 90-accuracy model to run at 30 fps on a device with 2 GB of RAM. Every percentage point of accuracy is traded against milliseconds of latency.

**Type:** Learn + Build **Languages:** Python **Prerequisites:** Phase 4 Lesson 04 (Image Classification), Phase 10 Lesson 11 (Quantization) **Time:** ~75 minutes

## Learning Objectives

- Measure inference latency, peak memory, and throughput for any PyTorch model, and read the FLOPs / params / latency trade-off
- Quantise a vision model to INT8 using PyTorch's post-training quantisation and verify accuracy loss < 1%
- Export to ONNX and compile with ONNX Runtime or TensorRT; name the three most common export failures and their fixes
- Explain when to pick MobileNetV3, EfficientNet-Lite, ConvNeXt-Tiny, or MobileViT for an edge constraint

## The Problem

A training-time vision model is a floating-point monster. 100M parameters, 10 GFLOPs per forward pass, 2 GB of VRAM. None of that fits on a phone, a car's infotainment unit, an industrial camera, or a drone. Shipping a vision system means fitting the same predictions into a budget that is 100x smaller.

Three knobs do most of the work: model choice (a smaller architecture with the same recipe), quantisation (INT8 instead of FP32), and the inference runtime (ONNX Runtime, TensorRT, Core ML, TFLite). Getting them right is the difference between a demo that runs on a workstation and a product that ships on a \$30 camera module.

This lesson sets up the measurement discipline first (you cannot optimise what you cannot measure), then walks the three knobs. The goal is not to learn every edge runtime but to know what levers exist and how to verify each one does what you think.

## The Concept

### The three budgets

**Diagram.** Rendered live in the web edition: <https://aiengineeringfromscratch.com/lesson?path=phases/04-computer-vision/15-real-time-edge>

- **Latency:** p50, p95, p99. Averaging only p50 hides tail behaviour that matters for real-time systems.
- **Peak memory:** the maximum the device ever sees, not the steady-state average. Matters because OOMs are fatal on embedded targets.

- **Power / energy:** millijoules per inference on a battery-powered device. Often proxied by CPU/GPU utilisation * time.

A table of (model, latency, memory, accuracy) is what an edge decision is made from. Every cell is measured on the target device, not the workstation.

## Measurement discipline

Three rules that every edge profile should follow:

1. **Warm up** the model with 5-10 dummy forward passes before measuring. Cold caches and JIT compilation produce unrepresentative first numbers.
2. **Synchronise** GPU workloads with `torch.cuda.synchronize()` before and after the timed block. Without this you measure kernel dispatch, not kernel execution.
3. **Fix input sizes** to the production resolution. Latency on 224x224 is not latency on 512x512.

## FLOPs as a proxy

FLOPs (floating-point operations per inference) is a cheap, device-independent proxy for latency. Useful for architecture comparison, misleading as absolute wall-clock. A model with 10% more FLOPs can be 2x faster in practice because it uses hardware-friendly ops (depth-wise convs compile well, large 7x7 convs do not).

Rule: use FLOPs for architecture search, use on-device latency for deployment decisions.

## Quantisation in one paragraph

Replace FP32 weights and activations with INT8. Model size drops 4x, memory bandwidth drops 4x, compute drops 2-4x on hardware that has INT8 kernels (every modern mobile SoC, every NVIDIA GPU with Tensor Cores). Accuracy loss on vision tasks is typically 0.1-1 percentage points with post-training static quantisation.

Types:

- **Dynamic** — quantise weights to INT8, activations computed in FP. Easy, small speedup.
- **Static (post-training)** — quantise weights + calibrate activation ranges on a small calibration set. Much faster than dynamic.
- **Quantisation-aware training (QAT)** — simulate quantisation during training so the model learns around it. Best accuracy, needs labelled data.

For vision, post-training static quantisation gives 95% of the benefit with 5% of the effort. Use QAT only when accuracy loss from PTQ is unacceptable.

## Pruning and distillation

- **Pruning** — remove unimportant weights (magnitude-based) or channels (structured). Works well on overparameterised models; less useful on already-compact architectures.
- **Distillation** — train a small student to mimic a large teacher's logits. Often recovers most of the accuracy lost by shrinking the model. Standard for production edge models.

## The inference runtimes

- **PyTorch eager** — slow, not for deployment. Use for development only.
- **TorchScript** — legacy. Superseded by `torch.compile` and ONNX export.

- **ONNX Runtime** — the neutral runtime. CPU, CUDA, CoreML, TensorRT, OpenVINO all have ONNX providers. Start here.
- **TensorRT** — NVIDIA's compiler. Best latency on NVIDIA GPUs (workstation and Jetson). Integrates with ONNX Runtime or standalone.
- **Core ML** — Apple's runtime for iOS/macOS. Needs .mlmodel or .mlpackage.
- **TFLite** — Google's runtime for Android/ARM. Needs .tflite.
- **OpenVINO** — Intel's runtime for CPU/VPU. Needs .xml + .bin.

In practice: export PyTorch -> ONNX -> pick the runtime for the target. ONNX is the lingua franca.

## Edge architecture picker

Budget	Model	Why
< 3M params	MobileNetV3-Small	Compiles everywhere, good baseline
3-10M	EfficientNet-Lite-B0	Best accuracy per param on TFLite
10-20M	ConvNeXt-Tiny	Best accuracy-per-param, CPU-friendly
20-30M	MobileViT-S or EfficientViT	Transformer with ImageNet accuracy
30-80M	Swin-V2-Tiny	If stack supports window attention

Quantise all of these to INT8 unless you have a specific reason not to.

**Interactive figure: cnn-param-count.** This one moves. Watch it animate and drag its controls in the web edition: <https://aiengineeringfromscratch.com/lesson?path=phases/04-computer-vision/15-real-time-edge>

## Build It

### Step 1: Measure latency correctly

```
import time
import torch

def measure_latency(model, input_shape, device="cpu", warmup=10, iters=50):
    model = model.to(device).eval()
    x = torch.randn(input_shape, device=device)
    with torch.no_grad():
        for _ in range(warmup):
            model(x)
        if device == "cuda":
            torch.cuda.synchronize()
        times = []
        for _ in range(iters):
            if device == "cuda":
                torch.cuda.synchronize()
```

```

        t0 = time.perf_counter()
        model(x)
        if device == "cuda":
            torch.cuda.synchronize()
        times.append((time.perf_counter() - t0) * 1000)
    times.sort()
    return {
        "p50_ms": times[len(times) // 2],
        "p95_ms": times[int(len(times) * 0.95)],
        "p99_ms": times[int(len(times) * 0.99)],
        "mean_ms": sum(times) / len(times),
    }

```

Warm up, synchronise, use `time.perf_counter()`. Report percentiles, not just mean.

## Step 2: Parameter and FLOP counts

```

def parameter_count(model):
    return sum(p.numel() for p in model.parameters())

def flops_estimate(model, input_shape):
    """
    Rough FLOP count for a conv/linear-only model. For production use `fvcore` or
    ↪ `ptflops`.
    """
    total = 0
    def conv_hook(m, inp, out):
        nonlocal total
        c_out, c_in, kh, kw = m.weight.shape
        h, w = out.shape[-2:]
        total += 2 * c_in * c_out * kh * kw * h * w
    def linear_hook(m, inp, out):
        nonlocal total
        total += 2 * m.in_features * m.out_features
    hooks = []
    for m in model.modules():
        if isinstance(m, torch.nn.Conv2d):
            hooks.append(m.register_forward_hook(conv_hook))
        elif isinstance(m, torch.nn.Linear):
            hooks.append(m.register_forward_hook(linear_hook))
    model.eval()
    with torch.no_grad():
        model(torch.randn(input_shape))
    for h in hooks:
        h.remove()
    return total

```

For real projects use `fvcore.nn.FlopCountAnalysis` or `ptflops`; they handle every module type correctly.

## Step 3: Post-training static quantisation

```

def quantise_ptq(model, calibration_loader, backend="x86"):
    import torch.ao.quantization as tq

```

```

model = model.eval().cpu()
model.qconfig = tq.get_default_qconfig(backend)
tq.prepare(model, inplace=True)
with torch.no_grad():
    for x, _ in calibration_loader:
        model(x)
tq.convert(model, inplace=True)
return model

```

Three steps: configure, prepare (insert observers), calibrate with real data, convert (fuse + quantise). Requires the model to be fused (Conv -> BN -> ReLU -> ConvBnReLU), which `torch.ao.quantization.fuse_modules` handles.

## Step 4: Export to ONNX

```

def export_onnx(model, sample_input, path="model.onnx"):
    model = model.eval()
    torch.onnx.export(
        model,
        sample_input,
        path,
        input_names=["input"],
        output_names=["output"],
        dynamic_axes={"input": {0: "batch"}, "output": {0: "batch"}},
        opset_version=17,
    )
    return path

```

`opset_version=17` is the safe default in 2026. `dynamic_axes` lets you run the ONNX model with arbitrary batch size.

## Step 5: Benchmark and compare regimes

```

import torch.nn as nn
from torchvision.models import mobilenet_v3_small

def compare_regimes():
    model = mobilenet_v3_small(weights=None, num_classes=10)
    params = parameter_count(model)
    flops = flops_estimate(model, (1, 3, 224, 224))
    lat_fp32 = measure_latency(model, (1, 3, 224, 224), device="cpu")
    print(f"FP32 MobileNetV3-Small: {params:,} params {flops/1e9:.2f} GFLOPs "
          f"p50={lat_fp32['p50_ms']:.2f}ms p95={lat_fp32['p95_ms']:.2f}ms")

```

Run the same function for `resnet50`, `efficientnet_v2_s`, and `convnext_tiny` and you have the comparison table you need for a deployment decision.

## Use It

Production stacks converge on one of three paths:

- **Web / serverless:** PyTorch -> ONNX -> ONNX Runtime (CPU or CUDA provider). Easiest, good enough for most.

- **NVIDIA edge (Jetson, GPU server):** PyTorch -> ONNX -> TensorRT. Best latency, biggest engineering effort.
- **Mobile:** PyTorch -> ONNX -> Core ML (iOS) or TFLite (Android). Quantise before export.

For measurement, `torch-tb-profiler`, `nvprof` / `nsys`, and Instruments on macOS give layer-by-layer breakdowns. `benchmark_app` (OpenVINO) and `trtexec` (TensorRT) give standalone CLI numbers.

**This chapter ships an artifact.** The course version of this lesson produces a reusable prompt or agent skill. It lives in the repository, ready to install: <https://github.com/rohitg00/ai-engineering-from-scratch/tree/main/phases/04-computer-vision/15-real-time-edge>

## Exercises

Starter code and the lesson’s working implementation: <https://github.com/rohitg00/ai-engineering-from-scratch/tree/main/phases/04-computer-vision/15-real-time-edge/code>

1. **(Easy)** Measure p50 latency for `resnet18`, `mobilenet_v3_small`, `efficientnet_v2_s`, and `convnext_tiny` at 224x224 on CPU. Report the table and identify which architecture has the best accuracy-per-ms.
2. **(Medium)** Apply post-training static quantisation to `mobilenet_v3_small`. Report FP32 vs INT8 latency and accuracy loss on a held-out subset of CIFAR-10 or similar.
3. **(Hard)** Export `convnext_tiny` to ONNX, run it through `onnxruntime` with the `CPUExecutionProvider`, and compare latency to the PyTorch eager baseline. Identify the first layer where ONNX Runtime is faster and explain why.

## Key Terms

Term	What people say	What it actually means
Latency	“How fast”	Time from input to output; p50/p95/p99 percentiles, not mean
FLOPs	“Model size”	Floating-point ops per forward pass; rough proxy for compute cost
INT8 quantisation	“8-bit”	Replace FP32 weights/activations with 8-bit integers; ~4x smaller, 2-4x faster
PTQ	“Post-training quantisation”	Quantise a trained model without retraining; easy, usually enough
QAT	“Quantisation-aware training”	Simulate quantisation during training; best accuracy, requires labelled data
ONNX	“The neutral format”	Model exchange format supported by every mainstream inference runtime
TensorRT	“NVIDIA compiler”	Compiles ONNX into an optimised engine for NVIDIA GPUs
Distillation	“Teacher -> student”	Train a small model to mimic a big model’s logits; recovers most lost accuracy

## Further Reading

- EfficientNet (Tan & Le, 2019) — compound scaling for efficient architectures

- MobileNetV3 (Howard et al., 2019) — mobile-first architecture with h-swish and squeeze-excite
- A Practical Guide to TensorRT Optimization (NVIDIA) — how to actually get the throughput numbers in the paper
- ONNX Runtime docs — quantisation, graph optimisation, provider selection

**Continue online.** The living edition of this chapter has more than the page can hold:

- Animated, interactive figures and the web text: <https://aiengineeringfromscratch.com/lesson?path=phases/04-computer-vision/15-real-time-edge>
- Runnable code for every step: <https://github.com/rohitg00/ai-engineering-from-scratch/tree/main/phases/04-computer-vision/15-real-time-edge/code>
- The chapter quiz, graded in the browser: <https://aiengineeringfromscratch.com/lesson?path=phases/04-computer-vision/15-real-time-edge>

The repository moves faster than any printing. When the book and the repo disagree, trust the repo.



# Build a Complete Vision Pipeline — Capstone

---

A production vision system is a chain of models and rules stitched with data contracts. The pieces are already in this phase; the capstone wires them together end-to-end.

**Type:** Build **Languages:** Python **Prerequisites:** Phase 4 Lessons 01-15 **Time:** ~120 minutes

## Learning Objectives

- Design a production vision pipeline that detects objects, classifies them, and emits structured JSON — with every failure path handled
- Plug a detector (Mask R-CNN or YOLO), a classifier (ConvNeXt-Tiny), and a data contract (Pydantic) into one service
- Benchmark the end-to-end pipeline and identify the first bottleneck (usually preprocessing, then the detector)
- Ship a minimal FastAPI service that accepts an image upload, runs the pipeline, and returns detections with classifications

## The Problem

Individual vision models are useful; vision products are chains of them. A retail shelf audit is a detector plus a product classifier plus a price-OCR pipeline. Autonomous driving is a 2D detector plus a 3D detector plus a segmenter plus a tracker plus a planner. A medical pre-screen is a segmenter plus a region classifier plus a clinician UI.

Wiring those chains is the part that separates a ML prototype from a product. Every interface between models is a new place for bugs. Every coordinate transform, every normalisation, every mask resize is a silent-failure candidate. A pipeline is as strong as its weakest interface.

This capstone sets up the minimum viable pipeline: detection + classification + structured output + a serving layer. Everything else in Phase 4 slots into this skeleton: swap Mask R-CNN for YOLOv8, add a OCR head, add a segmentation branch, add a tracker. The architecture is stable; the pieces are pluggable.

## The Concept

### The pipeline

**Diagram.** Rendered live in the web edition: <https://aiengineeringfromscratch.com/lesson?path=phases/04-computer-vision/16-vision-pipeline-capstone>

Seven stages. The two model stages are expensive; the five other stages are where the bugs live.

## Data contracts with Pydantic

Every model boundary becomes a typed object. This turns silent failures into loud ones.

```
Detection(
    box: tuple[float, float, float, float],    # (x1, y1, x2, y2), absolute pixels
    score: float,                             # [0, 1]
    class_id: int,                            # from detector's label map
    mask: Optional[list[list[int]]],          # RLE-encoded if present
)

PipelineResult(
    image_id: str,
    detections: list[Detection],
    classifications: list[Classification],
    inference_ms: float,
)
```

When a detector returns boxes in (cx, cy, w, h) instead of (x1, y1, x2, y2), Pydantic's validation fails at the boundary and you find out immediately instead of debugging a downstream crop that silently returns empty regions.

## Where latency goes

Three truths hold in nearly every vision pipeline:

1. **Preprocessing is often the biggest single block.** Decoding JPEGs, converting colour spaces, resizing — these are CPU-bound and easy to forget.
2. **The detector dominates GPU time.** 70-90% of GPU time is in the detection forward pass.
3. **Postprocessing (NMS, RLE encode/decode) is cheap on GPU, expensive on CPU.** Always profile with the actual target.

Knowing the distribution is what turns optimisation into a prioritised list.

## Failure modes

- **Empty detections** — return empty list, do not crash. Log.
- **Out-of-bounds boxes** — clamp to image size before cropping.
- **Tiny crops** — skip classification for boxes smaller than the classifier's minimum input.
- **Corrupt upload** — 400 response with a specific error code, not 500.
- **Model load failure** — fail at service startup, not at first request.

A production pipeline handles each of these without writing generic try/except that hides the failure. Every failure gets a named code and a response.

## Batching

A production service serves multiple clients. Batching detections and classifications across requests multiplies throughput. The trade-off: extra latency from waiting for a batch to fill. Typical setup: collect requests for up to 20ms, batch together, process, distribute responses. torchserve and triton do this natively; small services with predictable load roll their own micro-batcher.

**Interactive figure: v4-vision-pipeline.** This one moves. Watch it animate and drag its controls in the web edition: <https://aiengineeringfromscratch.com/lesson?path=phases/04-computer-vision/16-vision-pipeline-capstone>

## Build It

### Step 1: Data contracts

```

from pydantic import BaseModel, Field
from typing import List, Optional, Tuple

class Detection(BaseModel):
    box: Tuple[float, float, float, float]
    score: float = Field(ge=0, le=1)
    class_id: int = Field(ge=0)
    mask_rle: Optional[str] = None

class Classification(BaseModel):
    detection_index: int
    class_id: int
    class_name: str
    score: float = Field(ge=0, le=1)

class PipelineResult(BaseModel):
    image_id: str
    detections: List[Detection]
    classifications: List[Classification]
    inference_ms: float

```

Five seconds of code saves an hour of debugging on any serious pipeline.

### Step 2: A minimal Pipeline class

```

import time
import numpy as np
import torch
from PIL import Image

class VisionPipeline:
    def __init__(self, detector, classifier, class_names,
                 device="cpu", min_crop=32):
        self.detector = detector.to(device).eval()
        self.classifier = classifier.to(device).eval()
        self.class_names = class_names
        self.device = device
        self.min_crop = min_crop

    def preprocess(self, image):
        """
        image: PIL.Image or np.ndarray (H, W, 3) uint8
        returns: CHW float tensor on device
        """
        if isinstance(image, Image.Image):
            image = np.asarray(image.convert("RGB"))
        tensor = torch.from_numpy(image).permute(2, 0, 1).float() / 255.0
        return tensor.to(self.device)

```

```

@torch.no_grad()
def detect(self, image_tensor):
    return self.detector([image_tensor])[0]

@torch.no_grad()
def classify(self, crops):
    if len(crops) == 0:
        return []
    batch = torch.stack(crops).to(self.device)
    logits = self.classifier(batch)
    probs = logits.softmax(-1)
    scores, cls = probs.max(-1)
    return list(zip(cls.tolist(), scores.tolist()))

def run(self, image, image_id="anonymous"):
    t0 = time.perf_counter()
    tensor = self.preprocess(image)
    det = self.detect(tensor)

    crops = []
    detections = []
    valid_indices = []
    for i, (box, score, cls) in enumerate(zip(det["boxes"], det["scores"],
        ↪ det["labels"])):
        x1, y1, x2, y2 = [max(0, int(b)) for b in box.tolist()]
        x2 = min(x2, tensor.shape[-1])
        y2 = min(y2, tensor.shape[-2])
        detections.append(Detection(
            box=(x1, y1, x2, y2),
            score=float(score),
            class_id=int(cls),
        ))
        if (x2 - x1) < self.min_crop or (y2 - y1) < self.min_crop:
            continue
        crop = tensor[:, y1:y2, x1:x2]
        crop = torch.nn.functional.interpolate(
            crop.unsqueeze(0),
            size=(224, 224),
            mode="bilinear",
            align_corners=False,
        )[0]
        crops.append(crop)
        valid_indices.append(i)

    class_preds = self.classify(crops)

    classifications = []
    for valid_idx, (cls_id, cls_score) in zip(valid_indices, class_preds):
        classifications.append(Classification(
            detection_index=valid_idx,
            class_id=int(cls_id),
            class_name=self.class_names[cls_id],
            score=float(cls_score),

```

```

    ))

    return PipelineResult(
        image_id=image_id,
        detections=detections,
        classifications=classifications,
        inference_ms=(time.perf_counter() - t0) * 1000,
    )

```

Every interface is typed. Every failure path has a specific handling decision.

### Step 3: Wire a detector and a classifier

```

from torchvision.models.detection import maskrcnn_resnet50_fpn_v2
from torchvision.models import convnext_tiny

# Use ImageNet-pretrained weights for a realistic pipeline without training
detector = maskrcnn_resnet50_fpn_v2(weights="DEFAULT")
classifier = convnext_tiny(weights="DEFAULT")
class_names = [f"imagenet_class_{i}" for i in range(1000)]

pipe = VisionPipeline(detector, classifier, class_names)

# Smoke test with a synthetic image
test_image = (np.random.rand(400, 600, 3) * 255).astype(np.uint8)
result = pipe.run(test_image, image_id="demo")
print(result.model_dump_json(indent=2)[:500])

```

### Step 4: FastAPI service

```

from fastapi import FastAPI, UploadFile, HTTPException
from io import BytesIO

app = FastAPI()
pipe = None # initialised on startup

@app.on_event("startup")
def load():
    global pipe
    detector = maskrcnn_resnet50_fpn_v2(weights="DEFAULT").eval()
    classifier = convnext_tiny(weights="DEFAULT").eval()
    pipe = VisionPipeline(detector, classifier, class_names=[f"c{i}" for i in
↪ range(1000)])

@app.post("/detect")
async def detect_endpoint(file: UploadFile):
    if file.content_type not in {"image/jpeg", "image/png", "image/webp"}:
        raise HTTPException(status_code=400, detail="unsupported image type")
    data = await file.read()
    try:
        img = Image.open(BytesIO(data)).convert("RGB")
    except Exception:
        raise HTTPException(status_code=400, detail="cannot decode image")

```

```

    result = pipe.run(img, image_id=file.filename or "upload")
    return result.model_dump()

```

Run with `uvicorn main:app --host 0.0.0.0 --port 8000`. Test with `curl -F 'file=@dog.jpg' http://localhost:8000/detect`.

## Step 5: Benchmark the pipeline

```
import time
```

```

def benchmark(pipe, num_runs=20, image_size=(400, 600)):
    img = (np.random.rand(*image_size, 3) * 255).astype(np.uint8)
    pipe.run(img)  # warm up

    stages = {"preprocess": [], "detect": [], "classify": [], "total": []}
    for _ in range(num_runs):
        t0 = time.perf_counter()
        tensor = pipe.preprocess(img)
        t1 = time.perf_counter()
        det = pipe.detect(tensor)
        t2 = time.perf_counter()
        crops = []
        for box in det["boxes"]:
            x1, y1, x2, y2 = [max(0, int(b)) for b in box.tolist()]
            x2 = min(x2, tensor.shape[-1])
            y2 = min(y2, tensor.shape[-2])
            if (x2 - x1) >= pipe.min_crop and (y2 - y1) >= pipe.min_crop:
                crop = tensor[:, y1:y2, x1:x2]
                crop = torch.nn.functional.interpolate(
                    crop.unsqueeze(0), size=(224, 224), mode="bilinear",
                    align_corners=False
                )[0]
                crops.append(crop)
        pipe.classify(crops)
        t3 = time.perf_counter()
        stages["preprocess"].append((t1 - t0) * 1000)
        stages["detect"].append((t2 - t1) * 1000)
        stages["classify"].append((t3 - t2) * 1000)
        stages["total"].append((t3 - t0) * 1000)

    for stage, times in stages.items():
        times.sort()
        print(f"{stage:12s} p50={times[len(times)//2]:7.1f} ms
              ↪ p95={times[int(len(times)*0.95)]:7.1f} ms")

```

Typical output on CPU: preprocess ~3 ms, detect 300-500 ms, classify 20-40 ms, total 350-550 ms. On GPU, detect is 20-40 ms and the preprocess + classify start to matter more in relative terms.

## Use It

Production templates converge to the same structure, plus:

- **Model versioning** — always log the model name and weights hash in the response.

- **Per-request trace IDs** — log every stage timing for every request so you can correlate slow responses with stages.
- **Fallback path** — if the classifier times out, return detections without classifications rather than failing the whole request.
- **Safety filters** — NSFW / PII filters run after classification, before the response leaves the service.
- **Batch endpoint** — a `/detect_batch` accepting a list of image URLs for bulk processing.

For production serving, `torchserve`, `Triton Inference Server`, and `BentoML` handle batching, versioning, metrics, and health checks out of the box. Running `FastAPI` directly is fine for prototypes and small-scale products.

**This chapter ships an artifact.** The course version of this lesson produces a reusable prompt or agent skill. It lives in the repository, ready to install: <https://github.com/rohitg00/ai-engineering-from-scratch/tree/main/phases/04-computer-vision/16-vision-pipeline-capstone>

## Exercises

Starter code and the lesson’s working implementation: <https://github.com/rohitg00/ai-engineering-from-scratch/tree/main/phases/04-computer-vision/16-vision-pipeline-capstone/code>

1. **(Easy)** Run the pipeline on 10 images from any open dataset. Report the average time per stage and the distribution of detection counts per image.
2. **(Medium)** Add a mask output field to `Detection` and encode it as RLE. Verify the JSON stays under 1MB even for a 10-object image.
3. **(Hard)** Add a micro-batcher in front of the classifier: collect crops for up to 10 ms, classify them all in one GPU call, return results per request. Measure the throughput gain at 5 concurrent requests per second and the latency added.

## Key Terms

Term	What people say	What it actually means
Pipeline	“The system”	An ordered chain of preprocessing, inference, and postprocessing steps with a typed interface between each pair
Data contract	“The schema”	Pydantic / dataclass definitions that every stage input and output conforms to; catches integration bugs at the boundary
Preprocessing	“Before the model”	Decoding, colour conversion, resizing, normalising; usually the biggest CPU time sink
Postprocessing	“After the model”	NMS, mask resize, threshold, RLE encode; cheap on GPU, expensive on CPU
Microbatcher	“Collect then forward”	Aggregator that waits a fixed window for multiple requests, runs a single batched forward pass
Trace ID	“Request id”	Per-request identifier logged at every stage so slow requests can be traced end-to-end
Failure code	“Named error”	Specific error code per failure class instead of generic 500; enables client retry logic

Term	What people say	What it actually means
Health check	“Readiness probe”	Cheap endpoint that reports whether the service can answer; loadbalancers rely on this

## Further Reading

- Full Stack Deep Learning — Deploying Models — the canonical overview of production ML deployment
- BentoML docs — serving framework with batching, versioning, and metrics
- torchserve docs — PyTorch’s official serving library
- NVIDIA Triton Inference Server — high-throughput serving with batching and multi-model support

**Continue online.** The living edition of this chapter has more than the page can hold:

- Animated, interactive figures and the web text: <https://aiengineeringfromscratch.com/lesson?path=phases/04-computer-vision/16-vision-pipeline-capstone>
- Runnable code for every step: <https://github.com/rohitg00/ai-engineering-from-scratch/tree/main/phases/04-computer-vision/16-vision-pipeline-capstone/code>
- The chapter quiz, graded in the browser: <https://aiengineeringfromscratch.com/lesson?path=phases/04-computer-vision/16-vision-pipeline-capstone>

The repository moves faster than any printing. When the book and the repo disagree, trust the repo.



# Self-Supervised Vision — SimCLR, DINO, MAE

---

Labels are the bottleneck of supervised vision. Self-supervised pretraining removes them: learn visual features from 100M unlabelled images, fine-tune on 10k labelled ones.

**Type:** Learn + Build **Languages:** Python **Prerequisites:** Phase 4 Lesson 04 (Image Classification), Phase 4 Lesson 14 (ViT) **Time:** ~75 minutes

## Learning Objectives

- Trace the three major self-supervised families — contrastive (SimCLR), teacher-student (DINO), masked reconstruction (MAE) — and state what each one optimises
- Implement an InfoNCE loss from scratch and explain why a batch of 512 works but a batch of 32 fails
- Explain why MAE's 75% masking ratio is not arbitrary and how it differs from BERT's 15% for text
- Use DINOv2 or MAE ImageNet checkpoints for linear probing and zero-shot retrieval

## The Problem

Supervised ImageNet has 1.3M labelled images, which cost an estimated \$10M to annotate. Medical and industrial datasets are smaller and even more expensive to label. Every vision team asks: can we pretrain on cheap unlabelled data — YouTube frames, web crawls, webcam footage, satellite sweeps — and then fine-tune on a small labelled set?

Self-supervised learning is the answer. A modern self-supervised ViT trained on LAION or JFT reaches or beats supervised ImageNet accuracy when fine-tuned. It also transfers better to downstream tasks (detection, segmentation, depth) than supervised pretraining. DINOv2 (Meta, 2023) and MAE (Meta, 2022) are the current production defaults for transferable vision features.

The conceptual shift is that the pretext task — the thing the model is trained to do — does not have to be the downstream task. What matters is that it forces the model to learn useful features. Predict the colour of grayscale images, rotate images and ask the model to classify the rotation, mask patches and reconstruct them — all have worked. The three approaches that scale are contrastive learning, teacher-student distillation, and masked reconstruction.

## The Concept

### Three families

**Diagram.** Rendered live in the web edition: <https://aiengineeringfromscratch.com/lesson?path=phases/04-computer-vision/17-self-supervised-vision>

## Contrastive learning (SimCLR)

Take one image, apply two random augmentations, get two views. Feed both through the same encoder plus a projection head. Minimise a loss that says “these two embeddings should be close” and “this embedding should be far from every other image’s embeddings in the batch.”

Loss for positive pair  $(z_i, z_j)$  among  $2N$  views per batch:

$$L_{ij} = -\log\left(\frac{\exp(\text{sim}(z_i, z_j) / \tau)}{\sum_{k \text{ in batch } \setminus \{i\}} \exp(\text{sim}(z_i, z_k) / \tau)}\right)$$

$\text{sim}$  = cosine similarity  
 $\tau$  = temperature (0.1 standard)

This is the InfoNCE loss. It requires many negatives per positive, so batch size matters — SimCLR needs 512-8192. MoCo introduced a momentum queue of past batches to decouple negative count from batch size.

## Teacher-student (DINO)

Two networks with the same architecture: student and teacher. The teacher is an exponential moving average (EMA) of the student’s weights. Both see augmented views of the image. The student’s output is trained to match the teacher’s — no explicit negatives.

```
loss = CE( student_output(view_1), teacher_output(view_2) )
      + CE( student_output(view_2), teacher_output(view_1) )
```

```
teacher_weights = m * teacher_weights + (1 - m) * student_weights    (m ≈ 0.996)
```

Why it does not collapse to “predict a constant”: the teacher’s output is centred (subtract per-dimension mean) and sharpened (divide by small temperature). Centering prevents one dimension from dominating; sharpening prevents output collapse to uniform.

DINO is what DINOv2 scales up, on 142M curated images. The resulting features are the current SOTA for zero-shot visual retrieval and dense prediction.

## Masked reconstruction (MAE)

Mask 75% of patches of a ViT input. Pass only the visible 25% through the encoder. A small decoder receives the encoder’s output plus mask tokens at masked positions, and is trained to reconstruct the pixels of the masked patches.

Encoder: visible 25% of patches -> features

Decoder: features + mask tokens at masked positions -> reconstructed pixels

Loss: MSE between reconstructed and original pixels on masked patches only

Key design choices that make MAE work:

- **75% mask ratio** — high. Forces the encoder to learn semantic features; reconstructing 25% would be near-trivial (neighbouring pixels are so correlated that a CNN could nail it).
- **Asymmetric encoder/decoder** — the big ViT encoder only sees visible patches; a small decoder (8-layer, 512-dim) handles reconstruction. 3x faster pretraining than naive BEiT.
- **Pixel-space reconstruction target** — simpler than BEiT’s tokenised target and works better on ViT.

After pretraining, discard the decoder. The encoder is the feature extractor.

## Why 75% and not 15%

BERT masks 15% of tokens. MAE masks 75%. The difference is information density.

- Natural language has high entropy per token. Predicting 15% of tokens is still hard because each masked position has many plausible completions.
- Image patches have low entropy — an unmasked neighbourhood often determines the masked patch's pixels almost exactly. To make prediction require semantic understanding, you have to mask aggressively.

75% is high enough that simple spatial extrapolation cannot solve the task; the encoder must represent the image content.

## Linear-probe evaluation

After self-supervised pretraining, the standard evaluation is a **linear probe**: freeze the encoder, train a single linear classifier on top on ImageNet labels. Reports top-1 accuracy.

- SimCLR ResNet-50: ~71% (2020)
- DINO ViT-S/16: ~77% (2021)
- MAE ViT-L/16: ~76% (2022)
- DINOv2 ViT-g/14: ~86% (2023)

Linear probe is a pure measure of feature quality; fine-tuning typically adds 2-5 points but also mixes in the effect of head retraining.

**Interactive figure: data-augmentation.** This one moves. Watch it animate and drag its controls in the web edition: <https://aiengineeringfromscratch.com/lesson?path=phases/04-computer-vision/17-self-supervised-vision>

## Build It

### Step 1: Two-view augmentation pipeline

```
import torch
import torchvision.transforms as T

two_view_train = lambda: T.Compose([
    T.RandomResizedCrop(96, scale=(0.2, 1.0)),
    T.RandomHorizontalFlip(),
    T.ColorJitter(0.4, 0.4, 0.4, 0.1),
    T.RandomGrayscale(p=0.2),
    T.ToTensor(),
])

class TwoViewDataset(torch.utils.data.Dataset):
    def __init__(self, base):
        self.base = base
        self.aug = two_view_train()

    def __len__(self):
        return len(self.base)

    def __getitem__(self, i):
```

```

img, _ = self.base[i]
v1 = self.aug(img)
v2 = self.aug(img)
return v1, v2

```

Each **getitem** returns two augmented views of the same image; labels are not needed.

## Step 2: InfoNCE loss

```
import torch.nn.functional as F
```

```

def info_nce(z1, z2, tau=0.1):
    """
    z1, z2: (N, D) L2-normalised embeddings of paired views
    """
    N, D = z1.shape
    z = torch.cat([z1, z2], dim=0) # (2N, D)
    sim = z @ z.T / tau             # (2N, 2N)

    mask = torch.eye(2 * N, dtype=torch.bool, device=z.device)
    sim = sim.masked_fill(mask, float("-inf"))

    targets = torch.cat([torch.arange(N, 2 * N), torch.arange(0, N)]).to(z.device)
    return F.cross_entropy(sim, targets)

```

L2-normalise embeddings before calling.  $\tau=0.1$  is the SimCLR default; lower makes the loss sharper and requires more negatives.

## Step 3: Sanity check InfoNCE

```

z1 = F.normalize(torch.randn(16, 32), dim=-1)
z2 = z1.clone()
loss_same = info_nce(z1, z2, tau=0.1).item()
z2_random = F.normalize(torch.randn(16, 32), dim=-1)
loss_random = info_nce(z1, z2_random, tau=0.1).item()
print(f"InfoNCE with identical pairs: {loss_same:.3f}")
print(f"InfoNCE with random pairs: {loss_random:.3f}")

```

Identical pairs should give a low loss (close to 0 for a large batch and cold temperature). Random pairs should give  $\log(2N-1) = \sim \log(31) = \sim 3.4$  with a 16-pair batch.

## Step 4: MAE-style masking

```

def random_mask_indices(num_patches, mask_ratio=0.75, seed=0):
    g = torch.Generator().manual_seed(seed)
    n_keep = int(num_patches * (1 - mask_ratio))
    perm = torch.randperm(num_patches, generator=g)
    visible = perm[:n_keep]
    masked = perm[n_keep:]
    return visible.sort().values, masked.sort().values

```

```

num_patches = 196
visible, masked = random_mask_indices(num_patches, mask_ratio=0.75)

```

```
print(f"visible: {len(visible)} / {num_patches}")
print(f"masked: {len(masked)} / {num_patches}")
```

Simple, fast, and deterministic for a given seed. Real MAE implementations batch this and keep per-sample masks.

## Use It

DINOv2 is the production standard in 2026:

```
import torch
from transformers import AutoImageProcessor, AutoModel

processor = AutoImageProcessor.from_pretrained("facebook/dinov2-base")
model = AutoModel.from_pretrained("facebook/dinov2-base")
model.eval()

# Per-image embeddings for zero-shot retrieval
with torch.no_grad():
    inputs = processor(images=[pil_image], return_tensors="pt")
    outputs = model(**inputs)
    embedding = outputs.last_hidden_state[:, 0] # CLS token
```

The resulting 768-dim embedding is the backbone of modern image retrieval, dense correspondence, and zero-shot transfer pipelines. Fine-tuning on a downstream task rarely needs more than a linear head.

For image-text embeddings, SigLIP or OpenCLIP is the equivalent; for MAE-style fine-tuning, the timm repo ships every MAE checkpoint.

**This chapter ships an artifact.** The course version of this lesson produces a reusable prompt or agent skill. It lives in the repository, ready to install: <https://github.com/rohitg00/ai-engineering-from-scratch/tree/main/phases/04-computer-vision/17-self-supervised-vision>

## Exercises

Starter code and the lesson's working implementation: <https://github.com/rohitg00/ai-engineering-from-scratch/tree/main/phases/04-computer-vision/17-self-supervised-vision/code>

1. **(Easy)** Verify that InfoNCE loss drops when you decrease temperature for well-aligned embeddings and rises when you decrease temperature for random embeddings. Produce a plot tau in [0.05, 0.1, 0.2, 0.5] vs loss.
2. **(Medium)** Implement a DINO-style centre buffer. Show that without the centring, the student collapses to a constant vector within a few epochs.
3. **(Hard)** Train MAE on CIFAR-100 using the TinyUNet from Lesson 10 as the backbone. Report linear-probe accuracy at 10, 50, and 200 epochs. Show that a MAE-pretrained linear probe beats a from-scratch supervised linear probe on the same 1,000-image subset.

## Key Terms

Term	What people say	What it actually means
Self-supervised	“Label-free”	A pretext task that produces useful representations from unlabelled data
Pretext task	“The fake task”	The objective used during SSL (reconstruct patches, match views); discarded after pretraining
Linear probe	“Frozen encoder + linear head”	Standard SSL evaluation: train only a linear classifier on top of frozen features
InfoNCE	“Contrastive loss”	softmax over cosine similarities; positive pair is the target class, all others are negatives
EMA teacher	“Moving-average teacher”	Teacher whose weights are an exponential moving average of the student’s; used by BYOL, MoCo, DINO
Mask ratio	“% of patches hidden”	Fraction of patches masked during MAE; 75% for vision, 15% for text
Representation collapse	“Constant output”	SSL failure where the encoder outputs a constant vector for all inputs; prevented by centring, sharpening, or negatives
DINOv2	“Production SSL backbone”	Meta’s 2023 self-supervised ViT; strongest general-purpose image features in 2026

## Further Reading

- SimCLR (Chen et al., 2020) — contrastive learning reference
- DINO (Caron et al., 2021) — teacher-student with momentum, centring, sharpening
- MAE (He et al., 2022) — masked autoencoder pretraining for ViT
- DINOv2 (Oquab et al., 2023) — scaling self-supervised ViT to production features

**Continue online.** The living edition of this chapter has more than the page can hold:

- Animated, interactive figures and the web text: <https://aiengineeringfromscratch.com/lesson?path=phases/04-computer-vision/17-self-supervised-vision>
- Runnable code for every step: <https://github.com/rohitg00/ai-engineering-from-scratch/tree/main/phases/04-computer-vision/17-self-supervised-vision/code>
- The chapter quiz, graded in the browser: <https://aiengineeringfromscratch.com/lesson?path=phases/04-computer-vision/17-self-supervised-vision>

The repository moves faster than any printing. When the book and the repo disagree, trust the repo.

# Open-Vocabulary Vision — CLIP

---

Train an image encoder and a text encoder together so that matching (image, caption) pairs land at the same point in a shared space. That is the whole trick.

**Type:** Build + Use **Languages:** Python **Prerequisites:** Phase 4 Lesson 14 (ViT), Phase 4 Lesson 17 (Self-Supervised) **Time:** ~45 minutes

## Learning Objectives

- Explain CLIP's two-tower architecture and contrastive training objective
- Use a pretrained CLIP (or SigLIP) for zero-shot classification without any task-specific training
- Implement zero-shot classification from scratch: encode class prompts, compute cosine similarity, take argmax
- Distinguish CLIP, SigLIP, OpenCLIP, and LLaVA/LLaMA-vision models — what each is for in 2026

## The Problem

Traditional classifiers are closed-vocabulary: a 1000-class ImageNet model can only predict 1000 labels. Every new category requires labelled data and a retrained head.

CLIP (Radford et al., OpenAI 2021) showed that training on 400M (image, caption) pairs scraped from the web produces a model that can classify into any set of categories at inference, described purely in natural language. You give it a new class by writing a sentence.

That capability — zero-shot transfer — is why every modern vision system starts with a CLIP-family checkpoint. Detection (Grounding DINO, OWL-ViT), segmentation (CLIPSeg, SAM), retrieval, content moderation, VLMs, and text-to-image generation all build on CLIP-style joint embeddings.

## The Concept

### Two towers

**Diagram.** Rendered live in the web edition: <https://aiengineeringfromscratch.com/lesson?path=phases/04-computer-vision/18-open-vocab-clip>

Both encoders end with a linear projection to the same embedding dimension (512 for CLIP-B/32, 1024 for CLIP-L/14). L2-normalise and compute cosine similarity.

### The objective

Given a batch of  $N$  (image, caption) pairs, build an  $N \times N$  similarity matrix. Train both encoders so the diagonal (matching pairs) has high similarity and off-diagonals (non-matching) have low similarity.

```

sim_matrix = image_embeddings @ text_embeddings.T / tau

loss_i2t = cross_entropy(sim_matrix, targets=arange(N))
loss_t2i = cross_entropy(sim_matrix.T, targets=arange(N))
loss = (loss_i2t + loss_t2i) / 2

```

Symmetric because both image-to-text and text-to-image retrieval should work.  $\tau$  (temperature) is typically learned as a scalar parameter, initialised to 0.07.

## SigLIP: a better loss

SigLIP (Zhai et al., 2023) replaced the softmax with per-pair sigmoid:

```

loss = mean over pairs of log(1 + exp(-y_ij * sim_ij))
y_ij = +1 if matching, -1 otherwise

```

Per-pair loss removes the batch-level normalisation that CLIP requires. SigLIP trains better at small batch sizes and matches or exceeds CLIP at equal data.

## Zero-shot classification

Given a trained CLIP:

1. For each class, compose a prompt: “a photo of a {class}”.
2. Encode all class prompts with the text encoder -> T shape (C, d).
3. Encode the test image -> I shape (1, d).
4. Similarity = I @ T.T shape (1, C).
5. Argmax -> predicted class.

Prompt engineering matters. OpenAI published 80 prompt templates for ImageNet (“a photo of a { }”, “a blurry photo of a { }”, “a sketch of a { }”, ...). Average the embeddings of all templates per class for an extra 1-3% top-1 accuracy.

## Where CLIP-style models are used in 2026

- **Zero-shot classification** — direct use.
- **Image retrieval** — encode all images once, embed query at inference.
- **Text-conditioned detection** — Grounding DINO, OWL-ViT wrap a CLIP text tower around a detector.
- **Text-conditioned segmentation** — CLIPSeg; SAM uses text-prompt inputs via CLIP.
- **VLMS** — LLaVA, Qwen-VL, InternVL wire a CLIP-family vision encoder into an LLM.
- **Text-to-image gen** — Stable Diffusion, DALL-E 3 condition on CLIP text embeddings.

Once you have a shared embedding space, every vision+language task becomes a distance computation.

**Interactive figure: clip-contrastive.** This one moves. Watch it animate and drag its controls in the web edition: <https://aiengineeringfromscratch.com/lesson?path=phases/04-computer-vision/18-open-vocab-clip>

## Build It

### Step 1: A tiny two-tower model

Real CLIP is ViT + transformer. For this lesson the towers are small MLPs over pre-extracted features so the training signal is visible on CPU.



```

import torch
import torch.nn as nn
import torch.nn.functional as F

class TwoTower(nn.Module):
    def __init__(self, img_in=128, txt_in=64, emb=64):
        super().__init__()
        self.image_proj = nn.Sequential(nn.Linear(img_in, 128), nn.ReLU(),
                                         ↪ nn.Linear(128, emb))
        self.text_proj = nn.Sequential(nn.Linear(txt_in, 128), nn.ReLU(),
                                         ↪ nn.Linear(128, emb))
        self.logit_scale = nn.Parameter(torch.ones([]) * 2.6592) # ln(1/0.07)

    def forward(self, img_feats, txt_feats):
        i = F.normalize(self.image_proj(img_feats), dim=-1)
        t = F.normalize(self.text_proj(txt_feats), dim=-1)
        return i, t, self.logit_scale.exp()

```

Two projections, shared-dim output, learned temperature. Same shape as the real CLIP API.

## Step 2: Contrastive loss

```

def clip_loss(image_emb, text_emb, logit_scale):
    N = image_emb.size(0)
    sim = logit_scale * image_emb @ text_emb.T
    targets = torch.arange(N, device=sim.device)
    l_i = F.cross_entropy(sim, targets)
    l_t = F.cross_entropy(sim.T, targets)
    return (l_i + l_t) / 2

```

Symmetric. Higher logit_scale = sharper softmax = more confident but risk of instability.

## Step 3: Zero-shot classifier

```

@torch.no_grad()
def zero_shot_classify(model, image_feats, class_text_feats, class_names):
    """
    image_feats:      (N, img_in)
    class_text_feats: (C, txt_in)  one averaged embedding per class
    """
    i = F.normalize(model.image_proj(image_feats), dim=-1)
    t = F.normalize(model.text_proj(class_text_feats), dim=-1)
    sim = i @ t.T
    pred = sim.argmax(dim=-1)
    return [class_names[p] for p in pred.tolist()]

```

One line per step. This is the exact zero-shot procedure used with a production CLIP checkpoint.

## Step 4: Sanity check

```

torch.manual_seed(0)
model = TwoTower()

```

```
img = torch.randn(8, 128)
txt = torch.randn(8, 64)
i, t, scale = model(img, txt)
loss = clip_loss(i, t, scale)
print(f"batch size: {i.size(0)}    loss: {loss.item():.3f}")
```

Loss should be close to  $\log(N) = \log(8) = 2.08$  for a randomly initialised model — the symmetric cross-entropy target when no structure is learned yet.

## Use It

OpenCLIP is the community default in 2026:

```
import open_clip
import torch
from PIL import Image

model, _, preprocess = open_clip.create_model_and_transforms("ViT-B-32",
    ↪ pretrained="laion2b_s34b_b79k")
tokenizer = open_clip.get_tokenizer("ViT-B-32")

image = preprocess(Image.open("dog.jpg")).unsqueeze(0)
text = tokenizer(["a photo of a dog", "a photo of a cat", "a photo of a car"])

with torch.no_grad():
    image_features = model.encode_image(image)
    text_features = model.encode_text(text)
    image_features = image_features / image_features.norm(dim=-1, keepdim=True)
    text_features = text_features / text_features.norm(dim=-1, keepdim=True)
    probs = (100.0 * image_features @ text_features.T).softmax(dim=-1)

print(probs)
```

SigLIP is newer, trains better at small scales, and is preferred for new work: [google/siglip-base-patch16-224](https://github.com/google/siglip-base-patch16-224). Hugging Face ships both.

**This chapter ships an artifact.** The course version of this lesson produces a reusable prompt or agent skill. It lives in the repository, ready to install: <https://github.com/rohitg00/ai-engineering-from-scratch/tree/main/phases/04-computer-vision/18-open-vocab-clip>

## Exercises

Starter code and the lesson’s working implementation: <https://github.com/rohitg00/ai-engineering-from-scratch/tree/main/phases/04-computer-vision/18-open-vocab-clip/code>

1. **(Easy)** Use a pretrained OpenCLIP ViT-B/32 and do zero-shot classification on CIFAR-10 with the 80-template prompt set. Report top-1 accuracy; it should be around 85-90%.
2. **(Medium)** Compare single-template (“a photo of a { }”) vs 80-template averaged embeddings on the same CIFAR-10 task. Quantify the gap and explain why templates help.
3. **(Hard)** Build a zero-shot image retrieval index: embed 1,000 images with CLIP, build a FAISS index, query with a natural language description. Report retrieval recall@5 for 20 held-out queries you write by hand.

## Key Terms

Term	What people say	What it actually means
Two-tower	“Dual encoder”	Separate image and text encoders ending in a shared-dim projection head
Zero-shot	“No task-specific training”	Classify into classes described only by text at inference; no labels touched
Temperature / logit_scale	“tau”	Learned scalar that scales the similarity matrix before softmax
Prompt template	“A photo of a { }”	Natural-language wrapper around class names; averaging many templates boosts zero-shot accuracy
CLIP	“Image+text model”	The 2021 OpenAI model; vocabulary of the field in 2026
SigLIP	“Sigmoid CLIP”	Swaps softmax for per-pair sigmoid; trains better at small batches
OpenCLIP	“Open reproduction”	Community-trained CLIP variants on LAION; production default for open-source pipelines
VLM	“Vision-language model”	A CLIP-family encoder plus an LLM, trained to answer questions about images

## Further Reading

- CLIP: Learning Transferable Visual Models from Natural Language Supervision (Radford et al., 2021)
- SigLIP: Sigmoid Loss for Language-Image Pre-Training (Zhai et al., 2023)
- OpenCLIP — the community codebase
- DINOv2 vs CLIP vs MAE: a features comparison — HF guide with side-by-side use cases

**Continue online.** The living edition of this chapter has more than the page can hold:

- Animated, interactive figures and the web text: <https://aiengineeringfromscratch.com/lesson?path=phases/04-computer-vision/18-open-vocab-clip>
- Runnable code for every step: <https://github.com/rohitg00/ai-engineering-from-scratch/tree/main/phases/04-computer-vision/18-open-vocab-clip/code>
- The chapter quiz, graded in the browser: <https://aiengineeringfromscratch.com/lesson?path=phases/04-computer-vision/18-open-vocab-clip>

The repository moves faster than any printing. When the book and the repo disagree, trust the repo.

# OCR & Document Understanding

---

OCR is a three-stage pipeline — detect text boxes, recognise the characters, then lay them out. Every modern OCR system reorders these stages or merges them.

**Type:** Learn + Use **Languages:** Python **Prerequisites:** Phase 4 Lesson 06 (Detection), Phase 7 Lesson 02 (Self-Attention) **Time:** ~45 minutes

## Learning Objectives

- Trace the classical OCR pipeline (detect -> recognise -> layout) and the modern end-to-end alternatives (Donut, Qwen-VL-OCR)
- Implement CTC (Connectionist Temporal Classification) loss for sequence-to-sequence OCR training
- Use PaddleOCR or EasyOCR for production document parsing without training
- Distinguish OCR, layout parsing, and document understanding — and pick the right tool per task

## The Problem

Images full of text are everywhere: receipts, invoices, IDs, scanned books, forms, whiteboards, signs, screenshots. Extracting structured data from them — not just the characters, but “this is the total amount” — is one of the highest-value applied-vision problems.

The field splits into three skill layers:

1. **OCR proper:** turn pixels into text.
2. **Layout parsing:** group OCR output into regions (title, body, table, header).
3. **Document understanding:** extract structured fields (“invoice_total = \$42.50”) from layout.

Each layer has classical and modern approaches, and the gap between “I want text from an image” and “I need the total amount from this receipt” is bigger than most teams realise.

## The Concept

### The classical pipeline

**Diagram.** Rendered live in the web edition: <https://aiengineeringfromscratch.com/lesson?path=phases/04-computer-vision/19-ocr-document-understanding>

- **Text detection** produces per-line or per-word quadrilaterals.
- **Recognition** crops each region to a fixed height, runs a CNN + BiLSTM + CTC to produce a character sequence.
- **Layout** rebuilds reading order (top-to-bottom, left-to-right for Latin; different for Arabic, Japanese).

## CTC in one paragraph

OCR recognition produces a variable-length sequence from a fixed-length feature map. CTC (Graves et al., 2006) lets you train this without character-level alignment. The model outputs a distribution over (vocab + blank) at every time step; CTC loss marginalises over all alignments that reduce to the target text after merging repeats and removing blanks.

raw output: "h h h _ _ e e l l _ l l o _ _"  
after merge repeats and remove blanks: "hello"

CTC is the reason CRNN worked in 2015 and still trains most production OCR models in 2026.

## Modern end-to-end models

- **Donut** (Kim et al., 2022) — a ViT encoder + a text decoder; reads an image and emits JSON directly. No text detector, no layout module.
- **TrOCR** — ViT + transformer decoder for line-level OCR.
- **Qwen-VL-OCR / InternVL** — full vision-language models fine-tuned for OCR tasks; best accuracy in 2026 on complex documents.
- **PaddleOCR** — classical DB + CRNN pipeline in a mature production package; still the open-source workhorse.

End-to-end models need more data and compute but skip the error accumulation of multi-stage pipelines.

## Layout parsing

For structured documents, run a layout detector (LayoutLMv3, DocLayNet) that labels each region: Title, Paragraph, Figure, Table, Footnote. Reading order then becomes “iterate through regions in layout order, concatenate.”

For forms, use **Key-Value extraction** models (Donut for visually-rich documents, LayoutLMv3 for plain scans). They take image + detected text + positions and predict structured key-value pairs.

## Evaluation metrics

- **Character Error Rate (CER)** — Levenshtein distance / length of reference. Lower is better. Production target: < 2% on clean scans.
- **Word Error Rate (WER)** — same at the word level.
- **F1 on structured fields** — for key-value tasks; measures whether {invoice_total: 42.50} appears correctly.
- **Edit distance on JSON** — for end-to-end document parsing; the Donut paper introduced normalised tree edit distance.

**Interactive figure: cv3-ctc-collapse.** This one moves. Watch it animate and drag its controls in the web edition: <https://aiengineeringfromscratch.com/lesson?path=phases/04-computer-vision/19-ocr-document-understanding>

## Build It

### Step 1: CTC loss + greedy decoder

```
import torch
import torch.nn as nn
```

```

import torch.nn.functional as F

def ctc_loss(log_probs, targets, input_lengths, target_lengths, blank=0):
    """
    log_probs:      (T, N, C) log-softmax over vocab including blank at index 0
    targets:        (N, S) int targets (no blanks)
    input_lengths:  (N,) per-sample time steps used
    target_lengths: (N,) per-sample target length
    """
    return F.ctc_loss(log_probs, targets, input_lengths, target_lengths,
                      blank=blank, reduction="mean", zero_infinity=True)

def greedy_ctc_decode(log_probs, blank=0):
    """
    log_probs: (T, N, C) log-softmax
    returns: list of index sequences (blanks removed, repeats merged)
    """
    preds = log_probs.argmax(dim=-1).transpose(0, 1).cpu().tolist()
    out = []
    for seq in preds:
        decoded = []
        prev = None
        for idx in seq:
            if idx != prev and idx != blank:
                decoded.append(idx)
            prev = idx
        out.append(decoded)
    return out

```

F.ctc_loss uses the efficient CuDNN implementation when available. The greedy decoder is simpler than a beam search and usually within 1% CER of it.

## Step 2: Tiny CRNN recogniser

Minimal CNN + BiLSTM for line OCR.

```

class TinyCRNN(nn.Module):
    def __init__(self, vocab_size=40, hidden=128, feat=32):
        super().__init__()
        self.cnn = nn.Sequential(
            nn.Conv2d(1, feat, 3, 1, 1), nn.BatchNorm2d(feat),
            nn.ReLU(inplace=True),
            nn.MaxPool2d(2),
            nn.Conv2d(feat, feat * 2, 3, 1, 1), nn.BatchNorm2d(feat * 2),
            nn.ReLU(inplace=True),
            nn.MaxPool2d(2),
            nn.Conv2d(feat * 2, feat * 4, 3, 1, 1), nn.BatchNorm2d(feat * 4),
            nn.ReLU(inplace=True),
            nn.MaxPool2d((2, 1)),
            nn.Conv2d(feat * 4, feat * 4, 3, 1, 1), nn.BatchNorm2d(feat * 4),
            nn.ReLU(inplace=True),
            nn.MaxPool2d((2, 1)),
        )

```

```

self.rnn = nn.LSTM(feats * 4, hidden, bidirectional=True, batch_first=True)
self.head = nn.Linear(hidden * 2, vocab_size)

def forward(self, x):
    # x: (N, 1, H, W)
    f = self.cnn(x) # (N, C, H', W')
    f = f.mean(dim=2).transpose(1, 2) # (N, W', C)
    h, _ = self.rnn(f)
    return F.log_softmax(self.head(h).transpose(0, 1), dim=-1) # (W', N, vocab)

```

Fixed-height input (the CNN max-pools height to 1). Width is the time dimension for CTC.

### Step 3: Synthetic OCR

Generate black-on-white digit strings for an end-to-end smoke test.

```

import numpy as np

def synthetic_line(text, height=32, char_width=16):
    W = char_width * len(text)
    img = np.ones((height, W), dtype=np.float32)
    for i, c in enumerate(text):
        x = i * char_width
        shade = 0.0 if c.isalnum() else 0.5
        img[6:height - 6, x + 2:x + char_width - 2] = shade
    return img

def build_batch(strings, vocab):
    H = 32
    W = 16 * max(len(s) for s in strings)
    imgs = np.ones((len(strings), 1, H, W), dtype=np.float32)
    target_lengths = []
    targets = []
    for i, s in enumerate(strings):
        imgs[i, 0, :, :16 * len(s)] = synthetic_line(s)
        ids = [vocab.index(c) for c in s]
        targets.extend(ids)
        target_lengths.append(len(ids))
    return torch.from_numpy(imgs), torch.tensor(targets),
    ↪ torch.tensor(target_lengths)

```

```

vocab = ["_"] + list("0123456789abcdefghijklmnopqrstuvwxyz")
imgs, targets, lengths = build_batch(["hello", "world"], vocab)
print(f"images: {imgs.shape}    targets: {targets.shape}    lengths:
    ↪ {lengths.tolist()}")

```

A real OCR dataset adds fonts, noise, rotation, blur, and colour. The pipeline above is identical.

### Step 4: Training sketch

```

model = TinyCRNN(vocab_size=len(vocab))
opt = torch.optim.Adam(model.parameters(), lr=1e-3)

```

```

for step in range(200):
    strings = ["abc" + str(step % 10)] * 4 + ["xyz" + str((step + 1) % 10)] * 4
    imgs, targets, target_lens = build_batch(strings, vocab)
    log_probs = model(imgs) # (W', 8, vocab)
    input_lens = torch.full((8,), log_probs.size(0), dtype=torch.long)
    loss = ctc_loss(log_probs, targets, input_lens, target_lens, blank=0)
    opt.zero_grad(); loss.backward(); opt.step()

```

Loss should drop from ~3 to ~0.2 over 200 steps on this trivial synthetic data.

## Use It

Three production paths:

- **PaddleOCR** — mature, fast, multilingual. One-line usage: `paddleocr.PaddleOCR(lang="en").ocr(image_path)`.
- **EasyOCR** — Python-native, multilingual, PyTorch backbone.
- **Tesseract** — classical; still useful for old scanned documents when models struggle.

For end-to-end document parsing, use Donut or a VLM:

```

from transformers import DonutProcessor, VisionEncoderDecoderModel

processor =
    ↪ DonutProcessor.from_pretrained("naver-clova-ix/donut-base-finetuned-cord-v2")
model = VisionEncoderDecoderModel.from_pretrained("naver-clova-ix/donut-base-fine
    ↪ tuned-cord-v2")

```

For receipts, invoices, and forms with repeatable structure, fine-tune Donut. For arbitrary documents or OCR with reasoning, a VLM like Qwen-VL-OCR is the current default.

**This chapter ships an artifact.** The course version of this lesson produces a reusable prompt or agent skill. It lives in the repository, ready to install: <https://github.com/rohitg00/ai-engineering-from-scratch/tree/main/phases/04-computer-vision/19-ocr-document-understanding>

## Exercises

Starter code and the lesson's working implementation: <https://github.com/rohitg00/ai-engineering-from-scratch/tree/main/phases/04-computer-vision/19-ocr-document-understanding/code>

1. **(Easy)** Train the TinyCRNN on 5-digit random numeric strings for 500 steps. Report CER on a held-out set.
2. **(Medium)** Replace greedy decoding with beam search (`beam_width=5`). Report CER delta. On which inputs does beam search win?
3. **(Hard)** Use PaddleOCR on a set of 20 receipts, extract line items, and compute F1 against hand-labelled ground truth for {item_name, price} pairs.

## Key Terms



Term	What people say	What it actually means
OCR	“Text from pixels”	Turning image regions into character sequences
CTC	“Alignment-free loss”	Loss that trains a sequence model without per-timestep labels; marginalises over alignments
CRNN	“Classic OCR model”	Conv feature extractor + BiLSTM + CTC; the 2015 baseline still used in production
Donut	“End-to-end OCR”	ViT encoder + text decoder; emits JSON directly from image
Layout parsing	“Find regions”	Detect and label Title/Table/Figure/Paragraph regions in a document
Reading order	“Text sequence”	Ordering of recognised regions into a sentence; trivial for Latin, non-trivial for mixed layouts
CER / WER	“Error rates”	Levenshtein distance / reference length at character or word granularity
VLM-OCR	“LLM that reads”	A vision-language model trained or prompted for OCR tasks; current SOTA on complex documents

## Further Reading

- CRNN (Shi et al., 2015) — the original CNN+RNN+CTC architecture
- CTC (Graves et al., 2006) — the original CTC paper; densely packed with the algorithmic ideas
- Donut (Kim et al., 2022) — OCR-free document understanding transformer
- PaddleOCR — the open-source production OCR stack

**Continue online.** The living edition of this chapter has more than the page can hold:

- Animated, interactive figures and the web text: <https://aiengineeringfromscratch.com/lesson?path=phases/04-computer-vision/19-ocr-document-understanding>
- Runnable code for every step: <https://github.com/rohitg00/ai-engineering-from-scratch/tree/main/phases/04-computer-vision/19-ocr-document-understanding/code>
- The chapter quiz, graded in the browser: <https://aiengineeringfromscratch.com/lesson?path=phases/04-computer-vision/19-ocr-document-understanding>

The repository moves faster than any printing. When the book and the repo disagree, trust the repo.

# Image Retrieval & Metric Learning

---

A retrieval system ranks candidates by a distance in embedding space. Metric learning is the discipline of shaping that space so the distances mean what you want.

**Type:** Build **Languages:** Python **Prerequisites:** Phase 4 Lesson 14 (ViT), Phase 4 Lesson 18 (CLIP) **Time:** ~45 minutes

## Learning Objectives

- Explain triplet, contrastive, and proxy-based metric learning losses and pick the right one for a given dataset
- Implement L2-normalisation and cosine similarity correctly and audit the difference between “same item” and “same class” retrieval
- Build a FAISS index, query it by text and by image, and report recall@K for a held-out query set
- Use DINOv2, CLIP, and SigLIP as off-the-shelf embedding backbones and know when each wins

## The Problem

Retrieval is everywhere in production vision: duplicate detection, reverse image search, visual search (“find similar products”), face re-identification, person re-ID for surveillance, instance-level matching for e-commerce. The product question is always the same: “given this query image, rank my catalogue.”

Two design decisions shape the whole system. The embedding — what model produces the vectors. The index — how to find nearest neighbours at scale. Both are commodity in 2026 (DINOv2 for the embedding, FAISS for the index), which raises the bar: the hard part is defining *what counts as similar* for your application, then shaping the embedding space so the distances match.

That shaping is metric learning. It is a small but high-leverage discipline.

## The Concept

### Retrieval at a glance

**Diagram.** Rendered live in the web edition: <https://aiengineeringfromscratch.com/lesson?path=phases/04-computer-vision/20-image-retrieval-metric>

### The four loss families

Loss	Requires	Pros	Cons
<b>Contrastive</b>	(anchor, positive) + negatives	Simple, works with any pair label	Slow to converge without many negatives
<b>Triplet</b>	(anchor, positive, negative)	Intuitive; direct margin control	Hard-triplet mining is expensive
<b>NT-Xent / InfoNCE</b>	Pairs + batch-mined negatives	Scales to large batches	Needs big batch or momentum queue
<b>Proxy-based (ProxyNCA)</b>	Class labels only	Fast, stable, no mining	Can overfit to proxies on small datasets

For most production use cases, start with a pretrained backbone and only add a metric-learning fine-tune if the off-the-shelf embeddings underperform on your test set.

## Triplet loss formally

$$L = \max(0, ||f(a) - f(p)||^2 - ||f(a) - f(n)||^2 + \text{margin})$$

Pull anchor  $a$  close to positive  $p$ , push it away from negative  $n$ , with a margin that ensures a gap. The three-image structure generalises to any similarity ordering.

Mining matters: easy triplets ( $n$  already far from  $a$ ) contribute zero loss; only hard triplets teach the network. Semi-hard mining ( $n$  further than  $p$  but within margin) is the 2016 FaceNet recipe and still dominates.

## Cosine similarity vs L2

Two metrics, two conventions:

- **Cosine**: angle between vectors. Requires L2-normalised embeddings.
- **L2**: Euclidean distance. Works on raw or normalised embeddings, but is usually paired with L2-normalised + squared L2.

For most modern nets the two are equivalent:  $||a - b||^2 = 2 - 2 \cos(a, b)$  when  $||a|| = ||b|| = 1$ . Pick the convention that matches your embedding training; mixing them silently changes what “nearest” means.

## Recall@K

The standard retrieval metric:

$\text{recall@K} = \text{fraction of queries where at least one correct match is in the top K results}$

Report  $\text{recall@1}$ ,  $\text{@5}$ ,  $\text{@10}$  side by side. A  $\text{recall@10}$  above 0.95 with  $\text{recall@1}$  below 0.5 means the embedding space has the right structure but the ranking is noisy — try longer fine-tunes or a re-ranking step.

For duplicate detection,  $\text{precision@K}$  matters more because every false positive is a user-visible mistake. For visual search,  $\text{recall@K}$  is the product signal.

## FAISS in one paragraph

Facebook AI Similarity Search. The de-facto library for nearest-neighbour search. Three index choices:

- IndexFlatIP / IndexFlatL2 — brute force, exact, no training. Use up to ~1M vectors.
- IndexIVFFlat — partition into K cells, search only the closest few cells. Approximate, fast, needs training data.
- IndexHNSW — graph-based, fastest for many queries, large index size.

For 100k vectors you probably want IndexFlatIP on cosine similarity. For 10M you want IndexIVFFlat. For 100M+ combined with product quantisation (IndexIVFPQ).

## Instance-level vs category-level retrieval

Two very different problems with the same name:

- **Category-level** — “find cats in my catalogue.” Class-conditional similarity; off-the-shelf CLIP / DINOv2 embeddings work well.
- **Instance-level** — “find *this exact product* in my catalogue.” Needs fine-grained discrimination between visually similar objects of the same class; off-the-shelf embeddings under-perform; fine-tuning with metric learning matters.

Always ask which one you are solving before picking a model.

**Interactive figure: metric-embedding.** This one moves. Watch it animate and drag its controls in the web edition: <https://aiengineeringfromscratch.com/lesson?path=phases/04-computer-vision/20-image-retrieval-metric>

## Build It

### Step 1: Triplet loss

```
import torch
import torch.nn.functional as F

def triplet_loss(anchor, positive, negative, margin=0.2):
    d_ap = F.pairwise_distance(anchor, positive, p=2)
    d_an = F.pairwise_distance(anchor, negative, p=2)
    return F.relu(d_ap - d_an + margin).mean()
```

One line. Works on L2-normalised or raw embeddings.

### Step 2: Semi-hard mining

Given a batch of embeddings and labels, find the hardest semi-hard negative for each anchor.

```
def semi_hard_negatives(emb, labels, margin=0.2):
    dist = torch.cdist(emb, emb)
    same_class = labels[:, None] == labels[None, :]
    diff_class = ~same_class
    N = emb.size(0)

    positives = dist.clone()
    positives[~same_class] = float("-inf")
    positives.fill_diagonal_(float("-inf"))
```

```

pos_idx = positives.argmax(dim=1)

semi_hard = dist.clone()
semi_hard[same_class] = float("inf")
d_ap = dist[torch.arange(N), pos_idx].unsqueeze(1)
semi_hard[dist <= d_ap] = float("inf")
neg_idx = semi_hard.argmin(dim=1)

fallback_mask = semi_hard[torch.arange(N), neg_idx] == float("inf")
if fallback_mask.any():
    hardest = dist.clone()
    hardest[same_class] = float("inf")
    neg_idx = torch.where(fallback_mask, hardest.argmin(dim=1), neg_idx)
return pos_idx, neg_idx

```

Each anchor gets the hardest positive in-class and a semi-hard negative that is further than the positive but within margin.

### Step 3: Recall@K

```

def recall_at_k(query_emb, gallery_emb, query_labels, gallery_labels, k=1):
    sim = query_emb @ gallery_emb.T
    _, top_k = sim.topk(k, dim=-1)
    matches = (gallery_labels[top_k] == query_labels[:, None]).any(dim=-1)
    return matches.float().mean().item()

```

Top-k by inner product on L2-normalised embeddings equals top-k by cosine. Report the mean proportion of queries with at least one correct neighbour.

### Step 4: Putting it together

```

import torch
import torch.nn as nn
from torch.optim import Adam

class Encoder(nn.Module):
    def __init__(self, in_dim=128, emb_dim=64):
        super().__init__()
        self.net = nn.Sequential(
            nn.Linear(in_dim, 128), nn.ReLU(),
            nn.Linear(128, emb_dim),
        )

    def forward(self, x):
        return F.normalize(self.net(x), dim=-1)

torch.manual_seed(0)
num_classes = 6
protos = F.normalize(torch.randn(num_classes, 128), dim=-1)

def sample_batch(bs=32):
    labels = torch.randint(0, num_classes, (bs,))
    x = protos[labels] + 0.15 * torch.randn(bs, 128)
    return x, labels

```

```

enc = Encoder()
opt = Adam(enc.parameters(), lr=3e-3)

for step in range(200):
    x, y = sample_batch(32)
    emb = enc(x)
    pos_idx, neg_idx = semi_hard_negatives(emb, y)
    loss = triplet_loss(emb, emb[pos_idx], emb[neg_idx])
    opt.zero_grad(); loss.backward(); opt.step()

```

After a few hundred steps the embedding clusters form one cluster per class.

## Use It

Production stacks in 2026:

- **DINOv2 + FAISS** — general-purpose visual retrieval. Works off-the-shelf.
- **CLIP + FAISS** — when queries are text.
- **Fine-tuned DINOv2 + FAISS** — instance-level retrieval, face re-ID, fashion, e-commerce.
- **Milvus / Weaviate / Qdrant** — managed vector DB wrappers around FAISS or HNSW.

For SOTA instance retrieval, the recipe is: DINOv2 backbone, add an embedding head, fine-tune with a triplet or InfoNCE loss on instance-labelled pairs, index in FAISS.

**This chapter ships an artifact.** The course version of this lesson produces a reusable prompt or agent skill. It lives in the repository, ready to install: <https://github.com/rohitg00/ai-engineering-from-scratch/tree/main/phases/04-computer-vision/20-image-retrieval-metric>

## Exercises

Starter code and the lesson’s working implementation: <https://github.com/rohitg00/ai-engineering-from-scratch/tree/main/phases/04-computer-vision/20-image-retrieval-metric/code>

1. **(Easy)** Run the toy example above. Plot the embeddings with PCA before and after training to see the six clusters form.
2. **(Medium)** Add a ProxyNCA loss implementation: one learned “proxy” per class, standard cross-entropy on cosine similarity. Compare convergence speed vs triplet loss on the toy data.
3. **(Hard)** Take 1,000 ImageNet validation images, embed with DINOv2 via HuggingFace, build a FAISS flat index, and report recall@{1, 5, 10} against the same images as queries (should be 1.0) and against a held-out split with ImageNet labels as ground truth.

## Key Terms

Term	What people say	What it actually means
Metric learning	“Shape the space”	Training an encoder so distances in its output space reflect a target similarity
Triplet loss	“Pull and push”	$L = \max(0, d(a, p) - d(a, n) + \text{margin})$ ; the canonical metric-learning loss

Term	What people say	What it actually means
Semi-hard mining	“Useful negatives”	Negatives further from the anchor than the positive but within margin; empirically the most informative
Proxy-based loss	“Class prototypes”	One learned proxy per class; cross-entropy over similarity-to-proxies; no pair mining
Recall@K	“Top-K hit rate”	Fraction of queries with at least one correct result in the top K
Instance retrieval	“Find this exact thing”	Fine-grained matching; off-the-shelf features usually underperform
FAISS	“The NN library”	Facebook’s nearest-neighbour library; supports exact and approximate indexes
HNSW	“Graph index”	Hierarchical navigable small world; fast approximate NN with small memory overhead

## Further Reading

- FaceNet: A Unified Embedding for Face Recognition (Schroff et al., 2015) — the triplet loss / semi-hard mining paper
- In Defense of the Triplet Loss for Person Re-Identification (Hermans et al., 2017) — practical guide to triplet fine-tuning
- FAISS documentation — every index, every trade-off
- SMoT: Metric Learning Taxonomy (Kim et al., 2021) — survey of modern losses and their connections

**Continue online.** The living edition of this chapter has more than the page can hold:

- Animated, interactive figures and the web text: <https://aiengineeringfromscratch.com/lesson?path=phases/04-computer-vision/20-image-retrieval-metric>
- Runnable code for every step: <https://github.com/rohitg00/ai-engineering-from-scratch/tree/main/phases/04-computer-vision/20-image-retrieval-metric/code>
- The chapter quiz, graded in the browser: <https://aiengineeringfromscratch.com/lesson?path=phases/04-computer-vision/20-image-retrieval-metric>

The repository moves faster than any printing. When the book and the repo disagree, trust the repo.

# Keypoint Detection & Pose Estimation

---

A pose is a set of ordered keypoints. A keypoint detector is a heatmap regressor. Everything else is bookkeeping.

**Type:** Build **Languages:** Python **Prerequisites:** Phase 4 Lesson 06 (Detection), Phase 4 Lesson 07 (U-Net) **Time:** ~45 minutes

## Learning Objectives

- Distinguish top-down and bottom-up pose estimation and state when each is used
- Regress heatmaps for K keypoints with a Gaussian-per-keypoint target and extract keypoint coordinates at inference
- Explain Part Affinity Fields (PAFs) and how bottom-up pipelines associate keypoints into instances
- Use MediaPipe Pose or MMPose for production keypoint estimation and understand their output format

## The Problem

Keypoint tasks hide under many names: human pose (17 body joints), face landmarks (68 or 478 points), hand (21 points), animal pose, robotic object pose, medical anatomy landmarks. Every one of them shares the same structure: detect K discrete points on an object and output their (x, y) coordinates.

Pose estimation is the foundation of motion capture, fitness apps, sports analytics, gesture control, animation, AR try-on, and robotic grasping. The 2D case is mature; 3D pose (estimating joint positions in world coordinates from a single camera) is the current research frontier.

The engineering question is scale. A single-image, single-person pose is a 20ms problem. Multi-person pose in a crowd at 30 fps is a different problem with different architectures.

## The Concept

### Top-down vs bottom-up

**Diagram.** Rendered live in the web edition: <https://aiengineeringfromscratch.com/lesson?path=phases/04-computer-vision/21-keypoint-pose>

- **Top-down** — detect people first, then run a per-person keypoint model on each crop. Highest accuracy; scales linearly with number of people.
- **Bottom-up** — one forward pass predicts all keypoints plus an association field; group them. Constant time regardless of crowd size.

Top-down (HRNet, ViTPose) is the accuracy leader; bottom-up (OpenPose, HigherHRNet) is the throughput leader for crowded scenes.



## Heatmap regression

Instead of regressing  $(x, y)$  directly, predict an  $H \times W$  heatmap per keypoint with a Gaussian blob centred at the true location.

```
target[k, y, x] = exp(-((x - cx_k)^2 + (y - cy_k)^2) / (2 sigma^2))
```

At inference, the argmax of each heatmap is the predicted keypoint location.

Why heatmaps work better than direct regression: the network's spatial structure (conv feature map) aligns naturally with spatial output. Gaussian targets also regularise — a small localisation error produces a small loss, not zero.

## Sub-pixel localisation

Argmax gives integer coordinates. For sub-pixel precision, refine by fitting a parabola to the argmax and its neighbours, or use the well-known offset  $(dx, dy) = 0.25 * (\text{heatmap}[y, x+1] - \text{heatmap}[y, x-1], \dots)$  direction.

## Part Affinity Fields (PAFs)

OpenPose's trick for bottom-up association. For each pair of connected keypoints (e.g. left shoulder to left elbow), predict a 2-channel field that encodes the unit vector pointing from one to the other. To associate a shoulder with its elbow, integrate the PAF along the line connecting candidate pairs; the pair with the highest integral is matched.

For each connection (limb):

PAF channels: 2 (unit vector  $x, y$ )

Line integral: sum over sample points of  $(\text{PAF} \cdot \text{line_direction})$

Higher integral = stronger match

Elegant and scales to arbitrary crowd sizes without per-person crops.

## COCO keypoints

The standard body-pose dataset: 17 keypoints per person, PCK (Percentage of Correct Keypoints) and OKS (Object Keypoint Similarity) as metrics. OKS is the keypoint analogue of IoU and is what COCO mAP@OKS reports.

## 2D vs 3D

- **2D pose** — image coordinates; solved at production quality (MediaPipe, HRNet, ViTPose).
- **3D pose** — world / camera coordinates; still active research. Common approaches:
  - Lift 2D predictions to 3D with a small MLP (VideoPose3D).
  - Direct 3D regression from image (PyMAF, MHFormer).
  - Multi-view setups (CMU Panoptic) for ground truth.

**Interactive figure: cv3-pose-heatmap.** This one moves. Watch it animate and drag its controls in the web edition: <https://aiengineeringfromscratch.com/lesson?path=phases/04-computer-vision/21-keypoint-pose>

## Build It

### Step 1: Gaussian heatmap target

```
import numpy as np
import torch

def gaussian_heatmap(size, cx, cy, sigma=2.0):
    yy, xx = np.meshgrid(np.arange(size), np.arange(size), indexing="ij")
    return np.exp(-((xx - cx) ** 2 + (yy - cy) ** 2) / (2 * sigma **
        ↪ 2)).astype(np.float32)

hm = gaussian_heatmap(64, 32, 32, sigma=2.0)
print(f"peak: {hm.max():.3f} at ({hm.argmax() % 64}, {hm.argmax() // 64})")
```

Per-keypoint heatmaps stacked along a channel axis give the full target tensor.

### Step 2: Tiny keypoint head

A U-Net-style model that outputs K heatmap channels.

```
import torch.nn as nn
import torch.nn.functional as F

class TinyKeypointNet(nn.Module):
    def __init__(self, num_keypoints=4, base=16):
        super().__init__()
        self.down1 = nn.Sequential(nn.Conv2d(3, base, 3, 2, 1),
            ↪ nn.ReLU(inplace=True))
        self.down2 = nn.Sequential(nn.Conv2d(base, base * 2, 3, 2, 1),
            ↪ nn.ReLU(inplace=True))
        self.mid = nn.Sequential(nn.Conv2d(base * 2, base * 2, 3, 1, 1),
            ↪ nn.ReLU(inplace=True))
        self.up1 = nn.ConvTranspose2d(base * 2, base, 2, 2)
        self.up2 = nn.ConvTranspose2d(base, num_keypoints, 2, 2)

    def forward(self, x):
        h1 = self.down1(x)
        h2 = self.down2(h1)
        h3 = self.mid(h2)
        u1 = self.up1(h3)
        return self.up2(u1)
```

Input (N, 3, H, W), output (N, K, H, W). Loss is per-pixel MSE against Gaussian targets.

### Step 3: Inference — extract keypoint coordinates

```
def heatmap_to_coords(heatmaps):
    """
    heatmaps: (N, K, H, W)
    returns: (N, K, 2) float coordinates in image pixels
    """
    N, K, H, W = heatmaps.shape
    hm = heatmaps.reshape(N, K, -1)
    idx = hm.argmax(dim=-1)
```

```

ys = (idx // W).float()
xs = (idx % W).float()
return torch.stack([xs, ys], dim=-1)

coords = heatmap_to_coords(torch.randn(2, 4, 32, 32))
print(f"coords: {coords.shape}") # (2, 4, 2)

```

One line at inference. For sub-pixel refinement, interpolate around the argmax.

## Step 4: Synthetic keypoint dataset

Simple: draw four points on a white canvas and learn to predict them.

```

def make_synthetic_sample(size=64):
    img = np.ones((3, size, size), dtype=np.float32)
    rng = np.random.default_rng()
    kps = rng.integers(8, size - 8, size=(4, 2))
    for cx, cy in kps:
        img[:, cy - 2:cy + 2, cx - 2:cx + 2] = 0.0
    hms = np.stack([gaussian_heatmap(size, cx, cy) for cx, cy in kps])
    return img, hms, kps

```

Easy enough for a tiny model to learn in a minute.

## Step 5: Training

```

model = TinyKeypointNet(num_keypoints=4)
opt = torch.optim.Adam(model.parameters(), lr=3e-3)

for step in range(200):
    batch = [make_synthetic_sample() for _ in range(16)]
    imgs = torch.from_numpy(np.stack([b[0] for b in batch]))
    hms = torch.from_numpy(np.stack([b[1] for b in batch]))
    pred = model(imgs)
    # Upsample pred to full resolution
    pred = F.interpolate(pred, size=hms.shape[-2:], mode="bilinear",
        ↪ align_corners=False)
    loss = F.mse_loss(pred, hms)
    opt.zero_grad(); loss.backward(); opt.step()

```

## Use It

- **MediaPipe Pose** — Google's production pose estimator; ships WebGL + mobile runtimes with sub-10ms latency.
- **MMPose** (OpenMMLab) — comprehensive research codebase; every SOTA architecture with pretrained weights.
- **YOLOv8-pose** — fastest real-time multi-person pose with a single forward pass.
- **transformers HumanDPT / PoseAnything** — newer vision-language approaches for open-vocabulary pose (any object, any keypoint set).

**This chapter ships an artifact.** The course version of this lesson produces a reusable prompt or agent skill. It lives in the repository, ready to install: <https://github.com/rohitg00/ai-engineering-from-scratch/tree/main/phases/04-computer-vision/21-keypoint-pose>

## Exercises

Starter code and the lesson's working implementation: <https://github.com/rohitg00/ai-engineering-from-scratch/tree/main/phases/04-computer-vision/21-keypoint-pose/code>

1. **(Easy)** Train the tiny keypoint model on the synthetic 4-point dataset. Report mean L2 error between predicted and true keypoints after 200 steps.
2. **(Medium)** Add sub-pixel refinement: given the argmax position, fit a 1D parabola along x and y from the neighbouring pixels. Report the accuracy gain vs integer argmax.
3. **(Hard)** Build a 2-person synthetic dataset where each image shows two instances of the 4-keypoint pattern. Train a bottom-up pipeline with PAFs that predict which keypoint belongs to which instance, and evaluate OKS.

## Key Terms

Term	What people say	What it actually means
Keypoint	"A landmark"	A specific ordered point on an object (joint, corner, feature)
Pose	"The skeleton"	An ordered set of keypoints belonging to one instance
Top-down	"Detect then pose"	Two-stage pipeline: person detector + per-crop keypoint model; highest accuracy
Bottom-up	"Pose first, group later"	Single-pass all-keypoint prediction + grouping; constant time in crowd size
Heatmap	"Gaussian target"	H x W tensor per keypoint with peak at the true location; the preferred regression target
PAF	"Part Affinity Field"	2-channel unit vector field encoding limb directions; used to group keypoints into instances
OKS	"Keypoint IoU"	Object Keypoint Similarity; the COCO metric for pose
HRNet	"High-Resolution Net"	The dominant top-down keypoint architecture; preserves high-res features throughout

## Further Reading

- OpenPose (Cao et al., 2017) — bottom-up with PAFs; still the best writeup of the approach
- HRNet (Sun et al., 2019) — the top-down reference architecture
- ViTPose (Xu et al., 2022) — plain ViT as a pose backbone; current SOTA on many benchmarks
- MediaPipe Pose — production real-time pose; the fastest deployed stack in 2026

**Continue online.** The living edition of this chapter has more than the page can hold:

- Animated, interactive figures and the web text: <https://aiengineeringfromscratch.com/lesson?path=phases/04-computer-vision/21-keypoint-pose>
- Runnable code for every step: <https://github.com/rohitg00/ai-engineering-from-scratch/tree/main/phases/04-computer-vision/21-keypoint-pose/code>

- The chapter quiz, graded in the browser: <https://aiengineeringfromscratch.com/lesson?path=phases/04-computer-vision/21-keypoint-pose>

The repository moves faster than any printing. When the book and the repo disagree, trust the repo.

# 3D Gaussian Splatting from Scratch

---

A scene is a cloud of millions of 3D Gaussians. Each one has a position, orientation, scale, opacity, and a colour that depends on viewing direction. Rasterise them, backprop through the rasterisation, done.

**Type:** Build **Languages:** Python **Prerequisites:** Phase 4 Lesson 13 (3D Vision & NeRF), Phase 1 Lesson 12 (Tensor Operations), Phase 4 Lesson 10 (Diffusion basics optional) **Time:** ~90 minutes

## Learning Objectives

- Explain why 3D Gaussian Splatting replaced NeRF as the production default for photo-realistic 3D reconstruction in 2026
- State the six per-Gaussian parameters (position, rotation quaternion, scale, opacity, spherical harmonics colour, optional feature) and how many floats each contributes
- Implement a 2D Gaussian splatting rasterizer from scratch using alpha compositing, then show how the 3D case projects to the same loop
- Use nerfstudio, gsplat, or SuperSplat to reconstruct a scene from 20-50 photos and export to the KHR_gaussian_splatting glTF extension or the OpenUSD 26.03 UsdVolParticleField3DGaussianSplat schema

## The Problem

A NeRF stores a scene as the weights of an MLP. Every rendered pixel is hundreds of MLP queries along a ray. Training takes hours, rendering takes seconds, and the weights cannot be edited — if you want to move a chair inside a scene, you have to retrain.

3D Gaussian Splatting (Kerbl, Kopanas, Leimkühler, Drettakis, SIGGRAPH 2023) replaced all of that. A scene is an explicit set of 3D Gaussians. Rendering is GPU rasterisation at 100+ fps. Training takes minutes. Editing is direct: translate a subset of Gaussians and you have moved the chair. By 2026 the Khronos Group has ratified a glTF extension for Gaussian splats, OpenUSD 26.03 ships a Gaussian splat schema, Zillow and Apartments.com render real estate with them, and most new research papers on 3D reconstruction are variants on the core 3DGS idea.

The mental model is simple, the math has enough moving parts that most introductions start at rasterisation and skip past the projections and spherical harmonics. This lesson builds the whole thing — a 2D version first, then the 3D extension.

## The Concept

### What a Gaussian carries

One 3D Gaussian is a parametric blob in space with these attributes:

position	mu	(3,)	centre in world coordinates
rotation	q	(4,)	unit quaternion encoding orientation
scale	s	(3,)	log-scales per axis (exponentiated at render time)
opacity	alpha	(1,)	post-sigmoid opacity [0, 1]
SH coefficients	c_lm	(3 * (L+1)^2,)	view-dependent colour

Rotation + scale build a 3x3 covariance:  $\Sigma = R S S^T R^T$ . That is the shape of the Gaussian in 3D. Spherical harmonics let the colour change with viewing direction — specular highlights, subtle sheen, view-dependent glow — without storing per-view textures. With SH degree 3 you get 16 coefficients per colour channel, 48 floats per Gaussian for colour alone.

A scene typically has 1-5 million Gaussians. Each stores roughly 60 floats (3 + 4 + 3 + 1 + 48 + misc). That is 240 MB for a five-million-Gaussian scene — far smaller than the equivalent point cloud with per-point texture, and an order of magnitude smaller than a NeRF's MLP weights re-rendered at high resolution.

## Rasterisation, not ray marching

**Diagram.** Rendered live in the web edition: <https://aiengineeringfromscratch.com/lesson?path=phases/04-computer-vision/22-3d-gaussian-splatting>

Five steps, all GPU-friendly. No MLP query per pixel. A single RTX 3080 Ti renders 6 million splats at 147 fps.

## The projection step

The 3D Gaussian at world position  $\mu$  with 3D covariance  $\Sigma$  projects to a 2D Gaussian at screen position  $\mu'$  with 2D covariance  $\Sigma'$ :

```
mu' = project(mu)
Sigma' = J W Sigma W^T J^T          (2 x 2)
```

$W$  = viewing transform (rotation + translation of camera)

$J$  = Jacobian of the perspective projection at  $\mu'$

The 2D Gaussian's footprint is an ellipse whose axes are the eigenvectors of  $\Sigma'$ . Every pixel inside that ellipse receives the Gaussian's contribution, weighted by  $\exp(-0.5 * (p - \mu')^T \Sigma'^{-1} (p - \mu'))$ .

## The alpha-compositing rule

For one pixel, the Gaussians that cover it are sorted back-to-front (or equivalently front-to-back with inverted formula). Colour is composited with the same equation as every semi-transparent rasteriser since the 1980s:

$$C_{\text{pixel}} = \sum_i \alpha_i * T_i * c_i$$

```
T_i = prod_{j < i} (1 - alpha_j)      transmittance up to i
alpha_i = opacity_i * exp(-0.5 * d^T Sigma'^-1 d)  local contribution
c_i = eval_SH(SH_i, view_direction)  view-dependent colour
```

This is **the same equation as NeRF's volumetric render**, just over an explicit sparse set of Gaussians instead of dense samples along a ray. That identity is why rendered quality matches NeRF — both are integrating the same radiance-field equation.

## Why this is differentiable

Every step — projection, tile assignment, alpha compositing, SH evaluation — is differentiable with respect to the Gaussian parameters. Given a ground-truth image, compute rendered pixel loss, backprop through the rasteriser, update all ( $\mu$ ,  $q$ ,  $s$ ,  $\alpha$ ,  $c_{lm}$ ) by gradient descent. Over  $\sim 30,000$  iterations the Gaussians find their right positions, scales, and colours.

## Densification and pruning

A fixed set of Gaussians cannot cover a complex scene. Training includes two adaptive mechanisms:

- **Clone** a Gaussian at its current position when its gradient magnitude is high but its scale is small — the reconstruction needs more detail here.
- **Split** a large-scale Gaussian into two smaller ones when its gradient is high — one big Gaussian is too smooth to fit the region.
- **Prune** Gaussians whose opacity drops below a threshold — they are not contributing.

Densification runs every  $N$  iterations. A scene typically grows from  $\sim 100k$  initial Gaussians (seeded from SfM points) to 1-5M at the end of training.

## Spherical harmonics in one paragraph

View-dependent colour is a function  $c(\text{direction})$  on the unit sphere. Spherical harmonics are the sphere's Fourier basis. Truncate at degree  $L$  and you get  $(L+1)^2$  basis functions per channel. Evaluating the colour for a new view is a dot product between the learned SH coefficients and the basis evaluated at the viewing direction. Degree 0 = one coefficient = constant colour. Degree 3 = 16 coefficients = enough to capture Lambertian shading, specular, and mild reflection. SD Gaussian Splatting papers use degree 3 by default.

## The 2026 production stack

- |               |                                                                       |
|---------------|-----------------------------------------------------------------------|
| 1. Capture    | smartphone / DJI drone / handheld scanner                             |
| 2. SfM / MVS  | COLMAP or GLOMAP derives camera poses + sparse points                 |
| 3. Train 3DGS | nerfstudio / gsplat / inria official / PostShot ( $\sim 10-30$ min on |
| ↪ RTX 4090)   |                                                                       |
| 4. Edit       | SuperSplat / SplatForge (clean floaters, segment)                     |
| 5. Export     | .ply -> glTF KHR_gaussian_splatting or .usd (OpenUSD 26.03)           |
| 6. View       | Cesium / Unreal / Babylon.js / Three.js / Vision Pro                  |

## 4D and generative variants

- **4D Gaussian Splatting** — Gaussians are functions of time; used for volumetric video (Superman 2026, A\$AP Rocky's "Helicopter").
- **Generative splats** — text-to-splat models (Marble by World Labs) that hallucinate entire scenes.
- **3D Gaussian Unscented Transform** — NVIDIA NuRec's variant for autonomous driving simulation.

**Interactive figure: cv3-gaussian-splat.** This one moves. Watch it animate and drag its controls in the web edition: <https://aiengineeringfromscratch.com/lesson?path=phases/04-computer-vision/22-3d-gaussian-splatting>



## Build It

### Step 1: A 2D Gaussian

We first build a 2D rasteriser. The 3D case reduces to it after projection.

```
import torch
import torch.nn as nn
import torch.nn.functional as F

def eval_2d_gaussian(means, covs, points):
    """
    means: (G, 2)      centres
    covs:  (G, 2, 2)    covariance matrices
    points: (H, W, 2)   pixel coordinates
    returns: (G, H, W) density at every pixel for every Gaussian
    """
    G = means.size(0)
    H, W, _ = points.shape
    flat = points.view(-1, 2)
    inv = torch.linalg.inv(covs)
    diff = flat[None, :, :] - means[:, None, :]
    d = torch.einsum("gpi,gij,gpj->gp", diff, inv, diff)
    density = torch.exp(-0.5 * d)
    return density.view(G, H, W)
```

einsum does the quadratic form  $\text{diff}^T \Sigma^{-1} \text{diff}$  for every (Gaussian, pixel) pair.

### Step 2: 2D splatting rasteriser

Alpha-compositing front-to-back. Depth in 2D is meaningless, so we use a learned per-Gaussian scalar for order.

```
def rasterise_2d(means, covs, colours, opacities, depths, image_size):
    """
    means:      (G, 2)
    covs:       (G, 2, 2)
    colours:    (G, 3)
    opacities:  (G,)      in [0, 1]
    depths:     (G,)      per-Gaussian scalar used for ordering
    image_size: (H, W)
    returns:    (H, W, 3) rendered image
    """
    H, W = image_size
    yy, xx = torch.meshgrid(
        torch.arange(H, dtype=torch.float32, device=means.device),
        torch.arange(W, dtype=torch.float32, device=means.device),
        indexing="ij",
    )
    points = torch.stack([xx, yy], dim=-1)

    densities = eval_2d_gaussian(means, covs, points)
    alphas = opacities[:, None, None] * densities
    alphas = alphas.clamp(0.0, 0.99)
```

```

order = torch.argsort(depths)
alphas = alphas[order]
colours_sorted = colours[order]

T = torch.ones(H, W, device=means.device)
out = torch.zeros(H, W, 3, device=means.device)
for i in range(means.size(0)):
    a = alphas[i]
    out += (T * a)[..., None] * colours_sorted[i][None, None, :]
    T = T * (1.0 - a)
return out

```

Not fast — a real implementation uses tile-based CUDA kernels — but exactly the right math and fully differentiable.

### Step 3: A trainable 2D splat scene

```

class Splats2D(nn.Module):
    def __init__(self, num_splats=128, image_size=64, seed=0):
        super().__init__()
        g = torch.Generator().manual_seed(seed)
        H, W = image_size, image_size
        self.means = nn.Parameter(torch.rand(num_splats, 2, generator=g) *
            ↪ torch.tensor([W, H]))
        self.log_scale = nn.Parameter(torch.ones(num_splats, 2) * math.log(2.0))
        self.rot = nn.Parameter(torch.zeros(num_splats)) # single angle in 2D
        self.colour_logits = nn.Parameter(torch.randn(num_splats, 3, generator=g)
            ↪ * 0.5)
        self.opacity_logit = nn.Parameter(torch.zeros(num_splats))
        self.depth = nn.Parameter(torch.rand(num_splats, generator=g))

    def covs(self):
        s = torch.exp(self.log_scale)
        c, si = torch.cos(self.rot), torch.sin(self.rot)
        R = torch.stack([
            torch.stack([c, -si], dim=-1),
            torch.stack([si, c], dim=-1),
        ], dim=-2)
        S = torch.diag_embed(s ** 2)
        return R @ S @ R.transpose(-1, -2)

    def forward(self, image_size):
        covs = self.covs()
        colours = torch.sigmoid(self.colour_logits)
        opacities = torch.sigmoid(self.opacity_logit)
        return rasterise_2d(self.means, covs, colours, opacities, self.depth,
            ↪ image_size)

```

`log_scale`, `opacity_logit`, and `colour_logits` are all unconstrained parameters mapped through the right activation at render time. This is the standard pattern for every 3DGS implementation.

## Step 4: Fit 2D Gaussians to a target image

```
import math
import numpy as np

def make_target(size=64):
    yy, xx = np.meshgrid(np.arange(size), np.arange(size), indexing="ij")
    img = np.zeros((size, size, 3), dtype=np.float32)
    # Red circle
    mask = (xx - 20) ** 2 + (yy - 20) ** 2 < 10 ** 2
    img[mask] = [1.0, 0.2, 0.2]
    # Blue square
    mask = (np.abs(xx - 45) < 8) & (np.abs(yy - 40) < 8)
    img[mask] = [0.2, 0.3, 1.0]
    return torch.from_numpy(img)
```

```
target = make_target(64)
model = Splats2D(num_splats=64, image_size=64)
opt = torch.optim.Adam(model.parameters(), lr=0.05)
```

```
for step in range(200):
    pred = model((64, 64))
    loss = F.mse_loss(pred, target)
    opt.zero_grad(); loss.backward(); opt.step()
    if step % 40 == 0:
        print(f"step {step:3d}   mse {loss.item():.4f}")
```

Over 200 steps the 64 Gaussians settle into the two shapes. That is the entire idea — gradient-descent on explicit geometric primitives.

## Step 5: From 2D to 3D

The 3D extension keeps the same loop. The additions:

1. Per-Gaussian rotation is a quaternion instead of a single angle.
2. Covariance is  $R S S^T R^T$  with  $R$  built from the quaternion and  $S = \text{diag}(\exp(\log_scale))$ .
3. Projection  $(\mu, \Sigma) \rightarrow (\mu', \Sigma')$  uses the camera extrinsics and the Jacobian of the perspective projection at  $\mu$ .
4. Colour becomes a spherical-harmonics expansion; evaluate it at the viewing direction.
5. Depth-sort is from actual camera-space  $z$  instead of a learned scalar.

Every production implementation (gsplat, inria/gaussian-splatting, nerfstudio) does exactly this on the GPU with tile-based CUDA kernels.

## Step 6: Spherical harmonics evaluation

The SH basis up to degree 3 has 16 terms per channel. Evaluation:

```
def eval_sh_degree_3(sh_coeffs, dirs):
    """
    sh_coeffs: (... , 16, 3)    last dim is RGB channels
    dirs:      (... , 3)        unit vectors
    returns:   (... , 3)
    """
```

```

C0 = 0.282094791773878
C1 = 0.488602511902920
C2 = [1.092548430592079, 1.092548430592079,
      0.315391565252520, 1.092548430592079,
      0.546274215296039]
x, y, z = dirs[..., 0], dirs[..., 1], dirs[..., 2]
x2, y2, z2 = x * x, y * y, z * z
xy, yz, xz = x * y, y * z, x * z

result = C0 * sh_coeffs[..., 0, :]
result = result - C1 * y[..., None] * sh_coeffs[..., 1, :]
result = result + C1 * z[..., None] * sh_coeffs[..., 2, :]
result = result - C1 * x[..., None] * sh_coeffs[..., 3, :]

result = result + C2[0] * xy[..., None] * sh_coeffs[..., 4, :]
result = result + C2[1] * yz[..., None] * sh_coeffs[..., 5, :]
result = result + C2[2] * (2.0 * z2 - x2 - y2)[..., None] * sh_coeffs[..., 6, :]
result = result + C2[3] * xz[..., None] * sh_coeffs[..., 7, :]
result = result + C2[4] * (x2 - y2)[..., None] * sh_coeffs[..., 8, :]

# degree 3 terms omitted here for brevity; full 16-coefficient version in the
↪ code file
return result

```

Learned `sh_coeffs` store the “colour in every direction” for that Gaussian. At render time you evaluate against the current view direction and get a 3-vector RGB.

## Use It

For real 3DGS work, use `gsplat` (Meta) or `nerfstudio`:

```

pip install nerfstudio gsplat
ns-download-data example
ns-train splatfacto --data path/to/data

```

`splatfacto` is `nerfstudio`’s 3DGS trainer. The run takes 10-30 minutes on an RTX 4090 for a typical scene.

Export options that matter in 2026:

- `.ply` — raw Gaussian cloud (portable, largest file).
- `.splat` — PlayCanvas / SuperSplat quantised format.
- `glTF KHR_gaussian_splatting` — Khronos standard, portable across viewers (Feb 2026 RC).
- `OpenUSD UsdVolParticleField3DGaussianSplat` — USD-native, for NVIDIA Omniverse and Vision Pro pipelines.

For 4D / dynamic scenes, 4DGS and Deformable-3DGS extend the same machinery with time-varying means and opacities.

**This chapter ships an artifact.** The course version of this lesson produces a reusable prompt or agent skill. It lives in the repository, ready to install: <https://github.com/rohitg00/ai-engineering-from-scratch/tree/main/phases/04-computer-vision/22-3d-gaussian-splatting>

## Exercises

Starter code and the lesson’s working implementation: <https://github.com/rohitg00/ai-engineering-from-scratch/tree/main/phases/04-computer-vision/22-3d-gaussian-splatting/code>

1. **(Easy)** Run the 2D splat trainer above on a different synthetic image. Vary `num_splats` in `[16, 64, 256]` and plot MSE vs step for each. Identify the point of diminishing returns.
2. **(Medium)** Extend the 2D rasteriser to support per-Gaussian RGB colours that depend on a scalar “view angle” through a degree-2 harmonic. Train on a pair of target images and verify the model reconstructs both.
3. **(Hard)** Clone `nerfstudio` and train `splatfacto` on a 20-photo capture of any scene you have (desk, plant, face, room). Export to glTF `KHR_gaussian_splatting` and open it in a viewer (Three.js `GaussianSplats3D`, `SuperSplat`, `Babylon.js V9`). Report training time, number of Gaussians, and rendered fps.

## Key Terms

Term	What people say	What it actually means
3DGS	“Gaussian splats”	Explicit scene representation as millions of 3D Gaussians with per-Gaussian position, rotation, scale, opacity, SH colour
Covariance	“Shape of the Gaussian”	$\Sigma = R S S^T R^T$ ; orientation and anisotropic scale of one Gaussian
Alpha compositing	“Back-to-front blend”	Same equation as NeRF’s volumetric render, now over an explicit sparse set
Densification	“Clone and split”	Adaptive addition of new Gaussians where reconstruction is under-fit
Pruning	“Delete low-opacity”	Remove Gaussians that have collapsed to near-zero opacity during training
Spherical harmonics	“View-dependent colour”	Fourier basis on the sphere; stores colour as a function of viewing direction
Splatfacto	“nerfstudio’s 3DGS”	The easiest path to training 3DGS in 2026
KHR_gaussian_splatting	“glTF standard”	Khronos 2026 extension that makes 3DGS portable across viewers and engines

## Further Reading

- 3D Gaussian Splatting for Real-Time Radiance Field Rendering (Kerbl et al., SIGGRAPH 2023) — the original paper
- `gsplat` (Meta/nerfstudio) — production-quality CUDA rasteriser
- `nerfstudio Splatfacto` — reference training recipe
- Khronos `KHR_gaussian_splatting` extension — the 2026 portable format
- OpenUSD 26.03 release notes — `UsdVolParticleField3DGaussianSplat` schema
- THE FUTURE 3D State of Gaussian Splatting 2026 — industry overview

**Continue online.** The living edition of this chapter has more than the page can hold:

- Animated, interactive figures and the web text: <https://aiengineeringfromscratch.com/lesson?path=phases/04-computer-vision/22-3d-gaussian-splatting>

- Runnable code for every step: <https://github.com/rohitg00/ai-engineering-from-scratch/tree/main/phases/04-computer-vision/22-3d-gaussian-splatting/code>
- The chapter quiz, graded in the browser: <https://aiengineeringfromscratch.com/lesson?path=phases/04-computer-vision/22-3d-gaussian-splatting>

The repository moves faster than any printing. When the book and the repo disagree, trust the repo.

# Diffusion Transformers & Rectified Flow

---

The U-Net is not the secret of diffusion. Replace it with a transformer, swap the noise schedule for a straight-line flow, and suddenly you have SD3, FLUX, and every 2026 text-to-image model.

**Type:** Learn + Build **Languages:** Python **Prerequisites:** Phase 4 Lesson 10 (Diffusion DDPM), Phase 4 Lesson 14 (ViT), Phase 7 Lesson 02 (Self-Attention) **Time:** ~75 minutes

## Learning Objectives

- Trace the evolution from U-Net DDPM (Lesson 10) to Diffusion Transformer (DiT), MMDiT (SD3), and single+double-stream DiT (FLUX)
- Explain rectified flow: why a straight-line trajectory between noise and data lets models sample in 20 steps instead of 1000
- Implement a tiny DiT block and a rectified-flow training loop, both under 100 lines
- Distinguish model variants (SD3, FLUX.1-dev, FLUX.1-schnell, Z-Image, Qwen-Image) by architecture, parameter count, and licensing

## The Problem

Lesson 10 built a DDPM with a U-Net denoiser. That recipe dominated 2020-2023: U-Net + beta schedule + noise-prediction loss. It produced Stable Diffusion 1.5 and 2.1 and DALL-E 2.

Every 2026 state-of-the-art text-to-image model has moved past it. Stable Diffusion 3, FLUX, SD4, Z-Image, Qwen-Image, Hunyuan-Image — none use a U-Net. They use Diffusion Transformers (DiT). SD3 and FLUX also swap the DDPM noise schedule for rectified flow, which straightens the path from noise to data and enables 1-4 step inference with consistency or distilled variants.

The shift matters because it is the reason diffusion-based image generation became controllable, prompt-accurate (SD3/SD4 solved text rendering), and production-fast. Understanding DiT + rectified flow is understanding the 2026 generative-image stack.

## The Concept

### From U-Net to transformer

**Diagram.** Rendered live in the web edition: <https://aiengineeringfromscratch.com/lesson?path=phases/04-computer-vision/23-diffusion-transformers-rectified-flow>

- **DiT** (Peebles & Xie, 2023) — replace the U-Net with a ViT-like transformer on latent patches. Conditioning via adaptive layer norm (AdaLN).
- **MMDiT** (SD3, Esser et al., 2024) — two streams with separate weights for text and image tokens that share a joint attention.

- **FLUX** (Black Forest Labs, 2024) — first N blocks double-stream like SD3, later blocks concatenate and share weights (single-stream) for efficiency at higher depth.
- **Z-Image** (2025) — an efficient single-stream DiT at 6B parameters that challenges “scale at all costs”.

## Rectified flow in one paragraph

DDPM defines the forward process as a noisy SDE where  $x_t$  is increasingly corrupted. The learned reverse is a second SDE, solved by 1000 small steps.

Rectified flow defines a **straight-line** interpolation between clean data and pure noise:

$$x_t = (1 - t) * x_0 + t * \text{epsilon}, \quad t \text{ in } [0, 1]$$

Train a network to predict the velocity  $v_{\text{theta}}(x_t, t) = \text{epsilon} - x_0$  — the forward direction along the straight-line path from clean data to noise ( $dx_t/dt$ ). During sampling, you integrate this velocity backward to step from noise toward data. The resulting ODE is much closer to a straight line, so far fewer integration steps are needed to sample.

SD3 calls this **Rectified Flow Matching**. FLUX, Z-Image, and most 2026 models use the same objective. Typical inference: 20-30 Euler steps (deterministic) vs 50+ DDIM steps in the old DDPM regime. Distilled / turbo / schnell / LCM variants take it down to 1-4 steps.

## AdaLN conditioning

DiTs condition on timestep and class/text via **adaptive layer norm**: predict scale and shift from the conditioning vector and apply them after LayerNorm. Much cleaner than FiLM-style modulation in U-Nets and the default in every modern DiT.

```
cond -> MLP -> (scale, shift, gate)
norm(x) * (1 + scale) + shift, then residual add * gate
```

## Text encoders in SD3 and FLUX

- **SD3** uses three text encoders: two CLIP models + T5-XXL. Embeddings concatenated and fed to the image stream as text conditioning.
- **FLUX** uses one CLIP-L + T5-XXL.
- **Qwen-Image / Z-Image** variants use their own in-house text encoders aligned with their base LLMs.

The text encoder is a big part of why SD3/FLUX reason about prompts so much better than SD1.5. T5-XXL alone is 4.7B params.

## Classifier-free guidance still holds

Rectified flow changes the sampler, not the conditioning. Classifier-free guidance (drop text with 10% probability during training, mix conditional and unconditional predictions at inference) works identically with rectified flow. Most 2026 models use guidance scale 3.5-5 — lower than SD1.5’s 7.5 because rectified-flow models follow prompts more tightly by default.

## Consistency, Turbo, Schnell, LCM

Four names for the same idea: distil a slow many-step model into a fast few-step model.

- **LCM (Latent Consistency Model)** — train a student that predicts the final  $x_0$  from any intermediate  $x_t$  in one step.



- **SDXL Turbo / FLUX schnell** — 1-4 step models trained with adversarial diffusion distillation.
- **SD Turbo** — OpenAI-style Consistency Models adapted to latent diffusion.

Production serving of any new model ships both a “full quality” checkpoint and a “turbo / schnell” variant. Schnell (“fast” in German, Black Forest Labs’ convention) runs in 1-4 steps and fits real-time pipelines.

## Model landscape in 2026

Model	Size	Architecture	License
Stable Diffusion 3 Medium	2B	MMDiT	SAI Community
Stable Diffusion 3.5 Large	8B	MMDiT	SAI Community
FLUX.1-dev	12B	Double + Single Stream DiT	non-commercial
FLUX.1-schnell	12B	same, distilled	Apache 2.0
FLUX.2	—	iterated FLUX.1	mixed
Z-Image	6B	S3-DiT (Scalable Single-Stream)	permissive
Qwen-Image	~20B	DiT + Qwen text tower	Apache 2.0
Hunyuan-Image-3.0	~80B	DiT	research
SD4 Turbo	3B	DiT + distillation	SAI Commercial

FLUX.1-schnell is the 2026 open-source default. Z-Image is the efficiency leader. FLUX.2 and SD4 are the current quality tips.

## Why this phase shift matters

DDPM + U-Net worked. DiT + rectified flow works **better, faster, and scales more cleanly**. The transition parallels the one from RNNs to transformers in NLP: both architectures solved the same problem, but transformers scaled and now dominate. Every 2026 paper on image, video, or 3D generation uses a DiT-shaped denoiser and usually a rectified flow objective. U-Net DDPM is now primarily pedagogical (Lesson 10).

**Interactive figure: cv3-rectified-flow.** This one moves. Watch it animate and drag its controls in the web edition: <https://aiengineeringfromscratch.com/lesson?path=phases/04-computer-vision/23-diffusion-transformers-rectified-flow>

## Build It

### Step 1: A DiT block with AdaLN

```
import torch
import torch.nn as nn
```

```
class AdaLNZero(nn.Module):
    """
```

```
    Adaptive LayerNorm with a gate. Predicts (scale, shift, gate) from the
    ↪ conditioning.
```

*Init such that the whole block starts as identity ("zero init").*

```

def __init__(self, dim, cond_dim):
    super().__init__()
    self.norm = nn.LayerNorm(dim, elementwise_affine=False)
    self.mlp = nn.Linear(cond_dim, dim * 3)
    nn.init.zeros_(self.mlp.weight)
    nn.init.zeros_(self.mlp.bias)

def forward(self, x, cond):
    scale, shift, gate = self.mlp(cond).chunk(3, dim=-1)
    h = self.norm(x) * (1 + scale.unsqueeze(1)) + shift.unsqueeze(1)
    return h, gate.unsqueeze(1)

class DiTBlock(nn.Module):
    def __init__(self, dim=192, heads=3, mlp_ratio=4, cond_dim=192):
        super().__init__()
        self.adaln1 = AdaLNZero(dim, cond_dim)
        self.attn = nn.MultiheadAttention(dim, heads, batch_first=True)
        self.adaln2 = AdaLNZero(dim, cond_dim)
        self.mlp = nn.Sequential(
            nn.Linear(dim, dim * mlp_ratio),
            nn.GELU(),
            nn.Linear(dim * mlp_ratio, dim),
        )

    def forward(self, x, cond):
        h, gate1 = self.adaln1(x, cond)
        a, _ = self.attn(h, h, h, need_weights=False)
        x = x + gate1 * a
        h, gate2 = self.adaln2(x, cond)
        x = x + gate2 * self.mlp(h)
        return x

```

AdaLNZero starts as an identity mapping because its MLP weights are initialised to zero. Training nudges the block away from identity; this stabilises deep transformer diffusion models dramatically.

## Step 2: A tiny DiT

```

def timestep_embedding(t, dim):
    import math
    half = dim // 2
    freqs = torch.exp(-math.log(10000) * torch.arange(half, device=t.device) /
        ↪ half)
    args = t[:, None].float() * freqs[None]
    return torch.cat([args.sin(), args.cos()], dim=-1)

class TinyDiT(nn.Module):
    def __init__(self, image_size=16, patch_size=2, in_channels=3, dim=96,
        ↪ depth=4, heads=3):

```

```

    super().__init__()
    self.patch_size = patch_size
    self.num_patches = (image_size // patch_size) ** 2
    self.patch = nn.Conv2d(in_channels, dim, kernel_size=patch_size,
        ↪ stride=patch_size)
    self.pos = nn.Parameter(torch.zeros(1, self.num_patches, dim))
    self.time_mlp = nn.Sequential(
        nn.Linear(dim, dim * 2),
        nn.SiLU(),
        nn.Linear(dim * 2, dim),
    )
    self.blocks = nn.ModuleList([DiTBlock(dim, heads, cond_dim=dim) for _ in
        ↪ range(depth)])
    self.norm_out = nn.LayerNorm(dim, elementwise_affine=False)
    self.head = nn.Linear(dim, patch_size * patch_size * in_channels)

    def forward(self, x, t):
        n = x.size(0)
        x = self.patch(x)
        x = x.flatten(2).transpose(1, 2) + self.pos
        t_emb = self.time_mlp(timestep_embedding(t, self.pos.size(-1)))
        for blk in self.blocks:
            x = blk(x, t_emb)
        x = self.norm_out(x)
        x = self.head(x)
        return self._unpatchify(x, n)

    def _unpatchify(self, x, n):
        p = self.patch_size
        h = w = int(self.num_patches ** 0.5)
        x = x.view(n, h, w, p, p, -1).permute(0, 5, 1, 3, 2, 4).reshape(n, -1, h *
        ↪ p, w * p)
        return x

```

### Step 3: Rectified flow training

```

import torch.nn.functional as F

def rectified_flow_train_step(model, x0, optimizer, device):
    model.train()
    x0 = x0.to(device)
    n = x0.size(0)
    t = torch.rand(n, device=device)
    epsilon = torch.randn_like(x0)
    x_t = (1 - t[:, None, None, None]) * x0 + t[:, None, None, None] * epsilon

    target_velocity = epsilon - x0
    pred_velocity = model(x_t, t)

    loss = F.mse_loss(pred_velocity, target_velocity)
    optimizer.zero_grad()
    loss.backward()
    optimizer.step()
    return loss.item()

```

Compare with DDPM's noise-prediction loss (Lesson 10): same structure, different target. Instead of predicting the noise epsilon, we predict the **velocity** epsilon -  $x_0$ , which points from data to noise along the straight-line interpolation.

## Step 4: Euler sampler

Rectified flow is an ODE. Euler's method is the simplest and, for a well-trained rectified-flow model, nearly as accurate as higher-order solvers at 20+ steps.

```
@torch.no_grad()
def rectified_flow_sample(model, shape, steps=20, device="cpu"):
    model.eval()
    x = torch.randn(shape, device=device)
    dt = 1.0 / steps
    t = torch.ones(shape[0], device=device)
    for _ in range(steps):
        v = model(x, t)
        x = x - dt * v
        t = t - dt
    return x
```

20 steps. On a trained model this produces samples comparable to 1000-step DDPM.

## Step 5: End-to-end smoke test

```
import numpy as np

def synthetic_blobs(num=200, size=16, seed=0):
    rng = np.random.default_rng(seed)
    out = np.zeros((num, 3, size, size), dtype=np.float32)
    yy, xx = np.meshgrid(np.arange(size), np.arange(size), indexing="ij")
    for i in range(num):
        cx, cy = rng.uniform(4, size - 4, size=2)
        r = rng.uniform(2, 4)
        mask = (xx - cx) ** 2 + (yy - cy) ** 2 < r ** 2
        colour = rng.uniform(-1, 1, size=3)
        for c in range(3):
            out[i, c][mask] = colour[c]
    return torch.from_numpy(out)
```

Train a TinyDiT on this with rectified flow. After 500 steps, sampled outputs should look like faint blobs of colour.

## Use It

For real image generation with FLUX / SD3 / Z-Image, diffusers ships every one with a unified API:

```
from diffusers import FluxPipeline, StableDiffusion3Pipeline
import torch

pipe = FluxPipeline.from_pretrained(
    "black-forest-labs/FLUX.1-schnell",
    torch_dtype=torch.bfloat16,
```

```

).to("cuda")

out = pipe(
    prompt="a golden retriever surfing a tsunami, hyperrealistic, studio lighting",
    guidance_scale=0.0,           # schnell was trained without CFG
    num_inference_steps=4,
    max_sequence_length=256,
).images[0]
out.save("surf.png")

```

Three lines. FLUX.1-schnell in four steps. Swap the model id for black-forest-labs/FLUX.1-dev for higher quality at 20-30 steps with CFG.

For SD3:

```

pipe = StableDiffusion3Pipeline.from_pretrained(
    "stabilityai/stable-diffusion-3.5-large",
    torch_dtype=torch.bfloat16,
).to("cuda")
out = pipe(prompt, guidance_scale=3.5, num_inference_steps=28).images[0]

```

**This chapter ships an artifact.** The course version of this lesson produces a reusable prompt or agent skill. It lives in the repository, ready to install: <https://github.com/rohitg00/ai-engineering-from-scratch/tree/main/phases/04-computer-vision/23-diffusion-transformers-rectified-flow>

## Exercises

Starter code and the lesson’s working implementation: <https://github.com/rohitg00/ai-engineering-from-scratch/tree/main/phases/04-computer-vision/23-diffusion-transformers-rectified-flow/code>

1. **(Easy)** Train the TinyDiT above on the synthetic blob dataset for 500 steps. Compare samples produced with 10, 20, and 50 Euler steps.
2. **(Medium)** Add text conditioning by concatenating a learned class embedding to the time embedding (10 blob “classes” by colour). Sample with class 0, 5, and 9 and verify colours match.
3. **(Hard)** Compute the Fréchet distance (FID proxy) between generated samples from rectified-flow and DDPM versions of the same-size network trained on the same data for the same number of steps. Report which converges faster.

## Key Terms

Term	What people say	What it actually means
DiT	“Diffusion transformer”	Transformer that replaces the U-Net as the diffusion denoiser; operates on patchified latents
AdaLN	“Adaptive layer norm”	Timestep/text conditioning via learned scale, shift, gate applied after LayerNorm; standard in every modern DiT
MMDiT	“Multi-modal DiT (SD3)”	Separate weight streams for text and image tokens that share a joint self-attention

Term	What people say	What it actually means
Single-stream / double-stream	“FLUX trick”	First N blocks double-stream (separate weights per modality), later blocks single-stream (concat + shared weights) for efficiency
Rectified flow	“Straight-line noise-to-data”	Linear interpolation between data and noise; network predicts velocity; fewer ODE steps needed at inference
Velocity target	“epsilon - $x_0$ ”	The regression target in rectified flow; points from clean data to noise
CFG guidance	“classifier-free guidance”	Mix conditional and unconditional predictions; still used in rectified-flow models
Schnell / turbo / LCM	“1-4 step distillation”	Small-step variants distilled from full-quality models; production real-time

## Further Reading

- Scalable Diffusion Models with Transformers (Peebles & Xie, 2023) — the DiT paper
- Scaling Rectified Flow Transformers (Esser et al., SD3 paper) — MMDiT and rectified-flow at scale
- FLUX.1 model card and technical report (Black Forest Labs) — double + single-stream details
- Z-Image: Efficient Image Generation Foundation Model (2025) — single-stream DiT at 6B
- Elucidating the Design Space of Diffusion (Karras et al., 2022) — the reference for every diffusion design trade-off
- Latent Consistency Models (Luo et al., 2023) — how LCM-LoRA gives you 4-step inference

**Continue online.** The living edition of this chapter has more than the page can hold:

- Animated, interactive figures and the web text: <https://aiengineeringfromscratch.com/lesson?path=phases/04-computer-vision/23-diffusion-transformers-rectified-flow>
- Runnable code for every step: <https://github.com/rohitg00/ai-engineering-from-scratch/tree/main/phases/04-computer-vision/23-diffusion-transformers-rectified-flow/code>
- The chapter quiz, graded in the browser: <https://aiengineeringfromscratch.com/lesson?path=phases/04-computer-vision/23-diffusion-transformers-rectified-flow>

The repository moves faster than any printing. When the book and the repo disagree, trust the repo.

# SAM 3 & Open-Vocabulary Segmentation

---

Give a model a text prompt and an image and get masks for every matching object. SAM 3 made that a single forward pass.

**Type:** Use + Build **Languages:** Python **Prerequisites:** Phase 4 Lesson 07 (U-Net), Phase 4 Lesson 08 (Mask R-CNN), Phase 4 Lesson 18 (CLIP) **Time:** ~60 minutes

## Learning Objectives

- Distinguish SAM (visual prompts only), Grounded SAM / SAM 2 (detector + SAM), and SAM 3 (native text prompts via Promptable Concept Segmentation)
- Explain the SAM 3 architecture: shared backbone + image detector + memory-based video tracker + presence head + decoupled detector-tracker design
- Use Hugging Face transformers SAM 3 integration for text-prompted detection, segmentation, and video tracking
- Pick between SAM 3, Grounded SAM 2, YOLO-World, and SAM-MI based on latency, concept complexity, and deployment target

## The Problem

The 2023 SAM was a visual-prompt-only model: you click a point or draw a box and it returns a mask. For “give me all the oranges in this photo” you needed a detector (Grounding DINO) to produce boxes, then SAM to segment each. Grounded SAM turned this into a pipeline, but it was a cascade of two frozen models with inevitable error accumulation.

SAM 3 (Meta, Nov 2025, ICLR 2026) collapsed the cascade. It accepts a short noun phrase or an image exemplar as prompt and returns all matching masks and instance IDs in a single forward pass. That is **Promptable Concept Segmentation (PCS)**. Combined with the March 2026 Object Multiplex update (SAM 3.1), it tracks multiple instances of the same concept through video efficiently.

This lesson is about the structural shift this represents. 2D seg, detection, and text-image grounding have merged into one model. The production question is no longer “which pipeline do I chain together” but “which promptable model handles my use case end-to-end.”

## The Concept

### The three generations

**Diagram.** Rendered live in the web edition: <https://aiengineeringfromscratch.com/lesson?path=phases/04-computer-vision/24-sam3-open-vocab-segmentation>

## Promptable Concept Segmentation

A “concept prompt” is a short noun phrase (“yellow school bus”, “striped red umbrella”, “hand holding a mug”) or an image exemplar. The model returns segmentation masks for every instance in the image that matches the concept, plus a unique instance ID per match.

This differs from classic visual-prompt SAM in three ways:

1. No per-instance prompting required — one text prompt returns all matches.
2. Open-vocabulary — the concept can be anything describable in natural language.
3. Returns multiple instances at once rather than one mask per prompt.

## Key architectural pieces

- **Shared backbone** — a single ViT processes the image. Both the detector head and the memory-based tracker read from it.
- **Presence head** — predicts whether the concept is present in the image at all. Decouples “is this here?” from “where is it?”. Reduces false positives on absent concepts.
- **Decoupled detector-tracker** — image-level detection and video-level tracking have separate heads so they do not interfere.
- **Memory bank** — stores per-instance features across frames for video tracking (same mechanism SAM 2 used).

## Training at scale

SAM 3 was trained on **4 million unique concepts** generated by a data engine that iteratively annotates and corrects using AI + human review. The new **SA-CO benchmark** contains 270K unique concepts, 50x larger than prior benchmarks. SAM 3 reaches 75-80% of human performance on SA-CO and doubles existing systems on image + video PCS.

## SAM 3.1 Object Multiplex

March 2026 update: **Object Multiplex** introduces a shared-memory mechanism for joint tracking of many instances of the same concept at once. Previously, tracking N instances meant N separate memory banks. Multiplex collapses that into one shared memory with per-instance queries. Result: substantially faster multi-object tracking without sacrificing accuracy.

## Where Grounded SAM still matters in 2026

- When you need a specific open-vocabulary detector swapped in (DINO-X, Florence-2).
- When the SAM 3 license (gated on HF) is a blocker.
- When you need more control over the detector threshold than SAM 3 exposes.
- For research / ablation work on the detector component.

Modular pipelines still have a place. For most production work, SAM 3 is the simpler answer.

## YOLO-World vs SAM 3

- **YOLO-World** — open-vocabulary detector only (no masks). Real-time. Best when you need boxes at high fps.
- **SAM 3** — full segmentation + tracking. Slower but richer output.

Production split: YOLO-World for fast detection-only pipelines (robotics navigation, fast dashboards), SAM 3 for anything that needs masks or tracking.



## SAM-MI efficiency

SAM-MI (2025-2026) addresses SAM's decoder bottleneck. Key ideas:

- **Sparse point prompting** — uses a few well-chosen points instead of dense prompts; reduces decoder calls by 96%.
- **Shallow mask aggregation** — merges rough mask predictions into one sharper mask.
- **Decoupled mask injection** — decoder receives pre-computed mask features instead of re-running.

Result:  $\sim 1.6\times$  speedup over Grounded-SAM on open-vocabulary benchmarks.

## Output format for the three models

All return the same general structure (boxes + labels + scores + masks + IDs), which is helpful — your pipeline downstream does not have to branch on which model ran.

**Interactive figure: cv3-open-vocab.** This one moves. Watch it animate and drag its controls in the web edition: <https://aiengineeringfromscratch.com/lesson?path=phases/04-computer-vision/24-sam3-open-vocab-segmentation>

## Build It

### Step 1: Prompt construction

Build a helper that turns a user sentence into a list of SAM 3 concept prompts. This is the boundary where “what the user typed” meets “what the model consumes”.

```
def split_concepts(sentence):
    """
    Heuristic splitter for multi-concept prompts.
    Returns list of short noun phrases.
    """
    for sep in [",", ";", "and", "or", "&"]:
        if sep in sentence:
            parts = [p.strip() for p in sentence.replace("and ", ",").split(",")]
            return [p for p in parts if p]
    return [sentence.strip()]
```

```
print(split_concepts("cats, dogs and balloons"))
```

SAM 3 accepts one concept per forward pass; for multi-concept queries, loop or batch them.

### Step 2: Post-processing helpers

Turn SAM 3's raw outputs into a clean list of detections that match our Phase 4 Lesson 16 pipeline contract.

```
from dataclasses import dataclass
from typing import List

@dataclass
class ConceptDetection:
    concept: str
    instance_id: int
    box: tuple          # (x1, y1, x2, y2)
```

```
score: float
mask_rle: str          # run-length encoded
```

```
def rle_encode(binary_mask):
    flat = binary_mask.flatten().astype("uint8")
    runs = []
    prev, count = flat[0], 0
    for v in flat:
        if v == prev:
            count += 1
        else:
            runs.append((int(prev), count))
            prev, count = v, 1
    runs.append((int(prev), count))
    return ";".join(f"{v}x{c}" for v, c in runs)
```

RLE keeps response payloads small even for many high-resolution masks. The same format works across SAM 2, SAM 3, Grounded SAM 2.

### Step 3: A unified open-vocab segmentation interface

Wrap whatever backend you have (SAM 3, Grounded SAM 2, YOLO-World + SAM 2) behind a single method. Your downstream code does not change when the backend does.

```
from abc import ABC, abstractmethod
import numpy as np
```

```
class OpenVocabSeg(ABC):
    @abstractmethod
    def detect(self, image: np.ndarray, concept: str) -> List[ConceptDetection]:
        ...
```

```
class StubOpenVocabSeg(OpenVocabSeg):
    """
    Deterministic stub used for pipeline testing when real models are not loaded.
    """
    def detect(self, image, concept):
        h, w = image.shape[:2]
        return [
            ConceptDetection(
                concept=concept,
                instance_id=0,
                box=(w * 0.2, h * 0.3, w * 0.5, h * 0.8),
                score=0.89,
                mask_rle="0x100;1x50;0x200",
            ),
            ConceptDetection(
                concept=concept,
                instance_id=1,
                box=(w * 0.55, h * 0.25, w * 0.85, h * 0.75),
                score=0.74,
                mask_rle="0x80;1x40;0x220",
            ),
        ]
```

]

The real SAM3OpenVocabSeg subclass would wrap `transformers.Sam3Model` and `Sam3Processor`.

## Step 4: Hugging Face SAM 3 usage (reference)

For the actual model, the transformers integration:

```
from transformers import Sam3Processor, Sam3Model
import torch

processor = Sam3Processor.from_pretrained("facebook/sam3")
model = Sam3Model.from_pretrained("facebook/sam3").eval()

inputs = processor(images=pil_image, return_tensors="pt")
inputs = processor.set_text_prompt(inputs, "yellow school bus")

with torch.no_grad():
    outputs = model(**inputs)

masks = processor.post_process_masks(
    outputs.masks, inputs.original_sizes, inputs.resized_input_sizes
)
boxes = outputs.boxes
scores = outputs.scores
```

One prompt, all matches returned in a single call.

## Step 5: Measure what Grounded SAM 2 gave you for free

An honest benchmark: what happens when you replace Grounded SAM 2 with SAM 3 in a real pipeline?

- **Latency:** SAM 3 saves one forward pass (no separate detector) but the model itself is heavier; usually net-neutral or a slight speedup.
- **Accuracy:** SAM 3 substantially better on rare or compositional concepts ("striped red umbrella"). Similar on common single-word concepts.
- **Flexibility:** Grounded SAM 2 lets you swap detectors (DINO-X, Florence-2, Grounding DINO 1.5); SAM 3 is monolithic.

Conclusion: SAM 3 is the default for 2026 open-vocab seg. Grounded SAM 2 is still the right answer when you need detector flexibility or different license terms.

## Use It

Production deployment patterns:

- **Real-time annotation** — SAM 3 + CVAT's label-as-text-prompt feature. Annotators select a label name; SAM 3 pre-labels every matching instance. Review and correct.
- **Video analytics** — SAM 3.1 Object Multiplex for multi-object tracking; feed frames to the memory-based tracker.
- **Robotics** — SAM 3 for open-vocab manipulation ("pick up the red cup"); runs as a planning primitive.
- **Medical imaging** — SAM 3 fine-tuned on medical concepts; requires access request on HF.

Ultralytics wraps SAM 3 in its Python package:

```
from ultralytics import SAM
```

```
model = SAM("sam3.pt")
results = model(image_path, prompts="yellow school bus")
```

Same interface as YOLO and SAM 2.

**This chapter ships an artifact.** The course version of this lesson produces a reusable prompt or agent skill. It lives in the repository, ready to install: <https://github.com/rohitg00/ai-engineering-from-scratch/tree/main/phases/04-computer-vision/24-sam3-open-vocabulary-segmentation>

## Exercises

Starter code and the lesson’s working implementation: <https://github.com/rohitg00/ai-engineering-from-scratch/tree/main/phases/04-computer-vision/24-sam3-open-vocabulary-segmentation/code>

1. **(Easy)** Run SAM 3 on 10 images with concept prompts you choose. Compare against SAM 2 + Grounding DINO 1.5 on the same images. Report which concepts each model missed.
2. **(Medium)** Build a “click-to-include / click-to-exclude” UI on top of SAM 3: a text prompt returns candidate instances; user clicks keep which ones count as positive. Output the final concept set as JSON.
3. **(Hard)** Fine-tune SAM 3 on a custom concept set (e.g. 5 types of electronic components) with 20 labelled images each. Compare to zero-shot SAM 3 on the same test set; measure mask IoU improvement.

## Key Terms

Term	What people say	What it actually means
Open-vocabulary segmentation	“Segment by text”	Produce masks for objects described in natural language, not a fixed label set
PCS	“Promptable Concept Segmentation”	SAM 3’s core task — given a noun-phrase or image exemplar, segment all matching instances
Concept prompt	“The text input”	Short noun phrase or image exemplar; not a full sentence
Presence head	“Is it here?”	SAM 3 module that decides whether the concept exists in the image before localisation
SA-CO	“SAM 3 benchmark”	270K-concept open-vocabulary segmentation benchmark; 50x larger than prior open-vocab benchmarks
Object Multiplex	“SAM 3.1 update”	Shared-memory multi-object tracking; fast joint tracking of many instances
Grounded SAM 2	“Modular pipeline”	Detector + SAM 2 cascade; still relevant when detector swap matters

Term	What people say	What it actually means
SAM-MI	“Efficient SAM variant”	Mask Injection for 1.6x speedup over Grounded-SAM

## Further Reading

- SAM 3: Segment Anything with Concepts (arXiv 2511.16719)
- SAM 3.1 Object Multiplex (Meta AI, March 2026)
- SAM 3 model page on Hugging Face
- Grounded SAM 2 tutorial (PyImageSearch)
- Ultralytics SAM 3 docs
- SAM3-I: Instruction-aware SAM (arXiv 2512.04585)

**Continue online.** The living edition of this chapter has more than the page can hold:

- Animated, interactive figures and the web text: <https://aiengineeringfromscratch.com/lesson?path=phases/04-computer-vision/24-sam3-open-vocab-segmentation>
- Runnable code for every step: <https://github.com/rohitg00/ai-engineering-from-scratch/tree/main/phases/04-computer-vision/24-sam3-open-vocab-segmentation/code>
- The chapter quiz, graded in the browser: <https://aiengineeringfromscratch.com/lesson?path=phases/04-computer-vision/24-sam3-open-vocab-segmentation>

The repository moves faster than any printing. When the book and the repo disagree, trust the repo.

# Vision-Language Models — The ViT-MLP-LLM Pattern

---

A vision encoder converts an image into tokens. An MLP projector maps those tokens into the LLM's embedding space. A language model does the rest. That pattern — ViT-MLP-LLM — is every production VLM in 2026.

**Type:** Learn + Use **Languages:** Python **Prerequisites:** Phase 4 Lesson 14 (ViT), Phase 4 Lesson 18 (CLIP), Phase 7 Lesson 02 (Self-Attention) **Time:** ~75 minutes

## Learning Objectives

- State the ViT-MLP-LLM architecture and explain what each of the three components contributes
- Compare Qwen3-VL, InternVL3.5, LLaVA-Next, and GLM-4.6V on parameter count, context length, and benchmark performance
- Explain DeepStack: why multi-level ViT features tighten vision-language alignment better than a single last-layer feature
- Measure VLM hallucination in production with Cross-Modal Error Rate (CMER) and act on the signal

## The Problem

CLIP (Phase 4 Lesson 18) gives you a shared embedding space for images and text, which is enough for zero-shot classification and retrieval. It cannot answer “how many red cars are in this image?” because CLIP does not generate text — it only scores similarities.

Vision-Language Models (VLMs) — Qwen3-VL, InternVL3.5, LLaVA-Next, GLM-4.6V — bolt a CLIP-family image encoder to a full language model. The model sees an image plus a question and generates an answer. In 2026 open-source VLMs rival or beat GPT-5 and Gemini-2.5-Pro on multimodal benchmarks (MMMU, MMBench, DocVQA, ChartQA, MathVista, OSWorld).

The trio of pieces (ViT, projector, LLM) is the standard. The differences between models are in which ViT, which projector, which LLM, the training data, and the alignment recipe. Once you understand the pattern, swapping any component is mechanical.

## The Concept

### The ViT-MLP-LLM architecture

**Diagram.** Rendered live in the web edition: <https://aiengineeringfromscratch.com/lesson?path=phases/04-computer-vision/25-vision-language-models>

1. **Vision encoder** — a pretrained ViT (CLIP-L/14, SigLIP, DINOv3, or a fine-tuned variant). Produces patch tokens.

2. **Projector** — a small module (2-4 layer MLP, or a Q-former) that maps vision tokens into the LLM’s embedding dimension. This is where most of the fine-tuning happens.
3. **LLM** — a decoder-only language model (Qwen3, Llama, Mistral, GLM, InternLM). Reads the vision + text tokens in sequence, generates text.

All three pieces are trainable in principle. In practice, the vision encoder and LLM stay mostly frozen while the projector trains — a few billion parameters of signal for cheap.

## DeepStack

Vanilla projection uses only the last ViT layer. DeepStack (Qwen3-VL) samples features from multiple ViT depths and stacks them. Deeper layers carry high-level semantics; shallower layers carry fine-grained spatial and textural information. Feeding both into the LLM closes the gap between “what does the image contain” (semantics) and “where exactly” (spatial grounding).

## Three training stages

Modern VLMs train in stages:

1. **Alignment** — freeze ViT and LLM. Train only the projector on image-caption pairs. Teaches the projector to map vision space into language space.
2. **Pre-training** — unfreeze everything. Train on large-scale interleaved image-text data (500M+ pairs). Builds the model’s visual knowledge.
3. **Instruction tuning** — fine-tune on curated (image, question, answer) triples. Teaches conversational behaviour and task formats. This is what turns a “vision-aware LM” into a usable assistant.

Most LoRA fine-tunes target stage 3 with a small labelled dataset.

## Model family comparison (early 2026)

Model	Params	Vision encoder	LLM	Context	Strengths
Qwen3-VL-235B-A22B (MoE)	235B (22B active)	custom ViT + DeepStack	Qwen3	256K	General SOTA, GUI agent
Qwen3-VL-30B-A3B (MoE)	30B (3B active)	custom ViT + DeepStack	Qwen3	256K	Smaller MoE alternative
Qwen3-VL-8B (dense)	8B	custom ViT	Qwen3	128K	Production dense default
InternVL3.5-38B		InternViT-6B	Qwen3 + GPT-OSS	128K	Strong MMBench / MMVet
InternVL 3.5-241B-A28B	241B (28B active)	InternViT-6B	Qwen3	128K	Competitive with GPT-4o
LLaVA-Next 72B	72B	SigLIP	Llama-3	32K	Open, easy to fine-tune

Model	Params	Vision encoder	LLM	Context	Strengths
GLM-4.6V	~70B	custom	GLM	64K	Open-source, strong OCR
MiniCPM-V-2.6	8B	SigLIP	MiniCPM32K		Edge-friendly

## Visual agents

Qwen3-VL-235B reaches top global performance on OSWorld — a benchmark for **visual agents** that operate GUIs (desktop, mobile, web). The model sees a screenshot, understands the UI, and emits actions (click, type, scroll). Combined with tools, it closes the loop on common desktop tasks. This is what most 2026 “AI PC” demos run under the hood.

## Agentic capabilities + RoPE variants

VLMs need to know **when** a frame is in a video. Qwen3-VL evolved from T-RoPE (temporal rotary position embeddings) to **text-based time alignment** — explicit timestamp text tokens interleaved with video frames. The model sees “<timestamp 00:32> frame, prompt” and can reason about temporal relationships.

## The alignment problem

12% of image-text pairs in a crawled dataset contain descriptions not fully grounded in the image. A VLM trained on this silently learns to hallucinate — fabricate objects, misread numbers, invent relationships. In production this is the dominant failure mode.

Skywork.ai introduced the **Cross-Modal Error Rate (CMER)** to track it:

CMER = fraction of outputs where the text confidence is high but the image-text  
 ↪ similarity (via a CLIP-family checker) is low

High CMER means the model is confidently saying things not grounded in the image. Monitoring CMER and treating it as a production KPI cut hallucination rate by ~35% in their deployment. The trick is not “fix the model” but “route high-CMER outputs to human review.”

## Fine-tuning with LoRA / QLoRA

Full fine-tuning of a 70B VLM is out of reach for most teams. LoRA (rank 16-64) on attention + projector layers, or QLoRA with 4-bit base weights, fits on a single A100 / H100. Cost: 5,000-50,000 examples, \$100-\$5,000 in compute, 2-10 hours of training.

## Spatial reasoning is still weak

Current VLMs score 50-60% on spatial reasoning benchmarks (above-below, left-right, counting, distance). If your use case depends on “which object is on top of which,” validate heavily — generic VLM performance is below human. Better-than-VLM alternatives for pure spatial tasks: a specialised keypoint / pose estimator, a depth model, or a detection model with box geometry post-processed.

**Interactive figure: v4-vlm-projector.** This one moves. Watch it animate and drag its controls in the web edition: <https://aiengineeringfromscratch.com/lesson?path=phases/04-computer-vision/25-vision-language-models>



## Build It

### Step 1: The projector

The part you will train most often. 2-4 layer MLP with GELU.

```
import torch
import torch.nn as nn

class Projector(nn.Module):
    def __init__(self, vit_dim=768, llm_dim=4096, hidden=4096):
        super().__init__()
        self.net = nn.Sequential(
            nn.Linear(vit_dim, hidden),
            nn.GELU(),
            nn.Linear(hidden, llm_dim),
        )

    def forward(self, x):
        return self.net(x)
```

Input is a (N_patches, d_vit) token tensor. Output is (N_patches, d_llm). The LLM treats every output row as just another token.

### Step 2: Assemble ViT-MLP-LLM end-to-end

Skeleton of the forward pass for a minimal VLM. Real code uses transformers; this is the conceptual layout.

```
class MinimalVLM(nn.Module):
    def __init__(self, vit, projector, llm, image_token_id):
        super().__init__()
        self.vit = vit
        self.projector = projector
        self.llm = llm
        self.image_token_id = image_token_id  # placeholder token in text prompt

    def forward(self, image, input_ids, attention_mask):
        # 1. vision features
        vision_tokens = self.vit(image)  # (B, N_patches, d_vit)
        vision_embeds = self.projector(vision_tokens)  # (B, N_patches, d_llm)

        # 2. text embeddings
        text_embeds = self.llm.get_input_embeddings()(input_ids)  # (B, M, d_llm)

        # 3. replace image placeholder tokens with vision embeds
        merged = self._merge(text_embeds, vision_embeds, input_ids)

        # 4. run LLM
        return self.llm(inputs_embeds=merged, attention_mask=attention_mask)

    def _merge(self, text_embeds, vision_embeds, input_ids):
        out = text_embeds.clone()
        expected = vision_embeds.size(1)
```

```

        for b in range(input_ids.size(0)):
            positions = (input_ids[b] ==
                ↪ self.image_token_id).nonzero(as_tuple=True)[0]
            if len(positions) != expected:
                raise ValueError(
                    f"batch item {b} has {len(positions)} image tokens but
                    ↪ vision_embeds has {expected} patches."
                    " Every sample in the batch must be pre-padded to the same
                    ↪ number of image placeholder tokens.")
            out[b, positions] = vision_embeds[b]
        return out

```

The <image> placeholder token in the text gets replaced with real image embeddings — same pattern LLaVA, Qwen-VL, and InternVL use.

### Step 3: CMER computation

A lightweight runtime check.

```
import torch.nn.functional as F
```

```

def cross_modal_error_rate(image_emb, text_emb, text_confidence,
    ↪ sim_threshold=0.25, conf_threshold=0.8):
    """
    image_emb, text_emb: embeddings of image and generated text (normalised
    ↪ internally)
    text_confidence:    mean per-token probability in [0, 1]
    Returns:           fraction of high-confidence outputs with low image-text
    ↪ alignment
    """
    image_emb = F.normalize(image_emb, dim=-1)
    text_emb = F.normalize(text_emb, dim=-1)
    sim = (image_emb * text_emb).sum(dim=-1)           # cosine similarity
    high_conf_low_sim = (text_confidence > conf_threshold) & (sim < sim_threshold)
    return high_conf_low_sim.float().mean().item()

```

Treat CMER as a production KPI. Monitor it per endpoint, per prompt type, per customer. Rising CMER indicates the model is starting to hallucinate on some input distribution.

### Step 4: Toy VLM classifier (runnable)

Demonstrate the projector trains. Fake “ViT features” go in; a tiny LLM-style token predicts a class.

```

class ToyVLM(nn.Module):
    def __init__(self, vit_dim=32, llm_dim=64, num_classes=5):
        super().__init__()
        self.projector = Projector(vit_dim, llm_dim, hidden=64)
        self.head = nn.Linear(llm_dim, num_classes)

    def forward(self, vision_tokens):
        projected = self.projector(vision_tokens)
        pooled = projected.mean(dim=1)
        return self.head(pooled)

```

One can fit this on synthetic (feature, class) pairs in under 200 steps — enough to show the projector pattern works.

## Use It

Three ways production teams use VLMs in 2026:

- **Hosted API** — OpenAI Vision, Anthropic Claude Vision, Google Gemini Vision. Zero infra, vendor risk.
- **Open-source self-host** — Qwen3-VL or InternVL3.5 via transformers and vllm. Full control, higher up-front effort.
- **Fine-tune on domain** — load Qwen2.5-VL-7B or LLaVA-1.6-7B, LoRA on 5k-50k custom examples, serve with vllm or TGI.

```
from transformers import AutoProcessor, AutoModelForVision2Seq
import torch
from PIL import Image

model_id = "Qwen/Qwen3-VL-8B-Instruct"
processor = AutoProcessor.from_pretrained(model_id)
model = AutoModelForVision2Seq.from_pretrained(model_id,
    ↳ torch_dtype=torch.bfloat16, device_map="auto")

messages = [{
    "role": "user",
    "content": [
        {"type": "image", "image": Image.open("plot.png")},
        {"type": "text", "text": "What does this chart show?"},
    ],
}]

inputs = processor.apply_chat_template(messages, add_generation_prompt=True,
    ↳ tokenize=True, return_dict=True, return_tensors="pt").to("cuda")
generated = model.generate(**inputs, max_new_tokens=256)
answer = processor.decode(generated[0][inputs["input_ids"].shape[1]:],
    ↳ skip_special_tokens=True)
```

`apply_chat_template` hides the `<image>` placeholder tokenisation; the model handles the merge internally.

**This chapter ships an artifact.** The course version of this lesson produces a reusable prompt or agent skill. It lives in the repository, ready to install: <https://github.com/rohitg00/ai-engineering-from-scratch/tree/main/phases/04-computer-vision/25-vision-language-models>

## Exercises

Starter code and the lesson’s working implementation: <https://github.com/rohitg00/ai-engineering-from-scratch/tree/main/phases/04-computer-vision/25-vision-language-models/code>

1. **(Easy)** Run three prompts (“what is this?”, “count the objects”, “describe the scene”) through any open VLM on five images. Score each answer as correct / partially correct / hallucinated by hand. Compute a first-pass CMER-like rate.
2. **(Medium)** Fine-tune Qwen2.5-VL-3B or LLaVA-1.6-7B with LoRA (rank 16) on 500 images of a target domain with captions. Compare zero-shot vs fine-tuned MMBench-style accuracy.

3. **(Hard)** Replace the VLM’s image encoder with DINOv3 instead of its default SigLIP/CLIP. Re-train only the projector (frozen LLM + frozen DINOv3). Measure whether dense-prediction tasks (counting, spatial reasoning) improve.

## Key Terms

Term	What people say	What it actually means
ViT-MLP-LLM	“The VLM pattern”	Vision encoder + projector + language model; every 2026 VLM
Projector	“The bridge”	2-4 layer MLP (or Q-former) that maps vision tokens into LLM embedding space
DeepStack	“Qwen3-VL feature trick”	Multi-level ViT features stacked rather than last-layer only
Image token	“<image> placeholder”	Special token in the text stream replaced by projected vision embeddings
CMER	“Hallucination KPI”	Cross-Modal Error Rate; high when text confidence is high but image-text similarity is low
Visual agent	“VLM that clicks”	VLM operating GUIs (OSWorld, mobile, web) with tool calls
Q-former	“Fixed-count token bridge”	BLIP-2 style projector producing a fixed number of visual query tokens
Alignment / pre-training / instruction tuning	“Three stages”	Standard VLM training pipeline

## Further Reading

- Qwen3-VL Technical Report (arXiv 2511.21631)
- InternVL3.5 Advancing Open-Source Multimodal Models (arXiv 2508.18265)
- LLaVA-Next series
- BentoML: Best Open-Source VLMs 2026
- MMMU: Multi-discipline Multimodal Understanding benchmark
- VLMs in manufacturing (Robotics Tomorrow, March 2026)

**Continue online.** The living edition of this chapter has more than the page can hold:

- Animated, interactive figures and the web text: <https://aiengineeringfromscratch.com/lesson?path=phases/04-computer-vision/25-vision-language-models>
- Runnable code for every step: <https://github.com/rohitg00/ai-engineering-from-scratch/tree/main/phases/04-computer-vision/25-vision-language-models/code>
- The chapter quiz, graded in the browser: <https://aiengineeringfromscratch.com/lesson?path=phases/04-computer-vision/25-vision-language-models>

The repository moves faster than any printing. When the book and the repo disagree, trust the repo.

# Monocular Depth & Geometry Estimation

---

A depth map is a single-channel image where each pixel is a distance from the camera. Predicting it from one RGB frame used to be impossible without stereo or LiDAR. In 2026 a frozen ViT encoder plus a lightweight head gets within a few percent of ground truth.

**Type:** Build + Use **Languages:** Python **Prerequisites:** Phase 4 Lesson 14 (ViT), Phase 4 Lesson 17 (Self-Supervised Vision), Phase 4 Lesson 07 (U-Net) **Time:** ~60 minutes

## Learning Objectives

- Distinguish relative and metric depth and state which one each production model (MiDaS, Marigold, Depth Anything V3, ZoeDepth) solves
- Use Depth Anything V3 (DINOv2 backbone) to predict depth for arbitrary single images with no calibration
- Explain why monocular depth works at all from a single image (perspective cues, texture gradients, learned priors) and what it cannot recover (absolute scale, occluded geometry)
- Lift 2D detections to 3D points using a depth map and pinhole camera intrinsics

## The Problem

Depth is the missing axis in 2D computer vision. Given RGB, you know where things appear in the image plane; you do not know how far they are. Depth sensors (stereo rigs, LiDAR, time-of-flight) solve this directly but are expensive, fragile, and limited in range.

Monocular depth estimation — predicting depth from a single RGB frame — used to produce blurry, unreliable output. By 2026 large pretrained encoders changed that: Depth Anything V3 uses a frozen DINOv2 backbone and produces depth maps that generalise across indoor, outdoor, medical, and satellite domains. Marigold reframes depth as a conditional diffusion problem. ZoeDepth regresses true metric distances.

Depth is also the bridge between 2D detection and 3D understanding: multiply a detected box's pixels by depth and you lift the 2D object into a 3D point cloud. That is the core of every AR occlusion system, every obstacle-avoidance pipeline, and every "pick up the cup" robot.

## The Concept

### Relative vs metric depth

- **Relative depth** — ordered z values without a real-world unit. "Pixel A is closer than pixel B, but the ratio of distances is not anchored to metres."

- **Metric depth** — absolute distance in metres from the camera. Requires the model to have learnt the statistical relationship between image cues and real distance.

MiDaS and Depth Anything V3 produce relative depth. Marigold produces relative depth. ZoeDepth, UniDepth, and Metric3D produce metric depth. Metric models are sensitive to camera intrinsics; relative models are not.

## The encoder-decoder pattern

**Diagram.** Rendered live in the web edition: <https://aiengineeringfromscratch.com/lesson?path=phases/04-computer-vision/26-monocular-depth>

Depth Anything V3 freezes the encoder and trains only the DPT-style decoder. The encoder provides rich features; the decoder interpolates them back to image resolution and regresses depth.

## Why a single image produces depth at all

A 2D image contains many monocular cues that correlate with depth:

- **Perspective** — parallel lines in 3D converge in 2D.
- **Texture gradient** — surfaces far away have smaller, denser texture.
- **Occlusion order** — nearer objects occlude farther ones.
- **Size constancy** — known objects (cars, humans) give approximate scale.
- **Atmospheric perspective** — distant objects appear hazier and bluer in outdoor scenes.

A ViT trained on billions of images internalises these cues. With enough data and a strong backbone, monocular depth hits reasonable accuracy without any explicit 3D supervision.

## What monocular depth cannot do

- **Absolute metric scale** without intrinsics or a known object in the scene. The network can predict “the cup is twice as far as the spoon” without knowing whether the cup is 1 m or 10 m away.
- **Occluded geometry** — the back of a chair is unseen and cannot be inferred reliably.
- **Truly untextured / reflective surfaces** — mirrors, glass, uniform walls. The network reports plausible but wrong depth.

## Depth Anything V3 in 2026

- Vanilla DINOv2 ViT-L/14 as encoder (frozen).
- DPT decoder.
- Trained on posed image pairs from diverse sources (no explicit depth supervision needed beyond photometric consistency).
- Predicts spatially consistent geometry from **an arbitrary number of visual inputs, with or without known camera poses**.
- SOTA across monocular depth, any-view geometry, visual rendering, camera pose estimation.

This is the drop-in model to call when you need depth in 2026.

## Marigold — diffusion for depth

Marigold (Ke et al., CVPR 2024) reframes depth estimation as conditional image-to-image diffusion. Conditioning: RGB. Target: depth map. Uses a pretrained Stable Diffusion 2 U-Net

as backbone. Output depth maps are exceptionally sharp at object boundaries. Trade-off: slower inference than feed-forward models (10-50 denoising steps).

## Intrinsics and the pinhole camera

To lift a pixel ( $u$ ,  $v$ ) with depth  $d$  to a 3D point ( $X$ ,  $Y$ ,  $Z$ ) in camera coordinates:

```
fx, fy, cx, cy = camera_intrinsics
X = (u - cx) * d / fx
Y = (v - cy) * d / fy
Z = d
```

Intrinsics come from EXIF metadata, a calibration pattern, or a monocular intrinsics estimator (Perspective Fields, UniDepth). Without intrinsics, you can still render a point cloud by assuming a 60-70° FOV and moderate-resolution principals — usable for visualisation, not for measurement.

## Evaluation

Two standard metrics:

- **AbsRel** (absolute relative error):  $\text{mean}(|d_{\text{pred}} - d_{\text{gt}}| / d_{\text{gt}})$ . Lower is better. 0.05-0.1 for production models.
- **delta < 1.25** (threshold accuracy): fraction of pixels where  $\max(d_{\text{pred}}/d_{\text{gt}}, d_{\text{gt}}/d_{\text{pred}}) < 1.25$ . Higher is better. 0.9+ for SOTA.

For relative depth (Depth Anything V3, MiDaS), evaluation uses scale-and-shift invariant versions of both metrics.

**Interactive figure: depth-sweep.** This one moves. Watch it animate and drag its controls in the web edition: <https://aiengineeringfromscratch.com/lesson?path=phases/04-computer-vision/26-monocular-depth>

## Build It

### Step 1: Depth metrics

```
import torch

def abs_rel_error(pred, target, mask=None):
    if mask is not None:
        pred = pred[mask]
        target = target[mask]
    return (torch.abs(pred - target) / target.clamp(min=1e-6)).mean().item()

def delta_accuracy(pred, target, threshold=1.25, mask=None):
    if mask is not None:
        pred = pred[mask]
        target = target[mask]
    ratio = torch.maximum(pred / target.clamp(min=1e-6), target /
        ↪ pred.clamp(min=1e-6))
    return (ratio < threshold).float().mean().item()
```

Always mask invalid depth pixels (zero, NaN, saturated) before evaluation.

## Step 2: Scale-and-shift alignment

For relative-depth models, align prediction to ground truth before computing metrics. Least-squares fit of  $a * \text{pred} + b = \text{target}$ :

```
def align_scale_shift(pred, target, mask=None):
    if mask is not None:
        p = pred[mask]
        t = target[mask]
    else:
        p = pred.flatten()
        t = target.flatten()
    A = torch.stack([p, torch.ones_like(p)], dim=1)
    coeffs, *_ = torch.linalg.lstsq(A, t.unsqueeze(-1))
    a, b = coeffs[:2, 0]
    return a * pred + b
```

Run `align_scale_shift` before `abs_rel_error` when evaluating MiDaS / Depth Anything.

## Step 3: Lift depth to a point cloud

```
import numpy as np

def depth_to_point_cloud(depth, intrinsics):
    H, W = depth.shape
    fx, fy, cx, cy = intrinsics
    v, u = np.meshgrid(np.arange(H), np.arange(W), indexing="ij")
    z = depth
    x = (u - cx) * z / fx
    y = (v - cy) * z / fy
    return np.stack([x, y, z], axis=-1)
```

```
depth = np.random.uniform(0.5, 4.0, (240, 320))
intr = (320.0, 320.0, 160.0, 120.0)
pc = depth_to_point_cloud(depth, intr)
print(f"point cloud shape: {pc.shape} (H, W, 3)")
```

One function, every 3D-lifted application. Export the point cloud to `.ply` and open in MeshLab or CloudCompare.

## Step 4: Smoke test with a synthetic depth scene

```
def synthetic_depth(size=96):
    yy, xx = np.meshgrid(np.arange(size), np.arange(size), indexing="ij")
    # Floor: linear gradient from near (top) to far (bottom)
    depth = 1.0 + (yy / size) * 4.0
    # Box in the middle: closer
    mask = (np.abs(xx - size / 2) < size / 6) & (np.abs(yy - size * 0.6) < size / 6)
    depth[mask] = 2.0
    return depth.astype(np.float32)
```

```
gt = torch.from_numpy(synthetic_depth(96))
pred = gt + 0.3 * torch.randn_like(gt) # simulated prediction
```



```
aligned = align_scale_shift(pred, gt)
print(f"before align  absRel = {abs_rel_error(pred, gt):.3f}")
print(f"after align   absRel = {abs_rel_error(aligned, gt):.3f}")
```

## Step 5: Depth Anything V3 usage (reference)

```
import torch
from transformers import pipeline
from PIL import Image

pipe = pipeline(task="depth-estimation",
    ↪ model="LiheYoung/depth-anything-v2-large")

image = Image.open("street.jpg").convert("RGB")
out = pipe(image)
depth_np = np.array(out["depth"])
```

Three lines. `out["depth"]` is a PIL grayscale; convert to numpy for math. For Depth Anything V3 specifically, swap the model id once released; the API is unchanged.

## Use It

- **Depth Anything V3** (Meta AI / ByteDance, 2024-2026) — the default for relative depth. Fastest ViT-large-backbone model in production.
- **Marigold** (ETH, 2024) — highest visual quality, slow inference.
- **UniDepth** (ETH, 2024) — metric depth with camera intrinsics estimation.
- **ZoeDepth** (Intel, 2023) — metric depth; older, still reliable.
- **MiDaS v3.1** — legacy but stable; good baseline for comparison.

Typical integration pattern:

1. RGB frame arrives.
2. Depth model produces depth map.
3. Detector produces boxes.
4. Lift box centroids through depth to 3D; merge with point cloud if available.
5. Downstream: AR occlusion, path planning, object-size estimation, stereo replacement.

For real-time use, Depth Anything V2 Small (INT8 quantised) hits ~30 fps on a consumer GPU at 518x518.

**This chapter ships an artifact.** The course version of this lesson produces a reusable prompt or agent skill. It lives in the repository, ready to install: <https://github.com/rohitg00/ai-engineering-from-scratch/tree/main/phases/04-computer-vision/26-monocular-depth>

## Exercises

Starter code and the lesson's working implementation: <https://github.com/rohitg00/ai-engineering-from-scratch/tree/main/phases/04-computer-vision/26-monocular-depth/code>

1. **(Easy)** Run Depth Anything V2 on any 10 images of your desk. Save depth as grayscale PNGs and inspect. Identify one object whose predicted depth looks wrong and explain why the monocular cues failed.

2. **(Medium)** Given RGB + depth from Depth Anything V2, lift to a point cloud and render with open3d. Compare two scenes (indoor / outdoor) and note which looks more believable.
3. **(Hard)** Take five pairs of images that differ only by a known object's position (e.g. bottle moved 30 cm closer). Use UniDepth to predict metric depth on both. Report the predicted distance delta vs the true 30 cm.

## Key Terms

Term	What people say	What it actually means
Monocular depth	"Single-image depth"	Depth estimation from one RGB frame, no stereo or LiDAR
Relative depth	"Ordered depth"	Ordered z-values without real-world units
Metric depth	"Absolute distance"	Depth in metres; requires calibration or a model trained with metric supervision
AbsRel	"Absolute relative error"	Mean of
Delta	"delta < 1.25"	Fraction of pixels with prediction within 25% of ground truth
accuracy		
Pinhole camera	"fx, fy, cx, cy"	The camera model used to lift (u, v, d) to (X, Y, Z)
DPT	"Dense Prediction Transformer"	The conv-based decoder used on top of frozen ViT encoders for depth
DINOv2 backbone	"The reason it works"	Self-supervised features that generalise across domains without depth labels

## Further Reading

- Depth Anything V3 paper page — SOTA monocular depth with DINOv2 encoder
- Marigold (Ke et al., CVPR 2024) — diffusion-based depth estimation
- UniDepth (Piccinelli et al., 2024) — metric depth with intrinsics
- MiDaS v3.1 (Intel ISL) — the canonical relative-depth baseline
- DINOv3 blog post (Meta) — the encoder family that lifts depth accuracy

**Continue online.** The living edition of this chapter has more than the page can hold:

- Animated, interactive figures and the web text: <https://aiengineeringfromscratch.com/lesson?path=phases/04-computer-vision/26-monocular-depth>
- Runnable code for every step: <https://github.com/rohitg00/ai-engineering-from-scratch/tree/main/phases/04-computer-vision/26-monocular-depth/code>
- The chapter quiz, graded in the browser: <https://aiengineeringfromscratch.com/lesson?path=phases/04-computer-vision/26-monocular-depth>

The repository moves faster than any printing. When the book and the repo disagree, trust the repo.

# Multi-Object Tracking & Video Memory

---

Tracking is detection plus association. Detect every frame. Match this frame's detections to last frame's tracks by ID.

**Type:** Build **Languages:** Python **Prerequisites:** Phase 4 Lesson 06 (YOLO Detection), Phase 4 Lesson 08 (Mask R-CNN), Phase 4 Lesson 24 (SAM 3) **Time:** ~60 minutes

## Learning Objectives

- Distinguish tracking-by-detection from query-based tracking and name the algorithm families (SORT, DeepSORT, ByteTrack, BoT-SORT, SAM 2 memory tracker, SAM 3.1 Object Multiplex)
- Implement IoU + Hungarian assignment from scratch for classic tracking-by-detection
- Explain SAM 2's memory bank and why it handles occlusion better than IoU-based association
- Read the three tracking metrics (MOTA, IDF1, HOTA) and pick which one matters for a given use case

## The Problem

A detector tells you where the objects are in a single frame. A tracker tells you which detection in frame  $t$  is the same object as a detection in frame  $t-1$ . Without that, you cannot count objects crossing a line, follow a ball through an occlusion, or know "car #4 has been in the lane for 8 seconds."

Tracking is essential to every video-facing product: sports analytics, surveillance, autonomous driving, medical video analysis, wildlife monitoring, wordmark counting. The core building blocks are shared: a per-frame detector, a motion model (Kalman filter or something richer), an association step (Hungarian algorithm on IoU / cosine / learned features), and a track lifecycle (birth, update, death).

2026 brought two new patterns: **SAM 2 memory-based tracking** (feature-memory instead of motion-model association) and **SAM 3.1 Object Multiplex** (shared memory for many instances of the same concept). This lesson walks the classical stack first, then the memory-based approach.

## The Concept

### Tracking-by-detection

**Diagram.** Rendered live in the web edition: <https://aiengineeringfromscratch.com/lesson?path=phases/04-computer-vision/27-multi-object-tracking>

Every tracker you will encounter in 2026 is a variation on this loop. The differences:

- **SORT** (2016): Kalman filter + IoU Hungarian. Simple, fast, no appearance model.

- **DeepSORT** (2017): SORT + a CNN-based appearance feature per track (ReID embedding). Handles crossings better.
- **ByteTrack** (2021): associates low-confidence detections as a second stage; no appearance features needed but top performer on MOT17.
- **BoT-SORT** (2022): Byte + camera motion compensation + ReID.
- **StrongSORT / OC-SORT** — ByteTrack descendants with better motion and appearance.

## Kalman filter in one paragraph

A Kalman filter maintains a per-track state ( $x, y, w, h, dx, dy, dw, dh$ ) with a covariance. At each frame, **predict** the state using a constant-velocity model, then **update** with the matched detection. The update trusts the detection more when the predict uncertainty is high. This gives smooth trajectories and the ability to continue a track through a short occlusion (1-5 frames).

Every classical tracker uses a Kalman filter in the motion-prediction step.

## The Hungarian algorithm

Given a  $M \times N$  cost matrix (tracks  $\times$  detections), find the one-to-one assignment that minimises total cost. Cost is usually  $1 - \text{IoU}(\text{track_bbox}, \text{detection_bbox})$  or negative cosine similarity of appearance features. Runtime is  $O((M+N)^3)$ ; for  $M, N$  up to  $\sim 1000$  it is fast enough in Python via `scipy.optimize.linear_sum_assignment`.

## ByteTrack's key idea

Standard trackers drop low-confidence detections ( $< 0.5$ ). ByteTrack keeps them around as **second-stage candidates**: after matching tracks to high-confidence detections, unmatched tracks try to match low-confidence detections with a slightly looser IoU threshold. Recovers short occlusions, ID switches near crowds.

## SAM 2 memory-based tracking

SAM 2 handles video by keeping a **memory bank** of per-instance spatio-temporal features. Given a prompt (click, box, text) on one frame, it encodes the instance into memory. On subsequent frames, the memory is cross-attended against the new frame's features, and the decoder produces a mask for the same instance in the new frame.

No Kalman filter, no Hungarian assignment. The association is implicit in the memory-attention operation.

Pros: - Robust to large occlusions (memory carries instance identity across many frames).  
- Open-vocabulary when combined with SAM 3's text prompts. - Works without a separate motion model.

Cons: - Slower than ByteTrack for many-object tracking. - Memory bank grows; limits the context window.

## SAM 3.1 Object Multiplex

Prior SAM 2 / SAM 3 tracking keeps a separate memory bank per instance. For 50 objects, 50 memory banks. Object Multiplex (March 2026) collapses them into one shared memory with **per-instance query tokens**. Cost scales sub-linearly in number of instances.

Multiplex is the new default for crowd tracking in 2026: concert crowds, warehouse workers, traffic intersections.

## Three metrics to know

- **MOTA (Multi-Object Tracking Accuracy)** —  $1 - (FN + FP + ID \text{ switches}) / GT$ . Weighted by error type; a single metric that conflates detection and association failures.
- **IDF1 (ID F1)** — harmonic mean of ID precision and recall. Focuses specifically on how well each ground-truth track keeps its ID over time. Better than MOTA for ID-switch-sensitive tasks.
- **HOTA (Higher Order Tracking Accuracy)** — decomposes into detection accuracy (DetA) and association accuracy (AssA). The community standard since 2020; most comprehensive.

For surveillance (who is who): IDF1 is what you report. For sports analytics (counting passes): HOTA. For general academic comparison: HOTA.

**Interactive figure: cv3-track-assoc.** This one moves. Watch it animate and drag its controls in the web edition: <https://aiengineeringfromscratch.com/lesson?path=phases/04-computer-vision/27-multi-object-tracking>

## Build It

### Step 1: IoU-based cost matrix

```
import numpy as np

def bbox_iou(a, b):
    """
    a, b: (N, 4) arrays of [x1, y1, x2, y2].
    Returns (N_a, N_b) IoU matrix.
    """
    ax1, ay1, ax2, ay2 = a[:, 0], a[:, 1], a[:, 2], a[:, 3]
    bx1, by1, bx2, by2 = b[:, 0], b[:, 1], b[:, 2], b[:, 3]
    inter_x1 = np.maximum(ax1, bx1)
    inter_y1 = np.maximum(ay1, by1)
    inter_x2 = np.minimum(ax2, bx2)
    inter_y2 = np.minimum(ay2, by2)
    inter = np.clip(inter_x2 - inter_x1, 0, None) * np.clip(inter_y2 - inter_y1, 0,
    ↪ None)
    area_a = (ax2 - ax1) * (ay2 - ay1)
    area_b = (bx2 - bx1) * (by2 - by1)
    union = area_a + area_b - inter
    return inter / np.clip(union, 1e-8, None)
```

### Step 2: Minimal SORT-style tracker

Fixed constant-velocity Kalman omitted for brevity — we use a simple IoU association here; in production the Kalman predict is essential. The sort Python package provides the full version.

```
from scipy.optimize import linear_sum_assignment
```

```
class Track:
    def __init__(self, tid, bbox, frame):
```

```

        self.id = tid
        self.bbox = bbox
        self.last_frame = frame
        self.hits = 1

    def update(self, bbox, frame):
        self.bbox = bbox
        self.last_frame = frame
        self.hits += 1

class SimpleTracker:
    def __init__(self, iou_threshold=0.3, max_age=5):
        self.tracks = []
        self.next_id = 1
        self.iou_threshold = iou_threshold
        self.max_age = max_age

    def step(self, detections, frame):
        if not self.tracks:
            for d in detections:
                self.tracks.append(Track(self.next_id, d, frame))
                self.next_id += 1
            return [(t.id, t.bbox) for t in self.tracks]

        track_boxes = np.array([t.bbox for t in self.tracks])
        det_boxes = np.array(detections) if len(detections) else np.empty((0, 4))

        iou = bbox_iou(track_boxes, det_boxes) if len(det_boxes) else
        ↪ np.zeros((len(track_boxes), 0))
        cost = 1 - iou
        cost[iou < self.iou_threshold] = 1e6

        matched_track = set()
        matched_det = set()
        if cost.size > 0:
            row, col = linear_sum_assignment(cost)
            for r, c in zip(row, col):
                if cost[r, c] < 1.0:
                    self.tracks[r].update(det_boxes[c], frame)
                    matched_track.add(r); matched_det.add(c)

        for i, d in enumerate(det_boxes):
            if i not in matched_det:
                self.tracks.append(Track(self.next_id, d, frame))
                self.next_id += 1

        self.tracks = [t for t in self.tracks if frame - t.last_frame <=
        ↪ self.max_age]
        return [(t.id, t.bbox) for t in self.tracks]

```

60 lines. Takes per-frame detections, returns per-frame track IDs. Real systems add the Kalman predict, ByteTrack's second-stage re-match, and appearance features.

### Step 3: Synthetic trajectory test

```
def synthetic_frames(num_frames=20, num_objects=3, H=240, W=320, seed=0):
    rng = np.random.default_rng(seed)
    starts = rng.uniform(20, 200, size=(num_objects, 2))
    velocities = rng.uniform(-5, 5, size=(num_objects, 2))
    frames = []
    for f in range(num_frames):
        dets = []
        for i in range(num_objects):
            cx, cy = starts[i] + f * velocities[i]
            dets.append([cx - 10, cy - 10, cx + 10, cy + 10])
        frames.append(dets)
    return frames
```

```
tracker = SimpleTracker()
for f, dets in enumerate(synthetic_frames()):
    tracks = tracker.step(dets, f)
```

Three objects moving in straight lines should keep their IDs across all 20 frames.

### Step 4: ID-switch metric

```
def count_id_switches(tracks_per_frame, gt_per_frame):
    """
    tracks_per_frame: list of list of (track_id, bbox)
    gt_per_frame:      list of list of (gt_id, bbox)
    Returns number of ID switches.
    """
    prev_assignment = {}
    switches = 0
    for tracks, gts in zip(tracks_per_frame, gt_per_frame):
        if not tracks or not gts:
            continue
        t_boxes = np.array([b for _, b in tracks])
        g_boxes = np.array([b for _, b in gts])
        iou = bbox_iou(g_boxes, t_boxes)
        for g_idx, (gt_id, _) in enumerate(gts):
            j = iou[g_idx].argmax()
            if iou[g_idx, j] > 0.5:
                t_id = tracks[j][0]
                if gt_id in prev_assignment and prev_assignment[gt_id] != t_id:
                    switches += 1
                prev_assignment[gt_id] = t_id
    return switches
```

This is a simplified IDF1-adjacent metric: count how many times a ground-truth object changes its assigned predicted track ID. Real MOTA / IDF1 / HOTA tooling lives in `py-motmetrics` and `TrackEval`.

## Use It

Production trackers in 2026:

- ultralytics — YOLOv8 + ByteTrack / BoT-SORT built-in. `results = model.track(source, tracker="bytetrack.yaml")`. The default.
- supervision (Roboflow) — ByteTrack wrappers plus annotation utilities.
- SAM 2 / SAM 3.1 — memory-based tracking via `processor.track()`.
- Custom stack: detector (YOLOv8 / RT-DETR) + `sort-tracker` / OC-SORT / StrongSORT.

Picking:

- Pedestrians / cars / boxes at 30+ fps: **ByteTrack with ultralytics**.
- Many instances of one class in a crowd: **SAM 3.1 Object Multiplex**.
- Heavy occlusions with identifiable appearance: **DeepSORT / StrongSORT** (ReID features).
- Sports / complex interactions: **BoT-SORT** or learned trackers (MOTRv3).

**This chapter ships an artifact.** The course version of this lesson produces a reusable prompt or agent skill. It lives in the repository, ready to install: <https://github.com/rohitg00/ai-engineering-from-scratch/tree/main/phases/04-computer-vision/27-multi-object-tracking>

## Exercises

Starter code and the lesson’s working implementation: <https://github.com/rohitg00/ai-engineering-from-scratch/tree/main/phases/04-computer-vision/27-multi-object-tracking/code>

1. **(Easy)** Run the synthetic tracker above with 3, 10, and 30 objects. Report ID-switch count in each case. Identify where the simple IoU-only association starts to fail.
2. **(Medium)** Add a constant-velocity Kalman predict step before association. Show that short (2-3 frame) occlusions no longer cause ID switches.
3. **(Hard)** Integrate SAM 2’s memory-based tracker (via transformers) as an alternative tracker backend. Run both SimpleTracker and SAM 2 on a 30-second clip of a crowd and compare ID-switch counts, manually labelling ground-truth IDs for 5 salient people.

## Key Terms

Term	What people say	What it actually means
Tracking-by-detection	“Detect then associate”	Per-frame detector + Hungarian assignment on IoU / appearance
Kalman filter	“Motion predict”	Linear dynamics + covariance for smooth track predictions and occlusion handling
Hungarian algorithm	“Optimal assignment”	Solves the minimum-cost bipartite matching problem; <code>scipy.optimize.linear_sum_assignment</code>
ByteTrack	“Low-confidence second pass”	Re-match unmatched tracks to low-confidence detections to recover short occlusions
DeepSORT	“SORT + appearance”	Adds a ReID feature for cross-frame matching; better for ID preservation
Memory bank	“SAM 2 trick”	Per-instance spatio-temporal features stored across frames; cross-attention replaces explicit association
Object Multiplex	“SAM 3.1 shared memory”	Single shared memory with per-instance queries for fast many-object tracking



Term	What people say	What it actually means
HOTA	“Modern tracking metric”	Decomposes into detection and association accuracy; community standard

## Further Reading

- SORT (Bewley et al., 2016) — the minimal tracking-by-detection paper
- DeepSORT (Wojke et al., 2017) — adds appearance feature
- ByteTrack (Zhang et al., 2022) — low-confidence second pass
- BoT-SORT (Aharon et al., 2022) — camera motion compensation
- HOTA (Luiten et al., 2020) — decomposed tracking metric
- SAM 2 video segmentation (Meta, 2024) — memory-based tracker
- SAM 3.1 Object Multiplex (Meta, March 2026)

**Continue online.** The living edition of this chapter has more than the page can hold:

- Animated, interactive figures and the web text: <https://aiengineeringfromscratch.com/lesson?path=phases/04-computer-vision/27-multi-object-tracking>
- Runnable code for every step: <https://github.com/rohitg00/ai-engineering-from-scratch/tree/main/phases/04-computer-vision/27-multi-object-tracking/code>
- The chapter quiz, graded in the browser: <https://aiengineeringfromscratch.com/lesson?path=phases/04-computer-vision/27-multi-object-tracking>

The repository moves faster than any printing. When the book and the repo disagree, trust the repo.

# World Models & Video Diffusion

---

A video model that predicts the next seconds of a scene is a world simulator. Condition that prediction on actions and you have a learned game engine.

**Type:** Learn + Build **Languages:** Python **Prerequisites:** Phase 4 Lesson 10 (Diffusion), Phase 4 Lesson 12 (Video Understanding), Phase 4 Lesson 23 (DiT + Rectified Flow) **Time:** ~75 minutes

## Learning Objectives

- Explain the difference between a pure video generation model (Sora 2) and an action-conditioned world model (Genie 3, DreamerV3)
- Describe a video DiT: spatio-temporal patches, 3D position encoding, joint attention across (T, H, W) tokens
- Trace how a world model plugs into robotics: VLM plans → video model simulates → inverse dynamics emits actions
- Pick between Sora 2, Genie 3, Runway GWM-1 Worlds, Wan-Video, and HunyuanVideo for a given use case (creative video, interactive sim, autonomous-driving synthesis)

## The Problem

Video generation and world modelling converged in 2026. A model that can generate a coherent minute of video has, in some sense, learned how the world moves: object permanence, gravity, causality, style. If you condition that prediction on actions (walk left, open the door), the video model becomes a learnable simulator that can replace a game engine, a driving simulator, or a robotics environment.

The stakes are concrete. Genie 3 generates playable environments from a single image. Runway GWM-1 Worlds synthesises infinite explorable scenes. Sora 2 produces minute-long videos with synchronised audio and modelled physics. NVIDIA Cosmos-Drive, Wayve Gaia-2, and Tesla DrivingWorld generate realistic driving video for autonomous-vehicle training data. The world-model paradigm is quietly taking over sim-to-real for robotics.

This lesson is the “big picture” lesson for Phase 4. It connects image generation, video understanding, and agentic reasoning into the architecture pattern dominant research is moving toward.

## The Concept

### Three families of world-modelling

**Diagram.** Rendered live in the web edition: <https://aiengineeringfromscratch.com/lesson?path=phases/04-computer-vision/28-world-models-video-diffusion>

- **Sora 2** is pure video generation conditioned on prompts. No action interface. You cannot “steer” it mid-rollout.

- **Genie 3, GWM-1 Worlds, Mirage / Magica** are action-conditioned world models. Infer latent actions from observed video, then condition future frame predictions on actions. Interactive — you press keys or move a camera and the scene responds.
- **DreamerV3** and the classic RL world-model family predict in a latent space with explicit action conditioning, trained on a reward signal. Less visual; more useful for sample-efficient RL.

## Video DiT architecture

Video latent: (C, T, H, W)  
 Patchify (spatial): grid of  $P_h \times P_w$  patches per frame  
 Patchify (temporal): group  $P_t$  frames into a temporal patch  
 Resulting tokens:  $(T / P_t) * (H / P_h) * (W / P_w)$  tokens

Positional encoding is 3D: a rotary or learned embedding per (t, h, w) coordinate. Attention can be:

- **Full joint** — all tokens attend to all tokens.  $O(N^2)$  with N tokens. Prohibitive for long videos.
- **Divided** — alternate temporal attention (same spatial position, across time:  $(H*W) * T^2$ ) and spatial attention (same timestep, across space:  $T * (H*W)^2$ ). Used by TimeSformer and most video DiTs.
- **Window** — local windows in (t, h, w). Used by Video Swin.

Every 2026 video diffusion model uses one of these three patterns plus AdaLN conditioning (Lesson 23) and rectified flow.

## Conditioning on actions: latent action models

Genie learns a **latent action** per frame by discriminatively predicting the action between a pair of consecutive frames. The model's decoder then conditions on the inferred latent action — not on explicit keyboard keys. At inference, a user can specify a latent action (or sample one from a fresh prior) and the model generates the next frame consistent with that action.

Sora skips the action interface entirely. Its decoder predicts next spacetime tokens from past spacetime tokens. Prompt conditions the start; nothing steers it mid-generation.

## Physical plausibility

Sora 2's 2026 release explicitly advertised **physical plausibility**: weight, balance, object permanence, cause-and-effect. Measured by the team via hand-rated plausibility scores; the model visibly improves on dropped objects, characters colliding, and failures-on-purpose (a missed jump) versus Sora 1.

Plausibility remains the dominant failure mode. 2024-2025 videos of people eating spaghetti or drinking from glasses revealed the model's lack of persistent object representation. 2026 models (Sora 2, Runway Gen-5, HunyuanVideo) reduce but do not eliminate these.

## Autonomous driving world models

Driving world models generate realistic road scenes conditioned on trajectories, bounding boxes, or navigation maps. Usage:

- **Cosmos-Drive-Dreams** (NVIDIA) — generates minutes of driving video for RL training.
- **Gaia-2** (Wayve) — trajectory-conditioned scene synthesis for policy evaluation.
- **DrivingWorld** (Tesla) — simulates varied weather, time-of-day, traffic conditions.

- **Vista** (ByteDance) — reactive driving scene synthesis.

They replace expensive real-world data collection for corner cases — pedestrian jaywalks at night, icy intersections, unusual vehicle types — that would otherwise require millions of miles of driving.

## Robotics stack: VLM + video model + inverse dynamics

The emerging three-component robotics loop:

1. **VLM** parses the goal (“pick up the red cup”), plans a high-level action sequence.
2. **Video generation model** simulates what executing each action would look like — predicts observations  $N$  frames ahead.
3. **Inverse dynamics model** extracts the concrete motor commands that would produce those observations.

This replaces reward shaping and sample-heavy RL. The world model does the imagination; the inverse dynamics closes the loop on actuation. Genie Envisioner is one instantiation; many research groups are converging on this structure.

## Evaluation

- **Visual quality** — FVD (Fréchet Video Distance), user studies.
- **Prompt alignment** — CLIPScore per frame, VQA-style evaluation.
- **Physical plausibility** — hand-rated on a benchmark suite (Sora 2’s internal benchmark, VBench).
- **Controllability** (for interactive world models) — action → observation consistency; can you go back to a prior state?

## Model landscape in 2026

Model	Use	Parameters	Output	License
Sora 2	text-to-video, audio	—	1-min 1080p + audio	API only
Runway Gen-5	text/image-to-video	—	10s clips	API
Runway GWM-1 Worlds	interactive world	—	infinite 3D rollout	API
Genie 3	interactive world from image	11B+	playable frames	research preview
Wan-Video 2.1	open text-to-video	14B	high-quality clips	non-commercial
HunyuanVideo	open text-to-video	13B	10s clips	permissive
Cosmos / Cosmos-Drive	autonomous driving sim	7-14B	driving scenes	NVIDIA open

Model	Use	Parameters	Output	License
Magica / Mirage 2	AI-native game engine	—	modifiable worlds	product

**Interactive figure: v4-world-rollout.** This one moves. Watch it animate and drag its controls in the web edition: <https://aiengineeringfromscratch.com/lesson?path=phases/04-computer-vision/28-world-models-video-diffusion>

## Build It

### Step 1: 3D patchify for video

```
import torch
import torch.nn as nn

class VideoPatch3D(nn.Module):
    def __init__(self, in_channels=4, dim=64, patch_t=2, patch_h=2, patch_w=2):
        super().__init__()
        self.proj = nn.Conv3d(
            in_channels, dim,
            kernel_size=(patch_t, patch_h, patch_w),
            stride=(patch_t, patch_h, patch_w),
        )
        self.patch_t = patch_t
        self.patch_h = patch_h
        self.patch_w = patch_w

    def forward(self, x):
        # x: (N, C, T, H, W)
        x = self.proj(x)
        n, c, t, h, w = x.shape
        tokens = x.reshape(n, c, t * h * w).transpose(1, 2)
        return tokens, (t, h, w)
```

A 3D conv with stride equal to kernel acts as the spatio-temporal patchifier. (T, H, W) -> (T/2, H/2, W/2) grid of tokens.

### Step 2: 3D rotary position encoding

Rotary Position Embeddings (RoPE) separately applied along t, h, w axes:

```
def rope_3d(tokens, t_dim, h_dim, w_dim, grid):
    """
    tokens: (N, T*H*W, D)
    grid: (T, H, W) sizes
    t_dim + h_dim + w_dim == D
    """
    T, H, W = grid
    n, seq, d = tokens.shape
    if t_dim + h_dim + w_dim != d:
```

```

        raise ValueError(f"t_dim+h_dim+w_dim ({t_dim}+{h_dim}+{w_dim}) must equal
        ↪ D={d}")
    assert seq == T * H * W
    t_idx = torch.arange(T, device=tokens.device).repeat_interleave(H * W)
    h_idx = torch.arange(H, device=tokens.device).repeat_interleave(W).repeat(T)
    w_idx = torch.arange(W, device=tokens.device).repeat(T * H)
    # Simplified: just scale channels by frequencies. Real RoPE rotates pairs.
    freqs_t = torch.exp(-torch.log(torch.tensor(10000.0)) * torch.arange(t_dim //
    ↪ 2, device=tokens.device) / (t_dim // 2))
    freqs_h = torch.exp(-torch.log(torch.tensor(10000.0)) * torch.arange(h_dim //
    ↪ 2, device=tokens.device) / (h_dim // 2))
    freqs_w = torch.exp(-torch.log(torch.tensor(10000.0)) * torch.arange(w_dim //
    ↪ 2, device=tokens.device) / (w_dim // 2))
    emb_t = torch.cat([torch.sin(t_idx[:, None] * freqs_t), torch.cos(t_idx[:,
    ↪ None] * freqs_t)], dim=-1)
    emb_h = torch.cat([torch.sin(h_idx[:, None] * freqs_h), torch.cos(h_idx[:,
    ↪ None] * freqs_h)], dim=-1)
    emb_w = torch.cat([torch.sin(w_idx[:, None] * freqs_w), torch.cos(w_idx[:,
    ↪ None] * freqs_w)], dim=-1)
    return tokens + torch.cat([emb_t, emb_h, emb_w], dim=-1)

```

Simplified additive form. Real RoPE rotates paired channels at frequencies; the positional information is the same.

### Step 3: Divided attention block

```

class DividedAttentionBlock(nn.Module):
    def __init__(self, dim=64, heads=2):
        super().__init__()
        self.time_attn = nn.MultiheadAttention(dim, heads, batch_first=True)
        self.space_attn = nn.MultiheadAttention(dim, heads, batch_first=True)
        self.ln1 = nn.LayerNorm(dim)
        self.ln2 = nn.LayerNorm(dim)
        self.ln3 = nn.LayerNorm(dim)
        self.mlp = nn.Sequential(nn.Linear(dim, 4 * dim), nn.GELU(), nn.Linear(4 *
        ↪ dim, dim))

    def forward(self, x, grid):
        T, H, W = grid
        n, seq, d = x.shape
        # time attention: same (h, w), across t
        xt = x.view(n, T, H * W, d).permute(0, 2, 1, 3).reshape(n * H * W, T, d)
        a, _ = self.time_attn(self.ln1(xt), self.ln1(xt), self.ln1(xt),
        ↪ need_weights=False)
        xt = (xt + a).reshape(n, H * W, T, d).permute(0, 2, 1, 3).reshape(n, seq, d)
        # space attention: same t, across (h, w)
        xs = xt.view(n, T, H * W, d).reshape(n * T, H * W, d)
        a, _ = self.space_attn(self.ln2(xs), self.ln2(xs), self.ln2(xs),
        ↪ need_weights=False)
        xs = (xs + a).reshape(n, T, H * W, d).reshape(n, seq, d)
        xs = xs + self.mlp(self.ln3(xs))
        return xs

```

The time attention attends within each spatial position across time; the space attention attends within each frame across positions. Two  $O(T^2 + (HW)^2)$  operations instead of one

$O((THW)^2)$ . This is the core of TimeSformer and every modern video DiT.

## Step 4: Compose a tiny video DiT

```
class TinyVideoDiT(nn.Module):
    def __init__(self, in_channels=4, dim=64, depth=2, heads=2):
        super().__init__()
        self.patch = VideoPatch3D(in_channels=in_channels, dim=dim, patch_t=2,
            ↪ patch_h=2, patch_w=2)
        self.blocks = nn.ModuleList([DividedAttentionBlock(dim, heads) for _ in
            ↪ range(depth)])
        self.out = nn.Linear(dim, in_channels * 2 * 2 * 2)

    def forward(self, x):
        tokens, grid = self.patch(x)
        for blk in self.blocks:
            tokens = blk(tokens, grid)
        return self.out(tokens), grid
```

Not a working video generator; a structural demo that every piece shapes correctly.

## Step 5: Check shapes

```
vid = torch.randn(1, 4, 8, 16, 16) # (N, C, T, H, W)
model = TinyVideoDiT()
out, grid = model(vid)
print(f"input {tuple(vid.shape)}")
print(f"tokens grid {grid}")
print(f"output {tuple(out.shape)}")
```

Expect grid = (4, 8, 8) and out = (1, 256, 32) after patching; the head then projects to per-token spatio-temporal patches, ready to be un-patchified back into a video.

## Use It

Production access patterns for 2026:

- **Sora 2 API** (OpenAI) — text-to-video, synchronized audio. Premium pricing.
- **Runway Gen-5 / GWM-1** (Runway) — image-to-video, interactive worlds.
- **Wan-Video 2.1 / HunyuanVideo** — open-source self-host.
- **Cosmos / Cosmos-Drive** (NVIDIA) — driving simulation open weights.
- **Genie 3** — research preview, request access.

For building an interactive world-model demo: start with Wan-Video for quality, layer on a latent-action adapter for interactivity. For autonomous driving simulation: Cosmos-Drive is the 2026 open reference.

For robotics, the stack in the wild:

1. Language goal -> VLM (Qwen3-VL) -> high-level plan.
2. Plan -> latent-action video model -> imagined rollout.
3. Rollout -> inverse dynamics model -> low-level actions.
4. Actions executed -> observation fed back into step 1.

**This chapter ships an artifact.** The course version of this lesson produces a reusable prompt or agent skill. It lives in the repository, ready to install: <https://github.com/rohitg00/ai->

engineering-from-scratch/tree/main/phases/04-computer-vision/28-world-models-video-diffusion

## Exercises

Starter code and the lesson’s working implementation: <https://github.com/rohitg00/ai-engineering-from-scratch/tree/main/phases/04-computer-vision/28-world-models-video-diffusion/code>

1. **(Easy)** Compute the token count for a 5-second 360p video at patch-t=2, patch-h=8, patch-w=8. Reason about memory for attention at this size.
2. **(Medium)** Swap the divided attention block above for a full joint attention block and measure the shape and parameter count. Explain why divided attention is necessary for real video models.
3. **(Hard)** Build a minimal latent-action video model: take a dataset of (frame_t, action_t, frame_{t+1}) triples (any simple 2D game), train a tiny video DiT conditioned on action embeddings, and show that different actions produce different next frames.

## Key Terms

Term	What people say	What it actually means
World model	“Learned simulator”	A model that predicts future observations given state and action
Video DiT	“Spacetime transformer”	Diffusion transformer with 3D patchification and divided attention
Latent action	“Inferred control”	Discrete or continuous action latent inferred from frame pairs; used to condition next-frame generation
Divided attention	“Time then space”	Two attention operations per block — across time then across space — to keep $O(N^2)$ manageable
Object permanence	“Things stay real”	Scene property that video models must learn; classic failure mode on food, glassware
FVD	“Fréchet Video Distance”	Video equivalent of FID; primary visual quality metric
Inverse dynamics model	“Observations to actions”	Given (state, next state), output the action that connects them; closes robotics loop
Cosmos-Drive	“NVIDIA driving sim”	Open-weights autonomous-driving world model for RL and evaluation

## Further Reading

- Sora technical report (OpenAI)
- Genie: Generative Interactive Environments (Bruce et al., 2024) — latent action world models
- TimeSformer (Bertasius et al., 2021) — divided attention for video transformers
- DreamerV3 (Hafner et al., 2023) — world models for RL



- Cosmos-Drive-Dreams (NVIDIA, 2025) — driving world model
- Top 10 Video Generation Models 2026 (DataCamp)
- From Video Generation to World Model — survey repo

**Continue online.** The living edition of this chapter has more than the page can hold:

- Animated, interactive figures and the web text: <https://aiengineeringfromscratch.com/lesson?path=phases/04-computer-vision/28-world-models-video-diffusion>
- Runnable code for every step: <https://github.com/rohitg00/ai-engineering-from-scratch/tree/main/phases/04-computer-vision/28-world-models-video-diffusion/code>
- The chapter quiz, graded in the browser: <https://aiengineeringfromscratch.com/lesson?path=phases/04-computer-vision/28-world-models-video-diffusion>

The repository moves faster than any printing. When the book and the repo disagree, trust the repo.

## Part III — Speech & Audio

---

*Course phase 06. Live edition with animated figures and quizzes: <https://aiengineeringfromscratch.com/catalog.html>*

# Audio Fundamentals — Waveforms, Sampling, Fourier Transform

---

Waveforms are the raw signal. Spectrograms are the representation. Mel features are the ML-friendly form. Every modern ASR and TTS pipeline walks this ladder, and the first rung is understanding sampling and Fourier.

**Type:** Learn **Languages:** Python **Prerequisites:** Phase 1 · 06 (Vectors & Matrices), Phase 1 · 14 (Probability Distributions) **Time:** ~45 minutes

## The Problem

A microphone produces a pressure-vs-time signal. Your neural net consumes tensors. Between them sits a stack of conventions that, when violated, produce silent bugs: the model trains fine but the WER doubles, or TTS ships a hiss, or a voice cloning system memorizes the microphone instead of the speaker.

Every bug in speech systems traces back to one of three questions:

1. What sample rate was the data recorded at, and what does the model expect?
2. Is the signal aliased?
3. Are you operating on raw samples or on a frequency representation?

Get these right and the rest of Phase 6 is tractable. Get them wrong and even Whisper-Large-v4 produces garbage.

## The Concept

**Waveform.** A one-dimensional array of floats in  $[-1.0, 1.0]$ . Indexed by sample number. To convert to seconds, divide by the sample rate:  $t = n / sr$ . A 10-second clip at 16 kHz is an array of 160,000 floats.

**Sampling rate (sr).** How many samples per second. Common rates in 2026:

Rate	Use
8 kHz	Telephony, legacy VOIP. Nyquist at 4 kHz kills consonants. Avoid for ASR.
16 kHz	ASR standard. Whisper, Parakeet, SeamlessM4T v2 all consume 16 kHz.
22.05 kHz	TTS vocoder training for older models.
24 kHz	Modern TTS (Kokoro, F5-TTS, xTTS v2).
44.1 kHz	CD audio, music.
48 kHz	Film, pro audio, high-fidelity TTS (VALL-E 2, NaturalSpeech 3).

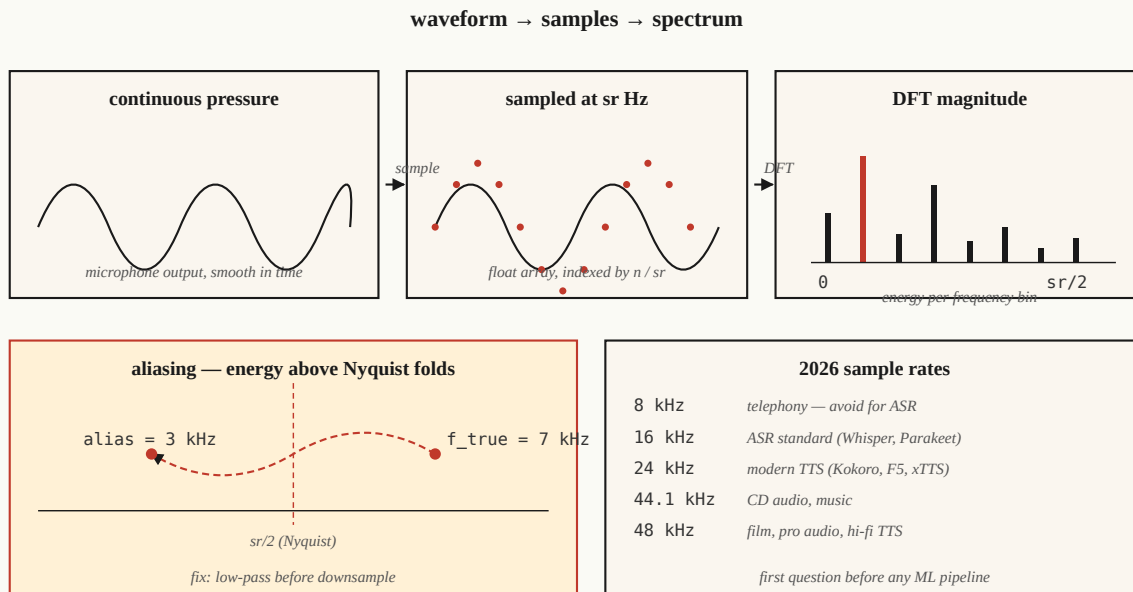


Figure 1: Waveform, sampling, DFT, and frequency bins visualized

**Nyquist-Shannon.** A sample rate of  $sr$  can unambiguously represent frequencies up to  $sr/2$ . The  $sr/2$  boundary is the *Nyquist frequency*. Energy above Nyquist gets *aliased* — folded down into lower frequencies — and corrupts the signal. Always low-pass filter before downsampling.

**Bit depth.** 16-bit PCM (signed int16, range  $\pm 32,767$ ) is the universal exchange format. 24-bit for music, 32-bit float for internal DSP. Libraries like soundfile read int16 but expose float32 arrays in  $[-1, 1]$ .

**Fourier Transform.** Any finite signal is a sum of sinusoids at different frequencies. The Discrete Fourier Transform (DFT) computes, for  $N$  samples,  $N$  complex coefficients — one per frequency bin. bin  $k$  maps to frequency  $k \cdot sr / N$  Hz. Magnitude is amplitude at that frequency, angle is phase.

**FFT.** Fast Fourier Transform: an  $O(N \log N)$  algorithm for the DFT when  $N$  is a power of 2. Every audio library uses FFT under the hood. A 1024-sample FFT at 16 kHz gives 512 usable frequency bins spanning 0–8 kHz at 15.6 Hz resolution.

**Framing + window.** We do not FFT an entire clip. We chop it into overlapping *frames* (typically 25 ms with 10 ms hop), multiply each frame by a window function (Hann, Hamming) to kill edge discontinuities, then FFT each frame. This is the Short-Time Fourier Transform (STFT). Lesson 02 picks up from here.

**Interactive figure: mel-scale.** This one moves. Watch it animate and drag its controls in the web edition: <https://aiengineeringfromscratch.com/lesson?path=phases/06-speech-and-audio/01-audio-fundamentals>

## Build It

### Step 1: read a clip and plot the waveform

code/main.py uses only the stdlib wave module to keep the demo dependency-free. For production you will use soundfile or torchaudio.load (both return (waveform, sr) tuples):

```
import soundfile as sf
waveform, sr = sf.read("clip.wav", dtype="float32") # shape (T,), sr=int
```

### Step 2: synthesize a sine wave from first principles

```
import math

def sine(freq_hz, sr, seconds, amp=0.5):
    n = int(sr * seconds)
    return [amp * math.sin(2 * math.pi * freq_hz * i / sr) for i in range(n)]
```

A 440 Hz sine (concert A) at 16 kHz for 1 second is 16,000 floats. Write with wave.open(..., "wb") using 16-bit PCM encoding.

### Step 3: compute the DFT by hand

```
def dft(x):
    N = len(x)
    out = []
    for k in range(N):
        re = sum(x[n] * math.cos(-2 * math.pi * k * n / N) for n in range(N))
        im = sum(x[n] * math.sin(-2 * math.pi * k * n / N) for n in range(N))
        out.append((re, im))
    return out
```

$O(N^2)$  — fine for  $N=256$  to confirm correctness, useless for real audio. Real code calls `numpy.fft.rfft` or `torch.fft.rfft`.

### Step 4: find the dominant frequency

Magnitude peak index `k_star` maps to frequency `k_star * sr / N`. Running this on the 440 Hz sine should return a peak at `bin 440 * N / sr`.

### Step 5: demonstrate aliasing

Sample a 7 kHz sine at 10 kHz (Nyquist = 5 kHz). The 7 kHz tone is above Nyquist and folds to  $10 - 7 = 3$  kHz. The FFT peak appears at 3 kHz. This is the classic aliasing demo and the reason every DAC/ADC ships with a brick-wall low-pass filter.

## Use It

The stack you will actually ship in 2026:

Task	Library	Why
Read/write WAV/FLAC/OGG	soundfile (libsndfile wrapper)	Fastest, stable, returns float32.
Resample	torchaudio.transforms.Resample or librosa.resample	Correct anti-aliasing built in.
STFT / Mel	torchaudio or librosa	GPU-friendly; PyTorch ecosystem.
Real-time streaming	sounddevice or pyaudio	Cross-platform PortAudio bindings.
Inspect a file	ffprobe or soxi	CLI, fast, reports sr/channels/codec.

Decision rule: **match sample rate before you match anything else**. Whisper expects 16 kHz mono float32. Pass it 44.1 kHz stereo and you will get garbage that looks like a model bug.

**This chapter ships an artifact.** The course version of this lesson produces a reusable prompt or agent skill. It lives in the repository, ready to install: <https://github.com/rohitg00/ai-engineering-from-scratch/tree/main/phases/06-speech-and-audio/01-audio-fundamentals>

## Exercises

Starter code and the lesson’s working implementation: <https://github.com/rohitg00/ai-engineering-from-scratch/tree/main/phases/06-speech-and-audio/01-audio-fundamentals/code>

1. **Easy.** Synthesize a 1-second mix of 220 Hz + 440 Hz + 880 Hz at 16 kHz. Run DFT. Confirm three peaks at the expected bins.
2. **Medium.** Record a 3-second WAV of your voice at 48 kHz. Downsample to 16 kHz using `torchaudio.transforms.Resample` (with anti-aliasing), then to 16 kHz using naive decimation (every third sample). FFT both. Where does the aliasing appear?
3. **Hard.** Build the STFT from scratch using only math and the DFT from Step 3. Frame size 400, hop 160, Hann window. Plot magnitudes with `matplotlib.pyplot.imshow`. This is the spectrogram of Lesson 02.

## Key Terms

Term	What people say	What it actually means
Sample rate	How many samples per second	Frequency in Hz at which the ADC measures the signal.
Nyquist	The max frequency you can represent	$sr/2$ ; energy above it aliases back down.
Bit depth	Resolution of each sample	<code>int16</code> = 65,536 levels; <code>float32</code> = 24-bit precision in $[-1, 1]$ .
DFT	The Fourier transform for sequences	$N$ samples $\rightarrow N$ complex frequency coefficients.
FFT	The fast DFT	$O(N \log N)$ algorithm requiring $N$ = power of 2.
Bin	Frequency column	$k \cdot sr / N$ Hz; resolution = $sr / N$ .
STFT	Spectrogram under the hood	Framed + windowed FFT over time.

Term	What people say	What it actually means
Aliasing	Weird frequency ghosts	Energy above Nyquist mirroring down to lower bins.

## Further Reading

- Shannon (1949). Communication in the Presence of Noise — the paper behind the sampling theorem.
- Smith — The Scientist and Engineer’s Guide to Digital Signal Processing — free, canonical DSP textbook.
- librosa docs — audio primer — practical walkthrough with code.
- Heinrich Kuttruff — Room Acoustics (6th ed.) — reference for why real-world audio is not a clean sinusoid.
- Steve Eddins — FFT Interpretation notebook — frequency bin intuition cleared up in 10 minutes.

**Continue online.** The living edition of this chapter has more than the page can hold:

- Animated, interactive figures and the web text: <https://aiengineeringfromscratch.com/lesson?path=phases/06-speech-and-audio/01-audio-fundamentals>
- Runnable code for every step: <https://github.com/rohitg00/ai-engineering-from-scratch/tree/main/phases/06-speech-and-audio/01-audio-fundamentals/code>

The repository moves faster than any printing. When the book and the repo disagree, trust the repo.

# Spectrograms, Mel Scale & Audio Features

---

Neural nets do not consume raw waveforms well. They consume spectrograms. They consume mel spectrograms even better. Every ASR, TTS, and audio classifier in 2026 lives or dies by this single preprocessing choice.

**Type:** Build **Languages:** Python **Prerequisites:** Phase 6 · 01 (Audio Fundamentals) **Time:** ~45 minutes

## The Problem

Take a 10-second 16 kHz clip. That is 160,000 floats, all in  $[-1, 1]$ , almost perfectly uncorrelated with the label “dog barking” or “the word cat”. The raw waveform has the information but in a form the model cannot easily extract. Two identical phonemes spoken 100 ms apart have completely different raw samples.

A spectrogram fixes this. It collapses the temporal detail where human perception ignores it (microsecond jitter) and preserves the structure where perception attends (which frequencies are energetic, over time windows of  $\sim 10$ –25 ms).

Mel spectrograms push further. Humans perceive pitch logarithmically: 100 Hz vs 200 Hz sounds “the same distance apart” as 1000 Hz vs 2000 Hz. The mel scale warps the frequency axis to match. A mel-scaled spectrogram is the single most important feature in speech ML from 2010 through 2026.

## The Concept

**STFT (Short-Time Fourier Transform).** Slice the waveform into overlapping frames (typical: 25 ms window, 10 ms hop = 400 samples / 160 samples at 16 kHz). Multiply each frame by a window function (Hann is the default; Hamming slightly different tradeoff). FFT each frame. Stack the magnitude spectra into a matrix of shape  $(n_frames, n_freq_bins)$ . That is your spectrogram.

**Log-magnitude.** Raw magnitudes span 5-6 orders of magnitude. Take  $\log(|X| + 1e-6)$  or  $20 * \log_{10}(|X|)$  to compress dynamic range. Every production pipeline uses log-magnitude, not raw magnitude.

**Mel scale.** Frequency  $f$  in Hz maps to mel  $m$  by  $m = 2595 * \log_{10}(1 + f / 700)$ . The mapping is roughly linear below 1 kHz and roughly logarithmic above. 80 mel bins covering 0–8 kHz is the standard ASR input.

**Mel filterbank.** A set of triangular filters spaced equally on the mel scale. Each filter is a weighted sum of adjacent FFT bins. Multiplying the STFT magnitude by the filterbank matrix gives the mel spectrogram in one matmul.

**Log-mel spectrogram.**  $\log(mel_spec + 1e-10)$ . Whisper’s input. Parakeet’s input. SeamlessM4T’s input. The universal 2026 audio frontend.



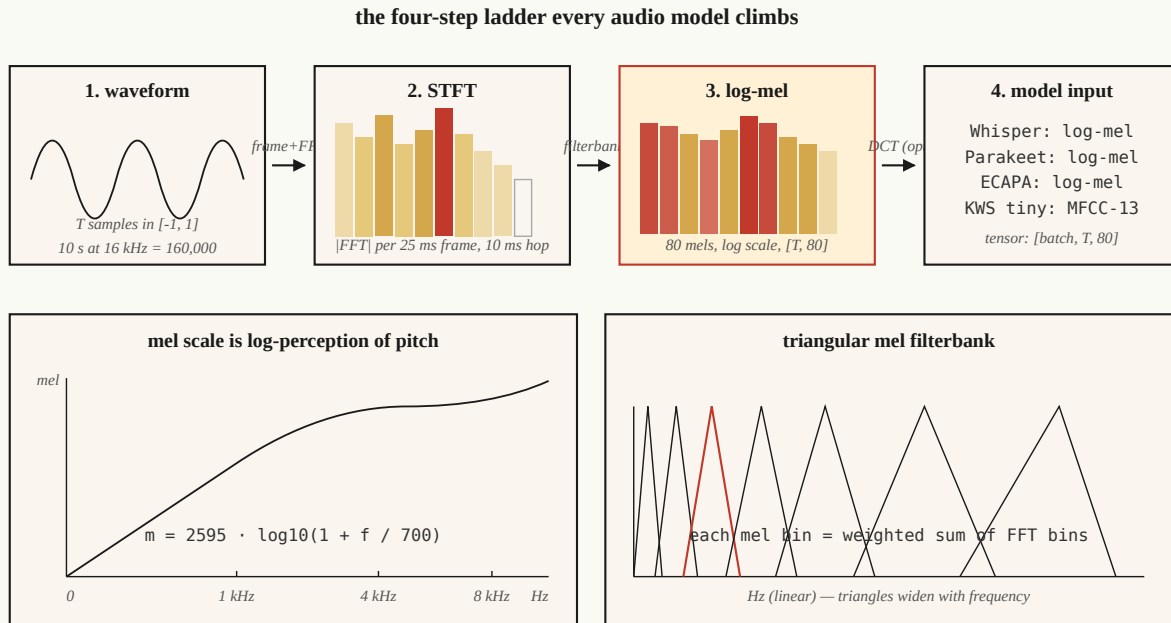


Figure 2: Waveform to STFT to mel spectrogram to MFCC ladder

**MFCCs.** Take the log-mel spectrogram, apply a DCT (type II), keep the first 13 coefficients. Decorrelates the features and compresses further. Dominant feature until about 2015 when CNNs/Transformers on raw log-mels caught up. Still used in speaker recognition (x-vectors, ECAPA).

**Resolution trade.** Larger FFT = better frequency resolution but worse time resolution. 25 ms / 10 ms is the audio-ML default; 50 ms / 12.5 ms for music; 5 ms / 2 ms for transient detection (drum hits, plosives).

**Interactive figure: spectrogram-window.** This one moves. Watch it animate and drag its controls in the web edition: <https://aiengineeringfromscratch.com/lesson?path=phases/06-speech-and-audio/02-spectrograms-mel-features>

## Build It

### Step 1: frame the waveform

```
def frame(signal, frame_len, hop):
    n = 1 + (len(signal) - frame_len) // hop
    return [signal[i * hop : i * hop + frame_len] for i in range(n)]
```

A 10-second 16 kHz clip with frame_len=400, hop=160 yields 998 frames.

### Step 2: Hann window

```
import math

def hann(N):
    return [0.5 * (1 - math.cos(2 * math.pi * n / (N - 1))) for n in range(N)]
```

Multiply element-wise before the FFT. Removes spectral leakage caused by truncating at non-zero endpoints.

### Step 3: STFT magnitude

```
def stft_magnitude(signal, frame_len=400, hop=160):
    win = hann(frame_len)
    frames = frame(signal, frame_len, hop)
    return [magnitudes(dft([w * s for w, s in zip(win, f)])) for f in frames]
```

Production uses `torch.stft` or `librosa.stft` (FFT-backed, vectorized). The loop here is pedagogical; it runs on short clips in `code/main.py`.

### Step 4: mel filterbank

```
def hz_to_mel(f):
    return 2595.0 * math.log10(1.0 + f / 700.0)

def mel_to_hz(m):
    return 700.0 * (10 ** (m / 2595.0) - 1)

def mel_filterbank(n_mels, n_fft, sr, fmin=0, fmax=None):
    fmax = fmax or sr / 2
    mels = [hz_to_mel(fmin) + (hz_to_mel(fmax) - hz_to_mel(fmin)) * i / (n_mels + 1)
            for i in range(n_mels + 2)]
    hzs = [mel_to_hz(m) for m in mels]
    bins = [int(h * n_fft / sr) for h in hzs]
    fb = [[0.0] * (n_fft // 2 + 1) for _ in range(n_mels)]
    for m in range(n_mels):
        for k in range(bins[m], bins[m + 1]):
            fb[m][k] = (k - bins[m]) / max(1, bins[m + 1] - bins[m])
        for k in range(bins[m + 1], bins[m + 2]):
            fb[m][k] = (bins[m + 2] - k) / max(1, bins[m + 2] - bins[m + 1])
    return fb
```

80 mels covering 0–8 kHz with `n_fft=400` gives an (80, 201) matrix. Multiply the (`n_frames`, 201) STFT magnitude by the transpose to get (`n_frames`, 80) mel spectrogram.

### Step 5: log-mel

```
def log_mel(mel_spec, eps=1e-10):
    return [[math.log(max(v, eps)) for v in frame] for frame in mel_spec]
```

Common alternatives: `librosa.power_to_db` (reference-normalized dB), `10 * log10(power + eps)`. Whisper uses a more involved clip + normalize routine (see Whisper’s `log_mel_spectrogram`).

### Step 6: MFCCs

```
def dct_ii(x, n_coeffs):
    N = len(x)
    return [
        sum(x[n] * math.cos(math.pi * k * (2 * n + 1) / (2 * N))) for n in range(N))
```

```
    for k in range(n_coeffs)
]
```

Apply DCT to each log-mel frame, keep the first 13 coefficients. That is your MFCC matrix. The first coefficient is usually dropped (it encodes overall energy).

## Use It

The 2026 stack:

Task	Features
ASR (Whisper, Parakeet, SeamlessM4T)	80 log-mels, 10 ms hop, 25 ms window
TTS acoustic model (VITS, F5-TTS, Kokoro)	80 mels, 5–12 ms hop for fine temporal control
Audio classification (AST, PANNs, BEATs)	128 log-mels, 10 ms hop
Speaker embedding (ECAPA-TDNN, WavLM)	80 log-mels or raw-waveform SSL
Music (MusicGen, Stable Audio 2)	EnCodec discrete tokens (not mels)
Keyword spotting	40 MFCCs for tiny devices

Rule of thumb: **if you are not working on music, start with 80 log-mels**. The burden of proof is on any deviation.

## Pitfalls that still ship in 2026

- **Mel count mismatch.** Training with 80 mels, inference with 128 mels. Silent failure. Log the feature shape at both ends.
- **Sample-rate mismatch upstream.** Mels computed at 22.05 kHz look different from 16 kHz. Fix SR *before* featurization.
- **dB vs log.** Whisper expects log-mel, not dB-mel. Some HF pipelines autodetect; your custom code will not.
- **Normalization drift.** Per-utterance normalization during training, global normalization during inference. Production bug that doubles WER.
- **Leakage from padding.** Zero-padding the end of a clip produces a flat spectrum in the trailing frames. Pad symmetrically or replicate.

**This chapter ships an artifact.** The course version of this lesson produces a reusable prompt or agent skill. It lives in the repository, ready to install: <https://github.com/rohitg00/ai-engineering-from-scratch/tree/main/phases/06-speech-and-audio/02-spectrograms-mel-features>

## Exercises

Starter code and the lesson’s working implementation: <https://github.com/rohitg00/ai-engineering-from-scratch/tree/main/phases/06-speech-and-audio/02-spectrograms-mel-features/code>

1. **Easy.** Run `code/main.py`. It synthesizes a chirp (frequency swept  $200 \rightarrow 4000$  Hz) and prints the `argmax` mel bin per frame. Plot (optional) and confirm it matches the sweep.
2. **Medium.** Re-run with `n_mels` in  $\{40, 80, 128\}$  and `frame_len` in  $\{200, 400, 800\}$ . Measure sharp-peak bandwidth across the time axis. Which combo resolves the chirp the best?
3. **Hard.** Implement `power_to_db` and compare ASR accuracy of a tiny CNN classifier on AudioMNIST using (a) raw log-mel, (b) dB-mel with `ref=max`, (c) MFCC-13 + `delta` + `delta-delta`. Report top-1 accuracy.

## Key Terms

Term	What people say	What it actually means
Frame	A slice	25 ms chunk of waveform fed to one FFT.
Hop	Stride	Samples between consecutive frames; 10 ms is ASR default.
Window	Hann/Hamming thing	Point-wise multiplier that tapers the frame edges to zero.
STFT	Spectrogram generator	Framed + windowed FFT; yields time $\times$ frequency matrix.
Mel	Warped frequency	Log-perception scale; $m = 2595 \cdot \log_{10}(1 + f/700)$ .
Filterbank	The matrix	Triangular filters that project STFT onto mel bins.
Log-mel	Whisper's input	$\log(\text{mel_spec} + \text{eps})$ ; standardized in 2026.
MFCC	Old-school feature	DCT of log-mel; 13 coeffs, decorrelated.

## Further Reading

- Davis, Mermelstein (1980). Comparison of parametric representations for monosyllabic word recognition — the MFCC paper.
- Stevens, Volkman, Newman (1937). A Scale for the Measurement of the Psychological Magnitude Pitch — the original mel scale.
- OpenAI — Whisper source, `log_mel_spectrogram` — read the reference implementation.
- librosa feature extraction docs — reference for `mfcc`, `melspectrogram`, and `hop/window`.
- NVIDIA NeMo — audio preprocessing — production-scale pipeline for Parakeet + Canary models.

**Continue online.** The living edition of this chapter has more than the page can hold:

- Animated, interactive figures and the web text: <https://aiengineeringfromscratch.com/lesson?path=phases/06-speech-and-audio/02-spectrograms-mel-features>
- Runnable code for every step: <https://github.com/rohitg00/ai-engineering-from-scratch/tree/main/phases/06-speech-and-audio/02-spectrograms-mel-features/code>

The repository moves faster than any printing. When the book and the repo disagree, trust the repo.

# Audio Classification — From k-NN on MFCCs to AST and BEATs

---

Everything from “dog barking vs siren” to “which language is this” is audio classification. The features are mels. The architecture moves each decade. The evaluation stays AUC, F1, and per-class recall.

**Type:** Build **Languages:** Python **Prerequisites:** Phase 6 · 02 (Spectrograms & Mel), Phase 3 · 06 (CNNs), Phase 5 · 08 (CNNs & RNNs for Text) **Time:** ~75 minutes

## The Problem

You get a 10-second clip. You want to know: “what is it?” Urban sound (siren, drill, dog), speech command (yes/no/stop), language ID (en/es/ar), speaker emotion (angry/neutral), or environmental sound (indoor/outdoor, babble). All of these are *audio classification*, and in 2026 the baseline architecture is mature: log-mel → CNN or Transformer → softmax.

The core difficulty is not the network. It is data. Audio datasets have brutal class imbalance, strong domain shift (clean vs noisy), and label noise (who decided “urban babble” vs “restaurant noise”?). The 80% of the problem is curation, augmentation, and evaluation, not swapping CNN for Transformer.

## The Concept

**k-NN on MFCCs (the 1990s baseline).** Flatten MFCCs per clip, compute cosine similarity to a labeled bank, return majority vote of the top K. Surprisingly strong on clean, small datasets (Speech Commands, ESC-50). Runs with no GPU.

**2D CNN on log-mels (2015-2019).** Treat the  $(T, n_mels)$  log-mel as an image. Apply ResNet-18 or VGG-style. Global mean pool the time axis. Softmax over classes. Still the baseline in most 2026 kaggle competitions.

**Audio Spectrogram Transformer, AST (2021-2024).** Patchify the log-mel (e.g.  $16 \times 16$  patches), add position embeddings, feed to a ViT. State of the art on AudioSet (mAP 0.485) for supervised learning.

**BEATs and WavLM-base (2024-2026).** Self-supervised pretraining on millions of hours. Fine-tune on your task with 1-10% of the supervised data you would have needed. In 2026 this is the default starting point for non-speech audio. BEATs-iter3 beats AST by 1-2 mAP on AudioSet while using 1/4 the compute.

**Whisper-encoder as a frozen backbone (2024).** Take Whisper’s encoder, drop the decoder, attach a linear classifier. Near-SOTA on language ID and simple event classification with zero audio augmentation. The “free lunch” baseline.

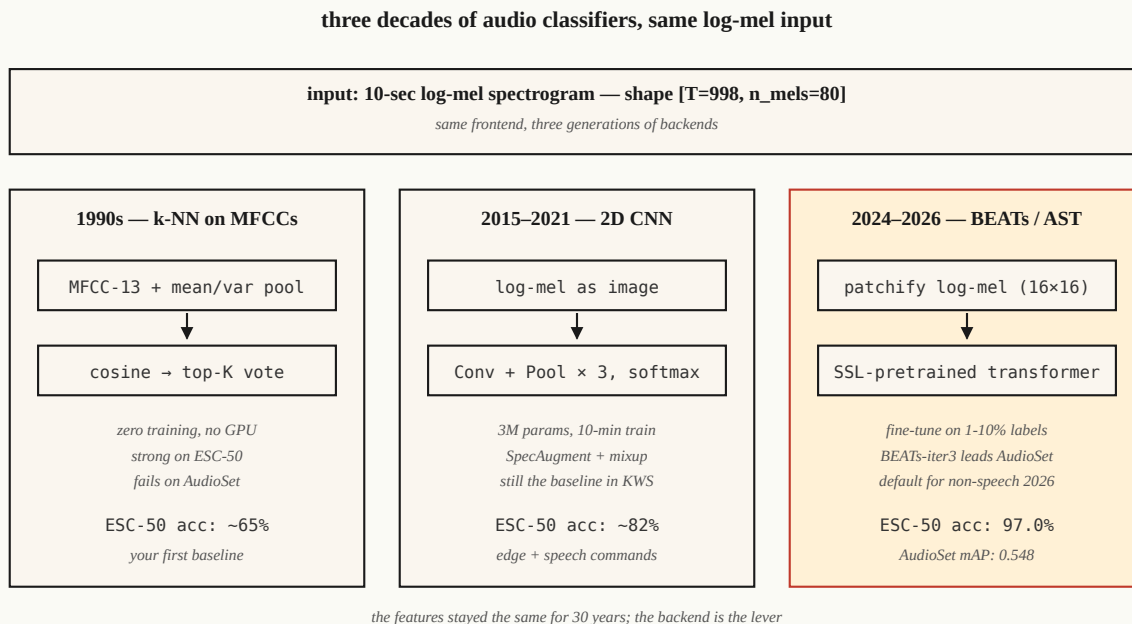


Figure 3: Audio classification ladder: k-NN on MFCCs to AST to BEATs

## Class imbalance is the real challenge

ESC-50: 50 classes, 40 clips each — balanced, easy. UrbanSound8K: 10 classes, imbalanced 10:1. AudioSet: 632 classes with a 100,000:1 long tail. Techniques that work:

- Balanced sampling during training (not in evaluation).
- Mixup: linearly interpolate two clips (and their labels) as augmentation.
- SpecAugment: mask random time and frequency bands. Simple; critical.

## Evaluation

- Multiclass exclusive (Speech Commands): top-1 accuracy, top-5 accuracy.
- Multiclass multi-label (AudioSet, UrbanSound-style): mean average precision (mAP).
- Heavily imbalanced: per-class recall + macro F1.

2026 numbers you should know:

Benchmark	Baseline	SOTA 2026	Source
ESC-50	82% (AST)	97.0% (BEATs-iter3)	BEATs paper (2024)
AudioSet mAP	0.485 (AST)	0.548 (BEATs-iter3)	HEAR leaderboard 2026
Speech Commands v2	98% (CNN)	99.0% (Audio-MAE)	HEAR v2 results

**Interactive figure: mfcc-pipeline.** This one moves. Watch it animate and drag its controls in the web edition: <https://aiengineeringfromscratch.com/lesson?path=phases/06-speech-and-audio/03-audio-classification>

## Build It

### Step 1: featurize

```
def featurize_mfcc(signal, sr, n_mfcc=13, n_mels=40, frame_len=400, hop=160):
    mag = stft_magnitude(signal, frame_len, hop)
    fb = mel_filterbank(n_mels, frame_len, sr)
    mels = apply_filterbank(mag, fb)
    log = log_transform(mels)
    return [dct_ii(frame, n_mfcc) for frame in log]
```

### Step 2: fixed-length summary

```
def summarize(mfcc_frames):
    n = len(mfcc_frames[0])
    mean = [sum(f[i] for f in mfcc_frames) / len(mfcc_frames) for i in range(n)]
    var = [
        sum((f[i] - mean[i]) ** 2 for f in mfcc_frames) / len(mfcc_frames) for i in
        range(n)
    ]
    return mean + var
```

Simple but strong: mean + variance across time gives a 26-dim fixed embedding for a 13-coef MFCC. Runs instantly. Beat state-of-the-art NN baselines on ESC-50 as recently as 2017.

### Step 3: k-NN

```
def cosine(a, b):
    dot = sum(x * y for x, y in zip(a, b))
    na = math.sqrt(sum(x * x for x in a)) or 1e-12
    nb = math.sqrt(sum(x * x for x in b)) or 1e-12
    return dot / (na * nb)

def knn_classify(q, bank, labels, k=5):
    sims = sorted(range(len(bank)), key=lambda i: -cosine(q, bank[i]))[:k]
    votes = Counter(labels[i] for i in sims)
    return votes.most_common(1)[0][0]
```

### Step 4: upgrade to CNN on log-mels

In PyTorch:

```
import torch.nn as nn

class AudioCNN(nn.Module):
    def __init__(self, n_mels=80, n_classes=50):
        super().__init__()
        self.body = nn.Sequential(
            nn.Conv2d(1, 32, 3, padding=1), nn.ReLU(), nn.MaxPool2d(2),
            nn.Conv2d(32, 64, 3, padding=1), nn.ReLU(), nn.MaxPool2d(2),
            nn.Conv2d(64, 128, 3, padding=1), nn.ReLU(),
            nn.AdaptiveAvgPool2d(1),
        )
        self.head = nn.Linear(128, n_classes)
```

```
def forward(self, x): # x: (B, 1, T, n_mels)
    return self.head(self.body(x).flatten(1))
```

3M parameters. Trains in ~10 min on ESC-50 with a single RTX 4090. 80%+ accuracy.

## Step 5: the 2026 default — fine-tune BEATs

```
from transformers import ASTFeatureExtractor, ASTForAudioClassification

ext =
    ↪ ASTFeatureExtractor.from_pretrained("MIT/ast-finetuned-audioset-10-10-0.4593")
model = ASTForAudioClassification.from_pretrained(
    "MIT/ast-finetuned-audioset-10-10-0.4593",
    num_labels=50,
    ignore_mismatched_sizes=True,
)

inputs = ext(audio, sampling_rate=16000, return_tensors="pt")
logits = model(**inputs).logits
```

For BEATs, use microsoft/BEATs-base via the beats library; the transformers API is the same shape.

## Use It

The 2026 stack:

Situation	Start with
Tiny dataset (<1000 clips)	k-NN on MFCC means (your baseline) + audio augmentation
Medium dataset (1K-100K)	BEATs or AST fine-tune
Large dataset (>100K)	Train from scratch or fine-tune
Real-time, edge	Whisper-encoder
Multi-label (AudioSet)	40-MFCC CNN, quantized to int8 (KWS-style)
Language ID	BEATs-iter3 with BCE loss + mixup + SpecAugment
	MMS-LID, SpeechBrain VoxLingua107 baseline

Decision rule: **start with a frozen backbone, not a fresh model.** Fine-tuning a BEATs head gets you 95% of SOTA in hours, not weeks.

**This chapter ships an artifact.** The course version of this lesson produces a reusable prompt or agent skill. It lives in the repository, ready to install: <https://github.com/rohitg00/ai-engineering-from-scratch/tree/main/phases/06-speech-and-audio/03-audio-classification>

## Exercises

Starter code and the lesson's working implementation: <https://github.com/rohitg00/ai-engineering-from-scratch/tree/main/phases/06-speech-and-audio/03-audio-classification/code>



1. **Easy.** Run `code/main.py`. It trains the k-NN MFCC baseline on a 4-class synthetic dataset (pure tones at different pitches). Report confusion matrix.
2. **Medium.** Replace `summarize` with `[mean, var, skew, kurtosis]`. Does 4-moment pooling beat `mean+var` on the same synthetic dataset?
3. **Hard.** Using `torchaudio`, train a 2D CNN on ESC-50 fold 1. Report 5-fold cross-validation accuracy. Add SpecAugment (time mask = 20, freq mask = 10) and report the delta.

## Key Terms

Term	What people say	What it actually means
AudioSet	The ImageNet of audio	Google's 2M-clip, 632-class weakly-labeled YouTube dataset.
ESC-50	Small classification benchmark	50 classes $\times$ 40 clips of environmental sounds.
AST	Audio Spectrogram Transformer	ViT on log-mel patches; 2021 SOTA.
BEATs	Self-supervised audio	Microsoft model, iter3 leads AudioSet as of 2026.
Mixup	Pair augmentation	$x = \lambda \cdot x_1 + (1-\lambda) \cdot x_2$ ; $y = \lambda \cdot y_1 + (1-\lambda) \cdot y_2$ .
SpecAugment	Mask-based augmentation	Zero-out random time and frequency bands of the spectrogram.
mAP	Main multi-label metric	Mean average precision across classes and thresholds.

## Further Reading

- Gong, Chung, Glass (2021). AST: Audio Spectrogram Transformer — the architecture of record from 2021–2024.
- Chen et al. (2022, rev. 2024). BEATs: Audio Pre-Training with Acoustic Tokenizers — the 2024+ default.
- Park et al. (2019). SpecAugment — the dominant audio augmentation.
- Piczak (2015). ESC-50 dataset — 50-class benchmark that lives on.
- Gemmeke et al. (2017). AudioSet — 632-class YouTube taxonomy; still the gold standard.

**Continue online.** The living edition of this chapter has more than the page can hold:

- Animated, interactive figures and the web text: <https://aiengineeringfromscratch.com/lesson?path=phases/06-speech-and-audio/03-audio-classification>
- Runnable code for every step: <https://github.com/rohitg00/ai-engineering-from-scratch/tree/main/phases/06-speech-and-audio/03-audio-classification/code>

The repository moves faster than any printing. When the book and the repo disagree, trust the repo.

# Speech Recognition (ASR) — CTC, RNN-T, Attention

---

Speech recognition is audio classification at every timestep, glued together by a sequence model that knows English and silence. CTC, RNN-T, and attention are the three ways to do it. Pick one and understand why.

**Type:** Build **Languages:** Python **Prerequisites:** Phase 6 · 02 (Spectrograms & Mel), Phase 5 · 08 (CNNs & RNNs for Text), Phase 5 · 10 (Attention) **Time:** ~45 minutes

## The Problem

You have a 10-second 16 kHz clip. You want a string: “turn on the kitchen lights”. The challenge is structural: audio frames do not align one-to-one with characters. The word “okay” might take 200 ms or 1200 ms. Silence punctuates the utterance. Some phonemes are longer than others. The number of output tokens is not known in advance.

Three formulations solve this:

1. **CTC (Connectionist Temporal Classification).** Emit per-frame token probabilities including a special *blank*. Collapse repeats and blanks at decode time. Non-autoregressive, fast. Used by wav2vec 2.0, MMS.
2. **RNN-T (Recurrent Neural Network Transducer).** Joint network predicts next token given encoder frame and previous tokens. Streamable. Used by Google’s on-device ASR, NVIDIA Parakeet.
3. **Attention encoder-decoder.** Encoder compresses audio to hidden states, decoder cross-attends to generate tokens autoregressively. Used by Whisper, SeamlessM4T.

In 2026, SOTA WER on LibriSpeech test-clean is 1.4% (Parakeet-TDT-1.1B, NVIDIA) and 1.58% (Whisper-Large-v3-turbo). The differences are tiny; the deployment differences are huge.

## The Concept

**CTC intuition.** Let the encoder output  $T$  frame-level distributions over  $V+1$  tokens ( $V$  chars + blank). For a target string  $y$  of length  $U < T$ , any frame alignment that collapses to  $y$  counts. CTC loss sums over all such alignments. Inference: per-frame argmax, collapse repeats, remove blanks.

Advantages: non-autoregressive, streamable, zero lookahead. Drawback: *conditional independence assumption* — each frame prediction is independent of the others, so there is no internal language model. Fix with an external LM via beam search or shallow fusion.

**RNN-T intuition.** Adds a *predictor* network that embeds the token history and a *joiner* that combines predictor state with encoder frame into a joint distribution over  $V+1$  (the +1 is a null / no-emit). Explicitly models the conditional dependence CTC ignored. Streamable because each step conditions only on past frames and past tokens.

## three ways to map T audio frames to U tokens

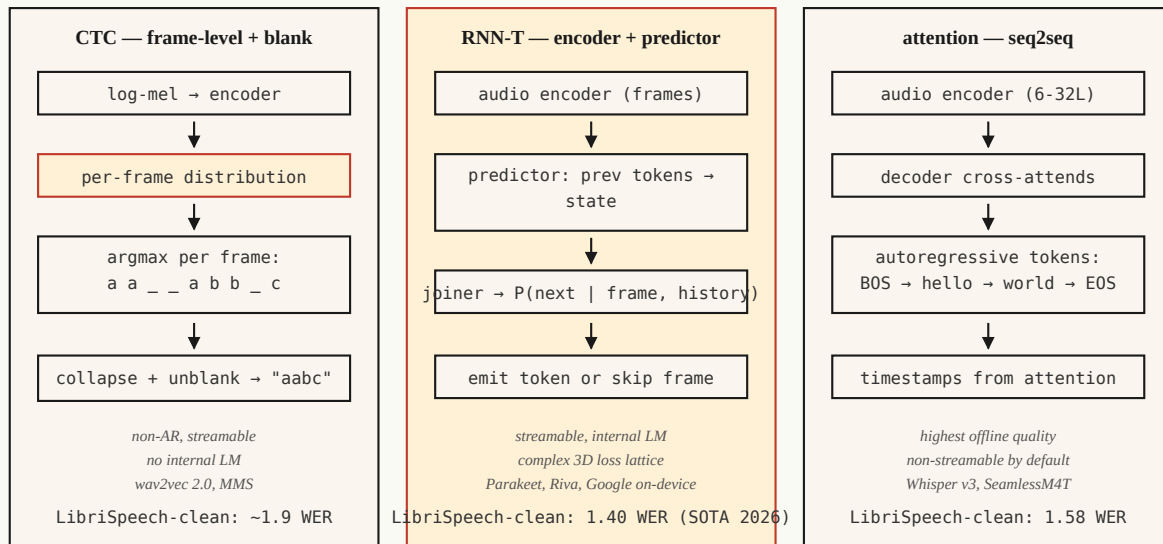


Figure 4: Three ASR formulations: CTC, RNN-T, attention-encoder-decoder

Advantages: streamable + internal LM. Drawback: training is more complex and memory-hungry (3D loss lattice); RNN-T loss kernels are a whole library category on their own.

**Attention encoder-decoder.** Encoder (6-32 transformer layers) over log-mel frames. Decoder (6-32 transformer layers) cross-attends to encoder outputs to generate tokens autoregressively. No alignment constraint — attention can look anywhere in the audio. Non-streamable unless you restrict attention (chunked Whisper-Streaming, 2024).

Advantages: highest quality on offline ASR, easy to train with standard seq2seq tooling. Drawback: autoregressive latency is proportional to output length; cannot stream without engineering.

## WER: the one number

**Word Error Rate** =  $(S + D + I) / N$ , where S=substitutions, D=deletions, I=insertions, N=reference word count. Matches Levenshtein edit distance at the word level. Lower is better. A WER above 20% is generally unusable; below 5% is human-parity for read speech. 2026 numbers on standard benchmarks:

Model	LibriSpeech test-clean	LibriSpeech test-other	Size
Parakeet-TDT-1.1B	1.40%	2.78%	1.1B params
Whisper-Large-v3-turbo	1.58%	3.03%	809M
Canary-1B Flash	1.48%	2.87%	1B
Seamless M4T v2	1.7%	3.5%	2.3B

All these are encoder-decoder or RNN-T based. Pure CTC systems (wav2vec 2.0) sit around 1.8–2.1% on test-clean.

**Interactive figure: ctc-collapse.** This one moves. Watch it animate and drag its controls in the web edition: <https://aiengineeringfromscratch.com/lesson?path=phases/06-speech-and-audio/04-speech-recognition-asr>

## Build It

### Step 1: greedy CTC decode

```
def ctc_greedy(frame_logits, blank=0, vocab=None):
    # frame_logits: list of per-frame probability vectors
    preds = [max(range(len(p)), key=lambda i: p[i]) for p in frame_logits]
    out = []
    prev = -1
    for p in preds:
        if p != prev and p != blank:
            out.append(p)
        prev = p
    return "".join(vocab[i] for i in out) if vocab else out
```

Two rules: collapse consecutive repeats, drop blanks. Example: a a _ _ a b b _ c → a a b c.

### Step 2: beam-search CTC

```
def ctc_beam(frame_logits, beam=8, blank=0):
    import math
    beams = [([], 0.0)] # (tokens, log_prob)
    for p in frame_logits:
        log_p = [math.log(max(pi, 1e-10)) for pi in p]
        candidates = []
        for seq, lp in beams:
            for t, lpt in enumerate(log_p):
                new = seq[:] if t == blank else (seq + [t] if not seq or seq[-1] !=
                ↪ t else seq)
                candidates.append((new, lp + lpt))
            candidates.sort(key=lambda x: -x[1])
            beams = candidates[:beam]
    return beams[0][0]
```

Production uses prefix tree beam search with LM fusion; this is the conceptual skeleton.

### Step 3: WER

```
def wer(ref, hyp):
    r, h = ref.split(), hyp.split()
    dp = [[0] * (len(h) + 1) for _ in range(len(r) + 1)]
    for i in range(len(r) + 1):
        dp[i][0] = i
    for j in range(len(h) + 1):
        dp[0][j] = j
    for i in range(1, len(r) + 1):
```

```

    for j in range(1, len(h) + 1):
        cost = 0 if r[i - 1] == h[j - 1] else 1
        dp[i][j] = min(
            dp[i - 1][j] + 1,
            dp[i][j - 1] + 1,
            dp[i - 1][j - 1] + cost,
        )
    return dp[len(r)][len(h)] / max(1, len(r))

```

## Step 4: inference against Whisper

```

import whisper
model = whisper.load_model("large-v3-turbo")
result = model.transcribe("clip.wav")
print(result["text"])

```

One-liner for the strongest general ASR in 2026. Runs on a 24 GB GPU at ~20× realtime.

## Step 5: streaming with Parakeet or wav2vec 2.0

```

from transformers import pipeline
asr = pipeline("automatic-speech-recognition", model="nvidia/parakeet-tdt-1.1b")
for chunk in streaming_audio():
    print(asr(chunk, return_timestamps=True))

```

Streaming ASR needs chunked encoder attention and carryover state; use a library that supports it (NeMo for Parakeet, transformers pipeline with chunk_length_s).

## Use It

The 2026 stack:

Situation	Pick
English, offline, max quality	Whisper-large-v3-turbo
Multilingual, robust	SeamlessM4T v2
Streaming, low latency	Parakeet-TDT-1.1B or Riva
Edge, mobile, <500 ms latency	Whisper-Tiny quantized or Moonshine (2024)
Long-form	Whisper with VAD-based chunking (WhisperX)
Domain-specific (medical, legal)	Fine-tune wav2vec 2.0 + domain LM fusion

## Pitfalls that still ship in 2026

- **No VAD.** Running Whisper on silence produces hallucinations (“Thanks for watching!”). Always gate with VAD.
- **Character vs word vs subword WER.** Report word-level WER *after* normalization (lowercase, punctuation stripped).
- **Language ID drift.** Whisper’s auto LID mis-routes noisy clips to Japanese or Welsh; force language="en" when you know.

- **Long clips without chunking.** Whisper has a 30-second window. Use `chunk_length_s=30`, `stride=5` for anything longer.

**This chapter ships an artifact.** The course version of this lesson produces a reusable prompt or agent skill. It lives in the repository, ready to install: <https://github.com/rohitg00/ai-engineering-from-scratch/tree/main/phases/06-speech-and-audio/04-speech-recognition-asr>

## Exercises

Starter code and the lesson’s working implementation: <https://github.com/rohitg00/ai-engineering-from-scratch/tree/main/phases/06-speech-and-audio/04-speech-recognition-asr/code>

1. **Easy.** Run `code/main.py`. It greedily decodes a hand-crafted CTC output and computes WER against a reference.
2. **Medium.** Implement the prefix-tree beam search in Step 2 properly (account for the blank merge rule). Compare with greedy on a 10-example synthetic dataset.
3. **Hard.** Use `whisper-large-v3-turbo` on LibriSpeech test-clean. Compute WER on the first 100 utterances. Compare with published numbers.

## Key Terms

Term	What people say	What it actually means
CTC	The blank-token loss	Marginal over all frame-to-token alignments; non-AR.
RNN-T	The streaming loss	CTC + next-token predictor; handles word-order.
Attention enc-dec	Whisper-style	Encoder + cross-attending decoder; best offline quality.
WER	The number you report	$(S+D+I)/N$ at word level.
Blank	The emptiness	Special token in CTC signalling “no emission this frame”.
LM fusion	External language model	Add weighted LM log-probs during beam search.
VAD	The silence gate	Voice activity detector; trims non-speech.

## Further Reading

- Graves et al. (2006). Connectionist Temporal Classification — the CTC paper.
- Graves (2012). Sequence Transduction with RNNs — the RNN-T paper.
- Radford et al. / OpenAI (2022). Whisper: Robust Speech Recognition via Large-Scale Weak Supervision — the 2022 canonical paper; v3-turbo extension in 2024.
- NVIDIA NeMo — Parakeet-TDT card — 2026 Open ASR Leaderboard leader.
- Hugging Face — Open ASR Leaderboard — live benchmark across 25+ models.

**Continue online.** The living edition of this chapter has more than the page can hold:

- Animated, interactive figures and the web text: <https://aiengineeringfromscratch.com/lesson?path=phases/06-speech-and-audio/04-speech-recognition-asr>
- Runnable code for every step: <https://github.com/rohitg00/ai-engineering-from-scratch/tree/main/phases/06-speech-and-audio/04-speech-recognition-asr/code>

The repository moves faster than any printing. When the book and the repo disagree, trust the repo.

# Whisper — Architecture & Fine-Tuning

---

Whisper is a 30-second-window transformer encoder-decoder, trained on 680k hours of multilingual weakly-supervised audio-text pairs. One architecture, multiple tasks, robust across 99 languages. The 2026 reference ASR.

**Type:** Build **Languages:** Python **Prerequisites:** Phase 6 · 04 (ASR), Phase 5 · 10 (Attention), Phase 7 · 05 (Full Transformer) **Time:** ~75 minutes

## The Problem

Whisper, released by OpenAI in September 2022, was the first ASR model to ship as a commodity: paste audio, get text, 99 languages, robust to noise, runs on a laptop. By 2024 OpenAI had shipped Large-v3 and Turbo variants; by 2026, Whisper is the default baseline for everything from podcast transcription to voice assistants to YouTube subtitles.

But Whisper is not a pipeline you can treat as a black box forever. Domain shift kills it — technical jargon, speaker accents, proper nouns, short clips, silence. You need to know:

1. What it actually is inside.
2. How to give it chunked, streaming, or long-form audio correctly.
3. When to fine-tune and how.

## The Concept

**Architecture.** Standard transformer encoder-decoder.

- Input: 30-second log-mel spectrogram, 80 mels, 10 ms hop → 3000 frames. Clips shorter are zero-padded, clips longer are chunked.
- Encoder: conv-downsample (stride 2) + N transformer blocks. For Large-v3: 32 layers, 1280-dim, 20 heads.
- Decoder: N transformer blocks with causal self-attn + cross-attn to encoder output. Same size as encoder.
- Output: BPE tokens over a 51,865-token vocab.

Large-v3 has 1.55B params. Turbo uses a 4-layer decoder (from 32), cutting latency 8× with a <1% WER hit.

**The prompt format.** Whisper is a multitask model steered by special tokens in the decoder prompt:

```
<|startoftranscript|><|en|><|transcribe|><|notimestamps|> Hello  
↪ world.<|endoftext|>
```

- <|en|> — language tag; forces translation-vs-transcription behavior.
- <|transcribe|> or <|translate|> — translate English output from any-language input, or verbatim.
- <|notimestamps|> — skip word-level timestamps (faster).



## Whisper: one architecture, many tasks, steered by prompt

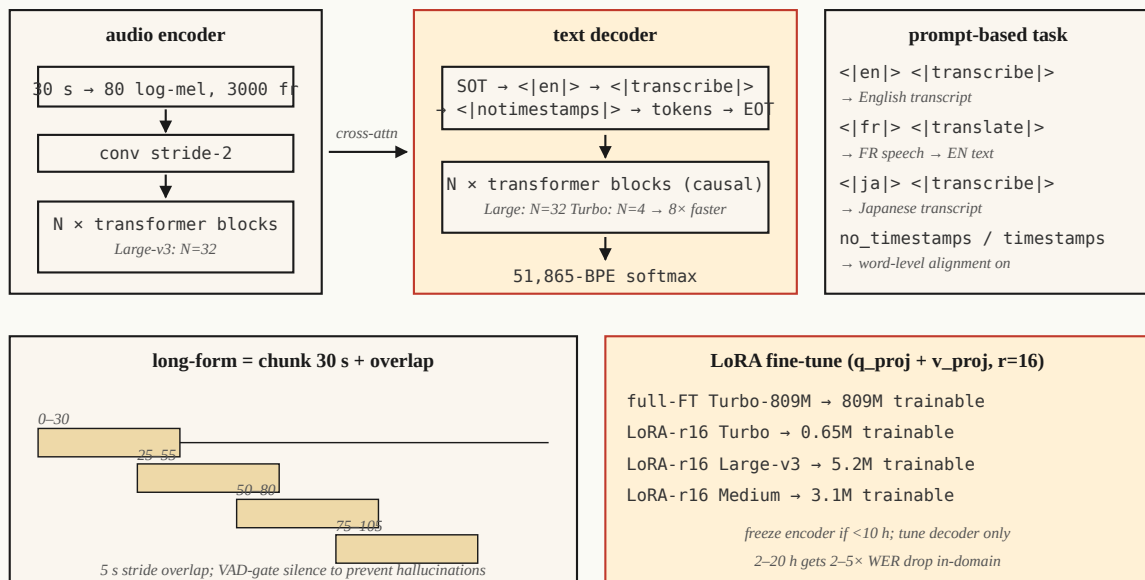


Figure 5: Whisper encoder-decoder, tasks, chunked inference, fine-tune

The prompt is what lets one model do many tasks. Change <|en|> to <|fr|> and it transcribes French.

**30-second window.** Everything is pinned to 30 seconds. Longer clips need chunking; shorter clips are padded. Windows are not streamed natively — this is why WhisperX, Whisper-Streaming, and faster-whisper exist.

**Log-mel normalization.**  $(\log_mel - \text{mean}) / \text{std}$  where the stats come from Whisper’s own training corpus. You *must* use Whisper’s preprocessing (`whisper.audio.log_mel_spectrogram`), not `librosa.feature.melspectrogram`.

## Variants in 2026

Variant	Params	Latency (A100)	WER (LibriSpeech-clean)
Tiny	39M	1× realtime	5.4%
Base	74M	1×	4.1%
Small	244M	1×	3.0%
Medium	769M	1×	2.7%
Large-v3	1.55B	2×	1.8%
Large-v3-turbo	809M	8×	1.58%
Whisper-Streaming (2024)	1.55B	streaming	2.0%

## Fine-tuning

Canonical workflow in 2026:

1. Collect 10–100 hours of target-domain audio with aligned transcripts.
2. Run `transformers.Seq2SeqTrainer` with `generate_with_loss` callback.
3. Parameter-efficient: LoRA on `q_proj`, `k_proj`, `v_proj` of attention layers reduces GPU memory 4× with <0.3 WER cost.
4. Freeze the encoder if you have <10 hours. Only tune the decoder.
5. Use Whisper’s own tokenizer and prompt format; never swap tokenizers.

Community results: fine-tuning Medium on 20 hours of medical dictation drops WER from 12% to 4.5% on medical vocabulary. Fine-tuning Turbo on 4 hours of Icelandic drops WER from 18% to 6%.

**Interactive figure: sp-asr-attention.** This one moves. Watch it animate and drag its controls in the web edition: <https://aiengineeringfromscratch.com/lesson?path=phases/06-speech-and-audio/05-whisper-architecture-finetuning>

## Build It

### Step 1: run Whisper out of the box

```
import whisper
model = whisper.load_model("large-v3-turbo")
result = model.transcribe(
    "clip.wav",
    language="en",
    task="transcribe",
    temperature=0.0,
    condition_on_previous_text=False, # prevents runaway repetition
)
print(result["text"])
for seg in result["segments"]:
    print(f"[{seg['start']:.2f}–{seg['end']:.2f}] {seg['text']}")
```

Key defaults you should always override: `temperature=0.0` (sampling defaults to 0.0 → 0.2 → 0.4 ... fallback chain), `condition_on_previous_text=False` (prevents the cascading hallucination problem), and `no_speech_threshold=0.6` (silence detection).

### Step 2: chunked long-form

```
# whisperx is the 2026 reference for long-form with word-level timestamps
import whisperx
model = whisperx.load_model("large-v3-turbo", device="cuda",
    ↪ compute_type="float16")
segments = model.transcribe("1hour.mp3", batch_size=16, chunk_size=30)
```

WhisperX adds (1) Silero VAD gating, (2) word-level alignment via `wav2vec 2.0`, (3) diarization via `pyannote.audio`. The 2026 workhorse for production transcription.

### Step 3: fine-tune with LoRA

```
from transformers import WhisperForConditionalGeneration, WhisperProcessor
from peft import LoraConfig, get_peft_model

model = WhisperForConditionalGeneration.from_pretrained("openai/whisper-large-v3-
    ↪ turbo")
```

```
lora = LoraConfig(
    r=16, lora_alpha=32, target_modules=["q_proj", "v_proj"],
    lora_dropout=0.1, bias="none", task_type="SEQ_2_SEQ_LM",
)
model = get_peft_model(model, lora)
# model.print_trainable_parameters() -> ~3M trainable / 809M total
```

Then standard Trainer loop. Checkpoint every 1000 steps. Evaluate with WER on held-out.

## Step 4: inspect what each layer learns

```
# Grab cross-attention weights during decode to see what the decoder attends to.
with torch.inference_mode():
    out = model.generate(
        input_features=features,
        return_dict_in_generate=True,
        output_attentions=True,
    )
# out.cross_attentions: layer × head × step × src_len
```

Visualize with a heatmap — you will see diagonal alignment as decoder steps scan through encoder frames. That diagonal is Whisper’s notion of word timestamps.

## Use It

The 2026 stack:

Situation	Pick
General English, offline	Large-v3-turbo via whisperx
Mobile / edge	Whisper-Tiny quantized (int8) or Moonshine
Multilingual long-form	Large-v3 via whisperx + diarization
Low-resource language	Fine-tune Medium or Turbo with LoRA
Streaming (2 s latency)	Whisper-Streaming or Parakeet-TDT
Word-level timestamps	WhisperX (forced alignment via wav2vec 2.0)

faster-whisper (CTranslate2 backend) is the fastest CPU+GPU inference runtime in 2026 — 4× faster than vanilla with identical output.

## Pitfalls that still ship in 2026

- **Hallucinated text on silence.** Whisper trained on captions includes “Thanks for watching!”, “Subscribe!”, song lyrics. Always VAD-gate before calling.
- **condition_on_previous_text cascade.** One hallucination pollutes subsequent windows. Set False unless you need fluency across chunks.
- **Short-clip padding.** A 2-second clip padded to 30 seconds can hallucinate in the trailing silence. Use pad=False or VAD-gate.

- **Wrong mel stats.** Using librosa’s mels instead of Whisper’s produces near-random output. Use `whisper.audio.log_mel_spectrogram`.

**This chapter ships an artifact.** The course version of this lesson produces a reusable prompt or agent skill. It lives in the repository, ready to install: <https://github.com/rohitg00/ai-engineering-from-scratch/tree/main/phases/06-speech-and-audio/05-whisper-architecture-finetuning>

## Exercises

Starter code and the lesson’s working implementation: <https://github.com/rohitg00/ai-engineering-from-scratch/tree/main/phases/06-speech-and-audio/05-whisper-architecture-finetuning/code>

1. **Easy.** Run `code/main.py`. It tokenizes a Whisper-style prompt, computes decoded shape budgets, and prints the chunk schedule for a 10-minute clip.
2. **Medium.** Install `faster-whisper`, transcribe a 10-minute podcast, compare WER against a human transcript. Try `language="auto"` vs forced `language="en"`.
3. **Hard.** Using HF datasets, pick a language Whisper struggles with (e.g., Urdu), fine-tune Medium with LoRA for 2 epochs on 2 hours, and report WER delta.

## Key Terms

Term	What people say	What it actually means
30-sec window	Whisper’s limit	Hard input cap; chunk longer audio.
SOT	Start-of-transcript	<code>&lt;\ startoftranscript\ &gt;</code> kicks off the decoder prompt.
Timestamps token	Temporal alignment	Every 0.02 s offset is a special token in the 51k vocab.
Turbo	The fast variant	4-decoder layers, 8× faster, <1% WER regression.
WhisperX	The long-form wrapper	VAD + Whisper + wav2vec alignment + diarization.
LoRA fine-tune	Efficient tuning	Add low-rank adapters to attention; train ~0.3% of params.
Hallucination	The silent failure	Whisper produces fluent English from noise/silence.

## Further Reading

- Radford et al. (2022). Whisper paper — the original architecture and training recipe.
- OpenAI (2024). Whisper Large-v3-turbo release — 4-layer decoder, 8× speedup.
- Bain et al. (2023). WhisperX — long-form, word-aligned, diarized.
- Systran — faster-whisper repo — CTranslate2-backed, 4× faster.
- HuggingFace — Whisper fine-tune tutorial — canonical LoRA / full-FT walkthrough.

**Continue online.** The living edition of this chapter has more than the page can hold:

- Animated, interactive figures and the web text: <https://aiengineeringfromscratch.com/lesson?path=phases/06-speech-and-audio/05-whisper-architecture-finetuning>

- Runnable code for every step: <https://github.com/rohitg00/ai-engineering-from-scratch/tree/main/phases/06-speech-and-audio/05-whisper-architecture-finetuning/code>

The repository moves faster than any printing. When the book and the repo disagree, trust the repo.

# Speaker Recognition & Verification

---

ASR asks “what did they say?” Speaker recognition asks “who said it?” The math looks the same — embeddings plus cosine — but every production decision hinges on a single EER number.

**Type:** Build **Languages:** Python **Prerequisites:** Phase 6 · 02 (Spectrograms & Mel), Phase 5 · 22 (Embedding Models) **Time:** ~45 minutes

## The Problem

A user says a passphrase. You want to know: is this the person they claim to be (*verification*, 1:1), or is it the first person in your enrollment bank (*identification*, 1:N)? Or neither — is this an unknown speaker (*open-set*)?

Pre-2018: GMM-UBM + i-vectors. Reasonable EER but fragile to channel shift (phone vs laptop) and emotion. 2018–2022: x-vectors (TDNN backbone trained with angular margin). 2022+: ECAPA-TDNN and WavLM-large embeddings. By 2026 the field is dominated by three models and one metric.

The metric is **EER** — Equal Error Rate. Set your decision threshold so False Accept Rate = False Reject Rate. The crossover is EER. Used in every paper, every leaderboard, every procurement call.

## The Concept

**The pipeline.** Enrollment: record 5–30 seconds of the target speaker; compute a fixed-dimension embedding (192-d for ECAPA-TDNN, 256-d for WavLM-large). Verification: get the test utterance embedding; compute cosine similarity; compare to a threshold.

**ECAPA-TDNN (2020, still dominant 2026).** Emphasized Channel Attention, Propagation and Aggregation - Time-Delay Neural Network. 1D conv blocks with squeeze-excitation, multi-head attention pooling, followed by a linear layer to 192-d. Trained on VoxCeleb 1+2 (2,700 speakers, 1.1M utterances) with Additive Angular Margin loss (AAM-softmax).

**WavLM-SV (2022+).** Fine-tune a pretrained WavLM-large SSL backbone with AAM loss. Higher quality but slower — 300+ MB vs 15 MB.

**x-vector (baseline).** TDNN + statistics pooling. Classic; still useful on CPU / edge.

**AAM-softmax.** Standard softmax with added margin  $m$  in the angular space:  $\cos(\theta + m)$  for the correct class. Forces inter-class angular separation. Typical  $m=0.2$ , scale  $s=30$ .

## Scoring

- **Cosine** between enrollment and test embeddings. Threshold-based decision.
- **PLDA (Probabilistic LDA).** Project embeddings into a latent space where same-speaker vs different-speaker has a closed-form likelihood ratio. Added on top of cosine for +10–20% EER reduction. Standard pre-2020; now used only in closed-set setups.

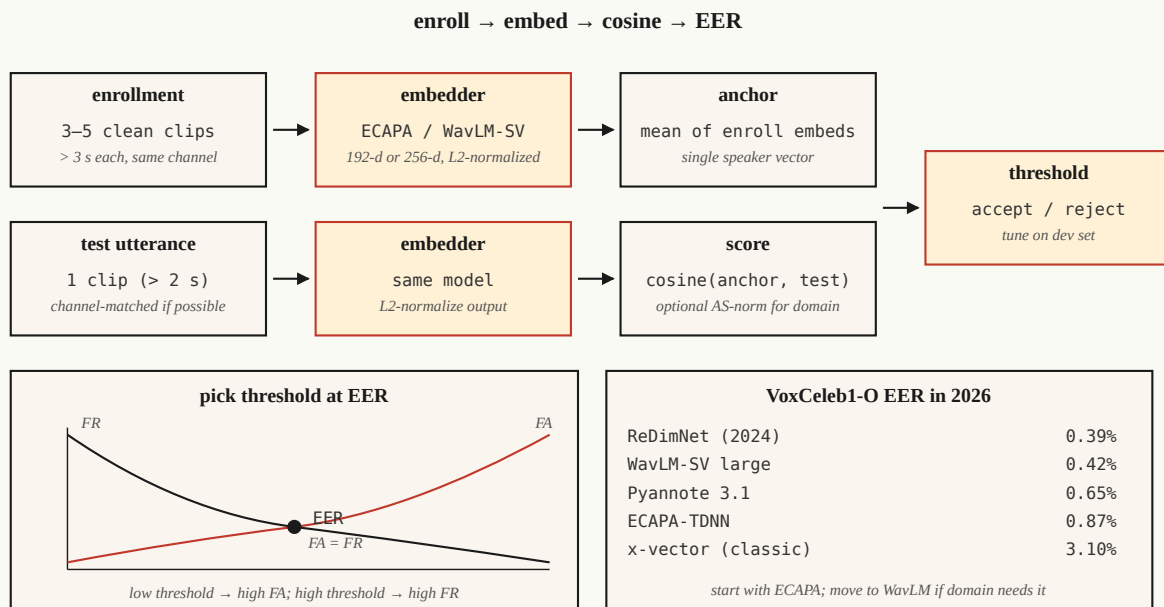


Figure 6: Enrollment + verification pipeline with embedding + cosine + EER

- **Score normalization.** S-norm or AS-norm: normalize each score against a cohort of imposter means and stds. Essential for cross-domain eval.

## Numbers you should know (2026)

Model	VoxCeleb1-O EER	Params	Throughput (A100)
x-vector (classic)	3.10%	5 M	400× RT
ECAPA-TDNN	0.87%	15 M	200× RT
WavLM-SV large	0.42%	316 M	20× RT
Pyannote 3.1 segmentation + embedding	0.65%	6 M	100× RT
ReDimNet (2024)	0.39%	24 M	100× RT

## Diarization

“Who spoke when” in a multi-speaker clip. Pipeline: VAD → segment → embed each segment → cluster (agglomerative or spectral) → smooth boundaries. Modern stack: pyannote.audio 3.1, which bundles speaker segmentation + embedding + clustering behind one call. 2026 SOTA DER on AMI is ~15% (down from 23% in 2022).

**Interactive figure: sp-eer-crossover.** This one moves. Watch it animate and drag its controls in the web edition: <https://aiengineeringfromscratch.com/lesson?path=phases/06->

speech-and-audio/06-speaker-recognition-verification

## Build It

### Step 1: toy embedding from MFCC statistics

```
def embed_mfcc_stats(signal, sr):
    frames = featurize_mfcc(signal, sr, n_mfcc=13)
    mean = [sum(f[i] for f in frames) / len(frames) for i in range(13)]
    std = [
        math.sqrt(sum((f[i] - mean[i]) ** 2 for f in frames) / len(frames))
        for i in range(13)
    ]
    return mean + std # 26-d
```

Not SOTA by a mile — for teaching only. `code/main.py` uses this as a proof-of-concept on synthetic speaker data.

### Step 2: cosine similarity + threshold

```
def cosine(a, b):
    dot = sum(x * y for x, y in zip(a, b))
    na = math.sqrt(sum(x * x for x in a))
    nb = math.sqrt(sum(x * x for x in b))
    return dot / (na * nb) if na and nb else 0.0

def verify(enroll, test, threshold=0.75):
    return cosine(enroll, test) >= threshold
```

### Step 3: EER from similarity pairs

```
def eer(same_scores, diff_scores):
    thresholds = sorted(set(same_scores + diff_scores))
    best = (1.0, 1.0, 0.0) # (fa, fr, threshold)
    for t in thresholds:
        fr = sum(1 for s in same_scores if s < t) / len(same_scores)
        fa = sum(1 for s in diff_scores if s >= t) / len(diff_scores)
        if abs(fa - fr) < abs(best[0] - best[1]):
            best = (fa, fr, t)
    return (best[0] + best[1]) / 2, best[2]
```

Returns (eer, threshold_at_eer). Report both.

### Step 4: production with SpeechBrain

```
from speechbrain.pretrained import EncoderClassifier

clf = EncoderClassifier.from_hparams(source="speechbrain/spkrec-ecapa-voxceleb")

# enroll: average the embeddings of 3-5 clean samples
enroll = torch.stack([clf.encode_batch(load(x)) for x in
    ↪ enrollment_clips]).mean(0)
# verify
```



```
score = clf.similarity(enroll, clf.encode_batch(load("test.wav"))).item()
verdict = score > 0.25 # ECAPA typical threshold; tune on your data
```

## Step 5: diarize with pyannote

```
from pyannote.audio import Pipeline

pipe = Pipeline.from_pretrained("pyannote/speaker-diarization-3.1")
diarization = pipe("meeting.wav", num_speakers=None)
for turn, _, speaker in diarization.itertracks(yield_label=True):
    print(f"{turn.start:.1f}-{turn.end:.1f} {speaker}")
```

## Use It

The 2026 stack:

Situation	Pick
Closed-set 1:1 verification, edge	ECAPA-TDNN + cosine threshold
Open-set verification, cloud	WavLM-SV + AS-norm
Diarization (meetings, podcasts)	pyannote/speaker-diarization-3.1
Anti-spoofing (replay / deepfake detection)	AASIST or RawNet2
Tiny embedded (KWS + enrollment)	Titanet-Small (NeMo)

## Pitfalls

- **Channel mismatch.** Model trained on VoxCeleb (web video)  $\neq$  phone-call audio. Always evaluate on target channel.
- **Short utterances.** EER degrades sharply below 3 seconds of test audio.
- **Enrollment with noise.** One noisy enrollment poisons the anchor. Use  $\geq 3$  clean samples and average.
- **Fixed threshold across conditions.** Always tune the threshold on a held-out dev set from the target domain.
- **Cosine on non-normalized embeddings.** L2-normalize first; otherwise the magnitude dominates.

**This chapter ships an artifact.** The course version of this lesson produces a reusable prompt or agent skill. It lives in the repository, ready to install: <https://github.com/rohitg00/ai-engineering-from-scratch/tree/main/phases/06-speech-and-audio/06-speaker-recognition-verification>

## Exercises

Starter code and the lesson's working implementation: <https://github.com/rohitg00/ai-engineering-from-scratch/tree/main/phases/06-speech-and-audio/06-speaker-recognition-verification/code>

1. **Easy.** Run code/main.py. Builds synthetic "speakers" (different tone profiles), enrolls, computes EER on a 100-pair trial list.

2. **Medium.** Use SpeechBrain ECAPA on 30 VoxCeleb1 utterances (5 speakers × 6 each). Compute EER with cosine vs PLDA.
3. **Hard.** Build the full enroll → diarize → verify pipeline with `pyannote.audio`. Evaluate DER on AMI dev set.

## Key Terms

Term	What people say	What it actually means
EER	The headline metric	Threshold where False Accept = False Reject.
Verification	1:1	“Is this Alice?”
Identification	1:N	“Who is speaking?”
Open-set	Unknown possible	Test set can contain unenrolled speakers.
Enrollment	Registering	Computing a speaker’s reference embedding.
AAM-softmax	The loss	Softmax with additive angular margin; forces cluster separation.
PLDA	Classic scoring	Probabilistic LDA; likelihood-ratio scoring on top of embeddings.
DER	Diarization metric	Diarization Error Rate — miss + false alarm + confusion.

## Further Reading

- Snyder et al. (2018). X-Vectors: Robust DNN Embeddings for Speaker Recognition — the classic deep-embedding paper.
- Desplanques et al. (2020). ECAPA-TDNN — dominant architecture 2020–2026.
- Chen et al. (2022). WavLM: Large-Scale Self-Supervised Pre-Training for Full Stack Speech Processing — SSL backbone for SV and diarization.
- Bredin et al. (2023). `pyannote.audio` 3.1 — production diarization + embedding stack.
- VoxCeleb leaderboard (updated 2026) — current EER standings across models.

**Continue online.** The living edition of this chapter has more than the page can hold:

- Animated, interactive figures and the web text: <https://aiengineeringfromscratch.com/lesson?path=phases/06-speech-and-audio/06-speaker-recognition-verification>
- Runnable code for every step: <https://github.com/rohitg00/ai-engineering-from-scratch/tree/main/phases/06-speech-and-audio/06-speaker-recognition-verification/code>

The repository moves faster than any printing. When the book and the repo disagree, trust the repo.

# Text-to-Speech (TTS) — From Tacotron to F5 and Kokoro

---

ASR inverts speech to text; TTS inverts text to speech. The 2026 stack is three parts: text → tokens, tokens → mel, mel → waveform. Each part has a default model that fits in a laptop.

**Type:** Build **Languages:** Python **Prerequisites:** Phase 6 · 02 (Spectrograms & Mel), Phase 5 · 09 (Seq2Seq), Phase 7 · 05 (Full Transformer) **Time:** ~75 minutes

## The Problem

You have a string: “Please remind me to water the plants at 6 pm.” You need a 3-second audio clip that sounds natural, has correct prosody (pauses, stress), pronounces “plants” with the right vowel, and runs in under 300 ms on a CPU for a live voice assistant. You also need to swap voices, handle code-switched input (“remind me at 6 pm, daijoubu?”), and not embarrass yourself on names.

Modern TTS pipelines look like this:

1. **Text frontend.** Normalize text (dates, numbers, emails), convert to phonemes or sub-word tokens, predict prosody features.
2. **Acoustic model.** Text → mel spectrogram. Tacotron 2 (2017), FastSpeech 2 (2020), VITS (2021), F5-TTS (2024), Kokoro (2024).
3. **Vocoder.** Mel → waveform. WaveNet (2016), WaveRNN, HiFi-GAN (2020), BigVGAN (2022), neural codec vocoders in 2024+.

In 2026 the acoustic + vocoder split blurs with end-to-end diffusion and flow-matching models. But the mental model of three parts still holds for debugging.

## The Concept

**Tacotron 2 (2017).** Seq2seq: char-embedding → BiLSTM encoder → location-sensitive attention → autoregressive LSTM decoder emits mel frames. Slow (AR), wobbly on long text. Still cited as a baseline.

**FastSpeech 2 (2020).** Non-autoregressive. Duration predictor outputs how many mel frames each phoneme gets. 1-pass, 10× faster than Tacotron. Loses some naturalness (monotonic alignment) but ships everywhere.

**VITS (2021).** Jointly trains encoder + flow-based duration + HiFi-GAN vocoder end-to-end with variational inference. High quality, single model. Dominant open-source TTS 2022–2024. Variants: YourTTS (multi-speaker zero-shot), XTTS v2 (2024, Coqui).

**F5-TTS (2024).** Diffusion transformer over flow matching. Natural prosody, zero-shot voice cloning with 5 seconds of reference audio. Top of the 2026 open-source TTS leaderboards. 335M params.

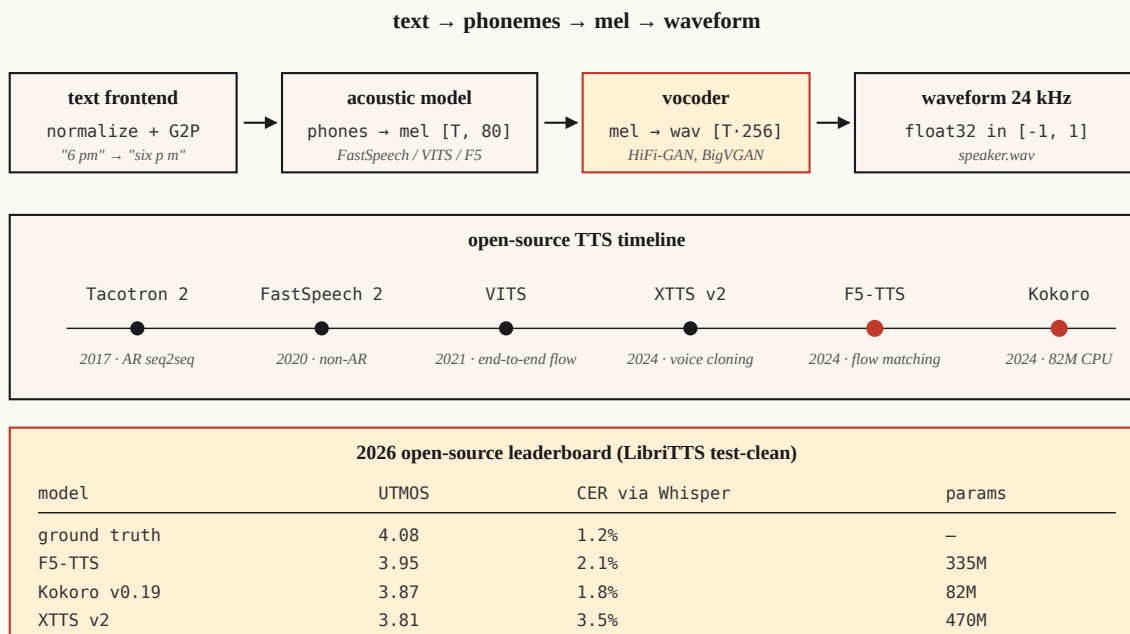


Figure 7: Tacotron, FastSpeech, VITS, F5/Kokoro side-by-side

**Kokoro (2024).** Small (82M), CPU-runnable, best-in-class English TTS for real-time use. Closed-vocabulary English-only, apache-2.0.

**OpenAI TTS-1-HD, ElevenLabs v2.5, Google Chirp-3.** Commercial state of the art. ElevenLabs v2.5 emotion tags (“[whispered]”, “[laughing]”) and character voices dominate audiobook production in 2026.

## Vocoder evolution

Era	Vocoder	Latency	Quality
2016	WaveNet	offline only	SOTA at release
2018	WaveRNN	~realtime	good
2020	HiFi-GAN	100× realtime	near-human
2022	BigVGAN	50× realtime	generalizes across speakers/langs
2024	SNAC, DAC (neural codecs)	integrated with AR models	discrete tokens, bit-efficient

By 2026 most “TTS” models are end-to-end from text to waveform; the mel spectrogram is an internal representation.

## Evaluation

- **MOS (Mean Opinion Score).** 1–5 scale, crowd-sourced. Still the gold standard; painfully slow.
- **CMOS (Comparative MOS).** A-vs-B preference. Tighter confidence intervals per annotation.

- **UTMOS, DNSMOS.** Reference-free neural MOS predictors. Used for leaderboards.
- **CER (Character Error Rate) via ASR.** Run TTS output through Whisper, compute CER against the input text. Proxy for intelligibility.
- **SECS (Speaker Embedding Cosine Similarity).** Voice-cloning quality.

2026 numbers on LibriTTS test-clean:

Model	UTMOS	CER (via Whisper)	Size
Ground truth	4.08	1.2%	—
F5-TTS	3.95	2.1%	335M
XTTS v2	3.81	3.5%	470M
VITS	3.62	3.1%	25M
Kokoro v0.19	3.87	1.8%	82M
Parler-TTS	3.76	2.8%	2.3B
Large			

**Interactive figure: sp-tts-stack.** This one moves. Watch it animate and drag its controls in the web edition: <https://aiengineeringfromscratch.com/lesson?path=phases/06-speech-and-audio/07-text-to-speech>

## Build It

### Step 1: phonemize input

```
from phonemizer import phonemize
ph = phonemize("Hello world", language="en-us", backend="espeak")
# 'həloʊ wɜːld'
```

Phonemes are the universal bridge. Avoid feeding raw text to anything below VITS-level quality.

### Step 2: run Kokoro (2026 CPU default)

```
from kokoro import KPipeline
tts = KPipeline(lang_code="a") # "a" = American English
audio, sr = tts("Please remind me to water the plants at 6 pm.", voice="af_bella")
# audio: float32 tensor, sr=24000
```

Runs offline, single file, 82M params.

### Step 3: run F5-TTS with voice cloning

```
from f5_tts.api import F5TTS
tts = F5TTS()
wav = tts.infer(
    ref_file="my_voice_5s.wav",
    ref_text="The quick brown fox jumps over the lazy dog.",
    gen_text="Please remind me to water the plants.",
)
```

Pass a 5-second reference clip + its transcript; F5 clones prosody and timbre.

## Step 4: HiFi-GAN vocoder from scratch

Too big to fit in a tutorial script, but the shape is:

```
class HiFiGAN(nn.Module):
    def __init__(self, mel_channels=80, upsample_rates=[8, 8, 2, 2]):
        super().__init__()
        # 4 upsample blocks, total 256x to go from mel-rate to audio-rate
        ...
    def forward(self, mel):
        return self.blocks(mel) # -> waveform
```

Training: adversarial (discriminator on short windows) + mel-spectrogram reconstruction loss + feature-matching loss. Commoditized — use pretrained checkpoints from hifi-gan repo or nvidia-NeMo.

## Step 5: the full pipeline (pseudocode)

```
text = "Please remind me at 6 pm."
phones = phonemize(text)
mel = acoustic_model(phones, speaker=alice) # [T, 80]
wav = vocoder(mel) # [T * 256]
soundfile.write("out.wav", wav, 24000)
```

## Use It

The 2026 stack:

Situation	Pick
Real-time English voice assistant	Kokoro (CPU) or XTTS v2 (GPU)
Voice cloning from 5 s reference	F5-TTS
Commercial character voices	ElevenLabs v2.5
Audiobook narration	ElevenLabs v2.5 or XTTS v2 + fine-tune
Low-resource language	Train VITS on 5–20 h target-lang data
Expressive / emotion tags	ElevenLabs v2.5 or StyleTTS 2 fine-tune

Open-source leader as of 2026: **F5-TTS for quality, Kokoro for efficiency**. Don't reach for Tacotron unless you are a historian.

## Pitfalls

- **No text normalizer.** “Dr. Smith” reads as “Doctor” or “Drive”? “2026” as “twenty twenty six” or “two zero two six”? Normalize BEFORE phonemizer.
- **OOV proper nouns.** “Ghumare” → “ghyu-mair”? Ship a fallback grapheme-to-phoneme model for unknown tokens.
- **Clipping.** Vocoder output rarely clips, but mel scaling mismatch at inference can overshoot  $\pm 1.0$ . Always `np.clip(wav, -1, 1)`.

- **Sample-rate mismatch.** Kokoro outputs 24 kHz; your downstream pipeline expects 16 kHz → resample or get aliasing.

**This chapter ships an artifact.** The course version of this lesson produces a reusable prompt or agent skill. It lives in the repository, ready to install: <https://github.com/rohitg00/ai-engineering-from-scratch/tree/main/phases/06-speech-and-audio/07-text-to-speech>

## Exercises

Starter code and the lesson’s working implementation: <https://github.com/rohitg00/ai-engineering-from-scratch/tree/main/phases/06-speech-and-audio/07-text-to-speech/code>

1. **Easy.** Run `code/main.py`. Builds a phoneme dictionary from a toy vocab, estimates duration per phoneme, and prints a fake “mel” schedule.
2. **Medium.** Install Kokoro, synthesize the same sentence at voice `af_bella` and `am_adam`. Compare audio durations and subjective quality.
3. **Hard.** Record a 5-second reference clip of yourself. Use F5-TTS to clone it. Report SECS between reference and cloned output.

## Key Terms

Term	What people say	What it actually means
Phoneme	Sound unit	Abstract sound class; 39 in English (ARPABet).
Duration predictor	How long each phoneme lasts	Non-AR model output; integer frames per phoneme.
Vocoder	Mel → waveform	Neural net mapping mel-spec to raw samples.
HiFi-GAN	Standard vocoder	GAN-based; dominant 2020–2024.
MOS	Subjective quality	1–5 mean opinion score from human raters.
SECS	Voice-clone metric	Cosine similarity between target and output speaker embedding.
F5-TTS	2024 open-source SOTA	Flow-matching diffusion; zero-shot cloning.
Kokoro	CPU English leader	82M-param model, Apache 2.0.

## Further Reading

- Shen et al. (2017). Tacotron 2 — the seq2seq baseline.
- Kim, Kong, Son (2021). VITS — end-to-end flow-based.
- Chen et al. (2024). F5-TTS — current open-source SOTA.
- Kong, Kim, Bae (2020). HiFi-GAN — the vocoder that still ships in 2026.
- Kokoro-82M on HuggingFace — 2024 CPU-friendly English TTS.

**Continue online.** The living edition of this chapter has more than the page can hold:

- Animated, interactive figures and the web text: <https://aiengineeringfromscratch.com/lesson?path=phases/06-speech-and-audio/07-text-to-speech>
- Runnable code for every step: <https://github.com/rohitg00/ai-engineering-from-scratch/tree/main/phases/06-speech-and-audio/07-text-to-speech/code>

The repository moves faster than any printing. When the book and the repo disagree, trust the repo.

# Voice Cloning & Voice Conversion

---

Voice cloning reads your text in someone else's voice. Voice conversion rewrites your voice into someone else's while preserving what you said. Both hang on the same decomposition: separate speaker identity from content.

**Type:** Build **Languages:** Python **Prerequisites:** Phase 6 · 06 (Speaker Recognition), Phase 6 · 07 (TTS) **Time:** ~75 minutes

## The Problem

In 2026, a 5-second audio clip is enough to produce a high-quality clone of anyone's voice with a consumer GPU. ElevenLabs, F5-TTS, OpenVoice v2, VoiceBox all ship zero-shot or few-shot cloning. The technology is a blessing (accessibility TTS, dubbing, assistive voices) and a weapon (scam calls, political deepfakes, IP theft).

Two closely-related tasks:

- **Voice cloning (TTS-side):** text + 5-second reference voice → audio in that voice.
- **Voice conversion (speech-side):** source audio (person A saying X) + reference voice of person B → audio of B saying X.

Both factor a waveform into (content, speaker, prosody) and recombine content from one source with speaker from another.

Key constraint you now ship under in 2026: **watermarking and consent gates are legally required in the EU (AI Act, enforceable August 2026) and in California (AB 2905, effective 2025)**. Your pipeline must emit an inaudible watermark and refuse non-consensual clones.

## The Concept

**Zero-shot cloning.** Pass a 5-second clip to a model that has been trained on thousands of speakers. The speaker encoder maps the clip to a speaker embedding; the TTS decoder conditions on that embedding plus text.

Used by: F5-TTS (2024), YourTTS (2022), XTTS v2 (2024), OpenVoice v2 (2024).

**Few-shot fine-tuning.** Record 5-30 minutes of the target voice. LoRA-fine-tune a base model for an hour. Quality leaps from "okay" to "indistinguishable". Coqui and ElevenLabs both support this pattern; community uses it with F5-TTS.

**Voice conversion (VC).** Two families:

- **Recognition-synthesis.** Run ASR-like model to extract content representation (e.g., soft phoneme posteriors, PPGs), then resynthesize with target speaker embedding. Robust to language and accent. Used by KNN-VC (2023), Diff-HierVC (2023).
- **Disentanglement.** Train an autoencoder that separates content, speaker, and prosody in latent space at the bottleneck. Swap speaker embedding at inference. Lower quality but faster. Used by AutoVC (2019), VITS-VC variants.



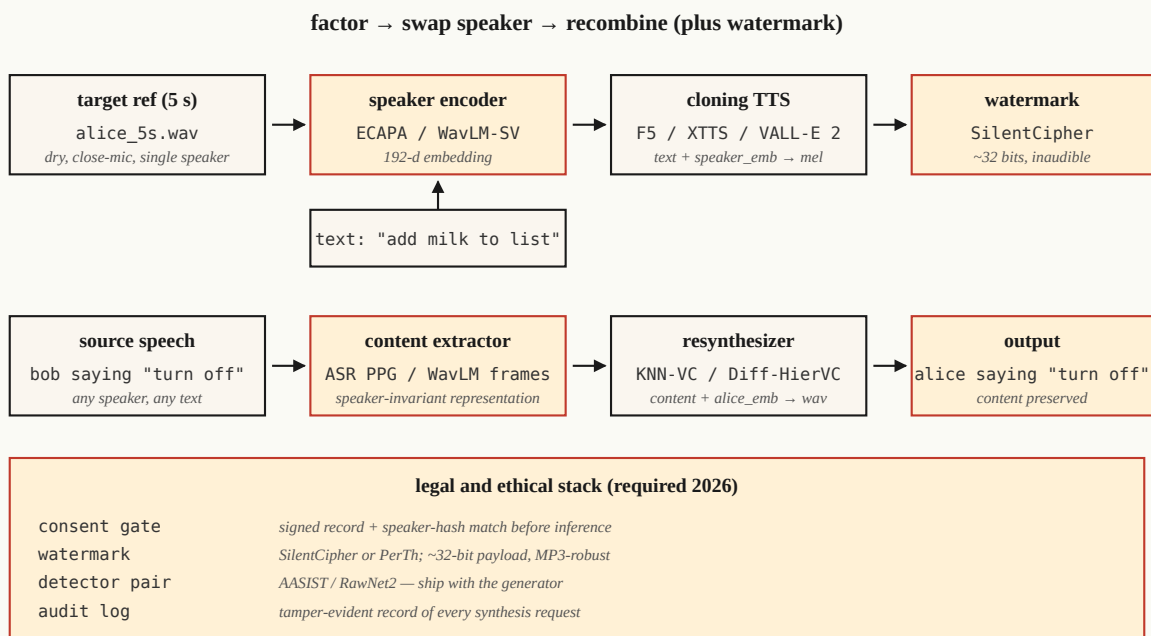


Figure 8: Voice cloning vs conversion: factorize, swap speaker, recombine

**Neural codec-based cloning (2024+).** VALL-E, VALL-E 2, NaturalSpeech 3, VoiceBox — treat audio as discrete tokens from SoundStream / EnCodec, train a large autoregressive or flow-matching model over codec tokens. Quality comparable to ElevenLabs on short prompts.

## The ethics bit, not a bolt-on

**Watermarking.** PerTh (Perth) and SilentCipher (2024) embed a ~16-32 bit ID imperceptibly in the audio. Survives re-encoding, streaming, and common edits. Production-ready open source.

**Consent gates.** Must pair every cloned output with a verifiable consent record. “I, Rohit, on 2026-04-22, authorize this voice for X purpose.” Store in a tamper-evident log.

**Detection.** AASIST, RawNet2, and Wav2Vec2-AASIST ship as detectors. ASVspoof 2025 challenge published EERs of 0.8–2.3% for state-of-the-art detectors against ElevenLabs, VALL-E 2, and Bark outputs.

## Numbers (2026)

Model	Zero-shot?	SECS (target sim)	WER (intel.)	Params
F5-TTS	Yes	0.72	2.1%	335M
XTTS v2	Yes	0.65	3.5%	470M
OpenVoice v2	Yes	0.70	2.8%	220M
VALL-E 2	Yes	0.77	2.4%	370M
VoiceBox	Yes	0.78	2.1%	330M

SECS > 0.70 is generally indistinguishable from the target for most listeners.

**Interactive figure: sp-voice-factorize.** This one moves. Watch it animate and drag its controls in the web edition: <https://aiengineeringfromscratch.com/lesson?path=phases/06-speech-and-audio/08-voice-cloning-conversion>

## Build It

### Step 1: decompose with recognition-synthesis (code-only demo in main.py)

```
def clone_pipeline(ref_audio, text, target_embedder, tts_model):
    speaker_emb = target_embedder.encode(ref_audio)
    mel = tts_model(text, speaker=speaker_emb)
    return vocoder(mel)
```

Conceptually simple; implementation mass is in `tts_model` and speaker encoder.

### Step 2: zero-shot clone with F5-TTS

```
from f5_tts.api import F5TTS
tts = F5TTS()
wav = tts.infer(
    ref_file="rohit_5s.wav",
    ref_text="The quick brown fox jumps over the lazy dog.",
    gen_text="Please add milk and bread to my list.",
)
```

Reference transcript must exactly match the audio; mismatch breaks alignment.

### Step 3: voice conversion with KNN-VC

```
import torch
from knnvc import KNNVC # 2023 model, https://github.com/bshall/knn-vc
vc = KNNVC.load("wavlm-base-plus")
out_wav = vc.convert(source="my_voice.wav", target_pool=["alice_1.wav",
    ↪ "alice_2.wav"])
```

KNN-VC runs WavLM to extract per-frame embeddings for source and target pool, then replaces each source frame with its nearest neighbor in the pool. Non-parametric, works with a minute of target speech.

### Step 4: embed a watermark

```
from silentcipher import SilentCipher
sc = SilentCipher(model="2024-06-01")
payload = b"consent_id:abc123;ts:1745353200"
watermarked = sc.embed(wav, sr=24000, message=payload)
detected = sc.detect(watermarked, sr=24000) # returns payload bytes
```

~32 bits of payload, detectable after MP3 re-encode and light noise.

### Step 5: consent gate

```
def cloned_inference(text, ref_audio, consent_record):
    assert verify_signature(consent_record), "Signed consent required"
```

```

assert consent_record["speaker_id"] == hash_speaker(ref_audio)
wav = tts.infer(ref_file=ref_audio, gen_text=text)
wav = watermark(wav, payload=consent_record["id"])
return wav

```

## Use It

The 2026 stack:

Situation	Pick
5-sec zero-shot clone, open-source	F5-TTS or OpenVoice v2
Commercial production cloning	ElevenLabs Instant Voice Clone v2.5
Voice conversion (rewriting)	KNN-VC or Diff-HierVC
Many-speaker fine-tune	StyleTTS 2 + speaker adapter
Cross-lingual cloning	XTTS v2 or VALL-E X
Deepfake detection	Wav2Vec2-AASIST

## Pitfalls

- **Misaligned reference transcript.** F5-TTS and similar require the reference text to match the reference audio exactly, punctuation included.
- **Reverberant reference.** Echo kills the clone. Record dry, close-mic.
- **Emotional mismatch.** Training reference “cheerful” produces cheerful clones of everything. Match reference emotion to target use.
- **Language leakage.** Cloning an English speaker then asking the model to speak French often carries the accent anyway; use cross-lingual models (XTTS, VALL-E X).
- **No watermark.** Legally unshippable in EU from Aug 2026.

**This chapter ships an artifact.** The course version of this lesson produces a reusable prompt or agent skill. It lives in the repository, ready to install: <https://github.com/rohitg00/ai-engineering-from-scratch/tree/main/phases/06-speech-and-audio/08-voice-cloning-conversion>

## Exercises

Starter code and the lesson’s working implementation: <https://github.com/rohitg00/ai-engineering-from-scratch/tree/main/phases/06-speech-and-audio/08-voice-cloning-conversion/code>

1. **Easy.** Run `code/main.py`. Demonstrates the speaker-embedding swap by computing the cosine between two “speakers” pre and post swap.
2. **Medium.** Use OpenVoice v2 to clone your own voice. Measure SECS between reference and clone. Measure CER via Whisper.
3. **Hard.** Apply SilentCipher watermark to 20 clones, run them through 128 kbps MP3 encode+decode, detect the payload. Report bit-accuracy.

## Key Terms

Term	What people say	What it actually means
Zero-shot clone	5 seconds is enough	Pretrained model + speaker embedding; no training.
PPG	Phonetic posteriorgram	Per-frame ASR posteriors used as language-agnostic content rep.
KNN-VC	Nearest-neighbor conversion	Replace each source frame with nearest target-pool frame.
Neural codec TTS	VALL-E style	AR model over EnCodec/SoundStream tokens.
Watermark SECS	Inaudible signature Cloning fidelity	Bits embedded in audio, survive re-encode. Cosine between target and clone speaker embeddings.
AASIST	Deepfake detector	Anti-spoof model; detects synthesized speech.

## Further Reading

- Chen et al. (2024). F5-TTS — open-source SOTA zero-shot cloning.
- Baevski et al. / Microsoft (2023). VALL-E and VALL-E 2 (2024) — neural-codec TTS.
- Qian et al. (2019). AutoVC — disentanglement-based voice conversion.
- Baas, Waubert de Puiseau, Kamper (2023). KNN-VC — retrieval-based VC.
- SilentCipher (2024) — Audio Watermarking — production-ready 32-bit audio watermark.
- ASVspoof 2025 results — detector vs synthesizer arms race, updated 2026.

**Continue online.** The living edition of this chapter has more than the page can hold:

- Animated, interactive figures and the web text: <https://aiengineeringfromscratch.com/lesson?path=phases/06-speech-and-audio/08-voice-cloning-conversion>
- Runnable code for every step: <https://github.com/rohitg00/ai-engineering-from-scratch/tree/main/phases/06-speech-and-audio/08-voice-cloning-conversion/code>

The repository moves faster than any printing. When the book and the repo disagree, trust the repo.

# Music Generation — MusicGen, Stable Audio, Suno, and the Licensing Earthquake

---

2026 music generation: Suno v5 and Udio v4 dominate commercial; MusicGen, Stable Audio Open, and ACE-Step lead open-source. The technical problem is mostly solved. The legal problem (Warner Music \$500M settlement, UMG settlement) reshaped the field in 2025-2026.

**Type:** Build **Languages:** Python **Prerequisites:** Phase 6 · 02 (Spectrograms), Phase 4 · 10 (Diffusion Models) **Time:** ~75 minutes

## The Problem

Text → a 30-second to 4-minute music clip, with lyrics, vocals, and structure. Three sub-problems:

1. **Instrumental generation.** Text like “lo-fi hip-hop drums with warm keys” → audio. MusicGen, Stable Audio, AudioLDM.
2. **Song generation (with vocals + lyrics).** “Country song about rainy Texas nights” → full song. Suno, Udio, YuE, ACE-Step.
3. **Conditional / controllable.** Extend an existing clip, regenerate a bridge, swap genre, stem-separate, or inpaint. Udio’s inpainting + stem separation is the 2026 feature to match.

## The Concept

### Token LM over neural-codec tokens

Meta’s **MusicGen** (2023, MIT) and many derivatives: condition on text/melody embeddings, autoregressively predict EnCodec tokens (32 kHz, 4 codebooks), decode with EnCodec. 300M - 3.3B params. Strong baseline; struggles past 30 seconds.

**ACE-Step** (open-source, 4B XL released April 2026) extends this for full-song lyric-conditioned generation. The open community’s closest thing to Suno.

### Diffusion over mels or latents

**Stable Audio (2023)** and **Stable Audio Open (2024)**: latent diffusion on compressed audio. Excels at loops, sound design, ambient textures. Not great at structured full songs.

**AudioLDM / AudioLDM2**: text-to-audio via T2I-style latent diffusion, generalized to music, sound effects, speech.

## music generation — two paradigms, one legal landscape

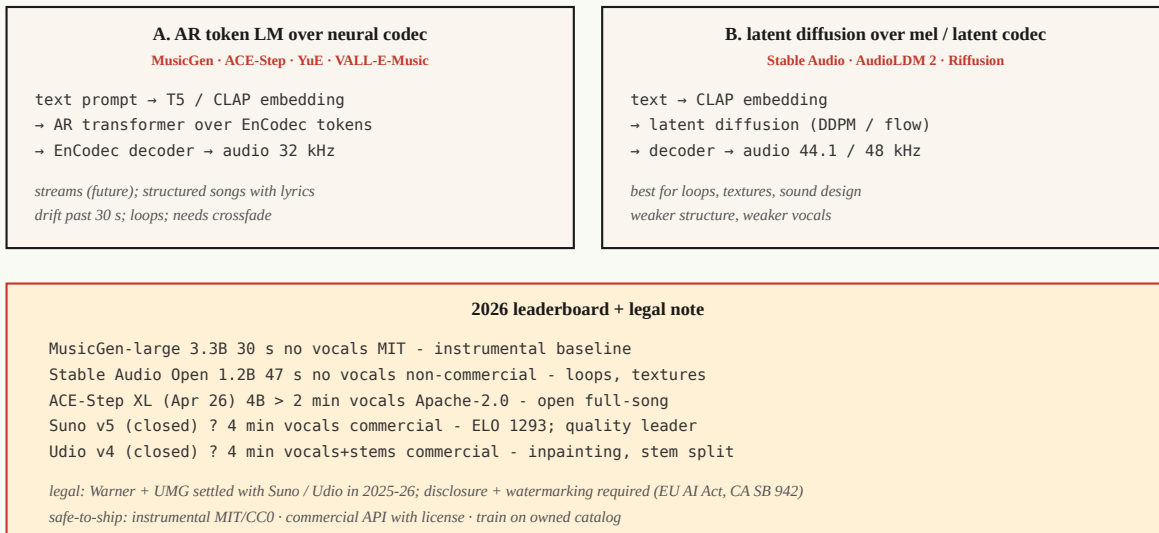


Figure 9: Music generation: token-LM vs diffusion, the 2026 model map

## Hybrid (production) — Suno, Udio, Lyria

Closed weights. Likely AR codec LM + diffusion-based vocoder with specialized voice / drum / melody heads. Suno v5 (2026) is the ELO 1293 quality leader. Udio v4 adds inpainting + stem separation (bass, drums, vocals separate downloads).

## Evaluation

- **FAD (Fréchet Audio Distance).** Embedding-level distance between generated vs real audio distribution using VGGish or PANNs features. Lower is better. MusicGen small: 4.5 FAD on MusicCaps; SOTA ~3.0.
- **Musicality (subjective).** Human preference. Suno v5 ELO 1293 leads.
- **Text-audio alignment.** CLAP score between prompt and output.
- **Musicality artifacts.** Off-beat transitions, vocal-phrase drift, loss of structure past 30 s.

## 2026 model map

Model	Params	Length	Vocals	License
MusicGen-large	3.3B	30 s	no	MIT
Stable Audio Open	1.2B	47 s	no	Stability non-commercial
ACE-Step XL (Apr 2026)	4B	> 2 min	yes	Apache-2.0
YuE	7B	> 2 min	yes, multilingual	Apache-2.0
Suno v5 (closed)	?	4 min	yes, ELO 1293	commercial

Model	Params	Length	Vocals	License
Udio v4 (closed)	?	4 min	yes + stems	commercial
Google Lyria 3 (closed)	?	real-time	yes	commercial
MiniMax Music 2.5	?	4 min	yes	commercial API

## The legal landscape (2025-2026)

- **Warner Music vs Suno settlement.** \$500M. WMG now has oversight of AI-likeness, music rights, and user-generated tracks on Suno. Similar UMG settlement on Udio.
- **EU AI Act + California SB 942:** AI-generated music must be disclosed.
- **Riffusion / MusicGen** under MIT have no compliance baggage but also no commercial vocals.

Safe-to-ship patterns:

1. Generate instrumental only (MusicGen, Stable Audio Open, MIT/CC0 outputs).
2. Use commercial APIs (Suno, Udio, ElevenLabs Music) with per-generation license.
3. Train on owned or licensed catalog (most enterprises end up here).
4. Tag generations with watermarks + metadata.

**Interactive figure: sp-codec-tokens.** This one moves. Watch it animate and drag its controls in the web edition: <https://aiengineeringfromscratch.com/lesson?path=phases/06-speech-and-audio/09-music-generation>

## Build It

### Step 1: generate with MusicGen

```
from audiocraft.models import MusicGen
import torchaudio

model = MusicGen.get_pretrained("facebook/musicgen-small")
model.set_generation_params(duration=10)
wav = model.generate(["upbeat synthwave with driving drums, 128 BPM"])
torchaudio.save("out.wav", wav[0].cpu(), 32000)
```

Three sizes: small (300M, fast), medium (1.5B), large (3.3B). Small is enough for “does the idea land.”

### Step 2: melody conditioning

```
melody, sr = torchaudio.load("humming.wav")
wav = model.generate_with_chroma(
    ["jazz piano cover"],
    melody.squeeze(),
    sr,
)
```

MusicGen-melody takes a chromagram and preserves the tune while swapping timbre. Useful for “give me this melody as a string quartet.”

### Step 3: FAD evaluation

```
from frechet_audio_distance import FrechetAudioDistance
fad = FrechetAudioDistance()
```

```
fad.get_fad_score("generated_folder/", "reference_folder/")
```

Computes VGGish-embedding distance. Useful for genre-level regression tests; not a substitute for human listeners.

### Step 4: adding to the LLM-music workflow

Combine with the ideas from Lessons 7-8:

```
prompt = "Write a 30-second jazz loop. Describe the drums, bass, and piano  
↪ voicing."  
description = llm.complete(prompt)  
music = musicgen.generate([description], duration=30)
```

## Use It

Goal	Stack
Instrumental sound design	Stable Audio Open
Game / adaptive music	Google Lyria RealTime (closed)
Full songs with vocals (commercial)	Suno v5 or Udio v4 with explicit license
Full songs with vocals (open)	ACE-Step XL or YuE
Short ad jingle	MusicGen melody-conditioned on a hummed reference
Music-video background	MusicGen + Stable Video Diffusion

## Pitfalls that still ship in 2026

- **Copyright-laundering prompts.** “Song in the style of Taylor Swift” — commercial Suno/Udio filter these now, open models do not. Add your own filter list.
- **Repetition / drift past 30 s.** AR models loop. Crossfade multiple generations, or use ACE-Step for structural coherence.
- **Tempo drift.** Models wander off the BPM. Use BPM tags in the prompt and post-filter with librosa’s `beat_track`.
- **Vocal intelligibility.** Suno is excellent; open models are often mushy on words. If lyrics matter, use a commercial API or fine-tune.
- **Mono output.** Open models generate mono or fake-stereo. Upgrade with a proper stereo reconstruction (ezst, Cartesia’s stereo diffusion).

**This chapter ships an artifact.** The course version of this lesson produces a reusable prompt or agent skill. It lives in the repository, ready to install: <https://github.com/rohitg00/ai-engineering-from-scratch/tree/main/phases/06-speech-and-audio/09-music-generation>

## Exercises

Starter code and the lesson’s working implementation: <https://github.com/rohitg00/ai-engineering-from-scratch/tree/main/phases/06-speech-and-audio/09-music-generation/code>



1. **Easy.** Run `code/main.py`. It produces a “generative” chord progression + drum pattern as ASCII symbols — a music-gen cartoon. Play it back via any MIDI renderer if you want.
2. **Medium.** Install `audiocraft`, generate 10-second clips across 4 genre prompts with `MusicGen-small`, measure FAD against a reference genre set.
3. **Hard.** Using `ACE-Step` (or `MusicGen-melody`), generate three variations of the same tune with different timbre prompts. Compute CLAP similarity to the prompt to verify alignment.

## Key Terms

Term	What people say	What it actually means
FAD	Audio FID	Fréchet distance between embedding distributions of real vs generated.
Chromagram	Melody as pitches	12-dim per-frame vector; input to melody conditioning.
Stems	Instrument tracks	Separated bass / drums / vocals / melody as WAV.
Inpainting	Regen a section	Mask a time window; model regenerates just that.
CLAP	Text-audio CLIP	Contrastive audio-text embedding; eval text-audio alignment.
EnCodec	Music codec	Meta’s neural codec used by MusicGen; 32 kHz, 4 codebooks.

## Further Reading

- Copet et al. (2023). MusicGen — the open autoregressive benchmark.
- Evans et al. (2024). Stable Audio Open — the sound-design default.
- ACE-Step — open 4B full-song generator, April 2026.
- Suno v5 platform docs — the commercial quality leader.
- AudioLDM2 — latent diffusion for music + sound effects.
- WMG-Suno settlement coverage — Nov 2025 precedent.

**Continue online.** The living edition of this chapter has more than the page can hold:

- Animated, interactive figures and the web text: <https://aiengineeringfromscratch.com/lesson?path=phases/06-speech-and-audio/09-music-generation>
- Runnable code for every step: <https://github.com/rohitg00/ai-engineering-from-scratch/tree/main/phases/06-speech-and-audio/09-music-generation/code>

The repository moves faster than any printing. When the book and the repo disagree, trust the repo.

# Audio-Language Models — Qwen2.5-Omni, Audio Flamingo, GPT-4o Audio

---

2026 audio-language models reason over speech + environmental sound + music. Qwen2.5-Omni-7B matches GPT-4o Audio on MMAU-Pro. Audio Flamingo Next beats Gemini 2.5 Pro on LongAudioBench. The gap between open and closed is essentially closed — except on multi-audio tasks, where everyone is near random.

**Type:** Learn **Languages:** Python **Prerequisites:** Phase 6 · 04 (ASR), Phase 12 · 03 (Vision-Language Models), Phase 7 · 10 (Audio Transformers) **Time:** ~45 minutes

## The Problem

You have 5 seconds of audio: dog barks, someone yells “stop!”, then silence. Useful questions span multiple axes:

- **Transcription.** “What was said?” — ASR territory.
- **Semantic reasoning.** “Is the person in danger?” — requires joint understanding of the bark + yell + silence.
- **Music reasoning.** “What instruments play the melody?”
- **Long-audio retrieval.** “Where in this 90-minute lecture did the instructor explain gradient descent?”

A single model that answers all of these with one prompt is an **audio-language model** (LALM / ALM). Separate from pure ASR: LALMs produce free-form natural-language answers, not just transcripts.

## The Concept

### The three-component template

Every 2026 LALM has the same skeleton:

1. **Audio encoder.** Whisper encoder · BEATs · CLAP · WavLM · or a custom encoder per model.
2. **Projector.** Linear or MLP bridging audio-encoder features into the LLM’s token embedding space.
3. **LLM.** Llama / Qwen / Gemma-based decoder. Takes interleaved text + audio tokens; generates text.

Training:

- **Stage 1.** Freeze encoder + LLM; train projector only on ASR / captioning data.
- **Stage 2.** Full / LoRA fine-tune on instruction-following audio tasks (QA, reasoning, music understanding).
- **Stage 3 (optional).** Voice-in / voice-out adds a speech decoder. Qwen2.5-Omni and AF3-Chat do this.

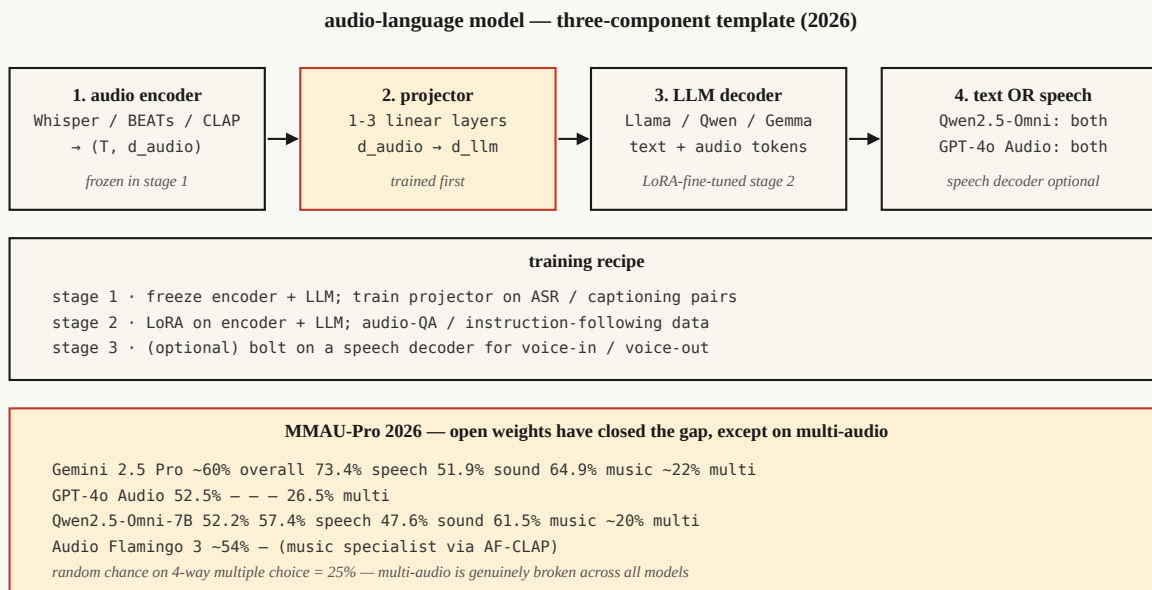


Figure 10: Audio-language model: audio encoder + projector + LLM decoder

## The 2026 model map

Model	Backbone	Audio encoder	Output modality	Access
Qwen2.5-Omni-7B	Qwen2.5-7B	Custom + Whisper	text + speech	Apache-2.0
Qwen3-Omni	Qwen3	Custom	text + speech	Apache-2.0
Audio Flamingo 3	Qwen2	AF-CLAP	text	NVIDIA non-commercial
Audio Flamingo Next	Qwen2	AF-CLAP v2	text	NVIDIA non-commercial
SALMONN	Vicuna	Whisper + BEATs	text	Apache-2.0
LTU / LTU-AS	Llama	CAV-MAE	text	Apache-2.0
GAMA	Llama	AST + Q-Former	text	Apache-2.0
Gemini 2.5 Flash/Pro (closed)	Gemini	proprietary	text + speech	API
GPT-4o Audio (closed)	GPT-4o	proprietary	text + speech	API

## Benchmark reality check (2026)

**MMAU-Pro.** 1800 QA pairs covering speech / sound / music / mixed. Multi-audio subset included.

Model	Overall	Speech	Sound	Music	Multi-audio
Gemini 2.5 Pro	~60%	73.4%	51.9%	64.9%	~22%
Gemini 2.5 Flash	~57%	73.4%	50.5%	64.9%	21.2%
GPT-4o Audio	52.5%	—	—	—	26.5%
Qwen2.5-Omni-7B Audio	52.2%	57.4%	47.6%	61.5%	~20%
Flamingo 3	~54%	—	—	—	—
Audio Flamingo Next	SOTA on LongAudioBench	—	—	—	—

The **multi-audio column is damning for everyone**. Random chance on 4-option multiple choice = 25%; most models score around there. LALMs still struggle to compare two clips.

## Where LALMs are useful in 2026

- **Compliance audit of call-center recordings.** “Did the agent mention the required disclosure?”
- **Accessibility.** Describe sound events to deaf users (not just transcription).
- **Content moderation.** Detect violent language + threatening tone + background context.
- **Podcast / meeting chaptering.** Semantic summary, not just speaker turns.
- **Music catalog analysis.** “Find all tracks with a B-section key change.”

## Where they are NOT (yet) useful

- Fine-grained music theory (below chord-level).
- Speaker-attributed reasoning over long conversations (degrades past 10 minutes).
- Multi-audio comparison (22-26% is barely above random).
- Real-time streaming reasoning (most are offline batch inference).

**Interactive figure: v4-alm-tokens.** This one moves. Watch it animate and drag its controls in the web edition: <https://aiengineeringfromscratch.com/lesson?path=phases/06-speech-and-audio/10-audio-language-models>

## Build It

### Step 1: query Qwen2.5-Omni

```
from transformers import AutoModelForCausalLM, AutoProcessor

processor = AutoProcessor.from_pretrained("Qwen/Qwen2.5-Omni-7B")
model = AutoModelForCausalLM.from_pretrained("Qwen/Qwen2.5-Omni-7B",
    ↪ torch_dtype="auto")

audio, sr = load_wav("clip.wav", sr=16000)
```

```

messages = [{
    "role": "user",
    "content": [
        {"type": "audio", "audio": audio},
        {"type": "text", "text": "What sounds do you hear, and what's happening?"},
    ],
}]
inputs = processor.apply_chat_template(messages, tokenize=True,
    ↪ return_tensors="pt")
output = model.generate(**inputs, max_new_tokens=200)
print(processor.decode(output[0], skip_special_tokens=True))

```

## Step 2: the projector pattern

```

import torch.nn as nn

class AudioProjector(nn.Module):
    def __init__(self, audio_dim=1280, llm_dim=4096):
        super().__init__()
        self.down = nn.Linear(audio_dim, llm_dim)
        self.act = nn.GELU()
        self.up = nn.Linear(llm_dim, llm_dim)

    def forward(self, audio_features):
        return self.up(self.act(self.down(audio_features)))

```

That's it. The projector is usually 1-3 linear layers. Training it on ASR pairs (audio → transcript) is the Stage-1 pretext task.

## Step 3: benchmarking MMAU / LongAudioBench

```

from datasets import load_dataset
mmau = load_dataset("MMAU/MMAU-Pro")

correct = 0
for item in mmau["test"]:
    answer = call_model(item["audio"], item["question"], item["choices"])
    if answer == item["correct_choice"]:
        correct += 1
print(f"Accuracy: {correct / len(mmau['test']):.3f}")

```

Report per-category (speech / sound / music / multi-audio) separately. Aggregate numbers hide where the model fails.

## Use It

Task	2026 pick
Free-form audio QA (open)	Qwen2.5-Omni-7B
Best open on long audio	Audio Flamingo Next
Best closed	Gemini 2.5 Pro
Voice-in / voice-out agent	Qwen2.5-Omni or GPT-4o Audio
Music reasoning	Audio Flamingo 3 or 2 (music-specialized AF-CLAP)

Task	2026 pick
Call-center audit	Gemini 2.5 Pro via API, with RAG over your policy docs

## Pitfalls

- **Over-trust on multi-audio.** If your task needs “which clip has X,” random-chance-level performance is real.
- **Long-audio degradation.** Past 10 minutes, most models’ speaker attribution breaks. Diarize first (Lesson 6), then summarize.
- **Hallucinations on silence.** Same Whisper-style issue inherited by LALMs that use Whisper encoder. VAD-gate.
- **Benchmark cherry-picking.** Vendor blog posts highlight best-case categories. Run MMAU-Pro multi-audio subset yourself.

**This chapter ships an artifact.** The course version of this lesson produces a reusable prompt or agent skill. It lives in the repository, ready to install: <https://github.com/rohitg00/ai-engineering-from-scratch/tree/main/phases/06-speech-and-audio/10-audio-language-models>

## Exercises

Starter code and the lesson’s working implementation: <https://github.com/rohitg00/ai-engineering-from-scratch/tree/main/phases/06-speech-and-audio/10-audio-language-models/code>

1. **Easy.** Run `code/main.py` to see a toy projector pattern + fake LALM routing of (audio-embedding, text-tokens) → output tokens.
2. **Medium.** Score Qwen2.5-Omni-7B on 100 MMAU-Pro speech items. Compare to the paper’s reported number.
3. **Hard.** Build a minimal audio-captioning baseline: BEATs encoder + 2-layer projector + frozen Llama-3.2-1B. Fine-tune only the projector on AudioCaps. Compare to SALMONN on Clotho-AQA.

## Key Terms

Term	What people say	What it actually means
LALM	Audio ChatGPT	Audio encoder + projector + LLM decoder.
Projector	Adapter	Small MLP mapping audio features into LLM embedding space.
MMAU	The benchmark	10k audio-QA pairs across speech, sound, music.
MMAU-Pro	Harder MMAU	1800 multi-audio / reasoning-heavy questions.
LongAudioBenchmark	Long-form eval	Multi-minute clips with semantic queries.
Voice-in / voice-out	Speech-native	Model ingests speech and emits speech without text detour.

## Further Reading

- Chu et al. (2024). Qwen2-Audio — reference architecture.

- Alibaba (2025). Qwen2.5-Omni — speech-in-speech-out.
- NVIDIA (2025). Audio Flamingo 3 — the open long-audio leader.
- NVIDIA (2026). Audio Flamingo Next — LongAudioBench SOTA.
- Tang et al. (2023). SALMONN — dual-encoder pioneer.
- MMAU-Pro leaderboard — live 2026 rankings.

**Continue online.** The living edition of this chapter has more than the page can hold:

- Animated, interactive figures and the web text: <https://aiengineeringfromscratch.com/lesson?path=phases/06-speech-and-audio/10-audio-language-models>
- Runnable code for every step: <https://github.com/rohitg00/ai-engineering-from-scratch/tree/main/phases/06-speech-and-audio/10-audio-language-models/code>

The repository moves faster than any printing. When the book and the repo disagree, trust the repo.

# Real-Time Audio Processing

Batch pipelines process a file. Real-time pipelines process the next 20 milliseconds before the next 20 arrive. Every conversational AI, broadcast studio, and telephony bot lives and dies by this latency budget.

**Type:** Build **Languages:** Python **Prerequisites:** Phase 6 · 02 (Spectrograms), Phase 6 · 04 (ASR), Phase 6 · 07 (TTS) **Time:** ~75 minutes

## The Problem

You want a voice assistant that feels alive. Human conversational turn-taking latency is ~230 ms (silence-to-response). Anything above 500 ms feels robotic; above 1500 ms feels broken. The budget for a full **hear** → **understand** → **respond** → **speak** loop in 2026 is:

Stage	Budget
Mic → buffer	20 ms
VAD	10 ms
ASR (streaming)	150 ms
LLM (first token)	100 ms
TTS (first chunk)	100 ms
Render → speaker	20 ms
<b>Total</b>	<b>~400 ms</b>

Moshi (Kyutai, 2024) clocked 200 ms full-duplex. GPT-4o-realtime (2024) clocks ~320 ms. Cascaded pipelines in 2022 shipped at 2500 ms. The 10× improvement came from three techniques: (1) streaming everywhere, (2) asynchronous pipelining with partial results, (3) interruptible generation.

## The Concept

**Frame / chunk / window.** Real-time audio flows as fixed-size blocks. Common choice: 20 ms (320 samples at 16 kHz). Everything downstream must keep up with this cadence.

**Ring buffer.** Fixed-size circular buffer. Producer thread writes new frames, consumer thread reads. Prevents allocations in the hot path. Size ≈ maximum-latency × sample-rate; a 2-second 16 kHz ring = 32,000 samples.

**VAD (Voice Activity Detection).** Gates downstream work when nobody is speaking. Silero VAD 4.0 (2024) runs <1 ms per 30 ms frame on CPU. webrtcvad is the older alternative.

**Streaming ASR.** Models that emit partial transcripts as audio arrives. Parakeet-CTC-0.6B in streaming mode (NeMo, 2024) does 2-5% WER at 320 ms latency. Whisper-Streaming (Macháček et al., 2023) chunks Whisper for near-streaming at ~2 s latency.



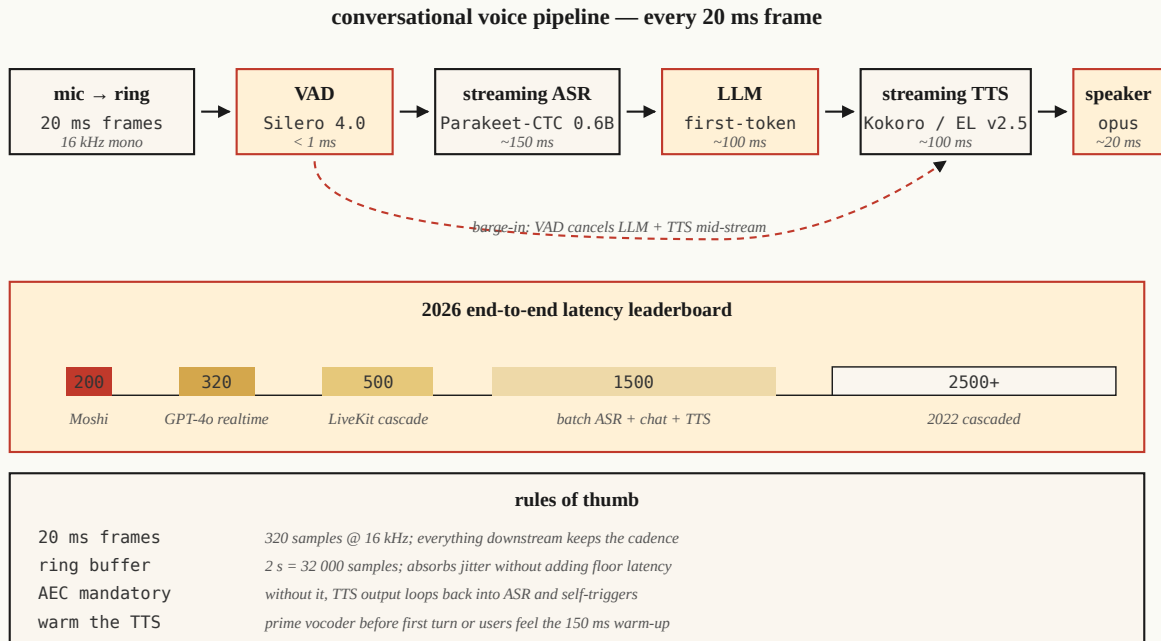


Figure 11: Streaming audio pipeline with ring buffer, VAD gate, interruption

**Interruption.** When the user speaks while the assistant is talking, you must (a) detect the barge-in, (b) stop the TTS, (c) discard the remaining LLM output. All within 100 ms, or the user perceives deaf assistant.

**WebRTC Opus transport.** 20 ms frames, 48 kHz, adaptive bitrate 8–128 kbps. Standard for browser and mobile. LiveKit, Daily.co, Pion are the 2026 stacks for building voice apps.

**Jitter buffer.** Network packets arrive out of order / late. The jitter buffer reorders and smooths; too small → audible gaps, too large → latency. 60–80 ms typical.

## Common gotchas

- **Thread contention.** Python’s GIL + heavy models can starve the audio thread. Use a C-callback audio library (sounddevice, PortAudio) and keep Python off the hot path.
- **Sample-rate conversion latency.** Resampling inside the pipeline adds 5–20 ms. Either resample upfront or use a zero-latency resampler (PolyPhase, soxr_hq).
- **TTS priming.** Even fast TTS like Kokoro has a 100–200 ms warm-up on first request. Cache model + warm it with a dummy run before the first real turn.
- **Echo cancellation.** Without AEC, TTS output re-enters the mic and triggers ASR on the bot’s own voice. WebRTC AEC3 is the open-source default.

**Interactive figure: nyquist-aliasing.** This one moves. Watch it animate and drag its controls in the web edition: <https://aiengineeringfromscratch.com/lesson?path=phases/06-speech-and-audio/11-real-time-audio-processing>

## Build It

### Step 1: ring buffer

```
import collections

class RingBuffer:
    def __init__(self, capacity):
        self.buf = collections.deque(maxlen=capacity)
    def write(self, frame):
        self.buf.extend(frame)
    def read(self, n):
        return [self.buf.popleft() for _ in range(min(n, len(self.buf)))]
    def level(self):
        return len(self.buf)
```

Capacity determines max buffering latency. 32,000 samples at 16 kHz = 2 s.

### Step 2: VAD gate

```
def simple_energy_vad(frame, threshold=0.01):
    return sum(x * x for x in frame) / len(frame) > threshold ** 2
```

Replace with Silero VAD in production:

```
import torch
vad, _ = torch.hub.load("snakers4/silero-vad", "silero_vad")
is_speech = vad(torch.tensor(frame), 16000).item() > 0.5
```

### Step 3: streaming ASR

```
# Parakeet-CTC-0.6B streaming via NeMo
from nemo.collections.asr.models import EncDecCTCModelBPE
asr = EncDecCTCModelBPE.from_pretrained("nvidia/parakeet-ctc-0.6b")
# chunk_ms=320 ms, look_ahead_ms=80 ms
for chunk in audio_stream():
    partial_text = asr.transcribe_streaming(chunk)
    print(partial_text, end="\r")
```

### Step 4: interruption handler

```
class Dialog:
    def __init__(self):
        self.tts_task = None

    def on_user_speech(self, frame):
        if self.tts_task and not self.tts_task.done():
            self.tts_task.cancel() # barge-in
            # then feed to streaming ASR

    def on_final_user_utterance(self, text):
        self.tts_task = asyncio.create_task(self.reply(text))

    async def reply(self, text):
```

```
async for tts_chunk in llm_then_tts(text):
    speaker.write(tts_chunk)
```

Hinges on async I/O and cancellable TTS streaming. WebRTC `peerconnection.stop()` on the audio track is the canonical way.

## Use It

The 2026 stack:

Layer	Pick
Transport	LiveKit (WebRTC) or Pion (Go)
VAD	Silero VAD 4.0
Streaming ASR	Parakeet-CTC-0.6B or Whisper-Streaming
LLM first-token	Groq, Cerebras, vLLM-streaming
Streaming TTS	Kokoro or ElevenLabs Turbo v2.5
Echo cancel	WebRTC AEC3
End-to-end native	OpenAI Realtime API or Moshi

## Pitfalls

- **Buffering 500 ms to be safe.** The buffer is your latency floor. Shrink it.
- **Not pinning threads.** Audio callback on a priority-lower-than-UI thread = glitches under load.
- **TTS chunks too small.** Sub-200 ms chunks make vocoder artifacts audible. 320 ms chunks are the sweet spot.
- **No jitter buffer.** Real networks are jittery; without smoothing you get pops.
- **Single-shot error handling.** Audio pipelines must be crash-proof. One exception kills the session.

**This chapter ships an artifact.** The course version of this lesson produces a reusable prompt or agent skill. It lives in the repository, ready to install: <https://github.com/rohitg00/ai-engineering-from-scratch/tree/main/phases/06-speech-and-audio/11-real-time-audio-processing>

## Exercises

Starter code and the lesson's working implementation: <https://github.com/rohitg00/ai-engineering-from-scratch/tree/main/phases/06-speech-and-audio/11-real-time-audio-processing/code>

1. **Easy.** Run `code/main.py`. Simulates a ring buffer + energy VAD; prints stage latencies for a fake 10-second stream.
2. **Medium.** Using `sounddevice`, build a passthrough loop that processes your mic in 20 ms frames and prints VAD state at each frame.
3. **Hard.** Build a full duplex echo test with `aiortc`: browser → WebRTC → Python → WebRTC → browser. Measure glass-to-glass latency with a 1 kHz pulse.

## Key Terms

Term	What people say	What it actually means
Ring buffer	The circular queue	Fixed-size, lock-free (or SPSC-locked) FIFO for audio frames.
VAD	Silence gate	Model or heuristic marking speech vs non-speech.
Streaming ASR	Real-time STT	Emits partial text as audio arrives; bounded lookahead.
Jitter buffer	Network smoother	Queue reordering out-of-order packets; 60-80 ms typical.
AEC	Echo cancellation	Subtracts speaker-to-mic feedback path.
Barge-in	User interrupt	System detects user speech mid-TTS; must cancel playback.
Full duplex	Simultaneous both ways	User and bot can talk at the same time; Moshi is full duplex.

## Further Reading

- Macháček et al. (2023). Whisper-Streaming — chunked near-streaming Whisper.
- Kyutai (2024). Moshi — full-duplex 200 ms latency.
- LiveKit Agents framework (2024) — production audio agent orchestration.
- Silero VAD repo — sub-1 ms VAD, Apache 2.0.
- WebRTC AEC3 paper — echo cancellation under open source.

**Continue online.** The living edition of this chapter has more than the page can hold:

- Animated, interactive figures and the web text: <https://aiengineeringfromscratch.com/lesson?path=phases/06-speech-and-audio/11-real-time-audio-processing>
- Runnable code for every step: <https://github.com/rohitg00/ai-engineering-from-scratch/tree/main/phases/06-speech-and-audio/11-real-time-audio-processing/code>

The repository moves faster than any printing. When the book and the repo disagree, trust the repo.

# Build a Voice Assistant Pipeline — The Phase 6 Capstone

---

Everything from lessons 01-11, stitched together. Build a voice assistant that listens, reasons, and talks back. In 2026 that is a solved engineering problem, not a research problem — but the integration details decide whether it ships.

**Type:** Build **Languages:** Python **Prerequisites:** Phase 6 · 04, 05, 06, 07, 11; Phase 11 · 09 (Function Calling); Phase 14 · 01 (Agent Loop) **Time:** ~120 minutes

## The Problem

Build an end-to-end assistant:

1. Captures mic input (16 kHz mono).
2. Detects start/end of user speech.
3. Transcribes streaming.
4. Passes transcript to an LLM that can call tools (timer, weather, calendar).
5. Streams LLM text to a TTS.
6. Plays audio back to the user.
7. Stops if the user interrupts mid-response.

Latency target: first TTS audio byte within 800 ms of the user finishing their utterance on a laptop CPU. Quality target: no missed words, no hallucinated subtitles on silence, no voice cloning leakage, no prompt injection success.

## The Concept

### The seven components

1. **Audio capture.** Mic → 16 kHz mono → 20 ms chunks. Usually sounddevice in Python or native AudioUnit/ALSA/WASAPI in production.
2. **VAD (Lesson 11).** Silero VAD @ threshold 0.5, min speech 250 ms, silence hang-over 500 ms. Signals “start” and “end.”
3. **Streaming STT (Lesson 4-5).** Whisper-streaming, Parakeet-TDT, or Deepgram Nova-3 (API). Partial + final transcripts.
4. **LLM with tool calling.** GPT-4o / Claude 3.5 / Gemini 2.5 Flash. JSON schema for tools. Stream tokens.
5. **Streaming TTS (Lesson 7).** Kokoro-82M (fastest open) or Cartesia Sonic (commercial). Start TTS after 20 LLM tokens.
6. **Playback.** Speaker out; opus-encode for low-bandwidth networks.
7. **Interruption handler.** If VAD fires during TTS playback, stop playback, cancel LLM, restart STT.

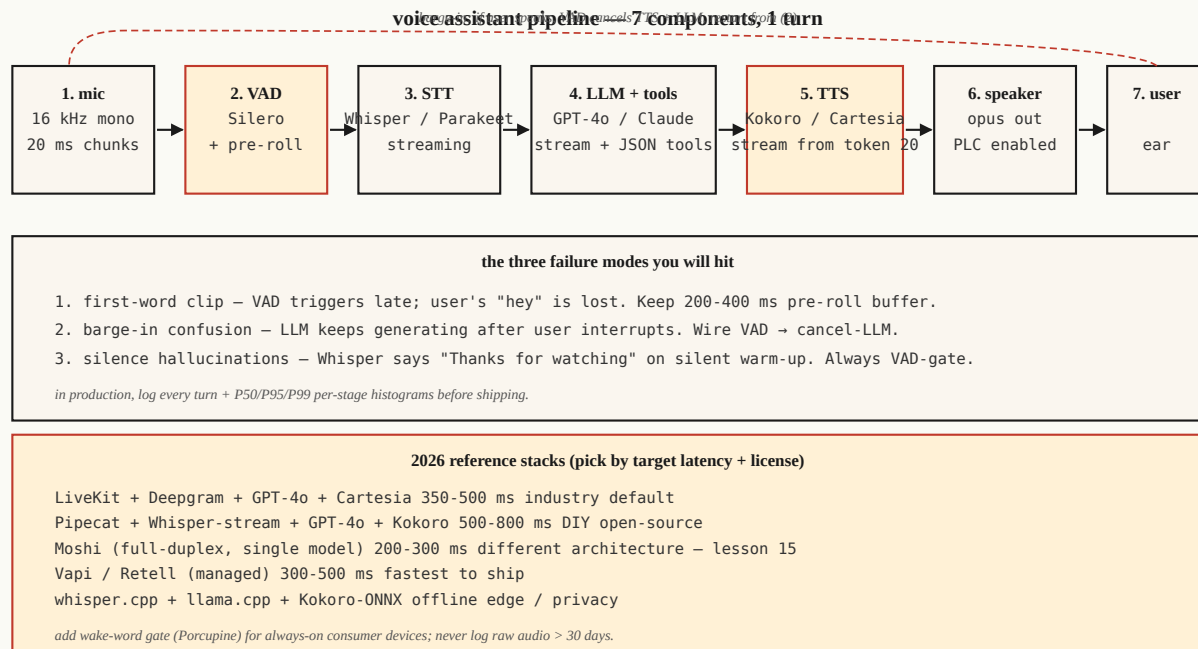


Figure 12: Voice assistant pipeline: mic → VAD → STT → LLM+tools → TTS → speaker

## The three failure modes you will hit

1. **First-word clip.** VAD starts a beat too late. User's "hey" is missing. Start threshold at 0.3, not 0.5.
2. **Mid-response interrupt confusion.** LLM keeps generating after user interrupts; assistant talks over user. Wire VAD → cancel-LLM.
3. **Silence hallucination.** Whisper outputs "Thanks for watching" on the silent warm-up frames. Always VAD-gate.

## 2026 production reference stacks

Stack	Latency	License	Notes
LiveKit + Deepgram + GPT-4o + Cartesia	350-500 ms	commercial API	Industry default 2026
Pipecat + Whisper-streaming + GPT-4o + Kokoro	500-800 ms	mostly open	DIY-friendly
Moshi (full-duplex)	200-300 ms	CC-BY 4.0	Single-model; different architecture, lesson 15
Vapi / Retell (managed)	300-500 ms	commercial	Fastest to launch; limited customization

Stack	Latency	License	Notes
Whisper.cpp + llama.cpp + Kokoro-ONNX	offline	open	Privacy / edge

**Interactive figure: v4-voice-latency.** This one moves. Watch it animate and drag its controls in the web edition: <https://aiengineeringfromscratch.com/lesson?path=phases/06-speech-and-audio/12-voice-assistant-pipeline>

## Build It

### Step 1: mic capture with chunking (pseudocode)

```
import sounddevice as sd

def mic_stream(chunk_ms=20, sr=16000):
    q = queue.Queue()
    def cb(indata, frames, time, status):
        q.put(indata.copy().flatten())
    with sd.InputStream(channels=1, samplerate=sr, blocksize=int(sr *
        ↪ chunk_ms/1000), callback=cb):
        while True:
            yield q.get()
```

### Step 2: VAD-gated turn capture

```
def capture_turn(stream, vad, pre_roll_ms=300, silence_ms=500):
    buf, pre, triggered = [], collections.deque(maxlen=pre_roll_ms // 20), False
    silent = 0
    for chunk in stream:
        pre.append(chunk)
        if vad(chunk):
            if not triggered:
                buf = list(pre)
                triggered = True
            buf.append(chunk)
            silent = 0
        elif triggered:
            silent += 20
            buf.append(chunk)
            if silent >= silence_ms:
                return b"".join(buf)
```

### Step 3: streaming STT → LLM → TTS

```
async def turn(audio_bytes):
    transcript = await stt.transcribe(audio_bytes)
    async for token in llm.stream(transcript):
        async for audio in tts.stream(token):
            await speaker.play(audio)
```

## Step 4: tool calling inside the LLM loop

```
tools = [
    {"name": "get_weather", "parameters": {"location": "string"}},
    {"name": "set_timer", "parameters": {"seconds": "int"}},
]

async for chunk in llm.stream(user_text, tools=tools):
    if chunk.type == "tool_call":
        result = dispatch(chunk.name, chunk.args)
        continue_streaming(result)
    if chunk.type == "text":
        await tts.stream(chunk.text)
```

## Step 5: interruption handling

```
tts_task = asyncio.create_task(tts_loop())
while True:
    chunk = await mic.get()
    if vad(chunk):
        tts_task.cancel()
        await speaker.stop()
        await new_turn()
        break
```

## Use It

See code/main.py for a runnable simulation that wires all seven components with stub models, so you can see the pipeline shape even without hardware. For a real implementation, swap stubs with:

- silero-vad (pip install silero-vad)
- deepgram-sdk or openai-whisper
- openai (gpt-4o) or anthropic
- kokoro or cartesia
- sounddevice for I/O

## Pitfalls

- **Logging PII forever.** Full-turn audio is PII in most jurisdictions. 30-day retention, encrypted at rest.
- **No barge-in.** Users will interrupt. Your assistant must stop talking.
- **TTS that blocks.** Synchronous TTS blocks the event loop. Use async or a separate thread.
- **No tool-call error handling.** Tools fail. LLM must get back the error + retry once, then gracefully degrade.
- **Overzealous hallucination filters.** Over-filter and the assistant repeats “I can’t help with that.” Under-filter and it says anything. Calibrate on a held-out set.
- **No wake-word option.** Always-listening is a privacy liability. Add a wake-word gate (Porcupine or openWakeWord).

**This chapter ships an artifact.** The course version of this lesson produces a reusable prompt or agent skill. It lives in the repository, ready to install: <https://github.com/rohitg00/ai->



[engineering-from-scratch/tree/main/phases/06-speech-and-audio/12-voice-assistant-pipeline](https://github.com/rohitg00/ai-engineering-from-scratch/tree/main/phases/06-speech-and-audio/12-voice-assistant-pipeline)

## Exercises

Starter code and the lesson’s working implementation: <https://github.com/rohitg00/ai-engineering-from-scratch/tree/main/phases/06-speech-and-audio/12-voice-assistant-pipeline/code>

1. **Easy.** Run `code/main.py`. It simulates one full turn end-to-end with stub modules and prints per-stage latency.
2. **Medium.** Replace the STT stub with a real Whisper model on a pre-recorded `.wav`. Measure WER and end-to-end latency.
3. **Hard.** Add tool calling: implement `get_weather` (any API) and `set_timer`. Route the LLM through the tools and verify that when the user says “set a 5 minute timer” the right function fires and the spoken reply confirms it.

## Key Terms

Term	What people say	What it actually means
Turn	A user + assistant round-trip	One VAD-bounded user speech + one LLM-TTS response.
Barge-in	Interruption	User speaks while assistant talks; assistant stops.
Wake word	“Hey assistant”	Short keyword detector; Porcupine, Snowboy, openWakeWord.
End-pointing	Turn ending	VAD + min-silence decision that user has finished.
Pre-roll	Pre-speech buffer	Keep 200-400 ms of audio before VAD fires to avoid first-word clip.
Tool call	Function invocation	LLM emits JSON; runtime dispatches; result feeds back in-loop.

## Further Reading

- LiveKit — voice agent quickstart — production-grade reference.
- Pipecat — voice agent examples — DIY-friendly framework.
- OpenAI Realtime API — the managed voice-native path.
- Kyutai Moshi — full-duplex reference (Lesson 15).
- Porcupine wake-word — wake-word gating.
- Anthropic — tool use guide — LLM function calling.

**Continue online.** The living edition of this chapter has more than the page can hold:

- Animated, interactive figures and the web text: <https://aiengineeringfromscratch.com/lesson?path=phases/06-speech-and-audio/12-voice-assistant-pipeline>
- Runnable code for every step: <https://github.com/rohitg00/ai-engineering-from-scratch/tree/main/phases/06-speech-and-audio/12-voice-assistant-pipeline/code>

The repository moves faster than any printing. When the book and the repo disagree, trust the repo.

# Neural Audio Codecs — EnCodec, SNAC, Mimi, DAC and the Semantic-Acoustic Split

---

2026 audio generation is almost all tokens. EnCodec, SNAC, Mimi, and DAC turn continuous waveforms into discrete sequences that a transformer can predict. The semantic-vs-acoustic token split — first-codebook as semantic, rest as acoustic — is the most important architectural shift since the Transformer for audio.

**Type:** Learn **Languages:** Python **Prerequisites:** Phase 6 · 02 (Spectrograms), Phase 10 · 11 (Quantization), Phase 5 · 19 (Subword Tokenization) **Time:** ~60 minutes

## The Problem

Language models work on discrete tokens. Audio is continuous. If you want an LLM-style model for speech / music — MusicGen, Moshi, Sesame CSM, VibeVoice, Orpheus — you first need a **neural audio codec**: a learned encoder that discretizes audio into a small vocabulary of tokens, and a matching decoder that reconstructs the waveform.

Two families have emerged:

1. **Reconstruction-first codecs** — EnCodec, DAC. Optimize perceptual audio quality. Tokens are “acoustic” — they capture everything including speaker identity, timbre, background noise.
2. **Semantic-first codecs** — Mimi (Kyutai), SpeechTokenizer. Force the first codebook to encode linguistic / phonetic content (often by distilling from WavLM). Subsequent codebooks are acoustic detail.

The 2024-2026 insight: **a pure reconstruction codec gives you blurry speech when you try to generate from text**. The LLM over codec tokens has to learn both language structure AND acoustic structure in the same codebook, which doesn't scale. Separating them — semantic codebook 0, acoustic codebooks 1-N — is what makes Moshi and Sesame CSM work.

## The Concept

### The core trick: Residual Vector Quantization (RVQ)

Rather than one big codebook (which would need millions of codes for good quality), all modern audio codecs use **RVQ**: a cascade of small codebooks. The first codebook quantizes the encoder output; the second quantizes the residual; etc. Each codebook is 1024 codes. 8 codebooks = effective vocabulary of  $1024^8 = 10^{24}$ .

At inference time, the decoder sums all chosen codes per frame to reconstruct.

## neural audio codecs — four variations on RVQ, one semantic/acoustic split

<b>EnCodec (Meta, 2022)</b> <i>acoustic RVQ baseline</i> wav → conv+transformer+conv → RVQ (4-32 codebooks) → decoder → wav 75 Hz · 24 kHz · used by MusicGen PESQ 3.2 @ 6 kbps	<b>DAC (Descript, 2023)</b> <i>reconstruction king</i> EnCodec + L2-norm codebooks periodic activations improved losses 44.1 kHz full-band · PESQ 3.5 @ 6 kbps	<b>SNAC (Siuzdak 2024)</b> <i>multi-scale RVQ</i> coarse codes @ 12 Hz medium @ 25 Hz fine @ 50 Hz hierarchical → AR-LM friendly used by Orpheus-3B	<b>Mimi (Kyutai 2024)</b> <i>semantic + acoustic split</i> codebook 0 — WavLM distilled (semantic) codebooks 1-7 — acoustic 12.5 Hz · 4.4 kbps · powers Moshi used by Sesame CSM
<b>why this matters for language modeling over audio</b> 10 s speech × 12.5 Hz × 8 codebooks = 1000 tokens → trivial transformer context factorize: LM predicts semantic codebook 0 first (text-aligned), then acoustic codebooks 1-7 result: zero-shot voice cloning (semantic = content, acoustic conditioned on reference)			
<b>pick by the problem</b> music generation → EnCodec-24k · MusicGen, AudioLDM highest fidelity → DAC-44.1k · audio restoration, editing AR LM over speech → SNAC or Mimi · TTS, codec LM full-duplex streaming → Mimi only · Moshi, Sesame CSM			

Figure 13: Four codec landscape: EnCodec, DAC, SNAC (multi-scale), Mimi (semantic+acoustic)

## The four codecs that matter in 2026

**EnCodec (Meta, 2022).** The baseline. Encoder-decoder over waveform, RVQ bottleneck. 24 kHz, 32 codebooks possible, default 4 codebooks @ 1.5 kbps. Uses 1D conv + transformer + 1D conv architecture. Used by MusicGen.

**DAC (Descript, 2023).** RVQ with L2-normalized codebooks, periodic activation functions, improved losses. Highest reconstruction fidelity of any open codec — sometimes indistinguishable from original speech with 12 codebooks. 44.1 kHz full-band.

**SNAC (Hubert Siuzdak, 2024).** Multi-scale RVQ — the coarse codebooks operate at a lower frame rate than fine ones. Effectively models audio hierarchically: a coarse “sketch” at ~12 Hz plus detail at 50 Hz. Used by Orpheus-3B because the hierarchical structure maps well onto LM-based generation.

**Mimi (Kyutai, 2024).** The 2026 game-changer. 12.5 Hz frame rate (extremely low), 8 codebooks @ 4.4 kbps. Codebook 0 is **distilled from WavLM** — trained to predict WavLM’s speech-content features. Codebooks 1-7 are acoustic residuals. This split powers Moshi (Lesson 15) and Sesame CSM.

## Frame rates matter for language modeling

Lower frame rate = shorter sequence = faster LM.

Codec	Frame rate	1 s = N frames	Good for
EnCodec-24k	75 Hz	75	music, general audio
DAC-44.1k	86 Hz	86	high-fidelity music

Codec	Frame rate	1 s = N frames	Good for
SNAC-24k (coarse)	~12 Hz	12	AR-LM efficient
Mimi	12.5 Hz	12.5	streaming speech

At 12.5 Hz, a 10-second utterance is only 125 codec frames — a transformer can easily predict them.

## Semantic vs acoustic tokens

```
frame_t → [semantic_token_t, acoustic_token_0_t, acoustic_token_1_t, ...,
           ↪ acoustic_token_6_t]
```

- **Semantic token (codebook 0 in Mimi).** Encodes what was said — phonemes, words, content. Distilled from WavLM via an auxiliary prediction loss.
- **Acoustic tokens (codebooks 1-7).** Encode timbre, speaker identity, prosody, background noise, fine detail.

An AR LM predicts the semantic token first (conditioned on text), then predicts acoustic tokens (conditioned on semantic + speaker reference). This factorization is why modern TTS can zero-shot-clone voices: the semantic model handles content; the acoustic model handles timbre.

## 2026 reconstruction quality (bits per sec, lower bitrate is better)

Codec	Bitrate	PESQ	ViSQOL
Opus-20kbps	20 kbps	4.0	4.3
EnCodec-6kbps	6 kbps	3.2	3.8
DAC-6kbps	6 kbps	3.5	4.0
SNAC-3kbps	3 kbps	3.3	3.8
Mimi-4.4kbps	4.4 kbps	3.1	3.7

Traditional codecs like Opus still win per bit on perceptual quality. Neural codecs win on **discrete tokens** (which Opus does not produce) and **generative-model quality** (what the LM can do with those tokens).

**Interactive figure: rvq-codec-cascade.** This one moves. Watch it animate and drag its controls in the web edition: <https://aiengineeringfromscratch.com/lesson?path=phases/06-speech-and-audio/13-neural-audio-codecs>

## Build It

### Step 1: encode with EnCodec

```
from encodec import EncodecModel
import torch

model = EncodecModel.encodec_model_24khz()
model.set_target_bandwidth(6.0) # kbps
```

```
wav = torch.randn(1, 1, 24000)
with torch.no_grad():
    encoded = model.encode(wav)
codes, scale = encoded[0]
# codes: (1, n_codebooks, n_frames), dtype=int64
n_codebooks=8 at 6 kbps. Each code is 0-1023 (10-bit).
```

## Step 2: decode and measure reconstruction

```
with torch.no_grad():
    wav_recon = model.decode([(codes, scale)])

from torchaudio.functional import compute_deltas
import torch.nn.functional as F

mse = F.mse_loss(wav_recon[:, :, :wav.shape[-1]], wav).item()
```

## Step 3: the semantic-acoustic split (Mimi-style)

```
from moshi.models import loaders
mimi = loaders.get_mimi()

with torch.no_grad():
    codes = mimi.encode(wav) # shape (1, 8, frames@12.5Hz)

semantic = codes[:, 0]
acoustic = codes[:, 1:]
```

Semantic codebook 0 is WavLM-aligned. You can train a text-to-semantic transformer — much smaller vocabulary than going direct-to-audio. Then a separate acoustic-to-waveform decoder conditions on a speaker reference.

## Step 4: why AR LM over codec tokens works

For a 10 s speech clip at Mimi’s 12.5 Hz × 8 codebooks:

$N_{\text{tokens}} = 10 * 12.5 * 8 = 1000$  tokens

1000 tokens is a trivial context for a transformer. A 256M-parameter transformer can generate 10 seconds of speech in milliseconds on a modern GPU.

## Use It

Map problem → codec:

Task	Codec
General music generation	EnCodec-24k
Highest-fidelity reconstruction	DAC-44.1k
AR LM over speech (TTS)	SNAC or Mimi
Streaming full-duplex speech	Mimi (12.5 Hz)
Sound-effect library with text	EnCodec + T5 condition
Fine-grained audio editing	DAC + inpainting

Rule of thumb: **if you’re building a generative model, start with Mimi or SNAC. If you’re building a compression pipeline, use Opus.**

## Pitfalls

- **Too many codebooks.** Adding codebooks increases fidelity linearly but LM sequence length linearly too. Stop at 8-12.
- **Frame-rate mismatch.** Training LM on 12.5 Hz Mimi then fine-tuning on 50 Hz EnCodec fails silently.
- **Assuming all codebooks equal.** In Mimi, codebook 0 carries content; losing it destroys intelligibility. Losing codebook 7 is barely noticeable.
- **Using reconstruction quality as the only metric.** A codec can have great reconstruction but be useless for LM-based generation if the semantic structure is bad.

**This chapter ships an artifact.** The course version of this lesson produces a reusable prompt or agent skill. It lives in the repository, ready to install: <https://github.com/rohitg00/ai-engineering-from-scratch/tree/main/phases/06-speech-and-audio/13-neural-audio-codecs>

## Exercises

Starter code and the lesson’s working implementation: <https://github.com/rohitg00/ai-engineering-from-scratch/tree/main/phases/06-speech-and-audio/13-neural-audio-codecs/code>

1. **Easy.** Run `code/main.py`. It implements a toy scalar + residual quantizer and measures reconstruction error as you add codebooks.
2. **Medium.** Install `encodec` and compare 1, 4, 8, 32 codebooks on a held-out speech clip. Plot PESQ or MSE vs bitrate.
3. **Hard.** Load Mimi. Encode a clip. Replace codebook 0 with random integers; decode. Then replace codebook 7 similarly. Compare the two corruptions — codebook 0 corruption should destroy intelligibility; codebook 7 corruption should barely change anything.

## Key Terms

Term	What people say	What it actually means
RVQ	Residual quantization	Cascade of small codebooks; each quantizes the previous residual.
Frame rate	Codec speed	How many token-frames per second. Lower = faster LM.
Semantic codebook	Codebook 0 (Mimi)	Codebook distilled from SSL features; encodes content.
Acoustic codebooks	Everything else	Timbre, prosody, noise, fine detail.
PESQ / ViSQOL	Perceptual quality	Objective metrics correlating with MOS.
EnCodec	Meta codec	The RVQ baseline; used by MusicGen.
Mimi	Kyutai codec	12.5 Hz frame rate; semantic-acoustic split; powers Moshi.

## Further Reading

- Défossez et al. (2023). EnCodec — the RVQ baseline.
- Kumar et al. (2023). Descript Audio Codec (DAC) — highest-fidelity open.
- Siuzdak (2024). SNAC — multi-scale RVQ.
- Kyutai (2024). Mimi codec — semantic-acoustic split, WavLM distillation.
- Borsos et al. (2023). AudioLM — the two-stage semantic/acoustic paradigm.
- Zeghidour et al. (2021). SoundStream — the original streamable RVQ codec.

**Continue online.** The living edition of this chapter has more than the page can hold:

- Animated, interactive figures and the web text: <https://aiengineeringfromscratch.com/lesson?path=phases/06-speech-and-audio/13-neural-audio-codecs>
- Runnable code for every step: <https://github.com/rohitg00/ai-engineering-from-scratch/tree/main/phases/06-speech-and-audio/13-neural-audio-codecs/code>

The repository moves faster than any printing. When the book and the repo disagree, trust the repo.

# Voice Activity Detection & Turn-Taking — Silero, Cobra, and the Flush Trick

---

Every voice agent lives or dies on two decisions: is the user speaking now, and are they done? VAD answers the first. Turn-detection (VAD + silence-hangover + semantic endpoint model) answers the second. Get either wrong and your assistant either cuts users off or never shuts up.

**Type:** Build **Languages:** Python **Prerequisites:** Phase 6 · 11 (Real-Time Audio), Phase 6 · 12 (Voice Assistant) **Time:** ~45 minutes

## The Problem

Three distinct decisions a voice agent makes on every 20 ms chunk:

1. **Is this frame speech?** — VAD. Binary, per-frame.
2. **Has the user started a new utterance?** — onset detection.
3. **Has the user finished?** — end-pointing (turn-end).

The naive answer (energy threshold) fails on any noise — traffic, keyboards, crowd babble. The 2026 answer: Silero VAD (open, deep-learned) + a turn-detection model (semantic end-pointing) + a VAD-calibrated silence hangover.

## The Concept

### The three-tier VAD cascade

**Tier 1: energy gate.** Cheapest. Threshold RMS at -40 dBFS. Filters obvious silence but fires on any noise above the threshold.

**Tier 2: Silero VAD** (2020-2026, MIT). 1M parameters. Trained on 6000+ languages. Runs in ~1 ms per 30 ms chunk on a single CPU thread. 87.7% TPR at 5% FPR. The open-source default.

**Tier 3: semantic turn detector.** LiveKit's turn-detection model (2024-2026) or your own small classifier. Distinguishes "pause mid-sentence" from "done talking." Uses linguistic context (intonation + recent words), not just silence.

### Key parameters and their defaults

- **Threshold.** Silero outputs a probability; classify speech at  $> 0.5$  (default) or  $> 0.3$  (sensitive). Lower threshold = fewer first-word clips, more false positives.
- **Minimum speech duration.** Reject speech shorter than 250 ms — usually coughs or chair noise.
- **Silence hangover (end-pointing).** After VAD returns to 0, wait 500-800 ms before declaring end-of-turn. Too short → interrupt user. Too long → feels sluggish.



## three-tier VAD cascade + turn-end state machine

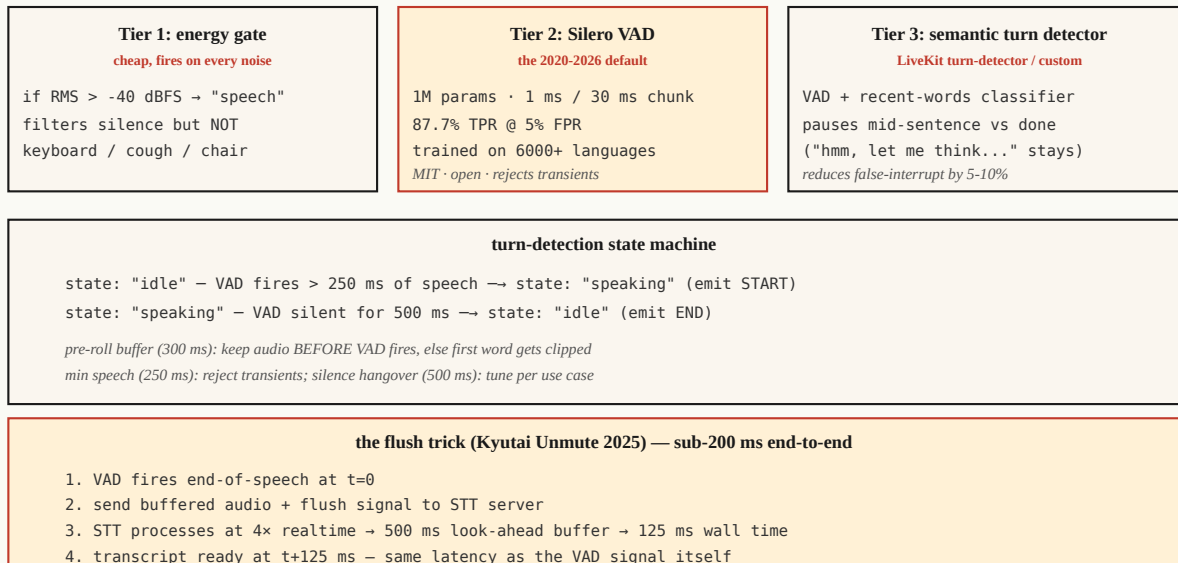


Figure 14: VAD cascade: energy → Silero → turn-detector → flush trick

- **Pre-roll buffer.** Keep 300-500 ms of audio before VAD fires. Prevents “hey” being clipped.

**The flush trick (Kyutai 2025)**

Streaming STT models have a look-ahead delay (500 ms for Kyutai STT-1B, 2.5 s for STT-2.6B). Normally you’d wait that long after end-of-speech for the transcript. Flush trick: when VAD fires end-of-speech, **send a flush signal to the STT** that forces immediate output. STT processes at ~4× realtime, so the 500 ms buffer finishes in ~125 ms.

End-to-end: 125 ms VAD + flush STT = conversational latency.

**2026 VAD comparison**

VAD	TPR @ 5% FPR	Latency	License
WebRTC VAD (Google, 2013)	50.0%	30 ms	BSD
Silero VAD (2020-2026)	87.7%	~1 ms	MIT
Cobra VAD (Picovoice)	98.9%	~1 ms	commercial
pyannote segmentation	95%	~10 ms	MIT-ish

Silero is the right default. Cobra is the compliance / accuracy upgrade. Energy-only VAD has no place in 2026 production.

**Interactive figure: sp-vad-cascade.** This one moves. Watch it animate and drag its controls in the web edition: <https://aiengineeringfromscratch.com/lesson?path=phases/06-speech-and-audio/14-voice-activity-detection-turn-taking>

## Build It

### Step 1: the energy gate

```
def energy_vad(chunk, threshold_dbfs=-40.0):
    rms = (sum(x * x for x in chunk) / len(chunk)) ** 0.5
    dbfs = 20.0 * math.log10(max(rms, 1e-10))
    return dbfs > threshold_dbfs
```

### Step 2: Silero VAD in Python

```
from silero_vad import load_silero_vad, get_speech_timestamps

vad = load_silero_vad()
audio = torch.tensor(waveform_16k, dtype=torch.float32)
segments = get_speech_timestamps(
    audio, vad, sampling_rate=16000,
    threshold=0.5,
    min_speech_duration_ms=250,
    min_silence_duration_ms=500,
    speech_pad_ms=300,
)
for s in segments:
    print(f"[{s['start']/16000:.2f}s - {s['end']/16000:.2f}s]")
```

### Step 3: turn-end state machine

```
class TurnDetector:
    def __init__(self, silence_hangover_ms=500, min_speech_ms=250):
        self.state = "idle"
        self.speech_ms = 0
        self.silence_ms = 0
        self.silence_hangover_ms = silence_hangover_ms
        self.min_speech_ms = min_speech_ms

    def update(self, is_speech, chunk_ms=20):
        if is_speech:
            self.speech_ms += chunk_ms
            self.silence_ms = 0
            if self.state == "idle" and self.speech_ms >= self.min_speech_ms:
                self.state = "speaking"
                return "START"
        else:
            self.silence_ms += chunk_ms
            if self.state == "speaking" and self.silence_ms >= self.silence_hangover_ms:
```

```

        self.state = "idle"
        self.speech_ms = 0
        return "END"
    return None

```

## Step 4: the flush trick skeleton

```

def flush_on_end(stt_client, audio_buffer):
    stt_client.send_audio(audio_buffer)
    stt_client.send_flush()
    return stt_client.recv_transcript(timeout_ms=150)

```

STT (Kyutai, Deepgram, AssemblyAI) must support flush for this to work. Whisper streaming does not — it's block-based and always waits for chunks.

## Use It

Situation	VAD choice
Open, fast, general	Silero VAD
Commercial call center	Cobra VAD
On-device (phone)	Silero VAD ONNX
Research / diarization	pyannote segmentation
Zero-dependency fallback	WebRTC VAD (legacy)
Need turn-ending quality	Silero + LiveKit turn-detector layered

Rule of thumb: never ship energy-only VAD unless you really have no other option.

## Pitfalls

- **Fixed threshold.** Works in quiet, fails in noisy. Either calibrate on-device or switch to Silero.
- **Too-short silence hangover.** Agent interrupts mid-sentence. 500-800 ms is the sweet spot for conversational speech.
- **Too-long hangover.** Feels sluggish. A/B test with target users.
- **No pre-roll buffer.** First 200-300 ms of user audio lost. Always keep a rolling pre-roll.
- **Ignoring semantic endpointing.** “Hmm, let me think...” contains long pauses. Users hate being cut off mid-thought. Use LiveKit’s turn-detector or similar.

**This chapter ships an artifact.** The course version of this lesson produces a reusable prompt or agent skill. It lives in the repository, ready to install: <https://github.com/rohitg00/ai-engineering-from-scratch/tree/main/phases/06-speech-and-audio/14-voice-activity-detection-turn-taking>

## Exercises

Starter code and the lesson’s working implementation: <https://github.com/rohitg00/ai-engineering-from-scratch/tree/main/phases/06-speech-and-audio/14-voice-activity-detection-turn-taking/code>

1. **Easy.** Run `code/main.py`. It simulates a speech + silence + speech + coughs sequence and tests three VAD tiers.
2. **Medium.** Install `silero-vad`, process a 5-min recording, tune threshold to minimize both first-word clips and false triggers. Report precision/recall.
3. **Hard.** Build a mini turn-detector: Silero VAD + a 3-layer MLP on the last 10 words' embeddings (use sentence-transformers). Train on a hand-labeled turn-end dataset. Beat Silero-only by 10% F1.

## Key Terms

Term	What people say	What it actually means
VAD	Voice detector	Binary per-frame: is this speech?
Turn detection	End-pointing	VAD + silence-hangover + semantic endpoint.
Silence hangover	Wait-after-speech	Time to wait before declaring turn end; 500-800 ms.
Pre-roll	Pre-speech buffer	Keep 300-500 ms audio before VAD fires.
Flush trick	Kyutai hack	VAD → flush-STT → 125 ms instead of 500 ms delay.
Semantic endpoint	“Did they mean to stop?”	ML classifier that looks at words, not just silence.
TPR @ FPR 5%	ROC point	Standard VAD benchmark; 87.7% for Silero, 50% WebRTC.

## Further Reading

- Silero VAD — the reference open VAD.
- Picovoice Cobra VAD — commercial accuracy leader.
- Kyutai — Unmute + flush trick — the sub-200 ms engineering trick.
- LiveKit — turn detection — semantic endpointing in production.
- WebRTC VAD — the legacy baseline.
- pyannote segmentation — diarization-grade segmentation.

**Continue online.** The living edition of this chapter has more than the page can hold:

- Animated, interactive figures and the web text: <https://aiengineeringfromscratch.com/lesson?path=phases/06-speech-and-audio/14-voice-activity-detection-turn-taking>
- Runnable code for every step: <https://github.com/rohitg00/ai-engineering-from-scratch/tree/main/phases/06-speech-and-audio/14-voice-activity-detection-turn-taking/code>

The repository moves faster than any printing. When the book and the repo disagree, trust the repo.

# Streaming Speech-to-Speech — Moshi, Hibiki, and Full-Duplex Dialogue

---

2024-2026 redefined voice AI. Moshi ships a single model that listens and speaks simultaneously at 200 ms latency. Hibiki does speech-to-speech translation chunk-by-chunk. Both abandon the ASR → LLM → TTS pipeline for a unified full-duplex architecture over Mimi codec tokens. This is the new reference design.

**Type:** Learn **Languages:** Python **Prerequisites:** Phase 6 · 13 (Neural Audio Codecs), Phase 6 · 11 (Real-Time Audio), Phase 7 · 05 (Full Transformer) **Time:** ~75 minutes

## The Problem

Every voice agent built from Lessons 11 + 12 has a fundamental latency floor around 300-500 ms: VAD fires, STT processes, LLM reasons, TTS generates. Each stage has its own minimum latency. You can tune and parallelize, but the pipeline shape caps you.

Moshi (Kyutai, 2024-2026) asks a different question: what if there is no pipeline? What if one model takes audio in and emits audio out directly, continuously, with text as an intermediate “inner monologue” instead of a required stage?

The answer is **full-duplex speech-to-speech**. Theoretical latency 160 ms (80 ms Mimi frame + 80 ms acoustic delay). Practical latency 200 ms on a single L4 GPU. That’s half what a best-in-class pipelined voice agent achieves.

## The Concept

### The Moshi architecture

**Inputs.** Two Mimi codec streams, both at 12.5 Hz × 8 codebooks:

- Stream 1: user audio (Mimi-encoded, constantly arriving)
- Stream 2: Moshi’s own audio (generated by Moshi)

**The transformer.** A 7B-parameter Temporal Transformer processes both streams and a text “inner monologue” stream. At each 80 ms step, it:

1. Consumes the latest user Mimi tokens (8 codebooks).
2. Consumes the most recent Moshi Mimi tokens (8 codebooks, as produced).
3. Generates the next Moshi text token (inner monologue).
4. Generates the next Moshi Mimi tokens (8 codebooks via a small Depth Transformer).

All three streams — user audio, Moshi audio, Moshi text — run in parallel. Moshi can hear the user while speaking; can interrupt itself when the user interrupts; can back-channel (“mhm”) without breaking its main utterance.

**The depth transformer.** Within a frame, the 8 codebooks are not predicted in parallel — they have inter-codebook dependencies. A small 2-layer “depth transformer” predicts them

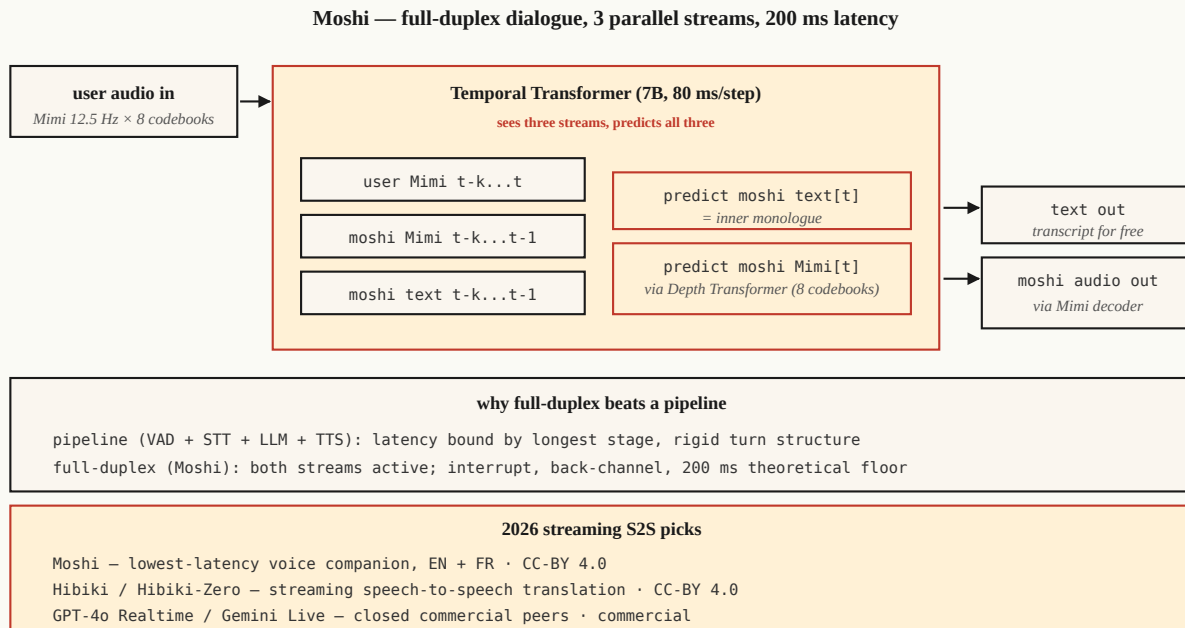


Figure 15: Moshi architecture: two parallel Mimi streams + inner-monologue text

sequentially within 80 ms. This is the standard factorization for AR codec LMs (also used by VALL-E, VibeVoice).

## Why inner-monologue text helps

Without explicit text, the model has to implicitly model language in its acoustic stream. Moshi’s insight: force it to emit text tokens alongside audio. The text stream is essentially the transcript of what Moshi is saying. This improves semantic coherence, makes it easier to swap out a language model head, and gives you transcripts for free.

## Hibiki: streaming speech-to-speech translation

Same architecture, trained on translation pairs. Source audio in, target-language audio out, continuously. Hibiki-Zero (Feb 2026) eliminates the need for word-level aligned training data — uses sentence-level data + GRPO reinforcement learning for latency optimization.

Four language pairs supported initially; can be adapted to a new language with  $\approx 1000$  hours.

## The broader Kyutai stack (2026)

- **Moshi** — full-duplex dialogue (French first, English well-supported)
- **Hibiki / Hibiki-Zero** — simultaneous speech translation
- **Kyutai STT** — streaming ASR (500 ms or 2.5 s look-ahead)
- **Kyutai Pocket TTS** — 100M-param TTS runs on CPU (Jan 2026)
- **Unmute** — full pipeline combining these on public servers

Throughput on an L40S GPU: 64 concurrent sessions at 3× real-time.

## Sesame CSM — the cousin

Sesame CSM (2025) uses a similar idea — a Llama-3 backbone with a Mimi codec head. But CSM is single-directional (takes context + text, produces speech) rather than full-duplex. It's the best “voice presence” TTS on the market; not quite the same as Moshi's full-duplex capability.

## 2026 performance numbers

Model	Latency	Use case	License
Moshi	200 ms (L4)	full-duplex English / French dialogue	CC-BY 4.0
Hibiki	12.5 Hz framerate	French ↔ English streaming translation	CC-BY 4.0
Hibiki-Zero	same	5 language-pairs, no aligned data	CC-BY 4.0
Sesame CSM-1B	200 ms TTFA	context-conditioned TTS	Apache-2.0
GPT-4o Realtime	~300 ms	closed, OpenAI API	commercial
Gemini 2.5 Live	~350 ms	closed, Google API	commercial

**Interactive figure: sp-fullduplex.** This one moves. Watch it animate and drag its controls in the web edition: <https://aiengineeringfromscratch.com/lesson?path=phases/06-speech-and-audio/15-streaming-speech-to-speech-moshi-hibiki>

## Build It

### Step 1: the interface

Moshi exposes a WebSocket server that takes 80 ms chunks of Mimi-encoded audio and returns 80 ms chunks of Mimi-encoded audio. Both ways. Constantly.

```
import asyncio
import websockets
from moshi.client_utils import encode_audio_mimi, decode_audio_mimi

async def moshi_chat():
    async with websockets.connect("ws://localhost:8998/api/chat") as ws:
        mic_task = asyncio.create_task(stream_mic_to(ws))
        spk_task = asyncio.create_task(stream_from_to_speaker(ws))
        await asyncio.gather(mic_task, spk_task)
```

### Step 2: the full-duplex loop

```
async def stream_mic_to(ws):
    async for chunk_80ms in mic_stream_at_12_5_hz():
        mimi_tokens = encode_audio_mimi(chunk_80ms)
        await ws.send(serialize(mimi_tokens))

async def stream_from_to_speaker(ws):
    async for msg in ws:
        mimi_tokens, text_token = deserialize(msg)
```

```
audio = decode_audio_mimi(mimi_tokens)
await play(audio)
```

Both directions run simultaneously. Python asyncio or Rust futures are the standard transport.

### Step 3: the training objective (conceptual)

For every 80 ms frame  $t$ :

- Input: `user_mimi[0..t]`, `moshi_mimi[0..t-1]`, `moshi_text[0..t-1]`
- Predict: `moshi_text[t]`, then `moshi_mimi[t]`, `codebook_0..7]`

Text is predicted before audio (inner monologue); audio is predicted codebook-sequential within the depth transformer.

### Step 4: where Moshi wins and where it doesn't

Moshi wins:

- Sub-250 ms end-to-end on cheap hardware.
- Natural back-channels and interruptions.
- No pipeline glue code.

Moshi does not win:

- Tool calling (not trained for it; you need a separate LLM path).
- Long reasoning (Moshi is an 8B-ish dialogue model, not Claude/GPT-4).
- Factual accuracy on niche topics.
- Most production enterprise use cases (still use pipelines in 2026).

## Use It

Situation	Pick
Lowest-latency voice companion	Moshi
Live translation call	Hibiki
Voice demo / research	Moshi, CSM
Enterprise agent with tools	Pipeline (Lesson 12), not Moshi
Custom-voice TTS in context	Sesame CSM
Speech-to-speech, any languages	GPT-4o Realtime or Gemini 2.5 Live (commercial)

## Pitfalls

- **Limited tool calling.** Moshi is a dialogue model, not an agent framework. Combine with pipeline for tools.
- **Specific-voice conditioning.** Moshi uses a single trained persona; cloning is a separate training run.
- **Language coverage.** French + English is excellent; others limited. Hibiki-Zero helps, but you still need training data.
- **Resource cost.** A full Moshi session holds a GPU slot; not a cheap shared-tenant deploy pattern.



**This chapter ships an artifact.** The course version of this lesson produces a reusable prompt or agent skill. It lives in the repository, ready to install: <https://github.com/rohitg00/ai-engineering-from-scratch/tree/main/phases/06-speech-and-audio/15-streaming-speech-to-speech-moshi-hibiki>

## Exercises

Starter code and the lesson’s working implementation: <https://github.com/rohitg00/ai-engineering-from-scratch/tree/main/phases/06-speech-and-audio/15-streaming-speech-to-speech-moshi-hibiki/code>

1. **Easy.** Run `code/main.py`. It simulates the two-stream + inner-monologue architecture symbolically.
2. **Medium.** Pull Moshi from HuggingFace, run the server, test one conversation. Measure wall-clock latency from end-of-user-speech to start-of-Moshi-response.
3. **Hard.** Take your Lesson 12 pipeline agent and compare P50 latency vs Moshi on 20 matched test utterances. Write up when a pipeline architecturally wins anyway.

## Key Terms

Term	What people say	What it actually means
Full-duplex	Hear-and-speak at once	Two audio streams active simultaneously on the same model.
Inner monologue	Model’s text stream	Moshi emits text tokens alongside its audio output.
Depth trans-former	Inter-codebook predictor	Small transformer that predicts 8 codebooks within one 80 ms frame.
Mimi	Kyutai’s codec	12.5 Hz × 8 codebooks; semantic+acoustic; powers Moshi.
Streaming S2S	Audio → audio live	Chunk-by-chunk translation/dialogue, no pipeline stages.
Back-channeling	“Mhm” reactions	Moshi can emit small acknowledgments without breaking its turn.

## Further Reading

- Défossez et al. (2024). Moshi — speech-text foundation model — the paper.
- Kyutai Labs (2026). Hibiki-Zero — streaming translation without aligned data.
- Sesame (2025). Crossing the uncanny valley of voice — CSM spec.
- Kyutai — Moshi repo — install + server.
- OpenAI — Realtime API — closed commercial peer.
- Kyutai — Delayed Streams Modeling — the STT/TTS framework under the hood.

**Continue online.** The living edition of this chapter has more than the page can hold:

- Animated, interactive figures and the web text: <https://aiengineeringfromscratch.com/lesson?path=phases/06-speech-and-audio/15-streaming-speech-to-speech-moshi-hibiki>
- Runnable code for every step: <https://github.com/rohitg00/ai-engineering-from-scratch/tree/main/phases/06-speech-and-audio/15-streaming-speech-to-speech-moshi-hibiki/code>

The repository moves faster than any printing. When the book and the repo disagree, trust the repo.

# Voice Anti-Spoofing & Audio Watermarking — ASVspoof 5, AudioSeal, WaveVerify

---

Voice cloning shipped faster than defenses. 2026 production voice systems need two things: a detector (AASIST, RawNet2) that classifies real vs fake speech, and a watermark (AudioSeal) that survives compression and editing. Ship both or do not ship voice cloning.

**Type:** Build **Languages:** Python **Prerequisites:** Phase 6 · 06 (Speaker Recognition), Phase 6 · 08 (Voice Cloning) **Time:** ~75 minutes

## The Problem

Three related defenses:

1. **Anti-spoofing / deepfake detection.** Given an audio clip, is it synthetic or real? ASVspoof benchmarks (ASVspoof 2019 → 2021 → 5) are the gold standard.
2. **Audio watermarking.** Embed an imperceptible signal in generated audio that a detector can extract later. AudioSeal (Meta) and WavMark are the open options.
3. **Authenticated provenance.** Cryptographic signing of audio files + metadata. C2PA / Content Authenticity Initiative.

Detection handles adversaries who don't cooperate. Watermarking handles compliance — AI-generated audio should be identifiable as such. Both are required in 2026.

## The Concept

### ASVspoof 5 — the 2024-2025 benchmark

Biggest change from prior editions:

- **Crowdsourced data** (not studio clean) — realistic conditions.
- **~2000 speakers** (vs ~100 before).
- **32 attack algorithms.** TTS + voice conversion + adversarial perturbation.
- **Two tracks.** Countermeasure (CM) standalone detection; Spoofing-robust ASV (SASV) for biometric systems.

State-of-the-art on ASVspoof 5: ~7.23% EER. On the older ASVspoof 2019 LA: 0.42% EER. Real-world deployment: expect 5-10% EER on in-the-wild clips.

### AASIST and RawNet2 — detection model families

**AASIST** (2021, updated through 2026). Graph-attention on spectral features. Current SOTA on ASVspoof 5 countermeasure task.

**RawNet2.** Convolutional front-end over raw waveform + TDNN backbone. Simpler baseline; still competitive with fine-tuning.

## three defensive layers — detection + watermarking + provenance

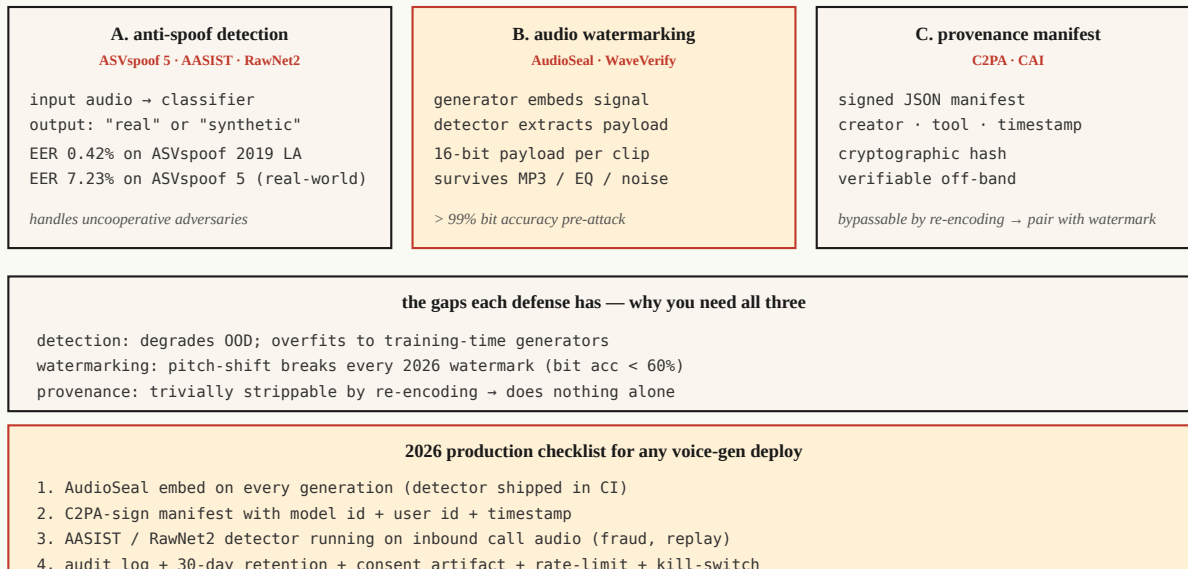


Figure 16: Anti-spoofing vs watermarking vs provenance — three defense layers

**NeXt-TDNN + SSL features.** 2025 variant: ECAPA-style + WavLM features + focal loss. Achieves the 0.42% EER on ASVspoof 2019 LA.

## AudioSeal — the 2024 watermark default

Meta's **AudioSeal** (Jan 2024, v0.2 Dec 2024). Key design:

- **Localized.** Detects the watermark per-frame at 16 kHz sample resolution (1/16000 s).
- **Generator + detector jointly trained.** Generator learns to embed inaudible signal; detector learns to find it through augmentations.
- **Robust.** Survives MP3 / AAC compression, EQ, speed-shift  $\pm 10\%$ , noise mix +10 dB SNR.
- **Fast.** Detector runs at 485× realtime; 1000× faster than WavMark.
- **Capacity.** 16-bit payload (can encode model ID, generation timestamp, user ID) embeddable in each utterance.

## WavMark

The pre-AudioSeal open baseline. Invertible neural network, 32 bits/sec. Problems:

- Synchronization brute-force is slow.
- Can be removed by Gaussian noise or MP3 compression.
- Not real-time friendly.

## WaveVerify (July 2025)

Addresses AudioSeal's weaknesses — specifically temporal manipulations (reversal, speed). Uses FiLM-based generator + Mixture-of-Experts detector. Competitive with AudioSeal on standard attacks; handles temporal edits.

## The gap adversaries exploit

From AudioMarkBench: “under pitch shift, all watermarks show Bit Recovery Accuracy below 0.6, indicating near-complete removal.” **Pitch-shift is the universal attack.** No 2026 watermark is fully robust to aggressive pitch modification. This is why you need detection (AASIST) alongside watermarking.

## C2PA / Content Authenticity Initiative

Not an ML technique — a manifest format. Audio files carry cryptographically signed metadata about creation tool, author, date. Audobox / Seamless use it. Good for provenance; does nothing if a bad actor re-encodes and strips metadata.

**Interactive figure: v4-audio-watermark.** This one moves. Watch it animate and drag its controls in the web edition: <https://aiengineeringfromscratch.com/lesson?path=phases/06-speech-and-audio/16-anti-spoofing-audio-watermarking>

## Build It

### Step 1: a simple spectral-feature detector (toy)

```
def spectral_rolloff(spec, percentile=0.85):
    cum = 0
    total = sum(spec)
    if total == 0:
        return 0
    threshold = total * percentile
    for k, v in enumerate(spec):
        cum += v
        if cum >= threshold:
            return k
    return len(spec) - 1

def is_suspicious(audio):
    spec = magnitude_spectrum(audio)
    rolloff = spectral_rolloff(spec)
    return rolloff / len(spec) > 0.92
```

Synthetic speech often has unusually flat high-frequency energy. Production detectors use AASIST, not this. But the intuition holds.

### Step 2: AudioSeal embed + detect

```
from audioseal import AudioSeal
import torch

generator = AudioSeal.load_generator("audioseal_wm_16bits")
detector = AudioSeal.load_detector("audioseal_detector_16bits")

audio = load_wav("generated.wav", sr=16000)[None, None, :]
payload = torch.tensor([[1, 0, 1, 1, 0, 1, 0, 0, 1, 1, 0, 1, 0, 1, 1, 0]])
watermark = generator.get_watermark(audio, sample_rate=16000, message=payload)
watermarked = audio + watermark
```

```

result, decoded_payload = detector.detect_watermark(watermarked,
    ↪ sample_rate=16000)
# result: float in [0, 1] – probability of watermark presence
# decoded_payload: 16 bits; match against embedded payload

```

### Step 3: evaluation — EER

```

def eer(real_scores, fake_scores):
    thresholds = sorted(set(real_scores + fake_scores))
    best = (1.0, 0.0)
    for t in thresholds:
        far = sum(1 for s in fake_scores if s >= t) / len(fake_scores)
        frr = sum(1 for s in real_scores if s < t) / len(real_scores)
        if abs(far - frr) < best[0]:
            best = (abs(far - frr), (far + frr) / 2)
    return best[1]

```

### Step 4: the production integration

```

def safe_tts(text, voice, clone_reference=None):
    if clone_reference is not None:
        verify_consent(user_id, clone_reference)
    audio = tts_model.synthesize(text, voice)
    audio_with_wm = audioseal_embed(audio, payload=build_payload(user_id,
    ↪ model_id))
    manifest = c2pa_sign(audio_with_wm, user_id, timestamp=now())
    return audio_with_wm, manifest

```

Every generation ships: (1) watermark, (2) signed manifest, (3) retention-policy-compliant audit log.

## Use It

Use case	Defense
Shipping TTS / voice cloning	AudioSeal embed on every output (non-negotiable)
Biometric voice unlock	AASIST + ECAPA ensemble; liveness challenge
Call-center fraud detection	AASIST on 20% sample of incoming calls
Podcast authenticity	C2PA signing on upload, AudioSeal if AI-generated
Research / training detectors	ASVspooof 5 train/dev/eval sets

## Pitfalls

- **Watermark without detector ever running.** Pointless. Ship the detector in your CI.
- **Detection without calibration.** AASIST trained on ASVspooof LA overfits; real-world accuracy drops. Calibrate on your domain.
- **Pitch-shift gap.** Aggressive pitch shift removes most watermarks. Have a detection fallback.

- **Metadata strip-and-rehost.** C2PA is trivially bypassable by re-encoding. Always add cryptographic + perceptual (watermark) defense together.
- **Liveness as detection.** Ask user to say a random phrase. Prevents replay attacks but not real-time cloning.

**This chapter ships an artifact.** The course version of this lesson produces a reusable prompt or agent skill. It lives in the repository, ready to install: <https://github.com/rohitg00/ai-engineering-from-scratch/tree/main/phases/06-speech-and-audio/16-anti-spoofing-audio-watermarking>

## Exercises

Starter code and the lesson’s working implementation: <https://github.com/rohitg00/ai-engineering-from-scratch/tree/main/phases/06-speech-and-audio/16-anti-spoofing-audio-watermarking/code>

1. **Easy.** Run `code/main.py`. Toy detector + toy watermark embed/detect on synthetic audio.
2. **Medium.** Install `audioseal`, embed a 16-bit payload in a TTS output, re-decode. Corrupt the audio with noise and measure Bit Recovery Accuracy.
3. **Hard.** Fine-tune a RawNet2 or AASIST on ASVspooF 2019 LA. Measure EER. Test on a held-out set of F5-TTS-generated clips — see how OOD detection degrades.

## Key Terms

Term	What people say	What it actually means
ASVspooF CM (counter-measure)	The benchmark Detector	Biennial challenge; 2024 = ASVspooF 5. Classifier: real speech vs synthetic / converted.
SASV AudioSeal	Speaker verif + CM Meta watermark	Integrated biometric + spoof detection. Localized, 16-bit payload, 485× faster than WavMark.
Bit Recovery Accuracy	Watermark survival	Fraction of payload bits recovered after attack.
C2PA	Provenance manifest	Cryptographic metadata about creation / authorship.
AASIST	Detector family	Graph-attention-based anti-spoofing SOTA.

## Further Reading

- Todisco et al. (2024). ASVspooF 5 — the current benchmark.
- Defossez et al. (2024). AudioSeal — the watermark default.
- Chen et al. (2025). WaveVerify — MoE detector for temporal attacks.
- Jung et al. (2022). AASIST — the SOTA detection backbone.
- AudioMarkBench (2024) — robustness evaluation.
- C2PA specification — provenance manifest format.

**Continue online.** The living edition of this chapter has more than the page can hold:

- Animated, interactive figures and the web text: <https://aiengineeringfromscratch.com/lesson?path=phases/06-speech-and-audio/16-anti-spoofing-audio-watermarking>
- Runnable code for every step: <https://github.com/rohitg00/ai-engineering-from-scratch/tree/main/phases/06-speech-and-audio/16-anti-spoofing-audio-watermarking/code>

The repository moves faster than any printing. When the book and the repo disagree, trust the repo.



# Audio Evaluation — WER, MOS, UTMOS, MMAU, FAD, and the Open Leaderboards

You cannot ship what you cannot measure. This lesson names the 2026 metrics for every audio task: ASR (WER, CER, RTFx), TTS (MOS, UTMOS, SECS, WER-on-ASR-round-trip), audio-language (MMAU, LongAudioBench), music (FAD, CLAP), and speaker (EER). Plus the leaderboards where you compare.

**Type:** Learn **Languages:** Python **Prerequisites:** Phase 6 · 04, 06, 07, 09, 10; Phase 2 · 09 (Model Evaluation) **Time:** ~60 minutes

## The Problem

Every audio task has multiple metrics, each measuring a different axis. Using the wrong metric is how you ship a model that looks great on your dashboard and terribly in production. The 2026 canonical list:

Task	Primary	Secondary
ASR	WER	CER · RTFx · first-token latency
TTS	MOS / UTMOS	SECS · WER-on-ASR-round-trip · CER · TTFA
Voice cloning	SECS (ECAPA cosine)	MOS · CER
Speaker verification	EER	minDCF · FAR / FRR at operating point
Diarization	DER	JER · speaker confusion
Audio classification	top-1 · mAP	macro F1 · per-class recall
Music generation	FAD	CLAP · listening panel MOS
Audio language model	MMAU-Pro	LongAudioBench · AudioCaps
Streaming S2S	latency P50/P95	FENSE
		WER · MOS

## The Concept

### ASR metrics

**WER (Word Error Rate).**  $(S + D + I) / N$ . Lowercase, strip punctuation, normalize numbers before scoring. Use jiwer or OpenAI's whisper_normalizer. < 5% = human-parity read speech.

**CER (Character Error Rate).** Same formula, character-level. Used for tone languages (Mandarin, Cantonese) where word segmentation is ambiguous.

**RTFx (inverse real-time factor).** Audio seconds processed per wall-clock second. Higher is better. Parakeet-TDT hits 3380×. Whisper-large-v3 is ~30×.

## audio evaluation — one metric per task, one leaderboard per metric

<b>ASR</b>   primary: WER (normalized) CER for tone langs speed: RTFx latency: first-token ms   <b>Open ASR Leaderboard</b>   Parakeet 6.05% avg WER   Whisper-v3-turbo 1.58% LS   Canary-Qwen 5.63% avg	<b>TTS</b>   primary: MOS / UTMOS SECS (cloning) round: ASR-WER on output latency: TTFA ms   <b>TTS Arena · Artificial Analysis</b>   Inworld TTS-1.5 ELO 1236   Kokoro-82M ELO 1059   ElevenLabs v3 ELO 1179	<b>speaker + diarization</b>   primary: EER minDCF diariz: DER / JER overlap: per-segment F1   <b>VoxCeleb1-O / VoxSRC</b>   ECAPA 0.87% EER   3D-Speaker 0.50% EER   pyannote Precision-2 &lt; 0.40%	<b>audio classification</b>   exclusive: top-1, top-5 multi: mAP imbalanced: macro F1 per-class recall   <b>HEAR · AudioSet · ESC-50</b>   BEATs-iter3 AudioSet 0.548   Audio-MAE Speech Cmd 99.0%   BEATs ESC-50 97.0%
<b>music generation</b>   primary: FAD (Fréchet Audio Distance) CLAP (text-audio alignment) consumer: blind-panel MOS / ELO   <b>Suno v5 ELO 1293 (quality leader)</b>   Udio v4 — producer tools + stems   MusicGen-large on MusicCaps ~4.5 FAD   always pair FAD with human MOS panel   open: ACE-Step XL / YuE / Stable Audio Open		<b>audio-language + streaming</b>   LALM reasoning: MMAU-Pro (1800 Q)   long-audio: LongAudioBench   captioning: FENSE / SPICE / CIDEr   <b>Gemini 2.5 Pro ~60% MMAU-Pro overall</b>   GPT-4o Audio 52.5% — Qwen2.5-Omni 52.2%   multi-audio subset: ~22% ALL models (near random)   streaming S2S: latency P50/P95 + WER   Moshi 200 ms L4 · GPT-4o Realtime ~300 ms	

Figure 17: Audio evaluation matrix — metrics vs tasks vs 2026 leaderboards

**First-token latency.** Wall-clock from audio input to first transcript token. Critical for streaming. Deepgram Nova-3: ~150 ms.

## TTS metrics

**MOS (Mean Opinion Score).** 1-5 human rating. Gold standard but slow. Collect 20+ listeners per sample, 100+ samples per model.

**UTMOS (2022-2026).** Learned MOS predictor. Correlates ~0.9 with human MOS on standard benchmarks. F5-TTS: UTMOS 3.95; ground truth: 4.08.

**SECS (Speaker Encoder Cosine Similarity).** For voice cloning. ECAPA embedding cosine between reference and cloned output. > 0.75 = recognizable clone.

**WER-on-ASR-round-trip.** Run Whisper over TTS output, compute WER against the input text. Catches intelligibility regressions. 2026 SOTA: < 2% CER.

**TTFA (time-to-first-audio).** Wall-clock latency. Kokoro-82M: ~100 ms; F5-TTS: ~1 s.

## Voice-cloning-specific

**SECS + MOS + CER** as a triple. Cloning that scores high SECS but low MOS means timbre-right-but-unnatural; the opposite means natural voice but wrong speaker.

## Speaker verification

**EER (Equal Error Rate).** The threshold where False Accept Rate equals False Reject Rate. ECAPA on VoxCeleb1-O: 0.87%.

**minDCF (min Detection Cost).** Weighted cost at a chosen operating point (often FAR=0.01). More production-relevant than EER.

## Diarization

**DER (Diarization Error Rate).**  $(FA + Miss + Confusion) / total_speaker_time$ . Missed speech + false-alarm speech + speaker-confusion, each as a fraction. AMI meetings: DER ~10-20% is realistic. pyannote 3.1 + Precision-2 commercial: <10% DER on well-recorded audio.

**JER (Jaccard Error Rate).** Alternative to DER, robust to short-segment bias.

## Audio classification

Multi-label: **mAP (mean Average Precision)** over all classes. AudioSet: 0.548 mAP for BEATs-iter3.

Multi-class exclusive: **top-1, top-5 accuracy**. Speech Commands v2: 99.0% top-1 (AudioMAE).

Imbalanced: **macro F1 + per-class recall**. Report per-class — aggregate accuracy hides which classes fail.

## Music generation

**FAD (Fréchet Audio Distance).** Distance between VGGish-embedding distributions of real vs generated audio. MusicGen-small on MusicCaps: 4.5. MusicLM: 4.0. Lower better.

**CLAP Score.** Text-audio alignment score using CLAP embeddings. > 0.3 = reasonable alignment.

**Listening panel MOS.** Still the final word for consumer-grade music. Suno v5 ELO 1293 on TTS Arena (from paired human preferences).

## Audio-language benchmarks

**MMAU (Massive Multi-Audio Understanding).** 10k audio-QA pairs.

**MMAU-Pro.** 1800 hard items, four categories: speech / sound / music / multi-audio. Random chance 25% on 4-way. Gemini 2.5 Pro overall ~60%; multi-audio ~22% across all models.

**LongAudioBench.** Multi-minute clips with semantic queries. Audio Flamingo Next beats Gemini 2.5 Pro.

**AudioCaps / Clotho.** Captioning benchmarks. SPICE, CIDEr, FENSE metrics.

## Streaming speech-to-speech

**Latency P50 / P95 / P99.** Wall-clock from end-of-user-speech to first audible response. Moshi: 200 ms; GPT-4o Realtime: 300 ms.

**WER / MOS** on the output.

**Barge-in responsiveness.** Time from user interrupt to assistant mute. Target < 150 ms.

## The 2026 leaderboards

Leaderboard	Tracks	URL
Open ASR Leaderboard (HF)	English + multilingual + long-form	<a href="https://huggingface.co/spaces/hf-audio/open_asr_leaderboard">huggingface.co/spaces/hf-audio/open_asr_leaderboard</a>
TTS Arena (HF)	English TTS	<a href="https://huggingface.co/spaces/TTS-AGI/TTS-Arena">huggingface.co/spaces/TTS-AGI/TTS-Arena</a>
Artificial Analysis Speech	TTS + STT, ELO from paired votes	<a href="https://artificialanalysis.ai/speech">artificialanalysis.ai/speech</a>
MMAU-Pro	LALM reasoning	<a href="https://mmaubenchmark.github.io">mmaubenchmark.github.io</a>
SpeakerBench / VoxSRC	Speaker recognition	<a href="https://voxsrc.github.io">voxsrc.github.io</a>
MMAU music subset	Music LALM	(within MMAU)
HEAR benchmark	Self-supervised audio	<a href="https://hearbenchmark.com">hearbenchmark.com</a>

**Interactive figure: sp-wer-align.** This one moves. Watch it animate and drag its controls in the web edition: <https://aiengineeringfromscratch.com/lesson?path=phases/06-speech-and-audio/17-audio-evaluation-metrics>

## Build It

### Step 1: WER with normalization

```
from jiwer import wer, Compose, ToLowerCase, RemovePunctuation, Strip

transform = Compose([ToLowerCase(), RemovePunctuation(), Strip()])
score = wer(
    truth="Please turn on the lights.",
    hypothesis="please turn on the light",
    truth_transform=transform,
    hypothesis_transform=transform,
)
# ~0.17
```

### Step 2: TTS round-trip WER

```
def ttr_wer(tts_model, asr_model, texts):
    errors = []
    for txt in texts:
        audio = tts_model.synthesize(txt)
        recog = asr_model.transcribe(audio)
        errors.append(wer(truth=txt, hypothesis=recog))
    return sum(errors) / len(errors)
```

### Step 3: SECS for voice cloning

```
from speechbrain.inference.speaker import EncoderClassifier
sv = EncoderClassifier.from_hparams("speechbrain/spkrec-ecapa-voxceleb")
```

```
emb_ref = sv.encode_batch(load_wav("reference.wav"))
emb_clone = sv.encode_batch(load_wav("cloned.wav"))
secs = torch.nn.functional.cosine_similarity(emb_ref, emb_clone, dim=-1).item()
```

## Step 4: FAD for music generation

```
from frechet_audio_distance import FrechetAudioDistance
fad = FrechetAudioDistance()
score = fad.get_fad_score("generated_folder/", "reference_folder/")
```

## Step 5: EER for speaker verification (same code as Lesson 6)

```
def eer(same_scores, diff_scores):
    thresholds = sorted(set(same_scores + diff_scores))
    best = (1.0, 0.0)
    for t in thresholds:
        far = sum(1 for s in diff_scores if s >= t) / len(diff_scores)
        frr = sum(1 for s in same_scores if s < t) / len(same_scores)
        if abs(far - frr) < best[0]:
            best = (abs(far - frr), (far + frr) / 2)
    return best[1]
```

## Use It

Pair every deploy with a fixed eval harness that runs on every model update. Three cardinal rules:

1. **Normalize before scoring.** Lowercase, punctuation-strip, number-expand. Report the normalization rule.
2. **Report distributions, not averages.** P50/P95/P99 for latency. Per-class recall for classification. Per-category for MMAU.
3. **Run one canonical public benchmark.** Even if your production data differs, reporting on Open ASR / TTS Arena / MMAU lets reviewers compare apples-to-apples.

## Pitfalls

- **UTMOS extrapolation.** Trained on VCTK-style clean speech; scores noisy / cloned / emotional audio poorly.
- **MOS panel bias.** 20 Amazon Mechanical Turk workers  $\neq$  20 target users. Pay for a domain panel if stakes are high.
- **FAD depends on reference set.** Compare against the same reference distribution across models.
- **Aggregate WER.** A 5% WER overall can hide 30% WER on accented speech. Report by demographic slice.
- **Public benchmark saturation.** Most frontier models are near the ceiling on standard benchmarks. Build an in-house held-out set that reflects your traffic.

**This chapter ships an artifact.** The course version of this lesson produces a reusable prompt or agent skill. It lives in the repository, ready to install: <https://github.com/rohitg00/ai-engineering-from-scratch/tree/main/phases/06-speech-and-audio/17-audio-evaluation-metrics>

## Exercises

Starter code and the lesson’s working implementation: <https://github.com/rohitg00/ai-engineering-from-scratch/tree/main/phases/06-speech-and-audio/17-audio-evaluation-metrics/code>

1. **Easy.** Run `code/main.py`. Compute WER / CER / EER / SECS / FAD-ish / MMAU-ish on toy inputs.
2. **Medium.** Build a TTS round-trip WER harness. Run your Kokoro or F5-TTS output through Whisper. Compute WER over 50 prompts. Flag prompts with WER > 10%.
3. **Hard.** Score your Lesson 10 LALM choice on MMAU-Pro speech + multi-audio subsets (50 items each). Report per-category accuracy and compare with the published number.

## Key Terms

Term	What people say	What it actually means
WER	ASR score	$(S+D+I)/N$ at word level after normalization.
CER	Character WER	For tone languages or char-level systems.
MOS	Human opinion	1-5 rating; 20+ listeners $\times$ 100 samples.
UTMOS	ML MOS predictor	Learned model; correlates $\sim 0.9$ with human MOS.
SECS	Voice-clone similarity	ECAPA cosine between reference and clone.
EER	Speaker verif score	Threshold where FAR = FRR.
DER	Diarization score	$(FA + Miss + Confusion) / \text{total}$ .
FAD	Music-gen quality	Fréchet distance on VGGish embeddings.
RTFx	Throughput	Audio seconds per wall-clock second.

## Further Reading

- `jiwer` — WER/CER library with normalization utilities.
- UTMOS (Saeki et al. 2022) — learned MOS predictor.
- Fréchet Audio Distance (Kilgour et al. 2019) — the music-gen standard.
- Open ASR Leaderboard — 2026 live rankings.
- TTS Arena — human-vote TTS leaderboard.
- MMAU-Pro benchmark — LALM reasoning leaderboard.
- HEAR benchmark — audio SSL benchmarks.

**Continue online.** The living edition of this chapter has more than the page can hold:

- Animated, interactive figures and the web text: <https://aiengineeringfromscratch.com/lesson?path=phases/06-speech-and-audio/17-audio-evaluation-metrics>
- Runnable code for every step: <https://github.com/rohitg00/ai-engineering-from-scratch/tree/main/phases/06-speech-and-audio/17-audio-evaluation-metrics/code>

The repository moves faster than any printing. When the book and the repo disagree, trust the repo.